

sercon User Manual

Embeddable TypeScript script engine — reference and
guide

0.102.0
2026-08-20
Thomas Björk

Contents

sercon manual	1
1. Overview	1
2. Quickstart	1
2.1 Concepts	2
2.2 Recipes	2
2.2.1 Write and run your first script	3
2.2.2 Read input: arguments and environment	3
2.2.3 Call an HTTP API	3
2.2.4 Read and write a file	3
2.2.5 Find the rest of the API	4
3. Library API — pkg/scriptengine	4
3.1 Engine, Options, New	4
3.2 Registering bindings	5
3.3 Run, RunFile	6
3.3.1 Per-Run options	6
3.4 Reset	7
3.5 WriteTypes	7
3.6 SetDocs and SetMemberDocs	7
3.7 PromisifyAsync[T] and AsyncBinding	8
3.8 Version	8
4. CLI — sercon	8
4.1 Passing arguments: -- and runtime.argv	10
4.2 --watch: re-run on file change	10
4.3 Editor autocomplete (sercon init)	11
4.4 Shebang lines and executable scripts (sercon run)	12
4.5 sercon serve: long-running scripts with production niceties	12
4.6 Recipes	14
4.6.1 Pass arguments to a script	14
4.6.2 Load config and secrets from a .env	14
4.6.3 Auto-rerun while developing	14
4.6.4 Bound a run and gate on the exit code	14
4.6.5 Use sercon in a shell pipeline	14
4.6.6 Run a long-lived server	14
4.6.7 Make a script directly executable	15
4.6.8 Emit tooling artifacts for your editor	15
5. Reserved globals (script surface)	15
5.1 console (browser/Node compatibility)	15
5.2 runtime	16
5.2.1 Script-controlled stdio: runtime.stdout / runtime.stderr / runtime.stdin ...	18
5.2.2 Recipes	20
5.2.2.1 Silence a noisy dependency for one call	20
5.2.2.2 Log to a file while still watching the terminal	20
5.2.2.3 Assert on what a function printed	20
5.2.2.4 Read piped input line by line	21
5.2.2.5 Route each line to your own handler	21
5.3 crypto	22

5.3.1	<code>crypto.hash.*</code>	22
5.3.2	<code>crypto.jwt.*</code>	22
5.3.3	<code>crypto.encrypt.*</code>	22
5.3.4	Concepts	23
5.3.5	Recipes	23
5.3.5.1	Hash a string or a file's contents	23
5.3.5.2	Verify a payload's integrity with a hash claim	23
5.3.5.3	Encrypt and decrypt with age	24
5.3.5.4	Sign and verify a JWT	25
5.4	<code>text</code>	25
5.4.1	Concepts	27
5.4.2	Recipes	28
5.4.2.1	Decode non-UTF-8 bytes to a string	28
5.4.2.2	Use the string utilities	28
5.4.2.3	Base64 encode/decode	28
5.4.2.4	Convert an identifier between conventions	29
5.4.2.5	Convert to a case chosen at runtime	29
5.4.2.6	Tokenize with split / guess with detect	29
5.5	<code>codec</code>	30
5.5.1	<code>codec.compression</code> — compress / decompress	31
5.5.2	<code>codec.barcode</code> — encode / decode	31
5.5.3	<code>codec.checkdigit</code> — validate / compute / inspect	32
5.5.4	<code>codec.php</code> — PHP dump formats	32
5.5.5	<code>codec.perl</code> — Perl Data::Dumper	33
5.5.6	opts and shared caveats	33
5.5.7	<code>codec.sheet</code> — tabular CSV / TSV / XLSX / ODS / XLS / SYLK / DIF	33
5.5.8	Concepts	35
5.5.9	Recipes	36
5.5.9.1	Parse and stringify a TOML document	36
5.5.9.2	Parse and stringify a YAML document	36
5.5.9.3	Round-trip tabular data as CSV	37
5.5.9.4	Convert a parsed document into another format	37
5.5.9.5	Parse and apply a <code>.env</code> file	37
5.5.9.6	Round-trip PHP and Perl dump formats	38
5.6	<code>fs</code>	38
5.6.1	Concepts	40
5.6.2	Recipes	40
5.6.2.1	Read a file as text or bytes	40
5.6.2.2	Write a file, creating its directory first	40
5.6.2.3	Guard an operation with <code>exists</code> / <code>stat</code>	40
5.6.2.4	Create and extract an archive	41
5.6.2.5	Find files by glob, respecting <code>.gitignore</code>	41
5.6.2.6	Search file contents for a pattern	41
5.6.2.7	Follow a log and react to error lines	41
5.7	<code>net</code>	42
5.7.1	HTTP client (<code>net.http</code>)	42
5.7.2	One-shot probes (<code>net.probe</code>)	43

5.7.3 Email-authentication probes (<code>net.email</code>)	44
5.7.4 Stateful HTTP session (<code>net.browser</code>)	44
5.7.5 Packet capture (<code>net.capture</code>)	45
5.7.6 Raw packets (<code>net.raw</code>)	47
5.7.7 HTTP load / resilience self-test (<code>net.load</code>)	48
5.7.8 Concepts	49
5.7.9 Recipes	49
5.7.9.1 Make an authenticated JSON request with retry	49
5.7.9.2 Probe a host:port for reachability + latency	49
5.7.9.3 Check a TLS certificate's expiry	50
5.7.9.4 Check a domain's email-authentication posture (SPF / DMARC / ...)	50
5.7.9.5 Capture packets to a file	50
5.7.9.6 Run a quick load / resilience self-test	50
5.8 db	51
5.8.1 Concepts	52
5.8.2 Recipes	54
5.8.2.1 Open a database and query rows	54
5.8.2.2 Insert with parameterized statements	54
5.8.2.3 Run a transaction	55
5.8.2.4 Apply a simple migration	55
5.8.2.5 Sweep an ETL load	56
5.8.2.6 Connect to a networked SQL engine	57
5.8.2.7 Use Redis or Valkey as a key-value store	57
5.8.2.8 Cache-aside with memcached	58
5.8.2.9 Inspect and search an LDAP directory	58
5.8.2.10 Define and match words over DICT	59
5.9 services	59
5.9.1 <code>services.agentBrowser</code>	61
5.9.2 <code>services.webdriver</code>	69
5.9.2.1 Frames / iframes	76
5.9.3 CDP trusted clicks (Chrome only)	77
5.9.4 <code>services.typst</code>	78
5.9.5 <code>services.pdf</code>	79
5.9.6 <code>services.doctor</code>	80
5.9.7 Concepts	81
5.9.8 Recipes	82
5.9.8.1 Feature-detect a tool before using it	82
5.9.8.2 Render a Typst document to a PDF	82
5.9.8.3 Drive a headless browser to screenshot / extract	82
5.9.8.4 Run a git / gh operation	83
5.10 tui	83
5.10.1 Layout	83
5.10.2 Pane handles	84
5.10.3 Subprocess routing	84
5.10.4 Keybindings (TTY mode only)	84
5.10.5 Limitations (v1)	85
5.11 image	85

5.11.1	Animated frames (GIF / APNG)	88
5.11.2	EXIF	89
5.11.3	Steganography (image.stego)	90
5.11.4	Concepts	92
5.11.5	Recipes	92
5.11.5.1	Resize an image and convert its format	92
5.11.5.2	Crop and correct orientation	93
5.11.5.3	Apply brightness / contrast / filter adjustments	93
5.11.5.4	Embed and extract steganographic data	93
5.11.5.5	Build a two-frame animation	94
5.12	web	94
5.12.1	web.feed	95
5.12.2	web.sitemap	95
5.12.3	web.html	95
5.12.4	Concepts	96
5.12.5	Recipes	97
5.12.5.1	Fetch and parse an RSS/Atom feed	97
5.12.5.2	Crawl a sitemap, expanding an index	97
5.12.5.3	Scrape a page with CSS selectors	97
5.12.5.4	Extract with XPath	97
5.13	audio	98
5.13.1	Format read/write	98
5.13.2	audio.stego	99
5.14	cloud	100
5.14.1	Concepts	100
5.14.2	cloud.google	102
	storage()	103
	compute()	104
	iam()	104
	secrets()	105
	call() — REST escape hatch	106
5.14.3	cloud.aws	106
	s3()	108
	ec2()	108
	iam()	109
	secretsmanager()	110
	sts()	110
	lambda()	111
	sqs()	111
	cloudwatch()	112
	cloudwatchlogs()	113
5.14.4	cloud.azure	114
	resourceGroups()	115
	compute()	116
	resources()	116
	call() — ARM REST escape hatch	117
	blob(accountUrl)	117

keyvaultSecrets(vaultUrl)	118
5.14.5 Recipes	118
5.15 mcp	120
5.15.1 Concepts	120
5.15.2 Recipes	124
5.15.2.1 Expose a sercon capability as an MCP tool	124
5.15.2.2 Run a stdio MCP server for Claude Desktop	125
5.15.2.3 Serve MCP over HTTP	126
5.15.2.4 Report progress and stream log lines from a tool	127
5.15.2.5 Grow or shrink your tool set at runtime	127
5.15.2.6 Expose a family of resources with a URI template	128
5.15.2.7 Push updates to subscribed clients (lazy-watch pattern)	128
5.15.2.8 Autocomplete a prompt argument	129
5.15.2.9 Cap list-page size for large tool sets	130
5.15.2.10 Bridge a tool to an external HTTP API	130
5.15.2.11 Ask the client's LLM to do work mid-call (sampling)	131
5.15.2.12 Confirm an action with the user mid-call (elicitation)	132
5.15.2.13 Read the client's filesystem roots	132
5.15.2.14 Protect an HTTP server with OAuth 2.1 bearer tokens	133
5.15.3 mcp.connect — sercon as an MCP client	134
5.15.3.1 Call a tool on an MCP server	136
5.15.3.2 Read a resource from an MCP server	137
5.15.3.3 Subscribe to a resource and react when it changes	137
5.15.3.4 Receive server log messages	138
5.15.3.5 Request an argument completion	138
5.15.3.6 Answer a server's sampling request (act as its LLM)	139
5.15.3.7 Confirm an action or collect input via elicitation	139
5.15.3.8 Expose the client's filesystem roots, and update them later	140
5.15.3.9 Connect over the legacy SSE transport	141
5.15.3.10 Connect to an OAuth-protected server with auth.getToken	141
5.15.3.11 Disable automatic reconnection with maxRetries	142
6. Servers	143
6.1 Foundation: HoldRun + LoopCallable	143
6.2 server.http.listen / server.https.listen	144
6.3 Static-file serving	147
6.4 Middleware	147
6.5 WebSocket upgrade	148
6.5.1 Server-Sent Events (res.sse)	149
6.6 Lifecycle	150
6.7 SMTP	150
6.7.1 server.smtp.listen({...})	150
6.7.2 net.email.send({...}) — outbound	153
6.8 Raw TCP / UDP	154
6.8.1 server.tcp.listen({...}, conn => {...})	154
6.8.2 server.udp.listen({...}, (msg, reply) => {...})	154
6.9 Concepts	155
6.10 Recipes	156

6.10.1	Serve a routed JSON API with middleware	156
6.10.2	Serve static files	156
6.10.3	Handle a WebSocket	157
6.10.4	Receive email	157
6.10.5	Run a TCP line server	158
7.	JavaScript runtime built-ins (goja)	158
7.1	Globals	158
7.2	Collections	158
7.3	Typed arrays	159
7.4	Iteration and generators	159
7.5	Not (yet) provided	159
8.	Async runtime additions (goja_nodejs)	159
8.1	Timers	159
8.2	console	160
8.3	require	160
9.	TypeScript support	160
10.	Top-level await	161
11.	Module resolution	161
12.	Timeouts and cancellation	162
13.	Error semantics	162
14.	Type generation (.d.ts)	162
14.1	JSDoc comments on declarations	163
15.	Limitations and gotchas	163
16.	Bundled libraries	164
16.1	paymentproviders	164
16.1.1	Shared core	164
16.1.2	kcov3 — Kustom / Klarna Checkout v3	165
16.1.3	netstv1 — Nexi / Nets Checkout Payment API v1	166
16.1.4	sveacheckout2 — Svea Checkout	167
16.1.5	qlirov2 — Qliro One	167
16.1.6	swedbankpayv2 / swedbankpayv3 — SwedbankPay Checkout (HAL)	168
16.1.7	Understanding payment providers	169
16.1.8	Recipes	170
16.1.8.1	Create a checkout and read its status	171
16.1.8.2	Capture — full, then partial	171
16.1.8.3	Refund a captured payment	171
16.1.8.4	Cancel an uncaptured payment	171
16.1.8.5	SwedbankPay: follow HAL operations	172
16.1.8.6	Signed-body providers (svea, qliro)	172
16.1.8.7	Retry safely with idempotency (kcov3)	172
16.1.8.8	Configure auth and pick test vs prod	173
16.1.8.9	Handle a PaymentError	173
16.1.8.10	Poll a payment to a terminal status	173
16.2	favro	174
16.2.1	Resource groups	174
16.2.2	Pagination	175
16.2.3	Retry and errors	175

16.2.4 Attachments	176
16.2.5 Understanding Favro's model	176
16.2.6 Recipes	177
16.2.6.1 Filter a column by lane	177
16.2.6.2 Resolve a name to an id	178
16.2.6.3 Filter by a custom-field value	178
16.2.6.4 Find a card by its human number	179
16.2.6.5 List a whole collection vs one board	179
16.2.6.6 Create a card	179
16.2.6.7 Move a card between columns	179
16.2.6.8 Comment on a card and attach a file	179
16.2.6.9 Add a dependency between cards	179
16.2.6.10 Set up auth and verify it	180
16.2.6.11 Receive a webhook	180
16.2.6.12 Read a lot of cards without tripping the rate limit	180
17. Binding reference (generated)	181
17.1 audio	181
17.1.1 audio.convert	181
17.1.2 audio.decode	181
17.1.3 audio.encode	182
17.1.4 audio.info	182
17.1.5 audio.stego	182
17.1.5.1 audio.stego.capacity	182
17.1.5.2 audio.stego.embed	183
17.1.5.3 audio.stego.extract	183
17.2 cloud	184
17.2.1 cloud.aws	184
17.2.1.1 cloud.aws.cloudwatch	187
17.2.1.1.1 cloud.aws.cloudwatch.describeAlarms	187
17.2.1.1.2 cloud.aws.cloudwatch.getMetricData	187
17.2.1.1.3 cloud.aws.cloudwatch.getMetricStatistics	188
17.2.1.1.4 cloud.aws.cloudwatch.listMetrics	188
17.2.1.1.5 cloud.aws.cloudwatch.putMetricData	188
17.2.1.2 cloud.aws.cloudwatchlogs	189
17.2.1.2.1 cloud.aws.cloudwatchlogs.describeLogGroups	189
17.2.1.2.2 cloud.aws.cloudwatchlogs.describeLogStreams	189
17.2.1.2.3 cloud.aws.cloudwatchlogs.filterLogEvents	190
17.2.1.2.4 cloud.aws.cloudwatchlogs.getLogEvents	190
17.2.1.2.5 cloud.aws.cloudwatchlogs.getQueryResults	191
17.2.1.2.6 cloud.aws.cloudwatchlogs.startQuery	191
17.2.1.3 cloud.aws.ec2	192
17.2.1.3.1 cloud.aws.ec2.describeAvailabilityZones	192
17.2.1.3.2 cloud.aws.ec2.describeInstances	192
17.2.1.3.3 cloud.aws.ec2.describeVolumes	193
17.2.1.3.4 cloud.aws.ec2.runInstances	193
17.2.1.3.5 cloud.aws.ec2.startInstances	193
17.2.1.3.6 cloud.aws.ec2.stopInstances	194

17.2.1.3.7 cloud.aws.ec2.terminateInstances	194
17.2.1.4 cloud.aws.iam	195
17.2.1.4.1 cloud.aws.iam.attachUserPolicy	195
17.2.1.4.2 cloud.aws.iam.createUser	195
17.2.1.4.3 cloud.aws.iam.deleteUser	195
17.2.1.4.4 cloud.aws.iam.getRole	196
17.2.1.4.5 cloud.aws.iam.getUser	196
17.2.1.4.6 cloud.aws.iam.listPolicies	196
17.2.1.4.7 cloud.aws.iam.listRoles	197
17.2.1.4.8 cloud.aws.iam.listUsers	197
17.2.1.5 cloud.aws.lambda	198
17.2.1.5.1 cloud.aws.lambda.createFunction	198
17.2.1.5.2 cloud.aws.lambda.deleteFunction	198
17.2.1.5.3 cloud.aws.lambda.getFunction	199
17.2.1.5.4 cloud.aws.lambda.invoke	199
17.2.1.5.5 cloud.aws.lambda.listFunctions	200
17.2.1.6 cloud.aws.s3	200
17.2.1.6.1 cloud.aws.s3.createBucket	200
17.2.1.6.2 cloud.aws.s3.deleteBucket	200
17.2.1.6.3 cloud.aws.s3.deleteObject	201
17.2.1.6.4 cloud.aws.s3.getObject	201
17.2.1.6.5 cloud.aws.s3.headObject	201
17.2.1.6.6 cloud.aws.s3.listBuckets	202
17.2.1.6.7 cloud.aws.s3.listObjects	202
17.2.1.6.8 cloud.aws.s3.putObject	203
17.2.1.7 cloud.aws.secretsmanager	203
17.2.1.7.1 cloud.aws.secretsmanager.createSecret	203
17.2.1.7.2 cloud.aws.secretsmanager.deleteSecret	204
17.2.1.7.3 cloud.aws.secretsmanager.describeSecret	204
17.2.1.7.4 cloud.aws.secretsmanager.getSecretValue	204
17.2.1.7.5 cloud.aws.secretsmanager.listSecrets	205
17.2.1.7.6 cloud.aws.secretsmanager.putSecretValue	205
17.2.1.8 cloud.aws.sqs	206
17.2.1.8.1 cloud.aws.sqs.createQueue	206
17.2.1.8.2 cloud.aws.sqs.deleteMessage	206
17.2.1.8.3 cloud.aws.sqs.deleteQueue	206
17.2.1.8.4 cloud.aws.sqs.getQueueAttributes	207
17.2.1.8.5 cloud.aws.sqs.listQueues	207
17.2.1.8.6 cloud.aws.sqs.receiveMessage	208
17.2.1.8.7 cloud.aws.sqs.sendMessage	208
17.2.1.9 cloud.aws.sts	209
17.2.1.9.1 cloud.aws.sts.assumeRole	209
17.2.1.9.2 cloud.aws.sts.getCallerIdentity	209
17.2.1.9.3 cloud.aws.sts.getSessionToken	209
17.2.2 cloud.azure	210
17.2.2.1 cloud.azure.blob	212
17.2.2.1.1 cloud.azure.blob.deleteBlob	212

17.2.2.1.2 cloud.azure.blob.download	212
17.2.2.1.3 cloud.azure.blob.listBlobs	213
17.2.2.1.4 cloud.azure.blob.listContainers	213
17.2.2.1.5 cloud.azure.blob.upload	214
17.2.2.2 cloud.azure.call	214
17.2.2.3 cloud.azure.compute	215
17.2.2.3.1 cloud.azure.compute.deallocate	215
17.2.2.3.2 cloud.azure.compute.delete	216
17.2.2.3.3 cloud.azure.compute.getVirtualMachine	216
17.2.2.3.4 cloud.azure.compute.listVirtualMachines	216
17.2.2.3.5 cloud.azure.compute.powerOff	217
17.2.2.3.6 cloud.azure.compute.start	217
17.2.2.4 cloud.azure.keyvaultSecrets	218
17.2.2.4.1 cloud.azure.keyvaultSecrets.deleteSecret	218
17.2.2.4.2 cloud.azure.keyvaultSecrets.getSecret	218
17.2.2.4.3 cloud.azure.keyvaultSecrets.listSecrets	219
17.2.2.4.4 cloud.azure.keyvaultSecrets.setSecret	219
17.2.2.5 cloud.azure.resourceGroups	219
17.2.2.5.1 cloud.azure.resourceGroups.create	220
17.2.2.5.2 cloud.azure.resourceGroups.delete	220
17.2.2.5.3 cloud.azure.resourceGroups.get	220
17.2.2.5.4 cloud.azure.resourceGroups.list	221
17.2.2.6 cloud.azure.resources	221
17.2.2.6.1 cloud.azure.resources.getById	221
17.2.2.6.2 cloud.azure.resources.listByResourceGroup	222
17.2.3 cloud.google	222
17.2.3.1 cloud.google.call	224
17.2.3.2 cloud.google.compute	225
17.2.3.2.1 cloud.google.compute.createInstance	225
17.2.3.2.2 cloud.google.compute.deleteInstance	225
17.2.3.2.3 cloud.google.compute.getInstance	226
17.2.3.2.4 cloud.google.compute.listDisks	226
17.2.3.2.5 cloud.google.compute.listInstances	227
17.2.3.2.6 cloud.google.compute.listZones	227
17.2.3.2.7 cloud.google.compute.startInstance	227
17.2.3.2.8 cloud.google.compute.stopInstance	228
17.2.3.3 cloud.google.iam	228
17.2.3.3.1 cloud.google.iam.createKey	228
17.2.3.3.2 cloud.google.iam.createServiceAccount	229
17.2.3.3.3 cloud.google.iam.deleteServiceAccount	229
17.2.3.3.4 cloud.google.iam.getIamPolicy	229
17.2.3.3.5 cloud.google.iam.getServiceAccount	230
17.2.3.3.6 cloud.google.iam.listKeys	230
17.2.3.3.7 cloud.google.iam.listServiceAccounts	231
17.2.3.3.8 cloud.google.iam.setIamPolicy	231
17.2.3.4 cloud.google.secrets	231
17.2.3.4.1 cloud.google.secrets.accessSecretVersion	232

17.2.3.4.2 cloud.google.secrets.addSecretVersion	232
17.2.3.4.3 cloud.google.secrets.createSecret	232
17.2.3.4.4 cloud.google.secrets.deleteSecret	233
17.2.3.4.5 cloud.google.secrets.getSecret	233
17.2.3.4.6 cloud.google.secrets.listSecrets	233
17.2.3.5 cloud.google.storage	234
17.2.3.5.1 cloud.google.storage.createBucket	234
17.2.3.5.2 cloud.google.storage.deleteBucket	234
17.2.3.5.3 cloud.google.storage.deleteObject	235
17.2.3.5.4 cloud.google.storage.getBucket	235
17.2.3.5.5 cloud.google.storage.listBuckets	235
17.2.3.5.6 cloud.google.storage.listObjects	236
17.2.3.5.7 cloud.google.storage.putObject	236
17.2.3.5.8 cloud.google.storage.readObject	237
17.2.3.5.9 cloud.google.storage.statObject	237
17.3 codec	237
17.3.1 codec.barcode	237
17.3.1.1 codec.barcode.decodableFormats	237
17.3.1.2 codec.barcode.decode	238
17.3.1.3 codec.barcode.encode	238
17.3.1.4 codec.barcode.formats	239
17.3.2 codec.checkdigit	239
17.3.2.1 codec.checkdigit.algos	239
17.3.2.2 codec.checkdigit.compute	239
17.3.2.3 codec.checkdigit.inspect	240
17.3.2.4 codec.checkdigit.validate	240
17.3.3 codec.compression	240
17.3.3.1 codec.compression.algos	240
17.3.3.2 codec.compression.compress	241
17.3.3.3 codec.compression.decompress	241
17.3.4 codec.doc	242
17.3.4.1 codec.doc.formats	242
17.3.4.2 codec.doc.read	242
17.3.4.3 codec.doc.write	242
17.3.5 codec.dotenv	243
17.3.5.1 codec.dotenv.parse	243
17.3.5.2 codec.dotenv.stringify	243
17.3.6 codec.perl	244
17.3.6.1 codec.perl.dumper	244
17.3.6.2 codec.perl.parseDumper	244
17.3.7 codec.php	245
17.3.7.1 codec.php.parseVarDump	245
17.3.7.2 codec.php.parseVarExport	245
17.3.7.3 codec.php.serialize	245
17.3.7.4 codec.php.unserialize	246
17.3.7.5 codec.php.varDump	246
17.3.7.6 codec.php.varExport	247

17.3.8 codec.sheet	247
17.3.8.1 codec.sheet.formats	247
17.3.8.2 codec.sheet.read	247
17.3.8.3 codec.sheet.write	248
17.3.9 codec.toml	249
17.3.9.1 codec.toml.parse	249
17.3.9.2 codec.toml.stringify	249
17.3.10 codec.xml	249
17.3.10.1 codec.xml.decode	249
17.3.10.2 codec.xml.encode	250
17.3.11 codec.yaml	250
17.3.11.1 codec.yaml.parse	250
17.3.11.2 codec.yaml.stringify	251
17.4 console	251
17.4.1 console.debug	251
17.4.2 console.error	251
17.4.3 console.info	252
17.4.4 console.log	252
17.4.5 console.table	252
17.4.6 console.warn	253
17.5 crypto	253
17.5.1 crypto.encrypt	253
17.5.1.1 crypto.encrypt.decrypt	253
17.5.1.2 crypto.encrypt.detectBackend	254
17.5.1.3 crypto.encrypt.encrypt	254
17.5.1.4 crypto.encrypt.keygen	254
17.5.1.5 crypto.encrypt.keygenPgp	255
17.5.1.6 crypto.encrypt.rekey	255
17.5.2 crypto.hash	256
17.5.2.1 crypto.hash.blake3	256
17.5.2.2 crypto.hash.crc32	256
17.5.2.3 crypto.hash.md5	256
17.5.2.4 crypto.hash.sha1	257
17.5.2.5 crypto.hash.sha256	257
17.5.2.6 crypto.hash.sha384	257
17.5.2.7 crypto.hash.sha3_256	257
17.5.2.8 crypto.hash.sha3_512	258
17.5.2.9 crypto.hash.sha512	258
17.5.3 crypto.jwt	258
17.5.3.1 crypto.jwt.sign	258
17.5.3.2 crypto.jwt.validate	259
17.5.3.3 crypto.jwt.view	259
17.6 db	260
17.6.1 db.clickhouse	260
17.6.1.1 db.clickhouse.open	260
17.6.2 db.dict	261
17.6.2.1 db.dict.define	261

17.6.2.2 db.dict.match	261
17.6.3 db.ldap	262
17.6.3.1 db.ldap.open	262
17.6.4 db.memcached	263
17.6.4.1 db.memcached.open	263
17.6.5 db.mssql	263
17.6.5.1 db.mssql.open	263
17.6.6 db.mysql	264
17.6.6.1 db.mysql.open	264
17.6.7 db.oracle	265
17.6.7.1 db.oracle.open	265
17.6.8 db.postgres	265
17.6.8.1 db.postgres.open	265
17.6.9 db.redis	266
17.6.9.1 db.redis.open	266
17.6.10 db.sqlite	267
17.6.10.1 db.sqlite.open	267
17.6.11 db.valkey	268
17.6.11.1 db.valkey.open	268
17.7 fs	268
17.7.1 fs.archive	268
17.7.1.1 fs.archive.create	268
17.7.1.2 fs.archive.extract	269
17.7.2 fs.exists	269
17.7.3 fs.find	270
17.7.4 fs.grep	271
17.7.5 fs.grepCount	271
17.7.6 fs.grepFiles	272
17.7.7 fs.grepStream	272
17.7.8 fs.mkdir	273
17.7.9 fs.path	273
17.7.9.1 fs.path.basename	273
17.7.9.2 fs.path.dirname	273
17.7.10 fs.readBytes	274
17.7.11 fs.readText	274
17.7.12 fs.remove	274
17.7.13 fs.stat	274
17.7.14 fs.tail	275
17.7.15 fs.writeBytes	275
17.7.16 fs.writeText	276
17.8 image	276
17.8.1 image.blur	276
17.8.2 image.brightness	276
17.8.3 image.bytes	277
17.8.4 image.contrast	277
17.8.5 image.crop	277
17.8.6 image.decode	278

17.8.7	image.decodeFrames	278
17.8.8	image.encodeFrames	279
17.8.9	image.exif	280
17.8.9.1	image.exif.clear	280
17.8.9.2	image.exif.read	280
17.8.9.3	image.exif.replace	281
17.8.9.4	image.exif.write	281
17.8.10	image.fit	282
17.8.11	image.flipH	282
17.8.12	image.flipV	282
17.8.13	image.gamma	283
17.8.14	image.grayscale	283
17.8.15	image.invert	283
17.8.16	image.open	283
17.8.17	image.orient	284
17.8.18	image.overlay	284
17.8.19	image.paste	285
17.8.20	image.rasterizeSVG	285
17.8.21	image.resize	286
17.8.22	image.rotate	286
17.8.23	image.rotate180	287
17.8.24	image.rotate270	287
17.8.25	image.rotate90	287
17.8.26	image.saturation	287
17.8.27	image.save	288
17.8.28	image.sharpen	288
17.8.29	image.stego	288
17.8.29.1	image.stego.analyze	288
17.8.29.2	image.stego.bitplane	289
17.8.29.3	image.stego.capacity	289
17.8.29.4	image.stego.detect	290
17.8.29.5	image.stego.embed	290
17.8.29.6	image.stego.extract	291
17.8.30	image.thumbnail	291
17.9	mcp	292
17.9.1	mcp.connect	292
17.9.1.1	mcp.connect.complete	292
17.9.1.2	mcp.connect.http	293
17.9.1.3	mcp.connect.onLoggingMessage	296
17.9.1.4	mcp.connect.onProgress	297
17.9.1.5	mcp.connect.onPromptsChanged	297
17.9.1.6	mcp.connect.onResourceUpdated	298
17.9.1.7	mcp.connect.onResourcesChanged	298
17.9.1.8	mcp.connect.onToolsChanged	299
17.9.1.9	mcp.connect.setLoggingLevel	299
17.9.1.10	mcp.connect.setRoots	300
17.9.1.11	mcp.connect.sse	301

17.9.1.12 mcp.connect.stdio	303
17.9.1.13 mcp.connect.subscribe	306
17.9.1.14 mcp.connect.unsubscribe	306
17.9.2 mcp.serve	307
17.9.2.1 mcp.serve.close	309
17.9.2.2 mcp.serve.completion	309
17.9.2.3 mcp.serve.listen	310
17.9.2.4 mcp.serve.onRootsChanged	312
17.9.2.5 mcp.serve.onSubscribe	313
17.9.2.6 mcp.serve.onUnsubscribe	313
17.9.2.7 mcp.serve.prompt	314
17.9.2.8 mcp.serve.removePrompt	315
17.9.2.9 mcp.serve.removeResource	315
17.9.2.10 mcp.serve.removeTool	316
17.9.2.11 mcp.serve.resource	316
17.9.2.12 mcp.serve.resourceTemplate	317
17.9.2.13 mcp.serve.resourceUpdated	318
17.9.2.14 mcp.serve.stdio	318
17.9.2.15 mcp.serve.tool	319
17.10 net	320
17.10.1 net.browser	320
17.10.1.1 net.browser.open	320
17.10.2 net.capture	321
17.10.2.1 net.capture.interfaces	321
17.10.2.2 net.capture.open	321
17.10.2.3 net.capture.openFile	322
17.10.2.4 net.capture.routes	323
17.10.2.5 net.capture.toFile	323
17.10.3 net.email	324
17.10.3.1 net.email.all	324
17.10.3.2 net.email.bimi	324
17.10.3.3 net.email.dmarc	325
17.10.3.4 net.email.mtaSts	325
17.10.3.5 net.email.send	326
17.10.3.6 net.email.spf	327
17.10.3.7 net.email.tlsRpt	327
17.10.4 net.http	328
17.10.4.1 net.http.get	328
17.10.4.2 net.http.post	328
17.10.4.3 net.http.request	328
17.10.5 net.icmp	330
17.10.5.1 net.icmp.open	330
17.10.6 net.load	331
17.10.6.1 net.load.http	331
17.10.7 net.netstatus	331
17.10.7.1 net.netstatus.check	331
17.10.8 net.probe	332

17.10.8.1 net.probe.dns	332
17.10.8.2 net.probe.ntp	332
17.10.8.3 net.probe.ping	333
17.10.8.4 net.probe.smtp	333
17.10.8.5 net.probe.tcp	334
17.10.8.6 net.probe.tls	334
17.10.8.7 net.probe.traceroute	335
17.10.8.8 net.probe.whois	336
17.10.8.9 net.probe.wss	336
17.10.9 net.raw	337
17.10.9.1 net.raw.open	337
17.10.9.2 net.raw.tcp	337
17.10.10 net.tcp	338
17.10.10.1 net.tcp.connect	338
17.10.11 net.udp	339
17.10.11.1 net.udp.open	339
17.11 runtime	340
17.11.1 runtime.argv	340
17.11.2 runtime.assert	340
17.11.2.1 runtime.assert.equal	340
17.11.2.2 runtime.assert.ok	340
17.11.3 runtime.clearDeadline	341
17.11.4 runtime.clipboard	341
17.11.4.1 runtime.clipboard.available	341
17.11.4.2 runtime.clipboard.imageAvailable	341
17.11.4.3 runtime.clipboard.read	342
17.11.4.4 runtime.clipboard.readImage	342
17.11.4.5 runtime.clipboard.write	342
17.11.4.6 runtime.clipboard.writeImage	342
17.11.5 runtime.env	343
17.11.5.1 runtime.env.get	343
17.11.5.2 runtime.env.load	343
17.11.6 runtime.getDeadline	344
17.11.7 runtime.log	344
17.11.8 runtime.open	344
17.11.9 runtime.openAvailable	345
17.11.10 runtime.secrets	345
17.11.10.1 runtime.secrets.available	345
17.11.10.2 runtime.secrets.delete	345
17.11.10.3 runtime.secrets.get	346
17.11.10.4 runtime.secrets.set	346
17.11.11 runtime.setDeadline	347
17.11.12 runtime.stderr	347
17.11.12.1 runtime.stderr.capture	347
17.11.12.2 runtime.stderr.reset	347
17.11.12.3 runtime.stderr.scoped	348
17.11.12.4 runtime.stderr.silence	348

17.11.12.5 runtime.stderr.target	349
17.11.12.6 runtime.stderr.to	349
17.11.12.7 runtime.stderr.toFile	350
17.11.13 runtime.stdin	350
17.11.13.1 runtime.stdin.from	350
17.11.13.2 runtime.stdin.fromFile	351
17.11.13.3 runtime.stdin.fromString	351
17.11.13.4 runtime.stdin.lines	352
17.11.13.5 runtime.stdin.read	352
17.11.13.6 runtime.stdin.readBytes	352
17.11.13.7 runtime.stdin.readLine	353
17.11.13.8 runtime.stdin.reset	353
17.11.13.9 runtime.stdin.scoped	353
17.11.13.10 runtime.stdin.source	354
17.11.14 runtime.stdout	354
17.11.14.1 runtime.stdout.capture	354
17.11.14.2 runtime.stdout.reset	355
17.11.14.3 runtime.stdout.scoped	355
17.11.14.4 runtime.stdout.silence	356
17.11.14.5 runtime.stdout.target	356
17.11.14.6 runtime.stdout.to	356
17.11.14.7 runtime.stdout.toFile	357
17.11.15 runtime.termSize	358
17.11.16 runtime.time	358
17.11.16.1 runtime.time.format	358
17.11.16.2 runtime.time.nowMs	359
17.11.16.3 runtime.time.sleep	359
17.12 server	359
17.12.1 server.http	359
17.12.1.1 server.http.listen	359
17.12.1.2 server.http.static	360
17.12.2 server.https	361
17.12.2.1 server.https.listen	361
17.12.2.2 server.https.static	362
17.12.3 server.icmp	362
17.12.3.1 server.icmp.listen	362
17.12.4 server.smtp	363
17.12.4.1 server.smtp.listen	363
17.12.5 server.tcp	364
17.12.5.1 server.tcp.listen	364
17.12.6 server.udp	365
17.12.6.1 server.udp.listen	365
17.13 services	366
17.13.1 services.agentBrowser	366
17.13.1.1 services.agentBrowser.auth	366
17.13.1.1.1 services.agentBrowser.auth.delete	366
17.13.1.1.2 services.agentBrowser.auth.list	366

17.13.1.1.3 services.agentBrowser.auth.save	366
17.13.1.1.4 services.agentBrowser.auth.show	367
17.13.1.2 services.agentBrowser.available	367
17.13.1.3 services.agentBrowser.batch	367
17.13.1.4 services.agentBrowser.chat	367
17.13.1.5 services.agentBrowser.clearDefaultOptions	368
17.13.1.6 services.agentBrowser.click	368
17.13.1.7 services.agentBrowser.clipboard	368
17.13.1.8 services.agentBrowser.close	369
17.13.1.9 services.agentBrowser.cmd	369
17.13.1.10 services.agentBrowser.cookies	369
17.13.1.11 services.agentBrowser.defaultOptions	370
17.13.1.12 services.agentBrowser.diff	370
17.13.1.13 services.agentBrowser.eval	370
17.13.1.14 services.agentBrowser.fill	371
17.13.1.15 services.agentBrowser.find	371
17.13.1.16 services.agentBrowser.get	372
17.13.1.17 services.agentBrowser.handle	372
17.13.1.17.1 services.agentBrowser.handle.snapshot	372
17.13.1.18 services.agentBrowser.inspect	373
17.13.1.19 services.agentBrowser.launch	373
17.13.1.20 services.agentBrowser.network	375
17.13.1.21 services.agentBrowser.open	375
17.13.1.22 services.agentBrowser.pdf	376
17.13.1.23 services.agentBrowser.profiler	376
17.13.1.24 services.agentBrowser.pushstate	376
17.13.1.25 services.agentBrowser.react	377
17.13.1.26 services.agentBrowser.record	377
17.13.1.27 services.agentBrowser.screenshot	378
17.13.1.28 services.agentBrowser.set	378
17.13.1.29 services.agentBrowser.setDefaultOptions	378
17.13.1.30 services.agentBrowser.snapshot	379
17.13.1.31 services.agentBrowser.storage	379
17.13.1.32 services.agentBrowser.stream	380
17.13.1.33 services.agentBrowser.tabs	380
17.13.1.34 services.agentBrowser.trace	381
17.13.1.35 services.agentBrowser.version	381
17.13.1.36 services.agentBrowser.vitals	381
17.13.2 services.ai	382
17.13.2.1 services.ai.providers	382
17.13.2.2 services.ai.send	382
17.13.3 services.doctor	382
17.13.4 services.exec	383
17.13.4.1 services.exec.http	383
17.13.4.2 services.exec.interactive	384
17.13.4.3 services.exec.shell	384
17.13.4.4 services.exec.stream	385

17.13.5 services.gh	386
17.13.5.1 services.gh.authStatus	386
17.13.5.2 services.gh.prList	387
17.13.5.3 services.gh.repoView	387
17.13.6 services.git	388
17.13.6.1 services.git.add	388
17.13.6.2 services.git.branch	388
17.13.6.3 services.git.commit	388
17.13.6.4 services.git.diffStat	389
17.13.6.5 services.git.isClean	389
17.13.6.6 services.git.log	389
17.13.6.7 services.git.revParse	390
17.13.6.8 services.git.runText	390
17.13.6.9 services.git.status	391
17.13.7 services.pdf	391
17.13.7.1 services.pdf.available	391
17.13.7.2 services.pdf.backend	392
17.13.7.3 services.pdf.info	392
17.13.7.4 services.pdf.toHtml	392
17.13.7.5 services.pdf.toImage	393
17.13.7.6 services.pdf.toText	393
17.13.7.7 services.pdf.tools	394
17.13.7.7.1 services.pdf.tools.pdfinfo	394
17.13.7.7.2 services.pdf.tools.pdfhtml	394
17.13.7.7.3 services.pdf.tools.pdfppm	394
17.13.7.7.4 services.pdf.tools.pdftext	394
17.13.7.8 services.pdf.version	394
17.13.8 services.typst	394
17.13.8.1 services.typst.available	394
17.13.8.2 services.typst.compile	395
17.13.8.3 services.typst.fonts	396
17.13.8.4 services.typst.query	396
17.13.8.5 services.typst.version	396
17.13.9 services.webdriver	397
17.13.9.1 services.webdriver.available	397
17.13.9.2 services.webdriver.connect	397
17.13.9.3 services.webdriver.probe	399
17.14 text	400
17.14.1 text.case	400
17.14.1.1 text.case.ada	400
17.14.1.2 text.case.camel	400
17.14.1.3 text.case.camelSnake	401
17.14.1.4 text.case.capital	401
17.14.1.5 text.case.cobol	401
17.14.1.6 text.case.convert	401
17.14.1.7 text.case.detect	402
17.14.1.8 text.case.dot	402

17.14.1.9 text.case.flat	403
17.14.1.10 text.case.header	403
17.14.1.11 text.case.kebab	403
17.14.1.12 text.case.names	403
17.14.1.13 text.case.pascal	404
17.14.1.14 text.case.path	404
17.14.1.15 text.case.screamingKebab	404
17.14.1.16 text.case.screamingSnake	405
17.14.1.17 text.case.sentence	405
17.14.1.18 text.case.slug	405
17.14.1.19 text.case.snake	405
17.14.1.20 text.case.split	406
17.14.1.21 text.case.title	406
17.14.1.22 text.case.train	406
17.14.1.23 text.case.upperFlat	407
17.14.2 text.charset	407
17.14.2.1 text.charset.decode	407
17.14.2.2 text.charset.detect	407
17.14.2.3 text.charset.encode	408
17.14.3 text.diff	408
17.14.3.1 text.diff.compare	408
17.14.4 text.jq	409
17.14.4.1 text.jq.query	409
17.14.4.2 text.jq.queryAll	409
17.14.5 text.markdown	410
17.14.5.1 text.markdown.toHtml	410
17.14.6 text.preg	410
17.14.6.1 text.preg.match	410
17.14.6.2 text.preg.matchAll	411
17.14.6.3 text.preg.replace	411
17.14.7 text.preg2	411
17.14.7.1 text.preg2.match	411
17.14.7.2 text.preg2.matchAll	412
17.14.7.3 text.preg2.replace	412
17.14.8 text.stego	413
17.14.8.1 text.stego.embed	413
17.14.8.2 text.stego.extract	413
17.14.9 text.str	413
17.14.9.1 text.str.base64Decode	413
17.14.9.2 text.str.base64Encode	414
17.14.9.3 text.str.base64UrlDecode	414
17.14.9.4 text.str.base64UrlEncode	414
17.14.9.5 text.str.br2nl	415
17.14.9.6 text.str.htmlEntityDecode	415
17.14.9.7 text.str.lpad	415
17.14.9.8 text.str.ltrim	416
17.14.9.9 text.str.nl2br	416

17.14.9.10 text.str.normalizeNewlines	416
17.14.9.11 text.str.pad	417
17.14.9.12 text.str.printf	417
17.14.9.13 text.str.reverse	417
17.14.9.14 text.str.rpad	418
17.14.9.15 text.str.rtrim	418
17.14.9.16 text.str.sprintf	418
17.14.9.17 text.str.stripHtml	419
17.14.9.18 text.str.trim	419
17.14.9.19 text.str.urlDecode	419
17.14.9.20 text.str.urlEncode	420
17.15 tui	420
17.15.1 tui.layout	420
17.15.2 tui.onKey	421
17.15.3 tui.pane	421
17.15.4 tui.waitKey	422
17.16 web	422
17.16.1 web.feed	422
17.16.1.1 web.feed.load	422
17.16.1.2 web.feed.parse	423
17.16.2 web.html	423
17.16.2.1 web.html.load	423
17.16.2.2 web.html.parse	424
17.16.3 web.sitemap	424
17.16.3.1 web.sitemap.load	424
17.16.3.2 web.sitemap.parse	425

sercon manual

Long-form reference for the `pkg/scriptengine` library, the `sercon` CLI, the built-in script API, and the JavaScript runtime surface that `scriptengine` exposes via `goja` and `goja_nodejs`. Examples are runnable — save snippets as `.ts` files and execute with `sercon file.ts` unless otherwise noted.

1. Overview

`sercon` is a Go program that runs TypeScript files. It's built from two parts:

- `pkg/scriptengine` — a library you embed in your own Go code. You register Go-callable bindings, then call `Run` / `RunFile` to execute a `.ts` script that talks to them.
- `cmd/sercon` — a thin CLI built on the library, useful for ad-hoc test scripts and as a working example of every binding kind.

sercon is CLI-first. The `sercon` command — reconnaissance, troubleshooting, testing — is the supported product; `pkg/scriptengine` exists to serve it. Embedding the engine as a library in your own Go program is **unsupported and at your own risk**: its API may change without notice, and there are no stability or sandboxing guarantees. The Library API in §3 is documented because the CLI is built on it, not as a stability contract.

The runtime is pure Go: `goja` is the JS engine, `esbuild` (used as a Go library) is the TS→JS transpiler, and `goja_nodejs` provides Promises, `setTimeout`, `console`, and CommonJS `require`. No `cgo`, no `Node`.

2. Quickstart

Install:

```
go install github.com/codedeviate/sercon/cmd/sercon@latest
```

Run one of the bundled examples:

```
sercon examples/scripts/smoke.ts examples/scripts/async.ts
```

...or run them all via `make demo`. `examples/scripts/` ships a runnable `.ts` (or `.tsx`) file per feature area — `hash.ts`, `strings.ts`, `path-and-time.ts`, `default-export.ts`, `tsx-demo.ts`, and the original `smoke.ts` / `async.ts` / `hang.ts` (`hang.ts` is the timeout demo and intentionally exits non-zero).

Generate a declaration file for editor autocomplete:

```
sercon --emit-dts sercon.d.ts
```

Embed in your own Go program:

```
eng := scriptengine.New(scriptengine.Options{
    Timeout:    5 * time.Second,
    ScriptRoot: "./scripts",
})
eng.Register("upper", strings.ToUpper)
val, err := eng.RunFile(ctx, "scripts/main.ts")
```

2.1 Concepts

sercon runs a `.ts` file as a program. It is a **pure-Go TypeScript runner**: goja executes the JavaScript, esbuild transpiles the TypeScript ahead of it, and there is no Node and no cgo. A few things are worth knowing before you write your first script:

- **Async is built in.** Top-level `await`, Promises, and `setTimeout` all work; a script that returns a pending Promise keeps running until it settles.
- **Each run is isolated.** Every invocation (and every `--watch` re-run) gets a fresh runtime — no global or module state leaks between runs.
- **Batteries are top-level globals.** You don't `import` the built-ins; HTTP, files, crypto, databases, and more are always-present globals (mapped below).
- **Binding errors throw.** When a built-in fails it throws a normal JS exception you can `try / catch` — see §13.

The reserved globals — what's in the box. sercon scripts get **fifteen** reserved top-level globals plus a `console` shim. Use them directly; there is no enclosing namespace. This is the map — follow the link for the full treatment.

Global	What it's for	Reference
<code>runtime</code>	script-host scaffolding: <code>argv</code> , <code>env</code> , <code>secrets</code> , <code>log</code> , <code>sleep</code>	§5.2
<code>crypto</code>	hashing, JWT, age encryption, random bytes	§5.3
<code>text</code>	charset decode, string utilities, <code>base64</code> , <code>jq</code>	§5.4
<code>codec</code>	encode/decode: barcode, check-digit, compression, docs, dotenv, perl, php, spreadsheets, TOML, XML	§5.5
<code>fs</code>	file read/write, <code>mkdir</code> , <code>stat</code> , <code>exists</code> , <code>remove</code>	§5.6
<code>net</code>	HTTP client, email send, DNS, TCP/UDP, ping	§5.7
<code>db</code>	SQL (query/exec/transactions) + Redis/Valkey/memcached, LDAP, DICT clients	§5.8
<code>server</code>	inbound listeners: HTTP/HTTPS, SMTP, WebSocket	§6
<code>services</code>	external-CLI wrappers: <code>git</code> , <code>gh</code> , <code>ai</code> , <code>agentBrowser</code> , <code>webdriver</code>	§5.9
<code>tui</code>	terminal-UI widgets	§5.10
<code>image</code>	decode/encode/transform images	§5.11
<code>web</code>	fetch + parse: feeds, sitemaps, HTML	§5.12
<code>audio</code>	read/write/convert audio, steganography	§5.13
<code>cloud</code>	provider APIs — AWS, Azure, Google Cloud (object storage, ...)	§5.14
<code>mcp</code>	Model Context Protocol — server (<code>serve</code>) and client (<code>connect</code>): tools/resources/prompts over stdio or HTTP	§5.15
<code>console</code>	browser/Node-compatible logging shim	§5.1

The full per-binding signatures live in the generated §17 binding reference and in `examples/scripts/sercon.d.ts`.

2.2 Recipes

Five short tasks to get from an empty file to a working script. Each hands off to the section with the full detail.

2.2.1 Write and run your first script

Save a `.ts` file and run it. `console.log` (or `runtime.log`) writes one clean line to stdout.

```
// first.ts
const name = "world";
console.log(`hello, ${name}`);
```

```
sercon first.ts          # → hello, world
```

Notes

- `console.log` / `runtime.log` print space-joined, prefix-free lines; objects render as JSON (see §5.1).

2.2.2 Read input: arguments and environment

User arguments arrive on `runtime.argv` (everything after `--`); environment variables come from `runtime.env.get` (returns `undefined` when unset).

```
// greet.ts
const who = runtime.argv[2] ?? "world";          // argv = [prog,
script, ...userArgs]
const shout = runtime.env.get("SHOUT") === "1";
console.log(shout ? `HELLO, ${who.toUpperCase()}!` : `hello, ${who}`);
```

```
sercon greet.ts -- Ada          # → hello, Ada          (see §4.1)
SHOUT=1 sercon greet.ts -- Ada  # → HELLO, ADA!
```

Notes

- Argument passing and `--` are covered in §4.1 `runtime` in §5.2.

2.2.3 Call an HTTP API

`net.http.request(method, url, opts?)` returns `{ status, ok, headers, body, bodyBytes, url }`. It does **not** throw on 4xx/5xx — check `ok` / `status` yourself.

```
// stars.ts
const res = await net.http.request("GET", "https://api.github.com/repos/
codeviate/sercon");
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const repo = JSON.parse(res.body);
console.log(`${repo.full_name}: ${repo.stargazers_count} stars`);
```

Notes

- Headers, retries, timeouts, auth, and multipart uploads: §5.7 / §17.

2.2.4 Read and write a file

`fs.readText` / `fs.writeText` are Promise-returning; there is no sandbox, so paths are ordinary filesystem paths.


```
// count.ts
const text = await fs.readText("input.txt");
const lines = text.split("\n").length;
await fs.writeText("report.txt", `lines: ${lines}\n`);
console.log(`wrote report.txt (${lines} lines)`);
```

Notes

- `writeText` fails if the parent directory is missing (Node-like) — `mkdir` first. Bytes, `stat`, `exists`, `remove`: §5.6.

2.2.5 Find the rest of the API

Generate editor autocomplete and browse the full surface.

```
sercon init                # writes sercon.d.ts + jsconfig.json for your
editor (§4.3)
sercon --emit-dts sercon.d.ts # emit just the .d.ts to a path
sercon --examples           # colorized, in-depth tour of every feature area
```

Notes

- The complete signatures are the generated §17 binding reference editor tooling is §4.3 / §4.6.8.

3. Library API — pkg/scriptengine

> **Unsupported surface.** `pkg/scriptengine` is CLI-serving — documented > here for completeness, not as a stability contract (see §1). Its API may > change without notice, and there are no sandboxing guarantees; embed it at > your own risk.

3.1 Engine, Options, New

```
type Options struct {
    Timeout      time.Duration // wall-clock limit per Run; 0 disables
    ScriptRoot   string        // base for require/import resolution
    DisableConsole bool          // turn off the goja_nodejs console module
    Verbose      io.Writer     // diagnostic traces, prefixed "[sercon] "
    ModuleLoader func(candidatePath string) (source string, found bool, err
error)
    ProgramName  string        // argv[0] for runtime.argv; defaults to
filepath.Base(os.Args[0])
    WatchMode    bool          // set by the CLI's --watch loop; read via
Engine.WatchMode()
}

func New(opts Options) *Engine
```

`ModuleLoader`, when non-nil, is consulted for every require/import candidate path **before** the filesystem — the hook for embedders that want to serve modules from somewhere other than disk (an in-memory FS, a network source, an embedded bundle). `goja` probes several candidates per specifier (`./x`, `./x.ts`, `./x/index.ts`, ...); the engine also tries the bare path plus the usual extension fallbacks (`.ts` / `.tsx` / `.js` / `.mjs` / `.cjs` / `.json`) against the loader, so a loader can match on a plain suffix. Return `(source, true, nil)` to serve the

module (transpiled when the path ends in `.ts` / `.tsx`, exactly like a disk read);
 (`""`, `false`, `nil`) to fall through to the filesystem; (`""`, `false`, `err`) to abort resolution.

```
eng := scriptengine.New(scriptengine.Options{
  ModuleLoader: func(path string) (string, bool, error) {
    if strings.HasSuffix(path, "greeting.ts") {
      return `export const hi = (n: string) => "hi " + n;`, true, nil
    }
    return "", false, nil // fall through to disk
  },
})
```

`Engine` is the host of all registered bindings. Construct it once, then call `Run` / `RunFile` as many times as needed — each call gets a *fresh* `*goja.Runtime` so script state never leaks between invocations.

`ScriptRoot` defaults to the working directory at `New` time if left empty. Both `Timeout` and `ScriptRoot` can be overridden on a per-Run basis only by constructing a fresh `Engine` see Out-of-scope for the per-Run override on the roadmap.

3.2 Registering bindings

Function	Use when...
<code>Register(name, value)</code>	The binding is a pure function, struct, or primitive that doesn't need access to the runtime.
<code>RegisterNamespace(name, members map[string]any)</code>	You want to expose <code>name.x</code> , <code>name.y</code> without defining a Go struct.
<code>RegisterConstructor(name, ctor)</code>	The binding is a Go function whose return value should be treated as a class in the emitted <code>.d.ts</code> . (Runtime semantics today are identical to <code>Register</code> the <code>d.ts</code> emitter is the only differentiator.)
<code>RegisterFactory(name, factory func(vm, loop) any)</code>	The binding needs the per-Run <code>*goja.Runtime</code> or <code>*eventloop.EventLoop</code> in scope (typical for Promise-returning I/O).
<code>RegisterNamespaceFactory(name, factory)</code>	Same, but the factory returns the full member map.

Example — synchronous function binding:

```
eng.Register("greet", func(name string) string { return "hi " + name })
```

Example — namespace with a sync member:

```
eng.RegisterNamespace("math2", map[string]any{
  "square": func(n float64) float64 { return n * n },
  "pi":     3.14159,
})
```

Example — async (Promise-returning) binding via `PromisifyAsync`:

```
eng.RegisterFactory("httpGet", func(vm *goja.Runtime, loop *eventloop.EventLoop)
any {
    return scriptengine.PromisifyAsync(vm, loop,
        func(ctx context.Context, call goja.FunctionCall) (map[string]any, error)
    {
        url := call.Argument(0).String()
        resp, err := http.Get(url)
        if err != nil {
            return nil, err
        }
        defer resp.Body.Close()
        body, _ := io.ReadAll(resp.Body)
        return map[string]any{"status": resp.StatusCode, "body":
string(body)}, nil
    })
})
```

3.3 Run, RunFile

```
func (e *Engine) Run(ctx context.Context, name, source string, opts ...RunOption)
(goja.Value, error)
func (e *Engine) RunFile(ctx context.Context, path string, opts ...RunOption)
(goja.Value, error)
```

`Run` executes `source` as an entry-script TS. `name` is used in stack traces. The returned value is the resolved value of the script's top-level `export default <expr>`, if any; scripts without a default export resolve to `undefined` (see §9. TypeScript support for how the `sercon` CLI uses this to print a script's result as JSON).

Both methods respect `Options.Timeout` and `ctx` cancellation, whichever fires first; the resulting error is either `ErrScriptTimeout`, `ctx.Err()`, or the underlying JS exception.

3.3.1 Per-Run options

```
type RunOption func(*runConfig)

func WithScriptRoot(dir string) RunOption
```

Reuse a single Engine across many scripts that each live in their own directory by passing `WithScriptRoot(dir)` to `Run` / `RunFile`. The override applies only to this call; `Options.ScriptRoot` is used for every other `Run`.

```
_, _ = eng.Run(ctx, "main.ts", source, scriptengine.WithScriptRoot("/path/to/
run42"))
```

```
func WithArgs(args []string) RunOption
```

Set the per-script argument vector. The script reads it as `runtime.argv`, a global with Node/Bun layout: `argv[0]` is `Options.ProgramName` (defaults to `filepath.Base(os.Args[0])` the CLI sets it to `"sercon"`), `argv[1]` is the running script's path, and the `WithArgs` values

follow from index 2. The global is always present — with no args it is just

```
[programName, scriptPath] .
```

```
_, _ = eng.Run(ctx, "main.ts", source, scriptengine.WithArgs([]string{"--port",
"8080"}))
// inside the script: runtime.argv == [ProgramName, "/abs/main.ts", "--port",
"8080"]
```

3.4 Reset

```
func (e *Engine) Reset()
```

Clears every registered binding. Useful when a long-lived Engine is reused across unrelated batches of scripts that each want a clean global namespace. **Not safe to call concurrently with Run / RunFile.**

3.5 WriteTypes

```
func (e *Engine) WriteTypes(w io.Writer) error
```

Emits a `.d.ts` describing the registered surface. See section

14. Type generation

for what the mapping looks like.

3.6 SetDocs and SetMemberDocs

```
func (e *Engine) SetDocs(path, doc string)
func (e *Engine) SetMemberDocs(namespace string, docs map[string]string)
```

Attach JSDoc strings to registered bindings so the emitted `.d.ts` grows readable editor hover. `SetDocs` takes a dotted path — either a bare top-level name (`"greet"`, `"http"`) or `"namespace.member"` for namespace members (`"http.get"`, `"exec.shell"`); `SetMemberDocs` is the convenience for documenting many members of a namespace at once (the namespace itself can still be documented separately via `SetDocs`).

Multi-line docs are written as `\n`-separated strings; the emitter expands them into a standard `*`-prefixed JSDoc block. Single-line docs collapse to `/** ... */`. Calling `SetDocs` with an empty string removes any previous doc for that path. Bindings without a doc entry get no JSDoc block at all — the emitter doesn't insert empty `/** */` placeholders.

`SetDocs` may be called before or after the matching `Register...` call; docs are looked up by name at emit time. Like the registration methods, `SetDocs` must not race with a `Run` / `WriteTypes` / `Reset` on the same engine.

```
eng.Register("greet", func(name string) string { return "hi " + name })
eng.SetDocs("greet", "Greet someone by name.")

eng.RegisterNamespace("math2", map[string]any{
    "pi": 3.14159,
    "squared": func(n float64) float64 { return n * n },
})
eng.SetDocs("math2", "Tiny math helpers.")
```

```
eng.SetMemberDocs("math2", map[string]string{
    "pi":      "Circumference / diameter ratio.",
    "squared": "Multiply a number by itself.",
})
```

3.7 PromisifyAsync[T] and AsyncBinding

```
type AsyncBinding struct {
    Func      func(goja.FunctionCall) goja.Value
    TSReturnType string
}

func PromisifyAsync[T any](vm *goja.Runtime, loop *eventloop.EventLoop,
    work func(ctx context.Context, call goja.FunctionCall) (T, error),
) AsyncBinding
```

`PromisifyAsync` turns blocking Go work into a JS Promise. It launches the work in a goroutine, parks a `SetTimeout` sentinel so `loop.Run` doesn't return early, and schedules the resolve/reject back onto the event loop. **Required** for any Promise-returning binding — `RunOnLoop` alone is not counted as a live job by the event loop.

`PromisifyAsync` returns an `AsyncBinding` carrier rather than the bare callback. The engine unwraps it to `.Func` at `vm.Set` time so goja's special-case detection of `func(goja.FunctionCall) goja.Value` host-callbacks still fires; the `.d.ts` emitter reads `.TSReturnType` to emit `Promise<T>` (mapped from the generic `T` via reflect at construction time) instead of the previous `unknown`. Hosts assigning the result into a `map[string]any` for `RegisterNamespace` / `RegisterNamespaceFactory` get the unwrap recursively too — no manual work required.

`RunContextFromVM(vm) context.Context`. Returns the `context.Context` of the `Engine.Run` currently executing on `vm`, or `context.Background()` when `vm` is not inside a `Run`. A host binding that spawns its own long-lived goroutine or subprocess should root it at this context so a `Run` timeout / cancellation reaches it (the built-in `mcp.connect` client uses it to tie an open connection and its stdio child to the `Run`). `PromisifyAsync` bindings already observe the `Run` context automatically and don't need this.

3.8 Version

```
import "github.com/codedeviate/sercon/pkg/scriptengine"

fmt.Println(scriptengine.Version) // "0.1.0"
```

Bumped in lockstep with the git tag.

4. CLI — sercon

```
sercon [flags] <script.ts> [script.ts ...]
sercon [flags] -                          # read entry script from stdin
sercon --examples | --help | --version
```

Flag	Purpose
------	---------

<code>-timeout DURATION</code>	Wall-clock limit per script (default 10s 0 disables).
<code>-root DIR</code>	Override the script root for <code>require</code> / <code>import</code> resolution.
<code>-emit-dts PATH</code>	Write the example bindings' <code>.d.ts</code> to <code>PATH</code> and exit.
<code>-emit-reference PATH</code>	Write the markdown binding reference to <code>PATH</code> and exit.
<code>-v, --verbose</code>	Print a <code>PASS <name> (<time>)</code> line per script on success (default is quiet — only failures print). Also traces the rewritten entry-script JS and each module resolution to stderr, and prints duration on failure.
<code>--silent</code>	Suppress the runner's <code>PASS</code> / <code>FAIL</code> status lines entirely. Script console output and the <code>export-default</code> result still print; failure is signalled only by the exit code.
<code>-h, --help</code>	In-depth colorized help.
<code>--examples</code>	In-depth colorized walkthrough of every feature.
<code>--no-pager</code>	Don't page <code>--help</code> / <code>--examples</code> . By default, when stdout is a terminal they pipe through <code>\$PAGER</code> (falling back to <code>less</code> with <code>LESS=FRX</code> , color preserved); a pipe/redirect, <code>--no-pager</code> , or <code>PAGER=cat</code> renders directly.
<code>--version</code>	Print the engine version (plus goja/esbuild build-info versions).
<code>--doctor</code>	Check external tool requirements (git, gh, AI providers, agent-browser, chromedriver/geckodriver, typst, poppler/pdf, recon/curl, clipboard/image): installed?, version, purpose — and validate the chromedriver↔Chrome major-version match; then exit. Prints a category-grouped table. Exits 0 normally (missing tools are optional), 5 on a detected compatibility conflict. Same report as the <code>services.doctor</code> binding.
<code>--watch</code>	Re-run on file change under the script root. Debounced 150 ms. Ctrl-C exits.
<code>--secrets-prefix=P</code>	Namespace prefix for <code>runtime.secrets</code> keystore items. Overrides <code>\$SERCON_SECRETS_PREFIX</code> default <code>sercon/</code> .
<code>--env-file=PATH</code>	Load <code>KEY=VALUE</code> pairs from a <code>.env</code> file into the environment before running (repeatable). Parses <code>KEY=VALUE</code> , <code>#</code> comments, blank lines, an optional leading <code>export</code> , and optional surrounding quotes — no shell expansion. A variable already in the real environment always wins; among multiple files, a later file overrides an earlier one. Replaces the <code>set -a; source .env; set +a</code> ritual and makes shebang scripts self-sufficient.

Each positional argument is either a path to a `.ts` / `.tsx` file or `-` to read an entry script from standard input:

```
echo 'runtime.log(1 + 2);' | sercon -
sercon prelude.ts -                # prelude then stdin
```

4.1 Passing arguments: `--` and `runtime.argv`

Everything after a standalone `--` is the script argument vector, exposed to every script via `runtime.argv`:

```
sercon run.ts -- --port 8080 hello
```

`runtime.argv` uses the Node/Bun layout `[programName, scriptPath, ...userArgs]` — `argv[0]` is `"sercon"`, `argv[1]` is the path of the script currently executing, and the post-`--` arguments start at index 2 (so `runtime.argv.slice(2)` is just your args). All scripts in one invocation share the same `--` tail, but each sees its own path at `argv[1]`. The global is always present; with no `--` it is just `[programName, scriptPath]`.

```
const args = runtime.argv.slice(2); // ["--port", "8080", "hello"]
```

Exit codes are distinct per failure type so shells can react sensibly. When several scripts run, the highest applicable code wins:

Code	Meaning
0	every script passed.
1	CLI usage error (unknown flag, missing script argument, ...).
2	at least one script failed to transpile — never ran.
3	at least one script timed out (<code>-timeout</code>) or was context-cancelled.
4	at least one script ran and threw a JS exception.
5	<code>--doctor</code> detected a compatibility conflict (e.g. chromedriver↔Chrome major mismatch).

The multi-script runner is **quiet on success by default** — it only prints

`FAIL <name>: <error> lines`. Pass `-v / --verbose` to also print a `PASS <name> (<time>)` line per script. `--silent` suppresses both status lines; the script's own `runtime.log / console.log` output and any `export default` result still appear, and the exit code is unchanged.

`-v` additionally writes lines prefixed with `[sercon]` to `stderr`. The traces include the full rewritten entry-script JS (the form goja actually runs, after the ESM→CJS rewrite + async IIFE wrapper) and every module-resolution event, so debugging an unexpected resolve target or a mis-rewritten import is straightforward.

The CLI registers fifteen reserved top-level globals; see the next section.

4.2 `--watch` : re-run on file change

`sercon --watch <script.ts>` runs the script once and then blocks, re-running it every time a watched file changes under the script root. Useful for iterating on a script body or on the binding surface during development.

```
sercon --watch examples/scripts/smoke.ts
```

What's watched:

- The script root (resolved the same way as the one-shot mode — `-root DIR` if set, else `dirname` of the first script argument).
- Recursively. Symlinks aren't followed.
- New directories that appear during the session are picked up automatically.

What's filtered:

- **File extensions:** `.ts` / `.tsx` / `.js` / `.jsx` / `.json` trigger a re-run; `.d.ts` files match via suffix too. Anything else (Markdown, images, editor swap files, build artefacts) is ignored.
- **Directories:** hidden directories (`.git`, `.vscode`, anything starting with `.`) and `node_modules` are excluded — both because they generate floods of irrelevant events and because recursing into them inflates the watcher's directory count for no gain.

Re-runs are **debounced 150 ms**. Editors typically fire several events per save (write → rename → chmod); collapsing them into a single re-run keeps the loop responsive without thrashing. The debounce window resets on every event, so a rapid burst of saves becomes one re-run after the burst settles.

Each run is delimited by a line like `--- sercon re-run @ HH:MM:SS ---` so the output is visually distinct from the previous run.

`Ctrl-C` (SIGINT) or `SIGTERM` exit cleanly. Per-script failures inside a watch session log as `FAIL` but don't terminate the loop — a syntax error you're trying to fix shouldn't kick you out of watch mode.

The watcher reuses the same `Engine` across runs. Each `Run` already gets a fresh `*goja.Runtime`, so re-running is just re-invoking `runOne` on the existing engine — there's no per-iteration setup cost beyond the transpile + module-resolution work.

Module-graph invalidation. Watch mode tracks each entry script's import graph (its own file plus every module it resolves, captured via `Engine.SetResolveHook`) during the run. On a file change it re-runs **only the entries whose graph includes the changed file** — editing a helper that just one of three entry scripts imports re-runs that one entry, not all three. An entry's own file counts (editing it re-runs it); stdin entries and any entry whose graph isn't known yet re-run unconditionally (conservative). Before each re-run the module cache is busted (`Engine.ResetModuleCache`) so an edited import's new source actually takes effect — the registry otherwise caches compiled bytecode across runs.

4.3 Editor autocomplete (`sercon init`)

```
sercon init [dir]
```

Drops two files into `dir` (default the current directory) so any editor backed by the TypeScript language server (VSCode, Zed, Neovim+coc, Sublime LSP, ...) gives completion and hover docs for the reserved globals with no plugin or manual config:

- **`sercon.d.ts`** — the binding declarations (same content as `sercon -emit-dts`): ambient `declare const` blocks for `runtime`, `crypto`, `text`, `codec`, `fs`, `net`, `db`, `server`, `services`, `tui`, `image`, `web`, `audio`, and `console`, each with JSDoc.
- **`jsconfig.json`** — points the language server at `sercon.d.ts` and sets `moduleResolution`: `"Bundler"` (so the extensionless relative imports sercon scripts use `resolve`) and `types`: `[]` (sercon isn't Node, so no stray `@types/*`).

Existing files are left untouched unless `--force` is given. The `examples/scripts/` directory ships with both files already, so the bundled demos autocomplete out of the box. For a single file without a config, the no-setup fallback is a leading

```
/// <reference path="./sercon.d.ts" /> .
```

4.4 Shebang lines and executable scripts (`sercon run`)

A script may begin with a `#!` shebang line. It is stripped before transpile (the line is blanked in place, so transpile/syntax error line numbers still match the source), which means a `.ts` / `.tsx` / `.js` file can be made directly executable:

```
#!/usr/bin/env sercon
runtime.log("hello from an executable script");
```

```
chmod +x hello.ts
./hello.ts
```

The kernel launches a shebang script as `sercon <script> <args...>`, and in the default mode every positional is treated as a *separate script* — so positional arguments to a shebang script would be mistaken for additional script paths. For an executable script that takes arguments, use the **run subcommand** in the shebang:

```
sercon run [flags] <script.ts> [args...]
```

```
#!/usr/bin/env -S sercon run
runtime.log("args:", JSON.stringify(runtime.argv.slice(2)));
```

`sercon run` executes exactly one script and hands every token after the script path to it as `runtime.argv[2:]` (Node/Bun layout: `[program, script, ...args]`) — no standalone `--` separator is needed (and a shebang line can't inject one). The `env -S` form splits the single shebang argument so the kernel runs `sercon run /abs/path/script.ts arg1 arg2 ...`. Flags (`-timeout`, `-root`, `-v`) are accepted before the script path; everything from the script path onward is positional and becomes script args. `env -S` is supported by GNU coreutils, macOS, and the BSDs.

```
sercon run script.ts --port 8080 alice    # argv[2:] = ["--port", "8080", "alice"]
```

Symlinked launchers. The natural way to expose a command is to keep the entry script in a project's `bin/` and symlink it onto `PATH` (`ln -s ~/proj/bin/tool ~/bin/tool`). `sercon` resolves the entry script through the symlink (`filepath.EvalSymlinks`) before computing the module root, so the script's relative `import / require` specifiers (and `runtime.argv[1]`) resolve against the **real** file's directory — `../lib/...` finds `~/proj/lib`, not `~/lib`. A broken symlink still errors cleanly. Only the entry path is resolved; already-required modules resolve against their own on-disk locations.

4.5 `sercon serve` : long-running scripts with production niceties

```
sercon serve [flags] <script.ts> [-- args...]
```

`sercon serve` is a sibling subcommand that wraps the same engine but adds the conveniences a process supervisor (systemd, docker, launchd) usually wants from a long-lived script. Use it whenever a script binds an HTTP/HTTPS listener (or any future `server.*` protocol) and is expected to keep running. Vanilla `sercon script.ts` also keeps the loop alive while listeners are bound, but gives you none of the deltas below.

Flag	Purpose
<code>--shutdown-timeout DURATION</code>	Graceful-shutdown deadline on SIGTERM/SIGINT (default <code>30s</code>). After the deadline the engine hard-cancels.
<code>--port-override N</code>	If non-zero, replaces every literal <code>port:</code> in <code>server.*.listen({...})</code> calls with <code>N</code> . Useful for spinning up the same script on a different port without editing source. Only literal port values are rewritten — computed expressions (<code>{port: getPort()}</code>) are left alone.
<code>-timeout DURATION</code>	Per-script wall-clock limit. Defaults to <code>0</code> (disabled) under <code>serve</code> so listeners don't get cut off; opt back in if you want one.
<code>-root DIR</code>	Script root for <code>require</code> / <code>import</code> resolution (same semantics as vanilla <code>sercon</code>).
<code>-v</code>	Verbose engine tracing to stderr.

Behavioural deltas vs vanilla `sercon` :

- **Access log to stderr.** Each request emits one line in the format `ts remote method path status dur_us`, e.g.
`2026-05-29T10:23:14Z 127.0.0.1:54321 GET /users/42 200 1843µs`. WebSocket upgrades log only the upgrade line; per-frame traffic is suppressed.
- **Readiness signal to stdout.** When each listener calls back into the binding with `bound`, the binding prints `READY` listening on `tcp/0.0.0.0:8080` to stdout. Multiple listeners produce one `READY` line each. Supervisors that read-line on startup (`systemd-notify`, `docker-compose` healthchecks) can wait on it.
- **Graceful shutdown.** SIGTERM / SIGINT invokes every active listener's close hook concurrently — the same underlying close path the script's own `srv.close()` takes (HTTP/HTTPS `Server.Shutdown`, SMTP `Server.Close`, TCP/UDP/ICMP socket close) — and waits up to `--shutdown-timeout` for them all to finish. Clean exit is exit code `0`. Past the deadline (or if a hook hangs), the run context is force-cancelled as a fallback, which drives the usual `vm.Interrupt + loop.Terminate` path.
- **No `--watch`.** Re-running while a listener is bound would race port rebinding; `sercon serve --watch ...` exits with a usage error. Use vanilla `sercon --watch script.ts` for the hot-reload development loop.

```
sercon serve examples/scripts/server-http.ts
sercon serve --port-override 9090 server.ts
sercon serve --shutdown-timeout 10s server.ts
```

4.6 Recipes

Common command-line patterns. (Flags are documented in the FLAGS table in §4 and in `sercon --help`.)

4.6.1 Pass arguments to a script

Everything after a standalone `--` becomes `runtime.argv` (see §4.1).

```
sercon report.ts -- --out /tmp/r.json prod
# report.ts: runtime.argv.slice(2) → ["--out", "/tmp/r.json", "prod"]
```

4.6.2 Load config and secrets from a `.env`

```
sercon --env-file .env deploy.ts
# .env lines (API_TOKEN=...) are readable via runtime.env.get("API_TOKEN");
# a real environment variable of the same name always wins.
sercon --secrets-prefix myapp/ deploy.ts
# namespaces runtime.secrets keystore items under "myapp/" instead of the
# default "sercon/" (also settable via $SERCON_SECRETS_PREFIX).
```

4.6.3 Auto-rerun while developing

Re-runs on every `.ts / .tsx / .js / .jsx / .json / .d.ts` change under the script root; `Ctrl-C` exits (see §4.2).

```
sercon --watch dashboard.ts
```

4.6.4 Bound a run and gate on the exit code

`-timeout` (default `10s`) caps the run; exceeding it exits non-zero (`ErrScriptTimeout`), as does any thrown/rejected script — so `$?` drives a CI gate (see §12 Timeouts, §13 Error semantics).

```
sercon -timeout 5s healthcheck.ts && echo "healthy" || echo "unhealthy ($?)"
```

4.6.5 Use `sercon` in a shell pipeline

`sercon -` reads the entry *script* (not data) from stdin; scripts write results to stdout via `console.log` / `runtime.log` for piping onward.

```
echo 'runtime.log(6 * 7)' | sercon -      # reads the script from stdin
sercon gen.ts | jq .                    # pipe a script's stdout into jq
```

Note: there is no `runtime.stdin` API for piped *data* — the positional `-` argument is for the script source itself. To consume piped data on Unix, read the pipe as a file from inside the script:

```
const input = await fs.readText("/dev/stdin");
```

4.6.6 Run a long-lived server

`sercon serve` keeps the event loop alive for a script that binds a listener, prints a `READY` line per listener, and shuts down gracefully on `SIGTERM/SIGINT` (see §4.5).

```
sercon serve --shutdown-timeout 5s app.ts
# app.ts calls server.http.listen({...}); -timeout defaults to 0 (disabled)
# under serve; --shutdown-timeout (default 30s) bounds the graceful drain.
```

4.6.7 Make a script directly executable

A plain shebang works for a script that takes no arguments (see §4.4):

```
#!/usr/bin/env sercon
runtime.log("hello from an executable script");
```

For a script that takes arguments, shebang into the **run** subcommand instead — **sercon run** hands every token after the script path straight to `runtime.argv[2:]`, with no `--` separator (a shebang line can't inject one):

```
#!/usr/bin/env -S sercon run
runtime.log("args:", JSON.stringify(runtime.argv.slice(2)));
```

```
chmod +x deploy.ts
./deploy.ts --dry-run alice          # runtime.argv.slice(2) → ["--dry-run", "alice"]
```

4.6.8 Emit tooling artifacts for your editor

```
sercon init                # writes ./sercon.d.ts + ./jsconfig.json (§4.3)
sercon init --force tools/  # same, into tools/, overwriting existing files
sercon -emit-dts sercon.d.ts # emit just the reserved-global .d.ts directly
```

5. Reserved globals (script surface)

sercon scripts get fifteen reserved top-level globals. Use them directly — there is no enclosing namespace. The full per-binding reference is the generated `examples/scripts/sercon.d.ts` this section is the at-a-glance prose.

Reserved names: `runtime`, `crypto`, `text`, `codec`, `fs`, `net`, `db`, `cloud`, `services`, `tui`, `image`, `web`, `audio`, `server`, `mcp`. User code can shadow these with a local `let / const / var` per normal JavaScript scoping — sercon does not intervene at runtime. `server` (added in v0.10.0) covers inbound listeners — see section 6 for the long-form treatment.

Deterministic key order. Objects returned from bindings enumerate their keys in a stable order (declaration order for fixed-shape results; source / column / insertion order for dynamic ones), so `JSON.stringify(result)` is byte-stable run-to-run — safe to hash for canonical serialization (payment signing, webhook signatures). The lone exception is `text.jq`, whose underlying engine discards key order internally.

5.1 console (browser/Node compatibility)

Beyond the fifteen reserved globals, sercon provides a `console` global so scripts pasted from a browser or Node run unchanged:

Method	Stream
<code>console.log</code> / <code>console.info</code> / <code>console.debug</code>	<code>stdout</code>
<code>console.warn</code> / <code>console.error</code>	<code>stderr</code>

console.table	stdout
---------------	--------

Each prints one space-joined line — clean and prefix-free (no timestamp), unlike the goja default console. Primitives print raw; objects and arrays render as JSON

(`console.log({a:1})` → `{ "a": 1 }`), with circular references falling back to `[object Object]` rather than throwing. The CLI disables the engine's built-in console so this stream-correct shim is the only `console` a script sees. `runtime.log` is the native equivalent of `console.log` (stdout) and formats the same way. Library embedders get the `goja_nodejs` console by default instead; toggle it with `Options.DisableConsole`.

`console.table(data, columns?)` renders tabular data as an aligned, bordered table (Node/Bun/Deno parity) — an array of objects, an array of primitives, or an object of objects/primitives. It prints a leading (index) column, one column per property (union of keys across rows, in first-seen order), and a `Values` column for primitive rows; the optional `columns` array restricts and orders the property columns. Cells are formatted like `console.log` (so strings are not quoted, unlike Node's `util.inspect`), and non-tabular input (a primitive) falls back to a `console.log`-style line rather than throwing.

```
console.table([
  { name: "web", status: "ok" },
  { name: "db", status: "down" },
]);
//
// | (index) | name | status |
// |-----|-----|-----|
// |    0    | web  | ok     |
// |    1    | db   | down   |
//
```

5.2 runtime

Script-host scaffolding. Members:

- `runtime.log(...args)` — print a space-joined line (primitives raw, objects/arrays as JSON).
- `runtime.assert.equal(actual, expected, msg?)` / `runtime.assert.ok(cond, msg?)` — throw on mismatch / falsy. `assert.equal` is **deep**: distinct objects/arrays with identical contents compare equal (structural, recursive — key order irrelevant).
- `runtime.time.nowMs()` — current wall-clock in milliseconds since the Unix epoch (host clock; never throws).
- `runtime.time.sleep(ms)` — Promise that resolves after `ms` on the event loop. It is **cancellable**: if the run hits its deadline or is cancelled before the delay elapses, the pending sleep rejects (the underlying context is cancelled) rather than holding the loop open.
- `runtime.time.format(unixMs, layout, tz?)` — render a Unix-ms timestamp through **strftime-style** tokens (not Go's reference layout). Supported tokens: `%Y %y %m %d %H %M %S %T %F %j %A %a %B %b %Z %z` and `%%` (a literal percent); unknown `%X` tokens pass through verbatim. `tz` is an optional IANA zone name (e.g. "Europe/Stockholm", "UTC"); it defaults to the host's local zone and **throws** if the name is

not loadable. For example `runtime.time.format(runtime.time.nowMs(), "%F %T", "UTC")` yields `"2026-06-24 13:45:09"`.

- `runtime.env.get(name)` — process environment variable; returns `undefined` when unset (never throws).
- `runtime.env.load(path, opts?)` — async; reads a `.env` file, applies its `KEY=VALUE` pairs to the process environment (same parser as the `--env-file` CLI flag: `#` comments, blank lines, optional `export`, surrounding quotes stripped, no shell expansion), and resolves to the parsed pairs. Already-set variables win unless `opts.override` is true. Rejects if the file is missing/unreadable or a line is malformed.
- **`runtime.setDeadline(ms)` / `runtime.clearDeadline()` / `runtime.getDeadline()`** — control the running script's own wall-clock kill deadline at runtime. This is the **same** deadline the `-timeout` flag sets, *not* the JS global `setTimeout` (which merely schedules a callback and does not move the run's kill timer).
 - `runtime.setDeadline(ms)` — move the kill deadline to **now + ms**, replacing any prior deadline. `ms <= 0` disables it (identical to `clearDeadline()`). Use it to extend a long-running task or to add a deadline to a run started with `-timeout 0`. Applies immediately to the in-flight run; never throws (a non-numeric argument coerces to `0`).
 - `runtime.clearDeadline()` — remove the deadline entirely (equivalent to `-timeout 0 / setDeadline(0)`); the script then runs without a timeout.
 - `runtime.getDeadline()` — milliseconds remaining until the kill deadline (`number`, `>= 0`), or `null` when no deadline is active (disabled, or started with `-timeout 0`).
- `runtime.argv` — `[programName, scriptPath, ...userArgs]`. User args come from the CLI args after a `--` separator. The layout mirrors Node/Bun: `argv[0]` is `"sercon"`, `argv[1]` is the path of the currently-executing script (so each script in a multi-arg invocation sees its own path), and `argv.slice(2)` is just your args. The property is always present; with no `--` it is just `[programName, scriptPath]`.
- **`runtime.secrets`** — read/write string credentials in the OS keystore (macOS Keychain, Linux Secret Service / libsecret, Windows Credential Manager). Pure-Go (no cgo) via `zalando/go-keyring`.
 - `runtime.secrets.available` — `boolean`, advisory: is a keystore backend reachable this run? Gate on it to self-skip on headless boxes.
 - `runtime.secrets.get(name, account)` — `Promise<string | null>` `null` when absent.
 - `runtime.secrets.set(name, account, secret)` — `Promise<void>`.
 - `runtime.secrets.delete(name, account)` — `Promise<boolean>` (`true` if removed).

All operations are confined to a **prefix namespace**: the keystore service is `PREFIX + name`, so a script can neither read nor clobber secrets outside it. `PREFIX` resolves from

`--secrets-prefix > SERCON_SECRETS_PREFIX > the default sercon/`. Store a secret once with your OS tools, e.g. macOS

`security add-generic-password -s 'sercon/devshop' -a tess -w`, Linux

`secret-tool store --label='sercon devshop' service 'sercon/devshop' account tess`.

Operations are async and bounded by a 10s timeout (a blocking macOS consent prompt rejects rather than hangs).

- **`runtime.clipboard`** — read/write the host OS system clipboard text. An external-CLI fallback (no pure-Go, no-cgo library covers every platform).

- `runtime.clipboard.available` — `boolean`, cheap advisory (no clipboard access): `true` when a backend is on `PATH`. Gate on it to self-skip on headless boxes.
- `runtime.clipboard.read()` — `Promise<string>` the clipboard text ("" when empty).
- `runtime.clipboard.write(text)` — `Promise<void>` replace the clipboard text.
- `runtime.clipboard.imageAvailable` — `boolean`, cheap advisory: `true` when a PNG image backend is usable on this host. Gated separately from `available` (text).
- `runtime.clipboard.readImage()` — `Promise<Uint8Array | null>` the clipboard image as PNG bytes, or `null` when no image is present.
- `runtime.clipboard.writeImage(png)` — `Promise<void>` set the clipboard image from PNG bytes (validated by signature; non-PNG rejects).

Backends: macOS `pbcopy` / `pbpaste`, Linux `wl-clipboard` (Wayland) or `xclip` / `xsel` (X11), Windows `clip` + PowerShell `Get-Clipboard`. When none is installed the calls throw a clean error and `available` is `false` the binary itself stays fully functional. Image is PNG-only and feature-detected separately (`imageAvailable`); macOS image read needs `pngpaste`, Linux needs `wl-clipboard` or `xclip` (not `xsel`). RTF/HTML/non-PNG are out of scope.

- `runtime.termSize()` — the controlling terminal’s size as { `columns`, `rows`, `tty` } (synchronous). When `stdout` is not a terminal (piped/redirected), `tty` is `false` and `columns` / `rows` fall back to `$COLUMNS` / `$LINES` then `80×24`, so scripts can format tables/progress bars without special-casing the non-TTY path. Pure-Go (`x/term`).
- `runtime.open(target)` — **async**; hand a URL or file path to the OS default handler (the user’s normal GUI browser for URLs). Fire-and-forget: resolves once the opener is spawned, not when the browser closes. The target is passed as a single argument (no shell, so special characters can’t inject). Throws if no opener is on `PATH`.
`runtime.openAvailable` is the cheap boolean advisory. Backends (feature-detected): macOS `open`, Linux `xdg-open` / `gnome-open`, Windows `rundll32` / `start`. This is the “open this in my browser” primitive — distinct from `net.http` / `web.load` (which *fetch*) and `services.agentBrowser` / `services.webdriver` (which drive an *automation* browser).

```
runtime.log("hello");
runtime.assert.equal(2 + 2, 4);
const args = runtime.argv.slice(2); // post-`--` user args
```

5.2.1 Script-controlled stdio: `runtime.stdout` / `runtime.stderr` / `runtime.stdin`

A script can redirect, silence, tee, or capture `sercon`’s own writers, and can swap what `runtime.stdin` reads from. Full member signatures are in the generated §17.11 reference (`runtime.stdout`, `runtime.stderr`, `runtime.stdin`); this section is the mental model.

The three handles. `runtime.stdout` and `runtime.stderr` are output handles with an identical member set — `to(target, opts?)`, `toFile(path, opts?)`, `silence()`, `reset()`, `target()`, `scoped(target, opts?, fn)`, `capture(fn)`. `runtime.stdin` is the input side — `read()`, `readBytes()`, `readLine()`, `lines()`, `from(source)`, `fromFile(path)`, `fromString(text)`, `reset()`, `source()`, `scoped(source, fn)`.

Redirects are a stack, not a single slot. Every setter that changes a destination or source returns an idempotent restore function; calling it pops *that* entry back off, wherever it ended up in the stack — so nested and out-of-order restores both behave, and a restore called twice is a no-op. `reset()` drops the whole stack at once (used by the CLI itself at the start of every Run — see below).

A stream-name target always folds onto the real process stream.

`runtime.stdout.to("stderr")` and `runtime.stderr.to("stdout")` both resolve to the actual `os.Stdout / os.Stderr`, never to whatever the other handle currently has pushed. That is deliberate: it makes a fold cycle (`stdout → stderr` while `stderr → stdout`) impossible by construction, at the cost that folding onto a stream bypasses whatever that stream's own redirect stack — a `capture()` buffer, a line callback, a file — currently holds.

tee writes to the destination *beneath* the one you're pushing, not unconditionally to the terminal. Teeing on top of an existing `silence()` still only reaches the new target and whatever was silenced beneath it — it does not resurrect the real terminal. `capture()` never tees; it is always exclusive. `toFile / to reject { tee: true }` combined with the `"null"` target (nothing to tee onto).

What this covers, and what it does not. Redirection reroutes sercon's own writers:

`console.*`, `runtime.log`, `text.str.printf`, the default-export JSON printed after a run, the `sercon run` subcommand's own `FAIL / verbose` output, the `--watch` banner, the TUI's non-TTY fallback renderer, and — notably — `sercon serve`'s `READY` line and its access log, so a served script can redirect its own supervisor's readiness line and request log.

A child process is a different story, and not a uniform one. `services.exec.shell / .run`, `services.git`, `services.gh`, and `services.typst.*` all **capture** the child's output into a buffer that they hand back to the script — the child never touches sercon's own process streams, so redirection is simply irrelevant to it either way; if the script then prints that captured string itself, *that* print follows the redirect like any other write. What genuinely bypasses redirection is a child that **inherits** file descriptor 1/2 directly:

`services.exec.interactive`, the “wire this subprocess to my real terminal” primitive (`ssh`, `docker exec -it`, interactive REPLs and pagers). The same boundary holds on the input side: the TUI's key-input reader and `services.exec.interactive` read file descriptor 0 directly, so a `runtime.stdin` source swap does not reach them either. The rule to remember: redirection covers what **sercon itself** reads/writes, not a separate process holding its own copy of the fd.

A function target's delivery has its own caveats. Lines reach the handler on a later tick, in write order, never synchronously with the write that produced them. The pending-line queue is bounded (1024 lines); on overflow, or on a write that re-enters the handler (the handler's own `console.log` while it is running), the write falls through to the destination *beneath* the callback instead of blocking or being lost twice. Any line still queued when the redirect is popped or the run ends is written to that same destination beneath rather than delivered late. A handler that throws has that one line reported once on the real `stderr` — not retried, not recovered — without aborting the rest of the run.

FAIL / PASS stay on stdout; --verbose's duration line is on stderr — and both follow whatever the script left `runtime.stdout / runtime.stderr` pointed at, because the CLI's own post-run reporting writes through the same registry the script does.

Redirects reset at the start of every Run, not the end. `sercon a.ts b.ts` gives `b.ts` clean streams regardless of what `a.ts` did, and each `--watch` re-run starts clean too. Resetting at the *start* (rather than after) is what lets the CLI's own `FAIL / PASS / --verbose` reporting still land wherever the script itself left the stream — the reset only clears the slate for the *next* run. There is one further reset on the way out, *after* that final reporting write: it exists so a function target left pushed by the last (or only) run cannot swallow the result JSON and

the `PASS / FAIL` line — delivery needs a live event loop, and by then there isn't one, so popping the entry flushes whatever it still holds to the destination beneath it.

`runtime.stdin` **reads block until EOF**, off the event loop, so `await` on them never stalls the loop; concurrent read calls serialise against each other so two callers can never split a line or a chunk. When the script itself was read from stdin (`sercon -`), the real stdin is already drained by the time the script runs, so every read resolves immediately (empty string / zero bytes / `null`) unless the script has swapped in a file or string source first.

5.2.2 Recipes

5.2.2.1 Silence a noisy dependency for one call

Some library call insists on printing to stdout/stderr and there is no quiet flag. Wrap just that call.

```
const restore = runtime.stderr.silence();
try {
  await noisyThirdPartyThing();
} finally {
  restore();
}
```

Notes

- `scoped()` (below) does the try/finally for you when the noisy call is a single function.

5.2.2.2 Log to a file while still watching the terminal

Append a run's output to a log file without losing the live terminal view — `tee` writes to both the new target and whatever was there before.

```
const stopLogging = runtime.stdout.toFile("/tmp/my-script.log", {
  append: true,
  tee: true,
});
console.log("this line lands in the file AND on screen");
stopLogging();
```

Notes

- Drop `append` to truncate the file on each run instead.
- A later write failure (disk full, file removed) does not throw — the destination is marked dead and every subsequent write falls through to the destination *beneath* it in the stack (here, with one redirect pushed, that is the process stream; deeper in a stack it is whatever sits below, not necessarily the terminal). So a destination that goes bad costs at most the one write that was in flight when it failed; nothing after it is lost. The failure is reported once on the real stderr, not once per line.
- The process stream itself is never taken out of service that way: a failed write straight to stdout/stderr is reported once and later writes are still attempted, so a single transient error can't silence the rest of the session.

5.2.2.3 Assert on what a function printed

Testing a function that logs, without touching its implementation.

```
const output = await runtime.stdout.capture(() => {
  reportStatus({ ok: true });
});
runtime.assert.ok(output.includes("ok"), "reportStatus should mention ok");
```

Notes

- `capture()` is always exclusive (never tees) and restores stdout — even if the callback throws — before its promise settles.
- Use `runtime.stderr.capture(fn)` for a function that logs warnings/errors instead.

5.2.2.4 Read piped input line by line

Consume `sercon script.ts < data.txt` (or a real interactive terminal) a line at a time, and make the same script testable without a real pipe.

```
for await (const line of runtime.stdin.lines()) {
  if (line === "") break;
  runtime.log("got:", line);
}
```

Notes

- To unit-test the same code path, swap the source first:
`runtime.stdin.fromString("a\nb\n\n");` then run the loop above — no pipe needed.
- If the *script itself* was read from stdin (`sercon -`), the real stdin is already drained before the script starts running, so `runtime.stdin.lines()` ends immediately unless a source has been swapped in with `from / fromFile / fromString`.

5.2.2.5 Route each line to your own handler

Send every line straight to your own logger instead of a file, so a script's output feeds a webhook, a structured logger, or a test spy.

```
const seen: string[] = [];
const stop = runtime.stdout.to((line) => {
  seen.push(line);
});
console.log("first");
console.log("second");
await runtime.time.sleep(0); // delivery is queued, not synchronous
stop();
runtime.assert.equal(seen.join(","), "first,second");
```

Notes

- Delivery is queued and asynchronous: a line lands on a later tick, never in the same call as the `console.log` that produced it. Any awaited microtask/macrotask (even `time.sleep(0)`) is enough to let it drain before inspecting the result.
- The pending-line queue is bounded (1024 lines). On overflow — or on a write that re-enters the handler, e.g. the handler itself calling `console.log` — the write falls through to whatever destination is beneath the callback instead of blocking or being silently dropped.
- Any line still queued when the redirect is popped or the run ends is written to the destination beneath rather than delivered to the handler.

- A handler that throws has that one line reported once on the real stderr (not retried); the rest of the run continues.

5.3 crypto

Hashing, JWT, and age/PGP encryption. Three sub-namespaces:

5.3.1 crypto.hash.*

`md5`, `sha1`, `sha256`, `sha384`, `sha512`, `sha3_256`, `sha3_512`, `blake3`, `crc32`. Each takes a single `string` (interpreted as a UTF-8 byte sequence) and returns a lowercase hex digest. Digest widths: MD5 32, SHA-1 40, SHA-256/SHA3-256/BLAKE3 64, SHA-384 96, SHA-512/SHA3-512 128. `crc32` is the IEEE polynomial, zero-padded to 8 hex chars. All are pure and deterministic. A missing/ `null` / `undefined` argument throws a `TypeError`. MD5 and SHA-1 are exposed for legacy fingerprinting only — do not use them for security. BLAKE3 is lukechampion.com/blake3 (256-bit output); the `sha3_*` underscore naming matches `recon`'s binding.

5.3.2 crypto.jwt.*

`sign(claims, secret, opts?)`, `view(token)`, `validate(token, secret, opts?)`.

The `secret` argument is polymorphic and the same across `sign/validate`: raw bytes for HMAC algorithms (`HS256` / `HS384` / `HS512`); a PEM-encoded key (`-----BEGIN ...`) for the asymmetric families (`RS*`, `PS*`, `ES*`, `EdDSA`); or a JWK JSON object (whose `key` selects the key type). `sign` uses the private key, `validate` the public key (a certificate is accepted). `opts.algorithm` is case-insensitive and defaults to `HS256`.

`sign` does **not** synthesise reserved claims — set `iat` / `exp` / `nbf` yourself if you want them; everything in `claims` is passed straight through as `MapClaims`. `view` decodes the header and payload **without verifying** the signature (handy for debugging an auth flow), preserving object key order and returning the raw signature segment alongside. `validate` verifies the signature plus the standard time claims (`exp` / `nbf` — `iat` is not checked, per RFC 7519 §4.1.6) and, when `opts.audience` / `opts.issuer` are set, those claims too; pinning `opts.algorithm` activates the algorithm-confusion guard. It resolves `{ valid: true, claims }` or `{ valid: false, reason }` — cryptographic and claim failures do *not* throw; only structural/wiring errors (empty token/secret, malformed token, key shape mismatching the token's algorithm) throw.

5.3.3 crypto.encrypt.*

`keygen()`, `keygenPgp(opts?)`, `encrypt(data, recipients, opts?)`,
`decrypt(ciphertext, identities)`,
`rekey(ciphertext, oldIdentities, newRecipients, opts?)`, `detectBackend(input)`.

Two interoperable backends, auto-dispatched on key format — you never name the backend explicitly. `age (filippo.io/age)` keys are bech32: `age1...` recipients (public) and `AGE-SECRET-KEY-1...` identities (private); `age` also accepts SSH public keys as recipients. PGP keys are ASCII-armored `-----BEGIN PGP PUBLIC/PRIVATE KEY BLOCK-----` blocks. `keygen` mints a fresh `age X25519` keypair; `keygenPgp` mints an `RSA-2048` PGP keypair (`opts.name` / `opts.email` populate the user ID). `encrypt` / `decrypt` route to `age` or PGP by inspecting the supplied keys (and, for `decrypt`, the ciphertext armor) — the two backends cannot be mixed in a single call.

`encrypt` accepts `string` / `Uint8Array` / `ArrayBuffer` data and one or many recipients; age output is binary unless `opts.armor` requests ASCII armor, PGP output is always armored. Multi-recipient ciphertext can be opened by any listed identity. `decrypt` returns the plaintext as a `Uint8Array` (decode with `new TextDecoder().decode(...)`). `rekey` is an age-only decrypt+re-encrypt that rotates a ciphertext to a new recipient set without exposing plaintext to JS; output armor defaults to the input's and `opts.armor` overrides. `detectBackend` is pure prefix matching (no parsing, no I/O) and never throws — it returns `{ backend: "age" | "pgp" | "unknown", kind?: "public" | "private" }`. The encrypt/decrypt/rekey calls throw on type/shape errors: wrong key kind (public where private is expected or vice versa), empty key lists, an unsupported data type, or no identity matching the ciphertext.

5.3.4 Concepts

Three independent families, none of which share state or keys:

- **`crypto.hash.*`** — one-shot digests (`md5` ... `blake3` , `crc32`). Each takes a single `string` (interpreted as UTF-8 bytes) and returns a lowercase hex digest; there's no separate “hash a file” entry point — read the file's text first (`fs.readText`), then hash the string.
- **`crypto.jwt.*`** — `sign` / `view` / `validate` . This is also `sercon`'s only exposed HMAC surface: there is no standalone `crypto.hmac.*` function, so “HMAC-sign a message” in `sercon` means `crypto.jwt.sign` with an `HS256` / `HS384` / `HS512` algorithm (raw secret bytes) rather than a bare MAC call. The same `sign` / `validate` pair also does asymmetric signing (`RS*` / `PS*` / `ES*` / `EdDSA`) when `secret` is a PEM key or JWK.
- **`crypto.encrypt.*`** — `keygen` / `encrypt` / `decrypt` / `rekey` over two auto-dispatched backends, age and PGP (armor auto-detected on decrypt).

All three families follow the same bytes convention used across `sercon`: inputs accept `string` | `Uint8Array` | `ArrayBuffer` (or, for `hash` , `string` only), and anything that returns raw bytes returns a `Uint8Array` — decode it with `new TextDecoder().decode(...)` (sync) or `await text.charset.decode(bytes, "utf-8")` (the codebase's async charset path, used throughout the example scripts) to get a string back.

5.3.5 Recipes

5.3.5.1 Hash a string or a file's contents

```
const digest = crypto.hash.sha256("abc");
runtime.log(digest); // 64-char lowercase hex

// "hash a file": crypto.hash.* only accepts a string, so read it as text
// first — this hashes the file's UTF-8 text, not arbitrary binary bytes.
const contents = await fs.readText("./notes.txt");
const fileDigest = crypto.hash.sha256(contents);
```

Signatures: §17.5.2.5 (`crypto.hash.sha256`); the same shape covers `md5` / `sha1` / `sha384` / `sha512` / `sha3_256` / `sha3_512` / `blake3` / `crc32` (§17.5.2.1–§17.5.2.9). runnable: `examples/scripts/hash.ts`

5.3.5.2 Verify a payload's integrity with a hash claim

A JWT can carry more than its own claims — embed a hash of some other payload as a custom claim, and the token becomes a tamper-evident anchor for data that travels

separately (e.g. a JSON body alongside an `Authorization` header, or a message queued for later processing). This is a different job from §5.3.5.4's "sign a claims object": there the JWT *is* the message; here the JWT *attests to* a message it doesn't carry.

```
// Sender: hash the payload, embed the digest as a claim, sign the token.
const payload = JSON.stringify({ userId: "u-42", action: "transfer", amount:
1234.56 });
const payloadHash = crypto.hash.sha256(payload);

const secret = "shared-with-the-verifier-only";
const now = Math.floor(Date.now() / 1000);
const token = crypto.jwt.sign(
  { sub: "u-42", iat: now, exp: now + 3600, phash: payloadHash },
  secret,
);

// ... payload and token travel separately; time passes ...

// Receiver: validate the token, then re-hash the payload and compare.
const verdict = crypto.jwt.validate(token, secret);
if (!verdict.valid) {
  throw new Error(`token rejected: ${verdict.reason}`);
}
const rehash = crypto.hash.sha256(payload);
if (rehash !== verdict.claims.phash) {
  throw new Error("payload does not match the hash claim — tampered in transit");
}
runtime.log("payload integrity verified");
```

`validate` resolves `{ valid: false, reason }` on a bad signature or expired token rather than throwing — check `.valid` before trusting `claims`. The hash comparison is a second, independent check: it catches a payload that was swapped out *without* touching the token (the signature alone can't detect that, since the token never contained the payload itself). Signatures: §17.5.2.5 (`crypto.hash.sha256`), §17.5.3.1 (`crypto.jwt.sign`), §17.5.3.2 (`crypto.jwt.validate`). runnable: `examples/scripts/advanced/crypto-pipeline.ts`

5.3.5.3 Encrypt and decrypt with age

```
const alice = crypto.encrypt.keygen(); // { publicKey: "age1...", privateKey: "AGE-
SECRET-KEY-1..." }

const ct = crypto.encrypt.encrypt("hello, alice", alice.publicKey);
const plain = crypto.encrypt.decrypt(ct, alice.privateKey);
runtime.log(await text.charset.decode(plain, "utf-8")); // "hello, alice"

// Multi-recipient: any listed identity can open it.
const bob = crypto.encrypt.keygen();
const shared = crypto.encrypt.encrypt("shared message", [alice.publicKey,
bob.publicKey]);

// ASCII armor for embedding in JSON / YAML / email; decrypt reads either form.
const armored = crypto.encrypt.encrypt("embed me", alice.publicKey, { armored:
true });
```

The wrong private key — or passing a public key where an identity is expected — throws a clean error rather than silently failing. Signatures: §17.5.1.4 (`keygen`), §17.5.1.3 (`encrypt`), §17.5.1.1 (`decrypt`), §17.5.1.6 (`rekey`), §17.5.1.2 (`detectBackend`). runnable: `examples/scripts/encrypt.ts`

5.3.5.4 Sign and verify a JWT

```
const secret = "topsecret-only-shared-with-the-verifier";
const now = Math.floor(Date.now() / 1000);

const tok = crypto.jwt.sign(
  { sub: "alice", iat: now, exp: now + 3600, aud: "sercon-demo" },
  secret,
);

const view = crypto.jwt.view(tok); // decodes without verifying – handy for
debugging
runtime.log(view.header.alg, view.payload.sub);

const verdict = crypto.jwt.validate(tok, secret, { audience: "sercon-demo" });
runtime.log(verdict.valid, verdict.claims?.sub);
```

`sign` does not synthesise `iat` / `exp` — set them yourself if you want them enforced. Asymmetric algorithms (`EdDSA` , `RS*` , `PS*` , `ES*`) use the same three calls with a PEM or JWK `secret` instead of raw bytes; a PEM key with an `HS*` algorithm is a cross-check error and throws at `sign` / `validate` rather than silently misbehaving. Signatures: §17.5.3.1 (`crypto.jwt.sign`), §17.5.3.3 (`crypto.jwt.view`), §17.5.3.2 (`crypto.jwt.validate`). runnable: `examples/scripts/jwt.ts`

5.4 text

String, regex, charset, and data manipulation. The `str.*` and `preg.*` families are **synchronous**; `charset.*` , `jq.*` , and `diff.*` return **Promises** (`await` them). Members:

- `text.str.*` — synchronous string helpers: `trim` , `ltrim` , `rtrim` , `reverse` , `stripHtml` , `nl2br` , `br2nl` , `base64Encode` , `base64Decode` , `base64UrlEncode` , `base64UrlDecode` , `urlEncode` , `urlDecode` , `htmlEntityDecode` , `pad` / `lpad` / `rpadd` , `sprintf` , `printf` , `normalizeNewlines` .
 - `trim` / `ltrim` / `rtrim` take a PHP-style **cutset** mask, not a prefix: every character in the mask is stripped (default mask is the whitespace set `" \t\n\r\v\f"`). `reverse` and the `pad` family count **runes**, not bytes, so multi-byte text stays intact (`reverse("café")` → `"éfac"`). `pad` 's side is `"right"` (default), `"left"` , or `"both"` . `"both"` splits the deficit with the extra rune going right.
 - `base64Encode` / `base64Decode` are the **standard** RFC 4648 alphabet with padding — URL-safe input (`-` / `_`) is *not* auto-detected and throws. Use `base64UrlEncode` / `base64UrlDecode` for the URL-safe alphabet (RFC 4648 §5, `-` / `_`): encode emits **no** padding (safe in URLs, filenames, and JWT segments), and decode accepts both padded and unpadded input.
 - `urlEncode` / `urlDecode` use `application/x-www-form-urlencoded` rules (space ↔ `+`); for path segments use `goja` 's built-in `encodeURIComponent` . `sprintf` / `printf` use **Go's** `fmt` verbs (`%s` , `%d` , `%.2f` , `%v` , `%q` , ...), not PHP's; `printf` writes straight to `stdout` and

returns nothing. `normalizeNewlines` rewrites any mix of CRLF/CR/LF to `"lf"` (default), `"crlf"`, or `"cr"`.

- `text.preg.*` and `text.preg2.*` — two distinct regex engines sharing a PHP-style `/pattern/flags` syntax (forward-slash delimiter only) and a `{ match, groups, index }` result shape. `preg` runs Go's **RE2**: linear-time, no catastrophic backtracking, but **no** lookahead or backreferences; flags `i / m / s` only (`u / U / x` are rejected). `preg2` runs the .NET-flavoured `regexp2` engine: lookahead, lookbehind, backreferences, and the extra `x` flag — at the cost of a backtracking engine, so guard untrusted input with a timeout. `match` returns the first hit or `null`; `matchAll` returns an array; `replace` substitutes every match. Replacement templates use the engine's `$1 / ${1}` group syntax — PHP's `\1` backref form is *not* translated.
- `text.charset.detect(data)` / `decode(data, charset)` / `encode(text, charset)` — **async** character-set detection and conversion over a `string`, `Uint8Array`, or `ArrayBuffer`. `detect` (saintfish/chardet) resolves `{ charset, confidence, language?, candidates }` where `confidence` is a 0–100 score and `candidates` ranks every guess. `decode` / `encode` map between a named WHATWG/HTML5 encoding (UTF-8, ISO-8859-1, Windows-1252, Shift_JIS, GBK, ...) and UTF-8; `encode` has **no lossy fallback** — a character with no representation in the target encoding rejects rather than being silently dropped.
- `text.jq.query(json, expr)` / `queryAll(json, expr)` — **async** jq filtering (itchyny/gojq) against any JSON-like JS value. `query` resolves the **first** value the filter emits (or `null` when it emits nothing, e.g. an optional path `.a.b?` that misses); `queryAll` drains the whole stream into an array.
- `text.diff.compare(a, b, opts?)` — **async** unified diff between two text or byte inputs. `opts.context` (default 3) sets the surrounding lines; `opts.fromFile` / `opts.toFile` (default `"a"` / `"b"`) label the headers. Resolves `{ identical, binary, added, removed, diff, format: "unified" }` inputs with a NUL byte in their first 8 KB are reported as `binary: true` with an empty `diff`.
- `text.stego.embed(cover, payload, opts?)` / `text.stego.extract(stegoText, opts?)` — synchronous zero-width character steganography. `embed` appends the payload as an invisible run of U+200B (zero-width space, bit 0) and U+200C (zero-width non-joiner, bit 1) at the end of `cover` the returned string is visually identical to the cover text. `extract` scans the string for those carrier runes and reconstructs the payload. Both accept `{ password }` for optional **AES-256-GCM** encryption (same key derivation as `image.stego`). `payload` may be a `string` (returned as a string on extract) or a `Uint8Array` (returned as `Uint8Array`). There is no separate capacity call — zero-width characters cost only 8 runes per payload byte, so practical limits are high.
- `text.markdown.toHtml(md, opts?)` — render a Markdown string to an HTML string (CommonMark, backed by the pure-Go **goldmark**). GFM extensions (tables, strikethrough, task lists, autolinks) are on by default; `opts.gfm: false` disables them, and `opts.hardBreaks: true` renders a single source newline as `<br>` instead of a space. Raw HTML in the source is escaped (goldmark's safe default). String in, string out — for Markdown → **PDF**, see `recon --md-to-pdf` (used by `make manual`).

5.4.1 Concepts

sercon is UTF-8 internally and at every script boundary — strings coming back from `fs`, `net`, and `text.str.*` are already UTF-8. `text.charset.*` is the escape hatch for bytes that *aren't*: a legacy log file, a mainframe export, or an HTTP body served as `Content-Type: text/html; charset=Shift_JIS`.

- `text.str.*` — synchronous string helpers. These operate purely on JS strings (already UTF-8) and never consult an external encoding table; `reverse` and the `pad` family count **runes**, not bytes, so multi-byte text survives round-trips intact.
- `text.charset.*` — **async** (Promise-returning) conversion between a named WHATWG/HTML5 encoding (ISO-8859-1, Windows-1252, Shift_JIS, GBK, ...) and UTF-8, plus best-guess `detect`. Reach for this whenever bytes didn't originate as UTF-8 — decoding a legacy file, an HTTP body with a non-UTF-8 `charset=` header, or data round-tripped through a system that only speaks Latin-1. `encode` has no lossy fallback: a character with no representation in the target charset rejects rather than being silently dropped.
- `text.case.*` — synchronous identifier case conversion. There is no “convert from X to Y” call: every converter runs the same auto-detecting tokenizer first (splitting on separators, lower→upper transitions, and acronym→word boundaries — `HTTPServer` → `[http, server]`), then re-emits the words in the target case. So `text.case.snake(x)` accepts *any* input convention — camelCase, kebab-case, spaced, dotted, path-like, or acronym-laden — with no “source case” to name. Sixteen named converters share this model:

Case	Example (myVarName)
camel	myVarName
pascal	MyVarName
snake	my_var_name
screamingSnake	MY_VAR_NAME
ada	My_Var_Name
camelSnake	my_Var_Name
kebab	my-var-name
train	My-Var-Name
screamingKebab	MY-VAR-NAME
flat	myvarname
upperFlat	MYVARNAME
dot	my.var.name
path	my/var/name
title	My Var Name
sentence	My var name
capital	My Var Name (synonym of title)

`flat` and `upperFlat` are **lossy** outputs — once the separators and case boundaries are gone (`myvarname`), the words can't be reliably re-split, so treat them as a one-way rendering, not a round-trippable case. Three aliases cover common names for the same converters: `header` → `train`, `cobol` → `screamingKebab`, `slug` → `kebab` (this `slug` is kebab-casing only — it does **not** transliterate or strip diacritics/punctuation; a real slugifier

is out of scope). Beyond the 16 named converters, `text.case.split(s)` exposes the tokenizer directly (returns the lowercased word array), `text.case.detect(s)` guesses which case an already-formatted string is in, `text.case.convert(s, name)` dispatches to a case chosen at runtime by name/alias, and `text.case.names()` lists the 16 canonical names.

5.4.2 Recipes

5.4.2.1 Decode non-UTF-8 bytes to a string

```
// Round-trip through a legacy charset: encode a UTF-8 string to
// Windows-1252 bytes, then decode those bytes back to UTF-8.
const bytes = await text.charset.encode("café crème – 1985", "Windows-1252");
const decoded = await text.charset.decode(bytes, "Windows-1252");
runtime.log(decoded); // "café crème – 1985"

// Unknown provenance? detect() ranks candidates by confidence (0-100).
const guess = await text.charset.detect(bytes);
runtime.log(guess.charset, guess.confidence);
```

`charset` names are WHATWG/HTML5 encoding names or aliases (UTF-8, ISO-8859-1, Windows-1252, Shift_JIS, GBK, ...); an unknown name or bytes invalid for that encoding rejects. Signatures: §17.14.2.3 (`encode`), §17.14.2.1 (`decode`), §17.14.2.2 (`detect`).
runnable: `examples/scripts/charset.ts`

5.4.2.2 Use the string utilities

```
text.str.trim(" hi "); // "hi"
text.str.ltrim("xxhi", "x"); // "hi"
text.str.rtrim("hixx", "x"); // "hi"

text.str.pad("7", 3, "0", "left"); // "007"
text.str.lpad("7", 3, "0"); // "007" – shortcut for pad(side: "left")
text.str.rpad("7", 3, "."); // "7.."

text.str.reverse("café"); // "éfac" – rune-aware, not byte-aware
text.str.stripHtml("<b>hi</b>"); // "hi"
```

`trim` / `ltrim` / `rtrim` take a PHP-style cutset **mask**, not a prefix — every character in the mask is stripped (default mask is the whitespace set `" \t\n\r\v\f"`). `reverse` and the `pad` family count runes, so multi-byte text stays intact. Signatures: §17.14.9.18 (`text.str.trim`), §17.14.9.8 (`text.str.ltrim`), §17.14.9.15 (`text.str.rtrim`), §17.14.9.11 (`text.str.pad`), §17.14.9.7 (`text.str.lpad`), §17.14.9.14 (`text.str.rpad`), §17.14.9.13 (`text.str.reverse`), §17.14.9.17 (`text.str.stripHtml`). runnable: `examples/scripts/strings.ts`

5.4.2.3 Base64 encode/decode

```
const enc = text.str.base64Encode("hi");
const dec = text.str.base64Decode(enc);
runtime.log(enc, dec); // "aGk=" "hi"

// URL-safe alphabet (no padding on encode) for URLs, filenames, JWT segments.
const urlSafe = text.str.base64UrlEncode("a?b");
```

```
const back = text.str.base64UrlDecode(urlSafe);
runtime.log(urlSafe, back); // "YT9i" "a?b"
```

`base64Encode` / `base64Decode` are the **standard** RFC 4648 alphabet with padding — URL-safe input (`-` / `_`) is *not* auto-detected and throws. `base64UrlEncode` / `base64UrlDecode` use the URL-safe alphabet (RFC 4648 §5): encode emits no padding, and decode accepts both padded and unpadded input. Signatures: §17.14.9.2 (`text.str.base64Encode`), §17.14.9.1 (`text.str.base64Decode`), §17.14.9.4 (`text.str.base64UrlEncode`), §17.14.9.3 (`text.str.base64UrlDecode`). runnable: `examples/scripts/strings.ts`

5.4.2.4 Convert an identifier between conventions

```
// Every converter auto-detects the input's boundaries — mixed input works fine.
text.case.snake("myVarName"); // "my_var_name"
text.case.kebab("HTTPServerConfig"); // "http-server-config"
text.case.pascal("user_id"); // "UserId"
text.case.camel("my-var--name"); // "myVarName"
```

Each named converter runs the same tokenizer (split on separators, lower→upper transitions, and acronym→word boundaries) then re-emits the words in its target case — there is no “from” case to specify. Signatures: §17.14.1.19 (`text.case.snake`), §17.14.1.11 (`text.case.kebab`), §17.14.1.13 (`text.case.pascal`), §17.14.1.2 (`text.case.camel`). runnable: none

5.4.2.5 Convert to a case chosen at runtime

```
const target = "screamingSnake"; // e.g. read from config or a CLI flag
runtime.log(text.case.names()); // ["camel","pascal","snake", ...] — valid names

const out = text.case.convert("parseHTTP2Response", target);
runtime.log(out); // "PARSE_HTTP2_RESPONSE"

// Aliases (header/cobol/slug) are accepted too:
text.case.convert("Content-Type", "header"); // "Content-Type"
```

`convert` accepts any canonical name from `names()` plus the aliases (`header` , `cobol` , `slug`) and throws — listing the valid names in the message — on anything else. Signatures: §17.14.1.6 (`text.case.convert`), §17.14.1.12 (`text.case.names`). runnable: none

5.4.2.6 Tokenize with split / guess with detect

```
text.case.split("getHTTPCode"); // ["get", "http", "code"]
text.case.split("__foo--bar//"); // ["foo", "bar"] — separator runs collapse

text.case.detect("my_var_name"); // "snake"
text.case.detect("MyVarName"); // "pascal"
text.case.detect("myvarname"); // "camel" — a single flat word matches many converters
text.case.detect("my Mixed_Up"); // "unknown" — no converter reproduces it exactly
```

`split` is the primitive every named converter builds on: it always returns lowercased words. `detect` is a heuristic — it returns the first case (in `names()` order) whose converter

round-trips the input exactly, or "unknown" when none does; a single lowercase word matches several converters and resolves to "camel" since it comes first in that order. Signatures: §17.14.1.20 (`text.case.split`), §17.14.1.7 (`text.case.detect`). runnable: none

5.5 codec

Binary-format codecs (was `format` in v0.8.0). Members:

- `codec.compression.{algorithms, compress, decompress}` — gzip, deflate, zlib, bzip2, zstd, brotli, lz4, xz, snappy.
- `codec.barcode.{formats, decodableFormats, encode, decode}` — QR, DataMatrix, Aztec, PDF417, Code128, Code39, Codabar, EAN-13/8, UPC-A, ITF. `encode` returns PNG bytes; `decode` reads image bytes.
- `codec.checkdigit.{algorithms, validate, compute, inspect}` — Luhn, ISBN-10/13, EAN-13/8, UPC-A.
- `codec.php.{serialize, unserialize, varExport, parseVarExport, varDump, parseVarDump}` — read and write PHP data dumps.
- `codec.perl.{dumper, parseDumper}` — read and write Perl `Data::Dumper` output.
- `codec.xml.encode(value, opts?)` / `codec.xml.decode(xml)` — `value` ↔ XML. Attributes are @-prefixed keys, text is `#text`, other keys are child elements, and an array value becomes repeated sibling elements; a text-only element collapses to a bare string and an empty/self-closing element to `null`. The value must be a single-key object naming the root (or pass `opts.rootName`). `opts.indent` pretty-prints and `opts.declaration` prepends the XML declaration. Object key order and namespace prefixes are preserved. Decoded values are **strings** (XML is untyped — a number round-trips as its string form); surrounding whitespace is trimmed and mixed text/element ordering is not preserved. Cycles, mismatched tags, multiple roots, and malformed XML throw.
- `codec.sheet.read(src, opts?)` / `codec.sheet.write(model, opts?)` / `codec.sheet.formats()` — tabular data (CSV / TSV / XLSX / ODS) to/from a workbook model; plus read-only legacy formats (XLS / SYLK / DIF) for extraction and convert-up workflows. XLSX and ODS cells are typed (numbers → `number`, booleans → `boolean`, empty → `null`); ODS dates come back as ISO-8601 strings; CSV/TSV cells are always `string`. XLS cells come back as formatted strings (extraction-grade). See §5.5.7 below.
- `codec.doc.read(src, opts?)` / `codec.doc.write(model, opts?)` / `codec.doc.formats()` — document text extraction and authoring. Five formats are supported: PDF and DOC (Word 97–2003) are **read-only** (writing throws); DOCX, RTF, and ODT are **read+write**. `read` returns `{ format, text, paragraphs }`: `format` is the detected or forced format, `text` is the full plain text, and `paragraphs` is the block breakdown (native for DOCX/ODT/RTF; best-effort for PDF/DOC). Format is auto-detected from file magic bytes (`%PDF`, `{\rtf`, `OLE2 \xD0\xCF`, or ZIP content inspection for DOCX/ODT) and falls back to extension; pass `opts.format` to force it. `write` accepts a `{ paragraphs }` array, a `{ text }` blob, or a bare string (split on newlines); without `opts.dest` the encoded bytes are returned, with `dest` they are written there. Extraction is **extraction-grade** (best-effort plain text — not a full-fidelity round-trip): styles, tables, images, and embedded objects are not preserved. Use the “convert-up” pattern — read from any source format, write to DOCX — for format migration. `codec.doc.formats()` returns the read/write capability matrix at runtime. Pure-Go (no cgo, no external tools).

- `codec.dotenv.parse(text)` / `codec.dotenv.stringify(obj)` — pure dotenv codec (same `--env-file` semantics: `#` comments, blank lines, optional `export`, surrounding quotes stripped, no shell expansion). `stringify` sorts keys and double-quotes values when needed so `parse(stringify(obj))` deep-equals `obj` — the round-trip guarantee. Neither function touches the process environment; use `runtime.env.load` to apply a file. Pure-Go.

5.5.1 `codec.compression` — `compress` / `decompress`

Nine algorithms, exposed by name (case-insensitive). `codec.compression.algos()` returns them in registration order: `gzip`, `deflate`, `zlib`, `bzip2`, `zstd`, `brotli`, `lz4`, `xz`, `snappy`.

- `codec.compression.compress(algo, data): Promise<Uint8Array>` — both `compress` and `decompress` are **async** (they run off the loop via `PromisifyAsync`). `data` is a string (taken as its UTF-8 bytes), a `Uint8Array`, or an `ArrayBuffer`.
- `codec.compression.decompress(algo, data, opts?): Promise<Uint8Array>` — the inverse; pass the **same** algorithm name you compressed with. The frame format is algorithm-specific and not self-describing, so there is no auto-detection. `opts.maxBytes` caps the decompressed output size in bytes (default 512 MB; a non-positive value falls back to the default) — a guard against a decompression bomb, a small crafted input that inflates to gigabytes in memory. Exceeding it throws.

Notes on the implementations: `bzip2` is compressed via `dsnet/compress` (Go's standard `compress/bzip2` is decompression-only); `snappy` uses its one-shot block form (no streaming frame header), while `lz4` uses the LZ4 frame format (`pierrec/lz4` writer/reader); `gzip` / `zlib` / `deflate` are `stdlib`. Outputs surface to JS as `Uint8Array` decode bytes back to text with `new TextDecoder().decode(bytes)`. Round-trips are byte-faithful for the same algorithm; cross-algorithm or truncated input throws.

5.5.2 `codec.barcode` — `encode` / `decode`

- `codec.barcode.encode(format, data, opts?): Promise<Uint8Array>` — **async**; renders `data` into PNG bytes. Encodable formats (`codec.barcode.formats()`): `qr`, `datamatrix`, `aztec`, `pdf417`, `code128`, `code39`, `codabar`, `ean13`, `ean8`, `upca`.
- `codec.barcode.decode(data, format?): Promise<{ format, text }>` — **async**; reads PNG / JPEG / WebP image bytes via `gozxing`. Decodable formats (`codec.barcode.decodableFormats()`): the encodable set minus `pdf417` (`gozxing` has no PDF417 reader) plus `code93`, `upce`, and `itf`. With no `format` hint every reader is tried in priority order and the first hit wins; a hint runs only that reader. `decode` shares the same hard 64-megapixel decode guard as `image.decode` (§5.11) — an image whose declared width x height exceeds it throws before the pixel buffer is allocated; there is no override.

Dimensions and the quiet zone. `opts.width` / `opts.height` default to 256×256 for the 2D codes (`qr`, `datamatrix`, `aztec`) and 400×120 for the 1D symbologies. `opts.quietZone` pads a white margin around the rendered bars: `true` uses 10 % of the width (minimum 10 px), a number uses that many pixels per side, and `false` / `0` / absent adds none. **EAN/UPC barcodes need a quiet zone to be decodable** — encode them with `quietZone: true` (or supply margin in the source image) before round-tripping through `decode`. Invalid payloads for a symbology (e.g. wrong EAN digit count) throw at encode time.

5.5.3 `codec.checkdigit` — validate / compute / inspect

Six algorithms (`codec.checkdigit.algos()`): `luhn`, `isbn10`, `isbn13`, `ean13`, `ean8`, `upca`. `isbn13` is an alias for `ean13` (identical check-digit math). Algorithm names are case-insensitive and trimmed.

- `codec.checkdigit.validate(algo, input): boolean` — does `input` (the full number *including* its trailing check digit) pass? Returns `false` — never throws — for the wrong length, non-digit characters, or an unknown algorithm.
- `codec.checkdigit.compute(algo, partial): string` — the single missing check digit for `partial` (the number *without* the check digit). Returns `'0' – '9'`, or `'X'` for ISBN-10 when the value is 10. **Throws** on an unknown algorithm, wrong length, or a non-digit. Fixed-length algorithms expect exactly `length – 1` digits (12 for `ean13`, 9 for `isbn10`, etc.).
- `codec.checkdigit.inspect(algo, input): { algo, input, valid, given, computed }` — a diagnostic that combines the two: `given` is the input's last character, `computed` is the recalculated check digit (empty when the input is too short to split), and `valid` is `given === computed` (case-insensitive). Never throws; malformed input yields `valid: false` with an empty `computed`. ISBN-10 accepts a trailing `X` as the check digit.

5.5.4 `codec.php` — PHP dump formats

Three PHP textual formats, each with an encoder and a matching parser:

- `codec.php.serialize(value, opts?): string` — PHP `serialize()`. Emits the canonical wire form (`s: , i: , d: , b: , N; , a: , O:`).
- `codec.php.unserialize(text, opts?): value` — inverse of `serialize`.
- `codec.php.varExport(value, opts?): string` — PHP `var_export()`: valid PHP source (`array ( ... ), \Cls::__set_state( ... )`).
- `codec.php.parseVarExport(text, opts?): value` — read a `var_export` literal back to a value.
- `codec.php.varDump(value, opts?): string` — PHP `var_dump()`: human-readable debug output with type/length annotations.
- `codec.php.parseVarDump(text, opts?): value` — **best-effort** read of `var_dump` output.

Array heuristic. A JS array, or a JS object whose keys are exactly the contiguous integer sequence `0..n-1`, maps to a PHP list array. Any other object maps to a PHP associative array (string/int keys preserved).

The `__class` sentinel. An object carrying a `__class` string key (configurable via `opts.classKey`, default `"__class"`) round-trips as a PHP *object* — `serialize` emits `0:...`, `var_export` emits `\Cls::__set_state(...)`, `var_dump` emits `object(Cls)#...`. The remaining keys become the object's properties. Decoding restores the `__class` key. Without the sentinel a value is treated as an array.

Shared references and cycles. `serialize` / `unserialize` model PHP's `r: / R:` reference markers: when the same object appears more than once, the decoder resolves both occurrences to the *same* JS object (a DAG is preserved). A true cycle (an object reachable from itself) **throws** on encode — there is no JS value that can represent it losslessly here.

`var_dump` parse is lossy. PHP's `var_dump` is a debug view, not a serialization format; `parseVarDump` is best-effort and **throws** when it hits something it cannot faithfully reconstruct: the `*RECURSION*` marker, a length-truncated string (its declared byte length

disagrees with the bytes present), or visibility-annotated property names (`[ "x": "Cls": private ]`).

5.5.5 `codec.perl` — Perl `Data::Dumper`

- `codec.perl.dumper(value, opts?): string` — emit a `Data::Dumper`-style dump (`$VAR1 = ... ;`) with normalized indentation.
- `codec.perl.parseDumper(text, opts?): value` — read `Data::Dumper` output back to a value.

The JSON bool convention. Perl has no native boolean, so JSON modules represent one as a blessed scalar reference (e.g. `bless( do{\(my $o = 1)}, 'JSON::XS::Boolean' )`). `dumper` emits JS booleans in this form, and `parseDumper` decodes a blessed scalar ref in the JSON-bool family (`JSON::XS::Boolean`, `JSON::PP::Boolean`, ...) back to a JS boolean. A bare `1 / 0` stays a number. The class used on encode is `opts.perlBoolClass` (default `"JSON::XS::Boolean"`). Blessed hashes carry the `__class` sentinel, mirroring the PHP side. Cycles **throw**.

5.5.6 `opts` and shared caveats

All `codec.php.* / codec.perl.*` functions accept an optional `opts` bag:

- `classKey` (default `"__class"`) — the key used as the class sentinel.
- `perlBoolClass` (default `"JSON::XS::Boolean"`) — class for Perl booleans.
- `indent` — override the default 2-space indentation step (`varExport`, `dumper`).

Caveats shared with JSON: strings are UTF-8 and lengths are reported in **bytes** (multibyte chars count as more than one); JS has a single number type, so `int` and `float` collapse the same way `JSON.stringify` does (e.g. `1.0` may re-emit as `1`). Decoded objects preserve a stable key order, so re-encoding is byte-stable — safe for canonical-JSON / payment hashing.

5.5.7 `codec.sheet` — tabular CSV / TSV / XLSX / ODS / XLS / SYLK / DIF

Read and write spreadsheet data. All three functions are **synchronous**.

Workbook model:

```
{
  format: "csv" | "tsv" | "xlsx" | "ods" | "xls" | "slk" | "dif",
  sheets: [
    { name: string, rows: (string | number | boolean | null)[][] },
    ...
  ]
}
```

A *cell* is a typed primitive. XLSX and ODS files preserve types: numeric cells come back as `number`, boolean cells as `boolean`, and empty cells as `null`. ODS date cells come back as ISO-8601 strings (e.g. `"2024-03-15"`). CSV and TSV are untyped text formats — every cell is always a `string`.

Legacy read-only formats (XLS / SYLK / DIF): `codec.sheet.read` also reads three legacy formats for extract-and-convert-up workflows. These formats are **read-only** — passing `"xls"`, `"slk"`, or `"dif"` to `codec.sheet.write` throws

"<fmt> is read-only (extract/convert only)". Use `codec.sheet.formats()` to query the capability matrix at runtime.

- **XLS** (Excel 97–2003, BIFF8) — detected by the OLE2 magic bytes `\xD0\xCF\x11\xE0` at the start of the file. Cells come back as the library's formatted strings (extraction-grade: numbers are strings like `"42"`, not JS numbers). Pure-Go via github.com/extrame/xls.
- **SYLK** (`.slk`) — detected by an `ID`; first line or a `.slk` extension. Cells come back as strings or numbers (SYLK's `K` field is typed).
- **DIF** (Data Interchange Format, `.dif`) — detected by a `TABLE` header line or a `.dif` extension. Cells come back as strings or numbers.

-
- `codec.sheet.read(src, opts?): { format, sheets }` — reads a file path **or** `Uint8Array` bytes. Format is auto-detected (XLSX and ODS by their ZIP magic bytes `PK\x03\x04`, then distinguished by the `mimetype` entry; OLE2 magic `\xD0\xCF` → XLS; `ID`; first line → SYLK; `TABLE` first line → DIF; everything else as CSV). A `.tsv` / `.slk` / `.dif` / `.xls` file extension also triggers the corresponding format. Pass `opts.format` to override the detection. CSV/TSV produces a single-sheet workbook; the sheet name is the file's basename (or `"Sheet1"` when reading raw bytes). XLSX, ODS, and XLS produce all sheets in document order.
 - `codec.sheet.write(model, opts): { format, bytes? } | { format, path? }` — encodes to XLSX, ODS, CSV, or TSV. `model` is either:
 - A workbook object: `{ sheets: [{ name?, rows }] }` — one or more named sheets (name defaults to `Sheet1`, `Sheet2`, ...).
 - A bare 2D array: `(string | number | boolean | null)[][]` — shorthand for a single sheet.

`opts.format` is required unless it can be inferred from a `opts.dest` extension (`.xlsx`, `.ods`, `.csv`, `.tsv`). Without `opts.dest` the encoded bytes are returned as `{ format, bytes: Uint8Array }` with `opts.dest` the file is written and `{ format, path }` is returned. Writing `"xls"`, `"slk"`, or `"dif"` throws immediately (read-only). **CSV/TSV single-sheet rule:** writing more than one sheet to CSV or TSV throws immediately — use XLSX/ODS or split the sheets yourself. **Type fidelity in XLSX and ODS:** `number` → numeric cell, `boolean` → boolean cell, `string` → text cell, `null` → empty (no cell written). Round-trips through XLSX or ODS are therefore type-preserving. ODS writes values and types only — no styles or formulas are preserved. Round-trips through CSV/TSV coerce everything to strings via `cellToStr` (numbers formatted without unnecessary decimals, booleans as `"true"` / `"false"`, `null` as `""`).

- `codec.sheet.formats(): { [format: string]: { read: boolean, write: boolean } }` — returns the capability matrix for every supported format. Read-only formats (`xls`, `slk`, `dif`) have `write: false` read+write formats (`csv`, `tsv`, `xlsx`, `ods`) have both `true`. Use this to branch on format support at runtime rather than hard-coding format names.

```
// XLSX round-trip (types preserved)
const xlsx = codec.sheet.write({ sheets: [{ name: "Data", rows: [
  ["item", "qty", "active"],
  ["widget", 42, true],
] }] }, { format: "xlsx" });
const wb = codec.sheet.read(xlsx.bytes);
// wb.sheets[0].rows[1] => ["widget", 42, true] (number + bool, not strings)
```

```

// ODS round-trip (typed cells, no styles/formulas)
const ods = codec.sheet.write({ sheets: [{ name: "Sales", rows: [
  ["item", "qty", "active"],
  ["widget", 42, true],
] ]}], { format: "ods" });
const odsWb = codec.sheet.read(ods.bytes);
// odsWb.sheets[0].rows[1] => ["widget", 42, true]

// CSV round-trip (cells become strings)
const csv = codec.sheet.write([["a", 1, true]], { format: "csv" });
const back = codec.sheet.read(csv.bytes, { format: "csv" });
// back.sheets[0].rows[0] => ["a", "1", "true"]

// Read a SYLK legacy file and convert up to XLSX
const slkBytes = new Uint8Array(slkString.length);
for (let i = 0; i < slkString.length; i++) slkBytes[i] = slkString.charCodeAt(i);
const slkWb = codec.sheet.read(slkBytes, { format: "slk" });
const converted = codec.sheet.write(slkWb, { format: "xlsx" });

// Capability matrix
const f = codec.sheet.formats();
// f.xlsx => { read: true, write: true }
// f.xls => { read: true, write: false }

```

5.5.8 Concepts

codec is a grab-bag of **independent format families**, not a single uniform `parse / stringify` pair applied across every format — each member below exists because a real script needed that specific format, and the method names differ per family:

- **codec.toml** — `parse(text) / stringify(value)`. The one family that mirrors native `JSON.parse / JSON.stringify` shape-for-shape.
- **codec.yaml** — `parse(text) / stringify(value)`, same JSON-shaped pair as `codec.toml`. Mappings ↔ objects, sequences ↔ arrays, scalars ↔ primitives. Only the **first** document of a multi-document (--- -separated) stream is parsed; non-string mapping keys are coerced to their string form.
- **codec.xml** — `decode(xml) / encode(value, opts?)`, using the @-attribute / #text convention (§5.5 above has the full rules).
- **codec.dotenv** — `parse(text) / stringify(obj)`, pure (never touches the process environment — see `runtime.env.load` in §17.11.5.2 to apply a file).
- **codec.sheet** — `read(src, opts?) / write(model, opts)` for tabular data; **CSV lives here**, not as its own namespace — pass `opts.format: "csv"` (see §5.5.7 above for the full CSV/XLSX/ODS/legacy walkthrough).
- **codec.php / codec.perl** — dump-format encoders/decoders (`serialize / unserialize`, `varExport / parseVarExport`, `varDump / parseVarDump`, `dumper / parseDumper`).
- **codec.doc**, **codec.barcode**, **codec.checkdigit**, **codec.compression** — document text extraction, barcode image codecs, check-digit validation, and binary compression, each with their own method names (§5.5.1–§5.5.4 above).

There is no `codec.json`. JSON needs no wrapper — use the built-in `JSON.parse / JSON.stringify`.

Round-trip fidelity varies by format. `codec.toml` and `codec.yaml` preserve ints/floats/booleans/nested structures (datetimes come back as strings); `codec.xml.decode` is fully untyped (every leaf is a string, per the XML spec); `codec.sheet` preserves types through XLSX/ODS but flattens everything to strings through CSV/TSV; `codec.dotenv.stringify` is designed so `parse(stringify(obj))` deep-equals `obj` `codec.php.varDump` / `codec.perl.dumper` are debug formats — `parseVarDump` is best-effort and throws on lossy markers it can't faithfully reconstruct (§5.5.4).

5.5.9 Recipes

5.5.9.1 Parse and stringify a TOML document

```
const cfg = codec.toml.parse(`
title = "sercon demo"
[server]
port = 8080
hosts = ["a.example", "b.example"]
`);
runtime.assert.equal(cfg.server.port, 8080, "parsed int");
runtime.assert.equal(cfg.server.hosts.length, 2, "parsed array");

const text = codec.toml.stringify({ name: "demo", flags: { debug: true } });
const back = codec.toml.parse(text);
runtime.assert.equal(back.flags.debug, true, "round-trip bool");
```

Tables become nested objects, arrays become JS arrays, and ints/floats/booleans map to their JS equivalents; only datetimes come back as strings. The top-level value passed to `stringify` must be an object (TOML documents are tables). Signatures: §17.3.9.1 (`codec.toml.parse`), §17.3.9.2 (`codec.toml.stringify`). runnable: `examples/scripts/codec-toml.ts`

5.5.9.2 Parse and stringify a YAML document

```
const cfg = codec.yaml.parse(`
title: sercon demo
server:
  port: 8080
  hosts: [a.example, b.example]
`);
runtime.assert.equal(cfg.server.port, 8080, "parsed int");
runtime.assert.equal(cfg.server.hosts.length, 2, "parsed sequence");

// A top-level sequence parses to a JS array.
const list = codec.yaml.parse("- one\n- two\n");
runtime.assert.equal(list[1], "two", "top-level sequence");

const text = codec.yaml.stringify({ name: "demo", flags: { debug: true } });
const back = codec.yaml.parse(text);
runtime.assert.equal(back.flags.debug, true, "round-trip bool");
```

Mappings become objects, sequences become arrays, and scalars map to their JS equivalents. Only the **first** document of a `---`-separated stream is parsed (a documented v1 limitation), and non-string mapping keys (`1:`, `true:`) are coerced to their string form so

the mapping stays reachable as an object. See the generated §17 reference for the exact signatures. runnable: `examples/scripts/codec-yaml.ts`

5.5.9.3 Round-trip tabular data as CSV

```
const wb = { sheets: [{ name: "Inventory", rows: [
  ["Name", "Qty", "InStock"],
  ["Widget", 42, true],
] ] }];

const csv = codec.sheet.write(wb, { format: "csv" });
const back = codec.sheet.read(csv.bytes, { format: "csv" });
runtime.assert.equal(back.sheets[0].rows[1][1], "42", "csv cells are strings");
```

CSV is one of `codec.sheet`'s formats, not a standalone codec — pass `opts.format: "csv"` to both `read` and `write`. Unlike XLSX/ODS, CSV is untyped: every cell round-trips as a string (42 becomes "42"). See §5.5.7 above for the XLSX/ODS-typed and legacy-format variants. Signatures: §17.3.8.2 (`codec.sheet.read`), §17.3.8.3 (`codec.sheet.write`). runnable: `examples/scripts/sheet.ts`

5.5.9.4 Convert a parsed document into another format

```
const cfg = codec.toml.parse('title = "demo"\n[server]\nport = 8080\n');

// codec.xml.encode needs a single-key root object — cfg has two top-level
// keys (title, server), so wrap it with opts.rootName instead of nesting
// it under one key by hand.
const xml = codec.xml.encode(cfg, { rootName: "config", indent: "  " });
runtime.log(xml);
// <config>
//   <title>demo</title>
//   <server>
//     <port>8080</port>
//   </server>
// </config>
```

Because every `codec.*` parser lands on plain JS values (objects, arrays, strings, numbers, booleans), the value one format parses is exactly what another format's encoder accepts — no adapter layer needed. `codec.xml.encode` without `opts.rootName` requires a single-key object naming the root, so a multi-key parse result (like a TOML table) needs `rootName` to wrap it. Signatures: §17.3.9.1 (`codec.toml.parse`), §17.3.10.2 (`codec.xml.encode`). runnable: `examples/scripts/codec-toml.ts`, `examples/scripts/codec-xml.ts`

5.5.9.5 Parse and apply a .env file

```
// Pure parse — no environment side effects.
const cfg = codec.dotenv.parse('GREETING="hello world"\n# comment\nexport
COUNT=3\n');
runtime.assert.equal(cfg.GREETING, "hello world", "parse strips quotes");
runtime.assert.equal(cfg.COUNT, "3", "parse handles export");

// Round-trip: stringify sorts keys and quotes where needed.
const text = codec.dotenv.stringify({ A: "1", B: "two words", FLAG: true });
runtime.assert.equal(codec.dotenv.parse(text).B, "two words", "stringify round-
```

```
trips");

// Load a real file AND apply it to the process environment (a `runtime`
// binding, not `codec` — see §17.11.5.2).
await fs.writeText("/tmp/demo.env", 'DEMO_KEY="from file"\n');
const loaded = await runtime.env.load("/tmp/demo.env");
runtime.assert.equal(runtime.env.get("DEMO_KEY"), "from file", "load applied to
env");
```

`codec.dotenv.parse / stringify` never touch the process environment — reach for `runtime.env.load` when a script needs the values applied, not just parsed. Signatures: §17.3.5.1 (`codec.dotenv.parse`), §17.3.5.2 (`codec.dotenv.stringify`), §17.11.5.2 (`runtime.env.load`). runnable: `examples/scripts/env.ts`

5.5.9.6 Round-trip PHP and Perl dump formats

```
const order = { __class: "Order", id: 7, items: ["a", "b"], paid: true, note:
null };

const s = codec.php.serialize(order);
const decoded = codec.php.unserialize(s);
// decoded deep-equals order — the __class sentinel round-trips as a PHP object

const flags = { active: true, archived: false, label: "vip" };
const pl = codec.perl.dumper(flags);
const back = codec.perl.parseDumper(pl);
// back deep-equals flags — JS booleans round-trip via the JSON::XS::Boolean
convention
```

`codec.php.serialize` is deterministic (stable key order), which matters when the output feeds a canonical-hash comparison; `codec.php.varDump / codec.perl.dumper` are debug formats and their parsers are best-effort (`parseVarDump` throws on lossy markers like `*RECURSION*` — see §5.5.4). Signatures: §17.3.7.3 (`codec.php.serialize`), §17.3.7.4 (`codec.php.unserialize`), §17.3.6.1 (`codec.perl.dumper`), §17.3.6.2 (`codec.perl.parseDumper`). runnable: `examples/scripts/dump-codec.ts`

5.6 fs

Filesystem operations:

- `fs.path.dirname(p) / fs.path.basename(p, suffix?)` — POSIX-style path splitting (forward slashes). `dirname` returns everything up to (not including) the final slash — `."` when there is no directory component, `"/` for a rooted single segment; trailing slashes are stripped first. `basename` returns the final segment and, when `suffix` is supplied and the segment ends with it (and is not equal to it), strips that suffix — handy for dropping a known extension. Both **throw a `TypeError`** if `p` is missing, `null`, or `undefined`. These are pure string transforms — they never touch disk. On Windows, normalise separators yourself before calling.
- `fs.archive.create(destPath, sources) / fs.archive.extract(archivePath, destDir, opts?)` — both **async** (return Promises). The archive format is **inferred from the file extension**: `.zip`, `.tar`, `.tar.gz`, and `.tgz` (a `.tgz` resolves to the `tar.gz` format); an unrecognised extension rejects. `create` takes a non-empty array of sources — a bare string uses the disk path as-is with its `basename`

inside the archive, while an `{ path, name }` object overrides the in-archive name; directory sources are recursed (their basename becomes the archive subdirectory) and archive paths always use forward slashes. It resolves to `{ path, format, entries, bytes? }` (`entries` lists the regular files written; `bytes` is the final archive size when `stat` succeeds). `extract` creates `destDir` recursively if absent, applies **zip-slip / tar-slip protection** (entries that resolve outside `destDir` via an absolute path or a `..` component reject the whole call), and honours `opts.overwrite` — the default `false` opens targets with `O_EXCL` so an entry colliding with an existing file fails; pass `true` to clobber. `opts.maxTotalBytes` (default 1 GB) caps the cumulative decompressed bytes written across all members, and `opts.maxEntries` (default 100000) caps the number of members processed — both are decompression-bomb guards (a non-positive value falls back to its default), and extraction aborts as soon as either cap is exceeded. It resolves to `{ path, format, dest, entries }`.

General file read/write (all async; paths used as given, no sandboxing — matching `image.save` and the WebDriver `screenshot` writers):

- `fs.writeText(path, text) → { path, bytes }` — UTF-8, truncating.
- `fs.writeBytes(path, data: Uint8Array) → { path, bytes }`.
- `fs.readText(path) → string` `fs.readBytes(path) → Uint8Array`.
- `fs.mkdir(path) → { path }` — creates parents (`mkdir -p`), idempotent.
- `fs.exists(path) → boolean` — never throws for a missing path.
- `fs.remove(path) → { path }` — file or directory tree; no error if absent.
- `fs.stat(path) → { size, isDir, modifiedMs }`.

`writeText / writeBytes` **do not create parent directories** — call `fs.mkdir` first (Node-like); reads and `stat` reject when the target is absent. See `examples/scripts/fs-report.ts` for a per-step screenshot report built on these.

- `fs.find(opts?)` — **fast file search** (fd-like). Walk one or more `root s` and return matching paths. Respects `.gitignore / .ignore` and skips hidden files **by default** (`gitignore: false / hidden: true` to opt out). Filter with `glob / exclude ( ** supported)`, `regex` (matches the basename, or the full path with `fullPath: true`), `type ( "file" / "dir" / "symlink" )`, and `extension . case` defaults to **smart** (case-insensitive unless the pattern has an uppercase char). Bound recursion with `maxDepth / minDepth` `absolute` returns absolute paths; `limit` caps results; `sort` gives deterministic order. Returns `Promise<string[]>`, or `Promise<FindEntry[]>` with `stat: true` (`{ path, type, size, mtimeMs }`), or an **async iterator** with `stream: true`.
- `fs.grep(opts)` — **fast content search** (rg-like). RE2 pattern (`fixed: true` for a literal fast-path), searched across the walked tree (or explicit `paths`). Same traversal/ignore options as `find`, plus `type` presets (`"ts"`, `"go"`, ...), `word`, `multiline`, `context` (or `before / after`), `invert`, `maxMatches` (per file), `maxResults` (total), and `includeBinary` (binary files are skipped by default). Returns `Promise<GrepMatch[]>` (`{ path, line, column, match, text, before?, after? }`, 1-based) or an async iterator with `stream: true`.
- `fs.grepFiles(opts)` — like `fs.grep` but returns just `string[]` of files with ≥ 1 match (rg `-l` stops at the first hit per file).
- `fs.grepCount(opts)` — like `fs.grep` but returns `{ path, count }[]` per-file counts (rg `-c`).

- `fs.tail(path, opts?)` — **follow a file** and yield each new line (async iterator), like `tail -f`. `opts.from` picks the start: "end" (default — only lines appended after the call), "start" (whole file then follow), or a positive integer `N` (last `N` lines then follow). Survives log rotation and truncation. `for await (const line of fs.tail(path)) { ... } break` (or a Run timeout) stops it. Pure-Go (`fsnotify` + a poll fallback); throws if the file is absent at start.
- `fs.grepStream(path, opts)` — **follow + match** (`tail -f | grep`): yield only lines matching `opts.pattern` as `{ line, column, match, text }`. Matching runs Go-side with `fs.grep`'s RE2 engine, so only matches cross into the script. Same `from/rotation` behaviour as `fs.tail`. `opts` also takes `fixed`, `case`, `word`, `invert`. `line` is a session-relative counter, not a file line number; multiline/context are not supported here (use `fs.grep`).

Unreadable files/dirs are **skipped** by default; pass `strict: true` to throw on the first error instead. Traversal is parallel and pure-Go (no `rg` / `fd` required).

v1 limitations. Ignore handling reads `.gitignore` and `.ignore` only (not `.git/info/exclude` or the global excludes file), and a nested ignore file's **negation** (`!pattern`) does not re-include a path an ancestor ignore file excluded (git's last-match-wins across the hierarchy is not resolved — the first ancestor match wins). `fs.grep`'s `paths` option takes explicit **files**; a directory in `paths` is not recursed. `fs.find`'s `regex` with `fullPath: true` matches the path **relative to its root**, which equals the displayed (cwd-relative) path only when `root` is `."`.

5.6.1 Concepts

`fs` reads and writes the real filesystem — there is **no sandbox**; paths are resolved as given. Text vs bytes: `readText` / `writeText` handle UTF-8 strings, `readBytes` / `writeBytes` handle a `Uint8Array` for binary data. A write's **parent directory must already exist** (Node-like) — create it with `fs.mkdir` first (which itself creates any missing parents, `mkdir -p`). `fs.stat` reports size, whether the path is a directory (`isDir`), and last-modified time (`modifiedMs`); `fs.exists` is a boolean guard that never throws for a missing path.

5.6.2 Recipes

5.6.2.1 Read a file as text or bytes

```
const text = await fs.readText("./config.json");
const bytes = await fs.readBytes("./logo.png"); // Uint8Array for binary
```

5.6.2.2 Write a file, creating its directory first

```
await fs.mkdir("./out/reports"); // creates parents
recursively
await fs.writeText("./out/reports/summary.txt", "done\n"); // parent must already exist
```

5.6.2.3 Guard an operation with exists / stat

```
if (await fs.exists("./cache.db")) {
  const st = await fs.stat("./cache.db");
  runtime.log("cache size:", st.size);
}
```

```

} else {
  runtime.log("no cache yet");
}

```

5.6.2.4 Create and extract an archive

```

await fs.archive.create("./backup.tar.gz", [".out", "config.json"]);
await fs.archive.extract("./backup.tar.gz", "restored");

```

Notes

- `archive.create` / `archive.extract` argument order and the `stat` field names are confirmed against §17.6 (the source of truth for the full surface).

5.6.2.5 Find files by glob, respecting .gitignore

Locate source files without descending into ignored directories (`node_modules`, `dist`, ...) — no external `fd` needed.

```

const files = await fs.find({ root: "src", glob: "**/*.ts", type: "file" });
runtime.log(`${files.length} TypeScript files`);

```

`.gitignore` is honored by default; pass `gitignore: false` to search everything. See §17 (`fs.find`) for the full option list.

5.6.2.6 Search file contents for a pattern

Grep a tree for a regex and print `path:line` for each hit — an in-process `rg`.

```

for await (const m of fs.grep({ pattern: "TODO\\(.*\\)", type: "ts", stream:
true })) {
  runtime.log(`${m.path}:${m.line}: ${m.text.trim()}`);
}

```

Use `fs.grepFiles(...)` for just the file list, or `fs.grepCount(...)` for per-file counts. See §17 (`fs.grep`).

5.6.2.7 Follow a log and react to error lines

Watch a growing log for problems without shelling out to `tail / grep` — the match runs in-process and only ERROR lines reach the script.

```

for await (const m of fs.grepStream("/var/log/app.log", { pattern: "ERROR|
FATAL" })) {
  runtime.log(`hit @${m.line}: ${m.text}`);
  await net.email.send({ /* alert ... */ });
}

```

`fs.grepStream` starts at end-of-file by default (only new lines); pass `from: 50` to replay the last 50 lines first, or `from: "start"` for the whole file. Use `fs.tail(path)` when you want every line (grep/parse in-script). Both survive log rotation. See §17 (`fs.tail`, `fs.grepStream`).

5.7 net

Network clients and probes:

- `net.http.{get, post, request}` — fetch-style HTTP client; `request` accepts headers, body, timeout, retry, follow, and basic auth.
- `net.probe.{tcp, dns, tls, ntp, whois, ping, traceroute, smtp, wss}` — one-shot reachability / handshake probes. Each returns a structured result; failures surface as `Error` objects with details.
- `net.probe.traceroute(host, opts?)` — trace the network path: send probes with increasing TTL and report each responding router. **Needs root / CAP_NET_RAW** (intermediate hops appear only via ICMP `time-exceeded`). `opts.protocol` is `"icmp"` (default), `"udp"` (to an incrementing high port), or `"tcp"` (SYN via a TTL-limited connect); `port` sets the udp/tcp target; `maxHops` (30), `timeout` ms per probe (2000), and `probes` per hop (3) bound the trace. Resolves to one `{ ttl, address, rttMs, reached }` per hop (`address` is `null` for a timed-out hop). IPv4 only.
- `net.probe.ping` also accepts `mode: "udp"` — a UDP datagram to a (closed) port whose ICMP `port-unreachable` proves reachability (needs root / `CAP_NET_RAW`), alongside the existing `"tcp"` (default) and `"icmp"` modes.
- `net.netstatus.check(host)` — combined reachability summary.
- `net.email.*` — `spf`, `dmARC`, `mtaSts`, `tlsRpt`, `bimi`, and `all(domain)` which runs every probe in parallel and aggregates; plus `send({...})` — outbound SMTP sender (see §6.7).
- `net.browser.open()` — open a stateful HTTP session (cookie jar + replayed default headers); **not** an OS-browser launcher.
- `net.tcp.connect(host, port, opts?)` — open a TCP client socket.
- `net.udp.open(opts)` — open a UDP socket (connected or bound).
- `net.icmp.open(opts?)` — open a raw ICMP socket (needs privileges).
- `net.capture.{interfaces, routes, open, openFile, toFile}` — packet capture (live + pcap file I/O), pure-Go gopacket.
- `net.raw.{open, tcp}` — craft & send raw IPv4 packets (TCP flags / UDP / arbitrary IP protocol) and receive replies (needs privileges).
- `net.load.http(opts)` — authorized HTTP load / resilience self-test (worker-pool, latency percentiles + error rate; public hosts need `confirm:true`).

5.7.1 HTTP client (`net.http`)

Three entry points, all returning a `Promise`. **A 4xx/5xx response never rejects** — it resolves with the status set; only transport-level failures (DNS, connection refused, TLS handshake) or a deadline reject.

- `net.http.get(url)` / `net.http.post(url, body?)` — quick one-liners with a **5-second** default timeout. Resolve to `{ status, body, bodyBytes }`: `body` is the response decoded as UTF-8, `bodyBytes` is the raw, undecoded `Uint8Array` (pair with `text.charset.decode` for ISO-8859-1 and other non-UTF-8 payloads). `post` sends `body` verbatim and sets **no** `Content-Type` — add one via `request` if the server needs it.
- `net.http.request(method, url, opts?)` — the full client. `opts` is `{ headers?, body?, multipart?, timeout? (ms, default 30000), retry? (extra attempts, default 0), follow? (default true), username?, password?, maxBytes? (default 256 MB) }`.
`retry` applies **only** to transport errors and 5xx, with linear backoff capped at 1 s; a malformed method/URL rejects immediately and is not retried. `follow: false` stops at

the first 3xx. `maxBytes` caps the response body size read into memory — a caller may raise or lower it; a response over the cap rejects instead of resolving. Resolves to `{ status, ok, headers, body, bodyBytes, url }` — `ok` is `status` in `[200, 400)`; `headers` is a lower-cased name→value map (last value wins); `url` is the final URL after redirects. `net.http.request` also accepts a binary `body` (`Uint8Array` / `ArrayBuffer`, sent byte-for-byte) and a `multipart` array for multipart/form-data uploads — each part is either a text field (`{ name, value }`) or a file (`{ name, filename, content, type? }`); `body` and `multipart` are mutually exclusive, and `multipart` sets the `Content-Type` header automatically.

5.7.2 One-shot probes (`net.probe`)

Each `net.probe.*` helper performs a single handshake/lookup and resolves with a structured result. Connection-level failures reject; *findings* (an expired cert, a missing STARTTLS, an unreachable host) are returned as data.

- `net.probe.tcp(target, opts?)` — dial and report `{ host, port, ip, latencyMs }`. `opts.port` (default "80") supplies a port when `target` is a bare host; `opts.timeout ms` (default 5000).
- `net.probe.dns(host, opts?)` — resolve A / AAAA / MX / TXT / CNAME / NS. `opts.types` restricts the set (case-insensitive). **Always resolves** — per-type failures and empties are simply omitted from the result, so test presence with "mx" in `result`. `mx` entries are `{ preference, host }`.
- `net.probe.tls(target, opts?)` — open a TLS connection with **verification skipped** (probe-only) and return the leaf cert summary `{ cn, issuer, notBefore, notAfter, daysRemaining, dnsNames, serialNumber, fingerprintSha256 }`. The host is sent as SNI; expired or name-mismatched certs still report (that's the point). A bare host uses port 443.
- `net.probe.ntp(host, opts?)` — query an NTPv4 server (UDP 123) and report `{ serverTime, offsetMs, rttMs, stratum, referenceTime, rootDelayMs, rootDispersionMs }`.
- `net.probe.whois(domain, opts?)` — two-hop WHOIS via the IANA referral. `raw` is always the full response text; `domain` / `registrar` are best-effort parsed and omitted for TLDs the parser doesn't recognise. A parse failure is non-fatal (only `raw` comes back); a wire failure rejects.
- `net.probe.smtp(host, opts?)` — SMTP **capability** probe (EHLO only, no mail). Returns `{ banner, ehloDomain, extensions, starttls, authMechanisms, sizeLimit }`. A server that omits STARTTLS/AUTH reports `false` / `[]` — a finding, not an error. `opts.port` default "25", `opts.ehloName` default "localhost".
- `net.probe.wss(url, opts?)` — WebSocket handshake probe. Returns `{ connected, subprotocol, status, handshakeMs, pingMs }` and closes immediately. `opts.ping` (default `true`) toggles the RTT measurement; `pingMs` is `-1` when skipped or unanswered.

`net.netstatus.check(host, opts?)` rolls DNS / TCP / TLS / HTTP into **one concurrent** sweep, resolving to `{ host, port, elapsedMs, reachable, dns, tcp, tls, http }`. Each sub-probe is `{ ok, ..., error? }` — sub-failures are **captured as data**, never thrown; only an empty `host` rejects. `reachable` is `dns.ok && tcp.ok` (TLS/HTTP are reported but don't gate it). `opts.port` defaults to "443".

5.7.3 Email-authentication probes (`net.email`)

DNS-based deliverability / anti-spoofing posture checks. Each takes a `domain` and resolves to a discriminated union — `{ present: false }` when the record is absent, or `{ present: true, record, ... }` with the parsed fields when found. **They reject only on a DNS error other than NXDOMAIN**; a missing record is the normal `{ present: false }` outcome, not a throw.

- `net.email.spf(domain)` → `{ record, mechanisms, allPolicy }` from the apex `TXT (v=spf1)`; `allPolicy` summarises the trailing `all` mechanism (`pass / fail / softfail / neutral`).
- `net.email.dmarc(domain)` → the `_dmarc.<domain> v=DMARC1` record with `{ tags, policy, subdomain, percent, rua, ruf }`.
- `net.email.mtaSts(domain)` → the `_mta-sts.<domain> TXT` **plus** the fetched well-known policy file (`{ mode, mx, maxAge }`); a policy-fetch failure surfaces in `policyError`, not as a throw.
- `net.email.tlsRpt(domain)` → the `_smtp._tls.<domain> v=TLSRPTv1` record with `{ tags, rua }`.
- `net.email.bimi(domain, opts?)` → the `<selector>._bimi.<domain>` record with `{ l, a }` (logo URL + VMC assertion); `opts.selector` defaults to `"default"` and is echoed back in the result.
- `net.email.all(domain)` runs all five **in parallel** and aggregates under `{ domain, spf, dmarc, mtaSts, tlsRpt, bimi }` a single probe's failure surfaces as `{ error }` under its key rather than failing the aggregate, so this call always resolves.

5.7.4 Stateful HTTP session (`net.browser`)

`net.browser.open()` (no arguments) opens a **stateful HTTP session** — an `http.Client` with an automatic, public-suffix-scoped **cookie jar** and a set of default headers replayed on every request, the way a browser carries a session across page loads. It does **not** launch the OS web browser. It resolves to a handle:

- `setUserAgent(ua) / setHeader(name, value)` — register default headers replayed on subsequent requests (synchronous).
- `get(url) / post(url, body?)` — issue a request through the session; resolve to the same `{ status, ok, headers, body, url }` shape as `net.http.request`. The jar is updated automatically, so a login `post` followed by a `get` replays the session cookie with no manual handling.
- `cookies(url)` — list the jar's cookies scoped to `url` as `{ name, value }[]`, e.g. to confirm a login set the expected cookie.

`open()` rejects only if the cookie jar can't be created; `get / post / cookies` reject on an empty/unparseable URL or a transport error (4xx/5xx resolve normally).

Raw sockets (`net.tcp / net.udp / net.icmp`) are long-lived, bidirectional client sockets with a *push / callback* read model — unlike the one-shot `net.probe.*` helpers, they stay open and deliver inbound data through callbacks until you `close()` them. Each constructor returns a `Promise` for a handle object:

- `net.tcp.connect(host, port, opts?)` — dial a TCP peer. `opts` `{ timeout? (ms, default 10000), readBuffer? (inbound channel capacity, default 64) }`. The handle has `write(data)` (`string` → UTF-8 bytes, `Uint8Array` → raw bytes; returns a

Promise), the `remote` / `local` address strings, plus the shared callbacks below. Inbound chunks arrive via `onData`.

- `net.udp.open(opts)` — two modes selected by `opts`. **Connected** `{ host, port, readBuffer? }` resolves the peer up front and exposes `send(data)`. **Bound** `{ bind: "127.0.0.1:0", readBuffer? }` listens on a local address and exposes `sendTo(data, host, port)` its inbound events also carry `address` / `port` of the sender. Either way the handle has `local` (the bound address — read the chosen port back from it when you bind to `:0`) and delivers datagrams via `onMessage`.
- `net.icmp.open(opts?)` — open a raw ICMP socket. **Requires root / CAP_NET_RAW**; without the privilege `open()` rejects with an error naming that requirement. `opts { network?: "ip4" | "ip6" (default "ip4"), readBuffer? }`. `send(opts)` writes a message in one of two modes: **Echo mode** `{ to, type?, code?, id?, seq?, payload? }` builds an Echo-shaped body (`type` defaults to the network's echo request), or **raw mode** `{ to, type, code?, body }` marshals `body (Uint8Array | string)` verbatim — for hand-built non-Echo messages such as a crafted destination-unreachable. In raw mode `type` is required and `body` is mutually exclusive with `id` / `seq` / `payload`. The handle has `network` / `local` inbound events carry `address` / `type` / `code` and arrive via `onMessage`.

All three handles share the same callback surface and lifecycle:

- `onData(cb)` (TCP) / `onMessage(cb)` (UDP, ICMP) — register a listener for inbound data. The event object carries `bytes` (a `Uint8Array`) and `text` (the bytes decoded as UTF-8); UDP-bound / ICMP events add the sender metadata noted above.
- `onClose(cb)` — fires once when the socket closes (locally or remotely).
- `onError(cb)` — fires on a read / connection error (benign teardown closes are reported through `onClose`, not `onError`).
- `close()` — shut the socket down; returns a `Promise`.

The socket keeps the engine's event loop alive while open (via the internal `HoldRun` refcount), so a script that only listens won't exit until it `close()`s. `sercon` scripts can't `bind` a listening TCP/UDP server socket yet — these are client sockets — so a TCP example needs a peer to dial; a UDP loopback pair is fully self-contained.

5.7.5 Packet capture (`net.capture`)

Packet capture and pcap file I/O, powered by **pure-Go gopacket** (no `libpcap`, no `cgo`). Five bindings:

- `net.capture.interfaces()` — **synchronous**; returns an array of `{ name, addresses: string[], up, loopback }` for the host's network interfaces. Pure-Go (`net.Interfaces`); no privileges, all platforms.
- `net.capture.routes()` — **synchronous**; returns an array of `{ destination, gateway, interface, family, metric }` for the host's IP routing table. `destination` is a CIDR (`0.0.0.0/0` / `::/0` are the default routes); `gateway` is the next-hop IP or `""` for a directly-connected route; `family` is `"ip"` or `"ip6"` `metric` is `0` where the platform doesn't report one. Pure-Go, unprivileged: Linux parses `/proc/net/route` + `/proc/net/ipv6_route` macOS/BSD read the routing socket via `golang.org/x/net/route`. **Windows is unsupported** (throws).
- `net.capture.open({ iface, promisc?, snaplen?, filter? }, pkt => {...})` — start a **live** capture on `iface`. Resolves to a handle `{ iface, link, close() }` the per-frame handler receives a decoded packet object (see below). `promisc` defaults `true` `snaplen` defaults

262144 . filter is an optional tcpdump-like expression string (see **Filtering** below).

Linux + macOS only — Linux uses AF_PACKET , macOS uses BPF (/dev/bpf); **Windows is unsupported** and open() rejects there. Requires **root / CAP_NET_RAW (Linux)** or **/dev/bpf read access (macOS)**; without the privilege open() rejects naming the requirement.

close() returns Promise<void> .

- net.capture.openFile(path, pkt => {...}, opts?) — read a .pcap / .pcapng file (format auto-detected from the magic bytes), calling the handler once per decoded packet. Returns a Promise that resolves at EOF. Offline; no privileges. opts is an optional trailing argument { filter? } (the two-argument form still works); filter is the same tcpdump-like expression string as open (see **Filtering** below).
- net.capture.toFile(path, { linkType?, snaplen? }) — open/create a .pcap file and return { write(bytes, { ts? }), close() }. write appends a raw frame (a Uint8Array); opts.ts (ms) overrides the per-frame timestamp (defaults to now). close() flushes and returns Promise<void> . linkType defaults to Ethernet, snaplen to 262144 . Offline; no privileges.

The **decoded packet object** passed to the open / openFile handlers:

```
{
  ts, length, captureLength, link,      // always present
  eth?: { src, dst, type },
  vlan?: { id, priority, drop, type }, // 802.1Q tag; `type` is the inner
ethertype
  arp?: { operation, senderMac, senderIp, targetMac, targetIp }, // operation:
"request"|"reply"
  ip?: { version, src, dst, protocol, ttl },
  tcp?: { srcPort, dstPort, seq, ack, window, checksum,
    flags: { syn, ack, fin, rst, psh, urg },
    options?: { mss?, windowScale?, sackPermitted?, timestamps?: { val,
ecr } } },
  udp?: { srcPort, dstPort, length },
  icmp?: { type, code },
  dns?: { id, qr, opcode, rcode,
    questions: { name, type }[],
    answers: { name, type, data }[] }, // data rendered by type (A/
AAAA→IP, CNAME→name, ...)
  payload?: Uint8Array, // application-layer bytes, if any
  bytes: Uint8Array, // the full raw frame (always present)
}
```

Layer keys (eth / vlan / arp / ip / tcp / udp / icmp / dns / payload) are present **only when that layer decodes** — a truncated or unrecognised frame still yields the always-present ts / length / captureLength / link / bytes . tcp.options is present only when the segment carries recognised options.

Filtering. Both open and openFile accept an optional filter string — a **subset of tcpdump syntax**. Supported grammar:

- protocols: tcp , udp , icmp , ip , ip6
- hosts: host X , src host X , dst host X (IPv4 or IPv6 address);
- networks: net X/Y , src net X/Y , dst net X/Y (IPv4 or IPv6 CIDR prefix, e.g. net 10.0.0.0/8 or net 2001:db8::/32);

- ports: port N, src port N, dst port N
- port ranges: portrange A-B, src portrange A-B, dst portrange A-B (inclusive, e.g. portrange 8000-8100);
- boolean composition: and, or, not, and parentheses;
- **implicit-and** between juxtaposed primaries — tcp port 80 is exactly tcp and port 80.

Example: `net.capture.open({ iface: "en0", filter: "tcp and port 80" }, pkt => { ... })`, or `net.capture.openFile("cap.pcap", pkt => { ... }, { filter: "udp or icmp" })`.

The filter is a **userspace, post-decode** predicate — it is **not** compiled to a kernel BPF program. It runs on each frame *after* gopacket has decoded it, so it saves the JS-callback dispatch and object-conversion cost for non-matching packets but does **not** avoid the kernel→userspace copy (a real kernel filter would). A malformed expression makes `open` / `openFile` **reject**.

Limitations. No live capture on Windows. The `filter` grammar is the tcpdump subset above; for anything beyond it, filter inside the callback on the decoded fields. Common-layer decode only (Ethernet / IPv4 / IPv6 / TCP / UDP / ICMP); exotic protocols surface only as raw bytes. At high packet rates frames backpressure and drop at the kernel, exactly like tcpdump.

The pcap file API round-trips: write raw frames with `toFile`, read them back decoded with `openFile`, no privileges required (see `examples/scripts/capture-file.ts`).

5.7.6 Raw packets (net.raw)

Craft and send raw IPv4 packets and receive the replies — for SYN scans, RST/port-state inference, custom-TTL probes, and exotic-packet testing. **Linux + macOS only; needs root / CAP_NET_RAW** (Windows rejects). Two bindings:

- `net.raw.open({ iface?, filter?, readBuffer? })` — open a raw packet engine.
`send(spec | Uint8Array)` crafts and fires a packet: a structured spec
`{ dst, dstPort?, srcPort?, src?, proto?: "tcp"|"udp"|"ip", protocol?, flags?: string[], seq?, ack?, window?, ttl?, ipId?, payload? }` (full source-IP / TTL / IP-ID control; sercon computes checksums), or a `Uint8Array` holding a complete IPv4 packet sent verbatim. `onPacket(cb)` delivers decoded replies (the **same decoded packet object** as `net.capture`); `onClose` / `onError` / `close()` (returns a `Promise`) round out the handle. `iface` defaults to the auto-detected default-route interface — set it explicitly on multi-homed hosts; `filter` is the same tcpdump-like expression as `net.capture`. Default `flags` ["SYN"], `ttl` 64, `window` 65535, `src` = the egress interface IP, `srcPort` = a random high port.
- `net.raw.tcp(host, port, opts?)` — one-shot: send a single crafted TCP segment (default a SYN) and resolve with the first reply correlated by the 4-tuple, or `null` on timeout. SYN → SYN/ACK = open, RST = closed, null = filtered/no answer. `opts`
`{ flags?, srcPort?, src?, seq?, ttl?, payload?, timeout? (ms, default 2000), iface? }`.

> **Kernel-RST caveat:** because the host kernel does not own the crafted > connection, it may answer the peer's SYN/ACK with its own RST. The probe > still captures the reply; only the target's connection state is perturbed. > To probe silently, add a host firewall rule dropping the outbound RST > (`iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP`, or the `pf` > equivalent). `sercon` does not modify your firewall.

5.7.7 HTTP load / resilience self-test (`net.load`)

`net.load.http(opts)` is an **authorized HTTP load / resilience self-test harness**: a worker pool drives a target you operate at a chosen concurrency, and the call resolves with a latency/error report. It is a **defensive** tool — for staging / CI / lab self-tests — not a flooding weapon: it is a plain HTTP client loop with **no raw packets, source spoofing, amplification, or randomized-target fan-out**.

> **Authorized-use guardrail.** A target whose host resolves to loopback, > a private/ULA range, link-local, the unspecified address, or `localhost` > runs unconditionally (it's your own infra). Any **public** host is > **refused unless you pass `confirm: true`** — throwing > `net.load.http: refusing to load-test public host <h> without confirm:true` (authorized self-testing only) `.>`
`confirm: true` is the script author asserting they are authorized to > load-test that host. Concurrency is hard-capped at **1000** (values above > throw); the default is a conservative **10**. The guardrail is re-applied > on every redirect hop, not just the initial URL — a redirect (a > response-controlled, not author-controlled, value) to a public host > without `confirm: true` is refused the same way, and the chain is capped > at 10 hops.

```
net.load.http({
  url: string,           // http/https target (required)
  method?: string,       // default "GET"
  headers?: Record<string, string>,
  body?: string,
  concurrency?: number,  // parallel workers; default 10, capped at 1000
  requests?: number,     // total requests to send ...
  duration?: number,     // ... OR run for this many ms (exactly one required)
  rps?: number,          // optional client-side rate cap (req/s; 0 =
unlimited)
  timeout?: number,      // per-request ms; default 10000
  confirm?: boolean,     // required true to target a public host
}): Promise<LoadReport>
```

- Provide **exactly one** of `requests` (> 0) or `duration` (> 0 ms); giving neither or both throws.
- A 4xx/5xx is a normal **recorded** response (it counts in `statusCounts`), not a thrown error; only transport failures (no response) count as `failed`. `errorRate` is `(failed + 5xx) / sent`.

The report:

```
interface LoadReport {
  target: string;
  method: string;
  concurrency: number;
  durationMs: number; // wall-clock of the run
  sent: number;        // requests started
  completed: number;   // got an HTTP response (any status)
  failed: number;      // transport errors / timeouts (no response)
  rps: number;         // achieved throughput (completed / durationSec)
  errorRate: number;   // (failed + 5xx) / sent, 0..1
  latency: {           // milliseconds, over completed requests
    min: number; mean: number; p50: number; p90: number;
```

```

    p95: number; p99: number; max: number;
  };
  statusCounts: Record<string, number>; // e.g. { "200": 990, "503": 10 }
  errors: Record<string, number>;      // transport error kind → count
}

```

Latency percentiles are nearest-rank over the completed-request latencies. Transport errors are bucketed by kind (`timeout` , `refused` , `dns` , `reset` , `canceled` , `error`). The run honours the engine's cancellation (Run end / `--timeout` aborts in-flight requests). See `examples/scripts/load.ts` .

5.7.8 Concepts

`net.http` is the general-purpose HTTP client — reach for it first for calling APIs and fetching pages. The rest of `net` is diagnostic tooling layered on top, aimed at host reconnaissance and testing: `net.probe.*` runs one-shot reachability/handshake checks (`tcp` , `dns` , `tls` , `ntp` , `whois` , `ping` , `traceroute` , `smtp` , `wss`); `net.email.*` checks a domain's DNS-based deliverability / anti-spoofing posture (SPF, DMARC, MTA-STS, TLS-RPT, BIMI); `net.capture` / `net.raw` / `net.icmp` work at the packet level, for traffic analysis and crafted-packet probing. **Packet capture and raw sockets need elevated privileges** — live capture (`net.capture.open`) and `net.raw.*` / `net.icmp.open` need `root` / `CAP_NET_RAW` on Linux or `/dev/bpf` access on macOS; the offline pcap-file path (`net.capture.toFile` / `openFile`) needs none. `net.load.http` is a separate, **authorized** in-process HTTP load / resilience self-test — a plain worker-pool HTTP client loop, not a raw-packet flood tool — for exercising a service you operate under rising concurrency.

5.7.9 Recipes

5.7.9.1 Make an authenticated JSON request with retry

```

const r = await net.http.request("GET", "https://api.example.com/status", {
  headers: { Authorization: "Bearer " + token },
  retry: 2,           // extra attempts on transport errors + 5xx (linear backoff,
                      // capped at 1s)
  timeout: 10000,
});
if (r.ok) {
  runtime.log(r.status, r.body);
} else {
  runtime.log("request failed with status", r.status);
}

```

See §17.10.4.3 (`net.http.request`) for the full `opts` shape.

5.7.9.2 Probe a host:port for reachability + latency

```

const tcp = await net.probe.tcp(TARGET + ":443", { timeout: 5000 });
runtime.log("port open, resolved to", tcp.ip, "latency", tcp.latencyMs.toFixed(1),
"ms");

```

runnable: `examples/scripts/advanced/recon-host-report.ts`

5.7.9.3 Check a TLS certificate's expiry

```
const tls = await net.probe.tls(TARGET, { timeout: 8000 });
const status = tls.daysRemaining > 30 ? "OK" : tls.daysRemaining > 7 ? "WARNING" :
"CRITICAL";
runtime.log(tls.cn, "expires", tls.notAfter.slice(0, 10), `(${tls.daysRemaining}d)
[${status}]`);
```

runnable: `examples/scripts/advanced/recon-host-report.ts`

5.7.9.4 Check a domain's email-authentication posture (SPF / DMARC / ...)

```
const posture = await net.email.all("example.com"); // spf, dmarc, mtaSts, tlsRpt,
bimi in parallel
for (const k of ["spf", "dmarc", "mtaSts", "tlsRpt", "bimi"] as const) {
  const probe = posture[k];
  runtime.log(k, probe.error ? "ERROR " + probe.error : probe.present ?
"present" : "absent");
}
```

See §17.10.3.1 (`net.email.all`). **Note:** there is no standalone DKIM probe — DKIM is a per-message signature keyed by a selector, not a single domain-wide DNS record the way SPF/DMARC/MTA-STS/TLS-RPT/BIMI are, so it isn't part of the `net.email` surface.

5.7.9.5 Capture packets to a file

```
const w = net.capture.toFile("/tmp/capture.pcap", { snaplen: 65536 });
w.write(frameBytes, { ts: Date.now() }); // frameBytes: a raw Ethernet/IPv4/...
frame
await w.close();

await net.capture.openFile("/tmp/capture.pcap", (pkt: any) => {
  const proto = pkt.tcp ? "tcp" : pkt.udp ? "udp" : "other";
  runtime.log(proto, pkt.tcp?.dstPort ?? pkt.udp?.dstPort);
});
```

Notes

- This file round-trip (`toFile` / `openFile`) needs no privileges.
- Live capture (`net.capture.open({ iface, ... }, pkt => {...})`) needs `root` / `CAP_NET_RAW` (Linux) or `/dev/bpf` access (macOS); pick `iface` from `net.capture.interfaces()`.

runnable: `examples/scripts/advanced/packet-analysis.ts`

5.7.9.6 Run a quick load / resilience self-test

```
const srv = await server.http.listen({
  port: 38090,
  routes: { "GET /work": (_req: any, res: any) => res.json({ ok: true }) },
});

const report = await net.load.http({
  url: `http://127.0.0.1:38090/work`,
  requests: 200,
  concurrency: 10,
```

```
});

runtime.log("sent:", report.sent, "completed:", report.completed, "rps:",
report.rps);
runtime.log("p50/p95/max ms:", report.latency.p50, report.latency.p95,
report.latency.max);
runtime.assert.equal(report.failed, 0, "no transport failures");

await srv.close();
```

Notes

- Loopback / private / link-local / localhost targets run unconditionally; a public host is refused unless `confirm: true` is passed (§5.7.7).
- `examples/scripts/advanced/load-resilience.ts` demonstrates the same ramp-and-assert pattern hand-rolled directly on `net.http.get` with custom latency-percentile math — reach for that shape when you need assertions or metrics beyond `net.load.http`’s built-in report.

runnable: `examples/scripts/load.ts`

5.8 db

Database / KV / directory clients:

- `db.sqlite.open(path)` — embedded SQLite (cgo-free driver).
- `db.postgres.open(dsn | opts)` — PostgreSQL via pure-Go `pgx`.
- `db.mysql.open(dsn | opts)` — MySQL / MariaDB via pure-Go `go-sql-driver`.
- `db.mssql.open(dsn | opts)` — Microsoft SQL Server via pure-Go `go-mssqldb`.
- `db.clickhouse.open(dsn | opts)` — ClickHouse via pure-Go `clickhouse-go v2` (`opts.secure` for TLS).
- `db.oracle.open(dsn | opts)` — Oracle via pure-Go `go-ora` (`opts.database` is the service name).
- `db.redis.open(url)` — Redis client (`redis://` / `rediss://` URL).
- `db.valkey.open(url)` — Valkey client (the RESP-compatible Redis fork; same `{do, ping, close}` handle, accepts `valkey://` / `valkeys://` as well as `redis://`).
- `db.memcached.open(addr)` — Memcached client.
- `db.ldap.open(url, opts?)` — LDAP client (search, bind).
- `db.dict.{define, match}` — RFC 2229 DICT client: remote word-definition lookup / prefix match.

SQL engines (`sqlite` / `postgres` / `mysql` / `mssql` / `clickhouse` / `oracle`) share one

handle: `open()` resolves to `{ exec, query, queryValue, begin, prepare, close }`. The three methods map onto the row shapes you’d expect — `exec(sql, ...params)` runs a non-`SELECT` statement and resolves to `{ rowsAffected, lastInsertId }`

`query(sql, ...params)` resolves to **one ordered object per row**, keyed by column name in column order (the key order is stable run-to-run, so a `JSON.stringify` of the result is reproducible for canonical-hash callers); `queryValue(sql, ...params)` resolves to the **first column of the first row**, or `null` when no rows match (handy for `COUNT(*)` / `EXISTS` -style probes). Transactions come from `begin()` →

`{ exec, query, queryValue, commit, rollback }`, and prepared statements from `prepare(sql)` → `{ exec, query, queryValue, close }` (the same `exec/query/queryValue`

surface, but the SQL is compiled once and the methods take only bind params). The server engines accept either a driver **DSN string** (used verbatim) or a connection **options object** (`{ host, port, user, password, database }`, plus `sslmode` for postgres and `secure` for clickhouse TLS; `database` is the *service name* for oracle); credentials in the assembled URL DSNs are percent-escaped. The connection is pinged on `open()` so a bad DSN or unreachable server fails there, not at the first query (and the underlying pool is closed on a failed ping rather than leaked). Bind parameters are positional and passed after the SQL string — write your engine’s placeholder syntax: `?` (sqlite, mysql, clickhouse), `$1` (postgres), `@p1` (mssql), `:1` (oracle). Values cross to Go losslessly (JS numbers, strings, booleans, `null`, and `Uint8Array` → `[]byte`); on the way back, a `[]byte` column that is valid UTF-8 surfaces as a `string` (the common `TEXT` case) while genuinely binary bytes surface as a `Uint8Array`. All six drivers are pure Go (no cgo). Scripts must `close()` the handle (and commit/rollback transactions, close prepared statements) — there is no GC finalizer, so a leaked transaction pins a pooled connection.

KV / directory / lookup engines. `redis` and `valkey` share the RESP handle

`{ do, ping, close } : do(cmd, ...args)` runs an arbitrary command (`do("SET", "k", "v")`, `do("HGETALL", "h")`), so the binding stays small while covering the whole command surface — the reply is coerced to the obvious JS value (string / number / array / `null`), and a nil reply (missing key) resolves to `null` rather than throwing; RESP-level errors (`WRONGTYPE`, unknown command) do throw. Both ping on `open()` `valkey` additionally accepts `valkey://` / `valkeys://` URLs (normalised to `redis://` / `rediss://`). `memcached` returns `{ get, set, delete }` — `get` resolves to the stored string or `null` on a miss, `set(key, value, expirySeconds?)` stores bytes (`0`/omitted = never expire), and `delete` resolves `true` / `false` for hit/miss; there is **no** ping-on-open and **no** `close` (`gomemcache` pools lazily, GC’d with the handle). `ldap` is a read-only directory inspector: `open()` does an anonymous bind (or `opts.bindDN` / `opts.password`) and returns `{ rootDSE, search, close }`, where `search(baseDN, filter, attrs?)` runs a whole-subtree search returning one ordered `{ dn, <attr>: string[] }` object per entry (multi-valued attributes stay arrays). `dict` is a one-shot RFC 2229 client (`define` / `match`) — a “no match” is data (`found: false` / empty list), not a thrown error.

Integration testing. The `db.*` tests above are unit-level. To exercise the networked bindings against real servers, point the `SERCON_TEST_*` env vars at a running database and run the `Integration` tests; unset vars skip, so `make test` / CI stay green without any server. `make test-integration` does it all against the `dbplayground` fleet: it brings the fleet up, runs the tests (postgres / mysql / mariadb / clickhouse / redis / valkey / memcached / ldap), and tears it down. Requires Docker + Compose v2.

5.8.1 Concepts

All six SQL engines (`sqlite` / `postgres` / `mysql` / `mssql` / `clickhouse` / `oracle`) share the same handle shape — `open()` resolves to

`{ exec, query, queryValue, begin, prepare, close }`. The recipes below use `db.sqlite`, the pure-Go, cgo-free engine that needs no external server (`":memory:"` or a file path), but the `exec/query/queryValue/ begin/prepare/close` vocabulary carries over verbatim to the other five — only the placeholder syntax and connection options differ (see the intro above).

- **`exec(sql, ...params)`** runs a non-`SELECT` statement (DDL, `INSERT` / `UPDATE` / `DELETE`) and resolves to `{ rowsAffected, lastInsertId }`.

- **query(sql, ...params)** runs a `SELECT` and resolves to one ordered object per row, keyed by column name.
- **queryValue(sql, ...params)** is a shortcut for a single scalar — the first column of the first row, or `null` when nothing matches (built for `COUNT(*)` / `EXISTS` -style probes).
- **Bind parameters, don't string-concatenate.** Pass values as `?` placeholders (sqlite's positional placeholder) after the SQL string rather than splicing them into the SQL text — bound params are injection-safe and let the driver do type coercion. Values cross to Go losslessly (numbers, strings, booleans, `null`, `Uint8Array` → `[]byte`).
- **begin()** returns a transaction handle with the same `exec/query/queryValue` plus `commit()` / `rollback()` — group related writes so they become visible atomically, or not at all on rollback (or a thrown error left unhandled).
- **prepare(sql)** compiles a statement once and returns `{ exec, query, queryValue, close }` that take only bind params — reach for it before a loop of many similar writes/reads instead of re-parsing the same SQL string on every call.
- **No finalizer.** Every `open()` needs a matching `close()`, every `begin()` a `commit()` or `rollback()`, and every `prepare()` a `close()` — a leaked transaction or statement pins a pooled connection until the process exits.

The six SQL engines at a glance. The handle is identical everywhere; only the connection argument, the placeholder token, and the default port differ. Pass either the driver's native **DSN string** (used verbatim) or the **options object** (assembled into a URL with credentials percent-escaped); every server engine pings on `open()`, so a bad DSN or unreachable host fails there rather than at the first query.

Engine	open() argument	Placeholder	Default port
sqlite	<code>":memory:"</code> or a file path	<code>?</code>	— (embedded)
postgres	DSN, or <code>{ host, port, user, password, database, sslmode }</code>	<code>\$1</code>	5432
mysql	DSN, or <code>{ host, port, user, password, database }</code>	<code>?</code>	3306
mssql	DSN, or <code>{ host, port, user, password, database }</code>	<code>@p1</code>	1433
clickhouse	DSN, or <code>{ ..., secure }</code> (TLS)	<code>?</code>	9000
oracle	DSN, or <code>{ ..., database }</code> (service name)	<code>:1</code>	1521

- **Key-value engines — redis / valkey / memcached.** `redis` and `valkey` share one RESP handle `{ do, ping, close }`: `do(cmd, ...args)` issues any command, so the whole Redis verb surface is reachable without a per-command binding (`do("SET", k, v)`, `do("INCR", c)`, `do("HGETALL", h)`). Replies coerce to the obvious JS value and a missing key resolves to `null`, while RESP errors (`WRONGTYPE`, unknown command) throw; `valkey` additionally accepts `valkey://` / `valkeys://` URLs. `memcached` is a narrower cache handle `{ get, set, delete }` — `set(key, value, expirySeconds?)` (`0` / omitted = never expire), `get` → the string or `null` on a miss, `delete` → `true` / `false`.
- **Directory and lookup — ldap / dict.** `ldap.open(url, opts?)` does an anonymous bind (or `opts.bindDN` / `opts.password`) and returns a read-only inspector `{ rootDSE, search, close }`: `search(baseDN, filter?, attrs?)` runs a whole-subtree search returning one ordered `{ dn, \<attr\>: string[] }` object per entry (multi-valued

attributes stay arrays; `filter` defaults to `(objectClass=*)`. `dict` is a one-shot RFC 2229 client — `define(host, word, opts?)` and `match(host, word, opts?)` each connect, query, and disconnect; a “no match” is `data ( found: false / an empty list)`, never a thrown error.

- **Close what you open — with two exceptions.** SQL handles, transactions, prepared statements, the KV (`redis / valkey`) handle, and the LDAP handle all expose `close()` and have no GC finalizer, so a leaked handle pins a connection. The exceptions are `memcached` (lazy pool, no ping and no `close`) and `dict` (one-shot, nothing to close).

5.8.2 Recipes

5.8.2.1 Open a database and query rows

```
const conn = await db.sqlite.open(":memory:");

await conn.exec(`
  CREATE TABLE users (
    id    INTEGER PRIMARY KEY,
    name  TEXT NOT NULL,
    age   INTEGER
  )
`);
await conn.exec("INSERT INTO users (name, age) VALUES (?, ?)", "Alice", 30);
await conn.exec("INSERT INTO users (name, age) VALUES (?, ?)", "Bob", 27);

const rows = await conn.query("SELECT name, age FROM users ORDER BY age DESC");
for (const r of rows) runtime.log(r.name, r.age);

// queryValue is a shortcut for a single scalar.
runtime.log("user count:", await conn.queryValue("SELECT count(*) FROM users"));

await conn.close();
```

`query()` resolves to an array of row objects keyed by column name; `queryValue()` resolves to the first column of the first row (or `null` when nothing matches). Always `close()` — there is no finalizer. Signatures: §17.6.10.1 (`db.sqlite.open`). runnable: `examples/scripts/sqlite.ts`

5.8.2.2 Insert with parameterized statements

```
// Bind params as ? placeholders — never string-concatenate values into SQL.
await conn.exec(
  "INSERT INTO users (name, age, email) VALUES (?, ?, ?)",
  "Alice", 30, "alice@example.com",
);

// Uint8Array round-trips as a BLOB column.
await conn.exec("CREATE TABLE files (name TEXT, data BLOB)");
await conn.exec(
  "INSERT INTO files (name, data) VALUES (?, ?)",
  "logo.png", new Uint8Array([0x89, 0x50, 0x4e, 0x47]),
);
const data = await conn.queryValue("SELECT data FROM files WHERE name = ?",
"logo.png");
```

Mixed types are fine — goja exports JS numbers/strings/ `null` / `Uint8Array` straight through to the driver; `exec()` resolves to `{ rowsAffected, lastInsertId }`, and a `BLOB` column round-trips back out as a `Uint8Array`. Signatures: §17.6.10.1 (`db.sqlite.open`). runnable: `examples/scripts/sqlite.ts`

5.8.2.3 Run a transaction

```
const tx = await conn.begin();
await tx.exec("INSERT INTO users (name, age) VALUES (?, ?)", "Dave", 50);
await tx.exec("INSERT INTO users (name, age) VALUES (?, ?)", "Eve", 22);
await tx.commit(); // both rows become visible atomically

// A constraint violation (or any thrown error) should be followed by a rollback.
const tx2 = await conn.begin();
try {
  await tx2.exec("INSERT INTO users (id, name) VALUES (?, ?)", 1, "duplicate-id");
} catch (e) {
  runtime.log("constraint caught:", String(e).slice(0, 60));
}
await tx2.rollback();
```

`begin()` returns a handle with the same `exec/query/queryValue` plus `commit()` / `rollback()` — every `begin()` must reach exactly one of the two, or the connection leaks. Bulk writes inside a transaction (looping `tx.exec(...)` over a batch, as in the ETL recipe below) is the standard shape for a batch load. Signatures: §17.6.10.1 (`db.sqlite.open`). runnable: `examples/scripts/sqlite.ts`

5.8.2.4 Apply a simple migration

```
await conn.exec(`CREATE TABLE IF NOT EXISTS schema_version (version INTEGER NOT NULL)`);
if (Number(await conn.queryValue("SELECT count(*) FROM schema_version")) === 0) {
  await conn.exec("INSERT INTO schema_version (version) VALUES (0)");
}

const migrations = [
  { to: 1, up: async () => {
    await conn.exec(`CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT NOT NULL)`);
  }},
  { to: 2, up: async () => {
    await conn.exec(`ALTER TABLE customers ADD COLUMN tier TEXT NOT NULL DEFAULT 'standard'`);
  }},
];

async function migrate() {
  let applied = 0;
  for (const m of migrations) {
    const current = Number(await conn.queryValue("SELECT version FROM schema_version"));
    if (current < m.to) {
      await m.up();
      await conn.exec("UPDATE schema_version SET version = ?", m.to);
      applied++;
    }
  }
}
```

```

    }
  }
  return applied; // 0 on a re-run – the hallmark of an idempotent migration
}

```

A `schema_version` table tracks the highest applied migration; the runner applies only migrations greater than the recorded version, so re-running `migrate()` is a no-op.

Signatures: §17.6.10.1 (`db.sqlite.open`). runnable:

`examples/scripts/advanced/sqlite-migration.ts`

5.8.2.5 Sweep an ETL load

```

const conn = await db.sqlite.open(":memory:");
await conn.exec(`
  CREATE TABLE events (
    region TEXT NOT NULL,
    product TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    revenue REAL NOT NULL
  )
`);

// Bulk insert inside a transaction so all writes are atomic.
const tx = await conn.begin();
for (const [region, product, quantity, revenue] of eventData) {
  await tx.exec(
    "INSERT INTO events (region, product, quantity, revenue) VALUES (?, ?, ?, ?)",
    region, product, quantity, revenue,
  );
}
await tx.commit();

// Prepared statement: compile once, reuse for repeated lookups.
const byRegion = await conn.prepare("SELECT count(*) FROM events WHERE region = ?");
for (const r of ["EU", "US", "APAC"]) {
  runtime.log(r, await byRegion.queryValue(r));
}
await byRegion.close();

// Aggregate query – hand the rows to codec.json / codec.xml for export.
const summary = await conn.query(`
  SELECT region, count(*) AS num_orders, sum(quantity) AS total_units,
    round(sum(revenue), 2) AS total_revenue
  FROM events
  GROUP BY region
  ORDER BY region
`);

```

The full script goes on to export `summary` as both JSON (`JSON.stringify`) and XML (`codec.xml.encode`) — a sweep like this is the usual shape for a scheduled load-and-report job: bulk-insert under one transaction, look up via a prepared statement, aggregate with `GROUP BY` , then hand the row objects to a codec. Signatures: §17.6.10.1 (`db.sqlite.open`), §17.3.10 (`codec.xml`). runnable: `examples/scripts/advanced/sqlite-etl.ts`

5.8.2.6 Connect to a networked SQL engine

The five server engines take either an options object (assembled into a URL with credentials escaped) or a driver DSN string used verbatim — pick whichever your config already has.

```
// Options object – host/port/user/password/database (+ engine extras).
const pg = await db.postgres.open({
  host: "db.internal", port: 5432, user: "app",
  password: "s3cr3t", database: "shop", sslmode: "require",
});
// ...equivalently, a libpq DSN used verbatim:
// const pg = await db.postgres.open("postgres://app:s3cr3t@db.internal/shop?
// sslmode=require");

// Placeholders are engine-specific – Postgres uses $1, $2, ...
const rows = await pg.query(
  "SELECT id, email FROM users WHERE region = $1 AND active = $2",
  "EU", true,
);
await pg.close();
```

Everything past the connection is the same `exec/query/queryValue/begin/prepare` vocabulary as the SQLite recipes above; only the placeholder token changes per the table in §5.8.1 — `?` for MySQL/ClickHouse, `@p1` for SQL Server, `:1` for Oracle. **Notes:** for Oracle, `database` is the *service name*; for ClickHouse, `secure: true` selects TLS (port 9440). Signatures: §17.6.8.1 (`db.postgres.open`), §17.6.6.1 (`db.mysql.open`), §17.6.5.1 (`db.mssql.open`), §17.6.1.1 (`db.clickhouse.open`), §17.6.7.1 (`db.oracle.open`).

5.8.2.7 Use Redis or Valkey as a key-value store

One `do(cmd, ...args)` method reaches the entire command surface — strings, counters, hashes, lists — so there's no per-verb binding to learn.

```
const r = await db.redis.open("redis://localhost:6379/0");

// A string with an expiry, and a miss that reads back as null (not an error).
await r.do("SET", "session:42", "active", "EX", 300);
const session = await r.do("GET", "session:42"); // "active"
const missing = await r.do("GET", "session:99"); // null

// Counters, hashes, lists – all through do().
await r.do("INCR", "visits");
await r.do("HSET", "user:42", "name", "Alice", "tier", "gold");
const user = await r.do("HGETALL", "user:42"); //
["name", "Alice", "tier", "gold"]
await r.do("RPUSH", "queue", "job-1", "job-2");

await r.close();
```

Notes: `db.valkey.open("valkey://...")` returns the identical `{ do, ping, close }` handle (the RESP-compatible fork). Replies coerce to strings / numbers / arrays; a nil reply is `null`, while RESP-level errors (`WRONGTYPE`, unknown command) throw. Always `close()`. Signatures: §17.6.9.1 (`db.redis.open`), §17.6.11.1 (`db.valkey.open`).

5.8.2.8 Cache-aside with memcached

get / set / delete with a per-key TTL — the classic read-through cache in front of a slower source of truth.

```
const mc = await db.memcached.open("localhost:11211");

async function getUser(id: number) {
  const key = `user:${id}`;
  const cached = await mc.get(key); // string | null
  if (cached !== null) return JSON.parse(cached);

  const fresh = await loadUserFromDB(id); // your source of truth
  await mc.set(key, JSON.stringify(fresh), 300); // expire after 5 minutes
  return fresh;
}

await mc.delete("user:42"); // invalidate on write
```

Notes: set(key, value, expirySeconds?) — 0 or omitted never expires. get resolves to null on a miss and delete to true / false for hit/miss — both are data, not errors. There is no close (the pool is lazy, GC'd with the handle). Signatures: §17.6.4.1 (db.memcached.open).

5.8.2.9 Inspect and search an LDAP directory

Read the server's Root DSE for metadata, then run a subtree search — each entry comes back as { dn, <attr>: string[] }.

```
const dir = await db.ldap.open("ldap://localhost:389", {
  bindDN: "cn=admin,dc=example,dc=com", // omit opts entirely for an anonymous bind
  password: "s3cr3t",
});

// Server metadata: naming contexts, supported controls, vendor, ...
const meta = await dir.rootDSE();
runtime.log(meta.namingContexts);

// Whole-subtree search; filter defaults to (objectClass=*) when omitted.
const people = await dir.search(
  "ou=people,dc=example,dc=com",
  "(mail=*@example.com)",
  ["cn", "mail"],
);
for (const p of people) runtime.log(p.dn, p.cn?.[0], p.mail?.[0]);

await dir.close();
```

Notes: this is a read-only inspector — there's no modify/write surface. Attribute values are always arrays, even when single-valued (hence p.cn?.[0]). Signatures: §17.6.3.1 (db.ldap.open).

5.8.2.10 Define and match words over DICT

A one-shot RFC 2229 client: `define` fetches definitions, `match` finds headwords by strategy. Each call connects, queries, and disconnects.

```
// Definition lookup — "no match" is data (found:false), not an error.
const def = await db.dict.define("dict.org", "serendipity");
if (def.found) {
  for (const d of def.definitions) runtime.log(`[${d.dbName}] ${d.text}`);
}

// Prefix match across every dictionary.
const hits = await db.dict.match("dict.org", "seren", { strategy: "prefix" });
runtime.log(hits.matches.map(m => m.word));
```

Notes: no handle to close — each call is self-contained. `define` resolves to { word, found, definitions[] } and `match` to { word, matches[] } `opts` covers database (default "*" = all), port (default "2628"), strategy (match only, default "prefix"), and timeout ms. Signatures: §17.6.2.1 (db.dict.define), §17.6.2.2 (db.dict.match).

5.9 services

Subprocess and external-CLI / service wrappers (was `tools` in v0.8.0). Members:

- `services.exec.shell(cmd, opts?)` — run a shell command; captures stdout/stderr or streams into a `tui` pane when `opts.pane` is set.
- `services.exec.stream(cmd, onLine, opts?)` — like `exec.shell` but streams the subprocess's output to `onLine(line, stream)` **line by line as it arrives** instead of buffering it. `cmd` and `opts` (`cwd` / `env` / `stdin` / `timeout`) match `exec.shell`, except `timeout` has **no default** (0 / absent = run until exit), since streaming targets long-running output. `stream` is "stdout" or "stderr". Returns a Promise resolving to { `exitCode`, `success`, `durationMs` } on exit; a non-zero exit resolves with `success: false`, while spawn failures and timeouts reject. Useful for processing large or incremental output without holding it all in memory.
- `services.exec.interactive(cmd, opts?)` — run a subprocess wired to `sercon`'s **own terminal** — the interactive counterpart to `exec.shell`. Where `shell` captures stdout/stderr, `interactive` inherits stdin/stdout/stderr so genuinely interactive children work: `docker exec -it, ssh, interactive mysql / redis-cli` REPLs, pagers, and full-screen TUIs. On Unix with a real TTY it allocates a pty and switches the terminal to **raw mode** (restored on every exit path), forwarding `SIGWINCH` resizes; on a non-TTY stdin (pipes, CI) or on Windows it inherits the raw handles with no pty. `opts` is { `cwd?`, `env?`, `timeout?` } — `timeout` has **no default** (0 / absent = run until the child exits). Nothing is captured (the child owns the terminal), so it resolves to just { `exitCode`, `success`, `durationMs` } a non-zero exit resolves with `success: false`, while spawn failures and timeouts throw. See `examples/scripts/exec-interactive.ts` (TTY-only; run it in a real terminal).
- `services.exec.http(method, url, opts?)` — shell-level `curl` -style HTTP, separate from `net.http` and intended for raw protocol use. It shells out to **recon first, then curl** (`opts.backend` = "auto" | "recon" | "curl" picks; "auto" is the default). 4xx/5xx resolve as a `status` only transport errors and timeouts throw. The request body is sent via a temp file + `--data-binary` so CR/LF survive intact, and `opts.{headers, timeout, follow, insecure}` map onto the underlying flags. Resolves

`{ status, headers, body, durationMs, backend }` (header names lower-cased, `backend` = whichever tool ran).

The `services.git.*` wrappers shell out to the local `git` binary and each accept `{ cwd }` to select the checkout (defaulting to the engine's working directory):

- `git.branch()` → `{ current, detached, all }` — current branch ("" when detached), the detached flag, and every local branch.
- `git.isClean()` → `boolean` — true when `git status --porcelain` is empty.
- `git.status()` → `Array<{ path, indexStatus, workingStatus }>` — parsed porcelain v1 entries (empty array = clean tree).
- `git.revParse(rev)` → the full 40-char SHA (`rev` is any ref/HEAD/short SHA; an unresolvable rev throws git's own message).
- `git.add(paths)` / `git.commit(message, { allowEmpty? })` — stage one or many paths (passed after `--`), then commit (`allowEmpty` adds `--allow-empty`); commit returns `{ sha }`.
- `git.log({ limit?, revRange? })` → newest-first `{ sha, shortSha, author, email, timestamp, subject }[]` (default limit 50, default range `HEAD~1..HEAD`).
- `git.diffStat({ revRange? })` → `{ files, insertions, deletions }` (default range `HEAD~1..HEAD`).
- `git.runText(args, { cwd? })` — escape hatch: run any `git <args>` and get `{ stdout, stderr, exitCode }` the exit code is **data**, so a non-zero status does not throw.

Every `services.git.*` call is bounded by a fixed 30-second subprocess timeout (there's no `opts.timeout` override on these positional bindings); a `git` process that hangs past it is killed and the call rejects instead of hanging the run — this matters most under `sercon serve`, where the Run's own `--timeout` is disabled.

The `services.gh.{authStatus, prList, repoView}` wrappers are thin `gh` CLI bindings. `authStatus()` is deliberately non-throwing — a missing `gh` or an unauthenticated session resolves with `{ authenticated: false, ... }` (only context cancellation throws), so it is safe to probe unconditionally. `prList({ cwd?, state?, limit?, author? })` returns one object per PR with `author` flattened from `gh's { login }` wrapper and ISO-8601 `createdAt` / `updatedAt` `repoView(repo?, { cwd? })` returns repo metadata with `owner` / `defaultBranch` pre-flattened (omit `repo` for the `cwd`'s repo, or pass `"owner/name"` for any repo `gh` can see). Like `services.git.*`, every `gh` invocation is bounded by a fixed 30-second subprocess timeout — a hung `gh` process is killed rather than left to hang the run.

The `services.ai.{providers, send}` client is provider-agnostic. `providers()` is **synchronous** and returns the subset of `claude` / `codex` / `copilot` / `gemini` found on `PATH` in preference order (empty array when none).

`send({ prompt, provider?, system?, context?, timeout? })` runs a one-shot prompt — when `provider` is omitted the first provider on `PATH` is chosen; `system` / `context` are prepended as `System:` / `Context:` blocks (a portable substitute for each CLI's own flags). It resolves `{ provider, output, exitCode }` a non-zero exit is **data**, not a throw (only a missing prompt, no provider on `PATH`, an unknown provider, or a timeout throw).

- `services.agentBrowser.*` — headless-Chrome automation via the `agent-browser` CLI. See the subsection below.

- `services.webdriver.*` — W3C WebDriver client (via `tebeka/selenium`). See the subsection below.
- `services.typst.*` — Typst typesetting via the external `typst` CLI. See the subsection below.
- `services.pdf.*` — PDF render/extract via the external `poppler-utils` CLI tools. See the subsection below.

5.9.1 `services.agentBrowser`

Bridge to the `agent-browser` headless-Chrome CLI. Every call shells out to `agent-browser --json --session \<name\> \<command\>` and returns the parsed JSON response.

Availability gate. `services.agentBrowser.available` is a sync boolean — check it before calling anything else. Every binding throws a clean error if the CLI is absent.

Launch model. `services.agentBrowser.launch(opts?)` is **synchronous**; no browser starts until the first command. It returns a handle whose I/O methods are all async (Promises). Sessions the script doesn't `close()` are best-effort closed when the Run ends.

Launch options (all optional):

Key	CLI flag	Description
<code>session</code>	<code>--session</code>	Name the session (auto-generated when omitted).
<code>headed</code>	<code>--headed</code>	Run with visible browser window.
<code>profile</code>	<code>--profile</code>	Browser profile directory.
<code>proxy</code>	<code>--proxy</code>	Proxy URL.
<code>userAgent</code>	<code>--user-agent</code>	Custom user-agent string.
<code>device</code>	<code>--device</code>	Emulate a device (e.g. "iPhone 12").
<code>colorScheme</code>	<code>--color-scheme</code>	"light" or "dark".
<code>ignoreHttpsErrors</code>	<code>--ignore-https-errors</code>	Skip TLS verification.
<code>engine</code>	<code>--engine</code>	Browser engine to use.
<code>executablePath</code>	<code>--executable-path</code>	Path to the browser binary.
<code>enable</code>	<code>--enable</code>	Enable a browser feature flag.
<code>args</code>	<code>--args</code>	Extra browser arguments.
<code>timeout</code>	<i>(sercon)</i>	Per-call subprocess timeout in ms (default 30 000, <code>0</code> disables). When the limit is reached the Promise rejects with a clear "timed out" message instead of hanging. Session <code>close()</code> is independently bounded at 10 s regardless of this value.

Response envelope. Every async method returns a parsed JSON object with the envelope `{ success: boolean, data: {...}, error: string | null }`. Drill into `.data` for the actual values (e.g. `data.title`, `data.value`, `data.visible`).

Handle method groups:

- **Navigation:** `open(url)`, `back()`, `forward()`, `reload()`, `wait(msOrSelector)`, `connect(portOrUrl)`
- **Interaction:** `click(sel)`, `dblclick(sel)`, `hover(sel)`, `focus(sel)`, `fill(sel, text)`, `type(sel, text)`, `press(key)`, `check(sel)`, `uncheck(sel)`, `select(sel, ...values)`, `scroll(dir, px?)`, `scrollIntoView(sel)`, `drag(src, dst)`, `upload(sel, files)`, `download(sel, path)`, `keyboard.{type, insertText}`, `mouse.{move, down, up, wheel}`
- **Inspection:** `get(what, sel?)`, `isVisible(sel)`, `isEnabled(sel)`, `isChecked(sel)`, `eval(jsCode)`, `snapshot(opts?)`, `console(opts?)`, `errors(opts?)`, `highlight(sel)`
- **Locators:** `find(locator, value, { action, text? })`, `locator(spec)` — returns a chainable handle bound to a `(locator, value)` spec.
- **Settings (Phase 2):** `set.viewport(w,h,scale?)`, `set.device(name)`, `set.geo(lat,lng)`, `set.offline(on?)`, `set.headers(obj)`, `set.credentials(user,pass)`, `set.media(scheme?,reducedMotion?)`
- **Recording (Phase 2):** `record.start(path.webm, url?)`, `record.stop()`
- **Capture (Phase 2):** `screenshot(path?,opts?)`, `pdf(path?)`
- **Network (Phase 3):** `network.route(url, opts?)`, `network.unroute(url?)`, `network.requests(opts?)`, `network.request(id)`, `network.har.start(path?)`, `network.har.stop(path?)`
- **Cookies (Phase 3):** `cookies.get()`, `cookies.set(name, value, opts?)`, `cookies.clear()`
- **Storage (Phase 3):** `storage.local.get(key?)`, `storage.local.set(key, value)`, `storage.local.clear()` and matching `storage.session.*`
- **Tabs (Phase 3):** `tabs.list()`, `tabs.new(url?, opts?)`, `tabs.close(ref?)`, `tabs.select(ref)`
- **Diff (Phase 3):** `diff.snapshot(opts?)`, `diff.screenshot(opts)`, `diff.url(url1, url2)`
- **Debug/Perf (Phase 4):** `trace.start()`, `trace.stop(path?)`, `profiler.start(opts?)`, `profiler.stop(path?)`, `inspect()`, `clipboard(op, text?)`, `vitals(url?)`, `pushstate(url)`
- **React DevTools (Phase 4):** `react.tree()`, `react.inspect(id)`, `react.renderers.start()`, `react.renderers.stop()`, `react.suspense(opts?)` — requires `launch({ enable: "react-devtools" })`
- **Streaming (Phase 4):** `stream.enable(opts?)`, `stream.disable()`, `stream.status()`
- **AI (Phase 4):** `chat(message, opts?)` — requires an AI gateway configured in agent-browser
- **Escape hatch (Phase 4):** `cmd(command, ...args)`, `batch(cmds, opts?)`
- **Auth login (Phase 4):** `auth.login(name)` — uses a saved auth profile on the current session
- **Lifecycle:** `session (read-only name)`, `close()`

Argument vocabularies (the agent-browser surface). The string arguments to `get`, the `is*` checks, and `find` are passed through to the `agent-browser` CLI verbatim, so the accepted values are `agent-browser`'s, not `sercon`'s. The set below is current for **agent-browser 0.27.1** (the version this manual was written against — see the authoritative-reference note at the end of this section):

- **get(what, sel?)** — `what` is one of `text`, `html`, `value`, `attr` (the attribute name goes in the `sel` slot, e.g. `get("attr", "href")`), `title`, `url`, `count`, `box` (bounding box), `styles`, or `cdp-url`. When `sel` is a plain CSS selector it scopes the query; a `snapshot @ref` (see below) targets a specific element; omit it to query the page or the active

element. The result is in the `.data` field of the envelope (e.g.

```
(await b.get("title")).data.title).
```

- **isVisible(sel) / isEnabled(sel) / isChecked(sel)** — map to agent-browser's `is visible|enabled|checked <selector>`. `sel` is required. The boolean lands in `.data` (e.g. `.data.visible`).
- **find(locator, value, { action, text? })** — `locator` is one of `role`, `text`, `label`, `placeholder`, `alt`, `title`, `testid`, `first`, `last`, or `nth` `value` is the locator's argument (the role/text/etc. to match). `action` is **required** and is an act-style verb — `click`, `dblclick`, `hover`, `focus`, `check`, `uncheck`, `fill`, or `type` (the last two consume text). agent-browser's `find` cannot return a handle without acting, so there is no read-only `find` for read-only matching use `snapshot()`. The chainable `locator(spec)` handle exposes the same verbs as methods.
- **snapshot(opts?)** — `opts` is `{ interactive?: boolean, compact?: boolean, depth?: number, selector?: string }`, mapping to agent-browser's `-i / -c / -d <n> / -s <sel>`. The snapshot is an accessibility tree whose interactive nodes carry refs like `@e2` **those refs are valid selectors** for `get`, `click`, `fill`, etc. on the same session — the snapshot → act-by-ref loop is agent-browser's intended workflow for AI agents.
- **console(opts?) / errors(opts?)** — `opts` is `{ clear?: boolean }` (passes `--clear`); returns captured console logs / page errors.

Reaching the rest of agent-browser. `sercon` binds the commonly-used subset; agent-browser ships more (sessions, dashboard, confirmation prompts, iOS-simulator and cloud-provider backends, init scripts, ...). Any subcommand that isn't a first-class binding is reachable through the `cmd(command, ...args)` escape hatch — `cmd` maps 1:1 onto an `agent-browser \<command> [args]` invocation. For the **authoritative, version-matched** command and flag reference — including data shapes returned in `.data`, which evolve with the CLI — run `agent-browser --help`, or the agent-oriented guide `agent-browser skills get core --full`. Treat the tables in this manual as a convenience snapshot, not a contract: agent-browser owns its own surface.

Capture (Phase 2). `screenshot` and `pdf` are path-first with opt-in in-memory bytes:

- `b.screenshot(path?, opts?)` — when `path` is given the image is written to that file and the result is `{ path: string, size: number, format: string }`. When omitted the raw bytes are returned as `{ bytes: number[], format: string }` — a plain JS `number[]` (byte-value array). Convert to a typed array with `new Uint8Array(bytes)`.
- `b.pdf(path?)` — same path-first model; format is always `"pdf"`.
- `opts` for `screenshot`:
`{ selector?, full?, annotate?, format?: "png"|"jpeg", quality? }`.

Defaults bag (Phase 2). A per-Run namespace-level defaults object is merged under per-call `launch()` `opts` so common flags don't need to be repeated:

```
services.agentBrowser.setDefaultOptions({ headed: false, proxy: "http://proxy:3128" });
const b = services.agentBrowser.launch(); // inherits headed + proxy
```

- `setDefaultOptions(obj)` — replace the defaults map entirely.
- `defaultOptions()` — return a shallow copy of the current defaults.

- `clearDefaultOptions()` — reset to `{}`.

Defaults are scoped to the current Run; they do not persist across `Run` calls.

Flat one-shot shortcuts (Phase 2). Launch an ephemeral session, open a URL, perform one action, and close — without managing a handle:

Shortcut	Equivalent handle sequence
<code>agentBrowser.screenshot(url, path?, opts?)</code>	<code>launch()+open(url)+screenshot(path?,opts?)+close()</code>
<code>agentBrowser.pdf(url, path?)</code>	<code>launch()+open(url)+pdf(path?)+close()</code>
<code>agentBrowser.snapshot(url, opts?)</code>	<code>launch()+open(url)+snapshot(opts?)+close()</code>
<code>agentBrowser.eval(url, js)</code>	<code>launch()+open(url)+eval(js)+close()</code>

Example (Phase 1 + Phase 2):

```
if (!services.agentBrowser.available) {
  runtime.log("install agent-browser first");
} else {
  const html = "data:text/html," + encodeURIComponent(
    "<title>sercon demo</title><h1 id=hi>Hello</h1><input id=box>"
  );
  const b = services.agentBrowser.launch({ headed: false });
  try {
    await b.open(html);

    const titleRes = await b.get("title");
    runtime.log("title:", titleRes.data?.title); // "sercon demo"

    await b.fill("#box", "typed by sercon");
    const valRes = await b.get("value", "#box");
    runtime.log("value:", valRes.data?.value); // "typed by sercon"

    const visRes = await b.isVisible("#hi");
    runtime.log("visible:", visRes.data?.visible); // true

    const snap = await b.snapshot({ compact: true });
    runtime.log("snap keys:", Object.keys(snap).join(", "));

    // Phase 2 — settings + capture
    await b.setViewport(1280, 800);
    const shot = await b.screenshot({ full: true }); // bytes path
    runtime.log("bytes:", new Uint8Array(shot.bytes).length, shot.format);

    const file = await b.screenshot("/tmp/demo.png"); // file path
    runtime.log("file:", JSON.stringify(file)); // { path, size, format }
  } finally {
    await b.close();
  }

  // One-shot shortcut
  const r = await services.agentBrowser.eval(html, "document.title");
  runtime.log("title:", r.data?.result); // "sercon demo"
}
```

Phase 3: network, cookies, storage, tabs, diff.

All Phase-3 groups are handle methods (operate on an open session) and return the standard { success, data, error } envelope.

Network interception (network.*).

```
// Block all XHR requests matching a glob pattern.
await b.network.route("**/api/*", { abort: true });

// Stub a JSON response body instead of blocking.
await b.network.route("**/data.json", { body: { mock: true } });

// Remove a specific route (or all routes when url is omitted).
await b.network.unroute("**/api/*");

// Inspect captured requests.
const reqs = await b.network.requests({ clear: true, method: "POST" });
runtime.log(JSON.stringify(reqs.data));

// Get a single request by id.
const req = await b.network.request("req-1");

// Record a HAR trace.
await b.network.har.start("/tmp/trace.har");
// ... navigate and interact ...
await b.network.har.stop();
```

Route opts: abort?: boolean, body?: unknown (JSON-serialised), resourceType?: string (e.g. "xhr").

Requests opts: clear?: boolean, filter?: string, type?: string, method?: string, status?: string.

Cookie management (cookies.*).

```
// Read the current cookie jar.
const jar = await b.cookies.get();
runtime.log(jar.data?.cookies); // array of cookie objects

// Set a cookie (requires a real HTTP origin – not available on data: URLs).
await b.cookies.set("session_id", "abc123", {
  domain: ".example.com",
  httpOnly: true,
  secure: true,
  sameSite: "Lax",
  expires: Math.floor(Date.now() / 1000) + 3600, // Unix seconds
});

// Clear all cookies.
await b.cookies.clear();
```

cookies.set opts: url?, domain?, path?, httpOnly?, secure?, sameSite?: "Strict" | "Lax" | "None", expires? (Unix seconds).

> **Note:** `cookies.set` and `storage.*` require a real HTTP origin. > They will fail on `data:` URLs — wrap in `try-catch` when testing with `> data:` URLs.

Web storage (`storage.*`).

```
// localStorage round-trip (requires a real HTTP origin).
await b.storage.local.set("theme", "dark");
const val = await b.storage.local.get("theme");
runtime.log(val.data);           // { theme: "dark" } or similar
await b.storage.local.clear();

// sessionStorage works identically under storage.session.
await b.storage.session.set("token", "xyz");
const tok = await b.storage.session.get("token");
await b.storage.session.clear();
```

`get(key?)` — omit `key` to get all entries; pass a key to get one.

Tab management (`tabs.*`).

```
// Open a new tab (optionally at a URL, with an optional label).
await b.tabs.new("https://docs.example.com", { label: "docs" });

// List all open tabs. Refs are t1/t2/... or the user-supplied label.
const list = await b.tabs.list();
runtime.log(list.data?.tabs); // [{ tabId, title, url, active, label, type }]

// Switch the active tab.
await b.tabs.select("docs"); // by label
await b.tabs.select("t1");   // by ref

// Close a specific tab (or the current tab when ref is omitted).
await b.tabs.close("docs");
```

Page diffing (`diff.*`).

```
// Snapshot the current DOM state (baseline snapshot when no -b given).
const snap = await b.diff.snapshot();

// Compare against a saved baseline snapshot.
const delta = await b.diff.snapshot({ baseline: "/tmp/base.json", compact:
true });
runtime.log(delta.data);

// Pixel-level screenshot comparison.
const px = await b.diff.screenshot({
  baseline: "/tmp/base.png",
  output: "/tmp/diff.png",
  threshold: 0.1,           // 0–1; pixels above this are flagged
});

// Compare two URLs side-by-side without an open session.
const cmp = await b.diff.url("https://staging.example.com", "https://prod.example.
com");
```

`diff.snapshot` opts: `baseline?: string` (path to a prior snapshot JSON),
`selector?: string` (scope to a subtree), `compact?: boolean`, `depth?: number`.

Phase 4: debug/perf, React DevTools, streaming, AI chat, escape hatch, auth vault.

All Phase-4 groups are handle methods (operate on an open session) unless noted. They return the standard `{ success, data, error }` envelope; `batch` returns an array of per-command envelopes.

Debug / perf (`trace.*`, `profiler.*`, `inspect`, `clipboard`, `vitals`, `pushstate`).

```
// Chrome DevTools tracing.
await b.trace.start();
// ... navigate and interact ...
await b.trace.stop("/tmp/trace.json");

// V8 CPU profiling (optional category filter).
await b.profiler.start({ categories: "v8,blink" });
await b.profiler.stop("/tmp/profile.json");

// Open the Chrome DevTools inspector and get the DevTools URL.
const devtools = await b.inspect();
runtime.log("DevTools:", devtools.data?.url);

// Clipboard read/write.
await b.clipboard("write", "copied text");
const clip = await b.clipboard("read");
runtime.log(clip.data?.text);

// Core Web Vitals for the current page (or a given URL).
const v = await b.vitals();
runtime.log("LCP:", v.data?.lcp);

// SPA client-side navigation without a full page reload.
await b.pushstate("/app/dashboard");
```

`profiler.start` opts: `{ categories?: string }` — a comma-separated list of V8/Blink profiling categories.

React DevTools (`react.*`).

Requires the session to have been launched with `launch({ enable: "react-devtools" })`. `agent-browser` returns a clear error (surfaced as a throw) when `react-devtools` was not enabled at launch time.

```
const b = services.agentBrowser.launch({ enable: "react-devtools" });
await b.open("https://my-react-app.example.com");

// Get the component tree.
const tree = await b.react.tree();
runtime.log(JSON.stringify(tree.data).slice(0, 200));

// Inspect a specific fiber node by id (id from tree.data).
const node = await b.react.inspect("fiber-123");
```



```
// Measure re-renders.
await b.react.renders.start();
// ... interact ...
const renders = await b.react.renders.stop();
runtime.log(JSON.stringify(renders.data));

// List Suspense boundaries (optionally only dynamic ones).
const suspense = await b.react.suspense({ onlyDynamic: true });
```

`react.suspense` opts: { `onlyDynamic?: boolean` } — when true, only boundaries that have actually suspended are included.

Live streaming (`stream.*`).

```
// Enable a browser-event stream on a chosen port.
await b.stream.enable({ port: 9229 });

const status = await b.stream.status();
runtime.log("streaming:", status.data?.enabled);

await b.stream.disable();
```

`stream.enable` opts: { `port?: number` } — selects the streaming port (agent-browser picks a default when omitted).

AI chat (`chat`).

Requires an AI gateway to be configured in agent-browser; errors cleanly if not configured.

```
// Single-shot natural-language instruction.
const r = await b.chat("Click the Login button");
runtime.log("chat ok:", r.success);

// Optionally specify the model.
const r2 = await b.chat("Fill out the form with test data", { model: "gpt-4o" });
```

Escape hatch (`cmd` / `batch`).

`cmd` is a generic passthrough for any agent-browser subcommand. `batch` runs multiple commands in a single round-trip and returns an **array** of per-command result envelopes (not the usual single envelope).

```
// cmd: call any agent-browser subcommand.
const title = await b.cmd("get", "title");
runtime.log("title:", title.data?.title);

// batch: multiple commands, one round-trip.
const results = await b.batch(["get title", "get url"], { bail: false });
for (const r of results) {
  runtime.log(r.success, JSON.stringify(r.data));
}
```

`batch` opts: { `bail?: boolean` } — stop at the first failing command and return results up to that point.

Auth vault — namespace (auth.save / list / show / delete) + handle (auth.login).

Vault CRUD is session-independent and lives on the namespace

(services.agentBrowser.auth.*). login fills a form on a live page and is a handle method (b.auth.login(name)).

Passwords are never placed in argv. auth.save feeds the password through the subprocess stdin using --password-stdin . This means the password never appears in ps output or shell history.

```
// Save a login profile (password is sent via stdin, not argv).
await services.agentBrowser.auth.save("prod", {
  url: "https://app.example.com/login",
  username: "admin",
  password: "s3cret",           // sent via stdin
  usernameSelector: "#user",
  passwordSelector: "#pass",
  submitSelector: "#submit",
});

// List all saved profiles (returns an envelope; data contains profile names).
const list = await services.agentBrowser.auth.list();
runtime.log(JSON.stringify(list.data));

// Show details of a saved profile.
const info = await services.agentBrowser.auth.show("prod");

// Delete a profile.
await services.agentBrowser.auth.delete("prod");

// Log in using a saved profile (handle method — requires an open session).
const b = services.agentBrowser.launch();
await b.open("https://app.example.com/login");
await b.auth.login("prod");    // fills the form and submits
runtime.log("logged in");
await b.close();
```

auth.save opts:

```
{ url, username, password, usernameSelector?, passwordSelector?, submitSelector? }.
```

All string opts except password map to the corresponding agent-browser flags; password is stripped from the option object and fed via stdin.

5.9.2 services.webdriver

W3C WebDriver client backed by github.com/tebeka/selenium (pure Go — no cgo). Drives a real browser via an installed chromedriver or geckodriver no driver binary is bundled or downloaded.

Availability gate. services.webdriver.available is a sync boolean — true when chromedriver or geckodriver is found on PATH. Gate all calls on this value.

Connect-or-start model. connect(opts?) is async. When opts.url is given, it dials the running driver endpoint at that URL. When opts.url is omitted, it locates the driver binary on PATH, picks an ephemeral port, starts the driver as a subprocess, and dials it. The resolved session handle has the same surface in both cases. Sessions quit (and any started

subprocess is stopped) when the Run ends; call `session.quit()` explicitly for deterministic teardown. Both the quit and the subprocess stop are bounded by a fixed 10-second timeout each, so a wedged driver process can't hang the call (or the Run) indefinitely — teardown proceeds best-effort past that point.

Probe. `probe({ url })` checks whether a WebDriver endpoint is ready without opening a session. Returns `{ ready: boolean, status?: number }` on HTTP success or `{ ready: false, error: string }` on transport failure; never throws on network errors.

Connect options (all optional):

Key	Type	Default	Description
<code>browser</code>	<code>"chrome" "firefox"</code>	<code>"chrome"</code>	Selects the driver binary (<code>chromedriver</code> or <code>geckodriver</code>).
<code>headless</code>	<code>boolean</code>	<code>true</code>	Run the browser in headless mode.
<code>url</code>	<code>string</code>	—	Dial an already-running WebDriver server instead of starting one.
<code>args</code>	<code>string[]</code>	—	Extra browser command-line flags appended to the default set.
<code>capabilities</code>	<code>object</code>	—	Raw W3C capability overrides merged last (escape hatch).
<code>commandTimeout</code>	<code>number</code>	<code>30000</code>	Per low-level WebDriver request timeout (ms). Bounds each command so a driver blocked behind an open alert or an unreachable endpoint can't hang the call. <code>0</code> /negative disables it. <code>quit()</code> (and Run-end cleanup) also cancels any in-flight command.

Browser flags (`args`) and capabilities (the WebDriver surface). Unlike `agentBrowser`, this binding speaks the W3C WebDriver protocol directly (via `tebeka/selenium`) rather than wrapping a CLI, so the “external surface” is the W3C capability set and the browser's own command-line flags:

- **`args`** are the browser's launch flags, appended *after* the headless flag `sercon` adds for you (`--headless=new` for Chrome, `-headless` for Firefox when `headless` is true). They land in the vendor capability block (`goog:chromeOptions.args` / `moz:firefoxOptions.args`). Common Chrome flags: `--no-sandbox`, `--disable-gpu`, `--disable-dev-shm-usage`, `--window-size=1280,800`, `--lang=en-US`, `--proxy-server=host:port`. Firefox takes `-width 1280`, `-height 800`, `-private`, etc.
- **`capabilities`** is an object of top-level **W3C standard capabilities**, merged into the session request *last* (so it overrides `sercon`'s defaults). Standard keys: `browserName`, `browserVersion`, `platformName`, `acceptInsecureCerts` (boolean — accept self-signed/expired TLS), `pageLoadStrategy` (`"normal" | "eager" | "none"`), `unhandledPromptBehavior` (`"dismiss" | "accept" | "ignore" | ...`), `timeouts`

({ script, pageLoad, implicit } in ms), and proxy ({ proxyType, httpProxy, sslProxy, ... }). Vendor blocks go here too — goog:chromeOptions and moz:firefoxOptions (each { args?, binary?, prefs?, ... }). Because the merge is by top-level key, supplying goog:chromeOptions wholesale *replaces* the one built from args / headless — set args for additive flags, capabilities only when you need full control. See the W3C WebDriver capabilities spec for the complete list.

Session handle methods (all async):

Method	Returns	Description
get(url)	{ ok: true }	Navigate to a URL.
url()	string	Current URL.
title()	string	Current page title.
back() / forward() / refresh()	{ ok: true }	Browser history controls.
find(by, value)	element handle	Find the first matching element.
findAll(by, value)	element handle[]	Find all matching elements.
source()	string	Current page source HTML.
screenshot(path?)	{ path?, size?, bytes?, format }	Capture the viewport. No path → bytes: number[] with path → { path, size, format }.
executeScript(js, args?)	unknown	Run a synchronous script in the page context.
executeScriptAsync(js, args?)	unknown	Run an async script (callback-resolved).
cookies()	object[]	Get all cookies.
setCookie(c)	{ ok: true }	Set a cookie { name, value, path?, domain?, secure?, (httpOnly?, expiry? }).
deleteCookie(name)	{ ok: true }	Delete a named cookie.
deleteAllCookies()	{ ok: true }	Clear all cookies.
setImplicitWait(ms)	{ ok: true }	Set the implicit element-wait timeout.
waitFor(by, value, opts?)	element handle	Poll in the active frame until an element appears (opts: timeout? ms, visible?: boolean, enabled?: boolean — wait past a disabled→enabled flip).
clickWhenReady(by, value, opts?)	{ ok: true }	Wait in the active frame (default visible + enabled)

		then issue a native (trusted) click that fires React handlers (opts: <code>timeout?</code> , <code>visible?</code> , <code>enabled?</code> , <code>poll?</code> ms).
<code>quit()</code>	<code>{ closed: true }</code>	Close the session (idempotent; also called by Run-end cleanup).

Locator strategies (by argument): `css`, `xpath`, `id`, `name`, `tag`, `className`, `linkText`, `partialLinkText`.

Element handle methods (all async):

Method	Description
<code>click()</code>	Click the element.
<code>sendKeys(text)</code>	Type text into the element.
<code>clear()</code>	Clear the element's value.
<code>submit()</code>	Submit the form.
<code>text()</code>	Inner text content.
<code>getAttribute(name)</code>	Get the named HTML attribute ; returns <code>null</code> when the attribute is absent (W3C semantics). This is the <i>attribute</i> , not the live DOM <i>property</i> — an <code><input></code> 's typed text lives in its <code>value</code> property, not a <code>value</code> attribute, so read it with <code>executeScript</code> (e.g. <code>executeScript("return document.querySelector('#box').value")</code>). Chrome's legacy fallback returns the property; Firefox correctly returns <code>null</code> .
<code>cssValue(name)</code>	Get a computed CSS property value.
<code>tagName()</code>	Tag name in lower case.
<code>isDisplayed()</code> / <code>isEnabled()</code> / <code>isSelected()</code>	Visibility / state checks.
<code>find(by, value)</code> / <code>findAll(by, value)</code>	Sub-tree element search.
<code>screenshot(path?)</code>	Screenshot the element (scrolls into view).

Example:

```
if (!services.webdriver.available) {
  runtime.log("no chromedriver/geckodriver on PATH – skip");
} else {
  const d = await services.webdriver.connect({ browser: "chrome", headless:
true });
  try {
    await d.get("data:text/html," + encodeURIComponent(
```

```

    "<title>wd demo</title><h1 id=hi>Hello</h1><input id=box>"));
runtime.log("title:", await d.title());
const h1 = await d.find("id", "hi");
runtime.log("text:", await h1.text(), "visible:", await h1.isDisplayed());
const box = await d.find("css", "#box");
await box.sendKeys("typed by sercon");
runtime.log("id attr:", await box.getAttribute("id")); // "box"
// Live input value is a DOM property, not an attribute – read via script
// so it works the same on Chrome and Firefox.
runtime.log("value:", await d.executeScript("return
document.querySelector('#box').value", []));
runtime.log("eval:", await d.executeScript("return 6*7", []));
const shot = await d.screenshot();
runtime.log("bytes:", new Uint8Array(shot.bytes).length, shot.format);
} finally {
    await d.quit();
}
}
}

```

Phase 2 (v0.41.0). The session handle gains the following additional async methods.

Phase 2 session methods:

Method	Returns	Description
<code>windowHandles()</code>	<code>string[]</code>	All open window/tab handles.
<code>currentWindow()</code>	<code>string</code>	Handle of the currently focused window/tab.
<code>switchToWindow(handle)</code>	<code>{ ok: true }</code>	Focus a window/tab by its handle.
<code>newWindow(type?)</code>	<code>{ handle, type }</code>	Open a new tab ("tab" , default) or window ("window"). Returns the new handle.
<code>closeWindow()</code>	<code>string[]</code>	Close the current window/tab; returns the remaining handles and auto-switches to a survivor.
<code>switchToFrame(target)</code>	<code>{ ok: true }</code>	Switch into a frame by index (number) or by passing an element handle for the <code><iframe></code> . A name/id string is not accepted — locate the iframe via <code>find</code> and pass the element handle.
<code>switchToParentFrame()</code>	<code>{ ok: true }</code>	Switch to the parent frame.
<code>switchToDefaultContent()</code>	<code>{ ok: true }</code>	Switch back to the top-level browsing context.

<code>acceptAlert()</code>	<code>{ ok: true }</code>	Accept (OK/confirm) the current alert/confirm/prompt dialog.
<code>dismissAlert()</code>	<code>{ ok: true }</code>	Dismiss (cancel) the current dialog.
<code>alertText()</code>	<code>string</code>	Get the message text of the current dialog.
<code>sendAlertText(text)</code>	<code>{ ok: true }</code>	Type into a prompt dialog.
<code>maximize()</code>	<code>{ x, y, width, height }</code>	Maximize the browser window; returns the new rect.
<code>minimize()</code>	<code>{ x, y, width, height }</code>	Minimize the browser window; returns the new rect.
<code>fullscreen()</code>	<code>{ x, y, width, height }</code>	Enter fullscreen; returns the new rect.
<code>setWindowRect({width?,height?,x?,y?,width?,height?})</code>	<code>{ x, y, width, height }</code>	Set position and/or size.
<code>getWindowRect()</code>	<code>{ x, y, width, height }</code>	Get the current window rect.
<code>hover(el)</code>	<code>{ ok: true }</code>	Move the mouse pointer over an element (W3C actions).
<code>dragAndDrop(src, dst)</code>	<code>{ ok: true }</code>	Drag from <code>src</code> element to <code>dst</code> element.
<code>keyChord(...keys)</code>	<code>{ ok: true }</code>	Send a key chord (e.g. <code>keyChord("Control", "a")</code>).
<code>performActions(sequence)</code>	<code>{ ok: true }</code>	Send a raw W3C actions array (escape hatch).
<code>releaseActions()</code>	<code>{ ok: true }</code>	Release any held action state (buttons/keys).

Element handle additions (Phase 2): Every element handle now carries an `elementId` string property (the raw W3C element reference id). Two new async methods are available: `el.hover()` moves the mouse over the element; `el.dragTo(target)` drags the element to a target element handle.

executeScript / executeScriptAsync — element handle return values: When a script returns a single W3C element reference, the binding wraps it in an element handle automatically. A top-level array of element references is also wrapped (each entry becomes an element handle). Nested references (e.g. an object with element-reference values) are returned as plain objects — return them flat or unwrap manually.

Caveat — alert unhandledPromptBehavior : chromedriver's default `unhandledPromptBehavior` is "dismiss and notify", which may auto-dismiss dialogs before your script can interact with them. To inspect alerts manually, pass `capabilities: { unhandledPromptBehavior: "ignore" }` in `connect()`. With "ignore", the alert is kept open but the renderer is blocked by the modal, so any WebDriver command that executes script in the page (including `executeScript`) will hang until the alert is dismissed. Trigger alerts via `<body onload="alert(...) ">` or a synchronous inline script so

that `get()` completes with the dialog already open — then `alertText()` and `acceptAlert()` work without racing against the modal state.

Caveat — file upload: For local `<input type="file">`, use `el.sendKeys("/absolute/path/to/file")` — this works with both Chrome and Firefox. Remote-grid file upload (the `/session/{id}/file` endpoint) is not supported by the underlying `tebeka/selenium` library and is out of scope.

Phase 2 example:

```
if (!services.webdriver.available) {
  runtime.log("no chromedriver/geckodriver on PATH — skip");
} else {
  const d = await services.webdriver.connect({
    browser: "chrome",
    headless: true,
    capabilities: { unhandledPromptBehavior: "ignore" },
  });
  try {
    // Windows / tabs
    const nw = await d.newWindow("tab");
    await d.switchToWindow(nw.handle);
    runtime.log("open tabs:", (await d.windowHandles()).length);
    await d.closeWindow();

    // Window rect
    await d.setWindowRect({ width: 1024, height: 768 });
    const r = await d.getWindowRect();
    runtime.log("rect:", r.width + "x" + r.height);

    // Frames
    await d.get("data:text/html," + encodeURIComponent(
      "<title>outer</title><iframe srcdoc='<p id=inner>hi</p>'></iframe>"));
    await d.switchToFrame(0);
    runtime.log("frame text:", await (await d.find("id", "inner")).text());
    await d.switchToParentFrame();
    await d.switchToDefaultContent();

    // Alerts (requires unhandledPromptBehavior: "ignore")
    // Navigate to a page that fires alert() synchronously on load; get()
    // blocks until the load event, so the dialog is open when it returns.
    try {
      await d.get("data:text/html," + encodeURIComponent(
        "<body onload='alert('hello')'><title>a</title>"));
    } catch (_) {
      // chromedriver may surface an unexpected-alert-open error — the alert
      // is still live, carry on.
    }
    runtime.log("alert:", await d.alertText());
    await d.acceptAlert();

    // Hover + drag
    await d.get("about:blank");
    await d.executeScript(`
      document.body.innerHTML =`
```



```

    '<div id=a
style="position:absolute;top:50px;left:50px;width:80px;height:80px"></div>' +
    '<div id=b
style="position:absolute;top:50px;left:200px;width:80px;height:80px"></div>';
    return 1;`, []);
    const a = await d.find("id", "a");
    const b = await d.find("id", "b");
    await a.hover();
    await d.dragAndDrop(a, b);

    // executeScript returning an element handle
    const el = await d.executeScript("return document.getElementById('b')", []);
    runtime.log("script element tag:", await el.tagName());
  } finally {
    await d.quit();
  }
}

```

5.9.2.1 Frames / iframes

WebDriver — nested + cross-origin. `switchToFrame(target)` accepts a frame index (number), an iframe **element handle** from `find()`, **or a CSS-selector string** (it finds the iframe in the current context and switches). After a switch, element queries are scoped to that frame; `switchToParentFrame()` goes up one level and `switchToDefaultContent()` returns to the top document. `frameChain([t1, t2, ...])` switches from the top document through each level in one call — the ergonomic way to reach a deeply nested frame. This uses the W3C `/frame` protocol, so cross-origin frames (e.g. a Klarna Checkout widget's nested payment iframe) work the same as same-origin ones.

```

await d.frameChain(["#klarna-checkout-iframe", "#klarna-fullscreen-iframe"]);
const confirm = await d.find("css", "button.kco-confirm"); // scoped to the inner
frame
await d.switchToDefaultContent();

```

WebDriver — waiting & reliable clicks. `find` and all element interaction follow the **active frame** set via `switchToFrame` / `frameChain` (W3C frame-scoping — there is no separate frame-context to manage). For buttons that render or enable asynchronously inside a frame (a common pattern in checkout widgets, e.g. a Klarna Checkout “Pay order” that flips disabled→enabled), combine a switch with `clickWhenReady`:

- `waitFor(by, value, opts?)` polls in the active frame until the element is present, and — when requested — `visible` and/or `enabled`. Opts: `timeout?` (ms, default 10000), `visible?: boolean`, `enabled?: boolean` (waits past a disabled→enabled transition). Resolves to the element handle.
- `clickWhenReady(by, value, opts?)` waits — by default both `visible` **and** `enabled` — in the active frame, then issues a **native (trusted)** W3C Element-Click that fires React/synthetic handlers. Opts: `timeout?`, `visible?`, `enabled?` (default `true` for both), and `poll?` (inter-attempt interval in ms). Resolves `{ ok: true }`.

```

await d.frameChain(["#klarna-checkout-iframe", "#payment-frame"]);
await d.clickWhenReady("css", "button.pay", { timeout: 8000 });

```

What made such nested-checkout flows flaky was **timing** (the button rendering or enabling late), not frame context — `find` already follows the chain. `clickWhenReady` is the fix: a wait (visible+enabled, in the active frame) followed by a trusted click.

5.9.3 CDP trusted clicks (Chrome only)

Some elements cannot be activated by the standard WebDriver click: when a button lives inside a **nested cross-origin iframe**, the W3C Element Click hit-tests to the parent

`<iframe>` element and fails with `element click intercepted` (synthetic `executeScript().click()` is untrusted and ignored by frameworks like React). The classic case is completing a Klarna Checkout “Pay order”.

`session.cdpClick(by, value, opts?)` solves this via chromedriver’s CDP passthrough. It locates the element across the **whole pierced frame tree** (no `switchToFrame` needed), reads its true viewport coordinates with `DOM.getContentQuads`, scrolls it into view, and dispatches a **trusted** `Input.dispatchEvent` press/release. `opts`: `timeout` (ms, default 10000), `poll` (ms, default 50), `button` (“left” / “right” / “middle”), `scrollIntoView` (default true), `offsetX` / `offsetY`. Resolves to `{ clicked: true, x, y }`. Throws on firefox.

`session.cdp(command, params?)` is a raw escape hatch that sends a single CDP command (e.g. `Input.dispatchEvent`, `Browser.getVersion`) and returns its result. Chrome-only.

```
const d = await services.webdriver.connect({ browser: "chrome" });
await d.get(checkoutUrl);
await d.cdpClick("xpath", '//button[normalize-space()="Pay order"]', { timeout:
8000 });
```

Out-of-process iframes (OOPIFs). A cross-site iframe (different eTLD+1 — e.g. a Klarna Checkout hosted on another domain) runs in a separate renderer process, which chromedriver’s page-session CDP passthrough cannot reach. `cdpClick` therefore opens a **browser-level CDP connection** (a direct DevTools WebSocket, the way Puppeteer/Playwright do), attaches to the page and every iframe target, locates the element in its owning session, and dispatches the trusted `Input.dispatchEvent` there — browser-level input is hit-tested across OOPIF boundaries. (A different *port* is still same-site and stays in-process; only a different site becomes an OOPIF.)

`session.targets()` lists the browser’s CDP targets (including OOPIF iframe targets); `session.attach(target)` (a target object or `targetId`) returns a `{ targetId, sessionId, cdp(method, params?), detach() }` handle for running CDP against that specific target. These require a CDP-capable session (local chromedriver, or a Grid advertising `se:cdp`); `cdpClick` falls back to the page-session path when browser-level CDP is unavailable.

agentBrowser — single level. `b.frame(target)` switches the session’s frame context to an iframe (CSS selector or `@ref` from snapshot); `b.frame("main")` returns to the top document. Subsequent `click` / `fill` / `get` / `snapshot` operate inside the switched frame. It is **single-level only**: `agent-browser` resolves the selector against the *main* document and cannot descend into nested frames, so sequential `frame()` calls won’t reach an inner frame. For nested or cross-origin nesting, use `WebDriver (frameChain)`. (A Klarna/KCO completion helper is out of scope — it needs the live payment widget.)

5.9.4 services.typst

External-CLI binding to the Typst compiler. Every call shells out to the local `typst` binary; nothing is linked into `sercon` (the `typst-as-lib` Rust embedding can't be used under our no-cgo constraint, so the external CLI is the only viable route — same opt-in, feature-detected model as `services.git` / `services.gh` / `services.ai` / `services.agentBrowser`).

Availability gate. `services.typst.available` is a sync boolean over `exec.LookPath("typst")` — resolved once per Run. Gate on it; every other `typst` binding throws a clean error when the CLI is absent. Install with `brew install typst` (or from `<https://typst.app>`).

Surface.

- `services.typst.available` — `boolean`. True when `typst` is on PATH.
- `services.typst.version()` — `Promise<string>`. The trimmed `typst --version` line.
- `services.typst.fonts()` — `Promise<string[]>`. Font families `typst` can see, de-duplicated and sorted.
- `services.typst.compile(opts)` — `Promise<{ format, bytes?, path? }>`. Compile a document to PDF/PNG/SVG.
- `services.typst.query(opts)` — `Promise<unknown>`. Query a document for elements matching a selector; returns parsed JSON.

Input. `compile` and `query` take exactly **one** of input (a path to a `.typ` file) or `source` (inline Typst markup). Passing both — or neither — throws.

compile output rule. With **no** `output`, a **PDF** is compiled to a temp file and returned as **bytes** (`{ format: "pdf", bytes: Uint8Array }`) — this bytes-return path is **PDF only**. With an `output path`, the file is written there (`{ format, path }`) and `format` is inferred from the extension (`.pdf` / `.png` / `.svg`); **PNG and SVG require an output path**. Other options: `root` (project root for imports), `inputs` (`sys.inputs` key/value pairs → `--input k=v`), `ppi` (PNG resolution), `fontPaths` (extra `--font-path` dirs), `timeout` (ms, default 60000).

Inline-source caveat. When you pass `source`, it is written to a temporary `main.typ` and compiled in that temp directory, so relative `#import` / `read()` paths resolve against the temp dir, **not** your working directory. For documents that pull in sibling files, pass `input` (or set `root`) instead of `source`.

query. `selector` is required (e.g. `"<label>"`, `"heading"`); `field` extracts a single field per match, `one` returns just the first match.

```
if (!services.typst.available) {
  runtime.log("install typst (brew install typst) first");
} else {
  runtime.log("typst:", await services.typst.version());

  // Inline source → PDF bytes (PDF-only without an output path).
  const pdf = await services.typst.compile({ source: "= Hi\nFrom Typst." });
  runtime.log(pdf.format, "bytes:", pdf.bytes.length);

  // Render to PNG on disk (png/svg require an output path).
  await services.typst.compile({ source: "= Hi", output: "/tmp/out.png", ppi:
144 });
```

```
// Query a labelled metadata value as JSON.
const v = await services.typst.query({
  source: "#metadata(42) <answer>",
  selector: "<answer>", field: "value", one: true,
});
runtime.log("answer:", v); // 42
}
```

5.9.5 services.pdf

External-CLI binding to poppler-utils. Every call shells out to the relevant poppler binary (pdftoppm, pdftotext, pdftohtml, pdfinfo) via the shared runTool helper — no shell, no cgo. Follows the same opt-in, feature-detected model as services.typst / services.git / services.agentBrowser.

Availability gate. services.pdf.available is a sync boolean — true when pdftoppm is on PATH. Gate on it; every other pdf binding throws a clean error when the tool is absent. Install with brew install poppler (macOS) or apt install poppler-utils (Debian/Ubuntu).

Granular tool availability. services.pdf.tools exposes per-binary flags (pdftoppm, pdftotext, pdftohtml, pdfinfo) so scripts can adapt when only a subset of poppler is installed.

Surface.

- services.pdf.available — boolean. True when pdftoppm is on PATH.
- services.pdf.backend — string. Always "poppler".
- services.pdf.tools — { pdftoppm, pdftotext, pdftohtml, pdfinfo: boolean }. Per-binary availability flags.
- services.pdf.version() — Promise<string>. The trimmed pdfinfo -v version line.
- services.pdf.info(src) — Promise<{ pages, title?, author?, subject?, creator?, producer?, creationDate?, modDate? }>. Document metadata from pdfinfo.
- services.pdf.toImage(src, opts?) — Promise<{ format, bytes?, path? }>. Render one or more pages to raster images via pdftoppm.
 - **Single-page, no dest:** { page: N } (1-based) returns { format, bytes: Uint8Array } — PNG bytes in memory.
 - **Any other case:** a dest path is required; the file(s) are written there and { format, path } is returned. Omitting dest when rendering multiple pages throws.
 - Options: page (1-based integer), pages (range [first, last]), format ("png" | "jpeg" | "tiff", default "png"), dpi (resolution, default 150), dest (output path), timeout (ms, default 60 000).
- services.pdf.toText(src, opts?) — Promise<string>. Extract plain text via pdftotext. Options: pages ([first, last]), layout (boolean, preserve physical layout), timeout (ms).

- `services.pdf.toHtml(src, opts?)` — `Promise<string>`. Extract HTML via `pdftohtml`. Options: `pages` (`[first, last]`), `noFrames` (boolean), `timeout` (ms).

```
if (!services.pdf.available) {
  runtime.log("install poppler (brew install poppler) first");
} else {
  runtime.log("pdf backend:", services.pdf.backend,
    "| version:", await services.pdf.version());

  const src = "cmd/sercon/testdata/sample.pdf";
  const meta = await services.pdf.info(src);
  runtime.log("pages:", meta.pages);

  // Render page 1 to PNG bytes (single-page + no dest → bytes).
  const img = await services.pdf.toImage(src, { page: 1, format: "png" });
  runtime.log(img.format, "bytes:", img.bytes.length); // e.g. "png bytes: 4459"

  // Render all pages to disk (multi-page requires a dest path).
  await services.pdf.toImage(src, { dest: "/tmp/out.png" });

  // Extract text.
  const txt = await services.pdf.toText(src);
  runtime.log("text snippet:", txt.trim().slice(0, 40));
}
```

5.9.6 services.doctor

`services.doctor(requires?)` reports on every external tool sercon can use and, optionally, asserts a script's prerequisites. It is the script-side form of the `--doctor` CLI flag and returns the same report.

```
const r = await services.doctor(requires?: string[]);
```

Report shape. The promise resolves to:

```
{
  ok: boolean;           // false only when a compatibility conflict was found
  satisfied: boolean;    // true when every `requires` entry is met (unmet === [])
  unmet: string[];       // requested requirements that are absent or conflicted
  tools: {
    name: string;        // binary name, e.g. "chromedriver"
    category: string;    // feature group, e.g. "webdriver"
    purpose: string;     // what sercon uses it for
    installed: boolean;
    version: string | null; // null when not installed or no --version
    ok: boolean;         // false only on a compatibility conflict
    detail?: string;     // e.g. "matches Chrome 149" or the conflict reason
  }[];
}
```

Tools probed include `git`, `gh`, the AI providers (`claude`, `codex`, `copilot`, `gemini`), `agent-browser`, `chromedriver` / `geckodriver`, `typst`, the poppler PDF tools (`pdftoppm`, `pdftotext`, `pdftohtml`, `pdftinfo`), the HTTP backends (`recon`, `curl`), and the platform

clipboard / image tools (pbcopy / pbpaste / osascript / pngpaste on macOS, wl-copy / wl-paste / xclip / xsel on Linux, clip / powershell on Windows).

requires — assert prerequisites. Each entry is either a feature/category name ("webdriver", "typst", "ai", "git", "gh", "agentBrowser", "pdf", "clipboard", "image", "http") or a specific binary name ("chromedriver", "pngpaste", "claude", "pdftoppm", ...). A category is met when at least one tool in it is installed and ok . Anything unmet lands in unmet — **a missing tool is reported, not thrown**, so a script can self-skip an optional feature. An **unknown** requirement name (a typo) throws.

The chromedriver↔Chrome check. When chromedriver is installed, sercon locates the installed Chrome/Chromium and compares the two major versions. A mismatch sets that tool's ok to false (with a detail explaining it), the top-level ok to false , and — for --doctor — exits with code 5 . A missing Chrome leaves ok true and notes that the version couldn't be verified. **Missing is fine; only a conflict fails.**

```
// Assert a hard prerequisite.
const need = await services.doctor(["git"]);
runtime.assert.ok(need.satisfied, "git is required");

// Self-skip an optional feature.
const opt = await services.doctor(["typst", "webdriver"]);
if (!opt.satisfied) runtime.log("skipping; unmet:", JSON.stringify(opt.unmet));
```

5.9.7 Concepts

Every member of services.* wraps a **separately-installed external CLI** — agent-browser , chromedriver / geckodriver , typst , poppler-utils, git , gh , and the AI provider CLIs (claude / codex / copilot / gemini). None of it is linked into the sercon binary; the static, no-cgo binary works with every one of them absent. Treat services as **enrichment, not a dependency** — feature-detect before calling in.

Detection is **not uniform across members**, and getting this right matters:

- services.agentBrowser , services.webdriver , services.typst , and services.pdf each expose their own synchronous **available** boolean (resolved once per Run from exec.LookPath). Check it before calling anything else on that member; every other binding on it throws a clean error when its CLI is missing.
- services.git , services.gh , and services.ai have **no available field of their own** — a missing git / gh binary surfaces as a thrown error from the call itself (except gh.authStatus() , which is deliberately non-throwing and resolves { authenticated: false, ... } instead), and services.ai.providers() just returns an empty array when no AI CLI is on PATH.
- services.doctor(requires?) (§5.9.6) is the **aggregate** surface — it probes every tool above (plus the HTTP backends and platform clipboard/image tools) in one call and can assert prerequisites via requires , so it's the tool of choice when a script depends on several services or wants a single up-front self-skip / hard-fail check instead of scattering available reads.

5.9.8 Recipes

5.9.8.1 Feature-detect a tool before using it

```
// Single-service gate – only agentBrowser/webdriver/typst/pdf have this.
if (!services.typst.available) {
  runtime.log("typst not on PATH – skipping render step");
} else {
  await services.typst.compile({ source: "= Hi", output: "/tmp/out.pdf" });
}

// Aggregate gate – covers everything services.* can shell out to,
// including git/gh/ai, which have no `available` field of their own.
const opt = await services.doctor(["git", "typst", "webdriver"]);
if (!opt.satisfied) {
  runtime.log("missing optional tools:", JSON.stringify(opt.unmet));
}

// Hard-require a prerequisite instead of self-skipping.
const need = await services.doctor(["git"]);
runtime.assert.ok(need.satisfied, "git must be installed");
```

See §17.13.3 (`services.doctor`) for the full report shape.

5.9.8.2 Render a Typst document to a PDF

```
if (!services.typst.available) {
  runtime.log("install typst (brew install typst) first");
} else {
  // typst renders; services.pdf only reads/extracts existing PDFs (poppler),
  // so use it here to sanity-check the render, not to create it.
  const doc = await services.typst.compile({
    source: "= Report\n\nGenerated by sercon.",
    output: "/tmp/report.pdf",
  });
  runtime.log("wrote", doc.path, doc.format);

  if (services.pdf.available) {
    const meta = await services.pdf.info(doc.path);
    runtime.log("pages:", meta.pages);
  }
}
```

See §17.13.8.2 (`services.typst.compile`) and §17.13.7.3 (`services.pdf.info`).

5.9.8.3 Drive a headless browser to screenshot / extract

```
if (!services.webdriver.available) {
  runtime.log("no chromedriver/geckodriver on PATH – skipping webdriver demo.");
} else {
  const d = await services.webdriver.connect({ browser: "chrome", headless:
true });
  try {
    await d.get("data:text/html," + encodeURIComponent(
      "<title>wd demo</title><h1 id=hi>Hello</h1>"
    ));
    runtime.log("title:", await d.title());
  }
}
```



```

const h1 = await d.find("id", "hi");
runtime.log("h1 text:", await h1.text(), "visible:", await h1.isDisplayed());
const shot = await d.screenshot();
runtime.log("screenshot bytes:", new Uint8Array(shot.bytes).length,
shot.format);
} finally {
  await d.quit();
}
}

```

runnable: `examples/scripts/webdriver.ts`

5.9.8.4 Run a git / gh operation

```

// git/gh have no `available` field – a missing binary throws from the call
// itself, except gh.authStatus(), which resolves { authenticated: false }
// instead of throwing.
const branch = await services.git.branch();
const clean = await services.git.isClean();
runtime.log("branch:", branch.current, "clean:", clean);

const auth = await services.gh.authStatus();
if (auth.authenticated) {
  const prs = await services.gh.prList({ state: "open", limit: 5 });
  runtime.log("open PRs:", prs.map((p) => `#${p.number} ${p.title}`));
} else {
  runtime.log("gh not authenticated – skipping PR list");
}

```

See §17.11 (`services.git.*` / `services.gh.*`).

5.10 tui

Multi-pane terminal UI (was `ui.tui` in v0.8.0). `tui.layout(tree)` declares panes;

`tui.pane(name)` returns a pane handle with `write`, `writeln`, `clear`, `title`.

`services.exec.shell({pane})` streams subprocess I/O into a pane.

Scripts that orchestrate multiple subprocesses (e.g. `brew`, `npm`, `cargo` updates running in parallel) can declare a multi-pane terminal layout and route each subprocess's stdout/stderr into its own pane. Activation is **script-driven**: the first call to `tui.layout(...)` enters TUI mode. If stdout is not a TTY (CI, pipes, `make demo`), the same calls fall back to prefixed plain-text lines (`[paneName] line`) so the same script runs in both contexts.

5.10.1 Layout

```

tui.layout({
  rows: [
    { name: "log", title: "Orchestrator", weight: 1 },
    { cols: [
      { name: "brew" },
      { name: "npm" },
    ],
    weight: 2,
  },
},

```



```
    ],
  });
```

Each layout node is one of:

- { name: string, title?: string, weight?: number } — a leaf pane.
- { rows: LayoutNode[], weight?: number } — a vertical split.
- { cols: LayoutNode[], weight?: number } — a horizontal split.

`weight` defaults to 1. Pane names must be unique across the whole tree. `layout` may be called **once** per Run; a second call throws.

5.10.2 Pane handles

```
const log = tui.pane("log");
log.writeln("Updating Homebrew...");
log.title("Done");
log.clear();
log.write("partial line ");
```

`tui.pane(name)` returns a handle with `write`, `writeln`, `clear`, `title`. Methods are synchronous from the script's perspective — they enqueue to the TUI goroutine.

5.10.3 Subprocess routing

`services.exec.shell(cmd, opts)` accepts `opts.pane` (a Pane handle or a pane name string):

```
await services.exec.shell("brew update && brew upgrade", { pane: "brew" });
```

When set, the subprocess's `stdout` **and** `stderr` stream into the pane line by line. The returned { `exitCode`, `durationMs`, `success` } is the same as without `pane`: except `stdout` and `stderr` are empty (data was streamed, not captured). ANSI colors are translated to `tview` color tags and rendered natively; `\r` without `\n` overwrites the current line so progress spinners render cleanly.

Pass { `pty`: `true` } to run the command under a pseudo-terminal (Unix only). The child then believes it is a terminal and emits color, progress bars, and spinners, which a `pane` renders (or, without a pane, are captured into `stdout`). This is the general alternative to `per-tool` force-color flags (`FORCE_COLOR=1`, `--color=always`): it works for any tool that gates output on a TTY. A `pty` merges `stdout` and `stderr` onto one stream, so `stderr` is empty in `pty` mode. On Windows `pty` is ignored (pipe fallback, no color).

5.10.4 Keybindings (TTY mode only)

Key	Action
Tab / Shift-Tab	Cycle focus through panes
PgUp / PgDn	Scroll focused pane one page
↑ / ↓	Scroll focused pane one line
Home / End	Jump to top / bottom
Ctrl-C	Abort the script (single press)

The focused pane has a yellow border; the bottom status bar shows its name and the active keys.

Panes follow the tail automatically; scroll up (keys or, when enabled, the mouse wheel) to pause following, and scroll back to the bottom (or press `End`) to resume. Set `{ autoscroll: false }` on a leaf to keep it pinned at the top. Pass `{ mouse: true }` at the layout root to enable mouse support: the wheel scrolls the pane **under the cursor** (without changing which pane is focused), and a left-click focuses the pane under the cursor. This disables the terminal's native click-drag selection while active; hold Shift/Option to select.

Per-leaf `{ wrap: "char" | "word" | "off" }` controls line wrapping (default `"char"` `"off"` lets long lines scroll horizontally), and `{ color: false }` strips a pane's ANSI to plain text instead of rendering it. Both affect TTY rendering only — the non-TTY fallback always emits plain prefixed lines.

`await tui.waitKey()` resolves with the next key (`{ name, rune, ctrl, alt, shift }`); `tui.onKey(handler)` registers a persistent per-keypress callback and returns an unsubscribe function. Both see every key except `Ctrl-C` (which always aborts) and coexist with the built-in navigation keys. Both require a TTY: `waitKey` rejects and `onKey` is a no-op in non-TTY (fallback) mode.

The `name` field is the tcell key name — `"Enter"`, `"Up"`, `"Tab"`, `"F1"`, or `"Ctrl-A"` for `Ctrl`+letter combos — or `"Rune"` for a printable character, in which case `rune` holds it. Because `Ctrl`+letter arrives as a combined name (`"Ctrl-A"`) rather than via the `ctrl` flag, branch on `name` to detect control keys. `waitKey` is one-shot: `await` it again for the next key, and concurrent waiters resolve FIFO (one keypress satisfies the oldest pending `await`). A pending `waitKey` keeps the TUI open, which is the idiomatic “press any key to close” hold. `onKey` does **not** keep the Run alive on its own — pair it with an outstanding `await` (a `waitKey` or a sleep) if the script should stay interactive.

5.10.5 Limitations (v1)

- `tui` is **incompatible with** `--watch`: calling `tui.layout()` under `--watch` throws. Use one or the other.
- No runtime layout mutation, no input panes (`tui.input`), no snapshot-to-normal-screen on exit. The alt screen is restored at script end and the usual `PASS / FAIL` line prints.

5.11 image

Image I/O and manipulation, all pure-Go and synchronous. Decode a raster image (PNG/JPEG/GIF/TIFF/BMP/WebP) or rasterize an SVG subset, then chain transforms on the returned **Image handle**. Each transform returns a *fresh* handle (immutable chaining); the source is never mutated.

Module functions

Function	Returns	Notes
<code>image.open(path, opts?)</code>	<code>Image</code>	Read + decode a file; format sniffed from magic bytes, not the extension. <code>opts.autoOrient: true</code> rotates pixels upright from the EXIF tag.

<code>image.decode(bytes, opts?)</code>	Image	Decode in-memory Uint8Array image bytes. <code>opts.autoOrient: true</code> rotates pixels upright from the EXIF tag.
<code>image.rasterizeSVG(src, { width, height })</code>	Image	Rasterize an SVG (subset) to a raster image at the given pixel size. <code>src</code> is a path or Uint8Array both dimensions required (> 0).
<code>image.decodeFrames(src)</code>	AnimDoc	Decode all frames of an animated GIF or APNG into a normalized frame model (see §5.11.2). A non-animated image yields a single frame. <code>src</code> is a path string or Uint8Array.
<code>image.encodeFrames(spec, opts?)</code>	AnimResult	Encode a frame set to an animated GIF or APNG (see §5.11.2). <code>opts.format</code> selects the encoder ("gif" / "apng", default "gif"); <code>opts.dest</code> writes the file and returns a path.

Handle properties (read-only): `width`, `height`, `format` (the decoded source format string, or "svg" for a rasterized SVG).

Handle methods (each returns a fresh Image unless noted):

Method	Effect
<code>resize(w, h, { filter? })</code>	Resize; a 0 dimension preserves aspect ratio. Filter ∈ <code>lanczos</code> (default), <code>nearest</code> , <code>linear</code> , <code>box</code> , <code>catmullrom</code> .
<code>fit(w, h)</code>	Scale to fit within <code>w×h</code> , preserving aspect (no crop).
<code>thumbnail(w, h)</code>	Scale + centre-crop to exactly <code>w×h</code> .
<code>crop(x, y, w, h)</code>	Crop a rectangle (bounds-checked).
<code>rotate(deg)</code>	Rotate CCW by an arbitrary angle; corners filled transparent.
<code>rotate90()</code> / <code>rotate180()</code> / <code>rotate270()</code>	Lossless quarter-turns (CCW).
<code>flipH()</code> / <code>flipV()</code>	Mirror horizontally / vertically.
<code>orient(n)</code>	Apply EXIF Orientation <code>n</code> (1..8) to pixels; 1 is a no-op copy. Throws <code>TypeError</code> if <code>n</code> is not an integer in 1..8.
<code>brightness(pct)</code> / <code>contrast(pct)</code> / <code>saturation(pct)</code>	Adjust by a percentage in <code>[-100, 100]</code> .
<code>gamma(g)</code>	Gamma correction (<1 darkens, >1 brightens).

<code>sharpen(sigma) / blur(sigma)</code>	Unsharp-mask / Gaussian blur.
<code>grayscale() / invert()</code>	Desaturate / photographic negative.
<code>overlay(other, x, y, opacity?)</code>	Alpha-blend another handle on top.
<code>paste(other, x, y)</code>	Paste another handle (no blend).
<code>bytes(format, { quality? })</code>	Encode → <code>Uint8Array</code> .
<code>save(path, { format?, quality? })</code>	Encode + write to disk (format inferred from the extension if not given). Returns <code>void</code> .

Encode formats: `png`, `jpeg`, `gif`, `tiff`, `bmp`, `webp`.

Worth knowing:

- `resize` with a `0` width or `0` height computes the missing dimension to preserve the aspect ratio.
- `quality` (1..100, default 90) applies to **JPEG** only. **WebP encode is lossless** (via `nativewebp`), so the `quality` option is accepted for API symmetry but ignored.
- **SVG is rasterize-in only** — there is no SVG *output*; `rasterizeSVG` renders a *subset* of SVG to a raster image you then encode as PNG/etc.
- `image.open` / `image.decode` **GIF decode is first-frame** — use `image.decodeFrames` to get all animation frames.
- `autoOrient` (`open` / `decode` `opts.autoOrient: true`) reads the source's EXIF `Orientation` tag and applies the corresponding pixel transform so the returned handle is always upright. Absent or unreadable `Orientation` is silently treated as 1 (no-op); the option never throws. **Strip-on-save is automatic** — the raster handle re-encodes from pixels with no EXIF carry-through.
- `orient(n)` applies one of the 8 EXIF pixel transforms directly (`n` 1..8; 1 is a no-op copy). Use this when you already know the orientation integer; use `autoOrient` to drive it from the source file's tag.
- **Hard 64-megapixel decode guard.** `image.open` / `image.decode` reject a source whose declared `width * height` exceeds 64,000,000 pixels *before* allocating the pixel buffer — a crafted header can claim an enormous resolution while the file itself stays a few dozen bytes (“decode bomb”). This cap is not configurable (no `opts` knob); `codec.barcode.decode` (§5.5.2) decodes through the same path and shares it.

```
const im = image.open("avatar.png");
const thumb = im.resize(128, 0).grayscale().blur(0.5);
thumb.save("thumb.webp"); // lossless webp
const png = im.crop(0, 0, 64, 64).bytes("png"); // → Uint8Array
// auto-orient a camera photo (reads EXIF Orientation, corrects pixels):
const photo = image.open("photo.jpg", { autoOrient: true });
// or apply an orientation value directly:
const cw = image.decode(rawBytes).orient(6); // 90° clockwise
```

Pure-Go stack: `disintegration/imaging` + `golang.org/x/image` (`webp/tiff/bmp` decode) + `HugoSmits86/nativewebp` (`webp` encode) + `srwiley/oksvg` + `rasterx` (SVG) + `kettek/apng` (APNG). No `cgo`.

5.11.1 Animated frames (GIF / APNG)

`image.decodeFrames` and `image.encodeFrames` provide full animated image support for GIF (stdlib) and APNG (`kettek/apng`). Both functions are synchronous.

`image.decodeFrames(src)` decodes every frame of an animated GIF or APNG and returns an `AnimDoc` object:

```
{
  format:    string;    // "gif" | "apng" | (source format for non-animated)
  width:     number;    // canvas width in pixels
  height:    number;    // canvas height in pixels
  loopCount: number;    // 0 = loop forever; n > 0 = play n times
  frames: {
    image:    Image;    // chainable Image handle for this frame's pixels
    delayMs:  number;    // display time in milliseconds
    xOffset:  number;    // frame's left edge on the canvas
    yOffset:  number;    // frame's top edge on the canvas
    disposal: "none" | "background" | "previous";
    blend?:  "source" | "over"; // APNG only
  }[];
}
```

Frames are returned *as stored* in the container — sub-rectangles with their placement metadata — not composited onto the canvas. Composite them yourself if you need the final rendered pixels.

A **non-animated input** (any image that is not a multi-frame GIF or APNG, including static PNGs, JPEGs, TIFFs, WebP, etc.) returns a single-element `frames` array so callers can treat all images uniformly.

`image.encodeFrames(spec, opts?)` encodes a frame set into an animated GIF or APNG. The `spec` object mirrors `AnimDoc`:

```
{
  width?:    number;    // canvas width (derived from frame extents if omitted)
  height?:   number;    // canvas height (derived from frame extents if omitted)
  loopCount?: number;    // default 0 (loop forever)
  frames: {
    image:    Image;    // required — an Image handle
    delayMs?: number;    // default 0
    xOffset?: number;    // default 0
    yOffset?: number;    // default 0
    disposal?: "none" | "background" | "previous"; // default "none"
    blend?:   "source" | "over"; // APNG only, default "over"
  }[];
}
```

`opts.format` selects the encoder ("gif" or "apng", default "gif"). Without `opts.dest` the result is `{ format, bytes }` (bytes as a `Uint8Array` -compatible array). With `opts.dest` the file is written and `{ format, path }` is returned instead.

GIF palettization: GIF requires an indexed (paletted) color model. Each frame is quantized to the 256-color Plan 9 palette with Floyd–Steinberg dithering, which handles photographic content reasonably but loses subtle gradients. For full-color animations use APNG. GIF

frame delays are stored in hundredths of a second, so `delayMs` quantizes to the nearest 10 ms on encode (`delayMs / 10`); APNG preserves exact-millisecond delays (up to 65 s, beyond which it falls back to 10 ms granularity).

APNG full-color: APNG frames are encoded at full RGBA color depth with no palette reduction.

```
// Decode all frames of an animated GIF:
const anim = image.decodeFrames("banner.gif");
runtime.log(anim.format, anim.frames.length, "frames");
runtime.log("first frame delay:", anim.frames[0].delayMs, "ms");

// Re-encode as APNG (full-color):
const out = image.encodeFrames(anim, { format: "apng" });
runtime.log(out.format, out.bytes.length, "bytes");

// Build a two-frame animation from scratch:
const frameA = image.open("a.png");
const frameB = image.open("b.png");
image.encodeFrames({
  loopCount: 3,
  frames: [{ image: frameA, delayMs: 500 }, { image: frameB, delayMs: 500 }],
}, { format: "gif", dest: "out.gif" });
```

5.11.2 EXIF

`image.exif` gives synchronous EXIF read/write for reconnaissance and metadata-aware tooling. Reads are broad (JPEG/PNG/TIFF full + HEIC/AVIF/RAW curated subset); writes are JPEG and PNG only.

Function	Returns	Notes
<code>image.exif.read(src)</code>	<code>ExifDoc</code>	Read grouped EXIF from a path or <code>Uint8Array</code> . Returns <code>{}</code> when no EXIF is present.
<code>image.exif.write(src, data, opts?)</code>	<code>ExifResult</code>	Merge tags (null deletes a tag). JPEG/PNG only.
<code>image.exif.replace(src, data, opts?)</code>	<code>ExifResult</code>	Replace the whole EXIF block. JPEG/PNG only.
<code>image.exif.clear(src, opts?)</code>	<code>ExifResult</code>	Remove all EXIF. JPEG/PNG only.

`src` is a file path string or a `Uint8Array` of raw image bytes.

ExifDoc — tags grouped by IFD:

```
{
  image?: Record<string, unknown>; // IFD0 (camera/software metadata)
  exif?: Record<string, unknown>; // ExifIFD (exposure, lens, ...)
  gps?: Record<string, unknown>; // GPSIFD
  thumbnail?: Record<string, unknown>; // IFD1 thumbnail
}
```

Value encoding inside the groups:

EXIF type	JS representation
RATIONAL / SRATIONAL	[numerator, denominator] array
GPS Latitude / Longitude	Signed decimal (number)
Date/time strings	EXIF string "YYYY:MM:DD HH:MM:SS"
UNDEFINED / BYTE arrays	base64 string
Integer / Short / Long	number
ASCII	string

ExifResult — returned by write/replace/clear:

- { format: string; bytes: Uint8Array } — when no `opts.dest` is given.
- { format: string; path: string } — when `opts.dest` names a destination path (the bytes are also written there).

Unsupported write targets: Calling write/replace/clear on a format other than JPEG or PNG throws ("image.exif.<op>: writing EXIF to ... is unsupported"). Read is broader and falls back to `imagemeta` for HEIC/AVIF/CR2/CR3/NEF/ARW/DNG/RAW.

```
// Replace EXIF block
const out = image.exif.replace(jpegBytes, { image: { Make: "sercon" } });
const e = image.exif.read(out.bytes);
runtime.log(e.image?.Make);    // "sercon"

// Merge / delete a tag
const out2 = image.exif.write(out.bytes, { image: { Artist: "Alice", Model:
null } });

// Strip all
const stripped = image.exif.clear(out2.bytes);
```

5.11.3 Steganography (`image.stego`)

`image.stego` hides and recovers payloads inside lossless images using **least-significant-bit (LSB)** steganography. One bit is stored per R/G/B channel; the alpha channel is **never modified** (opaque PNG carriers are recommended). The carrier can be any supported format (PNG/JPEG/GIF/TIFF/BMP/WebP), but the output of `embed` is always PNG — re-encoding to a lossy format would destroy the hidden data.

An optional **AES-256-GCM** password encrypts the payload before embedding (PBKDF2-SHA256 key derivation). Without a password the payload is stored as plain bytes. A fixed 10-byte header (`sercon` magic, version, flags, and a 4-byte payload length) is prepended to every LSB stream, so `extract` can detect foreign or corrupt carriers and report a clean error.

Function	Returns	Notes
<code>image.stego.embed(carrier, payload, opts?)</code>	{ bytes: Uint8Array } { path: string }	Hide payload in <code>carrier</code> . <code>opts.password</code> encrypts; <code>opts.dest</code> writes the PNG to disk; <code>opts.bits</code> sets the depth (1..4, default 1).

<code>image.stego.extract(carrier, string Uint8Array opts?)</code>		Recover the payload. Returns a <code>string</code> for text payloads, <code>Uint8Array</code> for binary. <code>opts.password</code> required when encrypted. Depth is auto-detected from the header.
<code>image.stego.capacity(carrier, opts?)</code>	<code>{ bytes: number; bits: number }</code>	Maximum plaintext payload size in bytes at the requested depth (<code>opts.bits</code> , default 1). Returns <code>{ bytes, bits }</code> .

```
// Check how many bytes a carrier can hold at 1-bit and 4-bit depth
const room1 = image.stego.capacity("cover.png", { bits: 1 }).bytes;
const room4 = image.stego.capacity("cover.png", { bits: 4 }).bytes; // room4 === room1 * 4

// Embed with 4-bit depth for higher capacity
const out = image.stego.embed("cover.png", "meet at noon", { password: "s3cret", bits: 4 });
// out.bytes is the stego PNG as a Uint8Array

// Recover – depth is auto-detected from the header, no bits arg needed
const msg = image.stego.extract(out.bytes, { password: "s3cret" });
// msg === "meet at noon"
```

Multi-bit depth (`bits` option). `embed` accepts `opts.bits` (integer 1..4, default 1) to store more payload bits per channel at the cost of greater perceptual change. The capacity formula is $(\text{pixels} \times \text{channels} - 80) \times N / 8$ bytes, where N is the chosen depth — so `bits: 4` gives 4× the capacity of `bits: 1`. The chosen depth is recorded in the stego header, so `extract` auto-detects it without any `bits` argument; existing 1-bit payloads are fully backward-compatible. `capacity` accepts the same `{ bits }` option and echoes the depth in the return value.

Worth knowing:

- `payload` may be a `string` (stored as UTF-8, extracted as `string`) or a `Uint8Array` (stored as binary, extracted as `Uint8Array`).
- The carrier must be large enough: `embed` throws `"payload too large (need N bytes, capacity M)"` when it isn't.
- `extract` throws `"no sercon stego payload found"` on a carrier that wasn't produced by `embed`, and `"wrong password or corrupt data"` on a decryption failure — so authentication errors are explicit.
- For write-to-disk, pass `opts.dest`:
`image.stego.embed("cover.png", data, { dest: "stego.png" }) → { path: "stego.png" }.`

Detection and analysis (read-only). Three companion functions let you inspect any image without modifying it. `image.stego.detect(carrier)` returns a quick verdict: a definitive

`sercon` flag (true when the "ScSt" header is found, with an accompanying `bits` field for the declared depth) plus a `suspicious / confidence` pair from statistical analysis.

`image.stego.analyze(carrier)` runs a full per-channel report — chi-square (pairs-of-values probability), LSB-plane Shannon entropy, and an RS embedding-rate estimate — and returns a `verdict` with a `reasons` array explaining the conclusion. The report now also includes `chiSquareByBits` (4-element array, chi-square signal at depths 1..4) and `entropyByPlane` (4-element array, Shannon entropy per bit-plane) on each channel entry, plus a top-level `estimatedBits` field and `sercon.bits`. `estimatedBits` is a coarse, best-effort hint — it needs substantial embedding coverage to register and returns 0 for small or partial payloads. For `sercon`-format carriers the authoritative depth is `sercon.bits` (from the header); `chiSquareByBits` and `entropyByPlane` are the real per-depth diagnostics.

`image.stego.bitplane(carrier, opts?)` renders a chosen bit-plane of the image as a PNG: hidden LSB data typically shows as structured noise in the low planes (plane 0 is the LSB). All three are purely read-only and never alter the carrier.

5.11.4 Concepts

`decode / open / rasterizeSVG` (and each frame's `image` field from `decodeFrames`) all return the same **Image handle**: read-only `width / height / format` plus the chainable transform methods listed in the handle-methods table above. Every transform returns a *fresh* handle — `im.resize(...).grayscale().blur(...)` composes without you tracking intermediate copies, and the source handle is never mutated. There's no separate "convert format" call: format conversion falls out of the same `bytes(format) / save(path)` step you'd already use to get output — decode a PNG, `.bytes("webp")` it, done.

Three families sit alongside the base `decode → transform → encode` loop without changing its shape:

- **Animated frames (§5.11.1)** — `decodeFrames / encodeFrames` treat a GIF/APNG as an ordered list of per-frame Image handles plus timing/placement metadata. Build or inspect an animation by working with individual frames exactly like a single image; a non-animated source just yields a one-frame list.
- **image.exif (§5.11.2)** — reads/writes metadata alongside pixels, independent of geometry/color adjustment; writes are JPEG/PNG only.
- **image.stego (§5.11.3)** — hides/recovers a payload in an image's least-significant bits. It works on raw bytes/paths rather than a chainable handle (`embed` returns `{bytes} / {path}`, not an `Image`), because the goal is bit-exact pixel data, not a transform pipeline; `detect / analyze / bitplane` are read-only companions for inspecting a carrier without modifying it.

SVG is rasterize-in only — `rasterizeSVG` turns an SVG subset into a normal Image handle you then encode as PNG/JPEG/etc.; there's no SVG encoder to go back the other way.

5.11.5 Recipes

5.11.5.1 Resize an image and convert its format

```
const im = image.decode(await fs.readBytes("medium.png"));
const thumb = im.resize(200, 0).grayscale(); // 0 height preserves aspect ratio

// "convert" is just encoding to a different target format —
// there is no separate convert call.
```

```
await fs.writeBytes("thumb.png", thumb.bytes("png"));
await fs.writeBytes("thumb.jpg", thumb.bytes("jpeg"));
runtime.log(`${thumb.width}x${thumb.height} grayscale, written as PNG + JPEG`);
```

`resize`'s 0 dimension computes the missing side to preserve aspect ratio; `bytes(format)` picks the encoder independently of how the source was decoded, so one resized handle can feed as many output formats as you need. Signatures: §17.8.6 (`image.decode`) — handle methods appear inline on its `Image` return type; see the handle-methods table above for the full list. runnable: `examples/recipes/image-pipeline.ts`

5.11.5.2 Crop and correct orientation

```
const im = image.decode(png);

// Crop a rectangle, then round-trip it through disk.
im.crop(2, 2, 10, 10).save(tmpPath);
const cropped = image.open(tmpPath);
runtime.assert.equal(cropped.width, 10, "cropped width");

// Apply a known EXIF orientation value directly...
const oriented = image.decode(png).orient(6); // 90° CW pixel transform

// ...or let decode/open read it from the source file's own EXIF tag.
const upright = image.open("photo.jpg", { autoOrient: true });
```

`orient(n)` applies one of the 8 EXIF pixel transforms directly when you already know the orientation integer; `autoOrient` drives the same transform from the source's own `Orientation` tag (a no-op when the tag is absent or unreadable — it never throws). `rotate90()` / `rotate180()` / `rotate270()` are the lossless quarter-turn siblings of `orient` for a fixed rotation that isn't EXIF-driven. Signatures: §17.8.6 (`image.decode`), §17.8.16 (`image.open`). runnable: `examples/scripts/image.ts`

5.11.5.3 Apply brightness / contrast / filter adjustments

```
const im = image.decode(png);
const out = im.resize(8, 0).grayscale().blur(0.5); // desaturate + soften after
resizing

runtime.assert.equal(out.width, 8, "resized width");
```

`brightness(pct)` / `contrast(pct)` / `saturation(pct)` (a percentage in `[-100, 100]`), `gamma(g)`, `sharpen(sigma)`, and `invert()` share the same shape as `grayscale()` / `blur(sigma)` above — chain any of them the same way; each returns a fresh handle so the source is never mutated. Signatures: §17.8.6 (`image.decode`) — see the handle-methods table above for the full adjustment/filter method list. runnable: `examples/scripts/image.ts`

5.11.5.4 Embed and extract steganographic data

```
const carrier = await fs.readBytes("cover.png");
const secret = "rendezvous at 0900";

const stego = image.stego.embed(carrier, secret, { password: "p@ss" });
```

```
await fs.writeBytes("stego.png", stego.bytes);

const recovered = image.stego.extract(await fs.readBytes("stego.png"), { password:
  "p@ss" });
runtime.assert.equal(recovered, secret, "stego round-trip");
```

`embed`'s output is always PNG regardless of the carrier's source format — re-encoding to anything lossy would destroy the hidden bits. `extract` auto-detects the bit depth from the stego header, so higher-capacity payloads (`opts.bits`, 1..4) need no extra argument on the way back out; `image.stego.capacity(carrier, { bits })` tells you how many bytes fit before you embed. Signatures: §17.8.29.5 (`image.stego.embed`), §17.8.29.6 (`image.stego.extract`), §17.8.29.3 (`image.stego.capacity`). runnable:

`examples/recipes/stego-hide.ts`, `examples/scripts/stego.ts`

5.11.5.5 Build a two-frame animation

```
const base = image.decode(png);

const spec = {
  width: base.width,
  height: base.height,
  loopCount: 0, // loop forever
  frames: [
    { image: base,          delayMs: 100 },
    { image: base.grayscale(), delayMs: 200 },
  ],
};

const gif = image.encodeFrames(spec, { format: "gif" });
const back = image.decodeFrames(gif.bytes);
runtime.assert.equal(back.frames.length, 2, "round-trip frame count");
```

Each frame's `image` is a regular `Image` handle, so building an animation is just assembling already-familiar handles into a `frames` array — swap `{ format: "gif" }` for `{ format: "apng" }` for full-color output (GIF palettizes to 256 colors with dithering). Signatures: §17.8.8 (`image.encodeFrames`), §17.8.7 (`image.decodeFrames`). runnable: `examples/scripts/image-anim.ts`

5.12 web

Fetch & parse web documents. Three families — `web.feed` (RSS/Atom/JSON feeds), `web.sitemap` (sitemaps), and `web.html` (lenient HTML scraping) — each exposing a synchronous `parse(string)` and an async `load(url, opts?)`. `parse` works on a string you already have; `load` GETs the URL first, then parses the body. This closes the HTML-scraping gap that `codec.xml` (strict XML only) left open. All pure-Go, no cgo.

Shared fetch options. Every `load(url, opts?)` reuses the `net.http` option surface — `timeout` (ms number or duration string), `headers`, `follow` (redirects), `userAgent`, `basic-auth` `username` / `password`, and `maxBytes` (response body size cap, default 256 MB; a response over the cap rejects) — and sends a default `sercon-web/<version>` User-Agent unless you override it. A non-2xx response throws.

5.12.1 web.feed

Function	Returns	Notes
<code>web.feed.parse(source)</code>	<code>Feed</code>	Parse RSS, Atom, or JSON-feed text; format auto-detected (<code>feedType</code> reports it).
<code>web.feed.load(url, opts?)</code>	<code>Promise<Feed></code>	Fetch then parse.

`Feed` is normalized to one model regardless of source format — the RSS/Atom field differences (`pubDate` / `updated` , `description` / `summary`) are unified:

```
{ feedType, title, description, link, updated, items[] }. Each item is  
{ title, link, published, updated, content, summary, author, guid, categories[],  
  raw }
```

The `raw` object is the escape hatch carrying format-specific extras (enclosures, namespaced elements like `media:*` / `dc:*`) that the normalized model drops.

5.12.2 web.sitemap

Function	Returns	Notes
<code>web.sitemap.parse(source)</code>	<code>Sitemap</code>	Parse a <code>urlset</code> or <code>sitemapindex</code> document.
<code>web.sitemap.load(url, opts?)</code>	<code>Promise<Sitemap></code>	Fetch then parse; transparently decompresses <code>.xml.gz</code> .

`Sitemap` is `{ type: "urlset" | "sitemapindex", urls[], sitemaps[], errors[] }`. `urls` entries are `{ loc, lastmod?, changefreq?, priority? }` an index lists child sitemap URLs in `sitemaps`. `load` detects `gzip` by magic bytes on the body (a `Content-Encoding: gzip` response is already unwrapped by the HTTP transport). For an index, pass `{ expand: true }` to fetch every child sitemap (bounded, single-level expansion — a child that is itself a `sitemapindex` is not recursed) and merge their URLs into `urls` per-child fetch/parse failures are collected in `errors[]` rather than thrown.

Same-site expansion (SSRF guardrail). A `<loc>` child URL is data read from the fetched `sitemapindex`, not a maintainer-authored path, so `expand` only follows children that are same-site — same scheme and registrable domain — as the index's own URL, and re-checks that on every redirect hop a child fetch takes. A cross-site child (including one reached only via redirect) is never fetched; it's recorded in `errors[]` instead (`"skipped: cross-site child (same-site only)"` or the redirect-hop variant). This restriction applies only to expansion; the top-level URL you pass to `load` is unaffected.

5.12.3 web.html

Function	Returns	Notes
<code>web.html.parse(source)</code>	<code>Node</code>	Parse HTML leniently — real-world tag soup is accepted, never throws on bad markup.
<code>web.html.load(url, opts?)</code>	<code>Promise<Node></code>	Fetch then parse.

`parse` / `load` return a chainable **Node** rooted at the document. Query it with CSS or XPath, read it with the accessor methods:

Method	Effect
<code>find(selector)</code>	First CSS match (or <code>null</code>).
<code>findAll(selector)</code>	All CSS matches as a <code>Node[]</code> .
<code>xpath(expr)</code>	First XPath match (or <code>null</code>).
<code>xpathAll(expr)</code>	All XPath matches as a <code>Node[]</code> .
<code>text()</code>	Concatenated text content.
<code>html()</code>	Outer HTML of the node.
<code>innerHTML()</code>	Inner HTML (children only).
<code>tag()</code>	Tag name.
<code>attr(name)</code>	Attribute value (or <code>null</code>).
<code>attrs()</code>	All attributes as a <code>Record<string, string></code> .

Sub-queries are scoped to the receiver node — use `./` for relative XPath; a leading `//` is document-wide.

```
const feed = await web.feed.load("https://example.com/feed.xml");
feed.items.forEach(i => console.log(i.title, i.link));

const sm = await web.sitemap.load("https://example.com/sitemap.xml", { expand:
true });
console.log(sm.urls.length, "urls");

const doc = await web.html.load("https://example.com");
doc.findAll("a.product").forEach(a => console.log(a.text(), a.attr("href")));
doc.xpathAll("//h2").forEach(h => console.log(h.text()));
```

Pure-Go stack: `mmcdole/gofeed` (feeds) + `andybalholm/cascadia` (CSS selectors over `golang.org/x/net/html`) + `antchfx/htmlquery` (XPath). No cgo.

5.12.4 Concepts

Reach for `parse` when you already have the bytes — a fixture string, a file you read yourself — and `load` when you want `sercon` to fetch first; `load(url, opts?)` is `parse` layered on the shared `net.http` fetch (same `timeout` / `headers` / `follow` / `userAgent` / `basic-auth` surface, same default `sercon-web/<version>` User-Agent, same `throw-on-non-2xx`). All three families share that shape, so a failure's origin tells you which half broke: a `load` throw is the fetch (bad URL, timeout, non-2xx status); a `parse` throw is the format assumption being wrong — except `web.html.parse`, which never throws on bad markup, it accepts real-world tag soup and does its best.

`web.html.parse` / `load` return a chainable **Node** instead of raw HTML text: `find` / `findAll` run CSS selectors (`andybalholm/cascadia`) and `xpath` / `xpathAll` run XPath (`antchfx/htmlquery`); `text()` / `html()` / `attr(name)` / ... read data back out. Sub-queries are scoped to the receiver node, so calling `.findAll(...)` on a node found earlier stays localized to that subtree — use `./` for a relative XPath, a leading `//` is document-wide.

5.12.5 Recipes

5.12.5.1 Fetch and parse an RSS/Atom feed

```
const feed = await web.feed.load("https://hnrss.org/frontpage", { timeout:
  "5s" });
runtime.log(feed.feedType, feed.items.length, "items");
for (const item of feed.items) {
  runtime.log(item.title, item.link, item.published);
}
```

Format (RSS/Atom/JSON-feed) is auto-detected; `feedType` reports which one. See §17.16.1.1 (`web.feed.load`); the feed result model is in §5.12.1. runnable: `examples/scripts/web-feed.ts`

5.12.5.2 Crawl a sitemap, expanding an index

```
const sm = await web.sitemap.load("https://www.sitemaps.org/sitemap.xml", {
  timeout: "5s",
  expand: true,
});
runtime.log(sm.type, sm.urls.length, "urls,", sm.sitemaps.length, "child
sitemaps");
```

`{ expand: true }` only matters when `type` is `"sitemapindex"` — it fetches every child sitemap and merges their URLs into `urls` (single-level, bounded; a child that is itself an index is not recursed). Per-child fetch/parse failures land in `errors[]` instead of throwing, so check that array rather than wrapping the call in `try/catch`. See §17.16.3.1 (`web.sitemap.load`); the sitemap result model is in §5.12.2. runnable: `examples/scripts/web-sitemap.ts`

5.12.5.3 Scrape a page with CSS selectors

```
const doc = await web.html.load("https://example.com", { timeout: "5s" });
const links = doc.findAll("li.p a").map((a: any) => a.attr("href"));
const title = doc.find("h1").text();
```

`find` returns the first match or `null` `findAll` returns a `Node[]` you can `.map` / `.forEach` over, each entry chainable the same way as the root. See §17.16.2.1 (`web.html.load`); the Node handle methods are in §5.12.3. runnable: `examples/scripts/web-html.ts`

5.12.5.4 Extract with XPath

```
const doc = web.html.parse(
  `

<li class="p"><a href="/a">Alpha<li class="p"><a href="/b">Beta</ul>`,
);
const firstHref = doc.xpath("//a/@href").text(); // "/a"
```

`xpath` / `xpathAll` mirror `find` / `findAll` but take an XPath expression instead of a CSS selector — useful when a selector can't express the match (attribute values, text-content predicates, ancestor axes). See §17.16.2.2 (`web.html.parse`); the Node handle methods are in §5.12.3. runnable: `examples/scripts/web-html.ts`

5.13 audio

The `audio` global reads and writes audio files and hides payloads inside them. It exposes four format members (`info`, `decode`, `encode`, `convert`) and one sub-namespace (`audio.stego`).

5.13.1 Format read/write

`audio` can probe and transcode audio files across WAV, FLAC, MP3, OGG (Vorbis), and AIFF. Internally all formats share a **canonical 16-bit signed PCM model** — every decoder normalises its output to 16-bit samples regardless of source bit depth, and encoders consume 16-bit PCM.

Format matrix:

Format	Decode	Encode
WAV	yes	yes
FLAC	yes	yes
AIFF	yes	yes
MP3	yes	no (no pure-Go encoder)
OGG	yes	no (no pure-Go encoder)

All operations are pure-Go (no cgo, no external process).

Function	Returns	Notes
<code>audio.info(src)</code>	<code>{ format, sampleRate, channels, bitDepth, frames, durationMs }</code>	Probe metadata without returning samples.
<code>audio.decode(src)</code>	<code>{ format, sampleRate, channels, bitDepth, frames, durationMs, pcm: Uint8Array }</code>	Decode to 16-bit LE interleaved PCM bytes.
<code>audio.encode(pcm, opts)</code>	<code>{ bytes: Uint8Array } { path: string }</code>	Encode raw 16-bit LE PCM to a lossless container.
<code>audio.convert(src, opts)</code>	<code>{ bytes: Uint8Array } { path: string }</code>	Decode any supported source and re-encode as a lossless container.

`src` may be a file path (string) or `Uint8Array`. `audio.encode` requires `opts.format` (`"wav"`, `"flac"`, or `"aiff"`), `opts.sampleRate`, and `opts.channels`. `audio.convert` requires `opts.format`. Both accept an optional `opts.dest` to write output to a file path instead of returning bytes.

```
// Probe an existing file.
const meta = audio.info("song.flac");
// → { format: "flac", sampleRate: 44100, channels: 2, bitDepth: 24, frames: ...,
durationMs: ... }

// Decode to raw PCM, modify samples, re-encode as WAV.
const d = audio.decode("song.wav");
const out = audio.encode(d.pcm, { format: "flac", sampleRate: d.sampleRate,
```

```

channels: d.channels });
// out.bytes is the FLAC as a Uint8Array

// One-step convert without touching PCM.
const flac = audio.convert("input.wav", { format: "flac" });
await fs.writeBytes("/tmp/out.flac", flac.bytes);

```

Errors: `audio.encode` / `audio.convert` throw for MP3 or OGG target formats (no pure-Go encoder). `audio.decode` / `audio.info` throw if the source is not a recognized or decodable format. `audio.encode` throws if `sampleRate` / `channels` are missing or ≤ 0 , or if the PCM byte length is not a whole number of frames.

16-bit normalization: source bit depths above 16 are shifted down ($>> (\text{srcBits} - 16)$); 8-bit sources are shifted up ($<< 8$). The reported `bitDepth` reflects the source; samples are always 16-bit on the wire.

5.13.2 `audio.stego`

`audio.stego` hides and recovers payloads inside WAV PCM files using least-significant-bit (LSB) embedding — the LSB of each PCM sample byte carries one bit of the payload stream. Both 8-bit and 16-bit PCM are supported; 16-bit files use the low byte of each sample (little-endian). The payload stream is prefixed with a `sercon`-magic header carrying the flags and length, and optionally sealed with AES-256-GCM.

Function	Returns	Notes
<code>audio.stego.embed(carrier, payload, opts?)</code>	<code>{ bytes: Uint8Array } { path: string }</code>	Hide payload in the WAV carrier. <code>opts.bits</code> sets depth (1..4, default 1).
<code>audio.stego.extract(carrier, opts?)</code>	<code>string Uint8Array</code>	Recover the hidden payload. Depth is auto-detected from the header.
<code>audio.stego.capacity(carrier, opts?)</code>	<code>{ bytes: number; bits: number }</code>	Maximum plaintext payload in bytes at the requested depth (<code>opts.bits</code> , default 1).

`carrier` may be a file path (string) or `Uint8Array`. `payload` may be a string (text flag set; `extract` returns a string) or `Uint8Array`. `opts.password` enables AES-256-GCM encryption. `embed` returns `{ bytes: Uint8Array }` by default, or `{ path }` when `opts.dest` is provided. `capacity` accounts for the stego header overhead; a WAV shorter than 128 samples returns 0. The `bits` option (1..4, default 1) controls how many bits are stored per PCM sample — higher values give up to 4× capacity at the cost of greater audible change. The chosen depth is stored in the stego header, so `extract` auto-detects it; existing 1-bit payloads are unaffected. The capacity formula is $(\text{samples} - 80) \times N / 8$ bytes where N is the depth.

```

const wav = fs.readBytes("input.wav");
const room = audio.stego.capacity(wav, { bits: 4 }).bytes; // 4× the 1-bit capacity

```



```
const out = audio.stego.embed(wav, "meet at noon", { password: "s3cret", bits:
4 });
// out.bytes is the stego WAV as a Uint8Array

const msg = audio.stego.extract(out.bytes, { password: "s3cret" });
// msg === "meet at noon" - depth auto-detected from header
```

Errors: `extract` throws "no sercon stego payload found" when the carrier has no embedded payload; "wrong password or corrupt payload" on decryption failure; "not a RIFF/WAVE file" / "unsupported WAV encoding" / "unsupported bit depth" on an invalid carrier; `embed` throws "payload too large" when the carrier does not have enough samples. Pure-Go, no cgo.

5.14 cloud

The `cloud` namespace drives the three major cloud providers — Google Cloud (`cloud.google`), Amazon Web Services (`cloud.aws`), and Microsoft Azure (`cloud.azure`) — natively, in pure Go, with no `gcloud` / `aws` / `az` CLI and no subprocess. Each provider is exposed as a **callable** that returns a handle:

```
const g = cloud.google({ project: "my-project" });
const aws = cloud.aws({ region: "eu-north-1" });
const az = cloud.azure({ subscriptionId:
"00000000-0000-0000-0000-000000000000" });
```

All three share the same shape, so once you know one you know the others:

- **Construction is synchronous; every operation is asynchronous.** Building the handle (and its service objects) does no I/O and never throws for network reasons; each *operation* returns a `Promise`, so you `await` it.
- **Services are accessed as functions on the handle.** `g.storage()`, `aws.s3()`, `az.compute()` return a service object whose methods are the async operations. Build the service once, call many methods on it.
- **Credentials are reused, never minted.** Each provider reads the same ambient credentials its own CLI produces (see *Concepts* below) — so if `gcloud` / `aws` / `az` already works on the machine, so does `sercon`. Credentials are never logged.
- **Errors throw.** A 4xx/5xx API response or a transport failure *rejects* the promise with a structured `Error { code, status, message, details }` — unlike `net.http`, where a 4xx is a normal return value. Wrap calls in `try/catch` and branch on `e.code` / `e.status`.

> **Two views of the surface.** This section is the *guide*: what each service is > for and how to use it, with runnable examples. §17.2 is the generated > *reference*: the exact TypeScript signature of every method. Use them together.

5.14.1 Concepts

Credentials — reuse the cloud's own login. None of the providers ask you to mint tokens; each consumes the ambient credentials its native tooling already set up, and each also accepts an explicit override on the handle:

Provider	Default (ambient) chain	Explicit override
----------	-------------------------	-------------------

<code>cloud.google</code>	Application Default Credentials — <code>gcloud auth application-default login</code> , <code>GOOGLE_APPLICATION_CREDENTIALS</code> , or the metadata server	<code>credentials</code> (service-account key path or inline JSON), <code>scopes</code> , <code>quotaProject</code>
<code>cloud.aws</code>	the standard AWS chain — env vars, <code>~/.aws/config</code> + <code>credentials</code> , <code>AWS_PROFILE</code> , SSO, IMDS (instance role)	<code>credentials: { accessKeyId, secretAccessKey, sessionToken? }, profile</code>
<code>cloud.azure</code>	<code>DefaultAzureCredential</code> — env, managed identity, <code>az login</code>	<code>{ tenantId, clientId, clientSecret }</code> (client-secret credential)

Secret material (`clientSecret` , SA keys, tokens) is never written to logs or error strings.

Scoping differs by provider — this is the main thing to get right:

- **Google** is scoped per **project**. `cloud.google({ project })` sets a default; most methods also take a `project` in their options.
- **AWS** is scoped per **region**, set once on the handle (`cloud.aws({ region })`), or from `AWS_REGION` /the profile). Clients are region-bound at construction — there is no per-call region.
- **Azure** has two planes. The **management plane** (ARM: `resourceGroups` , `compute` , `resources` , and the `call()` escape hatch) is scoped per **subscription** — set `subscriptionId` on the handle (or `AZURE_SUBSCRIPTION_ID`); calling one without a resolvable subscription throws. The **data plane** (`blob` , `keyvaultSecrets`) is scoped per **resource endpoint** — you pass the account/vault URL to the accessor itself:
`az.blob("https://acct.blob.core.windows.net")` .

Error shape. Every provider maps a failed call to the same structured error:

```
try {
  await cloud.aws({ region: "eu-north-1" }).s3().getObject({ bucket: "b", key:
    "missing" });
} catch (e) {
  runtime.log(`failed: code=${e.code} status=${e.status} - ${e.message}`);
  // e.code: provider error code ("NoSuchKey", "ResourceGroupNotFound", ...)
  // e.status: HTTP status (404, 403, ...); 0 for a transport/DNS/TLS failure
}
```

Result shapes and key casing. Responses come back as plain JS objects (the SDK response round-tripped to JSON), so the key casing follows each SDK:

- **Google** results are mostly lowercase/camelCase (`items` , `buckets`).
- **AWS** results are **PascalCase** (`Buckets` , `Reservations` , `LogGroups`) — the AWS SDK response structs carry no JSON tags, so their Go field names show through. Write `r.Buckets` , not `r.buckets` .
- **Azure** varies: ARM and Blob results are PascalCase (`Name`), Key Vault is lowercase (`id`).

Binary payloads (`s3().getObject` , `blob().download` , `GCS readObject`) come back as `{ bytes }` , where `bytes` renders in JS as an **array of numbers**, not a real `Uint8Array` —

wrap it yourself: `const buf = new Uint8Array(res.bytes)`. Decoded secret values (`getSecretValue` / `getSecret` / `accessSecretVersion`) come back as `{ value: string }`.

Escape hatches (reach anything). Where a cloud has a uniform REST surface, the provider offers a generic `call()` for operations no typed service covers:

- `cloud.google().call({ api, path, ... })` — any Google REST API (path-based).
- `cloud.azure().call({ path, apiVersion, ... })` — any ARM resource provider.
- **AWS has no escape hatch** — its per-service SDK clients have no uniform wire protocol, so `cloud.aws` is typed-services-only.

`cloud.azure` is PROVISIONAL. It is built against the Azure SDK but has **not been verified against a live Azure account** — the maintainer and CI have no Azure credentials, so it is exercised only by mocked unit tests. Treat its behaviour as unconfirmed until you have run it against a real subscription; the Azure examples below are illustrative.

Not covered this release: streaming / event-stream operations and gRPC-only data-plane services (e.g. Pub/Sub streaming, Firestore, Spanner). Operations are whole-value (bytes in / bytes out).

5.14.2 `cloud.google`

`cloud.google(opts?)` builds a handle onto Google Cloud: Cloud Storage, Compute Engine, IAM service accounts, and Secret Manager as typed services, plus a generic path-based REST `call()` for anything else. It is **pure Go, CGO-free** — built on `google.golang.org/api`, not the C++ gRPC client — so it runs the same way on every platform `sercon` supports, with no native dependency to install.

Credentials. If you've never used a Google Cloud SDK before, the concept to know is **Application Default Credentials (ADC)**: a small chain of places the client library looks for a usable identity, tried in order —

1. an explicit `credentials` value passed to `cloud.google({...})` (see below) — this always wins when present;
2. the `GOOGLE_APPLICATION_CREDENTIALS` environment variable, pointing at a service-account JSON key file on disk;
3. the credentials left behind by running `gcloud auth application-default login` on the machine (typical for local development);
4. the attached identity of the environment itself — a GCE/GKE/Cloud Run metadata server, when running inside Google Cloud.

You do not select which of these applies from script code — omit `credentials` entirely and the first one that's actually configured on the host wins. For a local dev machine, step 3 (`gcloud auth application-default login`) is usually the quickest way to get something working before scripting against a service account.

To bypass ADC and pin an explicit identity, pass `credentials` yourself: either a **string** (a filesystem path to a service-account JSON key) or an **object** (the key's JSON, inlined — handy when the key comes from a secret store rather than a file). Credential fields are never logged by `sercon`, in errors or otherwise.

Construct the handle synchronously — `cloud.google({...})` itself never returns a `Promise` and never touches the network:

```
const g = cloud.google({ project: "my-gcp-project" });
```

`opts` (all fields optional):

- `project` — the default GCP **project id** (not the numeric project number, not the display name) used by any service call that omits its own `project`. Most examples below rely on this default rather than repeating `project` on every call.
- `credentials` — a file path (string) or inline service-account JSON (object), as described above; omitted $\Rightarrow$ ADC.
- `scopes` — OAuth scopes to request; omitted $\Rightarrow$ each service's own default scope (broad enough for everything documented here).
- `quotaProject` — a billing/quota project override, for the case where the calling identity's home project shouldn't absorb the API quota (sent as `X-Goog-User-Project`).

Every service accessor (`storage()`, `compute()`, `iam()`, `secrets()`) and every method on them is **async** — call them with `await`, they resolve to plain data. A failure — a 403 from an IAM-permission gap, a 404 for a bucket that doesn't exist, a network timeout reaching Google — **rejects** with a structured `Error { code, status, message, details }` rather than returning an error value; `code / status` are `0 / "TRANSPORT"` for non-API failures (DNS, TLS, timeout) that never reached Google at all. Wrap calls in `try/catch` when you need to distinguish “the resource doesn't exist” from “the network is down.”

A first call — list the Cloud Storage buckets in a project (safe, read-only, and a good way to prove ADC actually resolved an identity):

```
const g = cloud.google({ project: "my-gcp-project" });
try {
  const r = await g.storage().listBuckets({ project: "my-gcp-project" });
  runtime.log(`ok: ${r.items ?? []}.length} bucket(s)`);
} catch (e: any) {
  runtime.log(`failed: ${e.message} (code=${e.code} status=${e.status})`);
}
```

See §17.2 for the full type signatures of every method below.

storage()

`g.storage()` returns a handle onto **Cloud Storage** (GCS) — Google's object store, the rough equivalent of S3: *buckets* hold *objects* (arbitrary blobs addressed by a string key). Methods:

- `listBuckets(opts: { project })` $\rightarrow$ `Promise<{ items?: [...] }>`
- `getBucket(opts: { bucket })` $\rightarrow$ `Promise<Record<string, unknown>>`
- `createBucket(opts: { project, bucket })` $\rightarrow$ `Promise<Record<string, unknown>>`
- `deleteBucket(opts: { bucket })` $\rightarrow$ `Promise<Record<string, unknown>>` — bucket must be empty
- `listObjects(opts: { bucket, prefix? })` $\rightarrow$ `Promise<{ items?: [...] }>`
- `statObject(opts: { bucket, key })` $\rightarrow$ `Promise<Record<string, unknown>>` — metadata only
- `readObject(opts: { bucket, key })` $\rightarrow$ `Promise<{ bytes: number[] }>`
- `putObject(opts: { bucket, key, body })` $\rightarrow$ `Promise<Record<string, unknown>>` — `body` is a string (UTF-8) or `Uint8Array / ArrayBuffer`
- `deleteObject(opts: { bucket, key })` $\rightarrow$ `Promise<Record<string, unknown>>`

`readObject`'s `{ bytes }` comes back as a **plain JS number array**, not a real `Uint8Array` (it's round-tripped through JSON) — wrap it before doing anything binary with it:

```
const gcs = cloud.google({ project: "p" }).storage();

const r = await gcs.listObjects({ bucket: "my-bucket", prefix: "logs/" });
runtime.log(`${r.items ?? []}.length` object(s) under logs/`);

const obj = await gcs.readObject({ bucket: "my-bucket", key: "logs/a.txt" });
const bytes = new Uint8Array(obj.bytes); // now a real Uint8Array
runtime.log(`read ${bytes.length} byte(s)`);
```

compute()

`g.compute()` returns a handle onto **Compute Engine** — GCP's VM service. Instances live in a **zone** (e.g. `europe-north1-a`), a sub-division of a **region**; most methods need both `project` and `zone`. Methods:

- `listInstances(opts: { project, zone })` → `Promise<{ items?: [...] }>`
- `getInstance(opts: { project, zone, name })` → `Promise<Record<string, unknown>>`
- `createInstance(opts: { project, zone, instance })` → `Promise<Record<string, unknown>>` — `instance` is a raw Compute Engine Instance resource body (`machineType`, `disks`, `networkInterfaces`, ...); returns the (typically long-running) operation resource, not the finished instance
- `deleteInstance(opts: { project, zone, name })` → `Promise<Record<string, unknown>>`
- `startInstance(opts: { project, zone, name })` → `Promise<Record<string, unknown>>`
- `stopInstance(opts: { project, zone, name })` → `Promise<Record<string, unknown>>`
- `listZones(opts: { project })` → `Promise<{ items?: [...] }>`
- `listDisks(opts: { project, zone })` → `Promise<{ items?: [...] }>`

```
const c = cloud.google({ project: "p" }).compute();

const zones = await c.listZones({ project: "p" });
runtime.log(`${zones.items ?? []}.length` zone(s) available`);

const vms = await c.listInstances({ project: "p", zone: "europe-north1-a" });
for (const vm of vms.items ?? []) {
  runtime.log(`${vm.name}: ${vm.status}`);
}
```

iam()

`g.iam()` returns a handle onto **Cloud IAM service accounts** — the robot identities scripts/workloads authenticate as (distinct from human Google accounts). A service account is addressed by its email (`<id>@<project>.iam.gserviceaccount.com`). Methods:

- `listServiceAccounts(opts: { project })` → `Promise<{ accounts?: [...] }>`
- `getServiceAccount(opts: { project, email })` → `Promise<Record<string, unknown>>`
- `createServiceAccount(opts: { project, accountId, displayName? })` → `Promise<Record<string, unknown>>`
- `deleteServiceAccount(opts: { project, email })` → `Promise<Record<string, unknown>>`
- `listKeys(opts: { project, email })` → `Promise<{ keys?: [...] }>`

- `createKey(opts: { project, email })` → `Promise<Record<string, unknown>>` — private key material is returned **only** by this call; capture it immediately, it can't be re-fetched
- `getIamPolicy(opts: { resource })` → `Promise<Record<string, unknown>>`
- `setIamPolicy(opts: { resource, policy })` → `Promise<Record<string, unknown>>` — **replaces** the whole policy; always read-modify-write via `getIamPolicy` first, or you'll clobber existing bindings

```
const iam = cloud.google({ project: "p" }).iam();

const sas = await iam.listServiceAccounts({ project: "p" });
runtime.log(`${(sas.accounts ?? []).length} service account(s)`);

const resource = "projects/p/serviceAccounts/sa@p.iam.gserviceaccount.com";
const policy = await iam.getIamPolicy({ resource });
policy.bindings = [
  ...(policy.bindings as any[] ?? []),
  { role: "roles/iam.serviceAccountUser", members: ["user:me@example.com"] },
];
await iam.setIamPolicy({ resource, policy });
```

secrets()

`g.secrets()` returns a handle onto **Secret Manager** — versioned, encrypted key/value storage for things like DB passwords or API keys. A *secret* is a named container; each write creates a new immutable *version*, addressed by number or the alias `"latest"`. Methods:

- `listSecrets(opts: { project })` → `Promise<{ secrets?: [...] }>` — metadata only, never values
- `getSecret(opts: { project, name })` → `Promise<Record<string, unknown>>`
- `createSecret(opts: { project, name })` → `Promise<Record<string, unknown>>` — creates the empty container (automatic replication); it has no value until you `addSecretVersion`
- `addSecretVersion(opts: { project, name, payload })` → `Promise<Record<string, unknown>>` — `payload` is plaintext; base64-encoded on the wire for you
- `accessSecretVersion(opts: { project, name, version? })` → `Promise<{ value: string }>` — `version` defaults to `"latest"`
- `deleteSecret(opts: { project, name })` → `Promise<Record<string, unknown>>` — deletes the secret and every version

`accessSecretVersion`'s `value` is already base64-**decoded** plaintext — you never see the raw wire-format base64:

```
const s = cloud.google({ project: "p" }).secrets();

await s.createSecret({ project: "p", name: "db-password" });
await s.addSecretVersion({ project: "p", name: "db-password", payload:
"s3cr3t" });

const { value } = await s.accessSecretVersion({ project: "p", name: "db-
password" });
runtime.log(value); // "s3cr3t"
```

call() — REST escape hatch

`storage()` / `compute()` / `iam()` / `secrets()` cover four services; Google Cloud has hundreds. `g.call(opts)` reaches **any** `{api}.googleapis.com` REST endpoint directly — no typed wrapper, no Discovery-document lookup — by giving it the request pieces yourself:

```
{
  api: string;                // subdomain, e.g. "run" for
run.googleapis.com
  version?: string;           // informational only; defaults to "v1" — path
is used as given
  httpMethod?: string;        // defaults to "GET"
  path: string;               // appended to https://{api}.googleapis.com
as-is
  params?: Record<string, string>; // query-string parameters
  body?: unknown;             // JSON-serialisable request body
}
```

Reach for `call()` whenever the service you need isn't one of the four typed ones above — Cloud Run, Pub/Sub, BigQuery, Cloud Functions, whatever REST surface Google publishes. It authenticates exactly like the parent `cloud.google({...})` handle (same ADC / explicit-credentials resolution) and returns the **raw decoded JSON** response body (`{}` for an empty body) — there's no per-service response shape to learn.

```
const g = cloud.google({ project: "my-gcp-project" });

// Same data as compute().listZones(), reached the raw way:
const zones = await g.call({
  api: "compute",
  path: "/compute/v1/projects/my-gcp-project/zones",
});
runtime.log(`${(zones as any).items?.length ?? 0} zone(s)`);

// A service with no typed wrapper above — list Cloud Run services in a region:
const services = await g.call({
  api: "run",
  path: "/v2/projects/my-gcp-project/locations/europe-north1/services",
});
runtime.log(JSON.stringify(services));
```

`call()` rejects with the same structured `Error { code, status, message, details }` as every other method here — plus if `api` or `path` is missing, or `body` isn't JSON-serialisable.

5.14.3 cloud.aws

`cloud.aws(opts?)` is a provider handle onto Amazon Web Services, built on the pure-Go `aws-sdk-go-v2` — no CGO, no vendored AWS CLI, no shelling out. Construction is **synchronous**; every service call it returns is **asynchronous** (a `Promise`).

If you've never touched the AWS SDK before, two concepts carry the whole surface:

- **Region.** Almost everything in AWS is scoped to a region (`us-east-1`, `eu-north-1`, ...) — resources in one region are invisible from another. `region` is a property of the *handle*, not of each call: set it once when you build `cloud.aws({...})` and every service accessor off that handle inherits it. If you omit it, `sercon` falls through to whatever the credential chain

supplies (`AWS_REGION` / `AWS_DEFAULT_REGION` , or a region baked into your `~/.aws/config` profile) — if none of those resolve, calls will fail with a client-side “missing region” error.

- **Credentials.** `cloud.aws` reuses the exact same lookup order the AWS CLI and every official SDK use — the “default credential chain”: environment variables (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_SESSION_TOKEN`), `~/.aws/credentials` + `~/.aws/config` (including named profiles and SSO sessions), `AWS_PROFILE` , and finally an attached instance/task identity via IMDS (EC2 instance role, ECS task role, Lambda execution role). You don’t have to do anything for this to work — just have `aws` configure ’d credentials or an attached role. To skip the chain and supply keys directly (e.g. a throwaway CI credential), pass `credentials: { accessKeyId, secretAccessKey, sessionToken? }` — but note it’s all-or-nothing: if you provide any of the three you must provide both `accessKeyId` and `secretAccessKey` (`cloud.aws` throws synchronously otherwise). `profile` selects a named profile from your local AWS config instead of the process’s default profile.

```
const aws = cloud.aws({ region: "eu-north-1" });
```

That’s the whole handle. Nothing talks to AWS yet — each service is accessed lazily via its own accessor (`aws.s3()` , `aws.sts()` , ...), and only building a *client* for that service, not making a call.

Every service method returns a `Promise` that **rejects with a structured Error** — `{ code, status, message, details }` — on failure, rather than throwing synchronously. `code` is the AWS API error code (e.g. `"AccessDenied"` , `"NoSuchBucket"`), `status` is the HTTP status AWS returned, and `details.fault` says whether AWS blamed the client or itself. A non-API failure (DNS, TLS, connection refused, timeout) still comes back as the same shape, just with `code` / `status` empty/zero and the raw transport error as `message` . Always wrap calls in `try/catch` (or handle the rejection) rather than assuming success.

There is no generic escape hatch here — unlike `cloud.google(...).call()` or `cloud.azure(...).call()` , `cloud.aws(...)` exposes **only** the nine typed services below. This isn’t an oversight: AWS has no single uniform wire protocol the way Google/Azure’s REST APIs do — S3 speaks a REST/XML-ish dialect, EC2/IAM/STS/CloudWatch speak classic query-string+XML, Lambda/SecretsManager/SQS speak different flavors of JSON — so there’s no one generic “make a request” primitive that could paper over all of them. If you need an AWS operation that isn’t listed under one of the nine services, it isn’t reachable from `sercon` yet; see this repo’s “missing `sercon` capabilities” process to request it.

One more thing worth knowing up front: because these are the AWS SDK’s own Go response structs (not hand-written JSON schemas) marshalled straight to JS, **result keys come back in the SDK’s PascalCase**, matching the Go struct field names — `Buckets` , `Reservations` , `LogGroups` , `Account` , and so on — *not* the lowerCamelCase you might expect from a typical REST API. Keep that in mind reading every example below and when destructuring results yourself.

The traditional “who am I, and is this working at all” first call is `sts().getCallerIdentity` , since it needs no arguments and no permissions beyond the ones every principal implicitly has:


```
const aws = cloud.aws({ region: "eu-north-1" });
try {
  const id = await aws.sts().getCallerIdentity({});
  runtime.log(`account=${id.Account} arn=${id.Arn} userId=${id.UserId}`);
} catch (e: any) {
  runtime.log(`sts.getCallerIdentity failed: ${e.code} ${e.status} ${e.message}`);
}
```

See §17.2 for the full typed signatures of every method below.

s3()

Amazon S3 is AWS’s object store — buckets holding arbitrary blobs (“objects”) addressed by key, not a filesystem or database.

- `listBuckets(opts?: Record<string, never>)`
- `createBucket(opts: { bucket })`
- `deleteBucket(opts: { bucket })` — the bucket must already be empty
- `listObjects(opts: { bucket; prefix? })`
- `headObject(opts: { bucket; key })` — metadata only, no body
- `getObject(opts: { bucket; key })` → `{ bytes }`
- `putObject(opts: { bucket; key; body })` — body is a string (UTF-8) or `Uint8Array` / `ArrayBuffer`
- `deleteObject(opts: { bucket; key })`

`getObject` is the one method here with a special return shape: rather than the SDK’s raw struct, you get back `{ bytes }` where `bytes` is a plain JS number array (not a real `Uint8Array`) — wrap it before treating it as binary:

```
const s3 = aws.s3();
try {
  const { bytes } = await s3.getObject({ bucket: "my-bucket", key: "logs/
app.log" });
  const data = new Uint8Array(bytes);
  runtime.log(`downloaded ${data.length} bytes`);
} catch (e: any) {
  runtime.log(`getObject failed: ${e.code} (${e.status}): ${e.message}`);
}
```

```
const list = await s3.listObjects({ bucket: "my-bucket", prefix: "logs/" });
for (const obj of list.Contents ?? []) {
  runtime.log(`${obj.Key} (${obj.Size} bytes)`);
}
```

ec2()

EC2 (Elastic Compute Cloud) is AWS’s virtual machine service — “instances” you launch from a machine image, plus the block-storage “volumes” attached to them.

- `describeInstances(opts?: { instanceIds? })`
- `runInstances(opts: { imageId; instanceType; minCount?: maxCount? })` — `minCount` / `maxCount` default to 1 when omitted (matching EC2’s own default)
- `terminateInstances(opts: { instanceIds })`

- `startInstances(opts: { instanceIds })`
- `stopInstances(opts: { instanceIds })`
- `describeVolumes(opts?: { volumeIds? })`
- `describeAvailabilityZones(opts?: Record<string, never>)`

```
const ec2 = aws.ec2();
const out = await ec2.describeInstances({});
for (const res of out.Reservations ?? []) {
  for (const inst of res.Instances ?? []) {
    runtime.log(`${inst.InstanceId} ${inst.State?.Name} ${inst.InstanceType}`);
  }
}
```

```
try {
  const zones = await ec2.describeAvailabilityZones({});
  runtime.log(`zones: ${zones.AvailabilityZones ?? []}.map((z: any) =>
z.ZoneName).join(", ")`);
} catch (e: any) {
  runtime.log(`describeAvailabilityZones failed: ${e.message}`);
}
```

iam()

IAM (Identity and Access Management) is AWS's account-wide authorization service — the users, roles, and policies that decide who can do what. Unlike most AWS services, IAM is **global**, not per-region.

- `listUsers(opts?: Record<string, never>)`
- `getUser(opts?: { userName? })` — omit `userName` to get the calling principal's own user
- `listRoles(opts?: Record<string, never>)`
- `getRole(opts: { roleName })`
- `listPolicies(opts?: Record<string, never>)`
- `createUser(opts: { userName })`
- `deleteUser(opts: { userName })`
- `attachUserPolicy(opts: { userName; policyArn })`

```
const iam = aws.iam();
const roles = await iam.listRoles({});
runtime.log(`roles: ${roles.Roles ?? []}.map((r: any) => r.RoleName).join(", ")`);
```

```
try {
  await iam.createUser({ userName: "ci-deploy" });
  await iam.attachUserPolicy({
    userName: "ci-deploy",
    policyArn: "arn:aws:iam::aws:policy/ReadOnlyAccess",
  });
  runtime.log("ci-deploy user created and policy attached");
} catch (e: any) {
  runtime.log(`iam setup failed: ${e.code}: ${e.message}`);
}
```

secretsmanager()

Secrets Manager stores and versions sensitive values (DB passwords, API keys) so you don't have to bake them into config files or environment variables.

- `listSecrets(opts?: Record<string, never>)`
- `describeSecret(opts: { secretId })` — metadata only, no value
- `createSecret(opts: { name; secretString? })`
- `getSecretValue(opts: { secretId }) → { value }`
- `putSecretValue(opts: { secretId; secretString })`
- `deleteSecret(opts: { secretId })`

`getSecretValue` is the method to reach for actual credential material — it hands back only the decoded plaintext, never the ARN/version metadata that the raw SDK response would also carry:

```
const sm = aws.secretsmanager();
try {
  const { value } = await sm.getSecretValue({ secretId: "prod/db/password" });
  runtime.log(`secret length: ${value.length}`);
} catch (e: any) {
  runtime.log(`getSecretValue failed: ${e.code}: ${e.message}`);
}
```

```
await sm.createSecret({ name: "prod/api/key", secretString: "s3cr3t-value" });
await sm.putSecretValue({ secretId: "prod/api/key", secretString: "rotated-value" });
```

sts()

STS (Security Token Service) issues and inspects temporary credentials — the “who am I” and “become this role” API.

- `getCallerIdentity(opts?: Record<string, never>) → the identity behind whatever credentials cloud.aws(...) resolved (Account, Arn, UserId)` — no secrets in the response
- `assumeRole(opts: { roleArn; roleSessionName; durationSeconds? }) → temporary AccessKeyId / SecretAccessKey / SessionToken for the assumed role`
- `getSessionToken(opts?: { durationSeconds? }) → temporary credentials for the current principal`

`durationSeconds` is omitted from the request entirely unless you pass a positive value (AWS's own minimum is 900 seconds).

```
const sts = aws.sts();
const id = await sts.getCallerIdentity({});
runtime.log(`running as ${id.Arn}`);
```

```
try {
  const creds = await sts.assumeRole({
    roleArn: "arn:aws:iam::123456789012:role/deploy",
    roleSessionName: "sercon-deploy",
    durationSeconds: 900,
```

```
});
// creds.Credentials.{AccessKeyId,SecretAccessKey,SessionToken} – never log
these.
runtime.log("assumed role ok");
} catch (e: any) {
  runtime.log(`assumeRole failed: ${e.code}: ${e.message}`);
}
```

Note: `assumeRole` / `getSessionToken` results carry live AWS credentials in their response — treat `runtime.log`-ing them the way you’d treat logging a password.

lambda()

Lambda runs your code on-demand without a server to manage — you upload a “function” (a zip or an S3 pointer), and AWS invokes it per-event.

- `listFunctions(opts?: Record<string, never>)`
- `getFunction(opts: { functionName })`
- `invoke(opts: { functionName; payload? }) → { statusCode, payload, functionError?, executedVersion? }`
- `createFunction(opts: { functionName; role; runtime; handler; zipFile?; s3Bucket?; s3Key? })`
— `zipFile` is a `string` / `Uint8Array` / `ArrayBuffer` of the deployment package; alternatively point at an existing package with `s3Bucket` / `s3Key`
- `deleteFunction(opts: { functionName })`

`invoke` hand-builds its result rather than round-tripping the raw SDK struct: `payload` is the invoked function’s JSON response **as a string** (parse it yourself with `JSON.parse` if you expect JSON back), and `functionError` is only present when the function itself threw.

```
const lambda = aws.lambda();
try {
  const res = await lambda.invoke({
    functionName: "my-function",
    payload: { ping: true },
  });
  runtime.log(`status=${res.statusCode} functionError=${res.functionError ??
"none"}`);
  const body = JSON.parse(res.payload);
  runtime.log(`response: ${JSON.stringify(body)}`);
} catch (e: any) {
  runtime.log(`invoke failed: ${e.code}: ${e.message}`);
}
```

```
const fns = await lambda.listFunctions({});
runtime.log(`functions: ${fns.Functions ?? []}.map((f: any) =>
f.FunctionName).join(", ")`);
```

sqs()

SQS (Simple Queue Service) is a message queue — producers `sendMessage`, consumers `receiveMessage` and then `deleteMessage` once they’ve processed it (SQS doesn’t remove a message just because you read it).

- `listQueues(opts?: { prefix? })`
- `createQueue(opts: { queueName })`
- `deleteQueue(opts: { queueUrl })`
- `sendMessage(opts: { queueUrl; messageBody })`
- `receiveMessage(opts: { queueUrl; maxMessages? })`
- `deleteMessage(opts: { queueUrl; receiptHandle })` — `receiptHandle` comes from the message you got back from `receiveMessage`, not the message id
- `getQueueAttributes(opts: { queueUrl; attributeNames? })`

```
const sqs = aws.sqs();
const queue = await sqs.createQueue({ queueName: "my-queue" });
await sqs.sendMessage({ queueUrl: queue.QueueUrl, messageBody: "hello" });

const received = await sqs.receiveMessage({ queueUrl: queue.QueueUrl, maxMessages:
5 });
for (const msg of received.Messages ?? []) {
  runtime.log(`body=${msg.Body}`);
  await sqs.deleteMessage({ queueUrl: queue.QueueUrl, receiptHandle:
msg.ReceiptHandle });
}
```

cloudwatch()

CloudWatch is AWS's metrics service — numeric time series (CPU%, request count, custom app metrics) organized under a “namespace”.

- `listMetrics(opts?: { namespace?; metricName? })`
- `getMetricData(opts)` — **pass-through**
- `getMetricStatistics(opts)` — **pass-through**
- `describeAlarms(opts?: { alarmNames? })`
- `putMetricData(opts)` — **pass-through**, always resolves to `{}`

The three pass-through methods are different from every other call in this guide: their `opts` object is **not** a small hand-mapped shape — it's JSON-round-tripped directly into the AWS SDK's own Go input struct, so it must be shaped like that struct, **PascalCase keys and all** (`Namespace`, `MetricData`, `MetricName`, `StartTime`, ...), not the lowerCamelCase used everywhere else in this API.

```
const cw = aws.cloudwatch();
try {
  await cw.putMetricData({
    Namespace: "MyApp/Recon",
    MetricData: [
      { MetricName: "ScriptRuns", Value: 1, Unit: "Count", Timestamp: new
Date().toISOString() },
    ],
  });
  runtime.log("putMetricData ok");
} catch (e: any) {
  runtime.log(`putMetricData failed: ${e.code}: ${e.message}`);
}
```

```
const stats = await cw.getMetricStatistics({
  Namespace: "AWS/EC2",
  MetricName: "CPUUtilization",
  Dimensions: [{ Name: "InstanceId", Value: "i-0123456789abcdef0" }],
  StartTime: new Date(Date.now() - 3600_000).toISOString(),
  EndTime: new Date().toISOString(),
  Period: 300,
  Statistics: ["Average"],
});
for (const p of stats.Datapoints ?? []) {
  runtime.log(`${p.Timestamp} avg=${p.Average}`);
}
```

cloudwatchlogs()

CloudWatch Logs is where most AWS services (and your own apps, if instrumented) ship their log lines — organized into “log groups” (one per app/service) containing “log streams” (one per instance/invoke). This is usually the most useful service here for recon: find the group, tail recent events, or run a full CloudWatch Logs Insights query.

- `describeLogGroups(opts?: { prefix? })`
- `describeLogStreams(opts: { logGroupName })`
- `getLogEvents(opts: { logGroupName; logStreamName; limit? })`
- `filterLogEvents(opts: { logGroupName; filterPattern? })`
- `startQuery(opts: { logGroupName; queryString; startTime; endTime })` — `startTime / endTime` here are **epoch seconds** (unlike `getLogEvents / filterLogEvents`, whose timestamps are epoch **milliseconds**)
- `getQueryResults(opts: { queryId })`

A log-tail: find the group, find its most recent stream, pull the latest events.

```
const logs = aws.cloudwatchlogs();
const groups = await logs.describeLogGroups({ prefix: "/aws/lambda/my-
function" });
const group = (groups.LogGroups ?? [])[0];
if (group) {
  const streams = await logs.describeLogStreams({ logGroupName:
group.LogGroupName });
  const latest = (streams.LogStreams ?? [])[0];
  if (latest) {
    const events = await logs.getLogEvents({
      logGroupName: group.LogGroupName,
      logStreamName: latest.LogStreamName,
      limit: 50,
    });
    for (const e of events.Events ?? []) {
      runtime.log(`${e.Timestamp} ${e.Message}`);
    }
  }
}
```

For a cross-stream search, `filterLogEvents` is usually more useful than walking streams by hand:

```
const hits = await logs.filterLogEvents({
  logGroupName: "/aws/lambda/my-function",
  filterPattern: "ERROR",
});
runtime.log(`${hits.Events ?? []}.length} matching log line(s)`);
```

For anything beyond a simple substring filter (aggregation, field extraction, stats over a time range), use Logs Insights via `startQuery / getQueryResults` — the query itself runs asynchronously on AWS's side, so poll `getQueryResults` until `status` reads "Complete":

```
const nowSec = Math.floor(Date.now() / 1000);
const started = await logs.startQuery({
  logGroupName: "/aws/lambda/my-function",
  queryString: "fields @timestamp, @message | filter @message like /ERROR/ | limit 20",
  startTime: nowSec - 3600,
  endTime: nowSec,
});
let results = await logs.getQueryResults({ queryId: started.QueryId });
while (results.Status === "Running" || results.Status === "Scheduled") {
  await new Promise((r) => setTimeout(r, 1000));
  results = await logs.getQueryResults({ queryId: started.QueryId });
}
runtime.log(`query status=${results.Status}, rows=${(results.Results ?? []).length}`);
```

5.14.4 cloud.azure

> **PROVISIONAL — unverified against a live Azure account.** `cloud.azure` > is built against `azure-sdk-for-go` and covered by `mock-server` > (`httptest`) tests, but no one on this project has a real Azure > subscription to run it against, and CI has no Azure credentials either. > Every method below is expected to work — the request shapes come > straight from the generated SDK clients — but nothing here has been > confirmed against a real `management.azure.com`, blob account, or Key > Vault. Treat every example as illustrative, not proven. If you run this > against a real subscription and it misbehaves, that's useful signal: > please report back what broke.

`cloud.azure` is sercon's Microsoft Azure provider: pure Go, no CGO, built on the official `azure-sdk-for-go` client libraries. If you've never scripted against Azure before, the two ideas to hold onto are **credentials** and **the two-plane model** — everything else follows from those.

Credentials. Azure SDKs authenticate through a *token credential*, not a raw API key.

`cloud.azure` gives you two ways to get one:

- Leave `tenantId / clientId / clientSecret` out entirely and it falls back to `DefaultAzureCredential` — Azure's standard credential chain, which tries (in order, roughly) environment variables in the `azidentity`-recognized shape, a managed identity if the process is running on an Azure host, and your local `az login` session. This is the right default for anything running on an Azure VM/App Service/Container App, or on a developer machine that's already run `az login`.

- Pass `{ tenantId, clientId, clientSecret }` explicitly to use a service-principal (client-secret) credential — the usual choice for CI or any environment without an interactive `az login` or managed identity available.

Nothing under `clientSecret` is ever logged by `sercon`, in errors or otherwise.

The two-plane model. Azure splits every service into a *management plane* and a *data plane*, and `cloud.azure`’s shape mirrors that split exactly:

- **ARM (management plane)** — Azure Resource Manager, the API that creates/lists/deletes the resources themselves: resource groups, VMs, and (generically) anything else ARM knows about. ARM is **subscription-scoped**: every ARM call needs a subscription id, which `cloud.azure` resolves from `opts.subscriptionId` or the `AZURE_SUBSCRIPTION_ID` env var — and throws if neither is set. ARM is exposed as `resourceGroups()`, `compute()`, `resources()`, and the generic `call()` escape hatch.
- **Data plane** — the services that do things *inside* a resource once it exists: reading/writing blobs inside a storage account, reading secrets out of a Key Vault. Data-plane accessors take the resource’s own endpoint URL directly (`blob(accountUrl)`, `keyvaultSecrets(vaultUrl)`) and need **no subscription id at all** — the URL you pass in *is* the target, full stop.

If you’re new to Azure: a **subscription** is the billing/administrative boundary (roughly: an AWS account or a GCP project), and a **resource group** is a folder-like container inside a subscription that most resources live in and get deleted together with.

Construction is synchronous and cheap — it just resolves config, it does not make a network call:

```
const az = cloud.azure({ subscriptionId:
  "00000000-0000-0000-0000-000000000000" });
```

Every service method after that is `async` — call it with `await` — and on failure rejects with a structured error carrying `{ code, status, message, details }` (`code / status` are `"" / 0` for non-API failures like DNS, TLS, or a credential/token problem — only a real ARM/data-plane HTTP error fills them in). A missing subscription (no `subscriptionId` and no `AZURE_SUBSCRIPTION_ID`) throws as soon as an ARM method or `call()` is invoked; `blob()` / `keyvaultSecrets()` have no such requirement.

One casing wrinkle worth knowing up front: ARM services and `keyvaultSecrets()` return lowercase-camelCase keys (`id`, `name`, `properties`, ...) straight from the SDK’s own JSON encoding, but `blob()` results come back **PascalCase** (`Name`, `Properties`, `Deleted`, ...) — the Storage Blob wire protocol is XML under the hood, so there’s no custom JSON marshalling to normalize the casing. Don’t assume one casing convention carries across every service.

See §17.2 for the full type signatures of every method below.

resourceGroups()

A resource group is Azure’s basic organizational container — most ARM resources belong to exactly one, and deleting a group cascades to everything inside it. `resourceGroups()` is subscription-scoped (via `cfg.subscriptionId / AZURE_SUBSCRIPTION_ID`) and gives you:

- `list(opts?)` → `Promise<{ value: [...] }>` — every resource group in the subscription, paged through automatically.
- `get(opts: { name })` → `Promise<{...}>` — a single group's details.
- `create(opts: { name, location })` → `Promise<{...}>` — create-or-update.
- `delete(opts: { name })` → `Promise<{}>` — starts the delete and waits for the long-running operation to finish before resolving.

```
const groups = await az.resourceGroups().list();
runtime.log(`resource groups: ${groups.value?.length ?? 0}`);
```

```
const rg = await az.resourceGroups().get({ name: "my-rg" });
runtime.log(`location: ${rg.location}`);
```

compute()

`compute()` wraps Azure's virtual-machine API (ARM's `Microsoft.Compute` provider) — start/stop/list/inspect VMs. Also subscription-scoped.

- `listVirtualMachines(opts?: { resourceGroup? })` → `Promise<{ value: [...] }>` — omit `resourceGroup` to list every VM in the subscription; supply it to scope the list to one group.
- `getVirtualMachine(opts: { resourceGroup, name })` → `Promise<{...}>`
- `start(opts: { resourceGroup, name })` → `Promise<{}>`
- `powerOff(opts: { resourceGroup, name })` → `Promise<{}>`
- `deallocate(opts: { resourceGroup, name })` → `Promise<{}>` — like `powerOff`, but also releases the underlying compute allocation (so you stop being billed for the VM size, not just for it running).
- `delete(opts: { resourceGroup, name })` → `Promise<{}>`

All of `start` / `powerOff` / `deallocate` / `delete` are long-running ARM operations under the hood; `cloud.azure` polls until each one completes before resolving, so `await` genuinely means “done”, not just “accepted”.

```
const vms = await az.compute().listVirtualMachines({ resourceGroup: "my-rg" });
runtime.log(`VMs in my-rg: ${vms.value?.length ?? 0}`);
```

```
const vm = await az.compute().getVirtualMachine({ resourceGroup: "my-rg", name:
"web-1" });
runtime.log(`vm state fields: ${Object.keys(vm)}`);
```

resources()

`resources()` is ARM's *generic* resource API — useful when you want to work across resource types without pulling in a type-specific service like `compute()`. It's the right tool when you already know a resource's ID (or just want everything in a group regardless of type) and don't need type-specific verbs like `start` / `powerOff`.

- `listByResourceGroup(opts: { resourceGroup })` → `Promise<{ value: [...] }>` — every resource in the group, any type.
- `getById(opts: { resourceId, apiVersion })` → `Promise<{...}>` — fetch by the resource's fully-qualified ARM ID. Unlike the typed services, the generic resources API has no built-in

default API version per resource type, so you must supply `apiVersion` yourself (check the resource provider's REST API reference for the right value).

```
const all = await az.resources().listByResourceGroup({ resourceGroup: "my-rg" });
runtime.log(`resources in my-rg: ${all.value?.length ?? 0}`);
```

```
const site = await az.resources().getById({
  resourceId: "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/
my-rg/providers/Microsoft.Web/sites/my-site",
  apiVersion: "2023-12-01",
});
runtime.log(site.name);
```

call() — ARM REST escape hatch

`resourceGroups()` / `compute()` / `resources()` cover three ARM providers; `call()` reaches **any** ARM resource provider by talking directly to `management.azure.com` — useful the moment you need a resource type (App Service, Cosmos DB, Storage accounts, anything) that doesn't have a dedicated service above. It shares the same subscription/credential resolution as the typed services (so it also throws if no subscription is configured), and takes:

```
call(opts: {
  path: string; // ARM path, e.g. "/subscriptions/{id}/
resourceGroups/..."
  apiVersion: string; // required — ARM has no per-path default
  method?: string; // default "GET"
  params?: Record<string, string>;
  body?: unknown; // JSON-serialisable request body
}): Promise<unknown>
```

```
// List every Microsoft.Web/sites (App Service) resource in a resource group.
const sites: any = await az.call({
  path: "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/my-rg/
providers/Microsoft.Web/sites",
  apiVersion: "2023-12-01",
});
runtime.log(`sites: ${sites.value?.length ?? 0}`);
```

blob(accountUrl)

`blob(accountUrl)` is Azure Blob Storage's data plane. Unlike the ARM services above, this accessor takes the storage account's own URL (e.g.

`https://myaccount.blob.core.windows.net`) directly — there's no subscription involved at all; the URL you hand it is the target account, authenticated with the same credential chain as everything else. Remember the PascalCase note above: results here come back as `Name` / `Properties` / etc., not lowercase.

- `listContainers(opts?)` → `Promise<{ value: [...] }>`
- `listBlobs(opts: { container })` → `Promise<{ value: [...] }>` — flat listing (no folder/delimiter hierarchy).

- `download(opts: { container, blob }) → Promise<{ bytes: number[] }>` — `bytes` is a plain JS array of byte values, not a real typed array; wrap it yourself:
`new Uint8Array(res.bytes)`.
- `upload(opts: { container, blob, body }) → Promise<{...}>` — `body` accepts a string (sent as UTF-8) or raw bytes (`Uint8Array` / `ArrayBuffer`); uploads as a single block blob.
- `deleteBlob(opts: { container, blob }) → Promise<{}>`

```
const blob = az.blob("https://myaccount.blob.core.windows.net");
const containers = await blob.listContainers();
runtime.log(`containers: ${containers.value?.length ?? 0}`);
```

```
const blob = az.blob("https://myaccount.blob.core.windows.net");
const res = await blob.download({ container: "logs", blob: "2026-07-01.log" });
const bytes = new Uint8Array(res.bytes);
runtime.log(`downloaded ${bytes.length} bytes`);
```

keyvaultSecrets(vaultUrl)

`keyvaultSecrets(vaultUrl)` is Azure Key Vault's secrets data plane — pass the vault's own URL (e.g. `https://myvault.vault.azure.net`) directly, same no-subscription pattern as `blob()`. Results here are lowercase-camelCase (`id`, `attributes`, `contentType`, ...), matching the ARM services rather than `blob()`'s PascalCase.

- `listSecrets(opts?) → Promise<{ value: [...] }>` — metadata only (names, attributes, tags); values are never included in a list, by Key Vault's own design.
- `getSecret(opts: { name }) → Promise<{ value: string }>` — the latest version's plaintext value.
- `setSecret(opts: { name, value }) → Promise<{...}>` — creates a new version of the named secret.
- `deleteSecret(opts: { name }) → Promise<{}>` — deletes all versions.

Secret values are never logged by sercon anywhere along this path — only ever handed back to your script to do with as you choose. Be equally careful in your own `runtime.log` calls.

```
const kv = az.keyvaultSecrets("https://myvault.vault.azure.net");
const { value } = await kv.getSecret({ name: "db-password" });
// don't log `value` — pass it straight to whatever needs it
```

```
const kv = az.keyvaultSecrets("https://myvault.vault.azure.net");
try {
  const secrets = await kv.listSecrets();
  runtime.log(`secrets in vault: ${secrets.value?.length ?? 0} (metadata only)`);
} catch (e: any) {
  runtime.log(`keyvaultSecrets.listSecrets failed: ${e.message} (code=${e.code} status=${e.status})`);
}
```

5.14.5 Recipes

“Who am I / what’s here” — a first call per provider. A cheap read to confirm credentials and orient yourself:

```
// AWS: caller identity + a bucket count
const aws = cloud.aws({ region: "eu-north-1" });
const who = await aws.sts().getCallerIdentity({});
runtime.log(`aws account ${who.Account} as ${who.Arn}`);
runtime.log(`buckets: ${await aws.s3().listBuckets().Buckets?.length ?? 0}`);

// Google: buckets in a project
const g = cloud.google({ project: "my-project" });
runtime.log(`gcs buckets: ${await g.storage().listBuckets({ project: "my-project" })).items?.length ?? 0}`);

// Azure (PROVISIONAL): resource groups in a subscription
const az = cloud.azure({ subscriptionId:
runtime.env.get("AZURE_SUBSCRIPTION_ID") });
runtime.log(`resource groups: ${await
az.resourceGroups().list().value?.length ?? 0}`);
```

Read a secret from each provider's secret store:

```
const awsSecret = (await cloud.aws({ region: "eu-north-1" })
  .secretsmanager().getSecretValue({ secretId: "prod/db" })).value;

const gcpSecret = (await cloud.google({ project: "my-project" })
  .secrets().accessSecretVersion({ project: "my-project", name: "db-
password" })).value;

const azSecret = (await cloud.azure({ subscriptionId: "..." })
  .keyvaultSecrets("https://myvault.vault.azure.net").getSecret({ name: "db-
password" })).value;
```

Tail recent CloudWatch logs (AWS) for a Lambda:

```
const logs = cloud.aws({ region: "eu-north-1" }).cloudwatchlogs();
const r = await logs.filterLogEvents({
  logGroupName: "/aws/lambda/my-fn",
  filterPattern: "ERROR",
});
for (const ev of r.Events ?? []) runtime.log(ev.Message);
```

Reach an untyped API via an escape hatch:

```
// Google: list Cloud Run services (no typed service – use call())
const runSvc = await cloud.google({ project: "my-project" }).call({
  api: "run", version: "v1",
  path: "/apis/serving.knative.dev/v1/namespaces/my-project/services",
});

// Azure (PROVISIONAL): list Web Apps via ARM call()
const sites = await cloud.azure({ subscriptionId: "..." }).call({
  path: "/subscriptions/.../providers/Microsoft.Web/sites",
  apiVersion: "2023-12-01",
});
```

Network-tolerant recon. Cloud calls throw on failure; in a recon script that should degrade rather than abort, catch and log:

```
try {
  const vms = await cloud.aws({ region: "eu-
north-1" }).ec2().describeInstances({});
  runtime.log(`reservations: ${vms.Reservations?.length ?? 0}`);
} catch (e) {
  runtime.log(`ec2 skip: ${e.code || "transport"} - ${e.message}`);
}
```

5.15 mcp

The `mcp` global turns a sercon script into a **Model Context Protocol (MCP) server**: a process that exposes tools, resources, and prompts to an MCP *client* — typically an LLM agent such as Claude Desktop, or any other MCP-speaking host.

`mcp.serve({name, version, instructions??})` returns a handle you register capabilities on, then serve over one of two transports.

> **Two views of the surface.** This section is the *guide*: what each > piece is for and how to use it, with runnable examples. §17.9 is the > generated *reference*: the exact TypeScript signature of every method > below. Use them together.

5.15.1 Concepts

The four primitives. A server exposes any mix of:

Primitive	Registered with	Purpose	Failure surfaces as
Tool	<code>srv.tool({name, description?, inputSchema, outputSchema?, handler})</code>	A callable action (an LLM invokes it with arguments and gets a result back).	An <code>isError: true</code> tool result — a thrown/rejected handler does not crash the server or the call.
Resource	<code>srv.resource({uri, name, mimeType?, read})</code>	A URI-addressed piece of content a client can fetch (a file, a config blob, a generated report).	A protocol-level <code>resources/read</code> error — there is no soft-failure shape for resources.
Resource template	<code>srv.resourceTemplate(name, mimeType?, read)</code>	Like a resource, but for a whole <i>family</i> of URIs matching an RFC 6570 pattern (e.g. <code>db:///table/{id}</code>) instead of one fixed URI.	Same as Resource — a protocol-level <code>resources/read</code> error.
Prompt	<code>srv.prompt({name, description?, arguments?, get})</code>	A named, parameterized conversation-starter	A protocol-level <code>prompts/get</code> error, same as resources.

		template a client can pull.	
--	--	-----------------------------	--

All handlers receive `(args, ctx)` (resource/resource-template read receives `(uri, ctx)`, prompt's get receives `(args, ctx)`), where `ctx` is `{ requestId, clientInfo: { name, version }, progress(progress, total?), log(level, message, data?) }` — see “Progress and logging” below for the last two. Handlers may be sync or async.

Registering and unregistering at runtime. Unlike Phase 1, `tool()` / `resource()` / `resourceTemplate()` / `prompt()` may be called **at any time** — before serving starts, or from inside another handler while clients are already connected. Each has a matching remove method (`srv.removeTool(name)`, `srv.removeResource(uri)`, `srv.removePrompt(name)`) that unregisters by the same name/uri; removing something that was never registered (or already removed) is a silent no-op. Adding or removing a capability after a transport has started fires the matching `list_changed` notification (`tools/list_changed`, `resources/list_changed`, `prompts/list_changed`) to every connected client, so well-behaved clients re-fetch their list rather than caching a stale one. See §5.15.2.5 for a worked example.

The two transports. A handle serves over exactly one transport, chosen by which method you call:

Transport	Call	Platform	Shape
stdio	<code>await srv.stdio()</code>	Unix-only this phase — rejects immediately on Windows with a clear error <code>mcp: stdio() is not supported on (windows ...)</code> ; use <code>listen()</code> there instead.	One peer per process, over stdin/stdout. The shape Claude Desktop and most CLI-launched MCP clients expect.
Streamable HTTP	<code>const h = await srv.listen({port, host?, path?})</code>	Cross-platform.	A plain TCP/HTTP endpoint any number of clients can connect to. Resolves as soon as the listener is bound (not when a client connects) to <code>{ url, stopped, close() }</code> .

Calling either a second time on the same handle throws — one handle serves one transport. `srv.close()` is currently a no-op placeholder: stop an HTTP listener via the `close()` on the handle `listen()` returned; a stdio server stops on its own once the peer disconnects.

The stdout rule (stdio only). MCP over stdio is a *framing* protocol — every byte on stdout must be a JSON-RPC message, or the client’s parser desyncs and the connection is unusable. From the moment `mcp.serve()` is called, `sercon` transparently redirects

`console.*` and `runtime.log` output to `stderr`, so ordinary debug logging in your script can't corrupt the stream. Nothing else needs to change in your script — just don't try to write to `stdout` yourself via a lower-level binding while a `stdio` server is running. This constraint doesn't apply to `listen()` : HTTP has no shared framing channel to protect.

Capability negotiation. During MCP's `initialize` handshake, the SDK advertises `tools / prompts / resources` support based on what's registered at connect time (a server with no prompts doesn't claim prompt support, and so on), plus `logging` support **unconditionally** (available even if a script never calls `ctx.log`) and `resources.subscribe / completions` support **unconditionally** as well — a script that never calls `srv.onSubscribe / srv.completion` still advertises those capabilities; the underlying handlers simply accept every subscribe request or answer “no suggestions” until a script wires up real logic. `serverInfo` reports your `name / version`, and `instructions` (optional) is surfaced to the client as free-text guidance on how to use the server. None of this requires script-side code.

Progress and logging. A handler's `ctx` carries two notification helpers alongside `requestId / clientInfo` :

Method	Signature	Sends	Requires
<code>ctx.progress</code>	<code>(progress: number, total?: number) => Promise<void></code>	<code>notifications /</code> A <code>progress</code> message correlated to this call.	The <i>client's</i> call must have attached a progress token. If it didn't, <code>ctx.progress</code> resolves immediately and sends nothing — this is normal, not an error.
<code>ctx.log</code>	<code>(level: string, message: string, data?: unknown) => Promise<void></code>	<code>notifications /</code> A message <code>log entry (level is one of the standard MCP/ syslog levels: "debug" , "info" , "notice" , "warning" , "error" , "critical" , "alert" , "emergency").</code>	The client must have called <code>session.setLoggingLevel(...)</code> first. Per the MCP spec (and confirmed against the go-sdk), a client that never sets a logging level receives <i>nothing</i> from <code>ctx.log</code> — the call still resolves (it's not an error), the message is just silently dropped. This is client/ SDK behaviour, not a sercon bug; don't assume a log line arrived just because <code>await ctx.log(...)</code> didn't throw.

Both are async and both are best-effort — call them freely from inside a tool/resource/prompt handler without worrying about breaking the handler’s own return value. See §5.15.2.4.

Resource subscriptions. A client can ask to be notified when a specific resource’s content changes (`resources/subscribe`), then unsubscribe later. `sercon` exposes this as three pieces you wire together yourself:

- `srv.onSubscribe(fn)` — `fn(uri)` fires when any client subscribes to `uri`.
- `srv.onUnsubscribe(fn)` — `fn(uri)` fires when any client unsubscribes.
- `srv.resourceUpdated(uri)` — notify every client *currently* subscribed to `uri` that it changed; the SDK tracks the actual subscriber set, this method doesn’t.

The common pattern is **lazy watching**: don’t start watching a resource’s backing data (a file, a DB row, a poll loop) until `onSubscribe` says a client actually cares, and stop when `onUnsubscribe` says none do anymore — see §5.15.2.7.

Argument completion. `srv.completion(fn)` registers one handler for the client’s `completion/complete` request — autocomplete suggestions as a user types a prompt argument or a resource-template URI variable. `fn(ref, argName, partial)` receives a normalized `ref: { type: "prompt" | "resource", name?, uri? }` (which prompt or resource-template is being completed), the argument/variable name, and the text typed so far; return a `string[]` or `{ values?, total?, hasMore? }`. If you never call `srv.completion`, the server still advertises the capability and answers every request with “no suggestions” rather than rejecting it. See §5.15.2.8.

List page size. `mcp.serve({ ..., pageSize })` caps how many tools/resources/prompts/resource templates one `list` response returns before the client must page with a cursor (defaults to the SDK’s built-in 1000). Useful mainly for exercising a client’s pagination handling, or if you register a very large capability set. See §5.15.2.9.

Sampling, elicitation, and roots — server-initiated client calls. Every recipe so far has the client calling *into* the server (a tool call, a resource read, a prompt fetch). A handler can also call back *out* to the client mid-request, via three additions to `ctx`:

Method	Asks the client for	Requires the client to have
<code>ctx.sample(opts)</code>	Run a completion through <i>the client’s own LLM</i> (<code>sampling/createMessage</code>) — not <code>sercon</code> calling an LLM provider directly.	Wired a <code>CreateMessageHandler</code> (i.e. advertised the <code>sampling</code> capability during <code>initialize</code>).
<code>ctx.elicit(opts)</code>	Ask <i>the user</i> , via the client’s UI, to confirm an action or fill in a small form (<code>elicitation/create</code>).	Wired an <code>ElicitationHandler</code> (the <code>elicitation</code> capability).
<code>ctx.roots()</code>	The client’s current list of filesystem/URI roots it’s willing to let the server operate within (<code>roots/list</code>).	Advertised the <code>roots</code> capability (most SDK clients do by default).

All three reject with a clear `"mcp: client does not support <capability>"` error when the connected client hasn’t advertised the matching capability — check for that error rather than

assuming every client supports every capability; none of the three has a silent degrade path. `srv.onRootsChanged(fn)` is the push-based counterpart to `ctx.roots()`: it fires whenever the client’s root set changes, so a server doesn’t have to poll.

Client-dependence. Unlike the recipes above — which are either self-testing (`mcp-server-http.ts`, `mcp-toolbox.ts`, `mcp-bridge.ts` speak raw JSON-RPC to themselves over HTTP) or need no client cooperation beyond the initial handshake — sampling/elicitation/roots only do something interesting against a *real* MCP client capable of answering these mid-call requests. `examples/scripts/mcp-sampling.ts`, `mcp-elicite.ts`, and `mcp-roots.ts` are therefore **not** make demo scripts: they serve over stdio and block until a peer disconnects. They’re exercised by `cmd/sercon/mcp_examples_test.go`, which connects the real go-sdk client with a canned handler standing in for “the client’s LLM” / “the human at the keyboard” / “a client with pre-seeded roots”. See §5.15.2.11–§5.15.2.13.

OAuth — sercon as a resource server, not an authorization server.

`srv.listen({ ..., auth })` turns a Streamable HTTP listener into an OAuth 2.1 *resource server* (RS): it validates bearer tokens (via your `verify` callback) and advertises RFC 9728 protected-resource metadata at `/.well-known/oauth-protected-resource`, so clients know which *authorization server* (AS) to obtain a token from. sercon deliberately does **not** implement dynamic client registration (DCR, RFC 7591) or any token issuance — minting and registering tokens is the AS’s job, not a resource server’s; sercon only ever validates tokens someone else issued. A request with a missing/invalid/expired/under-scoped token never reaches your handlers — the middleware answers it directly with `401` and a `WWW-Authenticate` header pointing at the metadata URL. See §5.15.2.14.

Handler timeout. Every script handler the SDK invokes — a server tool/resource/prompt/completion handler or `auth.verify`, and a client `onSample` / `onElicit` or `auth.getToken` — runs through an on-loop bridge that blocks the SDK’s request goroutine until the handler settles. The `handlerTimeout` option (on `mcp.serve({...})` and every `mcp.connect.*({...})`, in **milliseconds**) caps that wait: if the handler hasn’t settled by the deadline the bridge gives up and returns an error (a tool handler surfaces it as an `isError` result; others as a protocol error), reclaiming the request goroutine. It defaults to **5 minutes** — deliberately generous, so a handler legitimately awaiting `ctx.elicite` (a human) or `ctx.sample` (an LLM) is not cut off — and **0 disables** the timer (the bridge then waits until the request’s own context is cancelled). Note the deadline reclaims the *waiting goroutine*; the handler’s JavaScript keeps running on the event loop until it settles (goja can’t be preempted mid-execution — a runaway *synchronous* script is still the `--timeout` / Run-cancellation job, via `vm.Interrupt`).

5.15.2 Recipes

Each recipe assumes `const srv = mcp.serve({ name: "...", version: "1.0.0" });` has already registered whatever tools it needs (except where the recipe itself is about registration). Full signatures are in §17.9.

5.15.2.1 Expose a sercon capability as an MCP tool

Wrap an existing sercon binding in a tool so an MCP client can invoke it directly — here, a DNS-ish lookup via `net.probe`.

```
const srv = mcp.serve({ name: "recon-tools", version: "1.0.0" });

srv.tool({
  name: "resolve_host",
  description: "resolve a hostname's DNS records",
  inputSchema: {
    type: "object",
    properties: { host: { type: "string" } },
    required: ["host"],
  },
  async handler(args) {
    const result = await net.probe.dns(args.host);
    return { structuredContent: result };
  },
});
```

Notes

- Returning `{ structuredContent }` lets a client consume the result as structured data (validated against `outputSchema`, if you supply one) instead of parsing text.
- A handler that throws (e.g. the hostname doesn't resolve) becomes an `isError: true` tool result — the client sees a failed call, not a crashed server.
- This scales: `examples/scripts/mcp-toolbox.ts` wraps five different sercon globals (`net.http`, `db.sqlite`, `image`, `crypto`, `fs`) as five tools on one handle — “drop one binary in front of an LLM client and it already has a toolbox” is the same pattern as this recipe, just repeated.

5.15.2.2 Run a stdio MCP server for Claude Desktop

Claude Desktop (and similar hosts) launch an MCP server as a subprocess and speak to it over stdio. Point the client config at the script and call `srv.stdio()` :

```
// server.ts
const srv = mcp.serve({ name: "my-tools", version: "1.0.0" });

srv.tool({
  name: "add",
  description: "add two numbers",
  inputSchema: {
    type: "object",
    properties: { a: { type: "number" }, b: { type: "number" } },
    required: ["a", "b"],
  },
  async handler(args) {
    return String(args.a + args.b);
  },
});

await srv.stdio();
```

Add it to the client's MCP server config (e.g. Claude Desktop's `claude_desktop_config.json`, under `mcpServers`):

```
{
  "mcpServers": {
    "my-tools": {
      "command": "sercon",
      "args": ["server.ts"]
    }
  }
}
```

Notes

- Unix-only this phase — on Windows, `srv.stdio()` 's promise rejects immediately; use `listen()` there instead.
- Any `console.log` / `runtime.log` calls in `server.ts` land on `stderr`, never `stdout` — safe to leave debug logging in place.
- The script blocks on `await srv.stdio()` until the client disconnects; that's expected — the client owns the process lifecycle.

5.15.2.3 Serve MCP over HTTP

For a long-running server reachable by more than one client (or on Windows), bind the Streamable HTTP transport instead:

```
const srv = mcp.serve({ name: "my-tools", version: "1.0.0" });

srv.tool({
  name: "add",
  description: "add two numbers",
  inputSchema: {
    type: "object",
    properties: { a: { type: "number" }, b: { type: "number" } },
    required: ["a", "b"],
  },
  async handler(args) {
    return String(args.a + args.b);
  },
});

const h = await srv.listen({ port: 38080 });
runtime.log("MCP server listening at", h.url); // e.g. http://127.0.0.1:38080/mcp

// ... later, on shutdown:
await h.close();
```

Notes

- `listen()` 's promise resolves as soon as the port is bound, not when a client connects; `h.stopped` resolves once the HTTP server has fully stopped, and `h.close()` returns that same promise after starting a graceful shutdown.
- Point any Streamable-HTTP-capable MCP client at `h.url` the endpoint speaks the standard `initialize` → `notifications/initialized` → `request/response` handshake over POST, with SSE-framed responses — see `examples/scripts/mcp-server-http.ts` for a from-scratch client walking through that handshake by hand.

- `path` defaults to `/mcp` and `host` to `127.0.0.1` override either in the `listen()` options if you need to bind wider or mount elsewhere.

5.15.2.4 Report progress and stream log lines from a tool

A long-running tool can keep the client informed with periodic progress updates and structured log lines while it works.

```
srv.tool({
  name: "process_batch",
  description: "process a batch of items, reporting progress as it goes",
  inputSchema: {
    type: "object",
    properties: { items: { type: "array", items: { type: "string" } } },
    required: ["items"],
  },
  async handler(args: any, ctx) {
    const items: string[] = args.items;
    for (let i = 0; i < items.length; i++) {
      await ctx.log("info", `processing ${items[i]}`, { index: i });
      // ... do the actual work for items[i] ...
      await ctx.progress(i + 1, items.length);
    }
    return `processed ${items.length} items`;
  },
});
```

Notes

- `ctx.progress` only reaches the client if *its* call included a progress token — if not, the promise still resolves, it just sends nothing. Most MCP clients attach one automatically for long tool calls.
- `ctx.log` reaches the client only after it calls `session.setLogLevel(...)` — a client that never opts in receives no log lines at all, silently. Don't rely on `ctx.log` for anything the tool's own return value needs to convey; treat it as an optional debugging/progress channel.
- Both calls are cheap to await in sequence; there's no need to batch them.

5.15.2.5 Grow or shrink your tool set at runtime

Register or unregister tools/resources/prompts after `stdio()` / `listen()` has already started — useful for servers whose capability set depends on something discovered at runtime (an auth token, a plugin directory, ...).

```
const srv = mcp.serve({ name: "dynamic-tools", version: "1.0.0" });
await srv.listen({ port: 38090 });

// later, once some condition is met (e.g. a config file appears):
srv.tool({
  name: "beta_feature",
  description: "only available once enabled",
  inputSchema: { type: "object" },
  handler: () => "beta feature ran",
});
// connected clients receive a tools/list_changed notification here
```

```
// and to retract it again:
srv.removeTool("beta_feature");
// connected clients receive another tools/list_changed notification
```

Notes

- The same pattern applies to `srv.resource` / `srv.removeResource` and `srv.prompt` / `srv.removePrompt` — each pair fires its own `list_changed` notification (`resources/list_changed`, `prompts/list_changed`).
- Removing a name/uri that was never registered (or already removed) is a silent no-op — no need to track what's currently registered yourself.
- This works whether or not a transport has started; registering before `stdio()` / `listen()` is still the normal case for a static tool set.

5.15.2.6 Expose a family of resources with a URI template

Use `srv.resourceTemplate` instead of `srv.resource` when clients should be able to read any of a whole family of URIs (e.g. rows in a table) rather than one fixed resource.

```
srv.resourceTemplate({
  uriTemplate: "db://{table}/{id}",
  name: "row",
  mimeType: "application/json",
  async read(uri) {
    // uri is the concrete URI the client asked for, e.g. "db:///users/42"
    const [, , table, id] = uri.match(/^db:\\\\(?:[^\s/]+)\\\/(?:[^\s/]+)$/)? ? [];
    return { text: JSON.stringify({ table, id }) };
  },
});
```

A client reading `db:///users/42` invokes `read("db:///users/42", ctx)` — the template string itself (`db://{table}/{id}`) is only ever seen during `resources/templates/list`, never passed to `read`.

Notes

- `uriTemplate` follows RFC 6570 (the same template syntax the MCP spec uses) — `sercon` passes it through to the SDK as-is, it isn't parsed or validated by `sercon` itself.
- Like `srv.resource`, a `read` that throws or returns something other than `{text} / {blob}` is a protocol-level error, not a soft `isError` result.

5.15.2.7 Push updates to subscribed clients (lazy-watch pattern)

Combine `srv.onSubscribe` / `srv.onUnsubscribe` with `srv.resourceUpdated` to watch a resource's backing data only while at least one client cares about it.

```
const watchers = new Map<string, () => void>();

function startWatching(uri: string) {
  // stand-in for "start polling a file/DB row/etc for changes"
  const interval = setInterval(() => {
    srv.resourceUpdated(uri); // notify every currently-subscribed client
  }, 5000);
```

```

    watchers.set(uri, () => clearInterval(interval));
  }

  srv.onSubscribe((uri) => {
    if (!watchers.has(uri)) startWatching(uri);
  });

  srv.onUnsubscribe((uri) => {
    watchers.get(uri)?.();
    watchers.delete(uri);
  });

  srv.resource({
    uri: "config://app",
    name: "App config",
    mimeType: "application/json",
    read: async () => ({ text: JSON.stringify({ debug: true }) }),
  });

```

Notes

- `onSubscribe` / `onUnsubscribe` are fire-and-forget: their return value (and any thrown error) is ignored — a subscribe/unsubscribe request always succeeds from the client's point of view, this hook just observes it.
- `srv.resourceUpdated(uri)` is safe to call even if no client is currently subscribed to `uri` (it's a no-op fan-out to an empty set) — no need to track the subscriber count yourself, the SDK already does.
- Only one `onSubscribe` and one `onUnsubscribe` callback are held at a time; registering again replaces the previous one.

5.15.2.8 Autocomplete a prompt argument

Suggest values for a prompt's argument as a client's user types, via `srv.completion`.

```

const users = ["alice", "alicia", "bob", "carol"];

srv.prompt({
  name: "greet",
  description: "greet a user by name",
  arguments: [{ name: "user", required: true }],
  get: async (args) => ({
    messages: [{ role: "user", content: { type: "text", text: `Say hello to ${args.user}.` } }],
  }),
});

srv.completion((ref, argName, partial) => {
  if (ref.type === "prompt" && ref.name === "greet" && argName === "user") {
    return users.filter((u) => u.startsWith(partial));
  }
  return [];
});

```

Notes

- `ref` is normalized regardless of what's being completed, but `name` and `uri` are **both always present** on the object — for a prompt argument `ref.type` is "prompt" and `ref.name` is set (`ref.uri` is ""); for a resource-template URI variable `ref.type` is "resource" and `ref.uri` is set (`ref.name` is ""). The unused field is an empty string, never undefined / omitted, so always branch on `ref.type` — don't discriminate by checking a field for undefined.
- Only one completion callback is registered per handle; if your server has both prompts and resource templates that want completion, branch on `ref.type` / `ref.name` / `ref.uri` inside the single callback.
- Returning `{ values, total?, hasMore? }` instead of a plain array lets you hint at pagination for a large suggestion set; most clients are fine with a plain `string[]` for a short list.

5.15.2.9 Cap list-page size for large tool sets

Pass `pageSize` to `mcp.serve` to make `tools/list` (and the other list methods) paginate sooner — mainly useful for testing a client's cursor handling, or when a server genuinely registers hundreds of capabilities.

```
const srv = mcp.serve({ name: "many-tools", version: "1.0.0", pageSize: 50 });

for (const name of discoverToolNames()) {
  srv.tool({ name, inputSchema: { type: "object" }, handler: () => "ok" });
}

await srv.stdio();
```

Notes

- `pageSize` must be a positive integer if present; `mcp.serve` throws synchronously otherwise (zero, negative, and non-integer are all rejected).
- Omitting it uses the SDK's default (1000) — most servers with a modest, fixed tool set never need to set this.

5.15.2.10 Bridge a tool to an external HTTP API

A tool handler doesn't have to do the work itself — it can forward the call to an existing API and hand the result back to the model, translating transport failures into MCP's own error shape along the way.

```
srv.tool({
  name: "get_weather",
  description: "Look up current weather for a city via the upstream weather API.",
  inputSchema: {
    type: "object",
    properties: { city: { type: "string", description: "City name" } },
    required: ["city"],
  },
  async handler(args: any) {
    // net.http.request (not .get) — its richer { status, ok, headers, body }
    // shape is what lets the bridge tell a successful response from a failed one.
    const res = await net.http.request("GET", `https://api.example.com/weather?city=${encodeURIComponent(args.city)}`);
```

```

    if (!res.ok) {
      return { content: [{ type: "text", text: `upstream error: ${res.status}` }],
        isError: true };
    }
    return res.body; // upstream already returns JSON; pass it through as-is
  },
});

```

Notes

- Returning `{ content, isError: true }` on an upstream failure surfaces a normal MCP tool error to the client — not a thrown exception, which would also work (see §5.15.1's failure-surface table) but loses the chance to include a tailored message.
- `examples/scripts/mcp-bridge.ts` runs this end to end against a fake local upstream (started with `server.http` in the same script, so the demo stays offline) and exercises both the success and the `isError` path — it's a `make demo` script since it's its own client.

5.15.2.11 Ask the client's LLM to do work mid-call (sampling)

`ctx.sample` lets a tool hand a sub-task — summarizing, rephrasing, classifying — to whichever LLM the *connected client* is backed by, instead of the tool calling out to an LLM provider itself.

```

srv.tool({
  name: "summarize",
  description: "Ask the connected client's LLM to summarize the given text in one sentence.",
  inputSchema: {
    type: "object",
    properties: { text: { type: "string" } },
    required: ["text"],
  },
  async handler(args: any, ctx: any) {
    const r = await ctx.sample({
      messages: [{ role: "user", content: { type: "text", text: `Summarize in one sentence: ${args.text}` } }],
      maxTokens: 200,
      systemPrompt: "You are concise.",
    });
    // r => { content: { type, text }, model, stopReason, role }
    return JSON.stringify({ summary: r.content.text, model: r.model, stopReason: r.stopReason });
  },
});

```

Notes

- `messages / maxTokens` are the only required fields; `systemPrompt`, `temperature`, `stopSequences`, `includeContext`, and a partial `modelPreferences` (`cost/intelligence/speed` priorities, 0–1) are all optional — see §17.9 for the full `ctx.sample` signature.
- The resolved `content` is `{ type: "text", text }` for a text reply (the common case, and all this section's examples assume it). A client may instead return image or audio content, in which case `content` carries `{ type, data, mimeType }` with no `text` field — guard on `content.type` if your tool must handle non-text replies.

- Rejects with `"mcp: client does not support sampling"` if the connected client never wired a `CreateMessageHandler` — there is no soft-degrade path, so a tool that depends on sampling should let that rejection propagate (or catch it and explain why the tool is unavailable) rather than silently returning a placeholder.
- `examples/scripts/mcp-sampling.ts` is driven by `cmd/sercon/mcp_examples_test.go` (`TestMCPExamples/Sampling`), which connects a real SDK client with a canned `CreateMessageHandler` standing in for “the client’s LLM” — a plain HTTP client can’t answer this request, so it’s not a `make demo` script.

5.15.2.12 Confirm an action with the user mid-call (elicitation)

`ctx.elicit` pauses a handler to ask the *user* — via the client’s UI, not its LLM — to confirm an action or fill in a small form before the tool proceeds.

```
srv.tool({
  name: "deploy",
  description: "Deploy to the given target, after confirming with the user.",
  inputSchema: {
    type: "object",
    properties: { target: { type: "string" } },
    required: ["target"],
  },
  async handler(args: any, ctx: any) {
    const e = await ctx.elicit({
      message: `Confirm deploy to ${args.target}?`,
      schema: { type: "object", properties: { confirm: { type: "boolean" } } },
    });
    // e => { action: "accept"|"decline"|"cancel", content } (content present on
    accept)
    if (e.action !== "accept" || !e.content?.confirm) {
      return JSON.stringify({ deployed: false, action: e.action });
    }
    return JSON.stringify({ deployed: true, target: args.target, action:
    e.action });
  },
});
```

Notes

- `schema` is a JSON Schema describing the form fields you want back (passed to the SDK as-is); always branch on `action` first — `content` is only present when `action === "accept"`.
- Rejects with `"mcp: client does not support elicitation"` if the connected client never wired an `ElicitationHandler` — same no-soft-degrade caveat as `ctx.sample`.
- `examples/scripts/mcp-elicitation.ts` is driven by `cmd/sercon/mcp_examples_test.go` (`TestMCPExamples/Elicit`) with a canned `ElicitationHandler` standing in for “the human at the keyboard” — not a `make demo` script, for the same client-dependence reason as sampling.

5.15.2.13 Read the client’s filesystem roots

`ctx.roots()` asks the connected client which filesystem/URI roots it’s willing to let the server operate within; `srv.onRootsChanged(fn)` is the push-based counterpart, firing whenever that set changes.

```

srv.onRootsChanged((roots: any) => {
  const uris = roots.map((r: any) => r.uri).sort();
  runtime.log("roots changed:", JSON.stringify(uris));
});

srv.tool({
  name: "listRoots",
  description: "List the connected client's filesystem roots.",
  inputSchema: { type: "object" },
  async handler(_args: any, ctx: any) {
    const roots = await ctx.roots(); // Root[] : [{ uri, name? }, ...]
    return JSON.stringify(roots.map((r: any) => r.uri).sort());
  },
});

```

Notes

- `ctx.roots()` is the pull-based, per-request query; `onRootsChanged` is the persistent, push-based hook — register both if a tool needs to react to root changes even between calls, not just query them on demand.
- Rejects with "mcp: client does not support roots" if the client never advertises the `roots` capability — most SDK clients advertise it by default, but don't assume every client does.
- `examples/scripts/mcp-roots.ts` is driven by `cmd/sercon/mcp_examples_test.go` (`TestMCPEXamples/Roots`), which pre-seeds roots on a real SDK client via `client.AddRoots` and then adds another root post-connect to trigger `onRootsChanged` — not a `make demo` script, for the same client-dependence reason as `sampling/elic`.

5.15.2.14 Protect an HTTP server with OAuth 2.1 bearer tokens

Guard `listen()` with `auth` to turn it into an OAuth 2.1 resource server: every request needs a valid bearer token, and the well-known metadata endpoint tells clients which authorization server issues one.

```

const srv = mcp.serve({ name: "sercon-oauth-demo", version: "1.0.0" });

srv.tool({
  name: "add",
  inputSchema: { type: "object", properties: { a: { type: "number" }, b: { type: "number" } }, required: ["a", "b"] },
  async handler(args: any) { return String(args.a + args.b); },
});

const h = await srv.listen({
  port: 38080,
  auth: {
    // A real deployment would verify a signed JWT or call the authorization
    // server's introspection endpoint; this hardcodes one accepted token.
    verify: (token: string) =>
      token === "good-token" ? { subject: "demo-user", scopes: ["mcp"] } : null,
    resourceMetadata: {
      authorizationServers: ["https://auth.example.com"],
      scopesSupported: ["mcp"],
      resourceName: "sercon OAuth demo",
    }
  }
});

```

```

    },
    scopes: ["mcp"],
  },
});

runtime.log("listening at", h.url, "(bearer token required: good-token)");

```

Notes

- `verify(token, req)` may be sync or return a `Promise` returning `null` / `undefined` (or throwing/rejecting) rejects the request with `401` — sercon never treats a failed `verify` as a server error.
- `expiresAt` on the returned identity accepts a Unix-seconds number or an RFC 3339 string, and defaults to a 1-hour TTL if you omit it.
- This is the **resource-server** role only: sercon validates tokens and publishes `/.well-known/oauth-protected-resource` metadata pointing at `authorizationServers` it does not issue tokens and does not implement dynamic client registration (DCR, RFC 7591) — provisioning clients against the authorization server is out of scope here, by design.
- `examples/scripts/mcp-oauth.ts` binds a fixed port and never calls `close()` on purpose (an always-on OAuth-protected server is stopped by its operator, not itself); `cmd/sercon/mcp_examples_test.go` (`TestMCPExamples/OAuth`) drives it externally: no token → `401` with `WWW-Authenticate`, bad token → `401`, good token → `initialize` + `tools/call` succeeds, and a `GET` on the metadata URL → `200 JSON`. Not a make demo script — a real client is required to exercise it.

5.15.3 mcp.connect — sercon as an MCP client

Everything above is sercon *serving* MCP. `mcp.connect` is the other direction: sercon as an MCP **client**, connecting to someone else's server — over stdio (sercon launches it as a subprocess), Streamable HTTP, or legacy SSE (a server already listening) — then calling its tools, reading its resources, and fetching its prompts. Phase 2 adds the reactive half: change notifications, resource subscriptions, opt-in server logging, and argument completion. Phase 3 adds the **host responder** surface: the client can now answer the server's own mid-call requests — `onSample` / `onElicit` respond to `sampling/createMessage` and `elicitation/create` (the client-side counterpart to a server tool's `ctx.sample()` / `ctx.elicit()`, §5.15.2.11–§5.15.2.12), and `roots` seeds the client's filesystem/URI roots (answering `roots/list`, with `setRoots(roots)` updating that set at runtime). See §5.15.3.6–§5.15.3.8 below. Phase 4 (current, and the **final phase of the MCP client**) adds `mcp.connect.sse` (the legacy HTTP+SSE transport, §5.15.3.9), an OAuth 2.1 bearer-token client via `connect.http`'s `auth: { getToken }` option (§5.15.3.10), and a reconnect cap via `connect.http`'s `maxRetries` option (§5.15.3.11) — see those three recipes below. **With Phase 4 shipped, the MCP client is now full-spec: consume (Phase 1), react (Phase 2), host (Phase 3), and OAuth/SSE (Phase 4) are all covered.**

Call	Transport	Shape
<code>await mcp.connect.stdio({ command, env?, cwd?, onSample?, onElicit?, roots? })</code>	subprocess stdio	Launches command (argv, e.g. <code>["sercon", "server.ts"]</code>)

		and speaks JSON-RPC over its stdin/stdout — the mirror of <code>srv.stdio()</code> on the other end.
<pre>await mcp.connect.http(url, { headers?, maxRetries?, auth?, onSample?, onElicit?, roots? })</pre>	Streamable HTTP	Connects to an already-running listener (e.g. one started with <code>srv.listen(...)</code>); <code>headers</code> or <code>auth: { getToken }</code> can carry a bearer token for an OAuth-protected server; <code>maxRetries</code> caps the transport's own reconnect attempts.
<pre>await mcp.connect.sse(url, { headers?, onSample?, onElicit?, roots? })</pre>	legacy HTTP+SSE (2024-11-05)	Connects to a server that only exposes the older two-endpoint SSE handshake, for servers predating Streamable HTTP; no <code>maxRetries</code> / <code>auth</code> — this transport has no reconnect knob and no OAuth client wiring yet, use <code>headers</code> for a static bearer token.

Both resolve the same session handle: `serverInfo` / `capabilities` (what the server advertised during `initialize`), `listTools()` / `callTool(name, args?)` (a failed tool call surfaces as `{ isError: true }`, not a thrown error — same soft-failure contract as the server side), `listResources()` / `listResourceTemplates()` / `readResource(uri)`, `listPrompts()` / `getPrompt(name, args?)`, `ping()`, and `close()` (idempotent — safe to call more than once). A connection holds the script's event loop open for its lifetime, the same way an open `listen()` handle does.

The handle also carries the Phase-2 reactive surface: six single-slot notification setters — `onToolsChanged(fn)` / `onResourcesChanged(fn)` / `onPromptsChanged(fn)` (each `fn` takes no arguments — re-list to see what changed), `onResourceUpdated(fn)` (`fn(uri)`), `onLoggingMessage(fn)` (`fn({level, logger?, data})`), and `onProgress(fn)` (`fn({progressToken, progress, total?, message?})`) — plus four calls that make a mid-connection request: `subscribe(uri)` / `unsubscribe(uri)`, `setLoggingLevel(level)`, and `complete(ref, argName, partial)`. Each setter replaces any previously registered callback for that event (last-writer-wins), the same single-slot convention `srv.onSubscribe` / `srv.onUnsubscribe` use on the server side. Full signatures are in §17.9.1.

Host responders (Phase 3): `onSample` / `onElicit` / `roots`. Everything above is the client *asking* the server for things; these three connect opts flip that around and let the server ask the *client* for things — the same three server->client round trips a server tool handler can make via `ctx.sample()` / `ctx.elicit()` / `ctx.roots()` (§5.15.1/§5.15.2.11–13), now answerable from the connecting side instead of only the serving side. `onSample(req)` and

`onElicit(req)` are each **advertised as a capability only when provided**: the underlying SDK sets `ClientCapabilities.Sampling / .Elicitation` if and only if the matching opt is a function, so a server calling `ctx.sample()` / `ctx.elicit()` against a client that omitted the responder gets *that server-side call rejected* (`"mcp: client does not support sampling" / "...elicitation"`) — `mcp.connect` itself never throws for a missing responder. `roots` is different: the roots capability is advertised **unconditionally** by the SDK regardless of whether this option is given; passing `roots` only seeds the *initial* root set (via `AddRoots` before the handshake) so the server's first `roots/list` sees it immediately, rather than an empty set followed by a later `roots/list_changed . setRoots(roots)` on the handle replaces that set at runtime (synchronous — no Promise, since `AddRoots / RemoveRoots` are pure in-memory bookkeeping). See §5.15.3.6–§5.15.3.8 and the generated §17.9.1 reference (`connect.stdio / connect.http` 's Params, and `connect.setRoots`) for the exact request/response shapes.

Connection lifecycle: death, `close()`, and graceful HTTP shutdown. A connection whose peer disappears abruptly — a stdio subprocess exiting, or an abrupt TCP/transport failure — releases its hold on the script's event loop promptly: an internal watcher blocks on the SDK session's own `wait()`, which returns as soon as the session ends by any means, and then tears the connection down. This is what lets a script keep running (or exit) without waiting out the full run when a server it was talking to simply goes away. The one gap is a client connected over Streamable HTTP to a server that shuts down **gracefully** (e.g. the peer calling its own `h.close()` from `srv.listen()`, backed by Go's `http.Server.Shutdown`): a graceful shutdown does not force-close the client's long-lived SSE stream, so the session looks alive from the client's point of view and the watcher never fires. An idle client left connected to such a server keeps the event loop open until the run itself ends. The takeaway: **`close()` is the clean, explicit way to end a connection** — call it as soon as a script is done with a server, rather than relying on the peer's own shutdown to be noticed.

5.15.3.1 Call a tool on an MCP server

Connect over HTTP to a server already listening and invoke one of its tools:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");
runtime.log("connected to", c.serverInfo.name, c.serverInfo.version);

const result = await c.callTool("add", { a: 2, b: 40 });
if (result.isError) {
  runtime.log("tool call failed:", result.content[0].text);
} else {
  runtime.log("result:", result.content[0].text); // "42"
}

await c.close();
```

Notes

- `callTool` 's result never throws for a tool-level failure — check `isError` the same way a server's tool handler signals one; a thrown or rejected exception here means the *transport or protocol* failed, not that the tool itself errored.
- `examples/scripts/mcp-client-http.ts` is the hermetic version of this recipe: it starts its own `mcp.serve()` + `srv.listen()` in-script, so it needs no external server and runs under `make demo`.

5.15.3.2 Read a resource from an MCP server

Connect over stdio to a server launched as a subprocess, then read one of its resources:

```
const c = await mcp.connect.stdio({ command: ["sercon", "server.ts"] });

const doc = await c.readResource("cfg://app");
runtime.log(doc.contents[0].text);

const prompts = await c.listPrompts();
runtime.log("prompts:", prompts.map(p => p.name).join(", "));

await c.close();
```

Notes

- `readResource`'s `contents` array items carry `text` or `blob` (base64), matching the shape a server's `resource.read()` handler returns — see §5.15.2's resource recipes for the server side of the same contract.
- `examples/scripts/mcp-client-stdio.ts` exercises `connect.stdio` + `callTool` against a real subprocess (`examples/scripts/mcp-server-stdio.ts`); it needs the `sercon` binary on `PATH`, so it's driven by `cmd/sercon/mcp_client_test.go` rather than `make demo`. The `readResource` / `listPrompts` calls above use the same session-handle API already exercised hermetically over HTTP in `examples/scripts/mcp-client-http.ts` — the method set is identical across both transports.

5.15.3.3 Subscribe to a resource and react when it changes

Ask the server to notify this client whenever a resource's content changes, then re-read it each time:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");

c.onResourceUpdated(async (uri) => {
  const doc = await c.readResource(uri);
  runtime.log("updated:", uri, "->", doc.contents[0].text);
});

await c.subscribe("cfg://app");
// ... the server later calls its own srv.resourceUpdated("cfg://app"),
// which fires onResourceUpdated above with uri === "cfg://app" ...
await c.unsubscribe("cfg://app");
await c.close();
```

Notes

- `onResourceUpdated` holds one callback at a time (last-writer-wins) — if a script subscribes to several URIs, dispatch on the `uri` argument inside a single callback rather than expecting per-URI registration.
- `subscribe` / `unsubscribe` are plain request/response calls; the actual notification always arrives asynchronously and only for URIs currently subscribed.
- See §5.15.2's subscription recipes for the server side of this same contract (`srv.onSubscribe` / `srv.onUnsubscribe` / `srv.resourceUpdated`).

5.15.3.4 Receive server log messages

Opt into a server's log stream — nothing arrives until `setLoggingLevel` resolves, per the MCP spec:

```
const c = await mcp.connect.stdio({ command: ["sercon", "logging-server.ts"] });

c.onLoggingMessage((message) => {
  runtime.log(`[${message.level}]`, message.logger ?? "server", message.data);
});

await c.setLoggingLevel("info"); // required first — see the Note below
await c.callTool("do-something-noisy", {});

await c.close();
```

Notes

- Register `onLoggingMessage` *before* calling `setLoggingLevel` — the server may start pushing messages as soon as it acknowledges the level change, and a callback registered afterwards would miss anything sent in that window.
- Levels follow the RFC 5424 syslog severities the MCP spec reuses: "debug", "info", "notice", "warning", "error", "critical", "alert", "emergency" (least to most severe) — the server sends this level and anything more severe.
- This is the client-side half of a server's `ctx.log(level, message, data?)` calls inside a tool/resource/prompt handler — see §5.15.1's progress/logging concepts for the server side.

5.15.3.5 Request an argument completion

Ask a server for autocompletion suggestions for a prompt argument or a resource-template URI variable:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");

const suggestions = await c.complete({ type: "prompt", name: "greet" }, "user", "ali");
runtime.log(suggestions.values.join(", ")); // e.g. "alice, alicia"

const uriSuggestions = await c.complete(
  { type: "resource", uri: "db://{table}/{id}" },
  "table",
  "us",
);
runtime.log(uriSuggestions.values.join(", ")); // e.g. "users"

await c.close();
```

Notes

- `ref.type` selects which field applies: "prompt" uses `ref.name` (a prompt registered via `srv.prompt`); "resource" uses `ref.uri` (a resource template's `uriTemplate`, not a concrete URI).
- The result's `values` may be empty (no match is not an error); `total` / `hasMore` are optional hints, present only when the server supplies them.

- This is the client-side half of `srv.completion(fn)` — see §5.15.2's completion recipe for the server side of the same contract.

5.15.3.6 Answer a server's sampling request (act as its LLM)

Wire `onSample` to answer `ctx.sample()` calls a connected server's tool handlers make (§5.15.2.11) — the natural use is forwarding the request to a real LLM via `services.ai.send`, so the server gets an actual completion back instead of a canned string:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp", {
  async onSample(req) {
    const prompt = req.messages.map((m: any) => m.content.text).join("\n");
    const r = await services.ai.send({ prompt, system: req.systemPrompt });
    return r.output; // wrapped as { content: { type: "text", text }, model:
    "sercon", role: "assistant", stopReason: "endTurn" }
  },
});

const result = await c.callTool("summarize", { text: "..." });
runtime.log(result.content[0].text);

await c.close();
```

Notes

- `onSample` may return a plain string (the common case, auto-wrapped as shown above) or an object `{ content, model?, stopReason?, role? }` for explicit control — see §17.9.1's `connect.stdio / connect.http` Params for the full shape of both `req` and the accepted return values.
- **Capability gating:** providing `onSample` is what makes this connection advertise the sampling capability at all. Omit it and a server's `ctx.sample()` call against this client rejects with `"mcp: client does not support sampling"` — there is no partial/soft-degrade mode, and `mcp.connect` itself never throws for the omission.
- `onSample` may be sync or async (a returned Promise is awaited); a thrown/rejected `onSample` propagates back to the server as the `ctx.sample()` call's own rejection.
- `examples/scripts/mcp-client-http.ts` exercises this hermetically: its in-script server's `summarize` tool calls `ctx.sample`, answered here by a connected client's `onSample` (an echo, not a real `services.ai` call, so the demo stays offline).

5.15.3.7 Confirm an action or collect input via elicitation

Wire `onElicit` to answer `ctx.elicit()` calls (§5.15.2.12) — typically by prompting the actual human at the keyboard (`tui.*`, or a plain `runtime.prompt`-style read), or by returning a fixed decision for a scripted/unattended connection:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp", {
  onElicit(req) {
    // req => { message, requestedSchema, mode? }
    runtime.log("server asks:", req.message);
    // a real client would prompt the user here and shape the reply to
    // req.requestedSchema; this one auto-confirms:
    return { action: "accept", content: { confirm: true } };
  },
});
```



```
});

const result = await c.callTool("deploy", { target: "prod" });
runtime.log(result.content[0].text);

await c.close();
```

Notes

- The return value must be `{ action: "accept" | "decline" | "cancel", content? }` — `content` is only meaningful (and typically only present) when `action` is `"accept"` always branch on `action` first.
- Same capability-gating rule as `onSample`: omitting `onElicit` means this connection does not advertise elicitation, and a server's `ctx.elicit()` call against it rejects with `"mcp: client does not support elicitation"`.
- `onElicit` may be sync or async, same as `onSample`.

5.15.3.8 Expose the client's filesystem roots, and update them later

Seed the connection's roots up front with the `roots` opt so the server's first `ctx.roots()` (§5.15.2.13) sees them immediately, then swap the set at runtime with `setRoots`:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp", {
  roots: [{ uri: "file:///workspace", name: "workspace" }],
});

const initial = await c.callTool("listRoots", {});
runtime.log(initial.content[0].text); // sees "file:///workspace" right away

// later, e.g. the user switched workspaces:
c.setRoots([{ uri: "file:///workspace2", name: "workspace2" }]);
const updated = await c.callTool("listRoots", {});
runtime.log(updated.content[0].text); // sees "file:///workspace2"

await c.close();
```

Notes

- Unlike `onSample` / `onElicit`, the roots capability is advertised **unconditionally** by the underlying SDK — omitting `roots` just means the initial set is empty, not that the capability disappears.
- `setRoots` **replaces** the whole set (it does not merge with the previous one) and fires `roots/list_changed` to the server; a server watching via `srv.onRootsChanged(fn)` (§5.15.2.13) sees the push, and a server that only pulls via `ctx.roots()` sees the new set on its next call. `setRoots` is synchronous (`void`, not a `Promise`) — there's no network round trip to await.
- `examples/scripts/mcp-client-http.ts` exercises the full round trip: seeded `roots`, a server tool reading them back via `ctx.roots()`, then `setRoots` and a second read confirming the update.

5.15.3.9 Connect over the legacy SSE transport

Some MCP servers predate Streamable HTTP and only speak the older (2024-11-05) two-endpoint SSE handshake. `mcp.connect.sse` is otherwise a drop-in replacement for `mcp.connect.http` — same URL shape, same session handle, same Phase 1-3 surface:

```
const c = await mcp.connect.sse("http://127.0.0.1:38080/sse");
runtime.log("connected to", c.serverInfo.name, c.serverInfo.version);

const tools = await c.listTools();
runtime.log(tools.map(t => t.name).join(", "));

await c.ping();
await c.close();
```

Notes

- Use `connect.sse` only when the server genuinely doesn't support Streamable HTTP — prefer `connect.http` (§5.15.3.1) for anything new; the legacy transport exists for interop with older servers, not as a general-purpose alternative.
- `connect.sse` accepts the same `headers` / `onSample` / `onElicit` / `roots` `opts` as `connect.http` (§5.15.3.6–§5.15.3.8 all apply unchanged over SSE) but **not** `maxRetries` or `auth` — the underlying SDK's SSE client transport has neither a `reconnect-cap` field nor OAuth wiring; use `headers` for a static bearer token against a protected SSE server.
- This transport is Go-test-covered (`cmd/sercon/mcp_client_test.go` round-trips it against a Go-SDK-backed SSE server) rather than exercised by an `examples/scripts/ demo` — `sercon`'s own `srv.listen()` only serves Streamable HTTP, so there is no hermetic in-script SSE server to pair it with the way `mcp-client-http.ts` pairs `connect.http` with `srv.listen`.

5.15.3.10 Connect to an OAuth-protected server with `auth.getToken`

Pair with an HTTP server protected via `srv.listen({ auth })` (§5.15.2.14): instead of a static `headers.Authorization`, hand `connect.http` a `getToken` callback that supplies (and, on a 401/403, refreshes) a bearer token:

```
let token = "";
const c = await mcp.connect.http("http://127.0.0.1:38081/mcp", {
  auth: {
    async getToken() {
      if (!token) {
        // e.g. a client-credentials exchange against the resource
        // server's authorization server (see §5.15.2.14's
        // resourceMetadata.authorizationServers):
        // const r = await net.http.post("https://auth.example.com/token",
        //   JSON.stringify({ grant_type: "client_credentials", client_id,
client_secret }));
        // token = JSON.parse(r.body).access_token;
        token = "good-token";
      }
      return token;
    },
  },
});
```

```
const result = await c.callTool("add", { a: 2, b: 40 });
runtime.log(result.content[0].text);

await c.close();
```

Notes

- `getToken` may return a plain string or a `Promise<string>`, and MUST resolve to a non-empty string — a thrown/rejected `getToken`, or one that resolves to a non-string or an empty string, fails the connect (or the in-flight request) with a clear `"mcp.connect: auth.getToken must return a string" / "...a non-empty token string"` error.
- **Refresh on demand:** `getToken` is called once before the initial request and again automatically whenever a request comes back with a 401 or 403 — clear/replace your cached token inside `getToken` when that happens (or fetch a fresh one unconditionally) so the retry actually presents a different token, rather than caching forever and looping on the same rejected one.
- The token can come from anywhere a script can compute a string: a `net.http` call to a client-credentials token endpoint (as sketched above), a value read from `fs` / environment/a secrets store, or — for a quick local test, as in the snippet — a hardcoded string matching what the server's `auth.verify` expects (§5.15.2.14).
- `auth` is `connect.http` -only for now; `connect.sse` has no OAuth wiring (§5.15.3.9) and `connect.stdio` has no HTTP handshake to attach a bearer token to in the first place.
- `examples/scripts/mcp-client-http.ts` exercises this hermetically: an in-script `srv.listen({ auth })` paired with a `connect.http({ auth: { getToken } })` client that successfully calls a tool.

5.15.3.11 Disable automatic reconnection with `maxRetries`

`connect.http`'s Streamable HTTP transport reconnects its underlying SSE stream a limited number of times after a drop (5, by default). Set `maxRetries: 0` (or any negative number) to disable that behavior entirely — a single drop ends the session instead of retrying:

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp", {
  maxRetries: 0,
});

try {
  await c.callTool("add", { a: 2, b: 40 });
} finally {
  await c.close();
}
```

Notes

- Streamable-HTTP-only: `connect.sse`'s legacy transport has no equivalent field to plumb (§5.15.3.9).
- Omitting `maxRetries` leaves the go-sdk's own built-in default (5) in place — pass an explicit larger number to retry more persistently, or `0` /negative to fail fast (useful in tests,

or when a caller wants to implement its own backoff/retry loop around `mcp.connect.http` rather than relying on the transport's).

- This is a connect-time transport setting, not something adjustable after the fact — reconnect it with a different `maxRetries` by calling `mcp.connect.http` again rather than mutating an existing handle.

6. Servers

The `server` global (added in v0.10.0) hosts inbound listeners — scripts that *accept* connections rather than make them. It ships `server.http` and `server.https` (§6.2) — with static-file serving (§6.3), middleware (§6.4), and WebSocket / Server-Sent-Event upgrades (§6.5) — plus `server.smtp` (§6.7) and the raw `server.tcp` / `server.udp` / `server.icmp` listeners (§6.8). Lifecycle and the `sercon` `serve` production niceties are covered in §6.6. Future cycles will add IMAP, FTP, and POP3 against the same engine foundation.

Every listener follows the same shape: a **synchronous bind** (a port already in use throws immediately, before any Promise), a background serve loop kept alive by one `HoldRun` sentinel (§6.1), and a handle exposing `address` (a `proto/host:port` string; the raw TCP/UDP listeners accept `port: 0` for an OS-chosen ephemeral port, filled into `address` — HTTP, HTTPS, and SMTP require an explicit port) and a `close()` that returns `Promise<void>`. The HTTP and SMTP handles additionally carry a `stopped` Promise that resolves when the listener exits (and rejects on a non-close serve error).

6.1 Foundation: `HoldRun` + `LoopCallable`

Two engine primitives back every long-lived binding. They live on `*scriptengine.Engine` (CLI use of them is internal; the public contract is “use these if you write a similar binding yourself”).

`Engine.HoldRun(reason string) (release func())` — keeps `loop.Run` from exiting while a binding has resources outstanding. The event loop normally exits as soon as its `jobCount` drops to zero; `setTimeout` / `setInterval` / `setImmediate` are the only things that count. `HoldRun` parks a 24-hour sentinel timer under the hood (same trick `PromisifyAsync` uses for one-shot async work), returns a `release` function that clears it, and increments a `refcount` so multiple concurrent holders compose. `release` is idempotent. Any sentinels still parked when `Run` returns are cleanup-drained automatically as a safety net — a binding that forgets to call `release` does not leak into the next `Run`.

`scriptengine.NewLoopCallable(loop, fn)` — wraps a captured `goja.Callable` (a JS function reference) so it can be invoked from *any* goroutine. Goja's runtime is single-threaded; calling a `Callable` directly from a non-loop goroutine corrupts engine state. `LoopCallable.Call(buildArgs)` enqueues a `loop.RunOnLoop` callback, builds the arg list on the loop (where `vm.ToValue` is valid), invokes the callable, and returns the result to the caller's goroutine. `LoopCallable.CallOnLoop(vm, args...)` is the already-on-the-loop variant — using `Call` from inside a loop callback deadlocks because the new `RunOnLoop` job can't run until the current one returns.

You use them together. A typical server binding holds one or two `LoopCallable`s for the user's handlers, parks one `HoldRun` sentinel per active listener, and releases on close. The CLI bindings under `cmd/sercon/api_server*.go` follow this pattern.

Concurrency model. Because every handler invocation marshals back onto the single goja loop, **handlers serialize**: only one JS handler runs at a time, across all listeners. This is a

feature (no JS data races, no shared-state confusion) and a throughput ceiling. For CPU-bound work in a handler, offload to a Go binding that does the work in a goroutine and resolves a Promise.

6.2 `server.http.listen` / `server.https.listen`

```
// HTTP
const srv = await server.http.listen({
  port: 8080, // required
  host: "0.0.0.0", // default
  routes: { "GET /": (req, res) => res.text("hi") },
  use: [logger], // optional global middleware
});

// HTTPS – identical plus cert/key (file paths OR inline PEM strings)
const srv2 = await server.https.listen({
  port: 8443,
  cert: "/etc/ssl/server.pem",
  key: "/etc/ssl/server.key",
  routes,
});

// HTTPS dev shortcut – ephemeral self-signed cert, no key, no files
const srv3 = await server.https.listen({
  port: 8443,
  cert: "self-signed",
  routes,
});

srv.address; // "tcp/0.0.0.0:8080"
await srv.close(); // graceful: stop accepting, drain, release HoldRun
await srv.stopped; // Promise that resolves when the listener exits
```

Options:

Key	Type	Notes
<code>port</code>	<code>number</code>	Required. Bind port. <code>0</code> lets the kernel pick; read the chosen port off <code>srv.address</code> .
<code>host</code>	<code>string</code>	Default <code>"0.0.0.0"</code> . Pass <code>"127.0.0.1"</code> for loopback-only.
<code>routes</code>	<code>Record<string, RouteValue></code>	Required (may be <code>{}</code>). Keys are stdlib <code>http.ServeMux</code> patterns.
<code>use</code>	<code>Middleware[]</code>	Optional global middleware; runs in array order before any per-route middleware.
<code>onError</code>	<code>(err, req, res) => unknown</code>	Optional error handler; renders a custom response when a handler/middleware throws or rejects. See below.
<code>maxBodyBytes</code>	<code>number</code>	Caps an inbound request body read into memory before any route handler or middleware runs. Default 32 MB (a non-

		positive value falls back to the default). An over-limit body gets a 413 without invoking any JS.
cert	string	HTTPS only. File path, inline PEM, or the literal "self-signed" to mint an ephemeral in-process dev cert.
key	string	HTTPS only. File path or inline PEM. Not needed (ignored) when cert is "self-signed".

Custom error pages (`onError`). When a handler or middleware throws (or returns a rejected Promise) and `onError` is set, it is invoked as `onError(err, req, res)` instead of sending the stock 500 Internal Server Error. `err` is the thrown value (an `Error` for a plain `throw new Error(...)`); render whatever you like via the usual `res.*` terminals (`res.status(500).json({...})`, `res.html(...)`, etc.). `onError` may be `async`. If it finishes without finalizing a response, or itself throws/rejects, `sercon` falls back to the stock 500 — a buggy error handler can never wedge the request. Without `onError`, behaviour is unchanged (stock 500 + a log line); the per-route `try/catch` pattern still works and takes precedence (an error you catch never reaches `onError`).

Self-signed dev cert. `cert: "self-signed"` generates a fresh P-256 certificate in-process (valid 1 year), with SANs covering `localhost`, `127.0.0.1`, `::1`, and the `listen host`. The keypair lives only for the life of the run and is never written to disk. Self-signed certs fail normal client verification by design (`net.probe.tls` skips verification, so it still works against them) — this is a local-dev convenience, not a production path. For real deployments own the cert in your supervisor (`caddy` / `nginx` / `traefik`) in front.

Route patterns are stdlib Go 1.22+ `http.ServeMux` syntax:

`"METHOD /path/{param}/{rest...}"`. The leading method is optional (omit to match any). `{name}` captures one path segment; `{name...}` captures the tail (wildcard). No new routing library — stdlib does the matching, including longest-prefix wins. Captured values land in `req.params`.

RouteValue is either a bare handler — `(req, res) => res.text("...")` — or an object `{ use?: Middleware[], handler: HandlerFn }` for per-route middleware (see §6.4).

Request shape (`req`):

```
type Request = {
  method:    string;           // "GET"
  url:       string;           // full URL incl. scheme + host
  path:      string;           // "/users/42"
  query:     Record<string, string[]>; // ?a=1&a=2 → {a: ["1","2"]}
  headers:   Record<string, string[]>; // lowercase keys
  params:    Record<string, string>;   // path-pattern captures
  body:      string;            // UTF-8 decoded request body
  bodyBytes: Uint8Array;        // same data as raw bytes
  remote:    string;            // "1.2.3.4:54321"
  cookies:   Record<string, string>;  // parsed Cookie header
};
```

Response builder (`res`) — every method returns `res` for chaining; the terminal `.json` / `.text` / `.html` / `.bytes` / `.empty` / `.redirect` sets the body and flips an internal `finalized` flag:

```
type CookieOpts = {
  domain?: string;
  path?: string;
  maxAge?: number; // seconds; 0 = session; <0 = delete
  expires?: number; // unix ms
  secure?: boolean;
  httpOnly?: boolean;
  sameSite?: "strict" | "lax" | "none";
};

type Response = {
  status(code: number): Response;
  header(name: string, value: string): Response;
  cookie(name: string, value: string, opts?: CookieOpts): Response;
  // Terminals (all return res so they remain chainable but no further
  // write makes sense after one fires; a second terminal throws):
  json(value: unknown): Response; // → application/json
  text(s: string): Response; // → text/plain
  html(s: string): Response; // → text/html
  bytes(b: Uint8Array, ct?: string): Response; // default application/octet-
stream
  empty(): Response; // no body; pair with .status() for
204/304
  redirect(loc: string, code?: number): Response; // default 302
  // Upgrade (WebSocket):
  upgradeWebSocket(opts?: { readBuffer?: number }): Promise<WebSocket>;
  // Stream (Server-Sent Events):
  sse(opts?: { keepAlive?: number; retry?: number }): SSEStream;
};

type SSEStream = {
  send(data: string | { data: unknown; event?: string; id?: string; retry?:
number }): Promise<void>;
  close(): Promise<void>;
  readonly closed: Promise<void>; // resolves on close OR client disconnect
  readonly remote: string;
};
```

After the handler's returned Promise resolves, the engine checks `res.finalized`:

- **true** — buffered status / headers / body get written to the underlying `http.ResponseWriter`.
- **false** — engine sends `204 No Content`.
- **handler threw** (sync or async) — engine sends `500 Internal Server Error` and logs the error (`stderr` under vanilla `sercon` access log under `sercon serve`). The script keeps running; only that one request is affected.

Calling a terminal twice on the same `res` throws a `TypeError` (the second call sees `finalized === true` and refuses).

Multiple listeners compose naturally. A script can run

```
const [a, b] = await Promise.all([
  server.http.listen({ port: 80, routes }),
  server.https.listen({ port: 443, routes, cert, key }),
]);
```

Each `listen` call holds its own `HoldRun` sentinel; closing one listener does not affect the other.

6.3 Static-file serving

```
"GET /assets/{rest...}": server.http.static({
  dir:          "/var/www/public",    // root directory on disk
  stripPrefix:  "/assets/",          // URL prefix to strip before disk lookup
  index?:       "index.html",        // accepted but currently unused (stdlib
default applies)
  etag?:        true,                // accepted but currently unused (stdlib
default applies)
}),
```

`server.http.static({...})` returns an opaque handler the route compiler unwraps and registers directly on the mux. **The route pattern must include a `{rest...}` wildcard** — without it the match only ever produces the literal prefix and no subpath ever resolves on disk. Internally a stdlib `http.FileServer(http.Dir(dir))` wrapped in `http.StripPrefix(stripPrefix, ...)` ETag and range requests work out of the box. Symlinks outside `dir` are blocked (stdlib `http.Dir`'s default). No directory listing customisation yet — `index` and `etag` options are reserved for future tuning; v0.10.0 inherits the stdlib defaults.

6.4 Middleware

Onion model. A middleware is `(req, res, next) => Promise<void>`. `next` is an async function that runs the rest of the chain; awaiting it lets the middleware post-process.

```
const logger = async (req, res, next) => {
  const start = runtime.time.nowMs();
  await next(); // run downstream chain
  runtime.log(req.method, req.path, "→", runtime.time.nowMs() - start, "ms");
};

await server.http.listen({
  port: 8080,
  use: [logger], // global middleware
  routes: {
    "GET /api/secure": { // per-route middleware
      use: [authCheck],
      handler: (req, res) => res.json({ ok: true }),
    },
  },
});
```

Attachment points:

- **Global** — `use: array` in `listen({...})` options. Runs for every request, in declaration order, before any per-route middleware.

- **Per-route** — when a route value is `{ use: [...], handler: fn }`, those middleware run after globals but before the handler.

For a request to `POST /api/secure` with global middleware `[A, B]` and per-route middleware `[C]`, the composition is:

```
A(req, res, () => B(req, res, () => C(req, res, () => handler(req, res))))
```

Each layer can `await next()` then mutate response headers or record timings, exactly like Express / Koa.

Short-circuit: a middleware that does *not* call `await next()` stops the chain. It must terminate `res` itself (e.g. `res.status(401).text("denied")`) — otherwise the engine sends `204 No Content` per the unfinalized-response rule.

Throwing: any throw mid-chain (sync or via rejected Promise) bubbles to the engine's 500 path. Wrap with `try / catch` if you want custom error handling.

6.5 WebSocket upgrade

```
"GET /ws": async (req, res) => {
  const ws = await res.upgradeWebSocket({ readBuffer: 64 });
  // ws is both an async iterator AND has .send / .close.
  for await (const msg of ws) {
    if (msg.type === "text") {
      await ws.send("echo:" + msg.text);
    } else {
      await ws.send(msg.bytes);           // msg.type === "binary"
    }
  }
  // iterator ends when the peer closes or ws.close() is called
},
```

Type:

```
type WSMMessage =
  | { type: "text"; text: string }
  | { type: "binary"; bytes: Uint8Array };

type WebSocket = AsyncIterable<WSMMessage> & {
  send(data: string | Uint8Array): Promise<void>;
  close(code?: number, reason?: string): Promise<void>;
  remote: string;
  closeCode?: number; // set after the iterator ends on a peer close frame
  closeReason?: string;
};
```

Peer close info. When the peer closes with a proper WebSocket close frame, the frame's code and reason are surfaced on the socket object as `ws.closeCode` (number) and `ws.closeReason` (string) once the message iterator ends — so a script can inspect *why* the peer left after its `for await` loop exits. They stay `undefined` for an abnormal disconnect that carried no close frame (and for a locally-initiated `ws.close()`, where the script already knows the code/reason it sent).

Library: `github.com/coder/websocket` (the modern successor to `nhooyr.io/websocket` `gorilla/websocket` is in maintenance mode). Pure Go, zero deps.

Marshalling. Per upgraded connection, a Go goroutine reads frames into a buffered channel (capacity = `readBuffer`, default 64). Each `next()` on the async iterator pulls one message off the channel and resolves on the loop via `LoopCallable`. `ws.send()` schedules a write back through the connection-owning goroutine.

Backpressure. If the read buffer fills (slow JS consumer), back-pressure propagates into the websocket library's read loop — the client eventually sees `1009 message too big` after the library's own limit (1MB default). Bump `readBuffer` if your workload bursts and you want more in-flight queueing.

`ws.close()` closes the send channel, drains the receive channel, sends a close frame, and ends the async iterator on the next `next()` call. The script's `for await` exits cleanly; the handler's Promise resolves; the connection's goroutine returns. The HTTP server's `HoldRun` is **unaffected** — it stays alive for new connections until `srv.close()`.

`async function*` and `for await (...)` are not natively parsed by `goja`. `esbuild`'s `Supported` flag lowers both during transpile (see CHANGELOG), and every `Run` installs `Symbol.asyncIterator = Symbol.for("@@asyncIterator")` so the lowered helper and user code agree on the same iteration key.

6.5.1 Server-Sent Events (`res.sse`)

`res.sse(opts?)` starts a one-way text/event-stream response: the handler holds the connection open and pushes events until it (or the client) closes the stream.

```
"GET /events": (req, res) => {
  const stream = res.sse({ keepAlive: 15000 });
  let n = 0;
  const t = setInterval(() => stream.send({ event: "tick", data: { n: n++ } }),
    1000);
  stream.closed.then(() => clearInterval(t)); // stop on client disconnect
  return stream.closed;                      // keep the handler alive
},
```

Options. `keepAlive (ms)` sends : ping comment frames at that interval to defeat idle-proxy timeouts (off by default). `retry (ms)` emits a one-time `retry:` line telling the client its reconnect delay.

send(data) . `data` is a string (→ a single `data:` line) or an object `{ event?, data, id?, retry? }`. Object `data` is JSON-encoded; string `data` is passed through, and multi-line string `data` becomes one `data:` line per source line (per the SSE grammar). The returned Promise resolves once the frame is flushed to the socket, and rejects if the stream is already closed.

close() ends the stream and resolves once it is torn down. `closed` is a Promise that resolves on `close()` **or** when the client disconnects — use it to stop timers/producers.

Mechanics. Unlike `upgradeWebSocket`, SSE does **not** hijack the connection — it keeps writing to the same `http.ResponseWriter`. So the request's dispatcher goroutine parks until the stream closes (otherwise net/http would finish the response). A dedicated pump goroutine

owns the writer and flushes each event; the event loop never writes to the socket directly. The listener's `HoldRun` keeps the loop alive while streams are open, exactly like `WebSocket`.

6.6 Lifecycle

Vanilla `sercon script.ts`. Each `server.*.listen` call parks a `HoldRun` sentinel and the event loop stays alive while the listener is bound. `SIGINT` terminates abruptly: the engine's interrupt watcher calls `loop.Terminate()`, the cleanup drain runs (closing each listener), and the process exits. No access log, no readiness signal, no shutdown timeout.

`sercon serve script.ts`. Adds the production niceties documented in §4: structured access log to `stderr`, a `READY` listening on `tcp/...` line on `stdout` per listener, and graceful shutdown with `--shutdown-timeout` (default 30s). Clean `SIGTERM` exits 0. Choose `serve` whenever the script is supervised (systemd, docker compose, launchd).

```
# Quick local test:
sercon examples/scripts/server-http.ts

# Production-style supervised run:
sercon serve examples/scripts/server-http.ts
```

6.7 SMTP

SMTP is a **stateful, multi-stage transaction**: a client connects, optionally authenticates, declares a sender (`MAIL FROM`), one or more recipients (`RCPT TO`), then streams the message body (`DATA`). `sercon` maps each stage to a JavaScript callback so a script can accept or reject the transaction incrementally — refuse an unknown sender at `MAIL`, refuse an unroutable recipient at `RCPT`, or inspect the parsed message at `DATA`. The inbound listener is `server.smtp.listen({...})` the outbound side is `net.email.send({...})` (covered at the end of this section).

6.7.1 `server.smtp.listen({...})`

Synchronously binds the listening socket (so a port already in use throws immediately), then serves in the background. Returns a handle once the loop schedules it.

```
const srv = await server.smtp.listen({
  port: 2525,
  hostname: "mx.example.com",
  handlers: {
    onMail: (env) => env.from.endsWith("@example.com"),
    onRcpt: (env, rcpt) => isLocalMailbox(rcpt),
    onData: (env, msg) => { store(msg); return true; },
  },
});

// srv.address → "tcp/0.0.0.0:2525"
// srv.close() → Promise<void> (immediate — active sessions are severed)
// srv.stopped → Promise<void> (resolves when the listener exits)
```

Options:

Field	Type	Default	Notes
-------	------	---------	-------

port	number	—	Required. TCP port to bind.
host	string	"0.0.0.0"	Bind address.
hostname	string	os.Hostname()	EHLO greeting + Received: identity.
handlers	{onMail, onRcpt, onData}	—	Required. All three functions; see below.
auth	(user, pass, env) => bool Promise<bool>	—	Optional SASL verifier (PLAIN + LOGIN).
starttls	{cert, key}	—	Enables STARTTLS. File paths or inline PEM.
allowInsecureAuth	boolean	false	Permit AUTH on a plaintext connection (dev only).
maxMessageBytes	number	10485760 (10 MB)	DATA size cap; a non-positive value keeps the default.
maxRecipients	number	100	Max RCPT TO per transaction.
sessionTimeout	number	30000	Per-session idle timeout, milliseconds.

Handlers. `onMail(envelope)`, `onRcpt(envelope, recipient)`, and `onData(envelope, message)` are invoked at the matching protocol stage. All three are required. Each may be `async` — a returned Promise is awaited before the SMTP response is written. The return value (or a thrown exception) controls the SMTP reply, uniformly across all three:

Return value	SMTP response	Use case
true / undefined	250 OK	Accept the stage
false	550 Command refused	Generic permanent reject
string	550 <string>	Permanent reject with a reason
throw	451 Temporary failure	Transient error (client should retry)

Envelope (passed to all three handlers; recipients grows as RCPT TO accumulates):

```
type Envelope = {
  from: string; // "" for the null sender (DSN bounces)
  recipients: string[]; // accepted RCPT TO so far
  remote: string; // "1.2.3.4:54321"
  helo: string; // EHLO/HELO hostname the client gave
  authenticatedUser?: string; // set if AUTH succeeded
  tls?: { version: string; cipher: string }; // set under STARTTLS
};
```

Message (only `onData` parsed via `jhillierd/enmime`, with the exact original bytes always available as `raw`):

```
type Message = {
  from: string; // From: header (may differ from
  envelope.from)
  to: string[]; // To: header
  cc: string[]; // Cc: header
  subject: string;
  headers: Record<string, string[]>; // lowercase keys; multi-valued
  body: {
    text: string; // text/plain part; "" if absent
    html: string; // text/html part; "" if absent
  };
  attachments: Array<{
    filename: string;
    contentType: string;
    bytes: Uint8Array;
  }>;
  raw: Uint8Array; // exact DATA bytes (post dot-unstuffing)
};
```

Forwarders and DKIM-style signature verifiers should work from `raw` everyone else uses the parsed fields. `enmime` is lenient with malformed messages, so a script needing strict RFC 5322 conformance can re-parse `raw` itself.

AUTH. Supply an `auth` callback to enable SASL. Both **PLAIN** and **LOGIN** mechanisms are advertised; the callback receives the decoded `(username, password, envelope)` and returns a boolean (or a Promise of one). By default AUTH is refused on a plaintext connection — the client must complete STARTTLS first. Set `allowInsecureAuth: true` to accept credentials over plaintext; this is a **dev-only** escape hatch for trusted local networks and should never be set in production.

STARTTLS. Pass `starttls: {cert, key}` (file paths or inline PEM, same convention as `server.https`) to advertise STARTTLS. The upgrade happens mid-session on the same connection; once active, `envelope.tls` is populated. The listener itself is plaintext at bind time — there is no implicit-TLS (SMTPS / port 465) inbound mode.

Concurrency. Each connection runs on its own Go goroutine, but every handler invocation **serializes on the goja loop** — exactly one handler runs at a time (the same single-threaded model as the HTTP listener in §6.2). A slow `onData` therefore blocks other connections' handlers. For slow work, acknowledge the stage (`return true`) and process in the background (start a Promise without awaiting it).

A short, self-contained round-trip (bind, send to self, assert receipt):

```
const port = 38095;
let captured: { subject: string; from: string } | null = null;

const srv = await server.smtp.listen({
  port,
  hostname: "test.local",
  handlers: {
```

```

    onMail: () => true,
    onRcpt: () => true,
    onData: (env, msg) => {
      captured = { subject: msg.subject, from: env.from };
      return true;
    },
  },
});

await net.email.send({
  to: "alice@test.local",
  from: "bob@test.local",
  subject: "round-trip demo",
  body: "hello from the SMTP demo",
  server: { host: "127.0.0.1", port, tls: "none" },
});

runtime.assert.equal(captured!.subject, "round-trip demo", "subject");
await srv.close();

```

6.7.2 net.email.send({...}) — outbound

The outbound counterpart lives under `net.email` (next to the `spf` / `dmARC` / ... probes). It opens **one TCP connection per call** — no pooling, no queueing, no automatic retries — composes the MIME body in-tree, and returns a per-recipient outcome.

```

const result = await net.email.send({
  to: ["alice@example.com", "bob@example.com"], // string OR string[]
  from: "noreply@my.app",
  subject: "your verification code",
  body: "your code is 123456", // plain text
  html: "<p>your code is <b>123456</b></p>", // optional HTML alternative
  attachments: [
    { filename: "qr.png", contentType: "image/png", bytes: qrBytes },
  ],
  headers: { "X-App": "myapp" }, // optional extra headers
  server: {
    host: "smtp.example.com",
    port: 587, // default 587 (submission)
    auth: { username: "u", password: "p" }, // optional PLAIN auth
    tls: "starttls", // "starttls" (default) |
    "tls" | "none"
  },
  timeout: 30000, // ms; default 30s
});

// result = { accepted: string[], rejected: Array<{ address, reason }> }

```

MIME layout. `text only` → `text/plain; charset=utf-8` (no multipart). `text + html` → `multipart/alternative`. Any attachments → `multipart/mixed` (each part base64, `Content-Disposition: attachment`). `Date`, `Message-ID`, and `MIME-Version` are auto-generated; `headers` are merged on top.

TLS modes. "starttls" (default) connects plaintext then upgrades, refusing AUTH until TLS is active. "tls" is implicit TLS from connect (SMTPS, typically port 465). "none" is plaintext throughout, with AUTH disabled.

Outcomes. Transport-level failures (connection refused, TLS handshake, AUTH, MAIL FROM rejected) **throw** a JS exception. The `rejected` array captures only individual RCPT TO addresses the server permitted us to attempt but then refused — the transaction continues for the rest, so a partial send still delivers to the accepted recipients.

6.8 Raw TCP / UDP

`server.tcp.listen` and `server.udp.listen` are the inbound counterparts to the `net.tcp.connect` / `net.udp.open` clients (§5 net). They bind a listening socket **synchronously** and return the server handle directly — a port already in use throws immediately — then serve in the background, keeping the event loop alive via the same `HoldRun` model as the HTTP and SMTP listeners. `srv.close()` returns a `Promise<void>` (resolving once the listener is closed and accepted connections are drained), matching the rest of the `server.*` family. Passing `port: 0` binds an OS-chosen ephemeral port; read it back from the returned handle's `address`. Unlike the HTTP/SMTP handles, the raw listeners expose **no stopped Promise** — just `address` and `close()`.

Both return a server handle:

```
srv.address;           // "tcp/0.0.0.0:54321" or "udp/0.0.0.0:54321"
srv.close();           // Promise<void> — stop accepting, drain, release HoldRun
```

6.8.1 `server.tcp.listen({...}, conn => {...})`

The second argument is a **connection handler** invoked once per accepted socket. The `conn` it receives is the **same handle shape** as `net.tcp.connect` — there is no separate server-connection type to learn:

```
const srv = await server.tcp.listen({ port: 0 }, (conn) => {
  conn.onData(ev => conn.write(ev.bytes)); // ev = { bytes: Uint8Array, text }
  conn.onClose(() => runtime.log("peer disconnected"));
  conn.onError(e => runtime.log("conn error", String(e)));
  // conn.write(data) — string (UTF-8) or Uint8Array
  // conn.remote / conn.local — peer / local "host:port"
  // conn.close() — half/full close this connection
});

const port = Number(srv.address.split(":").pop()); // "tcp/0.0.0.0:PORT"
// ... net.tcp.connect("127.0.0.1", port) to talk to it ...
await srv.close();
```

Options: `port` (required; 0 for ephemeral), `host` (default all interfaces — pass "127.0.0.1" for loopback-only), `readBuffer` (per-connection inbound channel capacity — frames buffered before backpressure, default 64 same meaning as `net.tcp.connect`).

6.8.2 `server.udp.listen({...}, (msg, reply) => {...})`

UDP is connectionless, so the handler fires **once per inbound datagram** rather than once per connection. `msg` describes the datagram and its sender; `reply` sends a datagram back to that same sender:

```
const srv = await server.udp.listen({ port: 0 }, (msg, reply) => {
  // msg = { bytes: Uint8Array, text, address, port } — the sender
  runtime.log("got", msg.text, "from", msg.address + ":" + msg.port);
  reply("ack:" + msg.text); // string or Uint8Array; returns a Promise
});

const port = Number(srv.address.split(":").pop()); // "udp/0.0.0.0:PORT"
await srv.close();
```

Options: port (required; 0 for ephemeral), host (default all interfaces — pass "127.0.0.1" for loopback-only). UDP has no `readBuffer` option; inbound datagrams are read into a fixed 64 KiB buffer.

Lifecycle. Identical to §6.6: vanilla `sercon script.ts` keeps the loop alive while bound and terminates abruptly on SIGINT; `sercon serve script.ts` adds the access log, a `READY` listening on `tcp/...` (or `udp/...`) line on stdout per listener, and graceful shutdown. As with all `server.*` bindings, the connection / datagram handlers marshal back onto the single goja loop, so **handlers serialize** (one at a time across all listeners).

- `server.icmp.listen(opts?, (msg, reply) => ...)` — bind a raw ICMP listener. **Requires root / CAP_NET_RAW** (synchronous bind throws otherwise); raw ICMP has no ports, so the socket receives **all** host ICMP traffic. `opts { network?: "ip4" | "ip6" (default "ip4") }` (no `readBuffer`). The handler runs per received packet with `msg { bytes, text, address, type, code }` and a `reply(opts?)` that sends an ICMP message back to the sender (or `opts.to`) — Echo mode `{ type?, code?, id?, seq?, payload? }` or raw mode `{ type, code?, body }`, the same options as `net.icmp send`. The handle is `{ address: "icmp/<addr>", close() }` it emits a `READY` line under `sercon serve` and joins graceful shutdown.

6.9 Concepts

Every `server.*` listener follows the same lifecycle, whichever protocol you pick:

`await server.<proto>.listen({...})` binds **synchronously** (a port already in use throws before any Promise settles) and returns a **handle** — `address` (a `proto/host:port` string; `server.tcp.listen / server.udp.listen` accept `port: 0` for an OS-assigned ephemeral port, while `server.http`, `server.https`, and `server.smtp` require an explicit port) and `close()`. Binding parks a `HoldRun` sentinel (§6.1) so the engine's event loop stays alive while the listener is up; a script that just does `await server.http.listen(...)` and returns will keep running until something calls `close()` — that's why every example below ends with `await srv.close()`, exactly as the source scripts do. All handlers — HTTP routes, WebSocket loops, SMTP stage callbacks, TCP connection callbacks — run marshalled back onto the single goja loop, so **handlers serialize**: this rules out JS-level data races between requests, at the cost of no handler-level parallelism.

For `server.http / server.https`, routes are keyed by stdlib `http.ServeMux` Go 1.22+ patterns (`"GET /path"`, `"GET /items/{id}"`, `"GET /assets/{rest...}"`), and every handler receives a fluent `res` builder — `res.status(code)` is chainable into `.json(obj)`, `.text(s)`, `.html(s)`, or `.empty()`. Global middleware (`use: [...]`) and an optional `onError(err, req, res)` wrap the whole route table; see §6.4 and §6.2 for the full option surface. Full method reference: §17.10.

6.10 Recipes

6.10.1 Serve a routed JSON API with middleware

A request logger, bearer-token auth, and a try/catch error-catcher compose via `use: [...]`, applied in order to every route; a route can still short-circuit by not calling `next()`:

```
const TOKEN = "s3cr3t";

const requestLogger = async (req: any, res: any, next: any) => {
  const t0 = runtime.time.nowMs();
  await next();
  runtime.log(`${req.method} ${req.path} → ${runtime.time.nowMs() - t0}ms`);
};

const authMiddleware = async (req: any, res: any, next: any) => {
  if (req.path === "/health") return next();
  const authHeader: string = (req.headers["authorization"] ?? [""])[0];
  if (authHeader !== `Bearer ${TOKEN}`) {
    res.status(401).json({ error: "unauthorized" });
    return; // do NOT call next – response is finalized
  }
  return next();
};

const srv = await server.http.listen({
  port: 38200,
  use: [requestLogger, authMiddleware],
  routes: {
    "GET /health": (_req: any, res: any) => res.json({ ok: true }),
    "GET /items": (_req: any, res: any) =>
      res.json(Array.from(items.values())),
    "POST /items": (req: any, res: any) => {
      const body = JSON.parse(req.body || "{}");
      if (!body.name) return res.status(400).json({ error: "name required" });
      const id = nextId++;
      items.set(id, { id, name: body.name });
      res.status(201).json({ id, name: body.name });
    },
  },
});

await srv.close();
```

runnable: `examples/scripts/advanced/http-api.ts`

6.10.2 Serve static files

`server.http.static({dir, stripPrefix})` returns a route handler you mount under a wildcard pattern — it's a plain `RouteValue`, so it composes with the rest of your route table:

```
const srv = await server.http.listen({
  port: 38081,
  routes: {
    "GET /assets/{rest...}": server.http.static({
      dir: root,
```

```

        stripPrefix: "/assets/",
    }),
    "GET /": (req: any, res: any) => res.text("home"),
  },
});

await srv.close();

```

runnable: `examples/scripts/server-static.ts`

6.10.3 Handle a WebSocket

A route handler calls `res.upgradeWebSocket()` to hijack the connection; the returned handle is an `AsyncIterable<WSMessage>`, so `for await` walks frames until you `break` and close:

```

const srv = await server.http.listen({
  port: 38082,
  routes: {
    "GET /ws": async (req: any, res: any) => {
      const ws = await res.upgradeWebSocket();
      for await (const msg of ws) {
        if (msg.type === "text" && msg.text === "bye") {
          await ws.close(1000, "bye");
          break;
        }
        if (msg.type === "text") {
          await ws.send("echo:" + msg.text);
        }
      }
    },
  },
});

await srv.close();

```

runnable: `examples/scripts/server-ws.ts`

6.10.4 Receive email

`server.smtp.listen` takes per-stage handlers — `onMail` / `onRcpt` accept or reject the envelope, `onData` receives the fully parsed message:

```

let captured: { subject: string; body: string; from: string } | null = null;

const srv = await server.smtp.listen({
  port: 38095,
  hostname: "test.local",
  handlers: {
    onMail: () => true,
    onRcpt: () => true,
    onData: (env: any, msg: any) => {
      captured = { subject: msg.subject, body: msg.body.text, from: env.from };
      return true;
    },
  },
});

```

```
});

// ... net.email.send({... , server: { host: "127.0.0.1", port, tls: "none" }}) ...

await srv.close();
```

runnable: `examples/scripts/server-smtp.ts`

6.10.5 Run a TCP line server

`server.tcp.listen({...}, conn => {...})` invokes the connection handler once per accepted socket; `conn` is the same handle shape as `net.tcp.connect` — `onData` / `onClose` / `onError` / `write` / `close`:

```
const srv: any = await server.tcp.listen({ port: 0 }, (conn: any) => {
  conn.onData((ev: any) => conn.write(ev.bytes)); // echo bytes back
  conn.onError((e: any) => runtime.log("server conn error", String(e)));
});

const port = Number(srv.address.split(":").pop()); // "tcp/0.0.0.0:PORT"
// ... net.tcp.connect("127.0.0.1", port) to talk to it ...

await srv.close();
```

runnable: `examples/scripts/server-tcp.ts`

Notes

- Method names and option shapes above are confirmed against §17.10 (the source of truth for the full surface) and `examples/scripts/sercon.d.ts`.

7. JavaScript runtime built-ins (goja)

`scriptengine` runs on goja, which implements **ES5.1 + a large subset of ES6+**. The following are present and behave per spec:

7.1 Globals

- `Object`, `Array`, `String`, `Number`, `Boolean`, `BigInt`
- `Math`, `Date`, `JSON`, `RegExp`
- `Error`, `TypeError`, `RangeError`, `SyntaxError`, `ReferenceError`, `EvalError`, `URIError`
- `Promise` (provided via goja's native implementation; resolution microtasks are pumped by the event loop)
- `Symbol` (partial: well-known symbols `Symbol.iterator`, `Symbol.asyncIterator`, etc., are supported)
- `Proxy`, `Reflect` (partial)

```
runtime.log(Math.PI.toFixed(4), Math.max(1, 9, 3));
runtime.log(new Date().toISOString());
runtime.log(JSON.stringify({ a: 1, b: [2, 3] }));
runtime.log("abc".repeat(3), "abc".padStart(6, "_"));
```

7.2 Collections

- `Map`, `Set`, `WeakMap`, `WeakSet`

```
const counts = new Map<string, number>();
for (const ch of "banana") counts.set(ch, (counts.get(ch) ?? 0) + 1);
runtime.log([...counts]); // [{"b",1}, {"a",3}, {"n",2}]
```

7.3 Typed arrays

- `ArrayBuffer`, `DataView`
- `Int8Array`, `Uint8Array`, `Uint8ClampedArray`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `Float32Array`, `Float64Array`

```
const buf = new ArrayBuffer(4);
new DataView(buf).setUint32(0, 0xCAFEBAE);
runtime.log(new Uint8Array(buf)[0].toString(16)); // ca
```

7.4 Iteration and generators

- `for...of`, `for...in`, `spread`, `destructuring`
- Generator functions (`function*`)
- Async generators (`async function*`) — supported via goja's event loop

```
function* fib() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
const it = fib();
const first10 = Array.from({ length: 10 }, () => it.next().value);
runtime.log(first10);
```

7.5 Not (yet) provided

- `fetch` — use `net.http.*` from the CLI, or register your own binding.
- `Worker`, threading primitives.
- DOM globals (`window`, `document`).
- Native ES modules (`import` / `export` are *transpiled* to CommonJS at load time — see section 9).

8. Async runtime additions (goja_nodejs)

The event loop bundles a small Node-compatible runtime on top of goja:

8.1 Timers

```
setTimeout(() => runtime.log("after 50ms"), 50);
setInterval(() => runtime.log("tick"), 100);
setImmediate(() => runtime.log("next tick"));
clearTimeout(t); clearInterval(i); clearImmediate(im);
```

The event loop holds the script alive while *any* timer is pending. If your script ends with only `setTimeout` callbacks scheduled, the run will wait for them to fire before returning.

8.2 console

```
console.log("info");
console.error("oops");
console.warn("careful");
console.info("fyi");
console.debug("trace");
```

Disable with `Options.DisableConsole = true` if you want a clean global namespace.

8.3 require

CommonJS `require`, with TS-aware extension fallback added by `sercon` (see section 11):

```
const helpers = require("./helpers");
helpers.doThing();
```

Both `require("./foo")` and `import { x } from "./foo"` resolve to the same module instance per Run; esbuild rewrites `import` to `require` at transpile time.

9. TypeScript support

TS is transpiled by esbuild in-process. There is no `tsc`, no `tsconfig.json`, no type-checking — types are erased and the resulting JS is what executes. Practical implications:

- All TS syntax esbuild supports is allowed: type annotations, `interface`, `type`, generics, enums (compiled to objects), namespaces, decorators (legacy & TC39 stage 3).
- `import type { X } from "y"` is stripped entirely (no runtime require).
- TS errors are *not* surfaced as build errors — only esbuild parse errors are. Run a separate `tsc --noEmit` in CI if you want type enforcement.
- `tsconfig.json` is **not** consulted.
- **Runtime errors report TypeScript positions.** Stack traces in thrown errors point at the original `.ts` line/column, not the transpiled JS — for both the entry script and imported `.ts` / `.tsx` modules, and for both synchronous throws and async (top-level- `await`) rejections. This is done with inline source maps that `goja` consumes natively; the entry script's ESM→CJS rewrite is line-shift-aware so its frames map correctly too. If a map is ever unavailable the trace falls back to transpiled-JS positions.

`.tsx` and `.jsx` are supported via esbuild's `LoaderTSX` / `LoaderJSX`, including for required modules resolved by the engine's extension fallback. JSX has no React runtime in scope by default — either set the factory at the file level with an `@jsx` pragma:

```
/** @jsx h */
function h(tag: string, props: any, ...children: any[]) {
  return { tag, props: props ?? {}, children };
}
export const Box = (label: string) => <div className="box">{label}</div>;
```

...or provide a `React.createElement`-compatible binding from `Go` and rely on esbuild's default pragma. ESM `export default <value>` also works through the entry-script rewriter's `__esModule ? .default : m` unwrap, so `import answer from "./mod"` resolves to the default export.

An **entry script's own** `export default <expr>` is captured as the value `Engine.Run` resolves to, and the `sercon` CLI prints that value as JSON to stdout (scripts without a default export resolve to `undefined` and print nothing; a value that doesn't JSON-encode, such as a function, is skipped). This is the supported way for a script to emit a structured result.

10. Top-level `await`

Top-level `await` works in entry scripts:

```
const r = await net.http.get("https://example.com");
runtime.log(r.status);
```

How it works under the hood: esbuild rejects top-level `await` with the `cjs` output format, so the engine transpiles the *entry* script with `format=esm`, then rewrites `import` statements to `require()` calls and wraps the rest of the body in:

```
;(async () => { /* your transpiled body */ })().then(__resolve, __reject);
```

`__resolve` and `__reject` are bindings the engine sets on the runtime to capture the final settlement. *Required-module* code (anything loaded through `require`) is transpiled with `format=cjs` and does **not** support top-level `await` — wrap in `(async () => ... )()` yourself if you need it inside a module.

11. Module resolution

The engine plugs a custom source loader into `goja_nodejs/require` that adds:

1. **Direct path:** if the requested file exists on disk, use it. `.ts` / `.tsx` files are transpiled on the fly.
2. **Extension fallback** when the request has no extension: try `.ts`, `.tsx`, `.js`, `.cjs`, `.mjs`, `.json` in that order.
3. **.js → .ts swap:** if a path ending in `.js` / `.cjs` / `.mjs` is requested but the literal file doesn't exist, the same path ending in `.ts` (or `.tsx`) is tried. Handles `package.json` `main` fields that point at compiled output where only the TypeScript source is on disk.
4. **package.json source preference:** when reading a `package.json`, if a `source` field is present and points at an existing `.ts` / `.tsx` file, `main` is rewritten to that path before the registry consumes it. Matches the convention used by `parcel`, `microbundle`, and similar bundlers to mark the TS source of truth.
5. **Directory index:** if the resolved path is a directory, try `index.ts`, `index.tsx`, `index.js`.

The base directory follows Node's CommonJS rules — relative imports resolve against the requiring module's directory, courtesy of `goja_nodejs`.

```
// All resolve to ./helpers/assert.ts:
import { check } from "./helpers/assert";
import { check } from "./helpers/assert.ts";
const { check } = require("./helpers/assert");
const { check } = require("./helpers/assert.js");
```

JSON imports work out of the box via either the ESM-style default import (esbuild rewrites it to a `require`) or a direct `require`:

```
import data from "./data.json";
const r = require("./data.json");
```

`node_modules` lookup *works* via the upstream registry, but the source loader doesn't currently transpile through that path — keep TS helpers under `ScriptRoot`.

12. Timeouts and cancellation

Two mechanisms interrupt a running script:

- `Options.Timeout` — wall-clock limit per `Run`. Exceeding it returns `scriptengine.ErrScriptTimeout`.
- `ctx` passed to `Run` / `RunFile` — cancelling the context returns `ctx.Err()` (typically `context.Canceled` Or `context.DeadlineExceeded`).

Both call `vm.Interrupt(...)` and `loop.Terminate()` from a watcher goroutine. This works for **sync** JS — including `while(true){}` — because the goja VM checks the interrupt flag between bytecode steps. A host Go binding that's blocked in `cgo` or a long syscall isn't interruptible mid-call; if you write such a binding, plumb the engine `ctx` through to it yourself.

```
eng := scriptengine.New(scriptengine.Options{Timeout: 200 * time.Millisecond})
_, err := eng.Run(ctx, "loop.ts", `while (true) {}`)
// err is scriptengine.ErrScriptTimeout, returned ~200-300ms after Run.
```

13. Error semantics

- A Go binding returning `(T, error)` automatically throws as a JS exception when `err != nil`. The exception's `.message` is `err.Error()`.

```
try { mayFail(); } catch (e) { runtime.log(String(e)); }
```

- A Go binding that `panic`s with a `*goja.Object` (e.g. `vm.NewGoError(...)`) does the same.
- A script throwing a JS error out of the top level surfaces from `Run` as a Go `error` whose message is the JS error's `.message`.
- Timeout errors satisfy `errors.Is(err, scriptengine.ErrScriptTimeout)`. Cancellation errors satisfy `errors.Is(err, context.Canceled)` Or `context.DeadlineExceeded`.

14. Type generation (.d.ts)

`Engine.WriteTypes(w)` walks the registered bindings and emits a TypeScript declaration file. The mapping table (abbreviated):

Go type	TS type
<code>string</code>	<code>string</code>
any numeric / <code>float64</code>	<code>number</code>
<code>bool</code>	<code>boolean</code>
<code>[]T</code>	<code>T[]</code> <code>[]byte</code> → <code>Uint8Array</code>
<code>map[string]T</code>	<code>Record<string, T></code>
<code>*T</code>	<code>T</code> (transparent unwrap)

error (single return)	omitted (collapses to <code>void</code>)
(<code>T</code> , error) (tuple return)	<code>T</code>
<code>interface{}</code> / <code>any</code>	<code>unknown</code>
<code>goja.FunctionCall</code> parameter	folded into (<code>...args: unknown[]</code>)
<code>goja.Value</code> return	<code>unknown</code>
<code>scriptengine.AsyncBinding</code> value (from <code>PromisifyAsync[T]</code>)	<code>Promise<T></code>
self-referential types	<code>unknown</code> (cycle detection bails after depth 4)

```
sercon --emit-dts sercon.d.ts
```

In your editor, place the resulting `.d.ts` next to your scripts (or reference it via `/// <reference path="..." />`) for autocomplete.

14.1 JSDoc comments on declarations

The emitter pulls JSDoc strings from the engine's doc map (set via `Engine.SetDocs` / `Engine.SetMemberDocs`) and renders them above each declaration. Single-line docs collapse to `/** ... */` multi-line docs expand to a standard `*`-prefixed block. Bindings without a doc entry emit no JSDoc — the emitter never inserts empty placeholders. Docs are looked up by the same dotted path the binding was registered under, so namespace members get hover text too:

```
// AUTO-GENERATED by scriptengine.Engine.WriteTypes – do not edit by hand.

/** Hashing primitives. */
declare const crypto: {
  hash: {
    /** SHA-256 hex digest of a UTF-8 input. */
    sha256(...args: unknown[]): string;
    /** BLAKE3 hex digest (32-byte output, lukechampine.com/blake3). */
    blake3(...args: unknown[]): string;
    // ...
  };
  // ...
};
```

The CLI's reserved-global surface ships with a curated doc map; see `cmd/sercon/docs.go` for the source of truth and add new entries there when adding new bindings (the lockstep rule keeps `--examples` / `MANUAL` / `sercon.d.ts` in sync, and the doc map is now the seventh artifact in that chain).

15. Limitations and gotchas

- **No native ES modules at runtime.** All `import` is rewritten to `require` by esbuild. Side-effect-only imports run the module body exactly once per Run.
- **No top-level await inside required modules.** Wrap with `(async () => ... )()` inside the module if you need it.

- **No `tsconfig.json` honoured.** Module resolution and type checking are independent from any project-level TS config.
- **Per-Run runtime.** Side effects on `globalThis` do not persist between calls to `Run` on the same `Engine`. The require *compile* cache is shared; module *exports* are not.
- **Multi-line ESM imports may stress the entry rewriter.** The current rewriter is line-based and handles the common forms; especially gnarly inputs (comments inside the spec list, very unusual whitespace) fall back to executing as-is, which may throw.
- **RegisterConstructor gives real new semantics.** `new Foo(...)` runs the registered Go constructor; JS arguments are coerced to its parameter types (an argument that can't be coerced becomes the zero value rather than throwing); the result's exported methods/fields are reachable (subject to the `json` -tag field mapper); and a non-nil trailing `error` result throws. A panic inside the constructor propagates like any Go binding (it is not converted to a catchable JS error), and `instanceof Foo` is not guaranteed (the instance is the Go-wrapped object). This is a library-side API — the CLI registers namespaces, not constructors.
- **HTTP bindings are real network calls.** `net.http.*` uses `net/http` with a 5s timeout. They are not mockable from JS.

See OUT-OF-SCOPE.md for the active backlog of deferred ideas.

16. Bundled libraries

16.1 paymentproviders

A TypeScript payments library compiled into the `sercon` binary and importable by scripts — no install needed. It is **not** a reserved global; you `import` it:

```
import { kcov3 } from "paymentproviders";
const api = kcov3.client(); // reads KCO_* from the environment
const order = await api.getPayment(orderId);
await api.capturePayment(orderId, { amount: order.order_amount });
```

The module exposes one **version-labelled namespace per provider** — `kcov3`, `netstv1`, `sveacheckout2`, `qlirov2`, `swedbankpayv2`, `swedbankpayv3`. The version suffix is deliberate: a future API revision (e.g. `netstv2`) ships alongside the existing namespace instead of silently changing behaviour under callers, so a script pinned to `netstv1` keeps working.

Each namespace exports a single `client(overrides?)` factory that returns a ready-to-use client object. `client()` is **synchronous** and throws plainly (a regular `Error`, not `PaymentError`) if a required credential is missing — every provider validates its env vars up front. The client methods are all `async` and return the parsed JSON response.

16.1.1 Shared core

All six providers are built on a small shared core (`core/`):

- **Config resolution.** Every credential and the environment selector can come from the environment **or** be overridden per call. Precedence is `overrides.X ?? env(PROVIDER_X)`. The base URL is chosen by `pickBaseUrl`: an explicit `baseUrl` (override or `PROVIDER_BASE_URL`) always wins and has trailing slashes trimmed; otherwise `env === "prod"` selects the production URL and anything else (including `unset`) selects the test URL. So **test is the default** when `*_ENV` is absent.

- **Pluggable per-request signer.** Auth is not baked into the HTTP layer. Each client supplies a `sign(method, path, bodyStr)` function that returns the auth headers to merge for that request. This is what lets body-signing providers (Svea, Qliro) compute a per-request hash over the serialized body, while header/Bearer providers (KCO, Nets, SwedbankPay) ignore the arguments.
- **apiRequest** . The single request path: sets `accept: application/json` , serializes the body (when present) as JSON with `content-type: application/json` , calls the signer, issues the call via `net.http.request` (`follow: true` so redirects are honoured), and parses the response body as JSON (falling back to the raw string if it is not valid JSON).
- **Non-2xx throws PaymentError** . Any status outside 200–299 throws a `PaymentError` instead of returning. Its shape:

```
class PaymentError extends Error {
  provider: string; // e.g. "kco3"
  status: number; // the HTTP status
  body: unknown; // parsed JSON error body (or raw string)
  requestId?: string; // from x-correlation-id / klarna-correlation-id, if
  present
}
```

Catch it to branch on `status` (e.g. distinguish a 404 from a 401).

- **Idempotency keys.** Mutating KCO calls send an auto-generated `klarna-idempotency-key` by default; the generator (`idempotencyKey()`) is a timestamp+random base36 string. Every `kco3` mutation also accepts an optional `{ idempotencyKey? }` as its last argument so a caller can pin a stable key across retries of the same logical call — see §16.1.8.7.
- **Amounts are integer minor units** (öre / cents), matching the provider APIs.
`core/money.ts` exposes `major / minor` helpers if you need to convert.

What follows is a per-provider reference: auth model, env vars, base URLs, the method table, and a worked example.

16.1.2 kco3 — Kustom / Klarna Checkout v3

Wraps Kustom’s Order-Management v1 + Checkout v3 APIs.

- **Auth.** HTTP Basic, `Authorization: Basic base64(merchantId:sharedSecret)` .
- **Env vars.** `KCO_MERCHANT_ID` , `KCO_SHARED_SECRET` (both required), `KCO_ENV` (`test` | `prod`), `KCO_BASE_URL` (overrides env selection).
- **Base URLs.** `test` `https://api.playground.kustom.co` · `prod` `https://api.kustom.co` .
- **Idempotency.** All POST mutations send a fresh `klarna-idempotency-key` by default; pass `{ idempotencyKey }` as the trailing `opts` argument to pin a caller-chosen key instead (e.g. across retries).

Method	Params	Returns	Throws
<code>getPayment(id)</code>	order id	the order-management order	<code>PaymentError</code> on non-2xx
<code>acknowledge(id, opts?)</code>	order id + <code>{ idempotencyKey? }</code>	ack result	<code>PaymentError</code>

capturePayment(id, { amount, orderLines?, description? }, opts?)	id + capture input (sent as captured_amount / order_lines / description) + { idempotencyKey? }	capture result	PaymentError
refundPayment(id, { amount, orderLines?, description? }, opts?)	id + refund input (sent as refunded_amount / order_lines / description) + { idempotencyKey? }	refund result	PaymentError
cancelPayment(id, opts?)	order id + { idempotencyKey? }	cancel result	PaymentError
releaseRemainingAuthooid{id, opts?)	oid{id, { idempotencyKey? }	release result	PaymentError
createCheckout(order)	checkout order object	created checkout order	PaymentError
getCheckout(id)	checkout order id	checkout order	PaymentError

```
import { Kcov3 } from "paymentproviders";
const api = Kcov3.client(); // KCO_* from env (test by default)
const order = await api.getPayment("ord_123");
if (order.status === "AUTHORIZED") {
  await api.capturePayment("ord_123", { amount: order.order_amount });
}
```

16.1.3 netsv1 — Nexi / Nets Checkout Payment API v1

- **Auth.** The secret key is sent **verbatim** as the Authorization header value (no Bearer prefix — Nexi's scheme).
- **Env vars.** NETS_SECRET_KEY (required), NETS_ENV (test | prod), NETS_BASE_URL .
- **Base URLs.** test https://test.api.dibspayment.eu · prod https://api.dibspayment.eu .

Method	Params	Returns	Throws
createPayment(payment)	full payment-create body	created payment (incl. paymentId)	PaymentError
getPayment(id)	payment id	payment	PaymentError
capturePayment(id, { amount, orderItems? })	id + charge input (posts to /charges)	charge result	PaymentError
refundPayment(id, { amount, orderItems? })	id + refund input (posts to /refunds)	refund result	PaymentError
cancelPayment(id)	payment id (PUT to /terminate)	termination result	PaymentError

```
import { netsvl } from "paymentproviders";
const api = netsvl.client();
const created = await api.createPayment({ /* checkout, order, ... */ });
const p = await api.getPayment(created.paymentId);
await api.capturePayment(created.paymentId, { amount: p.summary.reservedAmount });
```

16.1.4 sveachekout2 — Svea Checkout

- **Auth.** Body-signed. The signer computes `hash = UPPER(SHA512(body + secretKey + timestamp))`, then `token = base64(merchantId:hash)`, and sends `Authorization: Svea <token>` plus a `Timestamp` header (UTC, YYYY-MM-DD HH:MM:SS). For GETs the body string is "", so the hash signs `secretKey + timestamp`.
- **Env vars.** `SCO_MERCHANT_ID`, `SCO_SECRET_KEY` (both required), `SCO_ENV` (`test` | `prod`), `SCO_BASE_URL`.
- **Base URLs.** `test https://checkoutapistage.svea.com · prod https://checkoutapi.svea.com`.

Method	Params	Returns	Throws
<code>createOrder(order)</code>	order body (POST <code>/api/orders</code>)	created order	<code>PaymentError</code>
<code>getOrder(id)</code>	order id	order	<code>PaymentError</code>
<code>getPayment(id)</code>	order id (alias of <code>getOrder</code> — the order is the payment)	order	<code>PaymentError</code>
<code>capturePayment(id, { amount, rows? })</code>	id + delivery input (POST <code>/deliveries</code>)	delivery result	<code>PaymentError</code>
<code>refundPayment(id, { amount, rows? })</code>	id + credit input (POST <code>/credits</code>)	credit result	<code>PaymentError</code>
<code>cancelPayment(id)</code>	order id (POST <code>/cancel</code>)	cancel result	<code>PaymentError</code>

```
import { sveachekout2 } from "paymentproviders";
const api = sveachekout2.client();
const order = await api.getOrder("12345");
await api.capturePayment("12345", { amount: order.OrderAmount });
```

16.1.5 qlirov2 — Qliro One

- **Auth.** Body-signed. `token = base64(SHA256(body + apiPassword))` sent as `Authorization: Qliro <token>`. For bodyless GETs the body string is "", so the token signs just the secret — Qliro's scheme for GETs.
- **Env vars.** `QLIRO_API_KEY`, `QLIRO_APIPASSWORD` (both required), `QLIRO_ENV` (`test` | `prod`), `QLIRO_BASE_URL`.
- **Base URLs.** `test https://pago.qit.nu · prod https://payments.qit.nu`.
- **Management calls.** `capturePayment` / `refundPayment` / `cancelPayment` hit the Admin API v2 (`MarkItemsAsShipped` / `ReturnItems` / `cancelOrder`). The library injects `RequestId` (a UUID v4), `OrderId`, and `MerchantApiKey` into the signed body for you; `MerchantApiKey` is

injected last so a caller's `input` cannot override it. `createOrder` likewise injects `MerchantApiKey` last.

Method	Params	Returns	Throws
<code>createOrder(order)</code>	order body (<code>MerchantApiKey</code> injected)	created order	<code>PaymentError</code>
<code>getOrder(id)</code>	order id (string number; GET)	order	<code>PaymentError</code>
<code>getPayment(id)</code>	order id (alias of <code>getOrder</code>)	order	<code>PaymentError</code>
<code>capturePayment(orderId, input?)</code>	order id + extra fields (→ <code>MarkItemsAsShipped</code>)	capture result	<code>PaymentError</code>
<code>refundPayment(orderId, input?)</code>	order id + extra fields (→ <code>ReturnItems</code>)	refund result	<code>PaymentError</code>
<code>cancelPayment(orderId)</code>	order id (→ <code>cancelOrder</code>)	cancel result	<code>PaymentError</code>

```
import { qlirov2 } from "paymentproviders";
const api = qlirov2.client();
const order = await api.getOrder(987654);
await api.capturePayment(987654, { /* OrderItemActions, etc. */ });
```

16.1.6 swedbankpayv2 / swedbankpayv3 — SwedbankPay Checkout (HAL)

Both versions share one client (`swedbankpay/common.ts`); they differ only in their `version` label — the create path (`/psp/paymentorders`) and the HAL model are identical. The request **payload** shape you supply to `createPaymentOrder` is where v2 and v3 diverge (that is the caller's responsibility).

- **Auth.** Authorization: Bearer <accessToken> .
- **Env vars.** `SWEDBANKPAY_ACCESS_TOKEN` , `SWEDBANKPAY_MERCHANT_ID` (both required; the merchant id is the `payee.payeeId`), `SWEDBANKPAY_ENV` (`test` | `prod`), `SWEDBANKPAY_BASE_URL` .
- **Base URLs.** `test` `https://api.externalintegration.payex.com` · `prod` `https://api.payex.com` .
- **HAL / hypermedia.** SwedbankPay responses carry an `operations` array of `{ rel, href, method }` links. The `operation(paymentOrderOrUrl, rel, body?)` primitive resolves a `rel` and POSTs (or uses the link's method) to its absolute `href` . The first argument may be an already-fetched payment-order payload **or** an id/URL string (which is GET-fetched first). Rel matching is exact or by hyphen-suffix, so `"capture"` also matches a qualified rel like `"create-paymentorder-capture"` . If the rel is not present on the payment order, `operation` throws a plain `Error` (not `PaymentError`). The `capturePayment` / `refundPayment` / `cancelPayment` convenience methods are thin wrappers over `operation` using the `capture` / `reversal` / `cancel` rels.
- The returned client also exposes `merchantId` as a plain string field.

Method	Params	Returns	Throws
--------	--------	---------	--------

<code>createPaymentOrder(body)</code>	full payment-order body (POST /psp/paymentorders)	created payment order (with operations)	PaymentError
<code>getPaymentOrder(idOrUrl)</code>	id or absolute URL (GET)	payment order	PaymentError
<code>getPayment(idOrUrl)</code>	alias of <code>getPaymentOrder</code>	payment order	PaymentError
<code>operation(paymentOrder, rel, body?)</code>	fetches PO or id/URL, the HAL rel, optional body	the operation's response	Error if rel absent; PaymentError on non-2xx
<code>capturePayment(paymentOrder, body)</code>	PO/id/URL + capture body (rel capture)	capture result	Error / PaymentError
<code>refundPayment(paymentOrder, body)</code>	PO/id/URL + reversal body (rel reversal)	reversal result	Error / PaymentError
<code>cancelPayment(paymentOrder, body?)</code>	PO/id/URL + optional body (rel cancel)	cancel result	Error / PaymentError

```
import { swedbankpayv3 } from "paymentproviders";
const api = swedbankpayv3.client();
const po = await api.getPaymentOrder("/psp/paymentorders/abc123");
// Capture using the HAL operation resolved from the fetched payment order:
await api.capturePayment(po, {
  transaction: { amount: 1500, vatAmount: 300, description: "Capture",
  payeeReference: "cap-1" },
});
```

16.1.7 Understanding payment providers

All six providers wrap the same payment lifecycle, so learn it once and map it onto each. (Snippets here are illustrative — see §16.1's lead.)

Lifecycle. create → get status → capture → refund → cancel :

- **create** a checkout / order / payment order (this authorises an amount).
- **capture** moves money — up to the authorised amount, in one or more calls (partial captures).
- **refund** returns captured money — up to what was captured.
- **cancel / void** releases an authorisation that was never (fully) captured.

Amounts are integer minor units (öre / cents) everywhere — 15000 means 150.00 SEK, never 150 .

Auth & the shared core (§16.1.1). Auth varies by provider, but the library handles it for you either way — you just supply credentials: **kcov3** sends **HTTP Basic** (base64(merchantId:sharedSecret)); **netsv1** sends its secret key **as the Authorization header value verbatim** (no Basic / Bearer scheme); **svea** and **qlirov2** **sign the request body** (a computed signature over the serialised JSON — svea uses SHA-512, qliro SHA-256); **swedbankpayv2/v3** send a **Bearer access token** (Authorization: Bearer <accessToken>). **kcov3** mutations carry an idempotency key —

auto-generated fresh per call by default, or pass `{ idempotencyKey }` to pin a stable value across retries so a retried capture/refund isn't double-applied (see §16.1.8.7). Any non-2xx throws a `PaymentError { provider, status, body, requestId? }`.

Config & test-vs-prod. `client(overrides?)` reads `env` with precedence

`overrides.X ?? env(PROVIDER_X)`. It targets the **test** environment by default; `env: "prod"` (or `PROVIDER_ENV=prod`) selects production, and an explicit `baseUrl` always wins.

Provider	Env vars
kcov3	KCO_MERCHANT_ID , KCO_SHARED_SECRET , KCO_ENV , KCO_BASE_URL
netstv1	NETS_SECRET_KEY , NETS_ENV , NETS_BASE_URL
sveacheckout2	SCO_MERCHANT_ID , SCO_SECRET_KEY , SCO_ENV , SCO_BASE_URL (note: SCO_ , not SVEA_)
qlirov2	QLIRO_API_KEY , QLIRO_APIPASSWORD , QLIRO_ENV , QLIRO_BASE_URL (note: QLIRO_APIPASSWORD , no underscore)
swedbankpayv2 / swedbankpayv3	SWEDBANKPAY_ACCESS_TOKEN , SWEDBANKPAY_MERCHANT_ID , SWEDBANKPAY_ENV , SWEDBANKPAY_BASE_URL

Method-equivalence table. Only *create/get* differ by provider; capture/refund/cancel share names:

Step	kcov3	netstv1	sveacheckout2	qlirov2	swedbankpayv2/v3
create	createCheckout	createPayment	createOrder	createOrder	createPaymentOrder
get	getCheckout / getPayment	getPayment	getOrder / getPayment	getOrder / getPayment	getPaymentOrder / getPayment
capture	capturePayment	capturePayment	capturePayment	capturePayment	capturePayment ¹
refund	refundPayment	refundPayment	refundPayment	refundPayment	refundPayment ¹
cancel	cancelPayment	cancelPayment	cancelPayment	cancelPayment	cancelPayment ¹

¹ SwedbankPay's capture/refund/cancel are thin wrappers over

`operation(paymentOrder, "capture" / "reversal" / "cancel", body)` `operation()` also follows any other HAL rel (see 16.1.8). `kcov3` additionally has `acknowledge` and `releaseRemainingAuthorization`. Capture/refund input shapes vary: `kcov3` `{ amount, orderLines?, description? }`, `netstv1` `{ amount, orderItems? }`, `sveacheckout2` `{ amount, rows? }`, `qlirov2` — optional management fields merged into the signed `{ RequestId, OrderId, MerchantApiKey }` envelope (capture → `MarkItemsAsShipped`, refund → `ReturnItems` no amount / rows), `swedbankpay` `{ transaction: { ... } }`.

16.1.8 Recipes

Task-oriented snippets. `kcov3` is used as the worked example unless a recipe is provider-specific; swap the create/get method per the §16.1.7 table for another provider. Signatures are in the §17 reference.

16.1.8.1 Create a checkout and read its status

```
import { kcov3 } from "paymentproviders";
const kco = kcov3.client(); // reads KCO_* from env; test by default

const order = await kco.createCheckout({
  purchase_country: "SE",
  purchase_currency: "SEK",
  locale: "sv-SE",
  order_amount: 15000, // minor units → 150.00 SEK
  order_lines: [{ name: "Widget", quantity: 1, unit_price: 15000, total_amount: 15000 }],
});
const status = await kco.getCheckout(order.order_id);
```

Notes

- Other providers: `netstv1.createPayment`, `sveacheckout2 / qlirov2.createOrder`, `swedbankpayv3.createPaymentOrder` (see §16.1.7).

16.1.8.2 Capture — full, then partial

Capture up to the authorised amount, in one or more calls. Amounts are minor units.

```
// full capture
await kco.capturePayment(orderId, { amount: 15000 });

// or capture in parts (e.g. per shipment)
await kco.capturePayment(orderId, { amount: 5000, description: "First shipment" });
await kco.capturePayment(orderId, { amount: 10000, description: "Second shipment" });
```

Notes

- `capturePayment` is the method name on every provider; the input shape varies — `netstv1 { amount, orderItems? }`, `sveacheckout2 { amount, rows? }`, `qlirov2` takes optional management fields merged into a signed `{ RequestId, OrderId, MerchantApiKey }` envelope (not `{ amount, rows }`), `swedbankpay { transaction: { amount, vatAmount, ... } }`.

16.1.8.3 Refund a captured payment

```
await kco.refundPayment(orderId, { amount: 5000, description: "Returned item" });
```

Notes

- Refund at most what was captured. Same method name across providers; input shape as for capture.

16.1.8.4 Cancel an uncaptured payment

```
await kco.cancelPayment(orderId);
```

Notes

- Cancel/void releases an authorisation that hasn't been captured; once captured, refund instead.

16.1.8.5 SwedbankPay: follow HAL operations

SwedbankPay is HAL-driven: capture/refund/cancel are wrappers over `operation(paymentOrder, rel, body)`, which follows the link with that `rel` on the fetched payment order. Use `operation()` directly for any other rel.

```
import { swedbankpayv3 } from "paymentproviders";
const spp = swedbankpayv3.client();

const po = await spp.getPaymentOrder("/psp/paymentorders/abc123");
await spp.capturePayment(po, {
  transaction: { amount: 15000, vatAmount: 3000, description: "Capture",
  payeeReference: "cap-1" },
});

// any other operation exposed on the payment order:
await spp.operation(po, "abort", { paymentorder: { operation: "Abort",
  abortReason: "CancelledByConsumer" } });
```

Notes

- Pass either the fetched payment-order object or its URL; the library resolves the operation's href from the object's `operations` /links.

16.1.8.6 Signed-body providers (svea, qliro)

sveacheckout2 and qlirov2 authenticate by signing the request body; the library computes the signature for you. You only control the body shape and amounts — a mismatch (wrong amount, malformed rows) comes back as a `PaymentError`, not a silent failure.

```
import { qlirov2 } from "paymentproviders";
const qliro = qlirov2.client(); // QLIRO_API_KEY + QLIRO_APIPASSWORD

const order = await qliro.createOrder({
  Currency: "SEK",
  MerchantReference: "order-1",
  OrderItems: [{ MerchantReference: "sku-1", Type: "Product", Quantity: 1,
  PricePerItemIncVat: 15000 }],
});
```

16.1.8.7 Retry safely with idempotency (kco3)

Every `kco3` mutation (`acknowledge`, `capturePayment`, `refundPayment`, `cancelPayment`, `releaseRemainingAuthorization`) takes an optional `{ idempotencyKey? }` as its last argument. Left unset, each *call* gets its own fresh, auto-generated `klarna-idempotency-key` — which means a naive retry loop that just calls the method again on failure sends a **different** key per attempt and is not deduped server-side. To make a retry loop safe, generate one key **before** the loop and pass the same value on every attempt:

```
async function captureWithRetry(kco: any, orderId: string, amount: number) {
  // Pinned once, before the loop – reused on every attempt below.
  const idempotencyKey = `${Date.now().toString(36)}-
```

```

    ${Math.random().toString(36).slice(2, 12)}`;
    for (let attempt = 0; attempt < 3; attempt++) {
      try {
        return await kco.capturePayment(orderId, { amount }, { idempotencyKey });
      } catch (e: any) {
        if (e.status >= 500 && attempt < 2) continue; // transient – safe to
        retry, same key
        throw e;
      }
    }
  }
}

```

Note: a stable key only dedupes retries of the *same logical operation*. Don't reuse it across unrelated calls (e.g. two different captures on the same order) — that would incorrectly collapse them into one.

16.1.8.8 Configure auth and pick test vs prod

Put credentials in a `.env` and run with `--env-file`. Mind the naming traps.

```

// sercon --env-file .env script.ts
// .env:
//   KCO_MERCHANT_ID=...      KCO_SHARED_SECRET=...      KCO_ENV=test
//   SCO_MERCHANT_ID=...      SCO_SECRET_KEY=...          (svea uses SCO_, not
SVEA_)
//   QLIRO_API_KEY=...        QLIRO_APIPASSWORD=...      (no underscore in
APIPASSWORD)
//   SWEDBANKPAY_ACCESS_TOKEN=... SWEDBANKPAY_MERCHANT_ID=...
import { kcov3 } from "paymentproviders";
const test = kcov3.client(); // test by default
const prod = kcov3.client({ env: "prod" }); // or set KCO_ENV=prod

```

16.1.8.9 Handle a PaymentError

```

import { kcov3 } from "paymentproviders";
try {
  await kcov3.client().getPayment("no-such-order");
} catch (e: any) {
  // PaymentError { provider, status, body, requestId? }
  if (e.status === 404) {
    runtime.log("not found");
  } else if (e.status >= 500) {
    runtime.log(`${e.provider} server error – safe to retry (reqId
${e.requestId})`);
  } else {
    throw e; // other 4xx are deterministic – fix the request, don't retry
  }
}

```

16.1.8.10 Poll a payment to a terminal status

```

async function waitForTerminal(kco: any, orderId: string, timeoutMs = 30000):
Promise<any> {
  const deadline = Date.now() + timeoutMs;

```

```

    for (;;) {
      const o = await kco.getPayment(orderId);
      if (o.status === "CAPTURED" || o.status === "CANCELLED") return o;
      if (Date.now() > deadline) throw new Error(`timed out waiting for ${orderId}`);
    };
    await new Promise((r) => setTimeout(r, 2000));
  }
}

```

Notes

- Terminal status strings and the `status` field name are provider-specific — check each provider's `get*` response shape; the polling structure is the reusable part.

16.2 favro

A TypeScript client for the Favro API, compiled into the `sercon` binary and importable by scripts — no install needed. Like `paymentproviders`, it is **not** a reserved global; you `import` it:

```

import { client } from "favro";
const favro = client(); // reads FAVRO_* from the environment
const cards = await favro.cards.listAll({ widgetCommonId: "w1" });

```

`client(overrides?)` is **synchronous** and throws a plain `Error` (not `FavroError`) if `email` or `apiToken` is missing. Config precedence is `overrides.X ?? env(FAVRO_X)` :

Field	Env var	Notes
<code>email</code>	<code>FAVRO_EMAIL</code>	required
<code>apiToken</code> (or <code>password</code>)	<code>FAVRO_API_TOKEN</code>	required; sent as <code>Authorization: Basic base64(email:apiToken)</code>
<code>organizationId</code>	<code>FAVRO_ORGANIZATION_ID</code>	fixed on the client; auto-sent as the <code>organizationId</code> header on every org-scoped route
<code>baseUrl</code>	<code>FAVRO_BASE_URL</code>	default <code>https://favro.com/api/v1</code> (Favro has one base URL, no test/prod split)
<code>retry</code>	—	<code>{ max?, maxWaitMs? }</code> (defaults <code>max: 2, maxWaitMs: 30000</code>) or <code>false</code> to disable 429 retry

16.2.1 Resource groups

The returned client exposes thirteen resource groups. Most expose the full CRUD + pagination surface (`list` / `listAll` / `iterate` / `get` / `create` / `update` / `remove`); a few expose a narrower subset because the underlying Favro API doesn't support the rest:

Group	Verbs	Notes
<code>organizations</code>	<code>list</code> , <code>listAll</code> , <code>iterate</code> , <code>get</code> , <code>create</code> , <code>update</code>	user-level — <code>organizationId</code> is never sent for this group

users	list, listAll, iterate, get	read-only
collections	full	
widgets	full	
columns	full	list / listAll / iterate require widgetCommonId
cards	full + dependencies + activities + uploadAttachment	list requires one of widgetCommonId / collectionId / cardCommonId / cardSequentialId / todoList
comments	full + uploadAttachment	list requires cardCommonId
tasks	full	list requires cardCommonId
tasklists	full	list requires cardCommonId
tags	full	
customFields	list, listAll, iterate, get	read-only
groups	full	
webhooks	list, listAll, iterate, create, remove	no get / update

cards.dependencies adds `list(cardId)` / `add(cardId, body)` / `set(cardId, body)` / `update(cardId, depId, body)` / `remove(cardId, depId)` / `removeAll(cardId)`
cards.activities adds `list(cardId, params?)` returning one page of activity entries.

16.2.2 Pagination

Every listable group returns three ways to walk results, all sharing the same scoped-list validation:

- **list(params?)** — one page envelope: { entities, page, pages, requestId, limit }.
- **listAll(params?)** — fetches every page and concatenates entities.
- **iterate(params?)** — an `AsyncIterableIterator` that yields entities page-by-page (for `await (const c of favro.collections.iterate())`).

Subsequent pages reuse the `requestId` from page 0 and pin to the same Favro backend process via the `X-Favro-Backend-Identifier` response/request header — Favro's own pagination contract for consistent results across pages.

16.2.3 Retry and errors

A 429 response is retried automatically (bounded): the wait comes from `Retry-After` (seconds) or `X-RateLimit-Reset` (a UTC timestamp), falling back to 1s, and retry stops once `retry.max` attempts are used or the computed wait exceeds `retry.maxWaitMs`. Pass `retry: false` on `client()` to disable it entirely.

Any non-2xx response (after retry is exhausted, if applicable) throws a `FavroError`:

```
class FavroError extends Error {
  status: number;           // the HTTP status
  body: unknown;            // parsed JSON error body (or raw string)
  requestId?: string;       // from the response body, if present
  rateLimit?: {             // parsed X-RateLimit-* / Retry-After headers
    limit?: number;
  };
}
```

```

    remaining?: number;
    reset?: string;
    retryAfter?: number;
  };
}

```

16.2.4 Attachments

`cards.uploadAttachment(cardId, input)` and `comments.uploadAttachment(commentId, input)` POST a `multipart/form-data` body via `net.http.request`'s `multipart` option:

```

await favro.cards.uploadAttachment("card123", {
  filename: "spec.txt",
  content: "hello",           // string | Uint8Array | ArrayBuffer
  type: "text/plain",         // optional
});

```

Caveat: Favro's exact multipart field name for the file part is unconfirmed against a live account. The library defaults to `field: "file"` pass `input.field` to override it if your Favro instance expects a different name. Verify against a real (non-mock) upload before relying on this in production.

16.2.5 Understanding Favro's model

The recipes below assume this mental model. (Snippets in this manual are illustrative — copy-paste and adapt — unless a section says otherwise.)

Favro nests like this:

```

organization
├── collection      (a group of boards)
│   └── widget      (a board – the thing you look at)
│       ├── column  (a vertical status: "In progress", "Done", ...)
│       ├── lane     (a horizontal swimlane, when the board has lanes on)
│       └── card     (lives on the widget; can also live on other widgets)

```

The identifiers that trip people up. A card can sit on several widgets at once, so Favro splits its identity:

Field	What it is	Use it for
<code>cardId</code>	id of one <i>placement</i> of the card (per widget)	direct card ops: <code>cards.get</code> / <code>update</code> / <code>remove</code> , <code>cards.dependencies.*</code> , <code>cards.activities.list</code>
<code>cardCommonId</code>	shared id across <i>all</i> placements of the same card	filtering <code>comments.list</code> / <code>tasks.list</code> / <code>tasklists.list</code> (that metadata belongs to the card, not a placement), and the <code>comments.create</code> body

<code>sequentialId</code>	the human number on the card (for readable links)	display; to <i>find</i> by it, pass the <code>cardSequentialId</code> query param to <code>cards.list</code> (note the different spelling)
<code>widgetCommonId</code>	the board's shared id	scoping <code>cards.list</code> / <code>columns.list</code>
<code>columnId</code>	the column a card is in on a widget	<code>cards.list({ widgetCommonId, columnId })</code> present only when the card is on a widget
<code>laneId</code>	the lane a card is in	present only when the card is on a widget and that widget has lanes enabled
<code>collectionId</code>	a collection's id	scoping <code>cards.list</code> to a whole collection

Custom fields. A card carries `customFields`, an array of `{ customFieldId, value }`. The `value` shape depends on the field's type — for a single/multiple-select field it is an array of the selected *item ids*, not their display names. The field's definition (name, type, and its selectable items) comes from `customFields.get(customFieldId)`.

The limitation that shapes every “filter” recipe. `GET /cards` can filter by `widgetCommonId` and `columnId`, but there is **no lane or custom-field filter**. So the pattern is always: *narrow by column on the server, then filter by `laneId` or a custom-field value in memory*.

16.2.6 Recipes

Task-oriented snippets. Each assumes `const favro = client();` (see §16.2 for config). Signatures are in the §17 reference; the shapes below are the Favro API's own.

16.2.6.1 Filter a column by lane

The one everyone hits: you want the cards in one status column *for a single product*, where products are lanes. There is no server-side lane filter, so narrow by column, then filter by `laneId` in memory.

```
const widgetCommonId = "your-board-common-id";

// name → columnId
const cols = await favro.columns.listAll({ widgetCommonId });
const columnId = cols.find((c) => c.name === "Ready for release")?.columnId;
if (!columnId) throw new Error("no such column");

// name → laneId (lanes live on the widget, not the card)
const widget = await favro.widgets.get(widgetCommonId);
const laneId = widget.lanes?.find((l) => l.name === "Product X")?.laneId;
if (!laneId) throw new Error("no such lane – is 'lanes' enabled on this board?");

// narrow by column server-side, filter by lane in memory
const inColumn = await favro.cards.listAll({ widgetCommonId, columnId });
const productX = inColumn.filter((c) => c.laneId === laneId);
```

Notes

- `laneId` is only on the card when the widget has lanes enabled; if it's `undefined`, your “lanes” may actually be a custom-field grouping — see 16.2.6.3.
- `columnId` / `laneId` are per-widget, so they're meaningful because you listed by `widgetCommonId`.

16.2.6.2 Resolve a name to an id

Columns, lanes, and custom fields are addressed by id but shown by name. A tiny resolver keeps recipes readable.

```
async function columnIdByName(widgetCommonId: string, name: string):
Promise<string> {
  const cols = await favro.columns.listAll({ widgetCommonId });
  const col = cols.find((c) => c.name === name);
  if (!col) throw new Error(`no column named "${name}" on ${widgetCommonId}`);
  return col.columnId;
}
```

Notes

- Same shape works for lanes (`widget.lanes`) and custom fields (`favro.customFields.listAll()` → `match .name`).

16.2.6.3 Filter by a custom-field value

When “product” is a single-select custom field rather than a lane, filter on the card's `customFields` entry. Select values are *item ids*, not names.

```
const fields = await favro.customFields.listAll();
const field = fields.find((f) => f.name === "Product");
if (!field) throw new Error("no 'Product' custom field");

// Resolve the option's item id from the field definition, then match cards.
// (The item id is what a select card value contains.)
const optionItemId = "the-option-item-id-for-Product-X";

// inColumn: cards already narrowed by column (see 16.2.6.1)
const widgetCommonId = "your-board-common-id";
const columnId = "your-column-id";
const inColumn = await favro.cards.listAll({ widgetCommonId, columnId });

const productX = inColumn.filter((c) =>
  (c.customFields ?? []).some(
    (v) => v.customFieldId === field.customFieldId && (v.value ??
  []).includes(optionItemId),
  ),
);
```

Notes

- A select field's `value` is an array of selected item ids. To turn the option *name* (“Product X”) into its item id, read the field definition via `favro.customFields.get(field.customFieldId)` and match its `items` array — the exact

item-array field name follows Favro's custom-field schema, so confirm it against a live field before relying on it.

16.2.6.4 Find a card by its human number

Use the `cardSequentialId` query param (note: the field on the card is `sequentialId`, but the filter is `cardSequentialId`).

```
const page = await favro.cards.list({ cardSequentialId: 1234 });
const card = page.entities[0];
```

16.2.6.5 List a whole collection vs one board

`cards.list` accepts any one scope. Use `collectionId` to sweep every board in a collection, or `widgetCommonId` for a single board.

```
const wholeCollection = await favro.cards.listAll({ collectionId: "col-common-id" });
const oneBoard        = await favro.cards.listAll({ widgetCommonId: "w-common-id" });
```

16.2.6.6 Create a card

```
const created = await favro.cards.create({
  name: "Ship the release notes",
  widgetCommonId: "w-common-id",
  columnId: "col-id",           // optional: drop it to land in the default column
});
```

16.2.6.7 Move a card between columns

Update the placement's `columnId` (and `laneId` if the board has lanes). Use the per-widget `cardId`, not `cardCommonId`.

```
await favro.cards.update("card-id-on-this-widget", {
  widgetCommonId: "w-common-id",
  columnId: "ready-for-release-col-id",
});
```

16.2.6.8 Comment on a card and attach a file

Comments and attachments hang off the card; comments filter by `cardCommonId`.

```
await favro.comments.create({ cardCommonId: "card-common-id", comment: "Reviewed ✓" });

await favro.cards.uploadAttachment("card-id", {
  filename: "notes.txt",
  content: "release checklist...", // string | Uint8Array | ArrayBuffer
  type: "text/plain",
});
```

16.2.6.9 Add a dependency between cards


```
await favro.cards.dependencies.add("card-id", {
  cardCommonId: "the-other-card-common-id",
  type: "blockedBy",
});
```

16.2.6.10 Set up auth and verify it

Put credentials in a `.env` and run with `--env-file` a cheap probe confirms the pairing without needing a known object.

```
// sercon --env-file .env script.ts
// .env: FAVRO_EMAIL=..., FAVRO_API_TOKEN=..., FAVRO_ORGANIZATION_ID=...
const favro = client(); // reads FAVRO_* from the environment
try {
  const orgs = await favro.organizations.listAll(); // user-level; no org id
  needed
  runtime.log("auth OK -", orgs.length, "organizations");
} catch (e: any) {
  runtime.assert.ok(e.status !== 401, "credentials rejected (401)");
  throw e;
}
```

16.2.6.11 Receive a webhook

Register an outgoing webhook, then handle its POST with `server.http`.

```
await favro.webhooks.create({
  name: "card-moves",
  url: "https://your-host.example/favro",
  options: { /* Favro webhook options */ },
});

const srv = await server.http.listen({ port: 8080, routes: {
  "POST /favro": (q: any, r: any) => {
    const event = JSON.parse(q.body);
    runtime.log("favro event:", event.action ?? event.type);
    return r.status(200).text("ok");
  },
}});
```

Notes

- Favro sends webhook payloads as JSON POST bodies; `q.body` is the raw string (parse it). Header values on `q.headers` are arrays.

16.2.6.12 Read a lot of cards without tripping the rate limit

For large sweeps prefer `iterate` (lazy, page-by-page) over `listAll` (buffers everything), and tune `retry` on the client. Favro's limits are per-hour and plan-dependent; a wide `listAll` fans out many page requests and can hit 429.

```
const favro = client({ retry: { max: 4, maxWaitMs: 60000 } });
for await (const card of favro.cards.iterate({ widgetCommonId: "w-common-id" })) {
  // process one card at a time; pages are fetched as you go
}
```

Notes

- A 429 is retried automatically within the `retry` bounds (§16.2.3); set `retry: false` to handle it yourself.

17. Binding reference (generated)

The per-function reference below is generated from the structured binding docs (MemberDoc) by `sercon --emit-reference`, spliced in by `make reference`. **Do not edit between the markers** — regenerate instead. Coverage grows per release as each namespace adopts the structured doc model (parameters, return shapes, errors, examples); namespaces not yet migrated show their one-line summary and a collapsed signature.

17.1 audio

Audio: read/write formats (WAV/FLAC/MP3/OGG/AIFF decode, WAV/FLAC/AIFF encode, convert, info) and WAV LSB steganography (stego).

17.1.1 audio.convert

```
convert(src: string | Uint8Array, opts: { format: "wav" | "flac" | "aiff"; dest?: string }): { bytes: Uint8Array } | { path: string }
```

Decode any supported audio source (WAV/FLAC/MP3/OGG/AIFF) and re-encode it as a lossless container (WAV, FLAC, or AIFF). Optionally write to a file via `opts.dest`.

Parameters

- `src` (*string* | *Uint8Array*) — Audio file path or raw bytes of any supported format.
- `opts` (*{ format: "wav" | "flac" | "aiff"; dest?: string }*) — format is required. `dest` writes to a file path instead of returning bytes.

Returns: The re-encoded audio bytes (`{ bytes }`), or `{ path }` when `opts.dest` was given.

Throws: Throws if `opts.format` is missing, if the source is unrecognized, or if the target format is MP3/OGG (no pure-Go encoder).

```
const flac = audio.convert("song.wav", { format: "flac" });
```

17.1.2 audio.decode

```
decode(src: string | Uint8Array): { format: string; sampleRate: number; channels: number; bitDepth: number; frames: number; durationMs: number; pcm: Uint8Array }
```

Decode an audio file (WAV/FLAC/MP3/OGG/AIFF) to canonical 16-bit LE interleaved PCM bytes plus metadata. Samples are normalized to 16-bit regardless of source bit depth.

Parameters

- `src` (*string* | *Uint8Array*) — Audio file path or raw bytes.

Returns: Metadata plus a `pcm` field containing raw 16-bit LE interleaved samples as a `Uint8Array`.

Throws: Throws if the source is not a recognized/decodable audio format.

```
const d = audio.decode("song.wav"); // d.pcm is a Uint8Array of 16-bit LE samples
```

17.1.3 audio.encode

```
encode(pcm: Uint8Array, opts: { format: "wav" | "flac" | "aiff"; sampleRate: number; channels: number; dest?: string }): { bytes: Uint8Array } | { path: string }
```

Encode raw 16-bit LE interleaved PCM bytes into a lossless container (WAV, FLAC, or AIFF). Optionally write to a file via `opts.dest`.

Parameters

- `pcm` (*Uint8Array*) — Raw 16-bit LE interleaved samples (as returned by `audio.decode`).
- `opts` (*{ format: "wav" | "flac" | "aiff"; sampleRate: number; channels: number; dest?: string }*) — `format`, `sampleRate`, and `channels` are required. `dest` writes to a file path instead of returning bytes.

Returns: The encoded audio bytes (`{ bytes }`), or `{ path }` when `opts.dest` was given.

Throws: Throws for MP3/OGG (no pure-Go encoder), if `sampleRate/channels` are missing or ≤ 0 , or if the PCM length is not a whole number of frames.

```
const out = audio.encode(d.pcm, { format: "flac", sampleRate: d.sampleRate, channels: d.channels });
```

17.1.4 audio.info

```
info(src: string | Uint8Array): { format: string; sampleRate: number; channels: number; bitDepth: number; frames: number; durationMs: number }
```

Probe an audio file's metadata without returning samples. Detects the container by magic bytes (WAV/FLAC/MP3/OGG/AIFF) and reports sample rate, channels, source bit depth, frame count, and duration.

Parameters

- `src` (*string | Uint8Array*) — Audio file path or raw bytes.

Returns: Metadata describing the source audio.

Throws: Throws if the source is not a recognized/decodable audio format.

```
const meta = audio.info("song.flac"); // { format: "flac", sampleRate: 44100, ... }
```

17.1.5 audio.stego

17.1.5.1 audio.stego.capacity

```
capacity(cover: string | Uint8Array, opts?: { bits?: number }): { bytes: number; bits: number }
```

Report the maximum payload size in bytes a WAV carrier can hold via LSB steganography — at the requested bit depth (1..4, default 1; one bit per PCM sample at the default), minus the fixed header. Encryption adds 44 bytes of overhead.

Parameters

- `cover` (*string* | *Uint8Array*) — The carrier WAV: a file path or raw bytes.
- `opts` (*{ bits?: number }, optional*) — `bits`: report capacity at this depth (integer 1..4, default 1).

Returns: An object: `bytes` is the maximum plaintext payload size at the requested depth; `bits` echoes that depth.

Throws: Throws if the cover is not a supported PCM WAV, or a `TypeError` if `bits` is not an integer 1..4.

```
const room = audio.stego.capacity("song.wav", { bits: 4 }).bytes;
```

17.1.5.2 audio.stego.embed

```
embed(cover: string | Uint8Array, payload: string | Uint8Array, opts?:  
  { password?: string; dest?: string; bits?: number }): { bytes: Uint8Array } |  
  { path: string }
```

Hide a payload in a WAV audio file using least-significant-bit steganography on the PCM samples (8- or 16-bit; one bit per sample's low byte — the audio is essentially unchanged). A non-empty password encrypts the payload (AES-256-GCM). Returns the modified WAV bytes. One to four bits are stored per PCM sample (the `bits` option, default 1); higher values raise capacity but make the change more audible and easier to detect.

Parameters

- `cover` (*string* | *Uint8Array*) — The carrier WAV: a file path or raw WAV bytes (PCM, 8- or 16-bit).
- `payload` (*string* | *Uint8Array*) — Data to hide. A string is stored as UTF-8 text; a `Uint8Array` as binary.
- `opts` (*{ password?: string; dest?: string; bits?: number }, optional*) — `password` encrypts the payload; `dest` writes the resulting WAV to that path instead of returning bytes. `bits`: payload bits per sample, an integer 1..4 (default 1).

Returns: The modified WAV bytes (`{ bytes }`), or `{ path }` when `opts.dest` was given.

Throws: Throws if the cover is not a PCM WAV, is an unsupported bit depth (only 8/16-bit), if the payload exceeds capacity, if the payload is not a string/Uint8Array, or if writing `dest` fails.

```
const out = audio.stego.embed("song.wav", "secret", { password: "p" });
```

17.1.5.3 audio.stego.extract

```
extract(cover: string | Uint8Array, opts?: { password?: string }): string |  
  Uint8Array
```

Recover a payload hidden by `audio.stego.embed`. Reads the PCM sample LSBs, verifies the sercon header, and returns the payload as a string (if embedded as text) or a `Uint8Array`. The bit depth is read from the header, so no `bits` argument is needed.

Parameters

- `cover` (*string* / *Uint8Array*) — The stego WAV: a file path or raw bytes.
- `opts` (*{ password?: string }*, *optional*) — The password used at embed time, required when the payload was encrypted.

Returns: The recovered payload — a string when embedded as text, otherwise a `Uint8Array`.

Throws: Throws if the cover is not a PCM WAV, if no sercon payload is present, if encrypted without a password, or on decryption failure.

```
const msg = audio.stego.extract("song.wav", { password: "p" });
```

17.2 cloud

Cloud provider clients: `cloud.google(opts?)` returns a handle with typed services (storage, compute, iam, secrets) plus a generic path-based REST escape hatch (`call`). Pure-Go, CGO-free; reuses Application Default Credentials.

17.2.1 cloud.aws

```
aws(opts?: { region?: string; profile?: string; credentials?: { accessKeyId:
string; secretAccessKey: string; sessionToken?: string } }): {
  s3(): {
    listBuckets(opts?: Record<string, never>): Promise<Record<string, unknown>>;
    createBucket(opts: { bucket: string }): Promise<Record<string, unknown>>;
    deleteBucket(opts: { bucket: string }): Promise<Record<string, unknown>>;
    listObjects(opts: { bucket: string; prefix?: string }): Promise<Record<string,
unknown>>;
    headObject(opts: { bucket: string; key: string }): Promise<Record<string,
unknown>>;
    getObject(opts: { bucket: string; key: string }): Promise<{ bytes: number[] }
>;
    putObject(opts: { bucket: string; key: string; body: string | Uint8Array |
ArrayBuffer }): Promise<Record<string, unknown>>;
    deleteObject(opts: { bucket: string; key: string }): Promise<Record<string,
unknown>>;
  };
  ec2(): {
    describeInstances(opts?: { instanceIds?: string[] }): Promise<Record<string,
unknown>>;
    runInstances(opts: { imageId: string; instanceType: string; minCount?: number;
maxCount?: number }): Promise<Record<string, unknown>>;
    terminateInstances(opts: { instanceIds: string[] }): Promise<Record<string,
unknown>>;
    startInstances(opts: { instanceIds: string[] }): Promise<Record<string,
unknown>>;
    stopInstances(opts: { instanceIds: string[] }): Promise<Record<string,
unknown>>;
    describeVolumes(opts?: { volumeIds?: string[] }): Promise<Record<string,
unknown>>;
    describeAvailabilityZones(opts?: Record<string, never>):
```

```

Promise<Record<string, unknown>>;
};
iam(): {
  listUsers(opts?: Record<string, never>): Promise<Record<string, unknown>>;
  getUser(opts?: { userName?: string }): Promise<Record<string, unknown>>;
  listRoles(opts?: Record<string, never>): Promise<Record<string, unknown>>;
  getRole(opts: { roleName: string }): Promise<Record<string, unknown>>;
  listPolicies(opts?: Record<string, never>): Promise<Record<string, unknown>>;
  createUser(opts: { userName: string }): Promise<Record<string, unknown>>;
  deleteUser(opts: { userName: string }): Promise<Record<string, unknown>>;
  attachUserPolicy(opts: { userName: string; policyArn: string }):
Promise<Record<string, unknown>>;
};
secretsmanager(): {
  listSecrets(opts?: Record<string, never>): Promise<Record<string, unknown>>;
  describeSecret(opts: { secretId: string }): Promise<Record<string, unknown>>;
  createSecret(opts: { name: string; secretString?: string }):
Promise<Record<string, unknown>>;
  getSecretValue(opts: { secretId: string }): Promise<{ value: string }>;
  putSecretValue(opts: { secretId: string; secretString: string }):
Promise<Record<string, unknown>>;
  deleteSecret(opts: { secretId: string }): Promise<Record<string, unknown>>;
};
sts(): {
  getCallerIdentity(opts?: Record<string, never>): Promise<Record<string,
unknown>>;
  assumeRole(opts: { roleArn: string; roleSessionName: string; durationSeconds?:
number }): Promise<Record<string, unknown>>;
  getSessionToken(opts?: { durationSeconds?: number }): Promise<Record<string,
unknown>>;
};
lambda(): {
  listFunctions(opts?: Record<string, never>): Promise<Record<string, unknown>>;
  getFunction(opts: { functionName: string }): Promise<Record<string, unknown>>;
  invoke(opts: { functionName: string; payload?: string | Record<string,
unknown> }): Promise<{ statusCode: number; payload: string; functionError?:
string; executedVersion?: string }>;
  createFunction(opts: { functionName: string; role: string; runtime: string;
handler: string; zipFile?: string | Uint8Array | ArrayBuffer; s3Bucket?: string;
s3Key?: string }): Promise<Record<string, unknown>>;
  deleteFunction(opts: { functionName: string }): Promise<Record<string,
unknown>>;
};
sqs(): {
  listQueues(opts?: { prefix?: string }): Promise<Record<string, unknown>>;
  createQueue(opts: { queueName: string }): Promise<Record<string, unknown>>;
  deleteQueue(opts: { queueUrl: string }): Promise<Record<string, unknown>>;
  sendMessage(opts: { queueUrl: string; messageBody: string }):
Promise<Record<string, unknown>>;
  receiveMessage(opts: { queueUrl: string; maxMessages?: number }):
Promise<Record<string, unknown>>;
  deleteMessage(opts: { queueUrl: string; receiptHandle: string }):
Promise<Record<string, unknown>>;
  getQueueAttributes(opts: { queueUrl: string; attributeNames?: string[] }):
Promise<Record<string, unknown>>;

```

```

};
cloudwatch(): {
  listMetrics(opts?: { namespace?: string; metricName?: string }):
Promise<Record<string, unknown>>;
  getMetricData(opts: Record<string, unknown>): Promise<Record<string,
unknown>>;
  getMetricStatistics(opts: Record<string, unknown>): Promise<Record<string,
unknown>>;
  describeAlarms(opts?: { alarmNames?: string[] }): Promise<Record<string,
unknown>>;
  putMetricData(opts: Record<string, unknown>): Promise<Record<string,
unknown>>;
};
cloudwatchlogs(): {
  describeLogGroups(opts?: { prefix?: string }): Promise<Record<string,
unknown>>;
  describeLogStreams(opts: { logGroupName: string }): Promise<Record<string,
unknown>>;
  getLogEvents(opts: { logGroupName: string; logStreamName: string; limit?:
number }): Promise<Record<string, unknown>>;
  filterLogEvents(opts: { logGroupName: string; filterPattern?: string }):
Promise<Record<string, unknown>>;
  startQuery(opts: { logGroupName: string; queryString: string; startTime:
number; endTime: number }): Promise<Record<string, unknown>>;
  getQueryResults(opts: { queryId: string }): Promise<Record<string, unknown>>;
};
}

```

Amazon Web Services provider. `cloud.aws(opts?)` returns a handle with typed services (s3, ec2, iam, secretsmanager, sts, lambda, sqs, cloudwatch, cloudwatchlogs). Pure-Go, CGO-free; reuses the standard AWS credential chain.

Parameters

- `opts` (*{ region?: string; profile?: string; credentials?: { accessKeyId: string; secretAccessKey: string; sessionToken?: string } }, optional*) — region: AWS region (default: from the credential chain / AWS_REGION). profile: named profile. credentials: static creds; omitted ⇒ default chain (env, /.aws, SSO, IMDS).

Returns: The AWS provider handle exposing the nine typed services. Most service methods take a small typed options object. The three CloudWatch metric methods — `cloudwatch().getMetricData/getMetricStatistics/putMetricData` — are pass-through: their argument is an AWS-SDK-shaped object with PascalCase keys (e.g. { Namespace, MetricData: [{ MetricName, Value, Unit, Timestamp }] }), forwarded to the SDK input as-is.

Throws: `cloud.aws(opts)` itself throws synchronously (not a rejected promise) if `opts` is provided but is not an object, or `credentials` is present but is not an object carrying `accessKeyId` and `secretAccessKey` (`sessionToken` optional). Each service method returns a promise that rejects with a structured Error { code, status, message, details } on API/transport failure.

```
const who = await cloud.aws({ region: "eu-north-1" }).sts().getCallerIdentity({});
```

17.2.1.1 cloud.aws.cloudwatch

CloudWatch — metrics and alarms (github.com/aws/aws-sdk-go-v2/service/cloudwatch). Reached via `cloud.aws({...}).cloudwatch()` each method below is called on that service handle.

17.2.1.1.1 cloud.aws.cloudwatch.describeAlarms

```
describeAlarms(opts?: { alarmNames?: string[] }): Promise<Record<string, unknown>>
```

Describe CloudWatch alarms, optionally filtered to specific alarm names.

Parameters

- `opts` (*{ alarmNames?: string[] }, optional*) — `alarmNames`: filter to specific alarm names; omitted or empty $\Rightarrow$ describe every alarm.

Returns: A promise resolving to the CloudWatch DescribeAlarms response — `MetricAlarms` (array of alarm descriptions), `CompositeAlarms`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatch().describeAlarms();
```

17.2.1.1.2 cloud.aws.cloudwatch.getMetricData

```
getMetricData(opts: Record<string, unknown>): Promise<Record<string, unknown>>
```

Fetch datapoints for one or more metrics via CloudWatch's metric-math query interface. Pass-through method: the argument is forwarded to the SDK's `GetMetricDataInput` as-is.

Parameters

- `opts` (*Record<string, unknown>*) — `opts`: an AWS-SDK-shaped object with PascalCase keys matching `GetMetricDataInput` (e.g. { `MetricDataQueries`: [{ `Id`, `MetricStat`: { `Metric`: { `Namespace`, `MetricName`, `Dimensions` }, `Period`, `Stat` } }], `StartTime`, `EndTime` }) — JSON round-tripped straight into the Go SDK input struct: RFC3339 timestamp strings unmarshal into the struct's `*time.Time` fields.

Returns: A promise resolving to the CloudWatch GetMetricData response — `MetricDataResults` (array of { `Id`, `Label`, `Timestamps`, `Values`, `StatusCode` }), `Messages`, `NextToken`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatch().getMetricData({
  MetricDataQueries: [{ Id: "m1", MetricStat: { Metric: { Namespace: "AWS/EC2",
    MetricName: "CPUUtilization", Period: 300, Stat: "Average" } } },
  StartTime: "2026-07-13T00:00:00Z", EndTime: "2026-07-13T01:00:00Z",
});
```


17.2.1.1.3 cloud.aws.cloudwatch.getMetricStatistics

```
getMetricStatistics(opts: Record<string, unknown>): Promise<Record<string, unknown>>
```

Fetch aggregated statistics for a single metric. Pass-through method: the argument is forwarded to the SDK's `GetMetricStatisticsInput` as-is.

Parameters

- `opts` (*Record<string, unknown>*) — `opts`: an AWS-SDK-shaped object with PascalCase keys matching `GetMetricStatisticsInput` (e.g. { `Namespace`, `MetricName`, `StartTime`, `EndTime`, `Period`, `Statistics`: ["Average"], `Dimensions?` }) — JSON round-tripped straight into the Go SDK input struct.

Returns: A promise resolving to the CloudWatch `GetMetricStatistics` response — `Datapoints` (array of { `Timestamp`, `Average`, `Sum`, `Minimum`, `Maximum`, `SampleCount`, `Unit` }), `Label`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatch().getMetricStatistics({
  Namespace: "AWS/EC2", MetricName: "CPUUtilization", StartTime:
  "2026-07-13T00:00:00Z", EndTime: "2026-07-13T01:00:00Z", Period: 300, Statistics:
  ["Average"],
});
```

17.2.1.1.4 cloud.aws.cloudwatch.listMetrics

```
listMetrics(opts?: { namespace?: string; metricName?: string }):
Promise<Record<string, unknown>>
```

List published CloudWatch metrics, optionally filtered by namespace/metric name.

Parameters

- `opts` (*{ namespace?: string; metricName?: string }, optional*) — `namespace`: e.g. "AWS/EC2"; omitted ⇒ all namespaces. `metricName`: filter to one metric name within the namespace; omitted ⇒ all metrics.

Returns: A promise resolving to the CloudWatch `ListMetrics` response — `Metrics` (array of { `Namespace`, `MetricName`, `Dimensions` }), `NextToken`, `OwningAccounts`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatch().listMetrics({ namespace: "AWS/EC2" });
```

17.2.1.1.5 cloud.aws.cloudwatch.putMetricData

```
putMetricData(opts: Record<string, unknown>): Promise<Record<string, unknown>>
```

Publish custom metric datapoints. Pass-through method: the argument is forwarded to the SDK's PutMetricDataInput as-is.

Parameters

- `opts` (*Record<string, unknown>*) — `opts`: an AWS-SDK-shaped object with PascalCase keys matching PutMetricDataInput (e.g. { Namespace, MetricData: [{ MetricName, Value, Unit, Timestamp, Dimensions? }] }) — JSON round-tripped straight into the Go SDK input struct.

Returns: A promise resolving to {} on success — PutMetricData's response carries no payload beyond request metadata.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).cloudwatch().putMetricData({
  Namespace: "MyApp", MetricData: [{ MetricName: "QueueDepth", Value: 42, Unit:
    "Count" }],
});
```

17.2.1.2 cloud.aws.cloudwatchlogs

CloudWatch Logs — log groups, streams, and Logs Insights queries (github.com/aws/aws-sdk-go-v2/service/cloudwatchlogs). Reached via `cloud.aws({...}).cloudwatchlogs()` each method below is called on that service handle.

17.2.1.2.1 cloud.aws.cloudwatchlogs.describeLogGroups

```
describeLogGroups(opts?: { prefix?: string }): Promise<Record<string, unknown>>
```

List log groups, optionally filtered by name prefix.

Parameters

- `opts` (*{ prefix?: string }, optional*) — `prefix`: only list log groups whose name starts with this string (sent as LogGroupNamePrefix); omitted ⇒ list all log groups.

Returns: A promise resolving to the CloudWatch Logs DescribeLogGroups response — LogGroups (array of { LogGroupName, Arn, CreationTime, RetentionInDays, ... }), NextToken.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatchlogs().describeLogGroups({ prefix: "/aws/lambda/" });
```

17.2.1.2.2 cloud.aws.cloudwatchlogs.describeLogStreams

```
describeLogStreams(opts: { logGroupName: string }): Promise<Record<string, unknown>>
```

List the log streams within a log group.

Parameters

- `opts` (*{ logGroupName: string }*) — `logGroupName`: the log group's name.

Returns: A promise resolving to the CloudWatch Logs DescribeLogStreams response — `LogStreams` (array of { `LogStreamName`, `Arn`, `CreationTime`, `FirstEventTimestamp`, `LastEventTimestamp`, ... }), `NextToken`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatchlogs().describeLogStreams({ logGroupName: "/aws/lambda/my-fn" });
```

17.2.1.2.3 cloud.aws.cloudwatchlogs.filterLogEvents

```
filterLogEvents(opts: { logGroupName: string; filterPattern?: string }): Promise<Record<string, unknown>>
```

Search log events across all (or filtered) streams in a log group.

Parameters

- `opts` (*{ logGroupName: string; filterPattern?: string }*) — `logGroupName`: the log group's name. `filterPattern`: a CloudWatch Logs filter pattern to restrict matching events; omitted $\Rightarrow$ all events.

Returns: A promise resolving to the CloudWatch Logs FilterLogEvents response — `Events` (array of { `LogStreamName`, `Timestamp`, `Message`, `IngestionTime`, `EventId` }), `SearchedLogStreams`, `NextToken`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatchlogs().filterLogEvents({ logGroupName: "/aws/lambda/my-fn", filterPattern: "ERROR" });
```

17.2.1.2.4 cloud.aws.cloudwatchlogs.getLogEvents

```
getLogEvents(opts: { logGroupName: string; logStreamName: string; limit?: number }): Promise<Record<string, unknown>>
```

Fetch log events from a specific log stream, in chronological order.

Parameters

- `opts` (*{ logGroupName: string; logStreamName: string; limit?: number }*) — `logGroupName`: the log group's name. `logStreamName`: the log stream's name. `limit`: max number of log events to return; omitted $\Rightarrow$ the API's own default.

Returns: A promise resolving to the CloudWatch Logs GetLogEvents response — Events (array of { Timestamp, Message, IngestionTime }), NextForwardToken, NextBackwardToken. Timestamps here are epoch milliseconds.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatchlogs().getLogEvents({ logGroupName: "/aws/lambda/my-fn", logStreamName: "2026/07/13/[$LATEST]abc123" });
```

17.2.1.2.5 cloud.aws.cloudwatchlogs.getQueryResults

```
getQueryResults(opts: { queryId: string }): Promise<Record<string, unknown>>
```

Poll for a Logs Insights query's results and status.

Parameters

- `opts` (*{ queryId: string }*) — `queryId`: the id returned by `startQuery`.

Returns: A promise resolving to the CloudWatch Logs GetQueryResults response — Results (array of arrays of { Field, Value } — one inner array per matched log record), Statistics, Status (e.g. "Scheduled", "Running", "Complete").

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).cloudwatchlogs().getQueryResults({ queryId: QueryId });
```

17.2.1.2.6 cloud.aws.cloudwatchlogs.startQuery

```
startQuery(opts: { logGroupName: string; queryString: string; startTime: number; endTime: number }): Promise<Record<string, unknown>>
```

Start a CloudWatch Logs Insights query. Returns immediately with a query id — poll `getQueryResults` for the results.

Parameters

- `opts` (*{ logGroupName: string; queryString: string; startTime: number; endTime: number }*) — `logGroupName`: the log group to query. `queryString`: the Logs Insights query. `startTime`/`endTime`: the query's time range, in epoch seconds (unlike `getLogEvents`/`filterLogEvents`, whose time fields are epoch milliseconds).

Returns: A promise resolving to the CloudWatch Logs StartQuery response — `QueryId`: pass it to `getQueryResults` to poll for the query's results.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const { QueryId } = await cloud.aws({ region: "eu-
north-1" }).cloudwatchlogs().startQuery({
  logGroupName: "/aws/lambda/my-fn", queryString: "fields @timestamp, @message |
filter @message like /ERROR/",
  startTime: Math.floor(Date.now() / 1000) - 3600, endTime:
Math.floor(Date.now() / 1000),
});
```

17.2.1.3 cloud.aws.ec2

EC2 — instances, volumes, and availability zones (github.com/aws/aws-sdk-go-v2/service/ec2). Reached via `cloud.aws({...}).ec2()` each method below is called on that service handle.

17.2.1.3.1 cloud.aws.ec2.describeAvailabilityZones

```
describeAvailabilityZones(opts?: Record<string, never>): Promise<Record<string,
unknown>>
```

List the availability zones available in the configured region.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `describeAvailabilityZones` takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the EC2 DescribeAvailabilityZones response — AvailabilityZones (array of { ZoneName, State, RegionName, ... }).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-
north-1" }).ec2().describeAvailabilityZones();
```

17.2.1.3.2 cloud.aws.ec2.describeInstances

```
describeInstances(opts?: { instanceIds?: string[] }): Promise<Record<string,
unknown>>
```

Describe EC2 instances, optionally filtered to specific instance ids.

Parameters

- `opts` (*{ instanceIds?: string[] }, optional*) — `instanceIds`: filter to specific instance ids; omitted or empty ⇒ describe every instance visible to the caller.

Returns: A promise resolving to the EC2 DescribeInstances response — Reservations (array of { Instances: [...], ReservationId, OwnerId, ... }); instances are nested inside each reservation, not returned as a flat list.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).ec2().describeInstances();
```

17.2.1.3.3 cloud.aws.ec2.describeVolumes

```
describeVolumes(opts?: { volumeIds?: string[] }): Promise<Record<string, unknown>>
```

Describe EBS volumes, optionally filtered to specific volume ids.

Parameters

- *opts* (*{ volumeIds?: string[] }, optional*) — volumeIds: filter to specific volume ids; omitted or empty ⇒ describe every volume visible to the caller.

Returns: A promise resolving to the EC2 DescribeVolumes response — Volumes (array of EBS volume descriptions).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).ec2().describeVolumes();
```

17.2.1.3.4 cloud.aws.ec2.runInstances

```
runInstances(opts: { imageId: string; instanceType: string; minCount?: number; maxCount?: number }): Promise<Record<string, unknown>>
```

Launch one or more EC2 instances from an AMI.

Parameters

- *opts* (*{ imageId: string; instanceType: string; minCount?: number; maxCount?: number }*) — imageId: the AMI id to launch. instanceType: e.g. "t3.micro". minCount/maxCount: the minimum/maximum number of instances to launch; both default to 1 when omitted (a sercon convenience — the EC2 RunInstances API itself marks both required).

Returns: A promise resolving to the EC2 RunInstances response — Instances (array of the newly launched instances' descriptions), ReservationId, OwnerId.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).ec2().runInstances({ imageId: "ami-0123456789abcdef0", instanceType: "t3.micro" });
```

17.2.1.3.5 cloud.aws.ec2.startInstances

```
startInstances(opts: { instanceIds: string[] }): Promise<Record<string, unknown>>
```

Start stopped EC2 instances.

Parameters

- `opts` (*{ instanceIds: string[] }*) — instanceIds: the instance ids to start.

Returns: A promise resolving to the EC2 StartInstances response — StartingInstances (array of { InstanceId, CurrentState, PreviousState }).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).ec2().startInstances({ instanceIds:
["i-0123456789abcdef0"] });
```

17.2.1.3.6 cloud.aws.ec2.stopInstances

```
stopInstances(opts: { instanceIds: string[] }): Promise<Record<string, unknown>>
```

Stop running EC2 instances.

Parameters

- `opts` (*{ instanceIds: string[] }*) — instanceIds: the instance ids to stop.

Returns: A promise resolving to the EC2 StopInstances response — StoppingInstances (array of { InstanceId, CurrentState, PreviousState }).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).ec2().stopInstances({ instanceIds:
["i-0123456789abcdef0"] });
```

17.2.1.3.7 cloud.aws.ec2.terminateInstances

```
terminateInstances(opts: { instanceIds: string[] }): Promise<Record<string,
unknown>>
```

Terminate EC2 instances.

Parameters

- `opts` (*{ instanceIds: string[] }*) — instanceIds: the instance ids to terminate.

Returns: A promise resolving to the EC2 TerminateInstances response — TerminatingInstances (array of { InstanceId, CurrentState, PreviousState }).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).ec2().terminateInstances({ instanceIds:
["i-0123456789abcdef0"] });
```

17.2.1.4 cloud.aws.iam

IAM — users, roles, and policies (github.com/aws/aws-sdk-go-v2/service/iam). Reached via `cloud.aws({...}).iam()` each method below is called on that service handle.

17.2.1.4.1 cloud.aws.iam.attachUserPolicy

```
attachUserPolicy(opts: { userName: string; policyArn: string }):  
Promise<Record<string, unknown>>
```

Attach a managed IAM policy to a user.

Parameters

- `opts` (*{ userName: string; policyArn: string }*) — `userName`: the user to attach the policy to.
`policyArn`: the managed policy's ARN.

Returns: A promise resolving to `{}` on success.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).iam().attachUserPolicy({ userName: "new-  
user", policyArn: "arn:aws:iam::aws:policy/ReadOnlyAccess" });
```

17.2.1.4.2 cloud.aws.iam.createUser

```
createUser(opts: { userName: string }): Promise<Record<string, unknown>>
```

Create an IAM user.

Parameters

- `opts` (*{ userName: string }*) — `userName`: the new user's name.

Returns: A promise resolving to the IAM CreateUser response — `User`: the newly created user's description.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).iam().createUser({ userName: "new-  
user" });
```

17.2.1.4.3 cloud.aws.iam.deleteUser

```
deleteUser(opts: { userName: string }): Promise<Record<string, unknown>>
```

Delete an IAM user.

Parameters

- `opts` (*{ userName: string }*) — `userName`: the user to delete.

Returns: A promise resolving to `{}` on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).iam().deleteUser({ userName: "new-user" });
```

17.2.1.4.4 cloud.aws.iam.getRole

```
getRole(opts: { roleName: string }): Promise<Record<string, unknown>>
```

Get an IAM role's metadata.

Parameters

- `opts` (*{ roleName: string }*) — `roleName`: the role's name.

Returns: A promise resolving to the IAM GetRole response — `Role: { RoleName, Arn, RoleId, AssumeRolePolicyDocument, ... }`.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const role = await cloud.aws({ region: "eu-north-1" }).iam().getRole({ roleName: "my-role" });
```

17.2.1.4.5 cloud.aws.iam.getUser

```
getUser(opts?: { userName?: string }): Promise<Record<string, unknown>>
```

Get an IAM user's metadata.

Parameters

- `opts` (*{ userName?: string }, optional*) — `userName`: the IAM user to look up; omitted ⇒ the user making the request (the credentials' own identity).

Returns: A promise resolving to the IAM GetUser response — `User: { UserName, Arn, UserId, CreateDate, ... }`.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const me = await cloud.aws({ region: "eu-north-1" }).iam().getUser();
```

17.2.1.4.6 cloud.aws.iam.listPolicies

```
listPolicies(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

List the customer-managed and AWS-managed IAM policies visible to the account.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `listPolicies` takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the IAM ListPolicies response — Policies (array of { PolicyName, Arn, PolicyId, ... }).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).iam().listPolicies();
```

17.2.1.4.7 cloud.aws.iam.listRoles

```
listRoles(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

List the IAM roles in the account.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `listRoles` takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the IAM ListRoles response — Roles (array of { RoleName, Arn, RoleId, AssumeRolePolicyDocument, ... }).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).iam().listRoles();
```

17.2.1.4.8 cloud.aws.iam.listUsers

```
listUsers(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

List the IAM users in the account.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `listUsers` takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the IAM ListUsers response — Users (array of { UserName, Arn, UserId, CreateDate, ... }), IsTruncated, Marker.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).iam().listUsers();
```

17.2.1.5 cloud.aws.lambda

Lambda — functions and invocations (github.com/aws/aws-sdk-go-v2/service/lambda). Reached via `cloud.aws({...}).lambda()` each method below is called on that service handle.

17.2.1.5.1 cloud.aws.lambda.createFunction

```
createFunction(opts: { functionName: string; role: string; runtime: string; handler: string; zipFile?: string | Uint8Array | ArrayBuffer; s3Bucket?: string; s3Key?: string }): Promise<Record<string, unknown>>
```

Create a Lambda function, from a zip file uploaded inline or a reference to one already in S3.

Parameters

- `opts` (*{ functionName: string; role: string; runtime: string; handler: string; zipFile?: string | Uint8Array | ArrayBuffer; s3Bucket?: string; s3Key?: string }*) — `functionName`: the new function's name. `role`: the execution role's ARN. `runtime`: e.g. "nodejs20.x", "provided.al2023". `handler`: the entry point, e.g. "index.handler". `zipFile`: the deployment package's bytes (string encoded as UTF-8, or Uint8Array/ArrayBuffer) uploaded inline. `s3Bucket/s3Key`: alternative to `zipFile` — reference an existing deployment package already uploaded to S3. Only the code-source field(s) actually supplied are attached to the request.

Returns: A promise resolving to the Lambda CreateFunction response, which carries the same fields as a function configuration (FunctionName, FunctionArn, Runtime, Role, Handler, CodeSha256, Version, ...).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).lambda().createFunction({ functionName: "my-fn", role: "arn:aws:iam::123456789012:role/lambda-role", runtime: "nodejs20.x", handler: "index.handler", s3Bucket: "my-bucket", s3Key: "my-fn.zip" });
```

17.2.1.5.2 cloud.aws.lambda.deleteFunction

```
deleteFunction(opts: { functionName: string }): Promise<Record<string, unknown>>
```

Delete a Lambda function.

Parameters

- `opts` (*{ functionName: string }*) — `functionName`: the function's name or ARN.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).lambda().deleteFunction({ functionName: "my-fn" });
```

17.2.1.5.3 cloud.aws.lambda.getFunction

```
getFunction(opts: { functionName: string }): Promise<Record<string, unknown>>
```

Get a Lambda function's configuration and code location.

Parameters

- `opts` (*{ functionName: string }*) — `functionName`: the function's name or ARN.

Returns: A promise resolving to the Lambda GetFunction response — Configuration (`{ FunctionName, Runtime, Handler, ... }`), Code (`{ Location, RepositoryType }`), Tags, Concurrency.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""`/`0` for non-API errors (DNS, TLS, timeout, connection refused).

```
const fn = await cloud.aws({ region: "eu-north-1" }).lambda().getFunction({ functionName: "my-fn" });
```

17.2.1.5.4 cloud.aws.lambda.invoke

```
invoke(opts: { functionName: string; payload?: string | Record<string, unknown> }): Promise<{ statusCode: number; payload: string; functionError?: string; executedVersion?: string }>
```

Synchronously invoke a Lambda function.

Parameters

- `opts` (*{ functionName: string; payload?: string | Record<string, unknown> }*) — `functionName`: the function's name or ARN. `payload`: the invocation payload — a string sent as-is, or a plain object JSON-serialised before sending; omitted $\Rightarrow$ no payload.

Returns: A promise resolving to a hand-built shape (not the raw SDK output, since Payload is raw bytes there): `statusCode` is the Lambda invocation's HTTP status; `payload` is the invoked function's raw JSON response body, always returned as a string (never JSON-parsed for you — call `JSON.parse` yourself); `functionError` and `executedVersion` are present only when the SDK response set them.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""`/`0` for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).lambda().invoke({ functionName: "my-fn", payload: { hello: "world" } });
runtime.log(JSON.parse(r.payload));
```

17.2.1.5.5 cloud.aws.lambda.listFunctions

```
listFunctions(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

List the Lambda functions in the account/region.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `listFunctions` takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the Lambda ListFunctions response — Functions (array of { FunctionName, FunctionArn, Runtime, Handler, ... }), NextMarker.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).lambda().listFunctions();
```

17.2.1.6 cloud.aws.s3

S3 — buckets and objects (github.com/aws/aws-sdk-go-v2/service/s3). Reached via `cloud.aws({...}).s3()` each method below is called on that service handle.

17.2.1.6.1 cloud.aws.s3.createBucket

```
createBucket(opts: { bucket: string }): Promise<Record<string, unknown>>
```

Create an S3 bucket.

Parameters

- `opts` (*{ bucket: string }*) — `bucket`: the new bucket's globally-unique name.

Returns: A promise resolving to the S3 CreateBucket response — Location.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).s3().createBucket({ bucket: "my-new-bucket" });
```

17.2.1.6.2 cloud.aws.s3.deleteBucket

```
deleteBucket(opts: { bucket: string }): Promise<Record<string, unknown>>
```

Delete an S3 bucket. The bucket must be empty.

Parameters

- `opts` (*{ bucket: string }*) — `bucket`: the bucket name.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).s3().deleteBucket({ bucket: "my-bucket" });
```

17.2.1.6.3 cloud.aws.s3.deleteObject

```
deleteObject(opts: { bucket: string; key: string }): Promise<Record<string, unknown>>
```

Delete an object.

Parameters

- `opts` (*{ bucket: string; key: string }*) — bucket: the bucket name. key: the object's key.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).s3().deleteObject({ bucket: "my-bucket", key: "logs/a.txt" });
```

17.2.1.6.4 cloud.aws.s3.getObject

```
getObject(opts: { bucket: string; key: string }): Promise<{ bytes: number[] }>
```

Download an object's content.

Parameters

- `opts` (*{ bucket: string; key: string }*) — bucket: the bucket name. key: the object's key.

Returns: A promise resolving to { bytes } where bytes is a plain JS number[] (byte-value array read from the object body), NOT a real Uint8Array — wrap it with new Uint8Array(res.bytes) before treating it as binary data (e.g. before fs.writeBytes or further decoding).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const res = await cloud.aws({ region: "eu-north-1" }).s3().getObject({ bucket: "my-bucket", key: "logs/a.txt" });
const bytes = new Uint8Array(res.bytes);
runtime.log(bytes.length);
```

17.2.1.6.5 cloud.aws.s3.headObject

```
headObject(opts: { bucket: string; key: string }): Promise<Record<string, unknown>>
```

Get an object's metadata without downloading its content.

Parameters

- `opts` (*{ bucket: string; key: string }*) — `bucket`: the bucket name. `key`: the object's key (path within the bucket).

Returns: A promise resolving to the S3 HeadObject response — `ContentLength`, `ContentType`, `ETag`, `LastModified`, `Metadata`, etc. (metadata only, no body).

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const meta = await cloud.aws({ region: "eu-north-1" }).s3().headObject({ bucket: "my-bucket", key: "logs/a.txt" });
```

17.2.1.6.6 cloud.aws.s3.listBuckets

```
listBuckets(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

List the S3 buckets owned by the caller.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `listBuckets` takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the S3 ListBuckets response — `Buckets` (array of { `Name`, `CreationDate` }) and `Owner`.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).s3().listBuckets();
```

17.2.1.6.7 cloud.aws.s3.listObjects

```
listObjects(opts: { bucket: string; prefix?: string }): Promise<Record<string, unknown>>
```

List objects in a bucket, optionally filtered by key prefix.

Parameters

- `opts` (*{ bucket: string; prefix?: string }*) — `bucket`: the bucket name. `prefix`: only list objects whose key starts with this string.

Returns: A promise resolving to the S3 ListObjectsV2 response — `Contents` (array of { `Key`, `Size`, `LastModified`, `ETag`, ... }), `IsTruncated`, `KeyCount`, `NextContinuationToken`.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).s3().listObjects({ bucket:
"my-bucket", prefix: "logs/" });
```

17.2.1.6.8 cloud.aws.s3.putObject

```
putObject(opts: { bucket: string; key: string; body: string | Uint8Array |
ArrayBuffer }): Promise<Record<string, unknown>>
```

Upload/overwrite an object's content.

Parameters

- `opts` (*{ bucket: string; key: string; body: string | Uint8Array | ArrayBuffer }*) — bucket: the bucket name. key: the object's key. body: a string (encoded as UTF-8) or raw bytes (Uint8Array/ArrayBuffer) to upload as the object's content.

Returns: A promise resolving to the S3 PutObject response — ETag, VersionId (when bucket versioning is enabled), etc.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).s3().putObject({ bucket: "my-bucket",
key: "logs/a.txt", body: "hello" });
```

17.2.1.7 cloud.aws.secretsmanager

Secrets Manager — secret containers and their values (github.com/aws/aws-sdk-go-v2/service/secretsmanager). Reached via `cloud.aws({...}).secretsmanager()` each method below is called on that service handle.

17.2.1.7.1 cloud.aws.secretsmanager.createSecret

```
createSecret(opts: { name: string; secretString?: string }):
Promise<Record<string, unknown>>
```

Create a secret, optionally with an initial value.

Parameters

- `opts` (*{ name: string; secretString?: string }*) — name: the new secret's name. secretString: an optional initial plaintext value; omitted $\Rightarrow$ the secret is created with no value.

Returns: A promise resolving to the Secrets Manager CreateSecret response — ARN, Name, VersionId.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).


```
await cloud.aws({ region: "eu-north-1" }).secretsmanager().createSecret({ name:
"db-password", secretString: "s3cr3t" });
```

17.2.1.7.2 cloud.aws.secretsmanager.deleteSecret

```
deleteSecret(opts: { secretId: string }): Promise<Record<string, unknown>>
```

Delete a secret.

Parameters

- `opts` (*{ secretId: string }*) — `secretId`: the secret's name or ARN.

Returns: A promise resolving to `{}` on success.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-
north-1" }).secretsmanager().deleteSecret({ secretId: "db-password" });
```

17.2.1.7.3 cloud.aws.secretsmanager.describeSecret

```
describeSecret(opts: { secretId: string }): Promise<Record<string, unknown>>
```

Get a secret's metadata (not its value — use `getSecretValue` for that).

Parameters

- `opts` (*{ secretId: string }*) — `secretId`: the secret's name or ARN.

Returns: A promise resolving to the Secrets Manager `DescribeSecret` response — ARN, Name, Description, `VersionIdsToStages`, ... (metadata only, not the value).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
const secret = await cloud.aws({ region: "eu-
north-1" }).secretsmanager().describeSecret({ secretId: "db-password" });
```

17.2.1.7.4 cloud.aws.secretsmanager.getSecretValue

```
getSecretValue(opts: { secretId: string }): Promise<{ value: string }>
```

Get a secret's current plaintext value.

Parameters

- `opts` (*{ secretId: string }*) — `secretId`: the secret's name or ARN.

Returns: A promise resolving to `{ value }` — the decoded secret value. `SecretString` is returned verbatim when present; otherwise `SecretBinary`'s raw bytes are converted to a

string as-is (no further base64 decoding — the SDK already decodes the wire-format blob for both fields).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const { value } = await cloud.aws({ region: "eu-  
north-1" }).secretsmanager().getSecretValue({ secretId: "db-password" });  
runtime.log(value);
```

17.2.1.7.5 cloud.aws.secretsmanager.listSecrets

```
listSecrets(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

List the secrets in the account (metadata only — not their values).

Parameters

- *opts* (*Record<string, never>, optional*) — *opts*: unused — *listSecrets* takes no filtering parameters; accepts only an empty object or omission.

Returns: A promise resolving to the Secrets Manager ListSecrets response — SecretList (array of { Name, ARN, Description, ... }), NextToken.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-  
north-1" }).secretsmanager().listSecrets();
```

17.2.1.7.6 cloud.aws.secretsmanager.putSecretValue

```
putSecretValue(opts: { secretId: string; secretString: string }):  
Promise<Record<string, unknown>>
```

Add a new value to an existing secret (creates a new version).

Parameters

- *opts* (*{ secretId: string; secretString: string }*) — *secretId*: the secret's name or ARN. *secretString*: the new plaintext value.

Returns: A promise resolving to the Secrets Manager PutSecretValue response — ARN, Name, VersionId, VersionStages.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-  
north-1" }).secretsmanager().putSecretValue({ secretId: "db-password",  
secretString: "n3w-s3cr3t" });
```

17.2.1.8 cloud.aws.sqs

SQS — queues and messages (github.com/aws/aws-sdk-go-v2/service/sqs). Reached via `cloud.aws({...}).sqs()` each method below is called on that service handle.

17.2.1.8.1 cloud.aws.sqs.createQueue

```
createQueue(opts: { queueName: string }): Promise<Record<string, unknown>>
```

Create a standard or FIFO SQS queue.

Parameters

- `opts` (*{ queueName: string }*) — `queueName`: the new queue's name (append `".fifo"` for a FIFO queue).

Returns: A promise resolving to the SQS CreateQueue response — `QueueUrl`: the new queue's URL.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).sqs().createQueue({ queueName: "my-queue" });
```

17.2.1.8.2 cloud.aws.sqs.deleteMessage

```
deleteMessage(opts: { queueUrl: string; receiptHandle: string }): Promise<Record<string, unknown>>
```

Delete a message from a queue after successful processing.

Parameters

- `opts` (*{ queueUrl: string; receiptHandle: string }*) — `queueUrl`: the queue's URL.
`receiptHandle`: the receipt handle returned by `receiveMessage` for this message (not the `MessageId`).

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).sqs().deleteMessage({ queueUrl: "https://sqs.eu-north-1.amazonaws.com/123456789012/my-queue", receiptHandle: r.Messages[0].ReceiptHandle });
```

17.2.1.8.3 cloud.aws.sqs.deleteQueue

```
deleteQueue(opts: { queueUrl: string }): Promise<Record<string, unknown>>
```

Delete a queue.

Parameters

- `opts` (*{ queueUrl: string }*) — `queueUrl`: the queue's URL (as returned by `createQueue/listQueues`).

Returns: A promise resolving to `{}` on success.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).sqs().deleteQueue({ queueUrl: "https://sqs.eu-north-1.amazonaws.com/123456789012/my-queue" });
```

17.2.1.8.4 cloud.aws.sqs.getQueueAttributes

```
getQueueAttributes(opts: { queueUrl: string; attributeNames?: string[] }): Promise<Record<string, unknown>>
```

Get a queue's attributes.

Parameters

- `opts` (*{ queueUrl: string; attributeNames?: string[] }*) — `queueUrl`: the queue's URL.
`attributeNames`: which attributes to fetch (e.g. "All", "ApproximateNumberOfMessages");
`omitted` $\Rightarrow$ none are requested.

Returns: A promise resolving to the SQS `GetQueueAttributes` response — `Attributes`: a flat map of attribute name to string value.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).sqs().getQueueAttributes({ queueUrl: "https://sqs.eu-north-1.amazonaws.com/123456789012/my-queue", attributeNames: ["All"] });
```

17.2.1.8.5 cloud.aws.sqs.listQueues

```
listQueues(opts?: { prefix?: string }): Promise<Record<string, unknown>>
```

List queue URLs, optionally filtered by name prefix.

Parameters

- `opts` (*{ prefix?: string }, optional*) — `prefix`: only list queues whose name starts with this string (sent as `QueueNamePrefix`); `omitted` $\Rightarrow$ list all queues.

Returns: A promise resolving to the SQS `ListQueues` response — `QueueUrls` (array of queue URL strings), `NextToken`.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are `""/0` for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).sqs().listQueues({ prefix:
"orders-" });
```

17.2.1.8.6 cloud.aws.sqs.receiveMessage

```
receiveMessage(opts: { queueUrl: string; maxMessages?: number }):
Promise<Record<string, unknown>>
```

Receive (poll for) messages from a queue.

Parameters

- `opts` (*{ queueUrl: string; maxMessages?: number }*) — `queueUrl`: the queue's URL.
`maxMessages`: the maximum number of messages to return (1-10); omitted $\Rightarrow$ the API's own default (1).

Returns: A promise resolving to the SQS ReceiveMessage response — `Messages` (array of `{ MessageId, ReceiptHandle, Body, MD5OfBody, ... }`); absent/empty when the queue currently has no visible messages.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-
north-1" }).sqs().receiveMessage({ queueUrl: "https://sqs.eu-north-1.amazonaws.
com/123456789012/my-queue", maxMessages: 5 });
```

17.2.1.8.7 cloud.aws.sqs.sendMessage

```
sendMessage(opts: { queueUrl: string; messageBody: string }):
Promise<Record<string, unknown>>
```

Send a message to a queue.

Parameters

- `opts` (*{ queueUrl: string; messageBody: string }*) — `queueUrl`: the queue's URL.
`messageBody`: the message text.

Returns: A promise resolving to the SQS SendMessage response — `MessageId`, `MD5OfMessageBody`, and (for FIFO queues) `SequenceNumber`.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.aws({ region: "eu-north-1" }).sqs().sendMessage({ queueUrl: "https://
sqs.eu-north-1.amazonaws.com/123456789012/my-queue", messageBody: "hello" });
```

17.2.1.9 cloud.aws.sts

STS — caller identity and temporary credentials (github.com/aws/aws-sdk-go-v2/service/sts). Reached via `cloud.aws({...}).sts()` each method below is called on that service handle.

17.2.1.9.1 cloud.aws.sts.assumeRole

```
assumeRole(opts: { roleArn: string; roleSessionName: string; durationSeconds?: number }): Promise<Record<string, unknown>>
```

Assume an IAM role, returning temporary credentials.

Parameters

- `opts` (*{ roleArn: string; roleSessionName: string; durationSeconds?: number }*) — `roleArn`: the ARN of the role to assume. `roleSessionName`: an identifier for the assumed-role session (shows up in CloudTrail). `durationSeconds`: session duration in seconds (AWS minimum 900); omitted $\Rightarrow$ the API's own default (3600).

Returns: A promise resolving to the STS AssumeRole response — Credentials: { AccessKeyId, SecretAccessKey, SessionToken, Expiration } plus AssumedRoleUser. These are temporary but sensitive credentials — handle/store them with the same care as long-lived ones; never log them.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).sts().assumeRole({ roleArn: "arn:aws:iam::123456789012:role/my-role", roleSessionName: "my-session" });
```

17.2.1.9.2 cloud.aws.sts.getCallerIdentity

```
getCallerIdentity(opts?: Record<string, never>): Promise<Record<string, unknown>>
```

Get the identity behind the credentials currently in use.

Parameters

- `opts` (*Record<string, never>, optional*) — `opts`: unused — `getCallerIdentity` takes no parameters; accepts only an empty object or omission.

Returns: A promise resolving to the STS GetCallerIdentity response — Account, Arn, UserId. Contains no secrets.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code is the AWS/smithy error code and status the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const who = await cloud.aws({ region: "eu-north-1" }).sts().getCallerIdentity({});
```

17.2.1.9.3 cloud.aws.sts.getSessionToken

```
getSessionToken(opts?: { durationSeconds?: number }): Promise<Record<string, unknown>>
```

Get temporary credentials for the current principal.

Parameters

- `opts` (*{ durationSeconds?: number }, optional*) — `durationSeconds`: session duration in seconds; omitted $\Rightarrow$ the API's own default (43200 / 12h for IAM-user credentials; 3600 / 1h for root-account credentials).

Returns: A promise resolving to the STS `getSessionToken` response — Credentials: { `AccessKeyId`, `SecretAccessKey`, `SessionToken`, `Expiration` }. Same sensitivity note as `assumeRole` applies.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code` is the AWS/smithy error code and `status` the HTTP status; both are ""/0 for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.aws({ region: "eu-north-1" }).sts().getSessionToken();
```

17.2.2 cloud.azure

```
azure(opts?: { subscriptionId?: string; tenantId?: string; clientId?: string;
clientSecret?: string }): {
  resourceGroups(): {
    list(opts?: Record<string, never>): Promise<{ value?: Array<Record<string,
unknown>> }>;
    get(opts: { name: string }): Promise<Record<string, unknown>>;
    create(opts: { name: string; location: string }): Promise<Record<string,
unknown>>;
    delete(opts: { name: string }): Promise<Record<string, unknown>>;
  };
  compute(): {
    listVirtualMachines(opts?: { resourceGroup?: string }): Promise<{ value?:
Array<Record<string, unknown>> }>;
    getVirtualMachine(opts: { resourceGroup: string; name: string }):
Promise<Record<string, unknown>>;
    start(opts: { resourceGroup: string; name: string }): Promise<Record<string,
unknown>>;
    powerOff(opts: { resourceGroup: string; name: string }):
Promise<Record<string, unknown>>;
    deallocate(opts: { resourceGroup: string; name: string }):
Promise<Record<string, unknown>>;
    delete(opts: { resourceGroup: string; name: string }): Promise<Record<string,
unknown>>;
  };
  resources(): {
    listByResourceGroup(opts: { resourceGroup: string }): Promise<{ value?:
Array<Record<string, unknown>> }>;
    getById(opts: { resourceId: string; apiVersion: string }):
Promise<Record<string, unknown>>;
  };
  call(opts: { path: string; apiVersion: string; method?: string; params?:
```

```
Record<string, string>; body?: unknown }): Promise<unknown>;
  blob(accountUrl: string): {
    listContainers(opts?: Record<string, never>): Promise<{ value?:
Array<Record<string, unknown>> }>;
    listBlobs(opts: { container: string }): Promise<{ value?: Array<Record<string,
unknown>> }>;
    download(opts: { container: string; blob: string }): Promise<{ bytes:
number[] }>;
    upload(opts: { container: string; blob: string; body: string | Uint8Array |
ArrayBuffer }): Promise<Record<string, unknown>>;
    deleteBlob(opts: { container: string; blob: string }): Promise<Record<string,
unknown>>;
  };
  keyvaultSecrets(vaultUrl: string): {
    listSecrets(opts?: Record<string, never>): Promise<{ value?:
Array<Record<string, unknown>> }>;
    getSecret(opts: { name: string }): Promise<{ value: string }>;
    setSecret(opts: { name: string; value: string }): Promise<Record<string,
unknown>>;
    deleteSecret(opts: { name: string }): Promise<Record<string, unknown>>;
  };
}
```

PROVISIONAL — built against the Azure SDK but not yet verified against a live Azure account. Microsoft Azure provider. `cloud.azure(opts?)` returns a handle with three ARM (Resource Manager) services — `resourceGroups`, `compute`, `resources` — plus a generic ARM `call()` REST escape hatch, and two data-plane services — `blob`, `keyvaultSecrets` — that take an endpoint URL directly. Pure-Go, CGO-free (`azure-sdk-for-go`); reuses the standard Azure credential chain.

Parameters

- `opts` (*{ subscriptionId?: string; tenantId?: string; clientId?: string; clientSecret?: string }, optional*) — `subscriptionId`: the ARM subscription id used by the ARM services `resourceGroups()`/`compute()`/`resources()`; omitted $\Rightarrow$ falls back to the `AZURE_SUBSCRIPTION_ID` env var (required only when an ARM service is actually invoked — `call()` targets `management.azure.com` with the subscription embedded in its path, and `blob()`/`keyvaultSecrets()` operate on a caller-supplied endpoint URL, so none of those need a configured subscription). `tenantId/clientId/clientSecret`: together select a client-secret (service-principal) credential; when any is omitted, falls back to `DefaultAzureCredential` (environment variables, managed identity, az login, and the other links in the default chain).

Returns: The Azure provider handle: `{ resourceGroups(), compute(), resources(), call(opts), blob(accountUrl), keyvaultSecrets(vaultUrl) }`. `resourceGroups()`/`compute()`/`resources()` return fresh ARM service handles bound to this call's subscription/credential; `call()` is the generic ARM REST escape hatch for APIs without a typed service above; `blob(accountUrl)/keyvaultSecrets(vaultUrl)` return data-plane handles bound directly to the given endpoint URL, independent of any subscription.

Throws: `cloud.azure(opts)` itself throws synchronously (not a rejected promise) only if `opts` is provided but is not a plain object — there is no further synchronous validation of the credential fields (a bad/incomplete `tenantId/clientId/clientSecret` combination fails later,

asynchronously, on first credential use). Every service method (ARM and data-plane alike) returns a promise that rejects with a structured Error { code, status, message, details } on API/transport failure — code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). resourceGroups()/compute()/resources() additionally reject if no subscription is configured (neither opts.subscriptionId nor AZURE_SUBSCRIPTION_ID); call(), blob() and keyvaultSecrets() have no such requirement — call() embeds the subscription in its path and targets management.azure.com, while blob()/keyvaultSecrets() operate directly against the caller-supplied accountUrl/vaultUrl.

```
// PROVISIONAL example – illustrative only, not run against a live account.
const az = cloud.azure({ subscriptionId:
  "00000000-0000-0000-0000-000000000000" });
const groups = await az.resourceGroups().list();
runtime.log(groups.value?.length ?? 0);

const kv = az.keyvaultSecrets("https://my-vault.vault.azure.net");
const { value } = await kv.getSecret({ name: "db-password" });
```

17.2.2.1 cloud.azure.blob

Blob Storage — containers and blobs on a storage account (azure-sdk-for-go azblob.Client). Data-plane: reached via cloud.azure({...}).blob(accountUrl) , where accountUrl is the target storage account's blob endpoint (e.g. https://myaccount.blob.core.windows.net) supplied directly by the caller rather than resolved from the subscription — no subscription is required for these methods. Each method below is called on that service handle.

17.2.2.1.1 cloud.azure.blob.deleteBlob

```
deleteBlob(opts: { container: string; blob: string }): Promise<Record<string,
unknown>>
```

Delete a blob.

Parameters

- `opts` (*{ container: string; blob: string }*) — container: the container's name. blob: the blob's name.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
await cloud.azure({}).blob("https://myaccount.blob.core.windows.
net").deleteBlob({ container: "logs", blob: "a.txt" });
```

17.2.2.1.2 cloud.azure.blob.download

```
download(opts: { container: string; blob: string }): Promise<{ bytes: number[] }>
```

Download a blob's entire content into memory.

Parameters

- `opts` (*{ container: string; blob: string }*) — `container`: the container's name. `blob`: the blob's name (path within the container).

Returns: A promise resolving to `{ bytes }` where `bytes` is a plain JS `number[]` (byte-value array), NOT a real `Uint8Array` — wrap it with `new Uint8Array(res.bytes)` before treating it as binary data (e.g. before `fs.writeBytes` or further decoding).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `""/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
const blob = cloud.azure({}).blob("https://myaccount.blob.core.windows.net");
const res = await blob.download({ container: "logs", blob: "a.txt" });
const bytes = new Uint8Array(res.bytes);
runtime.log(bytes.length);
```

17.2.2.1.3 cloud.azure.blob.listBlobs

```
listBlobs(opts: { container: string }): Promise<{ value?: Array<Record<string,
unknown>> }>
```

List every blob in a container (flat listing — no hierarchy/delimiter), paging through every page the SDK's pager returns.

Parameters

- `opts` (*{ container: string }*) — `container`: the container's name.

Returns: A promise resolving to `{ value }` — a list envelope wrapping every blob, with PascalCase keys (`Name`, `Properties`, `Deleted`, ...) — see `listContainers` for why.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `""/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
const r = await cloud.azure({}).blob("https://myaccount.blob.core.windows.net").listBlobs({ container: "logs" });
```

17.2.2.1.4 cloud.azure.blob.listContainers

```
listContainers(opts?: Record<string, never>): Promise<{ value?:
Array<Record<string, unknown>> }>
```

List every container in the storage account, paging through every page the SDK's pager returns.

Parameters

- `opts` (*Record<string, never>, optional*) — Unused — `listContainers` ignores any options object entirely.

Returns: A promise resolving to `{ value }` — a list envelope wrapping every container. The Storage Blob SDK's structs carry no JSON tags (the wire protocol is XML, not JSON), so

toPlain's JSON round-trip falls back to the Go struct field names verbatim: PascalCase keys (Name, Properties, Deleted, ...) — unlike the ARM services and keyvaultSecrets, which come back lowercase-camelCase.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
const blob = cloud.azure({}).blob("https://myaccount.blob.core.windows.net");
const r = await blob.listContainers();
runtime.log(r.value?.length ?? 0);
```

17.2.2.1.5 cloud.azure.blob.upload

```
upload(opts: { container: string; blob: string; body: string | Uint8Array |
ArrayBuffer }): Promise<Record<string, unknown>>
```

Upload a blob's content, creating or overwriting it. Bodies up to 256 MiB are sent in a single Put Blob request; larger bodies are split into staged blocks and committed (the SDK's UploadBuffer).

Parameters

- `opts` (*{ container: string; blob: string; body: string | Uint8Array | ArrayBuffer }*) — container: the container's name. blob: the blob's name. body: a string (encoded as UTF-8) or raw bytes (Uint8Array/ArrayBuffer) to upload as the blob's content.

Returns: A promise resolving to the SDK's UploadBuffer response, round-tripped via toPlain (PascalCase keys — see listContainers for why).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
await cloud.azure({}).blob("https://myaccount.blob.core.windows.
net").upload({ container: "logs", blob: "a.txt", body: "hello" });
```

17.2.2.2 cloud.azure.call

```
call(opts: { path: string; apiVersion: string; method?: string; params?:
Record<string, string>; body?: unknown }): Promise<unknown>
```

Generic path-based REST escape hatch onto the ARM (management.azure.com) API — for ARM APIs without a typed service above (resourceGroups/compute/resources). Authenticates the same way as the parent cloud.azure(...) handle, acquiring a management.azure.com/.default bearer token.

Parameters

- `opts` (*{ path: string; apiVersion: string; method?: string; params?: Record<string, string>; body?: unknown }*) — path: request path appended to https://management.azure.com as-is, e.g. "/subscriptions/{id}/providers/Microsoft.Compute/virtualMachines" — the caller supplies the subscription segment directly. apiVersion: the ARM api-version query

parameter (required — ARM has no default). method: HTTP verb, defaults to “GET”.
params: additional query-string parameters (merged with api-version). body: JSON-serialisable request body; sent with Content-Type: application/json when present.

Returns: A promise resolving to the decoded JSON response body ({} when the response body is empty).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are “”/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Also rejects if path or apiVersion is missing/empty, or if body is not JSON-serialisable. Note: call() does NOT require a configured subscription — it targets https://management.azure.com and the caller embeds the subscription segment in path directly.

```
// PROVISIONAL example — illustrative only, not run against a live account.  
const az = cloud.azure({ subscriptionId:  
  "00000000-0000-0000-0000-000000000000" });  
const vms = await az.call({  
  path: "/subscriptions/00000000-0000-0000-0000-000000000000/providers/  
  Microsoft.Compute/virtualMachines",  
  apiVersion: "2023-09-01",  
});
```

17.2.2.3 cloud.azure.compute

Virtual Machines — ARM (subscription-scoped) compute instances (azure-sdk-for-go armcompute.VirtualMachinesClient). Reached via cloud.azure({...}).compute() each method below is called on that service handle.

17.2.2.3.1 cloud.azure.compute.deallocate

```
deallocate(opts: { resourceGroup: string; name: string }): Promise<Record<string,  
unknown>>
```

Deallocate a virtual machine — releases the compute resources (and their billing) while retaining disks and configuration. Long-running ARM operation; the call blocks (polls) until it completes.

Parameters

- `opts` ({ *resourceGroup*: string; *name*: string }) — *resourceGroup*: the VM’s resource group. *name*: the VM’s name.

Returns: A promise resolving to {} once the deallocate operation completes.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are “”/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither opts.subscriptionId nor AZURE_SUBSCRIPTION_ID).

```
await cloud.azure({ subscriptionId: "...", }).compute().deallocate({ resourceGroup:  
  "my-rg", name: "web-1" });
```

17.2.2.3.2 cloud.azure.compute.delete

```
delete(opts: { resourceGroup: string; name: string }): Promise<Record<string, unknown>>
```

Delete a virtual machine. Long-running ARM operation; the call blocks (polls) until it completes.

Parameters

- `opts` (*{ resourceGroup: string; name: string }*) — `resourceGroup`: the VM's resource group.
`name`: the VM's name.

Returns: A promise resolving to `{}` once the delete operation completes.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `""/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
await cloud.azure({ subscriptionId: "..."}).compute().delete({ resourceGroup: "my-rg", name: "web-1" });
```

17.2.2.3.3 cloud.azure.compute.getVirtualMachine

```
getVirtualMachine(opts: { resourceGroup: string; name: string }): Promise<Record<string, unknown>>
```

Fetch a single virtual machine's metadata.

Parameters

- `opts` (*{ resourceGroup: string; name: string }*) — `resourceGroup`: the VM's resource group.
`name`: the VM's name.

Returns: A promise resolving to the VM's ARM JSON representation (lowercase-camelCase keys: `id`, `name`, `location`, `properties`, ...).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `""/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
const vm = await cloud.azure({ subscriptionId: "..."}).compute().getVirtualMachine({ resourceGroup: "my-rg", name: "web-1" });
```

17.2.2.3.4 cloud.azure.compute.listVirtualMachines

```
listVirtualMachines(opts?: { resourceGroup?: string }): Promise<{ value?: Array<Record<string, unknown>> }>
```

List virtual machines — scoped to a single resource group when one is given, else every VM in the subscription.

Parameters

- `opts` (*{ resourceGroup?: string }, optional*) — `resourceGroup`: restrict the listing to this resource group; omitted $\Rightarrow$ subscription-wide (the SDK's `NewListAllPager`).

Returns: A promise resolving to `{ value }` — the ARM list envelope wrapping every matching VM, each in the ARM SDK's own JSON shape (lowercase-camelCase keys: `id`, `name`, `location`, `properties`, ...).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `"/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
const r = await cloud.azure({ subscriptionId:
  "..."}).compute().listVirtualMachines({ resourceGroup: "my-rg" });
```

17.2.2.3.5 cloud.azure.compute.powerOff

```
powerOff(opts: { resourceGroup: string; name: string }): Promise<Record<string,
unknown>>
```

Power off a running virtual machine. The VM stays allocated and keeps incurring compute charges (use `deallocate()` to stop compute billing); disks remain attached. Long-running ARM operation; the call blocks (polls) until it completes.

Parameters

- `opts` (*{ resourceGroup: string; name: string }*) — `resourceGroup`: the VM's resource group.
`name`: the VM's name.

Returns: A promise resolving to `{}` once the power-off operation completes.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `"/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
await cloud.azure({ subscriptionId: "..."}).compute().powerOff({ resourceGroup:
  "my-rg", name: "web-1" });
```

17.2.2.3.6 cloud.azure.compute.start

```
start(opts: { resourceGroup: string; name: string }): Promise<Record<string,
unknown>>
```

Start a stopped virtual machine. Long-running ARM operation; the call blocks (polls) until it completes.

Parameters

- `opts` (*{ resourceGroup: string; name: string }*) — `resourceGroup`: the VM's resource group.
`name`: the VM's name.

Returns: A promise resolving to `{}` once the start operation completes.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither opts.subscriptionId nor AZURE_SUBSCRIPTION_ID).

```
await cloud.azure({ subscriptionId: "..."}).compute().start({ resourceGroup: "my-rg", name: "web-1" });
```

17.2.2.4 cloud.azure.keyvaultSecrets

Key Vault Secrets — secret values and their versioned metadata in a vault (azure-sdk-for-go azsecrets.Client). Data-plane: reached via cloud.azure({...}).keyvaultSecrets(vaultUrl) , where vaultUrl is the target vault's endpoint (e.g. https://myvault.vault.azure.net) supplied directly by the caller rather than resolved from the subscription — no subscription is required for these methods. Each method below is called on that service handle.

17.2.2.4.1 cloud.azure.keyvaultSecrets.deleteSecret

```
deleteSecret(opts: { name: string }): Promise<Record<string, unknown>>
```

Delete a secret and all of its versions.

Parameters

- `opts` ({ *name*: string }) — name: the secret's name.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
await cloud.azure({}).keyvaultSecrets("https://my-vault.vault.azure.net").deleteSecret({ name: "db-password" });
```

17.2.2.4.2 cloud.azure.keyvaultSecrets.getSecret

```
getSecret(opts: { name: string }): Promise<{ value: string }>
```

Fetch the latest version of a secret and its decoded plaintext value.

Parameters

- `opts` ({ *name*: string }) — name: the secret's name.

Returns: A promise resolving to { value } — the secret's current plaintext value (already decoded; never the raw wire form). The value is never logged anywhere in this path.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
const kv = cloud.azure({}).keyvaultSecrets("https://my-vault.vault.azure.net");
const { value } = await kv.getSecret({ name: "db-password" });
runtime.log(value);
```

17.2.2.4.3 cloud.azure.keyvaultSecrets.listSecrets

```
listSecrets(opts?: Record<string, never>): Promise<{ value?: Array<Record<string, unknown>> }>
```

List every secret's properties in the vault (metadata only — never includes values, matching the SDK's own List operation), paging through every page the SDK's pager returns.

Parameters

- `opts` (*Record<string, never>, optional*) — Unused — `listSecrets` ignores any options object entirely.

Returns: A promise resolving to `{ value }` — a list envelope wrapping every secret's properties. Uses the SDK's own MarshalJSON, so keys come back lowercase-camelCase (`id`, `attributes`, `contentType`, `managed`, `tags`, ...) — the same convention as the ARM services, unlike blob's PascalCase.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `"/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
const kv = cloud.azure({}).keyvaultSecrets("https://my-vault.vault.azure.net");
const r = await kv.listSecrets();
runtime.log(r.value?.length ?? 0);
```

17.2.2.4.4 cloud.azure.keyvaultSecrets.setSecret

```
setSecret(opts: { name: string; value: string }): Promise<Record<string, unknown>>
```

Create a new version of a secret with the given value.

Parameters

- `opts` (*{ name: string; value: string }*) — `name`: the secret's name. `value`: the plaintext value to store; never logged.

Returns: A promise resolving to the SDK's SetSecret response, round-tripped via `toPlain` (lowercase-camelCase keys: `id`, `attributes`, ... — same convention as the ARM services).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `"/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure).

```
await cloud.azure({}).keyvaultSecrets("https://my-vault.vault.azure.net").setSecret({ name: "db-password", value: "s3cr3t" });
```

17.2.2.5 cloud.azure.resourceGroups

Resource Groups — ARM (subscription-scoped) container objects for organizing resources (`azure-sdk-for-go armresources.ResourceGroupsClient`). Reached via

`cloud.azure({...}).resourceGroups()` each method below is called on that service handle.

17.2.2.5.1 cloud.azure.resourceGroups.create

```
create(opts: { name: string; location: string }): Promise<Record<string, unknown>>
```

Create a resource group, or update it if one with this name already exists (ARM's CreateOrUpdate semantics).

Parameters

- `opts` (*{ name: string; location: string }*) — name: the resource group's name. location: the Azure region, e.g. "westeurope".

Returns: A promise resolving to the created/updated resource group's ARM JSON representation.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
await cloud.azure({ subscriptionId: "..."}).resourceGroups().create({ name: "my-rg", location: "westeurope" });
```

17.2.2.5.2 cloud.azure.resourceGroups.delete

```
delete(opts: { name: string }): Promise<Record<string, unknown>>
```

Delete a resource group and everything in it. This is a long-running ARM operation; the call blocks (polls) until it completes.

Parameters

- `opts` (*{ name: string }*) — name: the resource group's name.

Returns: A promise resolving to {} once the delete operation completes.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
await cloud.azure({ subscriptionId: "..."}).resourceGroups().delete({ name: "my-rg" });
```

17.2.2.5.3 cloud.azure.resourceGroups.get

```
get(opts: { name: string }): Promise<Record<string, unknown>>
```

Fetch a single resource group by name.

Parameters

- `opts` (*{ name: string }*) — name: the resource group's name.

Returns: A promise resolving to the resource group's ARM JSON representation (lowercase-camelCase keys: id, name, location, properties, tags, ...).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither opts.subscriptionId nor AZURE_SUBSCRIPTION_ID).

```
const rg = await cloud.azure({ subscriptionId:
  "..."}).resourceGroups().get({ name: "my-rg" });
```

17.2.2.5.4 cloud.azure.resourceGroups.list

```
list(opts?: Record<string, never>): Promise<{ value?: Array<Record<string,
  unknown>> }>
```

List every resource group in the subscription, paging through every page the ARM API returns.

Parameters

- *opts* (*Record<string, never>*, *optional*) — Unused — list ignores any options object entirely; the subscription is that of the parent cloud.azure(...) handle.

Returns: A promise resolving to { value } — the ARM list envelope wrapping every resource group, each in the ARM SDK's own JSON shape (lowercase-camelCase keys: id, name, location, properties, tags, ...).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are ""/0 for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither opts.subscriptionId nor AZURE_SUBSCRIPTION_ID).

```
const az = cloud.azure({ subscriptionId:
  "00000000-0000-0000-0000-000000000000" });
const r = await az.resourceGroups().list();
runtime.log(r.value?.length ?? 0);
```

17.2.2.6 cloud.azure.resources

Generic Resources — ARM (subscription-scoped) cross-resource-type listing and lookup (azure-sdk-for-go armresources.Client — a distinct client from the resourceGroups() service above). Reached via cloud.azure({...}).resources() each method below is called on that service handle.

17.2.2.6.1 cloud.azure.resources.getById

```
getById(opts: { resourceId: string; apiVersion: string }): Promise<Record<string,
  unknown>>
```

Fetch a single resource by its fully qualified ARM resource ID. Unlike the typed services above, the generic resources API has no per-resource-type default api-version, so the caller must supply one explicitly.

Parameters

- `opts` (*{ resourceId: string; apiVersion: string }*) — `resourceId`: the resource's fully qualified ARM ID (e.g. `"/subscriptions/.../resourceGroups/my-rg/providers/Microsoft.Compute/virtualMachines/web-1"`). `apiVersion`: the ARM api-version to request for this resource type, e.g. `"2023-09-01"`.

Returns: A promise resolving to the resource's ARM JSON representation (lowercase-camelCase keys).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `""/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
const vm = await cloud.azure({ subscriptionId: "..."}).resources().getById({
  resourceId: "/subscriptions/.../resourceGroups/my-rg/providers/
Microsoft.Compute/virtualMachines/web-1",
  apiVersion: "2023-09-01",
});
```

17.2.2.6.2 cloud.azure.resources.listByResourceGroup

```
listByResourceGroup(opts: { resourceGroup: string }): Promise<{ value?:
Array<Record<string, unknown>> }>
```

List every resource (of any type) in a resource group, paging through every page the ARM API returns.

Parameters

- `opts` (*{ resourceGroup: string }*) — `resourceGroup`: the resource group's name.

Returns: A promise resolving to `{ value }` — the ARM list envelope wrapping every resource in the group, each in the ARM SDK's own generic-resource JSON shape (lowercase-camelCase keys: `id`, `name`, `type`, `location`, ...).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. `code/status` are `""/0` for non-API errors (DNS, TLS, timeout, connection refused, credential/token acquisition failure). Additionally rejects if no subscription is configured (neither `opts.subscriptionId` nor `AZURE_SUBSCRIPTION_ID`).

```
const r = await cloud.azure({ subscriptionId:
"..."}).resources().listByResourceGroup({ resourceGroup: "my-rg" });
```

17.2.3 cloud.google

```
google(opts?: { project?: string; credentials?: string | Record<string, unknown>;
scopes?: string[]; quotaProject?: string }): {
  storage(): {
    listBuckets(opts: { project: string }): Promise<{ items?: Array<Record<string,
unknown>> }>;
    getBucket(opts: { bucket: string }): Promise<Record<string, unknown>>;
    createBucket(opts: { project: string; bucket: string }):
Promise<Record<string, unknown>>;
```

```

    deleteBucket(opts: { bucket: string }): Promise<Record<string, unknown>>;
    listObjects(opts: { bucket: string; prefix?: string }): Promise<{ items?:
Array<Record<string, unknown>> }>;
    statObject(opts: { bucket: string; key: string }): Promise<Record<string,
unknown>>;
    readObject(opts: { bucket: string; key: string }): Promise<{ bytes: number[] }
>;
    putObject(opts: { bucket: string; key: string; body: string | Uint8Array |
ArrayBuffer }): Promise<Record<string, unknown>>;
    deleteObject(opts: { bucket: string; key: string }): Promise<Record<string,
unknown>>;
  };
  compute(): {
    listInstances(opts: { project: string; zone: string }): Promise<{ items?:
Array<Record<string, unknown>> }>;
    getInstance(opts: { project: string; zone: string; name: string }):
Promise<Record<string, unknown>>;
    createInstance(opts: { project: string; zone: string; instance: Record<string,
unknown> }): Promise<Record<string, unknown>>;
    deleteInstance(opts: { project: string; zone: string; name: string }):
Promise<Record<string, unknown>>;
    startInstance(opts: { project: string; zone: string; name: string }):
Promise<Record<string, unknown>>;
    stopInstance(opts: { project: string; zone: string; name: string }):
Promise<Record<string, unknown>>;
    listZones(opts: { project: string }): Promise<{ items?: Array<Record<string,
unknown>> }>;
    listDisks(opts: { project: string; zone: string }): Promise<{ items?:
Array<Record<string, unknown>> }>;
  };
  iam(): {
    listServiceAccounts(opts: { project: string }): Promise<{ accounts?:
Array<Record<string, unknown>> }>;
    getServiceAccount(opts: { project: string; email: string }):
Promise<Record<string, unknown>>;
    createServiceAccount(opts: { project: string; accountId: string; displayName?:
string }): Promise<Record<string, unknown>>;
    deleteServiceAccount(opts: { project: string; email: string }):
Promise<Record<string, unknown>>;
    listKeys(opts: { project: string; email: string }): Promise<{ keys?:
Array<Record<string, unknown>> }>;
    createKey(opts: { project: string; email: string }): Promise<Record<string,
unknown>>;
    getIamPolicy(opts: { resource: string }): Promise<Record<string, unknown>>;
    setIamPolicy(opts: { resource: string; policy: Record<string, unknown> }):
Promise<Record<string, unknown>>;
  };
  secrets(): {
    listSecrets(opts: { project: string }): Promise<{ secrets?:
Array<Record<string, unknown>> }>;
    getSecret(opts: { project: string; name: string }): Promise<Record<string,
unknown>>;
    createSecret(opts: { project: string; name: string }): Promise<Record<string,
unknown>>;
    addSecretVersion(opts: { project: string; name: string; payload: string }):

```

```

Promise<Record<string, unknown>>;
  accessSecretVersion(opts: { project: string; name: string; version?:
string }): Promise<{ value: string }>;
  deleteSecret(opts: { project: string; name: string }): Promise<Record<string,
unknown>>;
};
  call(opts: { api: string; version?: string; httpMethod?: string; path: string;
params?: Record<string, string>; body?: unknown }): Promise<unknown>;
}

```

Google Cloud provider handle. Pure-Go, CGO-free (google.golang.org/api); reuses Application Default Credentials unless credentials is given. Returns an object exposing `storage()`, `compute()`, `iam()`, `secrets()`, and the generic `call()` REST escape hatch.

Parameters

- `opts` (*{ project?: string; credentials?: string | Record<string, unknown>; scopes?: string[]; quotaProject?: string }, optional*) — `project`: the default GCP project id used by any service call that omits its own `project`. `credentials`: a file path (string) to a service-account JSON key, or an inline service-account JSON object; omitted $\Rightarrow$ Application Default Credentials (gcloud auth application-default login, `GOOGLE_APPLICATION_CREDENTIALS`, or attached-metadata identity). `scopes`: OAuth scopes to request; omitted $\Rightarrow$ each service's default scope. `quotaProject`: billing/quota project override (X-Goog-User-Project).

Returns: The provider handle: `{ storage(), compute(), iam(), secrets(), call(opts) }`. Each of `storage()/compute()/iam()/secrets()` returns a fresh service handle bound to this call's config; `call()` is the generic path-based REST escape hatch for APIs without a typed service above.

Throws: Throws synchronously (not a rejected promise) if `opts` is provided but is not an object, or `credentials` is neither a string path nor a plain object.

```

const g = cloud.google({ project: "my-proj" });
const gcs = g.storage();
const r = await gcs.listBuckets({ project: "my-proj" });
runtime.log(r.items?.length ?? 0);

```

17.2.3.1 cloud.google.call

```

call(opts: { api: string; version?: string; httpMethod?: string; path: string;
params?: Record<string, string>; body?: unknown }): Promise<unknown>

```

Generic path-based REST escape hatch onto any `{api}.googleapis.com` endpoint — for APIs without a typed service above (`storage/compute/iam/secrets`). Authenticates the same way as the parent `cloud.google(...)` handle.

Parameters

- `opts` (*{ api: string; version?: string; httpMethod?: string; path: string; params?: Record<string, string>; body?: unknown }*) — `api`: the API's subdomain, e.g. "compute" for `compute.googleapis.com`. `version`: URL-versioning hint some paths embed directly; defaults to "v1" (informational — path is used as given). `httpMethod`: HTTP verb, defaults to "GET". `path`: request path, e.g. `"/compute/v1/projects/p/zones"` — appended to `https://`

{api}.googleapis.com as-is. params: query-string parameters. body: JSON-serialisable request body; sent with Content-Type: application/json when present.

Returns: A promise resolving to the decoded JSON response body ({} when the response body is empty).

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused). Also rejects if `api` or `path` is missing/empty, or if body is not JSON-serialisable.

```
const g = cloud.google({ project: "my-proj" });
const zones = await g.call({ api: "compute", path: "/compute/v1/projects/my-proj/zones" });
runtime.log(zones.items?.length ?? 0);
```

17.2.3.2 cloud.google.compute

Compute Engine — VM instances, zones, and disks (google.golang.org/api/compute/v1). Reached via `cloud.google({...}).compute()` each method below is called on that service handle.

17.2.3.2.1 cloud.google.compute.createInstance

```
createInstance(opts: { project: string; zone: string; instance: Record<string, unknown> }): Promise<Record<string, unknown>>
```

Create a Compute Engine VM instance.

Parameters

- `opts` ({ *project*: string; *zone*: string; *instance*: Record<string, unknown> }) — *project*: the GCP project id. *zone*: the target zone. *instance*: a Compute Engine Instance resource body (machineType, disks, networkInterfaces, etc. — see the Compute Engine instances.insert API reference).

Returns: A promise resolving to the (typically long-running) Compute instances.insert operation resource.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const op = await cloud.google({ project: "p" }).compute().createInstance({
  project: "p", zone: "europe-north1-a",
  instance: { name: "web-1", machineType: "zones/europe-north1-a/machineTypes/e2-micro" },
});
```

17.2.3.2.2 cloud.google.compute.deleteInstance

```
deleteInstance(opts: { project: string; zone: string; name: string }):
Promise<Record<string, unknown>>
```

Delete a Compute Engine VM instance.

Parameters

- `opts` (*{ project: string; zone: string; name: string }*) — project: the GCP project id. zone: the instance's zone. name: the instance name.

Returns: A promise resolving to the Compute instances.delete operation resource.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).compute().deleteInstance({ project: "p",
zone: "europe-north1-a", name: "web-1" });
```

17.2.3.2.3 cloud.google.compute.getInstance

```
getInstance(opts: { project: string; zone: string; name: string }):
Promise<Record<string, unknown>>
```

Get a Compute Engine VM instance's metadata.

Parameters

- `opts` (*{ project: string; zone: string; name: string }*) — project: the GCP project id. zone: the instance's zone. name: the instance name.

Returns: A promise resolving to the Compute instances.get response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
const inst = await cloud.google({ project: "p" }).compute().getInstance({ project:
"p", zone: "europe-north1-a", name: "web-1" });
```

17.2.3.2.4 cloud.google.compute.listDisks

```
listDisks(opts: { project: string; zone: string }): Promise<{ items?:
Array<Record<string, unknown>> }>
```

List Compute Engine persistent disks in a zone.

Parameters

- `opts` (*{ project: string; zone: string }*) — project: the GCP project id. zone: e.g. "europe-north1-a".

Returns: A promise resolving to the Compute disks.list response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project: "p" }).compute().listDisks({ project: "p",
zone: "europe-north1-a" });
```

17.2.3.2.5 cloud.google.compute.listInstances

```
listInstances(opts: { project: string; zone: string }): Promise<{ items?:  
Array<Record<string, unknown>> }>
```

List Compute Engine VM instances in a zone.

Parameters

- `opts` (*{ project: string; zone: string }*) — `project`: the GCP project id. `zone`: e.g. “europe-north1-a”.

Returns: A promise resolving to the `Compute instances.list` response.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project: "p" }).compute().listInstances({ project:  
"p", zone: "europe-north1-a" });
```

17.2.3.2.6 cloud.google.compute.listZones

```
listZones(opts: { project: string }): Promise<{ items?: Array<Record<string,  
unknown>> }>
```

List the Compute Engine zones available to a project.

Parameters

- `opts` (*{ project: string }*) — `project`: the GCP project id.

Returns: A promise resolving to the `Compute zones.list` response.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project: "p" }).compute().listZones({ project:  
"p" });
```

17.2.3.2.7 cloud.google.compute.startInstance

```
startInstance(opts: { project: string; zone: string; name: string }):  
Promise<Record<string, unknown>>
```

Start a stopped Compute Engine VM instance.

Parameters

- `opts` (*{ project: string; zone: string; name: string }*) — `project`: the GCP project id. `zone`: the instance’s zone. `name`: the instance name.

Returns: A promise resolving to the `Compute instances.start` operation resource.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).compute().startInstance({ project: "p", zone: "europe-north1-a", name: "web-1" });
```

17.2.3.2.8 cloud.google.compute.stopInstance

```
stopInstance(opts: { project: string; zone: string; name: string }): Promise<Record<string, unknown>>
```

Stop a running Compute Engine VM instance.

Parameters

- `opts` (*{ project: string; zone: string; name: string }*) — project: the GCP project id. zone: the instance’s zone. name: the instance name.

Returns: A promise resolving to the Compute instances.stop operation resource.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).compute().stopInstance({ project: "p", zone: "europe-north1-a", name: "web-1" });
```

17.2.3.3 cloud.google.iam

Cloud IAM — service accounts, their keys, and resource IAM policies (google.golang.org/api/iam/v1). Reached via `cloud.google({...}).iam()` each method below is called on that service handle.

17.2.3.3.1 cloud.google.iam.createKey

```
createKey(opts: { project: string; email: string }): Promise<Record<string, unknown>>
```

Create a new IAM key for a service account. The private key material is only ever returned by this call — store it immediately.

Parameters

- `opts` (*{ project: string; email: string }*) — project: the GCP project id. email: the service account’s email address.

Returns: A promise resolving to the IAM serviceAccounts.keys.create response, including the base64-encoded private key material.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const key = await cloud.google({ project: "p" }).iam().createKey({ project: "p",
email: "sa@p.iam.gserviceaccount.com" });
```

17.2.3.3.2 cloud.google.iam.createServiceAccount

```
createServiceAccount(opts: { project: string; accountId: string; displayName?:
string }): Promise<Record<string, unknown>>
```

Create an IAM service account.

Parameters

- `opts` (*{ project: string; accountId: string; displayName?: string }*) — `project`: the GCP project id. `accountId`: the service account id (the part before @ in its email). `displayName`: an optional human-readable name.

Returns: A promise resolving to the IAM serviceAccounts.create response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
const sa = await cloud.google({ project:
"p" }).iam().createServiceAccount({ project: "p", accountId: "my-sa", displayName:
"My SA" });
```

17.2.3.3.3 cloud.google.iam.deleteServiceAccount

```
deleteServiceAccount(opts: { project: string; email: string }):
Promise<Record<string, unknown>>
```

Delete an IAM service account.

Parameters

- `opts` (*{ project: string; email: string }*) — `project`: the GCP project id. `email`: the service account's email address.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).iam().deleteServiceAccount({ project: "p",
email: "sa@p.iam.gserviceaccount.com" });
```

17.2.3.3.4 cloud.google.iam.getIamPolicy

```
getIamPolicy(opts: { resource: string }): Promise<Record<string, unknown>>
```

Get the IAM policy attached to a resource (e.g. a service account).

Parameters

- `opts` (*{ resource: string }*) — resource: the full resource name, e.g. “projects/p/serviceAccounts/sa@p.iam.gserviceaccount.com”.

Returns: A promise resolving to the IAM `getIamPolicy` response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const policy = await cloud.google({ project: "p" }).iam().getIamPolicy({ resource:
"projects/p/serviceAccounts/sa@p.iam.gserviceaccount.com" });
```

17.2.3.3.5 cloud.google.iam.getServiceAccount

```
getServiceAccount(opts: { project: string; email: string }):
Promise<Record<string, unknown>>
```

Get an IAM service account’s metadata.

Parameters

- `opts` (*{ project: string; email: string }*) — project: the GCP project id. email: the service account’s email address.

Returns: A promise resolving to the IAM `serviceAccounts.get` response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const sa = await cloud.google({ project: "p" }).iam().getServiceAccount({ project:
"p", email: "sa@p.iam.gserviceaccount.com" });
```

17.2.3.3.6 cloud.google.iam.listKeys

```
listKeys(opts: { project: string; email: string }): Promise<{ keys?:
Array<Record<string, unknown>> }>
```

List a service account’s IAM keys.

Parameters

- `opts` (*{ project: string; email: string }*) — project: the GCP project id. email: the service account’s email address.

Returns: A promise resolving to the IAM `serviceAccounts.keys.list` response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project: "p" }).iam().listKeys({ project: "p",
email: "sa@p.iam.gserviceaccount.com" });
```

17.2.3.3.7 cloud.google.iam.listServiceAccounts

```
listServiceAccounts(opts: { project: string }): Promise<{ accounts?:  
Array<Record<string, unknown>> }>
```

List the IAM service accounts in a project.

Parameters

- `opts` (*{ project: string }*) — `project`: the GCP project id.

Returns: A promise resolving to the IAM `serviceAccounts.list` response.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project:  
"p" }).iam().listServiceAccounts({ project: "p" });
```

17.2.3.3.8 cloud.google.iam.setIamPolicy

```
setIamPolicy(opts: { resource: string; policy: Record<string, unknown> }):  
Promise<Record<string, unknown>>
```

Replace the IAM policy attached to a resource. This is a full replace, not a merge — read-modify-write via `getIamPolicy` first to avoid clobbering existing bindings.

Parameters

- `opts` (*{ resource: string; policy: Record<string, unknown> }*) — `resource`: the full resource name. `policy`: the complete IAM Policy body (`{ bindings: [{ role, members }], etag?, version? }`).

Returns: A promise resolving to the IAM `setIamPolicy` response (the policy as stored).

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const resource = "projects/p/serviceAccounts/sa@p.iam.gserviceaccount.com";  
const current = await cloud.google({ project:  
"p" }).iam().getIamPolicy({ resource });  
current.bindings = [...(current.bindings ?? []), { role: "roles/  
iam.serviceAccountUser", members: ["user:me@example.com"] }];  
await cloud.google({ project: "p" }).iam().setIamPolicy({ resource, policy:  
current });
```

17.2.3.4 cloud.google.secrets

Secret Manager — secret containers and their versioned values (google.golang.org/api/secretmanager/v1). Reached via `cloud.google({...}).secrets()` each method below is called on that service handle.

17.2.3.4.1 cloud.google.secrets.accessSecretVersion

```
accessSecretVersion(opts: { project: string; name: string; version?: string }):  
Promise<{ value: string }>
```

Access (decrypt) a secret version's plaintext value.

Parameters

- `opts` (*{ project: string; name: string; version?: string }*) — project: the GCP project id. name: the secret id. version: a version number as a string, or “latest”; defaults to “latest” when omitted.

Returns: A promise resolving to `{ value }` — the decoded plaintext secret value (already base64-decoded; never the raw wire-format base64).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const { value } = await cloud.google({ project:  
  "p" }).secrets().accessSecretVersion({ project: "p", name: "db-password" });  
runtime.log(value);
```

17.2.3.4.2 cloud.google.secrets.addSecretVersion

```
addSecretVersion(opts: { project: string; name: string; payload: string }):  
Promise<Record<string, unknown>>
```

Add a new version (value) to an existing secret. The payload is base64-encoded on the wire automatically.

Parameters

- `opts` (*{ project: string; name: string; payload: string }*) — project: the GCP project id. name: the secret id. payload: the plaintext secret value.

Returns: A promise resolving to the Secret Manager secrets.addVersion response.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).secrets().addSecretVersion({ project: "p",  
  name: "db-password", payload: "s3cr3t" });
```

17.2.3.4.3 cloud.google.secrets.createSecret

```
createSecret(opts: { project: string; name: string }): Promise<Record<string,  
  unknown>>
```

Create a secret container (automatic replication). Does not set a value — call addSecretVersion to add one.

Parameters

- `opts` (*{ project: string; name: string }*) — project: the GCP project id. name: the new secret's id.

Returns: A promise resolving to the Secret Manager `secrets.create` response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).secrets().createSecret({ project: "p", name:
"db-password" });
```

17.2.3.4.4 cloud.google.secrets.deleteSecret

```
deleteSecret(opts: { project: string; name: string }): Promise<Record<string,
unknown>>
```

Delete a secret and all of its versions.

Parameters

- `opts` (*{ project: string; name: string }*) — project: the GCP project id. name: the secret id.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).secrets().deleteSecret({ project: "p", name:
"db-password" });
```

17.2.3.4.5 cloud.google.secrets.getSecret

```
getSecret(opts: { project: string; name: string }): Promise<Record<string,
unknown>>
```

Get a secret's metadata (not its value — use `accessSecretVersion` for that).

Parameters

- `opts` (*{ project: string; name: string }*) — project: the GCP project id. name: the secret id.

Returns: A promise resolving to the Secret Manager `secrets.get` response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
const secret = await cloud.google({ project: "p" }).secrets().getSecret({ project:
"p", name: "db-password" });
```

17.2.3.4.6 cloud.google.secrets.listSecrets

```
listSecrets(opts: { project: string }): Promise<{ secrets?: Array<Record<string, unknown>> }>
```

List the secrets in a project (metadata only — not their values).

Parameters

- `opts` (*{ project: string }*) — project: the GCP project id.

Returns: A promise resolving to the Secret Manager secrets.list response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project: "p" }).secrets().listSecrets({ project: "p" });
```

17.2.3.5 cloud.google.storage

Cloud Storage — buckets and objects (google.golang.org/api/storage/v1). Reached via `cloud.google({...}).storage()` each method below is called on that service handle.

17.2.3.5.1 cloud.google.storage.createBucket

```
createBucket(opts: { project: string; bucket: string }): Promise<Record<string, unknown>>
```

Create a Cloud Storage bucket.

Parameters

- `opts` (*{ project: string; bucket: string }*) — project: owning GCP project id. bucket: the new bucket’s globally-unique name.

Returns: A promise resolving to the Storage buckets.insert response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).storage().createBucket({ project: "p", bucket: "my-new-bucket" });
```

17.2.3.5.2 cloud.google.storage.deleteBucket

```
deleteBucket(opts: { bucket: string }): Promise<Record<string, unknown>>
```

Delete a Cloud Storage bucket. The bucket must be empty.

Parameters

- `opts` (*{ bucket: string }*) — bucket: the bucket name.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).storage().deleteBucket({ bucket: "my-bucket" });
```

17.2.3.5.3 cloud.google.storage.deleteObject

```
deleteObject(opts: { bucket: string; key: string }): Promise<Record<string, unknown>>
```

Delete an object.

Parameters

- `opts` (*{ bucket: string; key: string }*) — bucket: the bucket name. key: the object’s key.

Returns: A promise resolving to {} on success.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).storage().deleteObject({ bucket: "my-bucket", key: "logs/a.txt" });
```

17.2.3.5.4 cloud.google.storage.getBucket

```
getBucket(opts: { bucket: string }): Promise<Record<string, unknown>>
```

Get a Cloud Storage bucket’s metadata.

Parameters

- `opts` (*{ bucket: string }*) — bucket: the bucket name (not the gs:

Returns: A promise resolving to the Storage buckets.get response.

Throws: Rejects with a structured Error { code, status, message, details } on API or transport failure. code/status are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const b = await cloud.google({ project: "p" }).storage().getBucket({ bucket: "my-bucket" });
```

17.2.3.5.5 cloud.google.storage.listBuckets

```
listBuckets(opts: { project: string }): Promise<{ items?: Array<Record<string, unknown>> }>
```

List the Cloud Storage buckets in a project.

Parameters

- `opts` (*{ project: string }*) — `project`: the GCP project id.

Returns: A promise resolving to the Storage `buckets.list` response.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const gcs = cloud.google({ project: "p" }).storage(); const r = await
gcs.listBuckets({ project: "p" });
```

17.2.3.5.6 cloud.google.storage.listObjects

```
listObjects(opts: { bucket: string; prefix?: string }): Promise<{ items?:
Array<Record<string, unknown>> }>
```

List objects in a bucket, optionally filtered by key prefix.

Parameters

- `opts` (*{ bucket: string; prefix?: string }*) — `bucket`: the bucket name. `prefix`: only list objects whose key starts with this string.

Returns: A promise resolving to the Storage `objects.list` response.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
const r = await cloud.google({ project: "p" }).storage().listObjects({ bucket:
"my-bucket", prefix: "logs/" });
```

17.2.3.5.7 cloud.google.storage.putObject

```
putObject(opts: { bucket: string; key: string; body: string | Uint8Array |
ArrayBuffer }): Promise<Record<string, unknown>>
```

Upload/overwrite an object’s content.

Parameters

- `opts` (*{ bucket: string; key: string; body: string | Uint8Array | ArrayBuffer }*) — `bucket`: the bucket name. `key`: the object’s key. `body`: a string (encoded as UTF-8) or raw bytes (`Uint8Array/ArrayBuffer`) to upload as the object’s content.

Returns: A promise resolving to the Storage `objects.insert` response.

Throws: Rejects with a structured Error { `code`, `status`, `message`, `details` } on API or transport failure. `code/status` are 0/“TRANSPORT” for non-API errors (DNS, TLS, timeout, connection refused).

```
await cloud.google({ project: "p" }).storage().putObject({ bucket: "my-bucket",
key: "logs/a.txt", body: "hello" });
```

17.2.3.5.8 cloud.google.storage.readObject

```
readObject(opts: { bucket: string; key: string }): Promise<{ bytes: number[] }>
```

Download an object's content.

Parameters

- `opts` (*{ bucket: string; key: string }*) — bucket: the bucket name. key: the object's key.

Returns: A promise resolving to `{ bytes }` where `bytes` is a plain JS `number[]` (byte-value array), NOT a real `Uint8Array` — wrap it with `new Uint8Array(res.bytes)` before treating it as binary data (e.g. before `fs.writeBytes` or further decoding).

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
const res = await cloud.google({ project: "p" }).storage().readObject({ bucket:
"my-bucket", key: "logs/a.txt" });
const bytes = new Uint8Array(res.bytes);
runtime.log(bytes.length);
```

17.2.3.5.9 cloud.google.storage.statObject

```
statObject(opts: { bucket: string; key: string }): Promise<Record<string,
unknown>>
```

Get an object's metadata without downloading its content.

Parameters

- `opts` (*{ bucket: string; key: string }*) — bucket: the bucket name. key: the object's key (path within the bucket).

Returns: A promise resolving to the Storage objects.get response.

Throws: Rejects with a structured Error `{ code, status, message, details }` on API or transport failure. code/status are 0/"TRANSPORT" for non-API errors (DNS, TLS, timeout, connection refused).

```
const meta = await cloud.google({ project: "p" }).storage().statObject({ bucket:
"my-bucket", key: "logs/a.txt" });
```

17.3 codec

Binary-format codecs: compression, barcodes, check digits.

17.3.1 codec.barcode

17.3.1.1 codec.barcode.decodableFormats

```
decodableFormats(): string[]
```

Available decode formats (qr / datamatrix / aztec / code128 / code39 / code93 / codabar / ean13 / ean8 / upca / upce / itf). PDF417 is encode-only.

Returns: string[] — the twelve symbology names barcode.decode can recognise. PDF417 is absent (gozxing has no PDF417 decoder).

Throws: Never throws.

```
const fmts = codec.barcode.decodableFormats();
```

17.3.1.2 codec.barcode.decode

```
decode(data: string | Uint8Array | ArrayBuffer, format?: string):  
Promise<{ format: string, text: string }>
```

Decode a PNG/JPEG/WebP image to { format, text } via gozxing. Optional format hint skips the auto-detect walk. EAN/UPC need a quiet zone in the input. Async.

Parameters

- `data` (*string* / *Uint8Array* / *ArrayBuffer*) — Image bytes (PNG / JPEG / WebP). A string is treated as its raw UTF-8 bytes.
- `format` (*string*, *optional*) — Symbology hint (case-insensitive) from `decodableFormats`. When given, only that reader runs; otherwise every decoder is tried in priority order and the first hit wins.

Returns: Promise<{ format: string, text: string }> — the detected symbology name and the decoded payload.

Throws: Throws if data is missing/empty or not a string/ArrayBuffer/Uint8Array, the image cannot be decoded (including exceeding the shared hard 64-megapixel decode-bomb cap — see `image.decode`), the format hint is unsupported, or no barcode is recognised.

```
const { format, text } = await codec.barcode.decode(png);
```

17.3.1.3 codec.barcode.encode

```
encode(format: string, data: string, opts?: { width?: number, height?: number,  
quietZone?: boolean | number }): Promise<Uint8Array>
```

Render data into a PNG of the chosen format. `opts.width` / `opts.height` default to 256x256 (2D) or 400x120 (1D). `opts.quietZone` (true or px count) pads a white margin — required for EAN/UPC to decode. Async.

Parameters

- `format` (*string*) — Symbology (case-insensitive): qr / datamatrix / aztec / pdf417 / code128 / code39 / codabar / ean13 / ean8 / upca.
- `data` (*string*) — Payload to encode. EAN/UPC require the exact digit count for the variant; the encoder validates content per symbology.
- `opts` (*{ width?: number, height?: number, quietZone?: boolean | number }, optional*) — `width` / `height` set the output pixel dimensions (default 256x256 for 2D qr/datamatrix/aztec, 400x120 otherwise). `quietZone` pads a white margin: true uses 10% of width (min 10px), a number uses that many pixels per side, false/0/absent adds none. EAN/UPC need a quiet zone to be decodable.

Returns: Promise<Uint8Array> — PNG image bytes.

Throws: Throws if the format is unknown, the data is invalid for that symbology, or scaling / PNG encoding fails.

```
const png = await codec.barcode.encode("qr", "https://example.com", { width: 320, height: 320 });
```

17.3.1.4 codec.barcode.formats

```
formats(): string[]
```

Available encode formats (qr / datamatrix / aztec / pdf417 / code128 / code39 / codabar / ean13 / ean8 / upca).

Returns: string[] — the ten symbology names accepted by barcode.encode.

Throws: Never throws.

```
const fmts = codec.barcode.formats(); // ["qr", "datamatrix", ...]
```

17.3.2 codec.checkdigit

17.3.2.1 codec.checkdigit.algos

```
algos(): string[]
```

Supported algorithms (luhn / isbn10 / isbn13 / ean13 / ean8 / upca).

Returns: string[] — the six supported algorithm names. isbn13 is an alias for ean13 (same check-digit math).

Throws: Never throws.

```
const algos = codec.checkdigit.algos();
```

17.3.2.2 codec.checkdigit.compute

```
compute(algo: string, partial: string): string
```

Compute the missing trailing check digit for a partial input.

Parameters

- `algo` (*string*) — Algorithm name (case-insensitive, trimmed): luhn / isbn10 / isbn13 / ean13 / ean8 / upca.
- `partial` (*string*) — The number WITHOUT its check digit (whitespace trimmed). Fixed-length algorithms expect exactly length-1 digits (e.g. 12 for ean13, 9 for isbn10).

Returns: string — the single check digit ('0'–'9', or 'X' for isbn10 when the value is 10).

Throws: Throws if the algorithm is unknown, the input is empty / the wrong length, or contains a non-digit.

```
const cd = codec.checkdigit.compute("ean13", "123456789012"); // "8"
```

17.3.2.3 codec.checkdigit.inspect

```
inspect(algo: string, input: string): { algo: string, input: string, valid: boolean, given: string, computed: string }
```

Diagnostic combining validate + compute: { valid, given, computed, ... }.

Parameters

- `algo` (*string*) — Algorithm name (case-insensitive, trimmed): luhn / isbn10 / isbn13 / ean13 / ean8 / upca.
- `input` (*string*) — The full number including its trailing check digit (whitespace trimmed).

Returns: { algo, input, valid, given, computed } — algo/input echo the normalised arguments; given is the input's last character; computed is the recalculated check digit (empty when the input is too short or malformed to split); valid is true when given equals computed (case-insensitive).

Throws: Never throws; malformed input yields valid:false with an empty computed.

```
const r = codec.checkdigit.inspect("ean13", "1234567890128");  
// { algo: "ean13", input: "...", valid: true, given: "8", computed: "8" }
```

17.3.2.4 codec.checkdigit.validate

```
validate(algo: string, input: string): boolean
```

Return whether the input passes the named algorithm's check digit.

Parameters

- `algo` (*string*) — Algorithm name (case-insensitive, trimmed): luhn / isbn10 / isbn13 / ean13 / ean8 / upca.
- `input` (*string*) — The full number including its trailing check digit (whitespace trimmed). ISBN-10 may end in 'X'.

Returns: boolean — true when the input's check digit is valid for the algorithm. Returns false (does not throw) for wrong length, non-digit characters, or an unknown algorithm.

Throws: Never throws; any failure (unknown algorithm included) returns false.

```
const ok = codec.checkdigit.validate("luhn", "4539578763621486"); // true
```

17.3.3 codec.compression

17.3.3.1 codec.compression.algos

```
algos(): string[]
```

Available compression algorithm names (gzip / deflate / zlib / bzip2 / zstd / brotli / lz4 / xz / snappy).

Returns: `string[]` — the nine supported algorithm names, lowercase, in registration order.

Throws: Never throws.

```
const algos = codec.compression.algos(); // ["gzip", "deflate", ...]
```

17.3.3.2 `codec.compression.compress`

```
compress(algo: string, data: string | Uint8Array | ArrayBuffer):  
Promise<Uint8Array>
```

Compress data with the named algorithm. Returns `Uint8Array`. Async.

Parameters

- `algo` (*string*) — Algorithm name (case-insensitive): `gzip` / `deflate` / `zlib` / `bzip2` / `zstd` / `brotli` / `lz4` / `xz` / `snappy`.
- `data` (*string* / *Uint8Array* / *ArrayBuffer*) — Input bytes. Strings are interpreted as their UTF-8 byte sequence.

Returns: `Promise<Uint8Array>` — the compressed bytes.

Throws: Throws if data is undefined/null or an unsupported type, the algorithm name is unknown, or the underlying compressor errors.

```
const packed = await codec.compression.compress("gzip", "hello world");
```

17.3.3.3 `codec.compression.decompress`

```
decompress(algo: string, data: string | Uint8Array | ArrayBuffer, opts?:  
{ maxBytes?: number }): Promise<Uint8Array>
```

Decompress data previously produced by `compress` (same algorithm name required). Returns `Uint8Array`. Async.

Parameters

- `algo` (*string*) — Algorithm name (case-insensitive), matching the one used to compress: `gzip` / `deflate` / `zlib` / `bzip2` / `zstd` / `brotli` / `lz4` / `xz` / `snappy`.
- `data` (*string* / *Uint8Array* / *ArrayBuffer*) — Compressed input bytes.
- `opts` (*{ maxBytes?: number }, optional*) — `maxBytes` caps the decompressed output size in bytes (default 512 MB); a non-positive value also falls back to the default. Guards against a decompression bomb — a small crafted input that inflates to gigabytes.

Returns: `Promise<Uint8Array>` — the original decompressed bytes.

Throws: Throws if data is undefined/null or an unsupported type, the algorithm name is unknown, the input is not valid for that algorithm, or the decompressed output exceeds `maxBytes`.

```
const raw = await codec.compression.decompress("gzip", packed);  
const text = new TextDecoder().decode(raw);  
// cap output at 1 MB for untrusted input:
```

```
const capped = await codec.compression.decompress("gzip", packed, { maxBytes: 1 << 20 });
```

17.3.4 codec.doc

17.3.4.1 codec.doc.formats

```
formats(): { [format: string]: { read: boolean; write: boolean } }
```

Report the read/write capability of every document format codec.doc supports. Read-only formats (pdf/doc) have write:false.

Returns: An object keyed by format name (pdf/docx/doc/rtf/odt), each with read and write booleans.

Throws: Does not throw.

```
if (!codec.doc.formats().pdf.write) console.log("pdf is read-only");
```

17.3.4.2 codec.doc.read

```
read(src: string | Uint8Array, opts?: { format?: "pdf" | "docx" | "doc" | "rtf" | "odt" }): { format: string; text: string; paragraphs: string[] }
```

Extract text from a document: PDF, DOCX, DOC (Word 97–2003), RTF, or ODT. Returns { format, text, paragraphs }. Format is auto-detected by content (%PDF, {\rtf, OLE2 magic, or the zip's word/document.xml / opendocument.text mimetype) and extension, unless opts.format is given. Extraction is best-effort (extraction-grade): text is the full plain text; paragraphs is the block breakdown (native for docx/odt/rtf, best-effort for pdf/doc).

Parameters

- `src` (*string* / *Uint8Array*) — The document: a file path or raw bytes.
- `opts` (*{ format?: "pdf" | "docx" | "doc" | "rtf" | "odt" }, optional*) — Force the format instead of auto-detecting.

Returns: A document model: format is the detected/forced format; text is the full extracted text; paragraphs is the block breakdown.

Throws: Throws if the document cannot be parsed, if the format is unrecognized, or if the source is neither a path string nor a Uint8Array.

```
const d = codec.doc.read("report.pdf"); runtime.log(d.paragraphs.length, "paragraphs");
```

17.3.4.3 codec.doc.write

```
write(model: { paragraphs?: string[]; text?: string } | string, opts?: { format?: "docx" | "rtf" | "odt"; dest?: string }): { bytes: Uint8Array } | { path: string }
```

Write a document to DOCX, RTF, or ODT from { paragraphs } / { text } / a bare string (one paragraph per line). PDF and DOC are read-only and throw. Without opts.dest the encoded bytes are returned; with dest they are written there.

Parameters

- `model` (*{ paragraphs?: string[]; text?: string } | string*) — The content: a paragraph array, a text blob, or a bare string.
- `opts` (*{ format?: "docx" | "rtf" | "odt"; dest?: string }, optional*) — format: target format; dest: write to this path instead of returning bytes.

Returns: An object with the encoded bytes (`{ bytes }`), or `{ path }` when `opts.dest` was given.

Throws: Throws if the format is missing/unsupported, if writing a read-only format (pdf/doc), if the model is malformed, or on a write failure.

```
const out = codec.doc.write({ paragraphs: ["Hello", "World"] }, { format: "docx" });
```

17.3.5 codec.dotenv

17.3.5.1 codec.dotenv.parse

```
parse(text: string): { [key: string]: string }
```

Parse dotenv-format text into an object. Handles KEY=VALUE lines, # comments, blank lines, an optional leading `export`, and surrounding single/double quotes; no shell expansion. A later duplicate key overrides an earlier one. Pure — does not touch the environment.

Parameters

- `text` (*string*) — The .env-format text to parse.

Returns: An object of the parsed key/value pairs.

Throws: Throws on a line without `=` or with an empty key ("line N: ..."); `TypeError` if text is not a string.

```
const cfg = codec.dotenv.parse('PORT=8080\n# note\nHOST="0.0.0.0"');
```

17.3.5.2 codec.dotenv.stringify

```
stringify(obj: { [key: string]: string | number | boolean }): string
```

Serialize an object of string/number/boolean values to dotenv-format text (one KEY=VALUE line per entry, keys sorted). Values are double-quoted when needed so the result round-trips through `codec.dotenv.parse`: `parse(stringify(obj))` deep-equals `obj` (numbers/booleans become their string forms).

Parameters

- `obj` (*{ [key: string]: string | number | boolean }*) — The values to serialize.

Returns: The .env-format text.

Throws: Throws if a value contains a newline (not representable), if a key is empty or contains `=`/whitespace/newline, or if a value is not a string/number/boolean; `TypeError` if `obj` is not an object.


```
const text = codec.dotenv.stringify({ PORT: 8080, HOST: "0.0.0.0", DEBUG: true });
```

17.3.6 codec.perl

17.3.6.1 codec.perl.dumper

```
dumper(value: unknown, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): string
```

Perl Data::Dumper-style dump (\$VAR1 = ... ;), normalized indentation. JS booleans emit the JSON::XS::Boolean blessed-ref form (opts.perlBoolClass).

Parameters

- `value` (*unknown*) — Any JSON-like value (see `php.serialize`). Arrays/objects emit as Perl array/hash refs; class-key objects emit as blessed refs.
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `perlBoolClass` names the blessed class emitted for JS booleans (default “JSON::XS::Boolean”). `indent` overrides the indentation step. `classKey` overrides the class-name sentinel (default “__class”).

Returns: string — a Data::Dumper-style dump (\$VAR1 = ... ;) with normalized indentation.

Throws: Throws on a circular reference or an unsupported value type.

```
const d = codec.perl.dumper({ ok: true });
```

17.3.6.2 codec.perl.parseDumper

```
parseDumper(input: string, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): unknown
```

Read Data::Dumper output back. Blessed scalar refs in the JSON bool family decode to booleans; bare 1/0 stay numbers; cycles throw.

Parameters

- `input` (*string*) — Data::Dumper output to parse back (a \$VARn = ... ; assignment or a bare value).
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `classKey` sets the sentinel property used to tag decoded blessed refs (default “__class”). The JSON bool family (JSON::XS::Boolean, JSON::PP::Boolean, Types::Serialiser::Boolean) decodes to JS booleans regardless.

Returns: unknown — the decoded value. Blessed scalar refs in the JSON-bool family become JS booleans; bare 1/0 stay numbers; other blessed refs carry the `classKey` sentinel.

Throws: Throws on malformed input or a reference that would close a cycle.

```
const v = codec.perl.parseDumper("$VAR1 = [1, 2];"); // [1, 2]
```

17.3.7 codec.php

17.3.7.1 codec.php.parseVarDump

```
parseVarDump(input: string, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): unknown
```

Best-effort read of `var_dump()` output. Throws on lossy markers (*RECURSION*, truncation, visibility-annotated props).

Parameters

- `input` (*string*) — PHP `var_dump()` output to parse back.
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `classKey` sets the sentinel property used to tag decoded objects (default “`__class`”).

Returns: `unknown` — the reconstructed value (best-effort; `var_dump` is a lossy format).

Throws: Throws on lossy markers it cannot faithfully reverse: *RECURSION*, truncation, or visibility-annotated (private/protected) properties.

```
const v = codec.php.parseVarDump('int(42)'); // 42
```

17.3.7.2 codec.php.parseVarExport

```
parseVarExport(input: string, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): unknown
```

Read a `var_export()` literal (arrays, scalars, `NULL`, `\Cls::__set_state`) back to a value.

Parameters

- `input` (*string*) — A PHP `var_export()` literal: `array(...)`, scalars, `NULL`, or `\Cls::__set_state(array(...))`.
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `classKey` sets the sentinel property used to tag decoded `__set_state` objects (default “`__class`”).

Returns: `unknown` — the decoded value; `\Cls::__set_state` objects become plain objects carrying the `classKey` sentinel.

Throws: Throws on input that is not a parseable `var_export()` literal.

```
const v = codec.php.parseVarExport("array (\n 0 => 1,\n)");
```

17.3.7.3 codec.php.serialize

```
serialize(value: unknown, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): string
```

PHP `serialize()`: encode a value to PHP’s canonical serialization string. Objects use the `__class` sentinel; cycles throw.

Parameters

- `value` (*unknown*) — Any JSON-like value: null, boolean, number, string, array, or plain object. An object carrying the class-key sentinel (`opts.classKey`, default “`__class`”) encodes as a PHP object (O:).
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `classKey` overrides the class-name sentinel property (default “`__class`”). `indent` / `perlBoolClass` are unused by `serialize`.

Returns: string — the PHP `serialize()` string (e.g. `a:1:{...}`).

Throws: Throws on a circular reference or an unsupported value type (function, Symbol, BigInt, Date, Map, Set, RegExp).

```
const s = codec.php.serialize({ a: 1, b: [2, 3] });
```

17.3.7.4 codec.php.unserialize

```
unserialize(input: string, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): unknown
```

PHP `unserialize()`: decode a `serialize()` string back to a value. `r:/R`: references resolve to shared objects (DAGs); cycles throw.

Parameters

- `input` (*string*) — A PHP `serialize()` string.
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `classKey` sets the sentinel property used to tag decoded PHP objects (default “`__class`”).

Returns: unknown — the decoded value. PHP objects become plain objects carrying the `classKey` sentinel; `r:/R`: references rebuild as shared object identities (DAGs).

Throws: Throws on malformed input or a reference that would close a cycle.

```
const v = codec.php.unserialize('a:2:{i:0;i:1;i:1;i:2;}'); // [1, 2]
```

17.3.7.5 codec.php.varDump

```
varDump(value: unknown, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): string
```

PHP `var_dump()`: human-readable debug output. String lengths are byte counts.

Parameters

- `value` (*unknown*) — Any JSON-like value (see `php.serialize`).
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `indent` overrides the default indentation step. `classKey` overrides the class-name sentinel (default “`__class`”).

Returns: string — `var_dump()`-style output. String lengths in the output are byte counts, matching PHP.

Throws: Throws on a circular reference or an unsupported value type.

```
const dump = codec.php.varDump({ name: "ok" });
```

17.3.7.6 codec.php.varExport

```
varExport(value: unknown, opts?: { classKey?: string, perlBoolClass?: string, indent?: string }): string
```

PHP `var_export()`: emit valid PHP code for a value. `opts.indent` overrides the 2-space step.

Parameters

- `value` (*unknown*) — Any JSON-like value (see `php.serialize`). Objects with the class-key sentinel emit as `\Cls::__set_state(...)`.
- `opts` (*{ classKey?: string, perlBoolClass?: string, indent?: string }, optional*) — `indent` overrides the default 2-space indentation step. `classKey` overrides the class-name sentinel (default “`__class`”).

Returns: string — valid PHP source, the kind `var_export()` prints.

Throws: Throws on a circular reference or an unsupported value type.

```
const code = codec.php.varExport({ x: 1 }, { indent: "  " });
```

17.3.8 codec.sheet

17.3.8.1 codec.sheet.formats

```
formats(): { [format: string]: { read: boolean; write: boolean } }
```

Report the read/write capability of every spreadsheet format `codec.sheet` supports. Read-only formats (xls/slks/dif) have `write:false`.

Returns: An object keyed by format name (csv/tsv/xlsx/ods/xls/slks/dif), each with read and write booleans.

Throws: Does not throw.

```
const f = codec.sheet.formats(); if (!f.xls.write) console.log("xls is read-only");
```

17.3.8.2 codec.sheet.read

```
read(src: string | Uint8Array, opts?: { format?: "csv" | "tsv" | "xlsx" | "ods" | "xls" | "slk" | "dif" }): { format: string; sheets: { name: string; rows: (string | number | boolean | null)[][] }[] }
```

Read tabular data (CSV/TSV/XLSX/ODS) into a workbook model: `{ format, sheets:[{ name, rows }] }`. Cells are typed primitives — XLSX and ODS numbers/booleans come back as `number/boolean` (empty → `null`); ODS dates come back as ISO-8601 strings; CSV/TSV cells are always strings (CSV is untyped). Format is sniffed (XLSX/ODS by their ZIP magic, else CSV; a `.tsv` path is read as TSV) unless `opts.format` is given. Also reads legacy formats **read-only** — XLS (Excel 97–2003), SYLK (.slk), and DIF (.dif) — for extracting data to convert up; XLS

cells come back as the library's formatted strings (extraction-grade). Detection: OLE2 magic (\xD0\xCF)→xls, ID; →slk, TABLE header→dif, plus .xls/.slk/.dif extensions and opts.format.

Parameters

- `src` (*string* | *Uint8Array*) — A file path or the encoded bytes (csv/tsv/xlsx/ods/xls/slkl/dif).
- `opts` (*{ format?: "csv" | "tsv" | "xlsx" | "ods" | "xls" | "slk" | "dif" }, optional*) — Override the auto-detected format (e.g. force slk for a SYLK file with no extension).

Returns: A workbook: format echoes the detected/forced format; sheets has one entry for CSV/TSV (name from the file basename) or all sheets for XLSX/ODS/XLS, each with a rows grid of typed cells.

Throws: Throws if `src` is neither a path string nor *Uint8Array*, if the file can't be read, if the format is unsupported, or if the bytes can't be parsed.

```
const wb = codec.sheet.read("data.ods");
runtime.log(wb.sheets[0].name, wb.sheets[0].rows.length);
```

17.3.8.3 codec.sheet.write

```
write(model: { sheets: { name?: string; rows: (string | number | boolean | null)[] }[] } | (string | number | boolean | null)[][], opts: { format: "csv" | "tsv" | "xlsx" | "ods"; dest?: string }): { format: string; bytes?: Uint8Array; path?: string }
```

Write a workbook to CSV/TSV/XLSX/ODS. Pass `{ sheets:[{ name?, rows }] }` or a bare 2D array (one sheet). XLSX and ODS preserve types (number→numeric cell, boolean→bool, string→text, null→empty) and all sheets; ODS writes values+types only (no styles or formulas); CSV/TSV stringify cells and support a single sheet only (>1 throws). Without `opts.dest` the encoded bytes are returned; with `dest` they're written there.

Parameters

- `model` (*{ sheets: { name?: string; rows: (string | number | boolean | null)[] }[] } | (string | number | boolean | null)[][]*) — The workbook (`{ sheets }` with optional names) or a bare 2D array of cells (becomes a single sheet).
- `opts` (*{ format: "csv" | "tsv" | "xlsx" | "ods"; dest?: string }*) — format selects the writer (or is inferred from a `dest` extension); `dest`, when set, writes the file and returns its path instead of bytes.

Returns: `{ format, bytes }` with the encoded workbook, or `{ format, path }` when `opts.dest` is set.

Throws: Throws if the `model` isn't an object-with-sheets or a 2D array, if format is missing/unsupported, if a CSV/TSV write has more than one sheet, if the format is read-only (xls/slkl/dif throws "<fmt> is read-only (extract/convert only)"), or on an encode/write failure.

```
const out = codec.sheet.write([["a", 1, true]], { format: "ods" });
runtime.log(out.format, out.bytes.length);
```

17.3.9 codec.toml

17.3.9.1 codec.toml.parse

```
parse(text: string): Record<string, unknown>
```

Parse a TOML document string into a JS object. Tables become nested objects, arrays become arrays, and TOML integers/floats/booleans/strings map to the corresponding JS types (datetimes come back as strings).

Parameters

- `text` (*string*) — The TOML document text.

Returns: The parsed document as a plain object.

Throws: Throws (“codec.toml.parse: ...”) on malformed TOML.

```
const cfg = codec.toml.parse('port = 8080\n[db]\nhost = "localhost"');
```

17.3.9.2 codec.toml.stringify

```
stringify(value: Record<string, unknown>): string
```

Serialize a JS value to a TOML document string. The top-level value must be an object (TOML documents are tables); nested objects become tables, arrays become TOML arrays.

Parameters

- `value` (*Record<string, unknown>*) — An object to serialize as a TOML table.

Returns: The TOML document text.

Throws: Throws (“codec.toml.stringify: ...”) if the value can’t be represented as TOML (e.g. a non-object top level).

```
const text = codec.toml.stringify({ port: 8080, db: { host: "localhost" } });
```

17.3.10 codec.xml

17.3.10.1 codec.xml.decode

```
decode(xml: string): unknown
```

Parse an XML string to a value using the same @-prefix + #text convention as `xml.encode`. Attributes become @-keys, text becomes #text (or a bare string for a text-only element), child elements become keys, and repeated same-name siblings become an array. Empty/self-closing elements decode to null. Namespace prefixes are kept literally; all values are strings (no type coercion). Mismatched tags, multiple roots, and malformed XML throw.

Parameters

- `xml` (*string*) — The XML document to parse.

Returns: A single-key object whose key is the root element name and whose value is the parsed content (key order follows document order; all leaf values are strings).

Throws: Throws on malformed XML, mismatched/mis-nested end tags, multiple root elements, or no root element.

```
const v = codec.xml.decode("<note id=\"5\">hi</note>");  
// { note: { "@id": "5", "#text": "hi" } }
```

17.3.10.2 codec.xml.encode

```
encode(value: unknown, opts?: { rootName?: string, indent?: string, declaration?:  
boolean }): string
```

Serialize a value to an XML string. Convention: @-prefixed keys are attributes, #text is element text, other keys are child elements, and an array value becomes repeated sibling elements (a scalar key becomes a text-only element, null a self-closing tag). The value must be a single-key object naming the root element, or pass opts.rootName to wrap it. Scalars are stringified; object key order is preserved.

Parameters

- `value` (*unknown*) — A single-key object whose one key names the root element — e.g. `{ note: { "@id": "5", "#text": "hi", to: "alice" } }` → `<note id="5">hi<to>alice</to></note>`. Or any value plus opts.rootName to wrap it. Cycles throw.
- `opts` (`{ rootName?: string, indent?: string, declaration?: boolean }`, optional) — rootName wraps the value under that root element. indent pretty-prints with the given unit per level (default compact). declaration prepends `<?xml version="1.0" encoding="UTF-8"?>` (default off).

Returns: The XML string.

Throws: Throws if the value has no single root element and no opts.rootName, if the root content is an array, if a non-scalar is used as an attribute or #text value, or if the value contains a cycle.

```
const xml = codec.xml.encode({ note: { "@id": "5", "#text": "hi" } });  
// <note id="5">hi</note>
```

17.3.11 codec.yaml

17.3.11.1 codec.yaml.parse

```
parse(text: string): unknown
```

Parse a YAML document string into a JS value. Mappings become objects, sequences become arrays, and scalars map to the matching JS type (int/float/boolean/string/null). Only the first document of a multi-document (“—”-separated) stream is parsed; non-string mapping keys are coerced to their string form.

Parameters

- `text` (*string*) — The YAML document text.

Returns: The parsed value — a plain object for a mapping, an array for a sequence, or a primitive for a scalar document.

Throws: Throws (“codec.yaml.parse: ...”) on malformed YAML.

```
const cfg = codec.yaml.parse('port: 8080\n db: localhost');
```

17.3.11.2 codec.yaml.stringify

```
stringify(value: unknown): string
```

Serialize a JS value to a YAML document string. Objects become mappings, arrays become sequences, and primitives become scalars. The value round-trips through `codec.yaml.parse`.

Parameters

- `value` (*unknown*) — The value to serialize as YAML (typically an object).

Returns: The YAML document text (terminated with a trailing newline).

Throws: Throws (“codec.yaml.stringify: ...”) if the value can’t be represented as YAML.

```
const text = codec.yaml.stringify({ port: 8080, db: { host: "localhost" } });
```

17.4 console

Browser/Node-style console shim: `log/info/debug` to `stdout`, `warn/error` to `stderr`. For porting scripts; `runtime.log` is the native equivalent.

17.4.1 console.debug

```
debug(...args: unknown[]): void
```

Alias of `console.log` — stringified arguments, space-joined, to `stdout`.

Parameters

- `args` (*...unknown[]*) — Values to print; identical formatting and `stdout` destination as `console.log`.

Returns: `void` — output is written to `stdout` as a side effect.

Throws: Never throws; values JSON cannot serialise degrade to their `String()` form.

```
console.debug("cache hit", key);
```

17.4.2 console.error

```
error(...args: unknown[]): void
```

Like `console.log` but writes to `stderr`.

Parameters

- `args` (*...unknown[]*) — Values to print; same space-joined / JSON formatting as `console.log` but routed to `stderr`.

Returns: `void` — output is written to `stderr` as a side effect.

Throws: Never throws; values JSON cannot serialise degrade to their `String()` form.


```
console.error("request failed", { status: 500 });
```

17.4.3 console.info

```
info(...args: unknown[]): void
```

Alias of `console.log` — stringified arguments, space-joined, to `stdout`.

Parameters

- `args (...unknown[])` — Values to print; identical formatting and `stdout` destination as `console.log`.

Returns: `void` — output is written to `stdout` as a side effect.

Throws: Never throws; values JSON cannot serialise degrade to their `String()` form.

```
console.info("listening on", 8080);
```

17.4.4 console.log

```
log(...args: unknown[]): void
```

Print a space-joined line of the arguments to `stdout`. Primitives print raw; objects/arrays render as JSON. Browser/Node-compatible; same output as `runtime.log`.

Parameters

- `args (...unknown[])` — Values to print, joined by single spaces and terminated with a newline. Primitives (string/number/boolean/null/undefined) print raw; objects and arrays render as JSON via `JSON.stringify`, falling back to `String()` for functions and circular references.

Returns: `void` — output is written to `stdout` as a side effect.

Throws: Never throws; values JSON cannot serialise degrade to their `String()` form.

```
console.log("user", { id: 1, name: "ada" }); // user {"id":1,"name":"ada"}
```

17.4.5 console.table

```
table(data: unknown, columns?: string[]): void
```

Render tabular data as an aligned, bordered table on `stdout` (Node/Bun/Deno parity). Accepts an array of objects (rows), an array of primitives, or an object of objects/primitives. Prints a leading (index) column, one column per property (union of keys across rows, first-seen order), and a Values column for primitive rows. Non-tabular input (a primitive) falls back to `console.log`-style output without throwing.

Parameters

- `data (unknown)` — The rows to tabulate: an array (indices become the (index) column) or an object (keys become the (index) column). Cells are formatted like `console.log` — primitives raw, objects/arrays as compact JSON (strings are not quoted).

- `columns` (*string[], optional*) — Restrict and order the property columns to exactly these names; an absent column renders as an empty column. The (index) column is always shown.

Returns: void — the table is written to stdout as a side effect.

Throws: Never throws; non-tabular input degrades to a console.log-style line.

```
console.table([ { name: "web", status: "ok" }, { name: "db", status: "down" } ]);
```

17.4.6 console.warn

```
warn(...args: unknown[]): void
```

Like console.log but writes to stderr.

Parameters

- `args` (*...unknown[]*) — Values to print; same space-joined / JSON formatting as console.log but routed to stderr.

Returns: void — output is written to stderr as a side effect.

Throws: Never throws; values JSON cannot serialise degrade to their String() form.

```
console.warn("retrying in", 5, "seconds");
```

17.5 crypto

Hashing, JWT, age encryption — anything that produces a digest, signature, or ciphertext.

17.5.1 crypto.encrypt

17.5.1.1 crypto.encrypt.decrypt

```
decrypt(ciphertext: string | Uint8Array | ArrayBuffer, identities: string | string[]): Uint8Array
```

Open a payload with one of the supplied identities. Routes to age or PGP based on the identity / ciphertext format. age: binary or armored auto-detected. Wrong identity throws.

Parameters

- `ciphertext` (*string | Uint8Array | ArrayBuffer*) — The encrypted payload; age armor and PGP armor are auto-detected.
- `identities` (*string | string[]*) — One or more private keys. AGE-SECRET-KEY-1... identities use the age backend; —BEGIN PGP PRIVATE KEY BLOCK— entries (or an armored PGP message) use the PGP backend.

Returns: Uint8Array — the decrypted plaintext bytes (decode with new TextDecoder().decode(...) for a string).

Throws: Throws if ciphertext is empty or an unsupported type, no identities are given, an identity is actually a public key, or no supplied identity matches the ciphertext's recipients.

```
const pt = crypto.encrypt.decrypt(ct, privateKey);
const text = new TextDecoder().decode(pt);
```

17.5.1.2 crypto.encrypt.detectBackend

```
detectBackend(input: string): { backend: string; kind?: string }
```

Classify a recipient / identity string. Returns { backend: 'age'|'pgp'|'unknown', kind?: 'public'|'private' }. Pure prefix matching; no parsing or I/O.

Parameters

- `input` (*string*) — A recipient or identity string to classify (age bech32, age SSH public key, or PGP armored block).

Returns: { backend: "age" | "pgp" | "unknown", kind?: "public" | "private" } — kind is present only when the backend was identified.

Throws: Never throws; unrecognised input returns { backend: "unknown" }.

```
const info = crypto.encrypt.detectBackend("age1abc..."); // { backend: "age",
kind: "public" }
```

17.5.1.3 crypto.encrypt.encrypt

```
encrypt(data: string | Uint8Array | ArrayBuffer, recipients: string | string[],
opts?: { armored?: boolean }): Uint8Array
```

Seal data to recipients. age public keys (age1...) → age backend (opts.armored for ASCII); PGP public-key blocks → PGP backend (always armored). Auto-dispatched on key format. Multi-recipient: any listed identity decrypts.

Parameters

- `data` (*string | Uint8Array | ArrayBuffer*) — Plaintext to encrypt.
- `recipients` (*string | string[]*) — One or more recipient public keys. age1... bech32 keys use the age backend; —BEGIN PGP PUBLIC KEY BLOCK— entries use the PGP backend. Backends cannot be mixed in one call.
- `opts` (*{ armored?: boolean }, optional*) — armored (age backend only) wraps the output in age ASCII armor; defaults to false (binary). Ignored on the PGP path, which is always armored.

Returns: Uint8Array — the ciphertext (binary age, ASCII-armored age when opts.armored, or armored PGP message bytes).

Throws: Throws if data is an unsupported type, no recipients are given, a recipient is actually a private key, or a recipient string fails to parse.

```
const ct = crypto.encrypt.encrypt("secret", publicKey, { armored: true });
```

17.5.1.4 crypto.encrypt.keygen

```
keygen(): { publicKey: string; privateKey: string }
```

Generate a fresh age X25519 keypair. Returns { publicKey: 'age1...', privateKey: 'AGE-SECRET-KEY-1...' }.

Returns: { publicKey: string, privateKey: string } — publicKey is the shareable age1... recipient; privateKey is the secret AGE-SECRET-KEY-1... identity.

Throws: Throws if the system RNG fails to generate an identity.

```
const { publicKey, privateKey } = crypto.encrypt.keygen();
```

17.5.1.5 crypto.encrypt.keygenPgp

```
keygenPgp(opts?: { name?: string, email?: string }): { publicKey: string;  
privateKey: string }
```

Generate a PGP keypair (RSA 2048). opts.name / opts.email populate the user ID. Returns armored { publicKey, privateKey } blocks. encrypt/decrypt auto-route to PGP when they see these.

Parameters

- `opts` (*{ name?: string, email?: string }, optional*) — name and email populate the primary user ID; both default to empty (fine for throwaway keys).

Returns: { publicKey: string, privateKey: string } — both ASCII-armored PGP key blocks (—BEGIN PGP PUBLIC/PRIVATE KEY BLOCK—).

Throws: Throws if entity generation or armor serialisation fails.

```
const { publicKey, privateKey } = crypto.encrypt.keygenPgp({ name: "Test", email:  
"t@example.com" });
```

17.5.1.6 crypto.encrypt.rekey

```
rekey(ciphertext: string | Uint8Array | ArrayBuffer, oldIdentities: string |  
string[], newRecipients: string | string[], opts?: { armored?: boolean }):  
Uint8Array
```

Re-encrypt for a new recipient set without exposing plaintext to JS. Output format defaults to match the input; opts.armored forces. Internal decrypt+encrypt loop.

Parameters

- `ciphertext` (*string | Uint8Array | ArrayBuffer*) — The existing age ciphertext to re-key (armor auto-detected).
- `oldIdentities` (*string | string[]*) — Current AGE-SECRET-KEY-1... private keys used to decrypt the existing ciphertext.
- `newRecipients` (*string | string[]*) — New age1... public keys to encrypt the result for.
- `opts` (*{ armored?: boolean }, optional*) — armored forces the output armor state; default preserves the input's armor state.

Returns: Uint8Array — the re-encrypted ciphertext for the new recipient set.

Throws: Throws if ciphertext is empty, either key list is empty, a key is the wrong kind (public vs private), or the decrypt step fails (no matching identity / malformed input). age backend only.

```
const rotated = crypto.encrypt.rekey(ct, oldKey, newPublicKey);
```

17.5.2 crypto.hash

17.5.2.1 crypto.hash.blake3

```
blake3(input: string): string
```

BLAKE3 hex digest (32-byte output, lukechampion.com/blake3).

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 64-char lowercase hex digest (256-bit output).

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.blake3("hello");
```

17.5.2.2 crypto.hash.crc32

```
crc32(input: string): string
```

CRC-32 (IEEE polynomial), zero-padded to 8 hex chars.

Parameters

- `input` (*string*) — Data to checksum, interpreted as a UTF-8 byte sequence.

Returns: string — 8-char zero-padded lowercase hex checksum.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const c = crypto.hash.crc32("hello"); // "3610a686"
```

17.5.2.3 crypto.hash.md5

```
md5(input: string): string
```

MD5 hex digest of a UTF-8 input. Avoid for security purposes — exposed for compatibility with legacy fingerprints.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 32-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.md5("hello"); // "5d41402abc4b2a76b9719d911017c592"
```

17.5.2.4 crypto.hash.sha1

```
sha1(input: string): string
```

SHA-1 hex digest of a UTF-8 input. Avoid for security purposes.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 40-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.sha1("hello");
```

17.5.2.5 crypto.hash.sha256

```
sha256(input: string): string
```

SHA-256 hex digest of a UTF-8 input.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 64-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.sha256("hello");
```

17.5.2.6 crypto.hash.sha384

```
sha384(input: string): string
```

SHA-384 hex digest of a UTF-8 input.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 96-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.sha384("hello");
```

17.5.2.7 crypto.hash.sha3_256

```
sha3_256(input: string): string
```

SHA-3 256-bit hex digest. The underscore in the name matches `recon`'s binding.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 64-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.sha3_256("hello");
```

17.5.2.8 crypto.hash.sha3_512

```
sha3_512(input: string): string
```

SHA-3 512-bit hex digest.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 128-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.sha3_512("hello");
```

17.5.2.9 crypto.hash.sha512

```
sha512(input: string): string
```

SHA-512 hex digest of a UTF-8 input.

Parameters

- `input` (*string*) — Data to hash, interpreted as a UTF-8 byte sequence.

Returns: string — 128-char lowercase hex digest.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
const d = crypto.hash.sha512("hello");
```

17.5.3 crypto.jwt

17.5.3.1 crypto.jwt.sign

```
sign(claims: Record<string, unknown>, secret: string, opts?: { algorithm?: string }): string
```

Sign a claims object. `secret` is raw bytes for HS*; PEM-encoded private key for RS*/PS*/ES*/EdDSA; or a JWK JSON object (`kty` picks the key type) for any algorithm. `opts.algorithm` defaults to HS256.

Parameters

- `claims` (*Record<string, unknown>*) — Claims payload. Passed through to `MapClaims`; RFC 7519 reserved claims (`exp`/`nbfiat`/`iss`/`aud`/`sub`) are honoured. Reserved claims are NOT synthesised — set `iat`/`exp` explicitly if you want them.
- `secret` (*string*) — Key material: raw HMAC bytes for HS256/384/512, a PEM-encoded private key (—BEGIN ...) for RS*/PS*/ES*/EdDSA, or a JWK JSON object (`{“kty”:...}`).

- `opts` (*{ algorithm?: string }, optional*) — algorithm names the RFC 7518 alg (HS256/HS384/HS512, RS256/384/512, PS256/384/512, ES256/384/512, EdDSA); case-insensitive. Defaults to HS256.

Returns: string — the signed compact-serialisation JWT (header.payload.signature).

Throws: Throws if claims is null, secret is empty, the algorithm is unsupported, or the secret shape mismatches the algorithm (e.g. HMAC algo with a PEM key, or asymmetric algo with plain bytes).

```
const tok = crypto.jwt.sign({ sub: "u1" }, "topsecret", { algorithm: "HS256" });
```

17.5.3.2 crypto.jwt.validate

```
validate(token: string, secret: string, opts?: { algorithm?: string, audience?: string, issuer?: string }): { valid: boolean; claims?: object; reason?: string }
```

Verify signature + standard time claims (exp/nbf; iat is not checked, per RFC 7519) + optional aud/iss. secret accepts raw bytes / PEM public key / JWK. Set `opts.algorithm` for the algo-confusion guard. Resolves `{ valid:true, claims }` or `{ valid:false, reason }`.

Parameters

- `token` (*string*) — The compact-serialisation JWT to verify.
- `secret` (*string*) — Verification key: raw HMAC bytes, a PEM-encoded public key (or certificate), or a JWK JSON object.
- `opts` (*{ algorithm?: string, audience?: string, issuer?: string }, optional*) — algorithm pins the accepted alg (algorithm-confusion guard); when unset, any supported alg is accepted. audience / issuer enforce the aud / iss claims when set.

Returns: `{ valid: true, claims: object }` on success, or `{ valid: false, reason: string }` on any verification / claim failure.

Throws: Throws only on structural / wiring errors (empty token or secret, malformed token, or a secret shape that mismatches the token's algorithm). Cryptographic and claim failures resolve as `{ valid: false, reason }` instead of throwing.

```
const r = crypto.jwt.validate(tok, "topsecret", { algorithm: "HS256" });
if (r.valid) runtime.log(r.claims.sub);
```

17.5.3.3 crypto.jwt.view

```
view(token: string): { header: object; payload: object; signature: string }
```

Decode header + payload WITHOUT verifying the signature. Useful for inspection / debugging auth flows. Malformed input throws.

Parameters

- `token` (*string*) — A compact-serialisation JWT (three dot-separated base64url segments).

Returns: `{ header: object, payload: object, signature: string }` — decoded header and payload (object key order preserved) plus the raw signature segment.

Throws: Throws if the token is not three dot-separated segments or a segment fails base64url / JSON decoding.

```
const { header, payload } = crypto.jwt.view(tok);
```

17.6 db

Database / KV / directory clients (all pure-Go, no cgo): SQLite, PostgreSQL, MySQL/MariaDB, SQL Server, ClickHouse, Oracle, Redis, Valkey, memcached, LDAP, and DICT. The six SQL engines share ONE handle — `open()` resolves to `{ exec, query, queryValue, begin, prepare, close }` — so the query API is identical across them; only the DSN/options and placeholder syntax differ per engine (shown on each `open()` below). `exec` runs a non-SELECT and resolves `{ rowsAffected, lastInsertId }`; `query` resolves one ordered object per row; `queryValue` resolves the first column of the first row (or null); `begin()` returns a transaction and `prepare()` a compiled statement (both with the same `exec/query/queryValue` surface). Redis/Valkey use `{ do, ping, close }` (`do(cmd, ...args)` runs any RESP command); memcached uses `{ get, set, delete }`; LDAP uses `{ rootDSE, search, close }`; DICT is a one-shot define/match. Each subsection below documents its connection call — for the handle methods, per-engine placeholder syntax (`? / $1 / @p1 / :1`), JS $\leftrightarrow$ SQL type mapping, transactions, prepared statements, and runnable recipes, see the db guide in §5.8.

17.6.1 db.clickhouse

17.6.1.1 db.clickhouse.open

```
open(dsn: string | { host?: string, port?: number, user?: string, password?:
string, database?: string, secure?: boolean }): Promise<{ exec(sql:
string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId:
number }>; query(sql: string, ...params: unknown[]): Promise<Record<string,
unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>;
begin(): Promise<{ exec(sql: string, ...params: unknown[]):
Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql:
string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql:
string, ...params: unknown[]): Promise<unknown>; commit(): Promise<void>;
rollback(): Promise<void> }>; prepare(sql: string): Promise<{ exec(...params:
unknown[]): Promise<{ rowsAffected: number, lastInsertId: number }>;
query(...params: unknown[]): Promise<Record<string, unknown>[]>;
queryValue(...params: unknown[]): Promise<unknown>; close(): Promise<void> }>;
close(): Promise<void> }>
```

Connect to ClickHouse via the pure-Go clickhouse-go v2 driver. Arg is a clickhouse:

Parameters

- `dsn` (*string* | *{ host?: string, port?: number, user?: string, password?: string, database?: string, secure?: boolean }*) — A clickhouse:

Returns: `Promise<handle>` resolving to the shared SQL handle: `exec(sql, ...params) → Promise<{ rowsAffected, lastInsertId }>`; `query(sql, ...params) → Promise<object[]>` (one ordered object per row); `queryValue(sql, ...params) → Promise<any>` (first column of the first row, or null); `begin() → Promise<tx>`; `prepare(sql) → Promise<stmt>`; `close() → Promise<void>`. ClickHouse uses `?` positional placeholders (or `@name`).

Throws: Throws if no argument is given, the DSN string is empty, the argument is neither a string nor an object, or the connection ping fails.

```
const db = await db.clickhouse.open({ host: "localhost", database: "metrics",
secure: false });
const rows = await db.query("SELECT name, value FROM stats WHERE host = ?",
"web1");
await db.close();
```

17.6.2 db.dict

17.6.2.1 db.dict.define

```
define(host: string, word: string, opts?: { database?: string, port?: string,
timeout?: number }): Promise<{ word: string, found: boolean, definitions: { db:
string, dbName: string, text: string }[] }>
```

RFC 2229 DICT word lookup. `define(host, word, opts?) -> { word, found, definitions: [{ db, dbName, text }] }`. `found:false` on no match (not an error). One-shot: connect, query, QUIT.

Parameters

- `host` (*string*) — The DICT server hostname.
- `word` (*string*) — The word to look up.
- `opts` (*{ database?: string, port?: string, timeout?: number }, optional*) — database selects a specific dictionary (default "*" = all); port is the DICT port (default "2628"); timeout is the dial/read deadline in milliseconds (default 10000).

Returns: `Promise<{ word: string, found: boolean, definitions: { db: string, dbName: string, text: string }[] }>` — definitions carries one entry per matching dictionary (db is the dictionary code, dbName its human name, text the definition body). A word with no definitions resolves with `found:false` and an empty list.

Throws: Throws if host or word is empty, on dial/banner failure, or on an unexpected DICT status code (e.g. 550 invalid database). A 552 "no match" is NOT an error — it resolves with `found:false`.

```
const r = await db.dict.define("dict.org", "serendipity");
if (r.found) runtime.log(r.definitions[0].text);
```

17.6.2.2 db.dict.match

```
match(host: string, word: string, opts?: { strategy?: string, database?: string,
port?: string, timeout?: number }): Promise<{ word: string, matches: { db: string,
word: string }[] }>
```

RFC 2229 word match. `match(host, word, opts?) -> { word, matches: [{ db, word }] }`. `opts.strategy` (default prefix), `opts.database` (default *), `opts.port` (default 2628). One-shot: connect, query, QUIT.

Parameters

- `host` (*string*) — The DICT server hostname.
- `word` (*string*) — The word (or pattern) to match.

- `opts` (*{ strategy?: string, database?: string, port?: string, timeout?: number }, optional*) — `strategy` is the match strategy (default “prefix”); `database` selects a specific dictionary (default “*” = all); `port` is the DICT port (default “2628”); `timeout` is the dial/read deadline in milliseconds (default 10000).

Returns: `Promise<{ word: string, matches: { db: string, word: string }[] }>` — `matches` carries one entry per matched word (`db` is the dictionary it was found in, `word` the matched headword). No matches resolves with an empty `matches` list.

Throws: Throws if `host` or `word` is empty, on dial/banner failure, or on an unexpected DICT status code. A 552 “no match” is NOT an error — it resolves with an empty `matches` list.

```
const r = await db.dict.match("dict.org", "seren", { strategy: "prefix" });
runtime.log(r.matches.map(m => m.word));
```

17.6.3 db.ldap

17.6.3.1 db.ldap.open

```
open(url: string, opts?: { bindDN?: string, password?: string }):
Promise<{ rootDSE(): Promise<Record<string, unknown>>; search(baseDN: string,
filter?: string, attrs?: string[]): Promise<Record<string, unknown>[]>; close():
Promise<void> }>
```

Dial LDAP (ldap:

Parameters

- `url` (*string*) — An LDAP URL: ldap:
- `opts` (*{ bindDN?: string, password?: string }, optional*) — When `bindDN` is set, the connection binds with `bindDN`/password instead of doing an anonymous bind.

Returns: `Promise<handle>` resolving to `{ rootDSE, search, close }`: `rootDSE()` → `Promise<object>` reads the server’s Root DSE (an ordered `{ dn, <attr>: string[] }` object advertising naming contexts, supported controls, vendor, etc.; an empty object when the server returns no entry); `search(baseDN, filter, attrs?)` → `Promise<object[]>` runs a whole-subtree search and returns one ordered `{ dn, <attr>: string[] }` object per entry (multi-valued attributes stay arrays; filter defaults to `(objectClass=*)`; `attrs` is an optional array of attribute names); `close()` → `Promise<void>`. A directory-inspection (read) binding, not a write/modify surface.

Throws: `open` throws if `url` is empty, the dial fails, or (when `bindDN` is set) the bind fails (the connection is closed on bind failure). `rootDSE` / `search` throw on the underlying LDAP search error.

```
const dir = await db.ldap.open("ldap://localhost:389");
const meta = await dir.rootDSE();
const people = await dir.search("ou=people,dc=example,dc=com", "(uid=alice)",
["cn", "mail"]);
await dir.close();
```

17.6.4 db.memcached

17.6.4.1 db.memcached.open

```
open(addr: string): Promise<{ get(key: string): Promise<string | null>; set(key: string, value: unknown, expirySeconds?: number): Promise<void>; delete(key: string): Promise<boolean> }>
```

Connect to memcached (host:port). Returns { get, set, delete }. get -> string or null (miss); delete -> bool (existed). set(key, value, expirySeconds?). No ping on open; the pool is lazy.

Parameters

- `addr` (*string*) — A memcached server address, host:port (e.g. localhost:11211).

Returns: Promise<handle> resolving to { get, set, delete }: get(key) → Promise<string | null> (null on a cache miss); set(key, value, expirySeconds?) → Promise<void> (value stored as bytes; expirySeconds 0 or omitted means never expire); delete(key) → Promise<boolean> (true if the key existed, false on a miss). gomemcache pools connections lazily, so there is no ping-on-open and no close method (the pool is GC'd with the handle).

Throws: open throws if addr is empty. set throws if key is empty or the value cannot be coerced to bytes. get / delete throw on transport errors; a cache miss is data (get → null, delete → false), not an error.

```
const mc = await db.memcached.open("localhost:11211");
await mc.set("session:42", "active", 300);
const v = await mc.get("session:42"); // "active" or null
const existed = await mc.delete("session:42"); // true
```

17.6.5 db.mssql

17.6.5.1 db.mssql.open

```
open(dsn: string | { host?: string, port?: number, user?: string, password?: string, database?: string }): Promise<{ exec(sql: string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql: string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>; begin(): Promise<{ exec(sql: string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql: string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>; commit(): Promise<void>; rollback(): Promise<void> }>; prepare(sql: string): Promise<{ exec(...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId: number }>; query(...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(...params: unknown[]): Promise<unknown>; close(): Promise<void> }>; close(): Promise<void> }>
```

Connect to Microsoft SQL Server via the pure-Go go-mssqldb driver. Arg is a sqlserver:

Parameters

- `dsn` (*string* | { host?: string, port?: number, user?: string, password?: string, database?: string }) — A sqlserver:

Returns: Promise<handle> resolving to the shared SQL handle: exec(sql, ...params) → Promise<{ rowsAffected, lastInsertId }>; query(sql, ...params) → Promise<object[]> (one ordered object per row); queryValue(sql, ...params) → Promise<any> (first column of the first row, or null); begin() → Promise<tx>; prepare(sql) → Promise<stmt>; close() → Promise<void>. SQL Server uses @p1, @p2, ... placeholders.

Throws: Throws if no argument is given, the DSN string is empty, the argument is neither a string nor an object, or the connection ping fails.

```
const db = await db.mssql.open({ host: "localhost", user: "sa", password: "P@ss",
  database: "shop" });
const rows = await db.query("SELECT TOP 5 id, name FROM users WHERE region = @p1",
  "EU");
await db.close();
```

17.6.6 db.mysql

17.6.6.1 db.mysql.open

```
open(dsn: string | { host?: string, port?: number, user?: string, password?:
  string, database?: string }): Promise<{ exec(sql: string, ...params: unknown[]):
  Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql:
  string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql:
  string, ...params: unknown[]): Promise<unknown>; begin(): Promise<{ exec(sql:
  string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId:
  number }>; query(sql: string, ...params: unknown[]): Promise<Record<string,
  unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>;
  commit(): Promise<void>; rollback(): Promise<void> }>; prepare(sql: string):
  Promise<{ exec(...params: unknown[]): Promise<{ rowsAffected: number,
  lastInsertId: number }>; query(...params: unknown[]): Promise<Record<string,
  unknown>[]>; queryValue(...params: unknown[]): Promise<unknown>; close():
  Promise<void> }>; close(): Promise<void> }>
```

Connect to MySQL/MariaDB via the pure-Go go-sql-driver. Arg is a go-sql-driver DSN string (user:pass@tcp(host:port)/db) or an options object { host, port, user, password, database }. Returns the shared SQL handle. Uses ? placeholders. Pings on open.

Parameters

- *dsn* (*string* | { *host?*: *string*, *port?*: *number*, *user?*: *string*, *password?*: *string*, *database?*: *string* }) — A go-sql-driver DSN string (user:pass@tcp(host:port)/db?params) used verbatim, OR an options object assembled into one (defaults: host localhost, port 3306, empty database). One driver serves both MySQL and MariaDB.

Returns: Promise<handle> resolving to the shared SQL handle: exec(sql, ...params) → Promise<{ rowsAffected, lastInsertId }>; query(sql, ...params) → Promise<object[]> (one ordered object per row); queryValue(sql, ...params) → Promise<any> (first column of the first row, or null); begin() → Promise<tx>; prepare(sql) → Promise<stmt>; close() → Promise<void>. MySQL uses ? positional placeholders.

Throws: Throws if no argument is given, the DSN string is empty, the argument is neither a string nor an object, or the connection ping fails.

```
const db = await db.mysql.open("app:s3cr3t@tcp(localhost:3306)/shop");
const n = await db.queryValue("SELECT COUNT(*) FROM orders WHERE status = ?",
"paid");
await db.close();
```

17.6.7 db.oracle

17.6.7.1 db.oracle.open

```
open(dsn: string | { host?: string, port?: number, user?: string, password?:
string, database?: string }): Promise<{ exec(sql: string, ...params: unknown[]):
Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql:
string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql:
string, ...params: unknown[]): Promise<unknown>; begin(): Promise<{ exec(sql:
string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId:
number }>; query(sql: string, ...params: unknown[]): Promise<Record<string,
unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>;
commit(): Promise<void>; rollback(): Promise<void> }>; prepare(sql: string):
Promise<{ exec(...params: unknown[]): Promise<{ rowsAffected: number,
lastInsertId: number }>; query(...params: unknown[]): Promise<Record<string,
unknown>[]>; queryValue(...params: unknown[]): Promise<unknown>; close():
Promise<void> }>; close(): Promise<void> }>
```

Connect to Oracle via the pure-Go go-ora driver (no cgo). Arg is an oracle:

Parameters

- *dsn* (*string* | { *host?*: *string*, *port?*: *number*, *user?*: *string*, *password?*: *string*, *database?*: *string* }) — An oracle:

Returns: Promise<handle> resolving to the shared SQL handle: `exec(sql, ...params)` → Promise<{ rowsAffected, lastInsertId }>; `query(sql, ...params)` → Promise<object[]> (one ordered object per row); `queryValue(sql, ...params)` → Promise<any> (first column of the first row, or null); `begin()` → Promise<tx>; `prepare(sql)` → Promise<stmt>; `close()` → Promise<void>. Oracle uses :1, :2, ... (or :name) bind placeholders.

Throws: Throws if no argument is given, the DSN string is empty, the argument is neither a string nor an object, or the connection ping fails.

```
const db = await db.oracle.open({ host: "localhost", user: "app", password:
"s3cr3t", database: "ORCLPDB1" });
const rows = await db.query("SELECT id, name FROM users WHERE id = :1", 42);
await db.close();
```

17.6.8 db.postgres

17.6.8.1 db.postgres.open

```
open(dsn: string | { host?: string, port?: number, user?: string, password?:
string, database?: string, sslmode?: string }): Promise<{ exec(sql:
string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId:
number }>; query(sql: string, ...params: unknown[]): Promise<Record<string,
unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>;
begin(): Promise<{ exec(sql: string, ...params: unknown[]):
```

```

Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql:
string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql:
string, ...params: unknown[]): Promise<unknown>; commit(): Promise<void>;
rollback(): Promise<void> }>; prepare(sql: string): Promise<{ exec(...params:
unknown[]): Promise<{ rowsAffected: number, lastInsertId: number }>;
query(...params: unknown[]): Promise<Record<string, unknown>[]>;
queryValue(...params: unknown[]): Promise<unknown>; close(): Promise<void> }>;
close(): Promise<void> }>

```

Connect to PostgreSQL via the pure-Go pgx driver. Arg is a libpq DSN/URL string or an options object { host, port, user, password, database, sslmode }. Returns the shared SQL handle { exec, query, queryValue, begin, prepare, close }. Uses \$1,\$2,... placeholders. Pings on open.

Parameters

- `dsn` (*string* | { *host?: string*, *port?: number*, *user?: string*, *password?: string*, *database?: string*, *sslmode?: string* }) — A libpq DSN/URL string used verbatim, OR an options object assembled into a postgres:

Returns: Promise<handle> resolving to the shared SQL handle: `exec(sql, ...params)` → Promise<{ rowsAffected, lastInsertId }>; `query(sql, ...params)` → Promise<object[]> (one ordered object per row, keyed by column name in column order); `queryValue(sql, ...params)` → Promise<any> (first column of the first row, or null); `begin()` → Promise<tx> ({ `exec`, `query`, `queryValue`, `commit`, `rollback` }); `prepare(sql)` → Promise<stmt> ({ `exec`, `query`, `queryValue`, `close` }); `close()` → Promise<void>. Postgres uses \$1, \$2, ... positional placeholders.

Throws: Throws if no argument is given, the DSN string is empty, the argument is neither a string nor an object, or the connection ping fails (the pool is closed on ping failure).

```

const db = await db.postgres.open({ host: "localhost", user: "app", password:
"s3cr3t", database: "shop", sslmode: "disable" });
const rows = await db.query("SELECT id, name FROM users WHERE id = $1", 42);
await db.close();

```

17.6.9 db.redis

17.6.9.1 db.redis.open

```

open(url: string): Promise<{ do(cmd: string, ...args: unknown[]):
Promise<unknown>; ping(): Promise<string>; close(): Promise<void> }>

```

Connect to Redis (redis:

Parameters

- `url` (*string*) — A standard Redis URL: redis:

Returns: Promise<handle> resolving to { `do`, `ping`, `close` }: `do(cmd, ...args)` → Promise<any> runs an arbitrary RESP command (the first arg is the command name, the rest its arguments) and returns the reply coerced to a JS value — strings, numbers, arrays, or null; a nil reply (missing key) resolves to null rather than throwing. `ping()` → Promise<string> ('PONG'). `close()` → Promise<void>.

Throws: open throws if url is empty, the URL fails to parse, or the open-time ping fails (the client is closed on ping failure). do throws on Redis-level errors (WRONGTYPE, unknown command, etc.); a missing-key nil reply is data, not an error.

```
const r = await db.redis.open("redis://localhost:6379/0");
await r.do("SET", "greeting", "hi");
const v = await r.do("GET", "greeting"); // "hi"
const missing = await r.do("GET", "nope"); // null
await r.close();
```

17.6.10 db.sqlite

17.6.10.1 db.sqlite.open

```
open(path: string): Promise<{ exec(sql: string, ...params: unknown[]):
Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql:
string, ...params: unknown[]): Promise<Record<string, unknown>[]>; queryValue(sql:
string, ...params: unknown[]): Promise<unknown>; begin(): Promise<{ exec(sql:
string, ...params: unknown[]): Promise<{ rowsAffected: number, lastInsertId:
number }>; query(sql: string, ...params: unknown[]): Promise<Record<string,
unknown>[]>; queryValue(sql: string, ...params: unknown[]): Promise<unknown>;
commit(): Promise<void>; rollback(): Promise<void> }>; prepare(sql: string):
Promise<{ exec(...params: unknown[]): Promise<{ rowsAffected: number,
lastInsertId: number }>; query(...params: unknown[]): Promise<Record<string,
unknown>[]>; queryValue(...params: unknown[]): Promise<unknown>; close():
Promise<void> }>; close(): Promise<void> }>
```

Open a SQLite database (':memory:' or a file path; created if absent). Resolves to a handle { exec, query, queryValue, begin, prepare, close }. Connection is Ping-ed before resolving.

Parameters

- `path` (*string*) — “:memory:” for an in-RAM database, or a filesystem path. Missing files are created by the modernc.org/sqlite (pure-Go, no cgo) driver.

Returns: Promise<handle> resolving to the shared SQL handle object: exec(sql, ...params) → Promise<{ rowsAffected: number, lastInsertId: number }>; query(sql, ...params) → Promise<object[]> (one ordered object per row, keyed by column name in column order); queryValue(sql, ...params) → Promise<any> (first column of the first row, or null when no rows match); begin() → Promise<tx> ({ exec, query, queryValue, commit, rollback }); prepare(sql) → Promise<stmt> ({ exec, query, queryValue, close }); close() → Promise<void>. SQLite uses ? positional placeholders. UTF-8 byte columns scan to strings; genuinely binary bytes surface as Uint8Array.

Throws: Throws if path is missing or empty, or if the connection ping fails (the *sql.DB is closed on ping failure rather than leaked). Subsequent exec/query/etc. throw the driver error on bad SQL or bind mismatch.

```
const conn = await db.sqlite.open(":memory:");
await conn.exec("CREATE TABLE t (id INTEGER PRIMARY KEY, name TEXT)");
await conn.exec("INSERT INTO t (name) VALUES (?)", "alice");
const rows = await conn.query("SELECT * FROM t WHERE name = ?", "alice");
await conn.close();
```


17.6.11 db.valkey

17.6.11.1 db.valkey.open

```
open(url: string): Promise<{ do(cmd: string, ...args: unknown[]):  
  Promise<unknown>; ping(): Promise<string>; close(): Promise<void> }>
```

Connect to Valkey (valkey):

Parameters

- `url` (*string*) — A connection URL: valkey:

Returns: Promise<handle> resolving to { do, ping, close }: do(cmd, ...args) → Promise<any> runs an arbitrary RESP command (the first arg is the command name, the rest its arguments) and returns the reply coerced to a JS value — strings, numbers, arrays, or null; a nil reply (missing key) resolves to null rather than throwing. ping() → Promise<string> ('PONG'). close() → Promise<void>.

Throws: open throws if url is empty, the URL fails to parse, or the open-time ping fails (the client is closed on ping failure). do throws on RESP-level errors (WRONGTYPE, unknown command, etc.); a missing-key nil reply is data, not an error.

```
const r = await db.valkey.open("valkey://localhost:6379/0");  
await r.do("SET", "greeting", "hi");  
const v = await r.do("GET", "greeting"); // "hi"  
await r.close();
```

17.7 fs

Filesystem operations: path manipulation and archive create/extract.

17.7.1 fs.archive

17.7.1.1 fs.archive.create

```
create(destPath: string, sources: (string | { path: string, name?: string })[]):  
  Promise<{ path: string; format: string; entries: string[]; bytes?: number }>
```

Create a zip / tar / tar.gz at destPath from a list of paths. Format inferred from extension.

Parameters

- `destPath` (*string*) — Output archive path. Format is inferred from the extension: .zip, .tar, .tar.gz, or .tgz.
- `sources` ((*string* | { *path*: *string*, *name*?: *string* })[*i*]) — Non-empty array of inputs. A bare string uses the disk path as-is and its basename inside the archive; an object overrides the in-archive name via name. Directory sources are recursed (the directory's basename becomes the archive subdir). Archive paths always use forward slashes.

Returns: Promise<{ path: string, format: string, entries: string[], bytes?: number }> — path is destPath, format is the inferred format ("zip" | "tar" | "tar.gz"), entries lists the file paths written (directories excluded), and bytes is the final archive size when stat succeeds.

Throws: Rejects if destPath is empty, sources is not an array / is empty / contains a bad entry (object missing 'path', unsupported element type), the format cannot be inferred from the extension, or any disk read / write fails (e.g. a source path does not exist).

```
const r = await fs.archive.create("out.tar.gz", ["dist", { path: "README.md",
name: "docs/README.md" }]);
runtime.log(r.format, r.entries.length);
```

17.7.1.2 fs.archive.extract

```
extract(archivePath: string, destDir: string, opts?: { overwrite?: boolean,
maxTotalBytes?: number, maxEntries?: number }): Promise<{ path: string; format:
string; dest: string; entries: string[] }>
```

Extract a zip / tar / tar.gz to destDir. opts.overwrite controls O_EXCL behaviour; opts.maxTotalBytes/maxEntries cap decompression-bomb risk.

Parameters

- `archivePath` (*string*) — Path to the archive. Format is inferred from its extension (.zip, .tar, .tar.gz, .tgz).
- `destDir` (*string*) — Destination directory; created (recursively) if absent. All entries are confined to this directory via zip-slip / tar-slip protection.
- `opts` (*{ overwrite?: boolean, maxTotalBytes?: number, maxEntries?: number }, optional*) — overwrite (default false) clobbers existing files; when false, an entry colliding with an existing file fails the call (O_EXCL). maxTotalBytes caps the cumulative decompressed bytes written across all members (default 1 GB; a non-positive value falls back to the default). maxEntries caps the number of archive members processed (default 100000; a non-positive value falls back to the default). Both guard against a small crafted archive decompressing into an enormous amount of disk I/O (a decompression bomb); extraction stops as soon as either cap is exceeded.

Returns: Promise<{ path: string, format: string, dest: string, entries: string[] }> — path is archivePath, format is the inferred format, dest is destDir, and entries lists the extracted entry names (regular files only).

Throws: Rejects if archivePath or destDir is empty, the format cannot be inferred, destDir cannot be created, the archive cannot be opened / decoded, an entry escapes destDir (absolute path or '..' component), (with overwrite false) an entry collides with an existing file, the entry count exceeds maxEntries, or the cumulative decompressed size exceeds maxTotalBytes.

```
const r = await fs.archive.extract("out.tar.gz", "./unpacked", { overwrite: true,
maxTotalBytes: 1 << 28, maxEntries: 5000 });
runtime.log(r.entries.length, "files extracted");
```

17.7.2 fs.exists

```
exists(path: string): Promise<boolean>
```

Report whether a path exists. Never throws for a missing path.

Parameters

- `path` (*string*) — Path to test.

Returns: Promise resolving to true if the path exists, false if it does not.

Throws: Rejects only on an unexpected stat error (e.g. a permission error on a parent); a missing path resolves to false.

```
if (!(await fs.exists("report"))) await fs.mkdir("report");
```

17.7.3 fs.find

```
find(opts?: { root?: string | string[], glob?: string | string[], exclude?: string | string[], regex?: string, fullPath?: boolean, type?: "file"|"dir"|"symlink" | Array<"file"|"dir"|"symlink">, extension?: string | string[], case?: "smart"|"sensitive"|"insensitive", hidden?: boolean, gitignore?: boolean, followSymlinks?: boolean, maxDepth?: number, minDepth?: number, absolute?: boolean, limit?: number, sort?: boolean, strict?: boolean, stat?: boolean, stream?: boolean }): Promise<string[]> | Promise<FindEntry[]> | AsyncIterable<string>
```

Fast file/path search (fd-like): walk one or more roots and return matching paths. Respects .gitignore and skips hidden files by default (opt out with gitignore:false / hidden:true). Returns string[] of paths, or FindEntry[] with stat:true, or an async iterator with stream:true.

Parameters

- `opts` (*{ root?: string | string[], glob?: string | string[], exclude?: string | string[], regex?: string, fullPath?: boolean, type?: "file"|"dir"|"symlink"|Array<"file"|"dir"|"symlink">, extension?: string | string[], case?: "smart"|"sensitive"|"insensitive", hidden?: boolean, gitignore?: boolean, followSymlinks?: boolean, maxDepth?: number, minDepth?: number, absolute?: boolean, limit?: number, sort?: boolean, strict?: boolean, stat?: boolean, stream?: boolean }*, optional) — Search options. root defaults to ".". glob/exclude use ** globs; regex matches the basename (or, with fullPath, the path relative to root — which equals the displayed cwd-relative path only when root is "."). type/extension filter by kind. case defaults to smart (case-insensitive unless the regex has an uppercase char). hidden (default false) and gitignore (default true) control ignore-awareness. maxDepth/minDepth bound recursion. absolute returns absolute paths. limit caps results. sort yields deterministic path order. strict throws on the first traversal error (default: skip unreadable entries). stat returns { path, type, size, mtimeMs } objects. stream returns an async iterator.

Returns: By default Promise<string[]> of matching paths (relative to cwd, or absolute with absolute:true). With stat:true, Promise<{ path: string, type: "file"|"dir"|"symlink", size: number, mtimeMs: number }[]>. With stream:true, an async iterator (for await ...) yielding the same items.

Throws: Throws ("fs.find: ...") on an invalid regex, or on the first traversal error when strict:true. Unreadable files/dirs are skipped by default.

```
const files = await fs.find({ glob: "src/**/*.ts", type: "file" });
```

17.7.4 fs.grep

```
grep(opts: { pattern: string, fixed?: boolean, root?: string | string[], paths?: string[], glob?: string | string[], exclude?: string | string[], type?: string | string[], extension?: string | string[], case?: "smart"|"sensitive"|"insensitive", word?: boolean, multiline?: boolean, context?: number, before?: number, after?: number, invert?: boolean, maxMatches?: number, maxResults?: number, includeBinary?: boolean, hidden?: boolean, gitignore?: boolean, followSymlinks?: boolean, maxDepth?: number, absolute?: boolean, sort?: boolean, strict?: boolean, stream?: boolean }): Promise<GrepMatch[]> | AsyncIterable<GrepMatch>
```

Fast content search (rg-like): walk roots (or explicit paths) and return per-line matches. RE2 regex by default (fixed:true for a literal). Respects .gitignore, skips hidden and binary files by default. Returns GrepMatch[], or an async iterator with stream:true.

Parameters

- `opts` (*{ pattern: string, fixed?: boolean, root?: string | string[], paths?: string[], glob?: string | string[], exclude?: string | string[], type?: string | string[], extension?: string | string[], case?: "smart"|"sensitive"|"insensitive", word?: boolean, multiline?: boolean, context?: number, before?: number, after?: number, invert?: boolean, maxMatches?: number, maxResults?: number, includeBinary?: boolean, hidden?: boolean, gitignore?: boolean, followSymlinks?: boolean, maxDepth?: number, absolute?: boolean, sort?: boolean, strict?: boolean, stream?: boolean }*) — pattern is a RE2 regex (or literal with fixed:true). root defaults to "."; paths searches an explicit list of files instead (directories in paths are not recursed). type maps rg-style names (ts/js/go/md/json/py/rs/c) to extensions. case defaults to smart. context (or before/after) adds context lines. invert emits non-matching lines. maxMatches caps per file; maxResults caps total. Binary files are skipped unless includeBinary. stream returns an async iterator.

Returns: Promise<{ path: string, line: number, column: number, match: string, text: string, before?: string[], after?: string[] }[]> (1-based line/column). With stream:true, an async iterator yielding the same objects.

Throws: Throws ("fs.grep: ...") if pattern is missing/invalid, or on the first traversal/read error when strict:true. Unreadable and binary files are skipped by default.

```
const hits = await fs.grep({ pattern: "TODO\\(.*\\)", type: "ts", context: 1 });
```

17.7.5 fs.grepCount

```
grepCount(opts: GrepOptions): Promise<{ path: string; count: number }[]>
```

Like fs.grep but returns per-file match counts (rg -c). Files with zero matches are omitted.

Parameters

- `opts` (*GrepOptions*) — Same options as fs.grep.

Returns: Promise<{ path: string, count: number }[]> — one entry per file with >= 1 match.

Throws: Throws ("fs.grepCount: ...") if pattern is missing/invalid, or on the first error when strict:true.

```
const counts = await fs.grepCount({ pattern: "import", type: "ts" });
```

17.7.6 fs.grepFiles

```
grepFiles(opts: GrepOptions): Promise<string[]>
```

Like `fs.grep` but returns just the paths of files with at least one match (`rg -l`); one entry per file (deduplicated), no per-line detail.

Parameters

- `opts` (*GrepOptions*) — Same options as `fs.grep` (streaming/context options are ignored).

Returns: `Promise<string[]>` — paths (relative to `cwd`, or absolute with `absolute:true`) of files containing at least one match.

Throws: Throws (“`fs.grepFiles: ...`”) if pattern is missing/invalid, or on the first error when `strict:true`.

```
const files = await fs.grepFiles({ pattern: "deprecated", type: "go" });
```

17.7.7 fs.grepStream

```
grepStream(path: string, opts: { pattern: string, fixed?: boolean, case?: "smart" | "sensitive" | "insensitive", word?: boolean, invert?: boolean, from?: "end" | "start" | number }): AsyncIterable<{ line: number; column: number; match: string; text: string }>
```

Follow a file and yield only lines matching a pattern, as an async iterator (`tail + grep`). Matching runs Go-side with `fs.grep`’s RE2 engine, so only matches cross into the script. `from` works as in `fs.tail`. Long-lived: `break` (or a Run timeout) stops it.

Parameters

- `path` (*string*) — The file to follow. Must exist when the call is made (v1 throws otherwise).
- `opts` (*{ pattern: string, fixed?: boolean, case?: “smart” | “sensitive” | “insensitive”, word?: boolean, invert?: boolean, from?: “end” | “start” | number }*) — pattern is a RE2 regex (or a literal with `fixed:true`). case defaults to `smart` (case-insensitive unless the pattern has an uppercase char). word matches on word boundaries; invert yields non-matching lines. `from` is as in `fs.tail`. Multiline and context lines are not supported here — use `fs.grep` for those.

Returns: An async iterator yielding `{ line, column, match, text }` per matching line. `line` is a 1-based counter of lines observed since the follow started (session-relative, NOT a file line number); `column` is a 1-based rune offset; `match` is the matched substring; `text` is the full line.

Throws: Throws (“`fs.grepStream: ...`”) if path/pattern is missing, the pattern is an invalid regex, the file does not exist at start, or `from` is invalid.

```
for await (const m of fs.grepStream("/var/log/app.log", { pattern: "ERROR|FATAL" })) { runtime.log(m.text); }
```

17.7.8 fs.mkdir

```
mkdir(path: string): Promise<{ path: string }>
```

Create a directory, including any missing parents (mkdir -p). Idempotent.

Parameters

- `path` (*string*) — Directory path to create (mode 0755). Existing directories are fine.

Returns: Promise resolving to { path }.

Throws: Rejects if path is missing/empty or creation fails (e.g. a path component is an existing file).

```
await fs.mkdir("report/assets");
```

17.7.9 fs.path

17.7.9.1 fs.path.basename

```
basename(path: string, suffix?: string): string
```

Final segment of a path; optional suffix is stripped if it matches.

Parameters

- `path` (*string*) — A forward-slash path. Trailing slashes are stripped before taking the last segment.
- `suffix` (*string, optional*) — Trailing suffix to remove from the result (e.g. an extension). Only stripped when it matches and is not the entire segment; a non-matching or empty suffix is ignored.

Returns: string — the last path segment, with suffix removed when it applies.

Throws: Throws a TypeError if path is missing, null, or undefined. suffix is coerced to a string and is optional.

```
const b = fs.path.basename("/var/log/app.log", ".log"); // "app"
```

17.7.9.2 fs.path.dirname

```
dirname(path: string): string
```

Directory portion of a path. POSIX-style; trailing slashes are stripped.

Parameters

- `path` (*string*) — A forward-slash path. On Windows, normalise separators yourself first.

Returns: string — everything up to (not including) the final slash; “.” when the path has no directory component, “/” for a rooted single segment.

Throws: Throws a TypeError if path is missing, null, or undefined.

```
const d = fs.path.dirname("/var/log/app.log"); // "/var/log"
```

17.7.10 fs.readBytes

```
readBytes(path: string): Promise<Uint8Array>
```

Read an entire file as bytes.

Parameters

- `path` (*string*) — File to read (CWD-relative or absolute).

Returns: Promise resolving to the file contents as a Uint8Array.

Throws: Rejects if path is missing/empty or the file cannot be read.

```
const bytes = await fs.readBytes("shot.png");
```

17.7.11 fs.readText

```
readText(path: string): Promise<string>
```

Read an entire file as a UTF-8 string.

Parameters

- `path` (*string*) — File to read (CWD-relative or absolute).

Returns: Promise resolving to the file contents decoded as UTF-8.

Throws: Rejects if path is missing/empty or the file cannot be read (absent, permissions).

```
const html = await fs.readText("report/index.html");
```

17.7.12 fs.remove

```
remove(path: string): Promise<{ path: string }>
```

Remove a file or a directory tree (recursive). No error if the path is already absent.

Parameters

- `path` (*string*) — File or directory to remove. Directories are removed recursively.

Returns: Promise resolving to { path }.

Throws: Rejects only if removal fails (e.g. permissions); removing an absent path is a no-op.

```
await fs.remove("report"); // clean slate
```

17.7.13 fs.stat

```
stat(path: string): Promise<{ size: number; isDir: boolean; modifiedMs: number }>
```

File metadata: size, whether it is a directory, and last-modified time.

Parameters

- `path` (*string*) — Path to stat.

Returns: Promise resolving to { size (bytes), isDir, modifiedMs (epoch milliseconds) }.

Throws: Rejects if path is missing/empty or the target does not exist.

```
const st = await fs.stat("report/index.html");
runtime.log(st.size, st.isDir, st.modifiedMs);
```

17.7.14 fs.tail

```
tail(path: string, opts?: { from?: "end" | "start" | number }):
AsyncIterable<string>
```

Follow a file and yield each new line (like `tail -f`) as an async iterator. Survives log rotation and truncation. `from` controls where reading starts: “end” (default, new lines only), “start” (whole file then follow), or a positive integer N (last N lines then follow). Long-lived: iterate with `for await`, `break` (or a Run timeout) stops it. Backed by a pure-Go fsnotify follower with a poll fallback.

Parameters

- `path` (*string*) — The file to follow. Must exist when the call is made (v1 throws otherwise).
- `opts` (*{ from?: “end” | “start” | number }, optional*) — from: “end” (default) delivers only lines appended after the call; “start” replays the whole file then follows; a positive integer N replays the last N lines then follows.

Returns: An async iterator yielding each new line as a string (trailing newline/\r stripped).

`for await (const line of fs.tail(path)) { ... } break` stops the follow.

Throws: Throws (“fs.tail: ...”) if path is missing/empty, the file does not exist at start, or `from` is not “end”/“start”/a non-negative number.

```
for await (const line of fs.tail("/var/log/app.log")) { runtime.log(line);
break; }
```

17.7.15 fs.writeBytes

```
writeBytes(path: string, data: Uint8Array): Promise<{ path: string; bytes:
number }>
```

Write binary data (a Uint8Array) to a file, truncating. Fails if the parent directory does not exist.

Parameters

- `path` (*string*) — Output file path (CWD-relative or absolute).
- `data` (*Uint8Array*) — Bytes to write (mode 0644). Pass a Uint8Array (e.g. `new Uint8Array(shot.bytes)`).

Returns: Promise resolving to { path, bytes } where bytes is the number of bytes written.

Throws: Rejects if path is missing/empty, data is not a Uint8Array, the parent directory does not exist, or the write fails.


```
const shot = await d.screenshot();
await fs.writeBytes("shot.png", new Uint8Array(shot.bytes));
```

17.7.16 fs.writeText

```
writeText(path: string, text: string): Promise<{ path: string; bytes: number }>
```

Write a string to a file (UTF-8, truncating). Fails if the parent directory does not exist — call `fs.mkdir` first.

Parameters

- `path` (*string*) — Output file path (CWD-relative or absolute). Used as given; no sandboxing.
- `text` (*string*) — Content to write as UTF-8. The file is created (mode 0644) or truncated.

Returns: Promise resolving to `{ path, bytes }` where `bytes` is the number of bytes written.

Throws: Rejects if path is missing/empty, text is not a string, the parent directory does not exist, or the write fails (permissions, etc.).

```
await fs.mkdir("report");
await fs.writeText("report/index.html", "<h1>Hi</h1>");
```

17.8 image

Image decode/encode (PNG/JPEG/GIF/TIFF/BMP/WebP, SVG rasterize-in) and a chainable Image handle: `resize/crop/rotate/flip/adjust/filter/compose`.

17.8.1 image.blur

```
blur(sigma: number): Image
```

Gaussian blur with the given sigma (larger sigma → softer). Returns a fresh Image.

Parameters

- `sigma` (*number*) — Gaussian blur sigma; larger blurs more.

Returns: Image — blurred.

Throws: Does not throw; a sigma ≤ 0 leaves the image effectively unchanged.

```
const soft = im.blur(2.0);
```

17.8.2 image.brightness

```
brightness(percent: number): Image
```

Adjust brightness by a percentage in `[-100, 100]`; positive brightens, negative darkens, 0 is a no-op. Returns a fresh Image.

Parameters

- `percent` (*number*) — Brightness change in percent, `-100` (black) .. `100` (white).

Returns: Image — brightness-adjusted.

Throws: Does not throw; values outside $[-100, 100]$ are clamped to the boundary (e.g. 150 behaves identically to 100).

```
const b = im.brightness(20);
```

17.8.3 image.bytes

```
bytes(format: 'png' | 'jpeg' | 'gif' | 'tiff' | 'bmp' | 'webp', opts?: { quality?: number }): Uint8Array
```

Encode the image to bytes in the given format. quality (jpeg) is 1..100, default 90. WebP encode is lossless — the quality option is accepted but ignored.

Parameters

- `format` (*'png' | 'jpeg' | 'gif' | 'tiff' | 'bmp' | 'webp'*) — Target encode format.
- `opts` (*{ quality?: number }, optional*) — Optional encode options. quality (1..100, default 90) applies to jpeg; ignored for png/gif/tiff/bmp and for the lossless webp encoder.

Returns: Uint8Array — the encoded image bytes.

Throws: Throws (“image: unsupported encode format ...”) for an unknown format, or (“image: webp encode: ...”) if WebP encoding fails.

```
const png = im.bytes("png");
```

17.8.4 image.contrast

```
contrast(percent: number): Image
```

Adjust contrast by a percentage in $[-100, 100]$; positive increases, negative flattens. Returns a fresh Image.

Parameters

- `percent` (*number*) — Contrast change in percent, $-100 .. 100$.

Returns: Image — contrast-adjusted.

Throws: Does not throw; values outside $[-100, 100]$ are clamped to the boundary (e.g. 150 behaves identically to 100).

```
const c = im.contrast(15);
```

17.8.5 image.crop

```
crop(x: number, y: number, w: number, h: number): Image
```

Crop a rectangular region at (x, y) of size $w \times h$. Coordinates are validated against the image bounds. Returns a fresh Image.

Parameters

- `x` (*number*) — Left edge of the crop rectangle, in pixels.
- `y` (*number*) — Top edge of the crop rectangle, in pixels.

- `w` (*number*) — Crop width in pixels (> 0).
- `h` (*number*) — Crop height in pixels (> 0).

Returns: Image — the cropped region.

Throws: Throws a `TypeError` when w/h are non-positive or the rectangle falls outside the image bounds.

```
const region = im.crop(10, 10, 100, 100);
```

17.8.6 image.decode

```
decode(data: Uint8Array, opts?: { autoOrient?: boolean }): { readonly width:
number; readonly height: number; readonly format: string; resize(width: number,
height: number, opts?: { filter?: "lanczos" | "nearest" | "linear" | "box" |
"catmullrom" }): unknown; fit(width: number, height: number): unknown;
thumbnail(width: number, height: number): unknown; crop(x: number, y: number, w:
number, h: number): unknown; rotate(degrees: number): unknown; rotate90():
unknown; rotate180(): unknown; rotate270(): unknown; flipH(): unknown; flipV():
unknown; orient(n: number): unknown; brightness(percent: number): unknown;
contrast(percent: number): unknown; gamma(gamma: number): unknown;
saturation(percent: number): unknown; sharpen(sigma: number): unknown; blur(sigma:
number): unknown; grayscale(): unknown; invert(): unknown; overlay(other: unknown,
x: number, y: number, opacity?: number): unknown; paste(other: unknown, x: number,
y: number): unknown; bytes(format: "png" | "jpeg" | "gif" | "tiff" | "bmp" |
"webp", opts?: { quality?: number }): Uint8Array; save(path: string, opts?:
{ format?: string; quality?: number }): void }
```

Decode in-memory image bytes into a chainable Image handle. The format is sniffed from the magic bytes (PNG/JPEG/GIF/TIFF/BMP/WebP); GIF decodes the first frame only.

Parameters

- `data` (*Uint8Array*) — The raw, encoded image bytes (e.g. from `net.http`, `fs`, or a clipboard read).
- `opts` (*{ autoOrient?: boolean }, optional*) — When `autoOrient` is true, the source's EXIF Orientation is read and the pixels are rotated upright; absent/unreadable Orientation is treated as a no-op (never throws).

Returns: Image — a handle exposing read-only width/height/format and chainable transform methods.

Throws: Throws a `TypeError` if data is not a `Uint8Array`, (“image.decode: ...”) if the bytes are not a recognised/decodable image, or if the declared width x height exceeds a hard 64-megapixel cap (a “decode bomb” guard; not configurable — see `DefaultMaxImagePixels`).

```
const im = image.decode(pngBytes, { autoOrient: true });
```

17.8.7 image.decodeFrames

```
decodeFrames(src: string | Uint8Array): { format: string; width: number; height:
number; loopCount: number; frames: { image: { readonly width: number; readonly
height: number; readonly format: string; resize(width: number, height: number,
opts?: { filter?: "lanczos" | "nearest" | "linear" | "box" | "catmullrom" }): }
```

```
unknown; fit(width: number, height: number): unknown; thumbnail(width: number,
height: number): unknown; crop(x: number, y: number, w: number, h: number):
unknown; rotate(degrees: number): unknown; rotate90(): unknown; rotate180():
unknown; rotate270(): unknown; flipH(): unknown; flipV(): unknown; orient(n:
number): unknown; brightness(percent: number): unknown; contrast(percent: number):
unknown; gamma(gamma: number): unknown; saturation(percent: number): unknown;
sharpen(sigma: number): unknown; blur(sigma: number): unknown; grayscale():
unknown; invert(): unknown; overlay(other: unknown, x: number, y: number,
opacity?: number): unknown; paste(other: unknown, x: number, y: number): unknown;
bytes(format: "png" | "jpeg" | "gif" | "tiff" | "bmp" | "webp", opts?: { quality?:
number }): Uint8Array; save(path: string, opts?: { format?: string; quality?:
number }): void }; delayMs: number; xOffset: number; yOffset: number; disposal:
"none" | "background" | "previous"; blend?: "source" | "over" }[] }
```

Decode all frames of an animated image (GIF or APNG) into a raw, normalized frame model. Frames are returned as stored (sub-rectangles, not composited) with per-frame timing/ placement metadata; the caller composites if needed. A non-animated image returns a single frame.

Parameters

- `src` (*string* / *Uint8Array*) — Image path or encoded bytes (GIF/APNG/PNG/JPEG/...).

Returns: An object with the container format/size/loopCount and a frames array; each frame's image is a chainable Image handle, delayMs the display time, xOffset/yOffset the placement, disposal the dispose method, and blend (APNG only) the blend op. loopCount 0 = loop forever.

Throws: Throws if the path can't be read or the bytes can't be decoded.

```
const a = image.decodeFrames("anim.gif");
runtime.log(a.format, a.frames.length, a.frames[0].delayMs);
```

17.8.8 image.encodeFrames

```
encodeFrames(spec: { width?: number; height?: number; loopCount?: number; frames:
{ image: Image; delayMs?: number; xOffset?: number; yOffset?: number; disposal?:
"none" | "background" | "previous"; blend?: "source" | "over" }[] }, opts?:
{ format?: "gif" | "apng"; dest?: string }): { format: string; bytes?: Uint8Array;
path?: string }
```

Encode a frame set into an animated GIF or APNG. Pass a spec shaped like decodeFrames' result (frames[], optional width/height/loopCount); choose the format via opts.format. GIF frames are palettized to 256 colors with Floyd–Steinberg dithering; APNG is full-color. Without opts.dest the encoded bytes are returned; with dest they're written to that path.

Parameters

- `spec` (*{ width?: number; height?: number; loopCount?: number; frames: { image: Image; delayMs?: number; xOffset?: number; yOffset?: number; disposal?: "none" | "background" | "previous"; blend?: "source" | "over" }[] }*) — The animation: a frames array (each with an Image handle + optional delayMs/offsets/disposal/blend) and optional canvas width/ height (derived from frame extents when omitted) and loopCount (default 0 = forever).

- `opts` (*{ format?: "gif" | "apng"; dest?: string }, optional*) — format selects the encoder (default gif); dest, when set, writes the file and returns its path instead of bytes.

Returns: { format, bytes } with the encoded animation, or { format, path } when `opts.dest` is set.

Throws: Throws if format is not gif/apng, if frames is empty, if a frame's image is not an Image handle, or on an encode/write failure.

```
const a = image.decodeFrames("in.gif");
const out = image.encodeFrames(a, { format: "apng" });
runtime.log(out.format, out.bytes.length);
```

17.8.9 image.exif

17.8.9.1 image.exif.clear

```
clear(src: string | Uint8Array, opts?: { dest?: string }): { format: string;
bytes: Uint8Array } | { format: string; path: string }
```

Remove all EXIF metadata from an image (JPEG/PNG only). Returns {format, bytes} with the stripped image bytes, or writes to {format, path} when `opts.dest` is given.

Parameters

- `src` (*string | Uint8Array*) — Image path or raw bytes. String values are read from disk; Uint8Array values are used directly.
- `opts` (*{ dest?: string }, optional*) — Optional. When `opts.dest` is a path, the result is written there and {format, path} is returned; otherwise {format, bytes} is returned.

Returns: {format, bytes} with a stripped image (no EXIF) when no dest, or {format, path} when `opts.dest` is given.

Throws: Throws if the format is not JPEG or PNG (unsupported write target), if src cannot be read, or if dest cannot be written.

```
const out = image.exif.clear(jpegBytes);
const e = image.exif.read(out.bytes); // e === {}
```

17.8.9.2 image.exif.read

```
read(src: string | Uint8Array): { image?: Record<string, unknown>; exif?:
Record<string, unknown>; gps?: Record<string, unknown>; thumbnail?: Record<string,
unknown> }
```

Read an image's EXIF metadata into a grouped-by-IFD object (image/exif/gps/thumbnail). JPEG/PNG/TIFF return all tags; HEIC/AVIF/RAW return a curated subset. Returns {} when the image has no EXIF.

Parameters

- `src` (*string | Uint8Array*) — Image path or raw bytes. String values are read from disk; Uint8Array values are parsed in-memory.

Returns: An object grouped by IFD; rationals serialised as [num,den] arrays, GPS latitude/longitude as signed decimals, dates as EXIF strings, binary/undefined-type values as base64.

Throws: Throws if the path cannot be read or the image bytes cannot be parsed.

```
const e = image.exif.read("photo.jpg");
runtime.log(e.image?.Make, e.gps?.GPSLatitude);
```

17.8.9.3 image.exif.replace

```
replace(src: string | Uint8Array, data: { image?: Record<string, unknown>; exif?:
Record<string, unknown>; gps?: Record<string, unknown>; thumbnail?: Record<string,
unknown> }, opts?: { dest?: string }): { format: string; bytes: Uint8Array } |
{ format: string; path: string }
```

Replace an image's entire EXIF block with the supplied tags (JPEG/PNG only). All pre-existing EXIF is discarded; only the tags in data are written. Returns {format, bytes} or writes to {format, path} when opts.dest is given.

Parameters

- **src** (*string* | *Uint8Array*) — Image path or raw bytes. String values are read from disk; Uint8Array values are used directly.
- **data** (*{ image?: Record<string, unknown>; exif?: Record<string, unknown>; gps?: Record<string, unknown>; thumbnail?: Record<string, unknown> }*) — The complete new EXIF block grouped by IFD. Any tag not listed is absent from the output.
- **opts** (*{ dest?: string }*, optional) — Optional. When opts.dest is a path, the result is written there and {format, path} is returned; otherwise {format, bytes} is returned.

Returns: {format, bytes} with the updated image bytes when no dest, or {format, path} when opts.dest is given.

Throws: Throws if the format is not JPEG or PNG (unsupported write target), if src cannot be read, or if dest cannot be written.

```
const out = image.exif.replace(jpegBytes, { image: { Make: "sercon" } });
const e = image.exif.read(out.bytes); // e.image.Make === "sercon"
```

17.8.9.4 image.exif.write

```
write(src: string | Uint8Array, data: { image?: Record<string, unknown>; exif?:
Record<string, unknown>; gps?: Record<string, unknown>; thumbnail?: Record<string,
unknown> }, opts?: { dest?: string }): { format: string; bytes: Uint8Array } |
{ format: string; path: string }
```

Merge EXIF tags into an image (JPEG/PNG only). Existing tags not mentioned in data are preserved; pass null for a tag value to delete that tag. Returns {format, bytes} or writes to {format, path} when opts.dest is given.

Parameters

- **src** (*string* | *Uint8Array*) — Image path or raw bytes. String values are read from disk; Uint8Array values are used directly.

- `data` (*{ image?: Record<string, unknown>; exif?: Record<string, unknown>; gps?: Record<string, unknown>; thumbnail?: Record<string, unknown> }*) — Tags to merge, grouped by IFD. A null value deletes that tag from the output.
- `opts` (*{ dest?: string }, optional*) — Optional. When `opts.dest` is a path, the result is written there and `{format, path}` is returned; otherwise `{format, bytes}` is returned.

Returns: `{format, bytes}` with the updated image bytes when no `dest`, or `{format, path}` when `opts.dest` is given.

Throws: Throws if the format is not JPEG or PNG (unsupported write target), if `src` cannot be read, or if `dest` cannot be written.

```
const out = image.exif.write(jpegBytes, { image: { Artist: "Alice" } });
// out.bytes is the updated JPEG
```

17.8.10 image.fit

```
fit(width: number, height: number): Image
```

Scale the image down to fit entirely within a width × height box, preserving aspect ratio (no cropping; may letterbox conceptually). Returns a fresh Image.

Parameters

- `width` (*number*) — Maximum width of the bounding box, in pixels.
- `height` (*number*) — Maximum height of the bounding box, in pixels.

Returns: Image — the largest image that fits within the box at the original aspect ratio.

Throws: Does not throw; non-positive or zero dimensions produce a degenerate (possibly empty) image rather than an error.

```
const fitted = im.fit(800, 600);
```

17.8.11 image.flipH

```
flipH(): Image
```

Flip horizontally (mirror left↔right). Returns a fresh Image.

Returns: Image — horizontally mirrored.

Throws: Does not throw.

```
const m = im.flipH();
```

17.8.12 image.flipV

```
flipV(): Image
```

Flip vertically (mirror top↔bottom). Returns a fresh Image.

Returns: Image — vertically mirrored.

Throws: Does not throw.

```
const m = im.flipV();
```

17.8.13 image.gamma

```
gamma(gamma: number): Image
```

Apply gamma correction. $\text{gamma} < 1$ darkens, $\text{gamma} > 1$ brightens, 1.0 is a no-op. Returns a fresh Image.

Parameters

- `gamma` (*number*) — Gamma factor (> 0). 1.0 leaves the image unchanged.

Returns: Image — gamma-corrected.

Throws: Does not throw; pass a positive gamma (a non-positive value yields an all-black or undefined result rather than an error).

```
const g = im.gamma(1.2);
```

17.8.14 image.grayscale

```
grayscale(): Image
```

Convert to grayscale (luminance). Returns a fresh Image.

Returns: Image — grayscale.

Throws: Does not throw.

```
const g = im.grayscale();
```

17.8.15 image.invert

```
invert(): Image
```

Invert colours (photographic negative). Returns a fresh Image.

Returns: Image — colour-inverted.

Throws: Does not throw.

```
const n = im.invert();
```

17.8.16 image.open

```
open(path: string, opts?: { autoOrient?: boolean }): { readonly width: number;
readonly height: number; readonly format: string; resize(width: number, height:
number, opts?: { filter?: "lanczos" | "nearest" | "linear" | "box" |
"catmullrom" }): unknown; fit(width: number, height: number): unknown;
thumbnail(width: number, height: number): unknown; crop(x: number, y: number, w:
number, h: number): unknown; rotate(degrees: number): unknown; rotate90():
unknown; rotate180(): unknown; rotate270(): unknown; flipH(): unknown; flipV():
unknown; orient(n: number): unknown; brightness(percent: number): unknown;
```



```
contrast(percent: number): unknown; gamma(gamma: number): unknown;
saturation(percent: number): unknown; sharpen(sigma: number): unknown; blur(sigma:
number): unknown; grayscale(): unknown; invert(): unknown; overlay(other: unknown,
x: number, y: number, opacity?: number): unknown; paste(other: unknown, x: number,
y: number): unknown; bytes(format: "png" | "jpeg" | "gif" | "tiff" | "bmp" |
"webp", opts?: { quality?: number }): Uint8Array; save(path: string, opts?:
{ format?: string; quality?: number }): void }
```

Read an image file from disk and decode it into a chainable Image handle. The format is sniffed from the file's magic bytes (PNG/JPEG/GIF/TIFF/BMP/WebP), not the extension. GIF decodes the first frame only.

Parameters

- `path` (*string*) — Filesystem path to the image file to read and decode.
- `opts` (*{ autoOrient?: boolean }, optional*) — When `autoOrient` is true, the source's EXIF Orientation is read and the pixels are rotated upright; absent/unreadable Orientation is treated as a no-op (never throws).

Returns: Image — a handle exposing read-only width/height/format and chainable transform methods.

Throws: Throws ("image.open: ...") if the file cannot be read, ("image.decode: ...") if the bytes are not a recognised/decodable image, or if the declared width x height exceeds a hard 64-megapixel cap (a "decode bomb" guard; not configurable).

```
const im = image.open("avatar.jpg", { autoOrient: true });
```

17.8.17 image.orient

```
orient(n: number): Image
```

Apply one of the 8 EXIF orientations to the raster pixels and return a fresh Image. `n` is the EXIF Orientation value (1=normal, 2=mirror-H, 3=180°, 4=mirror-V, 5/7=transpose/transverse, 6=90°CW, 8=90°CCW). 1 is a no-op copy. Pure raster — no EXIF is read (use `open/decode { autoOrient: true }` to drive this from a file's tag).

Parameters

- `n` (*number*) — EXIF orientation value, an integer 1..8.

Returns: Image — the reoriented raster (dimensions swap for `n` in 5..8).

Throws: Throws a `TypeError` if `n` is not an integer in 1..8.

```
const up = image.decode(bytes).orient(6); // 90° clockwise
```

17.8.18 image.overlay

```
overlay(other: Image, x: number, y: number, opacity?: number): Image
```

Composite another Image on top of this one at (`x`, `y`) with an optional opacity (alpha-blended). Returns a fresh Image the size of the base.

Parameters

- `other` (*Image*) — The overlay Image handle to draw on top.
- `x` (*number*) — Left offset of the overlay within the base, in pixels.
- `y` (*number*) — Top offset of the overlay within the base, in pixels.
- `opacity` (*number, optional*) — Blend opacity 0..1 (default 1.0 = fully opaque).

Returns: Image — the base with the overlay alpha-blended on top.

Throws: Throws a `TypeError` if `other` is not an Image handle.

```
const composed = base.overlay(watermark, 10, 10, 0.5);
```

17.8.19 image.paste

```
paste(other: Image, x: number, y: number): Image
```

Paste another Image at (x, y), replacing the base pixels (no blending). Returns a fresh Image the size of the base.

Parameters

- `other` (*Image*) — The Image handle to paste in.
- `x` (*number*) — Left offset of the paste, in pixels.
- `y` (*number*) — Top offset of the paste, in pixels.

Returns: Image — the base with other pasted over it (opaque).

Throws: Throws a `TypeError` if `other` is not an Image handle.

```
const stamped = base.paste(logo, 0, 0);
```

17.8.20 image.rasterizeSVG

```
rasterizeSVG(src: string | Uint8Array, opts: { width: number; height: number }):  
{ readonly width: number; readonly height: number; readonly format: string;  
resize(width: number, height: number, opts?: { filter?: "lanczos" | "nearest" |  
"linear" | "box" | "catmullrom" }): unknown; fit(width: number, height: number):  
unknown; thumbnail(width: number, height: number): unknown; crop(x: number, y:  
number, w: number, h: number): unknown; rotate(degrees: number): unknown;  
rotate90(): unknown; rotate180(): unknown; rotate270(): unknown; flipH(): unknown;  
flipV(): unknown; orient(n: number): unknown; brightness(percent: number):  
unknown; contrast(percent: number): unknown; gamma(gamma: number): unknown;  
saturation(percent: number): unknown; sharpen(sigma: number): unknown; blur(sigma:  
number): unknown; grayscale(): unknown; invert(): unknown; overlay(other: unknown,  
x: number, y: number, opacity?: number): unknown; paste(other: unknown, x: number,  
y: number): unknown; bytes(format: "png" | "jpeg" | "gif" | "tiff" | "bmp" |  
"webp", opts?: { quality?: number }): Uint8Array; save(path: string, opts?:  
{ format?: string; quality?: number }): void }
```

Rasterize an SVG (a supported subset) to a raster Image at the requested pixel size. SVG is rasterize-IN only — there is no SVG output; the result is a raster image you then encode as PNG/JPEG/etc. The accepts a path string or in-memory SVG bytes.

Parameters

- `src` (*string* / *Uint8Array*) — An SVG file path (string) or the raw SVG document bytes (Uint8Array).
- `opts` (*{ width: number; height: number }*) — Target raster size in pixels; both width and height are required and must be > 0.

Returns: Image — a raster handle (format reported as “svg”) sized to `opts.width × opts.height`.

Throws: Throws a `TypeError` if `src` is neither a string nor `Uint8Array`, or if width/height are missing/non-positive; throws (“image: svg parse: ...”) on a malformed/unsupported SVG.

```
const im = image.rasterizeSVG("logo.svg", { width: 256, height: 256 });
```

17.8.21 image.resize

```
resize(width: number, height: number, opts?: { filter?: 'lanczos' | 'nearest' | 'linear' | 'box' | 'catmullrom' }): Image
```

Resize to width × height with a resampling filter (default Lanczos). Passing 0 for one dimension preserves the aspect ratio (the other dimension is computed). Returns a fresh Image (immutable chaining).

Parameters

- `width` (*number*) — Target width in pixels. 0 means “derive from height to preserve aspect”.
- `height` (*number*) — Target height in pixels. 0 means “derive from width to preserve aspect”.
- `opts` (*{ filter?: 'lanczos' | 'nearest' | 'linear' | 'box' | 'catmullrom' }, optional*) — Optional resampling filter; defaults to lanczos.

Returns: Image — the resized image.

Throws: Does not throw on dimension; an unknown filter name falls back to lanczos.

```
const thumb = im.resize(200, 0); // width 200, height auto
```

17.8.22 image.rotate

```
rotate(degrees: number): Image
```

Rotate counter-clockwise by an arbitrary angle (degrees) about the centre, filling exposed corners with transparency. Returns a fresh Image (its bounds grow to contain the rotation).

Parameters

- `degrees` (*number*) — Rotation angle in degrees, counter-clockwise.

Returns: Image — the rotated image with transparent fill in the new corners.

Throws: Does not throw; any finite angle is accepted (multiples of 360 are a no-op).

```
const tilted = im.rotate(15);
```

17.8.23 image.rotate180

```
rotate180(): Image
```

Rotate 180° (lossless). Returns a fresh Image.

Returns: Image — rotated 180°.

Throws: Does not throw.

```
const r = im.rotate180();
```

17.8.24 image.rotate270

```
rotate270(): Image
```

Rotate 270° counter-clockwise / 90° clockwise (lossless). Returns a fresh Image.

Returns: Image — rotated 270° CCW.

Throws: Does not throw.

```
const r = im.rotate270();
```

17.8.25 image.rotate90

```
rotate90(): Image
```

Rotate 90° counter-clockwise (lossless, no resampling). Returns a fresh Image.

Returns: Image — rotated 90° CCW.

Throws: Does not throw.

```
const r = im.rotate90();
```

17.8.26 image.saturation

```
saturation(percent: number): Image
```

Adjust colour saturation by a percentage in $[-100, 100]$; -100 is fully desaturated (grayscale), positive boosts. Returns a fresh Image.

Parameters

- `percent` (*number*) — Saturation change in percent, -100 (gray) .. 100 .

Returns: Image — saturation-adjusted.

Throws: Does not throw; values outside $[-100, 100]$ are clamped to the boundary (e.g. 150 behaves identically to 100).

```
const s = im.saturation(30);
```

17.8.27 image.save

```
save(path: string, opts?: { format?: string; quality?: number }): void
```

Encode and write the image to a file. The format is inferred from the path extension unless `opts.format` overrides it. quality (jpeg) is 1..100, default 90; WebP is lossless (quality ignored).

Parameters

- `path` (*string*) — Destination file path; its extension drives the format unless `opts.format` is set.
- `opts` (*{ format?: string; quality?: number }, optional*) — Optional: override the encode format and/or set jpeg quality (1..100).

Returns: void — writes the encoded image to disk.

Throws: Throws (“image: cannot infer format from path ...”) when the extension is unknown and no `opts.format` is given, or (“image.save: ...”) if the file cannot be written.

```
im.resize(64, 64).save("out.png");
```

17.8.28 image.sharpen

```
sharpen(sigma: number): Image
```

Sharpen via an unsharp-mask whose strength is the sigma of the underlying Gaussian. Returns a fresh Image.

Parameters

- `sigma` (*number*) — Sharpening strength (Gaussian sigma); larger is stronger.

Returns: Image — sharpened.

Throws: Does not throw; a sigma <= 0 leaves the image effectively unchanged.

```
const s = im.sharpen(1.0);
```

17.8.29 image.stego

17.8.29.1 image.stego.analyze

```
analyze(carrier: string | Uint8Array): { width: number; height: number; capacity: number; estimatedBits: number; sercon: { present: boolean; encrypted?: boolean; text?: boolean; payloadBytes?: number; bits?: number }; channels: { channel: string; chiSquare: number; lsbEntropy: number; rsEstimate: number; chiSquareByBits: number[]; entropyByPlane: number[] }[]; verdict: { suspicious: boolean; confidence: number; reasons: string[] } }
```

Run a full LSB-steganalysis report on an image: per-channel chi-square (pairs-of-values) probability, LSB-plane Shannon entropy, and an RS embedding-rate estimate, plus a combined verdict with reasons. Read-only. The report includes a generalized chi-square at depths 1..4 (`chiSquareByBits`), per-plane entropy (`entropyByPlane`, planes 0..3), and an `estimatedBits` figure for the likely embedding depth.

Parameters

- `carrier` (*string* | *Uint8Array*) — The image to analyze: a file path or raw image bytes.

Returns: A report object: capacity in bytes, the sercon-header check, per-channel signals (chiSquare/lsbEntropy/rsEstimate, each 0..1), and the verdict. `estimatedBits` is a coarse, best-effort hint at the embedding depth — it needs substantial coverage to register and returns 0 for small or partial payloads. For sercon-format payloads the authoritative depth is `sercon.bits` (set by the header); `chiSquareByBits` and `entropyByPlane` are the raw per-depth diagnostics.

Throws: Throws if the carrier cannot be decoded.

```
const r = image.stego.analyze("suspect.png"); console.log(r.verdict.suspicious, r.verdict.reasons);
```

17.8.29.2 image.stego.bitplane

```
bitplane(carrier: string | Uint8Array, opts?: { channel?: "r" | "g" | "b" | "rgb"; plane?: number; dest?: string }): { bytes: Uint8Array } | { path: string }
```

Render one bit-plane of an image as a PNG for visual inspection. Hidden LSB data often shows as structured noise in the low planes. For a single channel a set bit is white and an unset bit black; for “rgb” each channel’s bit maps to its colour component.

Parameters

- `carrier` (*string* | *Uint8Array*) — The image: a file path or raw image bytes.
- `opts` (*{ channel?: “r” | “g” | “b” | “rgb”; plane?: number; dest?: string }, optional*) — `channel` selects the colour channel (default “rgb” composite); `plane` is the bit index 0 (LSB) to 7 (MSB), default 0; `dest` writes the PNG to that path instead of returning bytes.

Returns: An object with the PNG bytes (`{ bytes }`), or `{ path }` when `opts.dest` was given.

Throws: Throws a `TypeError` for an unknown channel or a plane outside 0..7, throws if the carrier cannot be decoded, or if writing `opts.dest` fails.

```
const lsb = image.stego.bitplane("photo.png", { plane: 0 });
```

17.8.29.3 image.stego.capacity

```
capacity(carrier: string | Uint8Array, opts?: { bits?: number }): { bytes: number; bits: number }
```

Report the maximum payload size (in bytes) a carrier can hold, after the fixed 10-byte header — at the requested bit depth (1..4, default 1; one bit per R/G/B channel at the default). Encryption adds roughly 44 bytes of overhead (salt + nonce + auth tag), so the effective capacity for an encrypted payload is correspondingly lower.

Parameters

- `carrier` (*string* | *Uint8Array*) — The carrier image: a file path or raw image bytes.
- `opts` (*{ bits?: number }, optional*) — `bits`: report capacity at this depth (integer 1..4, default 1).

Returns: An object: bytes is the maximum plaintext payload size at the requested depth; bits echoes that depth.

Throws: Throws if the carrier cannot be decoded, or a `TypeError` if bits is not an integer 1..4.

```
const room = image.stego.capacity("cover.png", { bits: 4 }).bytes;
```

17.8.29.4 image.stego.detect

```
detect(carrier: string | Uint8Array): { sercon: boolean; encrypted?: boolean;  
text?: boolean; payloadBytes?: number; bits?: number; suspicious: boolean;  
confidence: number }
```

Quickly check whether an image carries hidden data. Returns a definitive flag for a sercon-format LSB payload (the “ScSt” header) plus a heuristic suspicion verdict from statistical analysis. Read-only; never modifies the image.

Parameters

- `carrier` (*string* / *Uint8Array*) — The image to inspect: a file path or raw image bytes.

Returns: An object: sercon is true when a sercon stego header is present (with encrypted/text/payloadBytes); suspicious/confidence summarize the statistical analysis. bits is the declared payload depth when a sercon header is present.

Throws: Throws if the carrier cannot be decoded. Never throws for a clean image.

```
if (image.stego.detect("photo.png").sercon) console.log("hidden payload!");
```

17.8.29.5 image.stego.embed

```
embed(carrier: string | Uint8Array, payload: string | Uint8Array, opts?:  
{ password?: string; dest?: string; bits?: number }): { bytes: Uint8Array } |  
{ path: string }
```

Hide a payload inside a lossless image using least-significant-bit (LSB) steganography, returning PNG bytes. The carrier is decoded (PNG/JPEG/GIF/TIFF/BMP/WebP) and re-encoded as PNG (output is always PNG — re-encoding to a lossy format would destroy the hidden data). One bit is stored per R/G/B channel; the alpha channel is never modified (opaque carriers recommended). A non-empty password encrypts the payload with AES-256-GCM (PBKDF2-SHA256 key derivation). One to four bits are stored per R/G/B channel (the `bits` option, default 1); higher values raise capacity but make the change more visible and easier to detect.

Parameters

- `carrier` (*string* / *Uint8Array*) — The carrier image: a file path (string) or raw image bytes (Uint8Array).
- `payload` (*string* / *Uint8Array*) — The data to hide. A string is stored as UTF-8 and marked as text (extract returns a string); a Uint8Array is stored as binary (extract returns a Uint8Array).

- `opts` (*{ password?: string; dest?: string; bits?: number }, optional*) — `password`: encrypt the payload with AES-256-GCM. `dest`: write the resulting PNG to this path instead of returning its bytes. `bits`: payload bits per channel, an integer 1..4 (default 1).

Returns: An object with the PNG bytes (`{ bytes }`), or `{ path }` when `opts.dest` was given.

Throws: Throws if the carrier cannot be decoded, if the payload is neither a string nor `Uint8Array`, if the payload exceeds the carrier capacity (“payload too large (need N bytes, capacity M)”), or if writing `opts.dest` fails.

```
const out = image.stego.embed("cover.png", "meet at noon", { password:
"s3cret" });
```

17.8.29.6 image.stego.extract

```
extract(carrier: string | Uint8Array, opts?: { password?: string }): string |
Uint8Array
```

Recover a payload previously hidden by `image.stego.embed`. Reads the LSB stream, verifies the sercon stego header, and returns the payload as a string (if it was embedded as text) or a `Uint8Array` (if binary). If the payload was encrypted, the same password must be supplied; a wrong password fails the authentication check. The bit depth is read from the header, so no `bits` argument is needed.

Parameters

- `carrier` (*string | Uint8Array*) — The stego image (must be the lossless PNG produced by `embed`, or an identical copy): a file path or raw bytes.
- `opts` (*{ password?: string }, optional*) — The password used at embed time, required when the payload was encrypted.

Returns: The recovered payload — a string when embedded as text, otherwise a `Uint8Array`.

Throws: Throws if the carrier cannot be decoded, if no sercon stego payload is present (“no sercon stego payload found”), if the payload is truncated, if the payload is encrypted but no password is given, or if decryption fails (“wrong password or corrupt data”).

```
const msg = image.stego.extract("cover.png", { password: "s3cret" });
```

17.8.30 image.thumbnail

```
thumbnail(width: number, height: number): Image
```

Produce an exactly `width × height` thumbnail by scaling and centre-cropping to fill the box (aspect ratio preserved, overflow cropped). Returns a fresh `Image`.

Parameters

- `width` (*number*) — Output width in pixels.
- `height` (*number*) — Output height in pixels.

Returns: `Image` — a `width × height` thumbnail, centre-cropped to fill.

Throws: Does not throw; non-positive dimensions produce a degenerate image rather than an error.

```
const sq = im.thumbnail(128, 128);
```

17.9 mcp

Model Context Protocol server: `mcp.serve({name, version, instructions?})` returns a handle for registering tools/resources/prompts with JS handlers, then serving them over `stdio()` (Unix-only this phase) or `listen()` (Streamable HTTP, cross-platform). Built on the official `modelcontextprotocol/go-sdk`.

17.9.1 mcp.connect

17.9.1.1 mcp.connect.complete

```
complete(ref: { type: "prompt" | "resource"; name?: string; uri?: string },
  argName: string, partial: string): Promise<{ values: string[]; total?: number;
  hasMore?: boolean }>
```

Ask the server for argument-autocompletion suggestions (completion/complete) — the client-side counterpart to `serve.completion`'s handler. Useful for building an interactive prompt-argument or resource-template-URI-variable picker against a remote server without guessing valid values yourself.

Parameters

- `ref` (*{ type: "prompt" | "resource"; name?: string; uri?: string }*) — identifies what's being completed: type "prompt" completes an argument of a prompt the server registered (name: the prompt's name) or type "resource" completes a URI-template variable of a resource template the server registered (uri: the template's `uriTemplate` string, not a concrete URI). Set only the field matching type; the other is omitted.
- `argName` (*string*) — the argument name (for a prompt ref) or the URI template variable name (for a resource ref) being completed.
- `partial` (*string*) — the text typed so far; pass "" to ask for all suggestions with no filtering applied yet.

Returns: A promise resolving to the server's suggestions: `values` is the ordered suggestion list (possibly empty — no match is not an error); `total`, when present, is the server's estimate of the full match count (may exceed `values.length`); `hasMore`, when present, signals more results exist beyond this page (the SDK does not expose a way to fetch the next page — a narrower `partial` is the only lever).

Throws: Throws synchronously if `ref` is missing/not an object, `ref.type` is neither "prompt" nor "resource", or `argName` is missing/empty. The returned promise rejects if the connection is closed/dead or the server rejects the request as a protocol error (e.g. it doesn't support the completions capability, or `ref` names something it never registered).

```
const c = await mcp.connect.stdio({ command: ["sercon", "server.ts"] });
const suggestions = await c.complete({ type: "prompt", name: "greet" }, "user",
  "ali");
```

```
runtime.log(suggestions.values.join(", ")); // e.g. "alice, alicia"
await c.close();
```

17.9.1.2 mcp.connect.http

```
http(url: string, opts?: { headers?: Record<string, string>; maxRetries?: number;
auth?: { getToken(): string | Promise<string> }; onSample?: (req: { messages:
Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }) => string | { content:
{ type: string; text: string } | string; model?: string; stopReason?: string;
role?: string } | Promise<string | { content: { type: string; text: string } |
string; model?: string; stopReason?: string; role?: string }>; onElicit?: (req:
{ message: string; requestedSchema: Record<string, unknown>; mode?: string }) =>
{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> } |
Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string,
unknown> }>; roots?: Array<{ uri: string; name?: string }>; handlerTimeout?:
number }): Promise<{
  serverInfo: { name: string; version: string; title?: string };
  capabilities: Record<string, unknown>;
  listTools(): Promise<Array<{ name: string; title?: string; description?: string;
inputSchema: Record<string, unknown>; outputSchema?: Record<string, unknown> }>>;
  callTool(name: string, args?: Record<string, unknown>): Promise<{ content:
Array<{ type: string; text?: string; data?: string; mimeType?: string; uri?:
string; resource?: { uri: string; mimeType?: string; text?: string; blob?:
string } }>; structuredContent?: unknown; isError: boolean }>;
  listResources(): Promise<Array<{ uri: string; name: string; title?: string;
description?: string; mimeType?: string }>>;
  listResourceTemplates(): Promise<Array<{ uriTemplate: string; name: string;
title?: string; description?: string; mimeType?: string }>>;
  readResource(uri: string): Promise<{ contents: Array<{ uri: string; mimeType?:
string; text?: string; blob?: string } }>>;
  listPrompts(): Promise<Array<{ name: string; title?: string; description?:
string; arguments?: Array<{ name: string; title?: string; description?: string;
required: boolean } }>>>;
  getPrompt(name: string, args?: Record<string, string>): Promise<{ description?:
string; messages: Array<{ role: string; content: { type: string; text?: string;
data?: string; mimeType?: string; uri?: string } } }>>;
  ping(): Promise<void>;
  close(): Promise<void>;
  onToolsChanged(fn: () => void): void;
  onResourcesChanged(fn: () => void): void;
  onPromptsChanged(fn: () => void): void;
  onResourceUpdated(fn: (uri: string) => void): void;
  onLoggingMessage(fn: (message: { level: string; logger?: string; data:
unknown }) => void): void;
  onProgress(fn: (progress: { progressToken: string | number; progress: number;
total?: number; message?: string }) => void): void;
  subscribe(uri: string): Promise<void>;
  unsubscribe(uri: string): Promise<void>;
  setLoggingLevel(level: string): Promise<void>;
  complete(ref: { type: "prompt" | "resource"; name?: string; uri?: string },
argName: string, partial: string): Promise<{ values: string[]; total?: number;
hasMore?: boolean }>;
```

```
setRoots(roots: Array<{ uri: string; name?: string }>): void;
}>
```

Connect to an MCP server as a client over the Streamable HTTP transport — the cross-platform counterpart to `connect.stdio`, talking to a server already listening (e.g. one started with `srv.listen(...)`) instead of launching a subprocess. Same Phase 2/Phase 3 scope note as `connect.stdio`: change notifications, subscriptions, logging, and completion are supported (see the ten `onXxx/subscribe/unsubscribe/setLoggingLevel/complete` entries below), and `onSample/onElicit/roots` (see `setRoots`) let this connection act as a host for the server’s own requests. Phase 4 (current) adds `maxRetries` (a reconnect cap for this transport) and `auth` (an OAuth 2.1 bearer-token client) — see below.

Parameters

- `url` (*string*) — the server’s absolute MCP endpoint URL, e.g. “`http://127.0.0.1:38080/mcp`” (must be `http` or `https` with a host; anything else throws synchronously).
- `opts` (*{ headers?: Record<string, string>; maxRetries?: number; auth?: { getToken(): string | Promise<string> }; onSample?: (req: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number; intelligencePriority?: number; speedPriority?: number } }) => string | { content: { type: string; text: string } | string; model?: string; stopReason?: string; role?: string } | Promise<string | { content: { type: string; text: string } | string; model?: string; stopReason?: string; role?: string }>; onElicit?: (req: { message: string; requestedSchema: Record<string, unknown>; mode?: string }) => { action: “accept” | “decline” | “cancel”; content?: Record<string, unknown> } | Promise<{ action: “accept” | “decline” | “cancel”; content?: Record<string, unknown> }>; roots?: Array<{ uri: string; name?: string }>; handlerTimeout?: number }*, optional) — optional. `headers`: extra HTTP headers sent with every request on this connection (e.g. a bearer token for a `listen({auth})` protected server, as an alternative to `auth` below) — merged with `sercon`’s default `sercon-mcp/<version> User-Agent`, which `headers` may override. `maxRetries`: caps how many times the Streamable-HTTP transport’s own SSE-stream reconnect logic retries after a dropped connection; 0 or a negative number disables reconnection entirely (a single drop ends the session), omitted leaves the `go-sdk`’s built-in default (5) in place. Streamable-HTTP-only — `connect.sse`’s legacy SSE transport has no equivalent knob. `auth`: turns this connection into an OAuth 2.1 bearer-token client — the client-side counterpart to `serve.listen`’s `auth` (resource-server) option. `auth.getToken()` is called once before the initial request and again whenever a request comes back 401/403, so the script can refresh an expired token; it may return a plain string or a `Promise<string>` and MUST resolve to a non-empty string — the returned value is sent verbatim as an `Authorization: Bearer <token>` header on every request. `getToken` runs on-loop (the same bridge tool/resource/prompt handlers use), so it may safely call other `sercon` bindings (e.g. `net.http` for a client-credentials token endpoint) or read a value cached from an earlier step. `auth` and `headers.Authorization` are independent — setting both means `auth`’s bearer header is applied via the transport’s own OAuth plumbing while `headers` still layers in anything else you supply (do not also set `headers.Authorization` yourself; the two would conflict). `onSample(req)`: answers the server’s sampling/createMessage requests (`ctx.sample()` on the server side); `req` carries `messages/maxTokens/systemPrompt/temperature/stopSequences/includeContext/`

modelPreferences. May return a plain string (wrapped as text content, model “sercon”, role “assistant”, stopReason “endTurn”) or an object giving explicit control over content/model/stopReason/role; sync or async. onElicit(req): answers the server’s elicitation/create requests (ctx.elicit() on the server side); req carries { message, requestedSchema, mode? }. Must return { action: “accept”|“decline”|“cancel”, content? }; sync or async. roots: seeds this client’s filesystem/URI roots — an array of { uri, name? } — via AddRoots before the connection is established, so the server’s first roots/list sees them immediately. IMPORTANT: the roots capability is advertised by the underlying SDK unconditionally regardless of whether this option is given (unlike onSample/onElicit, each advertised only when provided — see Errors); omitting roots just means the initial set is empty, still updatable later via setRoots. handlerTimeout (milliseconds) caps how long a host responder (onSample/onElicit) or the OAuth getToken callback may run before the on-loop bridge returns an error; default 5 minutes, 0 disables.

Returns: Same handle shape as connect.studio: a promise that resolves once the initialize handshake completes to a session handle with serverInfo/capabilities, the listTools/callTool/listResources/listResourceTemplates/readResource/listPrompts/getPrompt/ping/close methods, the subscribe/unsubscribe/setLoggingLevel/complete mid-connection calls, the six onXxx notification setters, and setRoots(roots) to update the seeded roots set at runtime. Holds the script’s event loop open for the connection’s lifetime.

Throws: Throws synchronously if url is missing or not an absolute http(s) URL, maxRetries is present but not a number, auth is present but auth.getToken is not a function, onSample/onElicit are present but not functions, or a roots entry is missing a non-empty uri. The returned promise rejects if the HTTP connection or initialize handshake fails (wrapped as “mcp.connect: ...”) — including a 401 from a listen({auth})-protected server when neither opts.headers nor opts.auth carries a valid bearer token, or if auth.getToken throws/rejects, resolves to a non-string, or resolves to an empty string (“mcp.connect: auth.getToken must return a string”/“...a non-empty token string”). Capability gating happens at connect time, not as a throw here: the SDK client advertises sampling/elicitation if and only if onSample/onElicit (respectively) is provided — a server calling ctx.sample()/ctx.elicit() against a client that omitted the matching responder gets that server-side call rejected, not this connect call.

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");
runtime.log("connected to", c.serverInfo.name);
const tools = await c.listTools();
runtime.log(tools.map(t => t.name).join(", "));
await c.ping();
await c.close();

// Against an OAuth-protected listener via a static header (see serve.listen's
// auth option):
const authed = await mcp.connect.http("http://127.0.0.1:38081/mcp", {
  headers: { Authorization: "Bearer good-token" },
});
await authed.close();

// Same, but via auth.getToken (re-invoked on 401/403 to refresh):
let cachedToken = "";
const oauthed = await mcp.connect.http("http://127.0.0.1:38081/mcp", {
```

```

    auth: {
      async getToken() {
        if (!cachedToken) cachedToken = "good-token"; // e.g. fetch via net.http
        client-credentials
        return cachedToken;
      },
    },
  });
  await oauthed.close();

  // Disable the transport's automatic reconnect:
  const noRetry = await mcp.connect.http("http://127.0.0.1:38080/mcp", { maxRetries:
  0 });
  await noRetry.close();

  // As a host: answer the server's sampling/elicitation requests and seed roots.
  const host = await mcp.connect.http("http://127.0.0.1:38080/mcp", {
    onSample: async (req) => {
      // wire this to services.ai to answer with a real LLM instead of an echo:
      // const r = await services.ai.send({ prompt: req.messages[0].content.text });
      // return r.output;
      return "SUMMARY: " + req.messages[0].content.text;
    },
    onElicit: (req) => ({ action: "accept", content: { confirmed: true } }),
    roots: [{ uri: "file:///workspace", name: "workspace" }],
  });
  host.setRoots([
    { uri: "file:///workspace2", name: "workspace2" }
  ]);
  await host.close();

```

17.9.1.3 mcp.connect.onLoggingMessage

```

onLoggingMessage(fn: (message: { level: string; logger?: string; data: unknown })
=> void): void

```

Register the callback invoked for every server log message this connection has opted into (notifications/message) — see `connect.setLoggingLevel`, which this callback depends on. Registering `onLoggingMessage` alone, without ever calling `setLoggingLevel`, receives nothing: the server (per the MCP spec) does not send log messages until the client explicitly requests a level. Only one `onLoggingMessage` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn` (`((message: { level: string; logger?: string; data: unknown }) => void)`) — invoked once per log message: `level` is the severity the server tagged it with (one of the same eight RFC 5424 names accepted by `setLoggingLevel`); `logger`, when the server set one, names the logging source; `data` is whatever JSON-serializable value the server's `ctx.log(level, message, data?)` call passed (commonly a string, but may be an object). Its return value (and any thrown error or rejection) is ignored.

Returns: Nothing — replaces any previously registered `onLoggingMessage` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
c.onLoggingMessage((message) => {
  runtime.log(`[${message.level}]`, message.logger ?? "server", message.data);
});
await c.setLoggingLevel("debug"); // required – otherwise onLoggingMessage never
fires
```

17.9.1.4 mcp.connect.onProgress

```
onProgress(fn: (progress: { progressToken: string | number; progress: number;
total?: number; message?: string }) => void): void
```

Register the callback invoked for server-sent progress updates (notifications/progress) correlated to an in-flight call this connection made (e.g. a long-running callTool) — the client-side counterpart to a server tool handler's `ctx.progress(progress, total?)`. Only fires for calls where the server actually chose to send progress; not every tool call produces one. Only one `onProgress` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn ((progress: { progressToken: string | number; progress: number; total?: number; message?: string }) => void)` — invoked once per progress notification: `progressToken` identifies which in-flight request this update belongs to (opaque — correlate it yourself if issuing multiple concurrent calls); `progress` is the cumulative amount of work done so far (server-defined units, monotonically increasing); `total`, when the server provided one, is the expected end value (`progress/total` gives a completion fraction); `message`, when present, is a short human-readable status string. Its return value (and any thrown error or rejection) is ignored.

Returns: Nothing — replaces any previously registered `onProgress` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
c.onProgress((p) => {
  runtime.log(`progress: ${p.progress}${p.total ? "/" + p.total : ""}`,
  p.message ?? "");
});
await c.callTool("long-running-tool", {});
```

17.9.1.5 mcp.connect.onPromptsChanged

```
onPromptsChanged(fn: () => void): void
```

Register the callback invoked whenever the server's prompt list changes (notifications/prompts/list_changed) — the mirror of `onToolsChanged` for prompts. Only one `onPromptsChanged` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn (() => void)` — invoked with no arguments whenever the server announces its prompt list changed — call `listPrompts()` again inside `fn` if you need the fresh list. Its return value (and any thrown error or rejection) is ignored.

Returns: Nothing — replaces any previously registered `onPromptsChanged` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
c.onPromptsChanged(async () => {
  const prompts = await c.listPrompts();
  runtime.log("prompts now:", prompts.map(p => p.name).join(", "));
});
```

17.9.1.6 mcp.connect.onResourceUpdated

```
onResourceUpdated(fn: (uri: string) => void): void
```

Register the callback invoked whenever a subscribed resource's content changes (notifications/resources/updated) — the delivery half of the subscribe/unsubscribe pair (see `connect.subscribe`). Fires only for URIs this connection has subscribed to and only after the corresponding `subscribe(uri)` call has resolved. Only one `onResourceUpdated` callback is held at a time — a later call replaces the earlier registration, so a script juggling several subscribed URIs must dispatch on the `uri` argument itself, not register one callback per URI.

Parameters

- `fn ((uri: string) => void)` — invoked with the URI whose content changed — typically followed by a `readResource(uri)` call to fetch the new content. Its return value (and any thrown error or rejection) is ignored.

Returns: Nothing — replaces any previously registered `onResourceUpdated` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
c.onResourceUpdated(async (uri) => {
  const doc = await c.readResource(uri);
  runtime.log("new content for", uri, ":", doc.contents[0].text);
});
await c.subscribe("cfg://app");
```

17.9.1.7 mcp.connect.onResourcesChanged

```
onResourcesChanged(fn: () => void): void
```

Register the callback invoked whenever the server's resource list changes (notifications/resources/list_changed) — the mirror of `onToolsChanged` for resources. Only one `onResourcesChanged` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn () => void` — invoked with no arguments whenever the server announces its resource list changed — call `listResources()` again inside `fn` if you need the fresh list. Its return value (and any thrown error or rejection) is ignored.

Returns: Nothing — replaces any previously registered `onResourcesChanged` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
c.onResourcesChanged(async () => {
  const resources = await c.listResources();
  runtime.log("resources now:", resources.map(r => r.uri).join(", "));
});
```

17.9.1.8 mcp.connect.onToolsChanged

```
onToolsChanged(fn: () => void): void
```

Register the callback invoked whenever the server’s tool list changes (notifications/tools/list_changed) — e.g. a script connected to a server that itself calls `srv.tool(...)` at runtime after the initial connect. Only one `onToolsChanged` callback is held at a time — a later call replaces the earlier registration; there is no way to unregister short of closing the connection.

Parameters

- `fn () => void` — invoked with no arguments whenever the server announces its tool list changed — call `listTools()` again inside `fn` if you need the fresh list; the notification itself carries no payload. Its return value (and any thrown error or rejection) is ignored — the SDK’s notification handler has no way to report it back to the server.

Returns: Nothing — replaces any previously registered `onToolsChanged` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");
c.onToolsChanged(async () => {
  const tools = await c.listTools();
  runtime.log("tools now:", tools.map(t => t.name).join(", "));
});
```

17.9.1.9 mcp.connect.setLoggingLevel

```
setLoggingLevel(level: "debug" | "info" | "notice" | "warning" | "error" |
"critical" | "alert" | "emergency"): Promise<void>
```

Opt this connection into receiving the server’s log messages (logging/setLevel) at the given severity and higher. Per the MCP spec, a client receives nothing from a tool/resource/prompt handler’s `ctx.log(...)` calls (see `serve.tool`’s `ctx`) until it calls this — `onLoggingMessage(fn)` registered beforehand fires only after `setLoggingLevel` resolves, never before. Calling it again with a different level changes the threshold for all subsequent messages; there is no way to opt back out short of closing the connection.

Parameters

- `level` (“debug” | “info” | “notice” | “warning” | “error” | “critical” | “alert” | “emergency”) — the minimum severity to receive, using the RFC 5424 syslog severity names the MCP spec re-uses (from least to most severe: debug, info, notice, warning, error, critical, alert, emergency) — the server sends this level and everything more severe. `sercon` does not validate the string against this list itself; an unrecognized value is passed through to the server as-is and its handling is server-specific.

Returns: A promise that resolves once the server has acknowledged the level change. Does not itself deliver any messages — register `onLoggingMessage(fn)` beforehand (or at least before awaiting this) to observe the resulting notifications, since the server may start sending them immediately after acknowledging.

Throws: Throws synchronously if level is missing, null, or an empty string. The returned promise rejects if the connection is closed/dead or the server rejects the request as a protocol error (e.g. it doesn't support the logging capability at all).

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");
let lastLog: unknown = null;
c.onLoggingMessage((message) => { lastLog = message; });
await c.setLoggingLevel("info");
// now any ctx.log(...) call at "info" or more severe, on the server side,
// arrives here via onLoggingMessage as { level, logger?, data }
await c.close();
```

17.9.1.10 mcp.connect.setRoots

```
setRoots(roots: Array<{ uri: string; name?: string }>): void
```

Replace this connection's advertised filesystem/URI roots at runtime — the update counterpart to the `roots` connect opt's initial seed. Calls the SDK's `RemoveRoots` (for whatever this connection previously seeded/set) followed by `AddRoots` (for the new list), which together fire a `roots/list_changed` notification to the server; a server watching via `srv.onRootsChanged(fn)` sees the change pushed, and a server that only pulls via `ctx.roots()` sees the new set on its next call. Synchronous — there is no network round trip to await (`AddRoots/RemoveRoots` are pure in-memory bookkeeping on the client), so `setRoots` returns void, not a Promise, unlike almost every other method on this handle.

Parameters

- `roots` (`Array<{ uri: string; name?: string }>`) — the new complete root set — this REPLACES the previous set (whether it came from the connect opt's `roots` or an earlier `setRoots` call), it does not merge with it. Each entry needs a non-empty uri; name is an optional human-readable label. Pass an empty array to clear all roots.

Returns: Nothing. The server is not guaranteed to have observed the change by the time `setRoots` returns — only that the notification has been handed to the SDK client to send; a script that needs to be sure the server has picked up the new set should have the server confirm it back (e.g. via a tool call reading `ctx.roots()` after some delay, or by reacting to the server's own `srv.onRootsChanged` if scripting both sides).

Throws: Throws synchronously (a `TypeError`) if `roots` is not an array, or if any entry is not an object with a non-empty string uri. Never rejects — there is no Promise to reject.

```
const c = await mcp.connect.http("http://127.0.0.1:38080/mcp", {
  roots: [{ uri: "file:///a" }, { uri: "file:///b" }],
});
// ... later, the workspace changed:
c.setRoots([{ uri: "file:///c", name: "new workspace" }]);
await c.close();
```

17.9.1.11 mcp.connect.sse

```
sse(url: string, opts?: { headers?: Record<string, string>; onSample?: (req:
{ messages: Array<{ role: string; content: { type: string; text: string } }>;
maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?:
string[]; includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }) => string | { content:
{ type: string; text: string } | string; model?: string; stopReason?: string;
role?: string } | Promise<string | { content: { type: string; text: string } |
string; model?: string; stopReason?: string; role?: string }>; onElicit?: (req:
{ message: string; requestedSchema: Record<string, unknown>; mode?: string }) =>
{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> } |
Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string,
unknown> }>; roots?: Array<{ uri: string; name?: string }>; handlerTimeout?:
number }): Promise<{
  serverInfo: { name: string; version: string; title?: string };
  capabilities: Record<string, unknown>;
  listTools(): Promise<Array<{ name: string; title?: string; description?: string;
inputSchema: Record<string, unknown>; outputSchema?: Record<string, unknown> }>>;
  callTool(name: string, args?: Record<string, unknown>): Promise<{ content:
Array<{ type: string; text?: string; data?: string; mimeType?: string; uri?:
string; resource?: { uri: string; mimeType?: string; text?: string; blob?:
string } }>; structuredContent?: unknown; isError: boolean }>;
  listResources(): Promise<Array<{ uri: string; name: string; title?: string;
description?: string; mimeType?: string }>>;
  listResourceTemplates(): Promise<Array<{ uriTemplate: string; name: string;
title?: string; description?: string; mimeType?: string }>>;
  readResource(uri: string): Promise<{ contents: Array<{ uri: string; mimeType?:
string; text?: string; blob?: string } }> }>;
  listPrompts(): Promise<Array<{ name: string; title?: string; description?:
string; arguments?: Array<{ name: string; title?: string; description?: string;
required: boolean } }>>>;
  getPrompt(name: string, args?: Record<string, string>): Promise<{ description?:
string; messages: Array<{ role: string; content: { type: string; text?: string;
data?: string; mimeType?: string; uri?: string } }> }>;
  ping(): Promise<void>;
  close(): Promise<void>;
  onToolsChanged(fn: () => void): void;
  onResourcesChanged(fn: () => void): void;
  onPromptsChanged(fn: () => void): void;
  onResourceUpdated(fn: (uri: string) => void): void;
  onLoggingMessage(fn: (message: { level: string; logger?: string; data:
unknown }) => void): void;
  onProgress(fn: (progress: { progressToken: string | number; progress: number;
total?: number; message?: string }) => void): void;
  subscribe(uri: string): Promise<void>;
  unsubscribe(uri: string): Promise<void>;
  setLoggingLevel(level: string): Promise<void>;
  complete(ref: { type: "prompt" | "resource"; name?: string; uri?: string },
argName: string, partial: string): Promise<{ values: string[]; total?: number;
hasMore?: boolean }>;
  setRoots(roots: Array<{ uri: string; name?: string }>): void;
}>
```

Connect to an MCP server as a client over the legacy (2024-11-05) HTTP+SSE transport — for servers that predate Streamable HTTP (connect.http) and only expose the older two-

endpoint SSE handshake. Routes through the same `connectWith` lifecycle as `connect.stdio/`
`connect.http`, so everything else in Phase 1-4 works identically over this transport: the
consume surface (`listTools/callTool/etc.`), the Phase-2 reactive surface (`onToolsChanged/`
`subscribe/setLoggingLevel/complete/...`), and the Phase-3 host responder surface
(`onSample/onElicit/roots`). The one Phase-4 addition that does NOT apply here is
`maxRetries`: the `go-sdk`'s SSE client transport has no `reconnect-cap` field to plumb
(Streamable HTTP only) — see `connect.http`.

Parameters

- `url` (*string*) — the server's absolute SSE endpoint URL, e.g. "`http://127.0.0.1:38080/sse`" (must be `http` or `https` with a host; anything else throws synchronously).
- `opts` (*{ headers?: Record<string, string>; onSample?: (req: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number; intelligencePriority?: number; speedPriority?: number } }) => string | { content: { type: string; text: string } | string; model?: string; stopReason?: string; role?: string } | Promise<string | { content: { type: string; text: string } | string; model?: string; stopReason?: string; role?: string }>; onElicit?: (req: { message: string; requestedSchema: Record<string, unknown>; mode?: string }) => { action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> } | Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> }>; roots?: Array<{ uri: string; name?: string }>; handlerTimeout?: number }*, optional) — optional — the same shape as `connect.http`'s `opts` minus `maxRetries/auth` (neither applies to this transport: there's no reconnect cap to set, and no OAuth client wiring yet for SSE — use headers for a static bearer token instead). `headers`: extra HTTP headers sent with every request on this connection (e.g. a bearer token for a protected server) — merged with `sercon`'s default `sercon-mcp/<version> User-Agent`, which headers may override. `onSample(req)/onElicit(req)/roots`: identical host-responder semantics to `connect.http/connect.stdio` — see `connect.http`'s Params for the full field-by-field description of each. `handlerTimeout` (milliseconds) caps how long a host responder may run before the on-loop bridge returns an error; default 5 minutes, 0 disables.

Returns: Same handle shape as `connect.stdio/connect.http`: a promise that resolves once the initialize handshake completes to a session handle with `serverInfo/capabilities`, the `listTools/callTool/listResources/listResourceTemplates/readResource/listPrompts/getPrompt/ping/close` methods, the `subscribe/unsubscribe/setLoggingLevel/complete` mid-connection calls, the six `onXxx` notification setters, and `setRoots(roots)` to update the seeded roots set at runtime. Holds the script's event loop open for the connection's lifetime.

Throws: Throws synchronously if `url` is missing or not an absolute `http(s)` URL, `onSample/`
`onElicit` are present but not functions, or a `roots` entry is missing a non-empty uri. The returned promise rejects if the SSE handshake or initialize fails (wrapped as "`mcp.connect: ...`") — including a 401 from a protected server when `opts.headers` doesn't carry a valid bearer token (there is no `opts.auth` on this transport yet; retry/refresh it yourself before reconnecting). Capability gating happens at connect time, not as a throw here: the SDK client advertises sampling/elicitation if and only if `onSample/onElicit` (respectively) is provided — a server calling `ctx.sample()/ctx.elicit()` against a client that omitted the matching responder gets that server-side call rejected, not this connect call.

```

const c = await mcp.connect.sse("http://127.0.0.1:38080/sse");
runtime.log("connected to", c.serverInfo.name);
const tools = await c.listTools();
runtime.log(tools.map(t => t.name).join(", "));
await c.ping();
await c.close();

// Against a bearer-protected legacy SSE server (no opts.auth on this transport yet):
const authed = await mcp.connect.sse("http://127.0.0.1:38081/sse", {
  headers: { Authorization: "Bearer good-token" },
});
await authed.close();

// As a host: same onSample/onElicit/roots surface as connect.http/connect.stdio.
const host = await mcp.connect.sse("http://127.0.0.1:38080/sse", {
  onSample: (req) => "SUMMARY: " + req.messages[0].content.text,
  roots: [{ uri: "file:///workspace", name: "workspace" }],
});
await host.close();

```

17.9.1.12 mcp.connect.stdio

```

stdio(opts: { command: string[]; env?: Record<string, string>; cwd?: string;
onSample?: (req: { messages: Array<{ role: string; content: { type: string; text:
string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number;
stopSequences?: string[]; includeContext?: string; modelPreferences?:
{ costPriority?: number; intelligencePriority?: number; speedPriority?:
number } }) => string | { content: { type: string; text: string } | string;
model?: string; stopReason?: string; role?: string } | Promise<string | { content:
{ type: string; text: string } | string; model?: string; stopReason?: string;
role?: string }>; onElicit?: (req: { message: string; requestedSchema:
Record<string, unknown>; mode?: string }) => { action: "accept" | "decline" |
"cancel"; content?: Record<string, unknown> } | Promise<{ action: "accept" |
"decline" | "cancel"; content?: Record<string, unknown> }>; roots?: Array<{ uri:
string; name?: string }>; handlerTimeout?: number }): Promise<{
  serverInfo: { name: string; version: string; title?: string };
  capabilities: Record<string, unknown>;
  listTools(): Promise<Array<{ name: string; title?: string; description?: string;
inputSchema: Record<string, unknown>; outputSchema?: Record<string, unknown> }>>;
  callTool(name: string, args?: Record<string, unknown>): Promise<{ content:
Array<{ type: string; text?: string; data?: string; mimeType?: string; uri?:
string; resource?: { uri: string; mimeType?: string; text?: string; blob?:
string } }>; structuredContent?: unknown; isError: boolean }>;
  listResources(): Promise<Array<{ uri: string; name: string; title?: string;
description?: string; mimeType?: string }>>;
  listResourceTemplates(): Promise<Array<{ uriTemplate: string; name: string;
title?: string; description?: string; mimeType?: string }>>;
  readResource(uri: string): Promise<{ contents: Array<{ uri: string; mimeType?:
string; text?: string; blob?: string }> }>;
  listPrompts(): Promise<Array<{ name: string; title?: string; description?:
string; arguments?: Array<{ name: string; title?: string; description?: string;
required: boolean }> }>>;
  getPrompt(name: string, args?: Record<string, string>): Promise<{ description?:
string; messages: Array<{ role: string; content: { type: string; text?: string;

```

```

data?: string; mimeType?: string; uri?: string } }> }>;
  ping(): Promise<void>;
  close(): Promise<void>;
  onToolsChanged(fn: () => void): void;
  onResourcesChanged(fn: () => void): void;
  onPromptsChanged(fn: () => void): void;
  onResourceUpdated(fn: (uri: string) => void): void;
  onLoggingMessage(fn: (message: { level: string; logger?: string; data:
unknown }) => void): void;
  onProgress(fn: (progress: { progressToken: string | number; progress: number;
total?: number; message?: string }) => void): void;
  subscribe(uri: string): Promise<void>;
  unsubscribe(uri: string): Promise<void>;
  setLoggingLevel(level: string): Promise<void>;
  complete(ref: { type: "prompt" | "resource"; name?: string; uri?: string },
argName: string, partial: string): Promise<{ values: string[]; total?: number;
hasMore?: boolean }>;
  setRoots(roots: Array<{ uri: string; name?: string }>): void;
}>

```

Connect to an MCP server as a client, launching it as a subprocess and speaking newline-delimited JSON-RPC over its stdin/stdout — the shape most CLI-launched MCP servers (including sercon’s own `srv.stdio()`) expect. Phase 2: consume an already-running server’s tools/resources/prompts, react to its change notifications (`onToolsChanged/`
`onResourcesChanged/onPromptsChanged/onResourceUpdated`), subscribe/unsubscribe to individual resources, opt into server logs (`setLoggingLevel` + `onLoggingMessage`), and request argument completions (`complete`) — see the ten `onXxx/subscribe/unsubscribe/`
`setLoggingLevel/complete` entries below. Phase 3 adds the host responder surface: `onSample/onElicit` answer the server’s own sampling/createMessage and elicitation/create requests (the client-side counterpart to a server tool’s `ctx.sample()/ctx.elicit()`), and `roots` seeds the client’s filesystem/URI roots (answering the server’s `roots/list`) — see `setRoots` below for updating that set at runtime. Phase 4 rounds out the MCP client with `connect.sse` (the legacy SSE transport) and an OAuth client (`connect.http`’s `auth` option) — neither applies to `stdio` itself (there’s no HTTP handshake to attach a bearer token to, and this transport is a different wire format than SSE); Windows `stdio` support remains an unaddressed gap.

Parameters

- `opts` (`{ command: string[]; env?: Record<string, string>; cwd?: string; onSample?: (req: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number; intelligencePriority?: number; speedPriority?: number } }) => string | { content: { type: string; text: string } | string; model?: string; stopReason?: string; role?: string } | Promise<string | { content: { type: string; text: string } | string; model?: string; stopReason?: string; role?: string }>; onElicit?: (req: { message: string; requestedSchema: Record<string, unknown>; mode?: string }) => { action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> } | Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> }>; roots?: Array<{ uri: string; name?: string }>; handlerTimeout?: number }`) — `command`: argv for the subprocess, e.g. `["sercon", "server.ts"]` — `command[0]` is the executable (resolved

via PATH), the rest are its arguments; must be a non-empty array. env: extra environment variables merged into the child's inherited environment (does not replace it). cwd: working directory for the child process; defaults to sercon's own cwd when omitted.

onSample(req): answers the server's sampling/createMessage requests (ctx.sample() on the server side); req carries the same messages/maxTokens/systemPrompt/temperature/stopSequences/includeContext/modelPreferences fields ctx.sample's opts accepts. May return a plain string (wrapped as text content with model "sercon", role "assistant", stopReason "endTurn") or an object giving explicit control over content/model/stopReason/role; sync or async (a returned Promise is awaited).

onElicit(req): answers the server's elicitation/create requests (ctx.elicit() on the server side); req carries { message, requestedSchema, mode? }. Must return { action: "accept"|"decline"|"cancel", content? } (content only meaningful on "accept"); sync or async.

roots: seeds this client's filesystem/URI roots — an array of { uri, name? } — sent to the SDK via AddRoots before the connection is established, so the server's first roots/list sees them immediately rather than an empty set followed by a change notification. IMPORTANT: unlike onSample/onElicit (each advertised only when provided — see Errors), the roots capability itself is advertised by the underlying SDK unconditionally, regardless of whether this option is given; omitting roots simply means the initial root set is empty (still updatable later via setRoots).

handlerTimeout (milliseconds) caps how long a host responder (onSample/onElicit) may run before the on-loop bridge returns an error; default 5 minutes, 0 disables.

Returns: A promise that resolves once the MCP initialize handshake completes to a session handle: serverInfo/capabilities reflect what the server advertised (capabilities.sampling/capabilities.elicitation here describe what the SERVER supports, not this client's own onSample/onElicit — there is no client-facing field reflecting what this client advertised), listTools/callTool/listResources/listResourceTemplates/readResource/listPrompts/getPrompt/ping/close drive the session, subscribe/unsubscribe/setLoggingLevel/complete make mid-connection requests, the six onXxx setters register server-push notification callbacks, and setRoots(roots) updates the seeded roots set at runtime. Holds the script's event loop open for the connection's lifetime (until close() or the subprocess exits) the same way an open server listener does.

Throws: Throws synchronously if opts is missing/not an object, command is missing/not a non-empty string array, onSample/onElicit are present but not functions, or a roots entry is missing a non-empty uri. The returned promise rejects if the subprocess fails to start or the initialize handshake fails (wrapped as "mcp.connect: ..."). Capability gating happens at connect time, not as a throw: the SDK client advertises the sampling capability if and only if onSample is provided (likewise elicitation/onElicit) — a server calling ctx.sample()/ctx.elicit() against a client that omitted the matching responder gets that server-side call rejected ("mcp: client does not support sampling"/"...elicitation"), not this connect call.

```
const c = await mcp.connect.stdio({ command: ["sercon", "server.ts"] });
runtime.log("connected to", c.serverInfo.name, c.serverInfo.version);
const r = await c.callTool("add", { a: 2, b: 3 });
runtime.log(r.content[0].text); // "5"
await c.close();

// As a host: answer the server's sampling/elicitation requests and seed roots.
const host = await mcp.connect.stdio({
```

```

command: ["sercon", "server.ts"],
onSample: (req) => "SUMMARY: " + req.messages[0].content.text,
onElicit: (req) => ({ action: "accept", content: { confirmed: true } }),
roots: [{ uri: "file:///workspace", name: "workspace" }],
});
await host.close();

```

17.9.1.13 mcp.connect.subscribe

```
subscribe(uri: string): Promise<void>
```

Ask the server to notify this client whenever the given resource's content changes (resources/subscribe). The actual notification arrives via onResourceUpdated(fn) — call subscribe once per URI you care about, then register onResourceUpdated to react. Subscribing to a URI you're already subscribed to is a harmless no-op on most servers (a plain protocol round trip either way).

Parameters

- `uri` (*string*) — the resource URI to subscribe to — must match a URI the server actually tracks; subscribing to an unknown URI is a server-specific choice (some accept it silently, some reject it as a protocol error).

Returns: A promise that resolves once the server has acknowledged the subscription. Does not itself deliver any content — register onResourceUpdated(fn) beforehand to observe the resulting notifications.

Throws: Throws synchronously if uri is missing, null, or an empty string. The returned promise rejects if the connection is closed/dead or the server rejects the subscribe request as a protocol error (e.g. it doesn't support resource subscriptions at all).

```

const c = await mcp.connect.http("http://127.0.0.1:38080/mcp");
c.onResourceUpdated((uri) => { runtime.log("changed:", uri); });
await c.subscribe("cfg://app");
// ... some time later, once the server calls resourceUpdated("cfg://app") server-
side ...
await c.unsubscribe("cfg://app");
await c.close();

```

17.9.1.14 mcp.connect.unsubscribe

```
unsubscribe(uri: string): Promise<void>
```

Ask the server to stop notifying this client about changes to the given resource (resources/unsubscribe) — the mirror of subscribe. Unsubscribing from a URI you were never subscribed to is a harmless no-op on most servers.

Parameters

- `uri` (*string*) — the resource URI to unsubscribe from — typically one previously passed to subscribe(uri).

Returns: A promise that resolves once the server has acknowledged the unsubscription. `onResourceUpdated` stops firing for this uri afterwards (assuming no other client-side subscription logic re-subscribes it).

Throws: Throws synchronously if uri is missing, null, or an empty string. The returned promise rejects if the connection is closed/dead or the server rejects the unsubscribe request as a protocol error.

```
await c.subscribe("cfg://app");
// ...
await c.unsubscribe("cfg://app");
```

17.9.2 mcp.serve

```
serve(config: { name: string; version: string; instructions?: string; pageSize?:
number; handlerTimeout?: number }): {
  tool(spec: { name: string; description?: string; inputSchema: Record<string,
unknown>; outputSchema?: Record<string, unknown>; handler(args: unknown, ctx:
{ requestId: string; clientInfo: { name: string; version: string } };
progress(progress: number, total?: number): Promise<void>; log(level: string,
message: string, data?: unknown): Promise<void>; sample(opts: { messages:
Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
>> }): unknown | Promise<unknown> }): void;
  resource(spec: { uri: string; name: string; mimeType?: string; read(uri: string,
ctx: { requestId: string; clientInfo: { name: string; version: string } };
progress(progress: number, total?: number): Promise<void>; log(level: string,
message: string, data?: unknown): Promise<void>; sample(opts: { messages:
Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
>> }): unknown | Promise<unknown> }): void;
  resourceTemplate(spec: { uriTemplate: string; name: string; mimeType?: string;
read(uri: string, ctx: { requestId: string; clientInfo: { name: string; version:
string } }; progress(progress: number, total?: number): Promise<void>; log(level:
string, message: string, data?: unknown): Promise<void>; sample(opts: { messages:
Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
```



```

>> }): unknown | Promise<unknown> }): void;
  prompt(spec: { name: string; description?: string; arguments?: Array<{ name:
string; description?: string; required?: boolean }>; get(args: Record<string,
string> | undefined, ctx: { requestId: string; clientInfo: { name: string;
version: string } }; progress(progress: number, total?: number): Promise<void>;
log(level: string, message: string, data?: unknown): Promise<void>; sample(opts:
{ messages: Array<{ role: string; content: { type: string; text: string } }>;
maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?:
string[]; includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
>> }): unknown | Promise<unknown> }): void;
  removeTool(name: string): void;
  removeResource(uri: string): void;
  removePrompt(name: string): void;
  onSubscribe(fn: (uri: string) => void): void;
  onUnsubscribe(fn: (uri: string) => void): void;
  onRootsChanged(fn: (roots: Array<{ uri: string; name?: string }>) => void):
void;
  resourceUpdated(uri: string): Promise<void>;
  completion(fn: (ref: { type: "prompt" | "resource"; name: string; uri: string },
argName: string, partial: string) => string[] | { values?: string[]; total?:
number; hasMore?: boolean } | Promise<string[] | { values?: string[]; total?:
number; hasMore?: boolean }> | null | undefined): void;
  stdio(): Promise<void>;
  listen(opts: { port: number; host?: string; path?: string; auth?:
{ verify(token: string, req: { method: string; path: string; header:
Record<string, string> }): { subject?: string; scopes?: string[]; expiresAt?:
number | string } | null | Promise<{ subject?: string; scopes?: string[];
expiresAt?: number | string } | null>; resourceMetadata?: { authorizationServers?:
string[]; scopesSupported?: string[]; resourceName?: string;
resourceDocumentation?: string; jwksUri?: string; bearerMethodsSupported?:
string[]; resource?: string }; scopes?: string[] } }): Promise<{ url: string;
stopped: Promise<void>; close(): Promise<void> }>;
    close(): void;
}

```

Create an MCP (Model Context Protocol) server. Register zero or more tools/resources/prompts/resource templates on the returned handle — at any time, including after a transport has started, since registering them fires a list-changed notification to already-connected clients — then serve them over `stdio()` (Unix-only this phase) or `listen()` (Streamable HTTP, cross-platform). Built on the official `modelcontextprotocol/go-sdk`; only one transport may be started per handle — starting a second one throws.

Parameters

- `config` (`{ name: string; version: string; instructions?: string; pageSize?: number; handlerTimeout?: number }`) — `name/version` identify this server to clients during MCP's initialize handshake (surfaced to the client as `serverInfo`). `instructions` is an optional free-text hint about how to use this server (tone, expected workflow), surfaced to clients/LLMs during capability negotiation. `pageSize` is an optional positive integer capping how many

tools/resources/prompts/resource templates a single list response returns before the client must page with a cursor; defaults to the SDK's built-in page size (1000) when omitted. Present-but-invalid (zero, negative, or non-integer) throws synchronously. handlerTimeout (milliseconds) caps how long a tool/resource/prompt/completion handler or the auth verify may run before the on-loop bridge abandons the wait and returns an error (the handler's JS keeps running on the loop; the timeout just reclaims the waiting request goroutine); defaults to 5 minutes (generous, so a handler awaiting ctx.elicit/ctx.sample is not cut off), and 0 disables the cap (honour the request context only).

Returns: A handle for registering capabilities and starting a transport: tool()/resource()/resourceTemplate()/prompt() register handlers and removeTool()/removeResource()/removePrompt() unregister them — all callable at any time; onSubscribe()/onUnsubscribe() register resource-subscription hooks and resourceUpdated() notifies subscribers; completion() registers an argument-autocompletion handler; stdio()/listen() start serving — mutually exclusive, starting a second transport on the same handle throws; close() is currently an inert placeholder (see serve.close).

Throws: Throws synchronously if config is missing/not an object, name/version is missing or empty, or pageSize/handlerTimeout is present but not a valid non-negative integer.

```
const srv = mcp.serve({ name: "my-tools", version: "1.0.0" });
srv.tool({
  name: "add",
  description: "add two numbers",
  inputSchema: { type: "object", properties: { a: { type: "number" }, b: { type:
"number" } }, required: ["a", "b"] },
  async handler(args) { return String(args.a + args.b); },
});
await srv.stdio();
```

17.9.2.1 mcp.serve.close

```
close(): void
```

Present on the handle for interface symmetry, but currently a no-op — it does not stop a running transport. To stop an HTTP listener, call the close() on the handle returned by listen(); a stdio server stops on its own once the peer disconnects (its stdio() promise settles then). A future phase may wire this into an explicit shutdown path.

Returns: Nothing; calling it has no observable effect on a running transport.

Throws: Never throws.

```
const srv = mcp.serve({ name: "my-tools", version: "1.0.0" });
srv.close(); // currently a no-op; use the listen() handle's close(), or let
stdio() resolve on disconnect
```

17.9.2.2 mcp.serve.completion

```
completion(fn: (ref: { type: "prompt" | "resource"; name: string; uri: string },
argName: string, partial: string) => string[] | { values?: string[]; total?:
```

```
number; hasMore?: boolean } | Promise<string[] | { values?: string[]; total?:
number; hasMore?: boolean }> | null | undefined): void
```

Register the handler invoked for a client’s argument-autocompletion request (completion/complete) — suggesting values for a prompt argument or a resource-template URI variable as the user types. Only one completion callback is held at a time — a later call replaces the earlier registration. If never called, the server still advertises the completions capability and answers every request with an empty (“no suggestions”) result rather than rejecting it.

Parameters

- `fn` ((`ref`: { `type`: “prompt” | “resource”; `name`: string; `uri`: string }, `argName`: string, `partial`: string) => string[] | { `values`?: string[]; `total`?: number; `hasMore`?: boolean } | Promise<string[] | { `values`?: string[]; `total`?: number; `hasMore`?: boolean }> | null | undefined) — `ref` identifies what’s being completed: type “prompt” (the prompt registered via `serve.prompt`, in `name`) or type “resource” (the resource-template URI registered via `serve.resourceTemplate`, in `uri`). Both `name` and `uri` are always set on `ref` — whichever one doesn’t apply to the current type is set to the empty string “”, never omitted or undefined — so discriminate on `ref.type`, not on checking a field for undefined. `argName` is the argument/variable name being completed and `partial` is the text typed so far. `fn` may return a plain string[] (the suggestions, in order), an object { `values`?, `total`?, `hasMore`? } for pagination hints, a Promise of either, or null/undefined/nothing to mean no suggestions.

Returns: Nothing — replaces any previously registered completion callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function. Unlike a tool handler, a completion handler that throws or rejects propagates as a real completion/complete protocol error to the client — there is no `isError`-style soft failure for completion.

```
srv.prompt({
  name: "greet",
  arguments: [{ name: "user", required: true }],
  get: async (args) => ({ messages: [{ role: "user", content: { type: "text",
text: "hi " + args.user } }] }),
});

const users = ["alice", "alicia", "bob"];
srv.completion((ref, argName, partial) => {
  if (ref.type === "prompt" && ref.name === "greet" && argName === "user") {
    return users.filter((u) => u.startsWith(partial));
  }
  return [];
});
```

17.9.2.3 mcp.serve.listen

```
listen(opts: { port: number; host?: string; path?: string; auth?: { verify(token:
string, req: { method: string; path: string; header: Record<string, string> }):
{ subject?: string; scopes?: string[]; expiresAt?: number | string } | null |
Promise<{ subject?: string; scopes?: string[]; expiresAt?: number | string } |
null>; resourceMetadata?: { authorizationServers?: string[]; scopesSupported?:
string[]; resourceName?: string; resourceDocumentation?: string; jwksUri?: string;
```

```
bearerMethodsSupported?: string[]; resource?: string }; scopes?: string[] } }):  
Promise<{ url: string; stopped: Promise<void>; close(): Promise<void> }>
```

Serve this handle over the Streamable HTTP transport — a cross-platform, multi-client-capable alternative to `stdio()`: any number of clients can connect to a plain TCP/HTTP endpoint, rather than one client per subprocess. Optionally protect it as an OAuth 2.1 resource server via `opts.auth` — `sercon` validates bearer tokens and advertises where to obtain one; it never issues or registers tokens itself (that is the authorization server’s job, out of scope here).

Parameters

- `opts` (*{ port: number; host?: string; path?: string; auth?: { verify(token: string, req: { method: string; path: string; header: Record<string, string> }): { subject?: string; scopes?: string[]; expiresAt?: number | string } | null | Promise<{ subject?: string; scopes?: string[]; expiresAt?: number | string } | null>; resourceMetadata?: { authorizationServers?: string[]; scopesSupported?: string[]; resourceName?: string; resourceDocumentation?: string; jwksUri?: string; bearerMethodsSupported?: string[]; resource?: string }; scopes?: string[] } }*) — `port`: the TCP port to bind (required). `host`: bind interface, defaults to “127.0.0.1”. `path`: the HTTP path the MCP endpoint is mounted at, defaults to “/mcp”. `auth` (optional): turns this listener into an OAuth 2.1 protected resource server (RFC 6750 bearer tokens + RFC 9728 protected-resource metadata) — omit it for the unauthenticated Phase-1/2 behavior. `auth.verify(token, req)` is called once per request with the bearer token string and a plain `{method, path, header}` request-info object (header values are already flattened to strings); return an identity object to accept the request (subject/scopes/`expiresAt` — `expiresAt` accepts a Unix-seconds number or an RFC 3339 string, and defaults to a 1-hour TTL if omitted) or null/undefined to reject it with a 401 (verify may be sync or return a Promise; a thrown or rejected verify is also treated as a 401, not a 500). `auth.scopes` lists the scopes enforced on every request (all must be present on the verified identity’s scopes) and is also the WWW-Authenticate scope hint. `auth.resourceMetadata` fills the RFC 9728 metadata document served at `/well-known/oauth-protected-resource` — `authorizationServers` is the list of AS issuer URLs clients should obtain tokens from (the whole point of a resource server: it never issues tokens itself); `scopesSupported` falls back to `auth.scopes` when omitted; `resource` (the RS’s own identifier) defaults to the bound base URL when omitted. Dynamic client registration (DCR, RFC 7591) is intentionally out of scope — that’s the authorization server’s responsibility, not a resource server’s.

Returns: A promise that resolves as soon as the listener is bound (not when a client connects) to a handle: `url` is the full endpoint URL (e.g. “http://127.0.0.1:38080/mcp”); `stopped` resolves when the HTTP server stops (rejects on a non-close Serve error); `close()` begins a graceful shutdown and returns the same stopped promise.

Throws: Throws synchronously if a transport is already running on this handle, `opts` is missing/not an object, `port` is missing, or (with `auth` present) `auth.verify` is not a function. Throws (wrapping the bind error) if the listener fails to bind (e.g. address already in use) — a bind failure does NOT mark the handle as started, so `listen()` may be retried with a different port on the same handle. With `auth` configured, a request with a missing/invalid/expired/insufficiently-scoped bearer token never reaches your handlers — the middleware answers

it directly with 401 and a WWW-Authenticate header pointing at the protected-resource metadata URL; this is enforced per-request at the HTTP layer, not surfaced as a script-side error.

```
const srv = mcp.serve({ name: "my-tools", version: "1.0.0" });
srv.tool({ name: "ping", inputSchema: { type: "object" }, handler: () =>
  "pong" });
const h = await srv.listen({ port: 38080 });
runtime.log("listening at", h.url);
// ... later
await h.close();

// With OAuth 2.1 resource-server protection:
const h2 = await srv.listen({
  port: 38081,
  auth: {
    verify: (token) => (token === "good-token" ? { subject: "demo-user", scopes:
["mcp"] } : null),
    resourceMetadata: { authorizationServers: ["https://auth.example.com"],
scopesSupported: ["mcp"] },
    scopes: ["mcp"],
  },
});
await h2.close();
```

17.9.2.4 mcp.serve.onRootsChanged

```
onRootsChanged(fn: (roots: Array<{ uri: string; name?: string }>) => void): void
```

Register a best-effort hook invoked whenever the connected client's filesystem/URI roots list changes (the MCP roots list-changed notification) — the persistent counterpart to `ctx.roots()`'s pull-based, per-request query. Only one `onRootsChanged` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn ((roots: Array<{ uri: string; name?: string }>) => void)` — invoked with the client's fresh root list (the same `{uri, name?}` shape `ctx.roots()` resolves to; name is `""` when the client didn't set one). Its return value (and any thrown error or rejection) is ignored — the go-sdk's own notification handler has no way to fail the notification back to the client, so this hook is purely an observer.

Returns: Nothing — replaces any previously registered `onRootsChanged` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
srv.onRootsChanged((roots) => {
  runtime.log("roots changed:", JSON.stringify(roots.map((r) => r.uri)));
});

srv.tool({
  name: "listRoots",
  inputSchema: { type: "object" },
  async handler(_args, ctx) {
```

```
const roots = await ctx.roots();
return JSON.stringify(roots.map((r) => r.uri));
},
});
```

17.9.2.5 mcp.serve.onSubscribe

```
onSubscribe(fn: (uri: string) => void): void
```

Register a best-effort hook invoked whenever a client subscribes to a resource (resources/subscribe). Typically used to start watching a resource's backing data (a file, a DB row, ...) only when a client actually cares, pairing with `serve.onUnsubscribe` to stop watching and `serve.resourceUpdated` to notify once it changes. Only one `onSubscribe` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn ((uri: string) => void)` — invoked with the URI the client subscribed to. Its return value (and any thrown error or rejection) is ignored — the subscribe request always succeeds from the client's point of view; this hook cannot fail it.

Returns: Nothing — replaces any previously registered `onSubscribe` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```
const watchers = new Map<string, () => void>();
srv.onSubscribe((uri) => {
  runtime.log("client subscribed to", uri);
  // start watching uri's backing data here
});
srv.onUnsubscribe((uri) => {
  runtime.log("client unsubscribed from", uri);
  // stop watching uri's backing data here
});
```

17.9.2.6 mcp.serve.onUnsubscribe

```
onUnsubscribe(fn: (uri: string) => void): void
```

Register a best-effort hook invoked whenever a client unsubscribes from a resource (resources/unsubscribe) — the mirror of `serve.onSubscribe`. Only one `onUnsubscribe` callback is held at a time — a later call replaces the earlier registration.

Parameters

- `fn ((uri: string) => void)` — invoked with the URI the client unsubscribed from. Its return value (and any thrown error or rejection) is ignored — the unsubscribe request always succeeds from the client's point of view; this hook cannot fail it.

Returns: Nothing — replaces any previously registered `onUnsubscribe` callback.

Throws: Throws synchronously (a `TypeError`) if `fn` is not a function.

```

srv.onUnsubscribe((uri) => {
  runtime.log("no longer watching", uri);
});

```

17.9.2.7 mcp.serve.prompt

```

prompt(spec: { name: string; description?: string; arguments?: Array<{ name:
string; description?: string; required?: boolean }>; get(args: Record<string,
string> | undefined, ctx: { requestId: string; clientInfo: { name: string;
version: string }; progress(progress: number, total?: number): Promise<void>;
log(level: string, message: string, data?: unknown): Promise<void>; sample(opts:
{ messages: Array<{ role: string; content: { type: string; text: string } }>;
maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?:
string[]; includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
>> }): unknown | Promise<unknown> }): void

```

Register a prompt: a named, parameterized template clients can fetch to seed a conversation. Callable at any time, including after a transport has started, for the same list-changed-notification reason as `serve.tool`.

Parameters

- *spec* (*{ name: string; description?: string; arguments?: Array<{ name: string; description?: string; required?: boolean }>; get(args: Record<string, string> | undefined, ctx: { requestId: string; clientInfo: { name: string; version: string }; progress(progress: number, total?: number): Promise<void>; log(level: string, message: string, data?: unknown): Promise<void>; sample(opts: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number; intelligencePriority?: number; speedPriority?: number } }): Promise<{ content: { type: string; text: string }; model: string; stopReason: string; role: string }>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?: string }): Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }>> }): unknown | Promise<unknown> }*) — *name*: unique prompt name. *description*: shown to clients when listing prompts. *arguments*: the prompt's declared parameters (name/description/required), advertised so clients know what to supply. *get(args, ctx)*: sync or async; must return *{ description?: string; messages: Array<{ role: string; content: unknown }> }* — each message's content follows the same content-item shape as a tool result's content array (e.g. *{ type: "text", text }*). Any other shape, or a thrown/rejected handler, is a protocol-level error. *ctx* is the same shape as a tool handler's (see `serve.tool`).

Returns: Nothing — registers the prompt on the handle. Call `srv.removePrompt(name)` to unregister it.

Throws: Throws synchronously if *spec* is not an object, *name* is missing/empty, *arguments* is present but not an array (or an entry is missing *name*), or *get* is not a function. A `get`

handler that throws, rejects, or returns a malformed result propagates as a prompts/get protocol error to the client — there is no `isError`-style soft failure for prompts.

```
srv.prompt({
  name: "greet",
  description: "greet a user by name",
  arguments: [{ name: "user", required: true }],
  get: async (args) => ({
    messages: [{ role: "user", content: { type: "text", text: `Say hello to
${args.user}.` } }],
  }),
});
```

17.9.2.8 mcp.serve.removePrompt

```
removePrompt(name: string): void
```

Unregister a previously added prompt by name. Callable at any time — before or after a transport has started; removing a name post-connect fires a `prompts/list_changed` notification to connected clients. Removing a name that was never registered (or already removed) is a silent no-op.

Parameters

- `name` (*string*) — the prompt name passed to `serve.prompt`'s `spec.name`.

Returns: Nothing.

Throws: Throws synchronously (a `TypeError`) if `name` is missing, null, or an empty string.

```
srv.prompt({ name: "temp", get: () => ({ messages: [] }) });
// ... later:
srv.removePrompt("temp");
```

17.9.2.9 mcp.serve.removeResource

```
removeResource(uri: string): void
```

Unregister a previously added resource by URI. Callable at any time — before or after a transport has started; removing a URI post-connect fires a `resources/list_changed` notification to connected clients. Removing a URI that was never registered (or already removed) is a silent no-op.

Parameters

- `uri` (*string*) — the resource URI passed to `serve.resource`'s `spec.uri`. There is no equivalent remove method for a resource template registered via `serve.resourceTemplate` — that's a currently-unbound gap in the script surface, not a missing SDK capability.

Returns: Nothing.

Throws: Throws synchronously (a `TypeError`) if `uri` is missing, null, or an empty string.


```

srv.resource({ uri: "config://temp", name: "temp", read: () => ({ text:
  "{}" }) });
// ... later:
srv.removeResource("config://temp");

```

17.9.2.10 mcp.serve.removeTool

```
removeTool(name: string): void
```

Unregister a previously added tool by name. Callable at any time — before or after a transport has started; removing a name post-connect fires a tools/list_changed notification to connected clients. Removing a name that was never registered (or already removed) is a silent no-op.

Parameters

- `name` (*string*) — the tool name passed to `serve.tool`'s `spec.name`.

Returns: Nothing.

Throws: Throws synchronously (a `TypeError`) if `name` is missing, null, or an empty string.

```

srv.tool({ name: "temp", inputSchema: { type: "object" }, handler: () => "hi" });
// ... later, once the capability is no longer relevant:
srv.removeTool("temp");

```

17.9.2.11 mcp.serve.resource

```

resource(spec: { uri: string; name: string; mimeType?: string; read(uri: string,
  ctx: { requestId: string; clientInfo: { name: string; version: string } };
  progress(progress: number, total?: number): Promise<void>; log(level: string,
  message: string, data?: unknown): Promise<void>; sample(opts: { messages:
  Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
  number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
  includeContext?: string; modelPreferences?: { costPriority?: number;
  intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
  { type: string; text: string }; model: string; stopReason: string; role: string }
  >; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
  string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
  Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
  >> }): unknown | Promise<unknown> }): void

```

Register a resource: a URI-addressed piece of content clients can read (e.g. a file, a config blob, a generated report). Callable at any time, including after a transport has started, for the same list-changed-notification reason as `serve.tool`.

Parameters

- `spec` (*{ uri: string; name: string; mimeType?: string; read(uri: string, ctx: { requestId: string; clientInfo: { name: string; version: string }; progress(progress: number, total?: number): Promise<void>; log(level: string, message: string, data?: unknown): Promise<void>; sample(opts: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number;*

intelligencePriority?: number; speedPriority?: number } }): Promise<{ content: { type: string; text: string }; model: string; stopReason: string; role: string }>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?: string }): Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }>> }>): unknown | Promise<unknown> } } — uri: the resource's identifier (any URI scheme, e.g. "file:

Returns: Nothing — registers the resource on the handle. Call `srv.removeResource(uri)` to unregister it.

Throws: Throws synchronously if `spec` is not an object, `uri` or `name` is missing/empty, or `read` is not a function. A read handler that throws, rejects, or returns a value without text or blob propagates as a resources/read protocol error to the client — there is no `isError`-style soft failure for resources.

```
srv.resource({
  uri: "config://app",
  name: "App config",
  mimeType: "application/json",
  read: async () => ({ text: JSON.stringify({ debug: true }) }),
});
```

17.9.2.12 mcp.serve.resourceTemplate

```
resourceTemplate(spec: { uriTemplate: string; name: string; mimeType?: string;
read(uri: string, ctx: { requestId: string; clientInfo: { name: string; version:
string }; progress(progress: number, total?: number): Promise<void>; log(level:
string, message: string, data?: unknown): Promise<void>; sample(opts: { messages:
Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
>> }> ): unknown | Promise<unknown> }): void
```

Register a resource template: an RFC 6570 URI template (e.g. "db:

Parameters

- `spec` (*{ uriTemplate: string; name: string; mimeType?: string; read(uri: string, ctx: { requestId: string; clientInfo: { name: string; version: string }; progress(progress: number, total?: number): Promise<void>; log(level: string, message: string, data?: unknown): Promise<void>; sample(opts: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number; intelligencePriority?: number; speedPriority?: number } }): Promise<{ content: { type: string; text: string }; model: string; stopReason: string; role: string }>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?: string }): Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> }>; roots():*

Promise<Array<{ uri: string; name?: string }>> > }): unknown | Promise<unknown> } —
uriTemplate: the RFC 6570 template string advertised to clients (e.g. “db:

Returns: Nothing — registers the resource template on the handle.

Throws: Throws synchronously if spec is not an object, uriTemplate or name is missing/empty, or read is not a function. A read handler that throws, rejects, or returns a value without text or blob propagates as a resources/read protocol error to the client, same as `serve.resource`.

```
srv.resourceTemplate({
  uriTemplate: "db://{table}/{id}",
  name: "row",
  mimeType: "application/json",
  read: (uri) => ({ text: "row at " + uri }),
});
// a client reading "db:///users/42" invokes read("db:///users/42", ctx)
```

17.9.2.13 mcp.serve.resourceUpdated

```
resourceUpdated(uri: string): Promise<void>
```

Notify every client currently subscribed to uri that the resource’s content has changed (resources/updated). This is the script’s half of the subscription pair: the SDK tracks the actual subscriber set itself (via the subscribe/unsubscribe plumbing backing `serve.onSubscribe/onUnsubscribe`) and fans this notification out to whoever is subscribed to uri right now — calling it for a URI with no subscribers is a harmless no-op. Callable at any time after `mcp.serve()`, including before any transport has started.

Parameters

- `uri` (*string*) — the resource URI that changed — must match a URI a client may have subscribed to (does not need to be a URI you’ve registered with `serve.resource`; it’s just an opaque identifier from the notification’s point of view).

Returns: A promise that resolves once the notification has been sent (or immediately if there is no active transport to send it over).

Throws: Throws synchronously (a `TypeError`) if uri is missing, null, or an empty string. The returned promise rejects if the underlying notification send fails.

```
srv.resourceUpdated("config://app");
```

17.9.2.14 mcp.serve.stdio

```
stdio(): Promise<void>
```

Serve this handle over stdio (newline-delimited JSON-RPC on stdin/stdout) — the transport clients like Claude Desktop use when they launch this script as a subprocess. Unix-only this phase: on Windows the returned promise rejects immediately with a clear error (stdout can’t be safely separated from the JSON-RPC stream there) — use `listen()` instead. `sercon`’s own output (`console.*`, `runtime.log`) is transparently redirected to `stderr` starting the

moment `mcp.serve()` is called (not just once `stdio()` begins), so `stdout` carries only protocol frames for the lifetime of the connection.

Returns: A promise that resolves once the client disconnects (`stdin` closes / the session ends) and rejects if the transport fails to start (e.g. on Windows) or the JSON-RPC session ends with an error.

Throws: Throws synchronously if a transport is already running on this handle (`stdio()`/`listen()` already called). The returned promise rejects on Windows with “mcp: `stdio()` is not supported on windows (console output cannot be separated from the JSON-RPC stream); use `listen()` instead”, or if the underlying session ends abnormally.

```
const srv = mcp.serve({ name: "my-tools", version: "1.0.0" });
srv.tool({ name: "ping", inputSchema: { type: "object" }, handler: () =>
  "pong" });
await srv.stdio();
```

17.9.2.15 mcp.serve.tool

```
tool(spec: { name: string; description?: string; inputSchema: Record<string,
unknown>; outputSchema?: Record<string, unknown>; handler(args: unknown, ctx:
{ requestId: string; clientInfo: { name: string; version: string } };
progress(progress: number, total?: number): Promise<void>; log(level: string,
message: string, data?: unknown): Promise<void>; sample(opts: { messages:
Array<{ role: string; content: { type: string; text: string } }>; maxTokens?:
number; systemPrompt?: string; temperature?: number; stopSequences?: string[];
includeContext?: string; modelPreferences?: { costPriority?: number;
intelligencePriority?: number; speedPriority?: number } }): Promise<{ content:
{ type: string; text: string }; model: string; stopReason: string; role: string }
>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?:
string }): Promise<{ action: "accept" | "decline" | "cancel"; content?:
Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }
>> }): unknown | Promise<unknown> }): void
```

Register a tool: a named, schema-described callable that MCP clients (typically an LLM agent) can invoke. Callable at any time, including after a transport has started — the SDK fires a `tools/list_changed` notification to already-connected clients when a tool is added post-connect, so a script can grow its tool set at runtime (e.g. from within another handler).

Parameters

- `spec` (`{ name: string; description?: string; inputSchema: Record<string, unknown>; outputSchema?: Record<string, unknown>; handler(args: unknown, ctx: { requestId: string; clientInfo: { name: string; version: string } }; progress(progress: number, total?: number): Promise<void>; log(level: string, message: string, data?: unknown): Promise<void>; sample(opts: { messages: Array<{ role: string; content: { type: string; text: string } }>; maxTokens?: number; systemPrompt?: string; temperature?: number; stopSequences?: string[]; includeContext?: string; modelPreferences?: { costPriority?: number; intelligencePriority?: number; speedPriority?: number } }): Promise<{ content: { type: string; text: string }; model: string; stopReason: string; role: string }>; elicit(opts: { message: string; schema: Record<string, unknown>; mode?: string }): Promise<{ action: "accept" | "decline" | "cancel"; content?: Record<string, unknown> }>; roots(): Promise<Array<{ uri: string; name?: string }>> }): unknown | Promise<unknown> }`) — `name`: unique tool name

presented to clients. `description`: shown to the client/LLM to help it decide when to call this tool. `inputSchema`: a JSON Schema object describing the call arguments — passed through to the SDK as-is (not validated by sercon itself). `outputSchema`: optional JSON Schema describing structuredContent. `handler(args, ctx)`: sync or async; may return a plain string (wrapped as a single text content item), an object shaped `{ content?, structuredContent?, isError? }`, or throw/reject — a thrown or rejected handler is NOT a protocol error, it surfaces to the client as a tool result with `isError: true`. `ctx` adds `progress(progress, total?)` and `log(level, message, data?)` to the `requestId/clientInfo` pair — see §5.15.1's progress/logging concepts for the `SetLogLevel` caveat on `log()`.

Returns: Nothing — registers the tool on the handle. Call multiple times to register multiple tools; call `srv.removeTool(name)` to unregister one.

Throws: Throws synchronously if `spec` is not an object, `name` is missing/empty, or `handler` is not a function. A handler that throws or returns a rejected promise does not throw here — it surfaces per-call as an `{ isError: true }` tool result seen by the client, not a synchronous or protocol-level error.

```
srv.tool({
  name: "add",
  description: "add two numbers",
  inputSchema: { type: "object", properties: { a: { type: "number" }, b: { type: "number" } }, required: ["a", "b"] },
  async handler(args) { return String(args.a + args.b); },
});
```

17.10 net

Network clients and probes: HTTP, TCP/DNS/TLS/NTP/WHOIS probes, netstatus, email auth, browser-style sessions.

17.10.1 net.browser

17.10.1.1 net.browser.open

```
open(...args: unknown[]): Promise<{ setUserAgent(ua: string): void,
setHeader(name: string, value: string): void, get(url: string): Promise<{ status:
number, ok: boolean, headers: Record<string, string>, body: string, url: string }>,
post(url: string, body?: string): Promise<{ status: number, ok: boolean,
headers: Record<string, string>, body: string, url: string }>, cookies(url:
string): Promise<{ name: string, value: string }[]> }>
```

Open a stateful HTTP session: `{ setUserAgent, setHeader, get, post, cookies }`. Cookie jar + default headers persist across requests (like a browser).

Returns: `Promise<{ setUserAgent(ua: string): void, setHeader(name: string, value: string): void, get(url: string): Promise<{ status: number, ok: boolean, headers: Record<string, string>, body: string, url: string }>, post(url: string, body?: string): Promise<{ status: number, ok: boolean, headers: Record<string, string>, body: string, url: string }>, cookies(url: string): Promise<{ name: string, value: string }[]> }>` — a session handle backed by an `http.Client` with an automatic cookie jar (public-suffix scoped). `setUserAgent/setHeader` register default headers replayed on every request; `get/post` resolve to `{ status, ok, headers, body, url }` (like `net.http.request` but without `bodyBytes`); `cookies` lists the jar's cookies for a URL.

Throws: `browser.open` rejects only if the cookie jar can't be created. `get/post` reject if the URL is empty or the request fails (transport error); `cookies` rejects if the URL is empty or unparseable. 4xx/5xx responses resolve normally.

```
const b = await net.browser.open();
b.setUserAgent("my-bot/1.0");
await b.post("https://site/login", "user=x&pass=y");
const home = await b.get("https://site/home");
runtime.log(await b.cookies("https://site/"));
```

17.10.2 net.capture

17.10.2.1 net.capture.interfaces

```
interfaces(): { name: string; addresses: string[]; up: boolean; loopback:
boolean }[]
```

List the host's network interfaces synchronously: `net.capture.interfaces()` → array of { name, addresses: string[], up, loopback }. Pure-Go (no privileges, all platforms).

Returns: { name: string, addresses: string[], up: boolean, loopback: boolean }[] — one entry per interface with its name, assigned addresses (CIDR strings), and up / loopback flags. Synchronous (not a Promise).

Throws: Throws if interface enumeration fails.

```
for (const i of net.capture.interfaces()) runtime.log(i.name, i.up);
```

17.10.2.2 net.capture.open

```
open(opts: { iface: string, promisc?: boolean, snaplen?: number, filter?:
string }, onPacket: (pkt: { ts: number, length: number, captureLength: number,
link: string, eth?: { src: string, dst: string, type: string }, vlan?: { id:
number, priority: number, drop: boolean, type: string }, arp?: { operation:
string, senderMac: string, senderIp: string, targetMac: string, targetIp:
string }, ip?: { version: number, src: string, dst: string, protocol: string, ttl:
number }, tcp?: { srcPort: number, dstPort: number, seq: number, ack: number,
window: number, checksum: number, flags: { syn: boolean, ack: boolean, fin:
boolean, rst: boolean, psh: boolean, urg: boolean }, options?: { mss?: number,
windowScale?: number, sackPermitted?: boolean, timestamps?: { val: number, ecr:
number } } }, udp?: { srcPort: number, dstPort: number, length: number }, icmp?:
{ type: number, code: number }, dns?: { id: number, qr: boolean, opcode: string,
rcode: string, questions: { name: string, type: string }[], answers: { name:
string, type: string, data: string }[] }, payload?: Uint8Array, bytes:
Uint8Array }) => void): Promise<{ iface: string, link: string, close():
Promise<void> }>
```

Live packet capture: `net.capture.open({ iface, promisc?, snaplen?, filter? }, pkt => {...})` → `Promise<{ iface, link, close() }>`. Linux + macOS only (Windows rejects); needs root / `CAP_NET_RAW` (Linux) or `/dev/bpf` access (macOS). `promisc` defaults true. The handler is called per frame with a decoded packet { ts, length, captureLength, link, eth?, vlan?, arp?, ip?, tcp?, udp?, icmp?, dns?, payload?, bytes }. Optional filter is a tcpdump-like expression string (e.g. 'tcp and port 80'), evaluated post-decode in userspace — NOT a kernel BPF program,

so it skips the JS callback for non-matching packets but does not avoid the kernel→userspace copy. Supports tcp/udp/icmp/ip/ip6, host/src host/dst host, net/src net/dst net (CIDR), port/src port/dst port, portrange/src portrange/dst portrange, and/or/not + parens, implicit-and between juxtaposed primaries. A malformed expression makes open reject. close() returns Promise<void>. Pure-Go gopacket (no libpcap/cgo).

Parameters

- **opts** (*{ iface: string, promisc?: boolean, snaplen?: number, filter?: string }*) — iface is the interface name to capture on (required); promisc enables promiscuous mode (default true); snaplen caps the per-packet capture length in bytes (default 262144); filter is an optional tcpdump-like expression (e.g. 'tcp and port 80') applied post-decode in userspace — supports tcp/udp/icmp/ip/ip6, host/net (CIDR), port/portrange, src/dst directions, and/or/not + parens.
- **onPacket** (*((pkt: { ts: number, length: number, captureLength: number, link: string, eth?: { src: string, dst: string, type: string }, vlan?: { id: number, priority: number, drop: boolean, type: string }, arp?: { operation: string, senderMac: string, senderIp: string, targetMac: string, targetIp: string }, ip?: { version: number, src: string, dst: string, protocol: string, ttl: number }, tcp?: { srcPort: number, dstPort: number, seq: number, ack: number, window: number, checksum: number, flags: { syn: boolean, ack: boolean, fin: boolean, rst: boolean, psh: boolean, urg: boolean }, options?: { mss?: number, windowScale?: number, sackPermitted?: boolean, timestamps?: { val: number, ecr: number } } }, udp?: { srcPort: number, dstPort: number, length: number }, icmp?: { type: number, code: number }, dns?: { id: number, qr: boolean, opcode: string, rcode: string, questions: { name: string, type: string }[], answers: { name: string, type: string, data: string }[] }, payload?: Uint8Array, bytes: Uint8Array }) => void)*) — Called once per matching frame with the decoded packet. ts is epoch ms; layer keys (eth/vlan/arp/ip/tcp/udp/icmp/dns) are present only when that layer decoded; bytes is always the raw frame; payload is the application-layer bytes when present.

Returns: Promise<{ iface: string, link: string, close(): Promise<void> }> — a live-capture handle. link is the link-type name; close() stops the capture and resolves when the source is torn down. The handler keeps firing until close() is called or the source errors.

Throws: Rejects if iface is missing, the filter expression is malformed, the platform is unsupported (Windows), or the capture can't be opened (missing root / CAP_NET_RAW on Linux, /dev/bpf access on macOS). Throws synchronously if onPacket is not a function.

```
const cap = await net.capture.open({ iface: "en0", filter: "tcp and port 443" },
  pkt => {
    runtime.log(pkt.ip?.src, "→", pkt.ip?.dst);
  });
await cap.close();
```

17.10.2.3 net.capture.openFile

```
openFile(path: string, onPacket: (pkt: { ts: number, length: number,
  captureLength: number, link: string, eth?: object, vlan?: object, arp?: object,
  ip?: object, tcp?: object, udp?: object, icmp?: object, dns?: object, payload?:
  Uint8Array, bytes: Uint8Array }) => void, opts?: { filter?: string }):
  Promise<void>
```


Read a .pcap / .pcapng file: `net.capture.openFile(path, pkt => {...}, opts?) → Promise<void>`. Calls the handler once per decoded packet (same shape as `capture.open`) and resolves at EOF. Offline; no privileges. `opts` is an optional trailing arg `{ filter? }` — the 2-arg form still works; `filter` is the same tcpdump-like expression string as `capture.open` (post-decode/userspace, not kernel BPF; supports host/net CIDR + port/portrange; malformed → rejects).

Parameters

- `path` (*string*) — Path to a .pcap or .pcapng file; the format is auto-detected from the magic bytes.
- `onPacket` (*((pkt: { ts: number, length: number, captureLength: number, link: string, eth?: object, vlan?: object, arp?: object, ip?: object, tcp?: object, udp?: object, icmp?: object, dns?: object, payload?: Uint8Array, bytes: Uint8Array }) => void)*) — Called once per decoded packet (same shape as `capture.open`'s handler).
- `opts` (*{ filter?: string, optional }*) — `filter` is the same tcpdump-like expression as `capture.open`, applied post-decode in userspace; omit (2-arg form) to deliver every packet.

Returns: `Promise<void>` — resolves at end-of-file after dispatching every (matching) packet to the handler.

Throws: Rejects if the filter expression is malformed, the file can't be opened or parsed, or the handler throws. Throws synchronously if `onPacket` is not a function.

```
await net.capture.openFile("/tmp/dump.pcap", pkt => runtime.log(pkt.tcp?.dstPort),
{ filter: "tcp" });
```

17.10.2.4 net.capture.routes

```
routes(): { destination: string; gateway: string; interface: string; family: "ip"
| "ip6"; metric: number }[]
```

List the host's IP routing table synchronously: `net.capture.routes() → array of { destination, gateway, interface, family, metric }`. Pure-Go, unprivileged: Linux reads `/proc/net/route + /proc/net/ipv6_route`; macOS/BSD read the routing socket via `x/net/route`. Windows is unsupported (throws).

Returns: `{ destination: string, gateway: string, interface: string, family: "ip" | "ip6", metric: number }[]` — one entry per route. `destination` is a CIDR ('0.0.0.0/0' for the default route, ':::0' for the IPv6 default); `gateway` is the next-hop IP or '' for a directly-connected/link route; `interface` is the outgoing NIC name (best-effort); `metric` is 0 when the platform doesn't report one (BSD/macOS). Synchronous (not a Promise).

Throws: Throws if route enumeration fails or the platform is unsupported (Windows).

```
const def = net.capture.routes().find(r => r.destination === "0.0.0.0/0");
\nruntime.log("default via", def?.gateway, "on", def?.interface);
```

17.10.2.5 net.capture.toFile


```
toFile(path: string, opts?: { snaplen?: number, linkType?: number }):
{ write(bytes: string | Uint8Array, opts?: { ts?: number }): void; close():
Promise<void> }
```

Write raw frames to a .pcap file: `net.capture.toFile(path, { linkType?, snaplen? }) → { write(bytes, { ts? }), close() }`. `write` appends a raw frame (Uint8Array); `ts` (ms) overrides the timestamp. `close()` flushes and returns `Promise<void>`. Offline; no privileges.

Parameters

- `path` (*string*) — Path of the .pcap file to create (overwritten if it exists).
- `opts` (*{ snaplen?: number, linkType?: number }, optional*) — `snaplen` is the pcap global-header snap length (default 262144); `linkType` is the numeric pcap link-type written into the header (default Ethernet).

Returns: `{ write(bytes, opts?): void, close(): Promise<void> }` — a writer handle (returned synchronously, not a Promise). `write` appends one raw frame to the file (`opts.ts` in ms overrides the timestamp, defaulting to now) and returns undefined. `close()` flushes and closes the file, resolving when done.

Throws: Throws synchronously if the file can't be created or the header can't be written, and on a per-frame write error. `write` throws if called after `close`. `close()` rejects on a close error.

```
const w = net.capture.toFile("/tmp/out.pcap");
w.write(frameBytes, { ts: Date.now() });
await w.close();
```

17.10.3 net.email

17.10.3.1 net.email.all

```
all(domain: string): Promise<{ domain: string, spf: object, dmarc: object, mtaSts:
object, tlsRpt: object, bimi: object }>
```

Run all five email probes in parallel — five-way handshake aggregate.

Parameters

- `domain` (*string*) — The domain to run all five email-auth probes against.

Returns: `Promise<{ domain: string, spf: object, dmarc: object, mtaSts: object, tlsRpt: object, bimi: object }>` — `domain` echoes the input; each probe key holds that probe's result (the same shape its individual binding returns) or `{ error: string }` when that single probe failed. A per-probe failure doesn't fail the aggregate.

Throws: Resolves even when individual probes fail (their failure surfaces under `<probe>.error`). Always resolves.

```
const a = await net.email.all("example.com"); runtime.log(a.spf, a.dmarc);
```

17.10.3.2 net.email.bimi

```
bimi(domain: string, opts?: { selector?: string }): Promise<{ present: false, selector: string } | { present: true, selector: string, record: string, tags: Record<string, string>, l: string, a: string }>
```

Probe BIMI: TXT(<selector>._bimi.<domain>); selector defaults to 'default'.

Parameters

- `domain` (*string*) — The domain whose <selector>._bimi.<domain> TXT record is queried for a v=BIMI1 record.
- `opts` (*{ selector?: string }, optional*) — selector names the BIMI selector to query (default 'default').

Returns: Promise<{ present: false, selector: string } | { present: true, selector: string, record: string, tags: Record<string, string>, l: string, a: string }> — selector echoes the queried selector; when found, l is the logo URL tag and a is the assertion (VMC) tag. Missing record resolves to { present: false, selector }.

Throws: Rejects only on a DNS lookup error other than NXDOMAIN; an absent record resolves to { present: false, selector }.

```
const r = await net.email.bimi("example.com", { selector: "v1" });
runtime.log(r.present && r.l);
```

17.10.3.3 net.email.dmarc

```
dmarc(domain: string): Promise<{ present: false } | { present: true, record: string, tags: Record<string, string>, policy: string, subdomain: string, percent: string, rua: string, ruf: string }>
```

Query TXT(_dmarc.<domain>) and parse policy / pct / rua / ruf tags.

Parameters

- `domain` (*string*) — The domain whose _dmarc.<domain> TXT record is queried for a v=DMARC1 record.

Returns: Promise<{ present: false } | { present: true, record: string, tags: Record<string, string>, policy: string, subdomain: string, percent: string, rua: string, ruf: string }> — tags is the full parsed tag map; policy/subdomain/percent/rua/ruf surface the common p/sp/pct/rua/ruf tags. Missing record resolves to { present: false }.

Throws: Rejects only on a DNS lookup error other than NXDOMAIN; an absent record resolves to { present: false }.

```
const r = await net.email.dmarc("example.com"); runtime.log(r.present && r.policy);
```

17.10.3.4 net.email.mtaSts

```
mtaSts(domain: string): Promise<{ present: false } | { present: true, record: string, txt: { v: string, id: string }, policy?: { version?: string, mode?: string, mx?: string[], maxAge?: number | string }, policyError?: string }>
```

Probe MTA-STS: TXT(_mta-sts.<domain>) plus the fetched policy file.

Parameters

- `domain` (*string*) — The domain whose _mta-sts.<domain> TXT record and well-known policy file are probed.

Returns: `Promise<{ present: false } | { present: true, record: string, txt: { v: string, id: string }, policy?: { version?: string, mode?: string, mx?: string[], maxAge?: number | string }, policyError?: string }>` — txt carries the versioned id from the TXT marker; policy is the parsed well-known file (mode + mx + maxAge), or policyError holds the fetch/parse error string when the file couldn't be retrieved. Missing TXT resolves to `{ present: false }`.

Throws: Rejects only on a DNS lookup error other than NXDOMAIN; an absent record resolves to `{ present: false }`. A policy-file fetch failure is captured in policyError, not thrown.

```
const r = await net.email.mtaSts("example.com"); runtime.log(r.present &&
r.policy?.mode);
```

17.10.3.5 net.email.send

```
send(opts: { to: string | string[], from: string, subject?: string, body?: string,
html?: string, attachments?: { filename: string, contentType?: string, bytes:
Uint8Array | ArrayBuffer }[], headers?: Record<string, string>, server: { host:
string, port?: number, auth?: { username: string, password: string }, tls?:
"starttls" | "tls" | "none" }, timeout?: number }): Promise<{ accepted: string[],
rejected: { address: string, reason: string }[]>
```

Send an outbound email: `net.email.send({to, from, subject, body, html?, attachments?, headers?, server: {host, port?, auth?, tls?}, timeout?})` → `Promise<{accepted: string[], rejected: [{address, reason}]}>`. One TCP connection per call; per-recipient outcome captured. Transport failures throw; per-RCPT rejections surface in the result. TLS modes: starttls (default), tls, none.

Parameters

- `opts` (*{ to: string | string[], from: string, subject?: string, body?: string, html?: string, attachments?: { filename: string, contentType?: string, bytes: Uint8Array | ArrayBuffer }[], headers?: Record<string, string>, server: { host: string, port?: number, auth?: { username: string, password: string }, tls?: "starttls" | "tls" | "none" }, timeout?: number }*) — to (string or array) and from are required, as is server.host. subject/body/html shape the message: body alone → text/plain, body+html → multipart/alternative, any attachments → multipart/mixed. attachments carry raw bytes (contentType defaults to application/octet-stream). headers adds custom headers (CR/LF stripped). server.port defaults to 587; server.auth enables PLAIN auth (skipped when tls is 'none'); server.tls picks the transport: 'starttls' (default), implicit 'tls', or 'none'. timeout is the dial / connection timeout in ms (default 30000).

Returns: `Promise<{ accepted: string[], rejected: { address: string, reason: string }[] }>` — accepted lists recipients the server accepted at RCPT TO; rejected pairs each refused address with the server's reason. The DATA body is sent only when at least one recipient was accepted.

Throws: Rejects on missing required fields (to / from / server.host), transport / protocol failures (dial, HELO, STARTTLS unavailable when requested, AUTH, MAIL FROM, DATA), or timeout. Per-recipient RCPT rejections are returned in rejected, not thrown.

```
const r = await net.email.send({
  to: "to@example.com", from: "from@example.com", subject: "hi", body: "hello",
  server: { host: "smtp.example.com", port: 587, auth: { username: "u", password:
    "p" } },
});
runtime.log(r.accepted, r.rejected);
```

17.10.3.6 net.email.spf

```
spf(domain: string): Promise<{ present: false } | { present: true, record: string,
mechanisms: string[], allPolicy: string }>
```

Query TXT(<domain>) for SPF, return record + parsed mechanisms + all-policy.

Parameters

- `domain` (*string*) — The domain whose apex TXT records are queried for an SPF (v=spf1) record.

Returns: `Promise<{ present: false } | { present: true, record: string, mechanisms: string[], allPolicy: string }>` — when found, record is the raw SPF string, mechanisms is the tokenised list after v=spf1, and allPolicy summarises the trailing all-style mechanism (pass / fail / softfail / neutral). Missing record resolves to `{ present: false }`.

Throws: Rejects only on a DNS lookup error other than NXDOMAIN; an absent record resolves to `{ present: false }`.

```
const r = await net.email.spf("example.com"); if (r.present)
runtime.log(r.allPolicy);
```

17.10.3.7 net.email.tlsRpt

```
tlsRpt(domain: string): Promise<{ present: false } | { present: true, record:
string, tags: Record<string, string>, rua: string }>
```

Probe TLS-RPT: TXT(_smtp._tls.<domain>) and parse rua.

Parameters

- `domain` (*string*) — The domain whose _smtp._tls.<domain> TXT record is queried for a v=TLSRPTv1 record.

Returns: `Promise<{ present: false } | { present: true, record: string, tags: Record<string, string>, rua: string }>` — tags is the parsed tag map; rua surfaces the report-URI tag. Missing record resolves to `{ present: false }`.

Throws: Rejects only on a DNS lookup error other than NXDOMAIN; an absent record resolves to `{ present: false }`.

```
const r = await net.email.tlsRpt("example.com"); runtime.log(r.present && r.rua);
```

17.10.4 net.http

17.10.4.1 net.http.get

```
get(url: string): Promise<{ status: number, body: string, bodyBytes: Uint8Array }>
```

Perform an HTTP GET with a 5-second default timeout. Returns { status, body, bodyBytes }.

Parameters

- `url` (*string*) — Absolute request URL (http:// or https://).

Returns: `Promise<{ status: number, body: string, bodyBytes: Uint8Array }>` — the HTTP status code, the response body as a UTF-8 string, and `bodyBytes` (the raw, undecoded bytes; pair with `text.charset.decode` for non-UTF-8 content like ISO-8859-1). Redirects are followed by the default client.

Throws: Rejects on transport errors (DNS failure, connection refused, TLS handshake) or if the 5s context deadline is exceeded. 4xx/5xx responses do NOT reject — they surface via status.

```
const r = await net.http.get("https://example.com"); runtime.log(r.status);
```

17.10.4.2 net.http.post

```
post(url: string, body?: string): Promise<{ status: number, body: string, bodyBytes: Uint8Array }>
```

Perform an HTTP POST with a 5-second default timeout. Returns { status, body, bodyBytes }.

Parameters

- `url` (*string*) — Absolute request URL (http:// or https://).
- `body` (*string, optional*) — Request body sent verbatim; omit or pass empty for no body. No Content-Type header is set automatically.

Returns: `Promise<{ status: number, body: string, bodyBytes: Uint8Array }>` — the HTTP status code, the response body as a UTF-8 string, and `bodyBytes` (the raw, undecoded bytes; pair with `text.charset.decode` for non-UTF-8 content).

Throws: Rejects on transport errors (DNS failure, connection refused, TLS handshake) or if the 5s context deadline is exceeded. 4xx/5xx responses do NOT reject.

```
const r = await net.http.post("https://api.example.com/x", JSON.stringify({ a: 1 }));
```

17.10.4.3 net.http.request

```
request(method: string, url: string, opts?: { headers?: Record<string, string>, body?: string | Uint8Array | ArrayBuffer, multipart?: Array<{ name: string, value: string } | { name: string, filename: string, content: string | Uint8Array | ArrayBuffer, type?: string }>, timeout?: number, retry?: number, follow?: boolean, username?: string, password?: string, maxBytes?: number }): Promise<{ status:
```

```
number, ok: boolean, headers: Record<string, string>, body: string, bodyBytes:
Uint8Array, url: string }>
```

Full HTTP client: method, url, opts {headers, body|multipart, timeout, retry, follow, username, password, maxBytes}. body is string|Uint8Array|ArrayBuffer; multipart builds a multipart/form-data upload. Returns {status, ok, headers, body, bodyBytes, url}. 4xx/5xx dont throw; retry covers transport errors + 5xx.

Parameters

- `method` (*string*) — HTTP method (GET, POST, PUT, ...); upper-cased internally. Required.
- `url` (*string*) — Absolute request URL. Required.
- `opts` (*{ headers?: Record<string, string>, body?: string | Uint8Array | ArrayBuffer, multipart?: Array<{ name: string, value: string } | { name: string, filename: string, content: string | Uint8Array | ArrayBuffer, type?: string }>, timeout?: number, retry?: number, follow?: boolean, username?: string, password?: string, maxBytes?: number } , optional*) — headers sets request headers; body is the request body (a string is sent as UTF-8, a Uint8Array/ArrayBuffer byte-for-byte); multipart builds a multipart/form-data body from an array of parts (a part with a filename is a file part whose content is string|Uint8Array|ArrayBuffer and type sets its Content-Type, default application/octet-stream; any other part is a text field carrying value) — body and multipart are mutually exclusive, and multipart sets the Content-Type header with its boundary, overriding any caller-set content-type; timeout is the per-attempt client timeout in ms (default 30000); retry is the number of extra attempts (default 0) applied only to transport errors and 5xx with linear backoff capped at 1s; follow toggles redirect following (default true — false stops at the first 3xx); username/password set HTTP Basic auth; maxBytes caps the response body size in bytes (default 256 MiB) — a caller may raise or lower it; exceeding it rejects with a size-limit error.

Returns: Promise<{ status: number, ok: boolean, headers: Record<string, string>, body: string, bodyBytes: Uint8Array, url: string }> — status is the final status code; ok is status in [200,400); headers is a lower-cased name → value map (last value wins, alphabetically ordered); body is the response text (UTF-8); bodyBytes is the raw, undecoded response bytes (pair with text.charset.decode for non-UTF-8 content like ISO-8859-1); url is the final URL after redirects.

Throws: Rejects on transport errors (DNS, connection refused, TLS) or context deadline, and after exhausting retries on a persistent transport error / 5xx. A malformed method or URL rejects immediately (not retried). 4xx/5xx that succeed at the transport level resolve normally.

```
const bytes = await fs.readBytes("logo.png");
const r = await net.http.request("POST", "https://api.example.com/upload",
{ multipart: [
  { name: "title", value: "hi" },
  { name: "file", filename: "logo.png", content: bytes, type: "image/png" },
] });
```

17.10.5 net.icmp

17.10.5.1 net.icmp.open

```
open(opts?: { network?: "ip4" | "ip6", readBuffer?: number }): Promise<{ network:
string, local: string, send(opts: { to: string, type?: number, code?: number, id?:
number, seq?: number, payload?: string | Uint8Array, body?: string |
Uint8Array }): Promise<void>, onMessage(cb: (ev: { bytes: Uint8Array, text:
string, address: string, type: number, code: number }) => void): void, onClose(cb:
() => void): void, onError(cb: (err: string) => void): void, close(): void }>
```

Open a raw ICMP socket: `net.icmp.open(opts?) → Promise<handle>`. Requires root / CAP_NET_RAW (open rejects otherwise). `opts { network?: 'ip4'|'ip6' (default 'ip4'), readBuffer? }`. `handle.send(opts)` writes a message in one of two modes: Echo mode `{ to, type?, code?, id?, seq?, payload? }` (type defaults to the network's echo request), or raw mode `{ to, type, code?, body }` where body (`Uint8Array|string`) is marshalled verbatim (`icmp.RawBody`) for hand-built non-Echo messages such as destination-unreachable — in raw mode type is required and body is mutually exclusive with id/seq/payload. `push/` callback model — `onMessage(cb)` events carry `{ address, type, code }`; `onClose(cb)/onError(cb)`; `handle.network/local`; `handle.close()`.

Parameters

- `opts` (*{ network?: "ip4" | "ip6", readBuffer?: number }*, optional) — network selects the IP version (default 'ip4'); `readBuffer` is the inbound channel capacity (default 64).

Returns: `Promise<{ network: string, local: string, send(opts: { to: string, type?: number, code?: number, id?: number, seq?: number, payload?: string | Uint8Array, body?: string | Uint8Array }): Promise<void>, onMessage(cb: (ev: { bytes: Uint8Array, text: string, address: string, type: number, code: number }) => void): void, onClose, onError, close(): void }>` — a raw-ICMP handle. `send` writes an ICMP message: omit body for an Echo-shaped body (type defaults to the network's echo request, id/seq/payload optional); provide body for a verbatim raw body (type required, mutually exclusive with id/seq/payload) to hand-build non-Echo messages such as destination-unreachable. `to` is the destination address. `onMessage` fires per received packet with the marshalled body plus `{ address, type, code }` meta.

Throws: Rejects if the raw socket can't be opened — typically because it needs root / CAP_NET_RAW. `send` rejects on resolve/marshal/write errors and throws synchronously: after close, if `opts.to` is missing, if a raw body is sent without `opts.type`, or if body is combined with id/seq/payload. Read errors surface via `onError`.

```
const p = await net.icmp.open();
p.onMessage(ev => runtime.log(ev.address, ev.type));
await p.send({ to: "8.8.8.8", id: 1, seq: 1, payload: "ping" });
// raw (non-Echo) body — e.g. a hand-built destination-unreachable:
await p.send({ to: "8.8.8.8", type: 3, code: 1, body: new Uint8Array([0, 0, 0,
0]) });
```


17.10.6 net.load

17.10.6.1 net.load.http

```
http(opts: { url: string, method?: string, headers?: Record<string, string>,
body?: string, concurrency?: number, requests?: number, duration?: number, rps?:
number, timeout?: number, confirm?: boolean }): Promise<{ target: string, method:
string, concurrency: number, durationMs: number, sent: number, completed: number,
failed: number, rps: number, errorRate: number, latency: { min: number, mean:
number, p50: number, p90: number, p95: number, p99: number, max: number },
statusCounts: Record<string, number>, errors: Record<string, number> }>
```

Authorized HTTP load / resilience self-test: drive a target with a worker pool at a given concurrency for a fixed `requests` count or `duration` (exactly one required), optional client-side `rps` cap, and return a latency/error report. Dual-use guardrail: public targets are refused unless `confirm:true` (loopback/private/localhost hosts are always allowed); concurrency is capped at 1000. Defensive self-testing only — no raw packets, spoofing, or amplification.

Parameters

- `opts` (*{ url: string, method?: string, headers?: Record<string, string>, body?: string, concurrency?: number, requests?: number, duration?: number, rps?: number, timeout?: number, confirm?: boolean }*) — `url` is the http/https target (required). `method` defaults to GET. `headers/body` set the request. `concurrency` is the number of parallel workers (default 10, clamped to [1,1000]). Provide exactly one of `requests` (total requests to send) or `duration` (run time in ms). `rps` caps client-side throughput (req/s, 0 = unlimited). `timeout` is the per-request timeout in ms (default 10000). `confirm:true` asserts you are authorized and is required to target a public host.

Returns: `Promise<LoadReport>` — `durationMs` is the wall-clock of the run; `sent` is requests started; `completed` got an HTTP response (any status); `failed` is transport errors/timeouts; `rps` is achieved throughput (completed/sec); `errorRate` is (failed + 5xx)/sent in [0,1]; `latency` is milliseconds over completed requests; `statusCounts` maps status code → count; `errors` maps transport error kind (timeout/refused/dns/reset/canceled/error) → count. A 4xx/5xx is a recorded response, not a failure.

Throws: Rejects if `url` is missing or not http/https, if neither or both of `requests` / `duration` are given, if `concurrency` exceeds the 1000 cap, or if the target host is public and `confirm:true` is not set.

```
const r = await net.load.http({ url: "http://127.0.0.1:8080/", requests: 200,
concurrency: 10 }); runtime.log(r.rps, r.latency.p95, r.errorRate);
```

17.10.7 net.netstatus

17.10.7.1 net.netstatus.check

```
check(host: string, opts?: { port?: string, timeout?: number }): Promise<{ host:
string, port: string, elapsedMs: number, reachable: boolean, dns: { ok: boolean,
ips: string[], error?: string }, tcp: { ok: boolean, latencyMs: number, error?:
string }, tls: { ok: boolean, daysRemaining: number, error?: string }, http: { ok:
boolean, status: number, error?: string } }>
```


Run DNS / TCP / TLS / HTTP against one host concurrently. Returns { reachable, dns, tcp, tls, http } — each sub-probe ok+error; reachable = dns.ok AND tcp.ok. Sub-failures are data, not throws.

Parameters

- `host` (*string*) — The host to check. Required.
- `opts` (*{ port?: string, timeout?: number }, optional*) — port is the TCP/TLS port (default “443”); timeout bounds all four sub-probes in ms (default 10000).

Returns: Promise<{ host: string, port: string, elapsedMs: number, reachable: boolean, dns: { ok: boolean, ips: string[], error?: string }, tcp: { ok: boolean, latencyMs: number, error?: string }, tls: { ok: boolean, daysRemaining: number, error?: string }, http: { ok: boolean, status: number, error?: string } }> — the four sub-probe results plus elapsed time. reachable is dns.ok AND tcp.ok; TLS/HTTP are reported but don’t gate it. Each sub-probe carries its own error string instead of failing the call.

Throws: Rejects only if host is empty. Sub-probe failures are captured as data (ok:false + error) rather than thrown.

```
const s = await net.netstatus.check("example.com"); runtime.log(s.reachable);
```

17.10.8 net.probe

17.10.8.1 net.probe.dns

```
dns(host: string, opts?: { types?: string[] }): Promise<{ a?: string[], aaaa?: string[], mx?: { preference: number, host: string }[], txt?: string[], cname?: string, ns?: string[] }>
```

Look up A / AAAA / MX / TXT / CNAME / NS records. Default: all five.

Parameters

- `host` (*string*) — The hostname to resolve.
- `opts` (*{ types?: string[] }, optional*) — types restricts the lookup to a subset (case-insensitive: ‘a’, ‘aaaa’, ‘mx’, ‘txt’, ‘cname’, ‘ns’); omit to query all.

Returns: Promise<{ a?: string[], aaaa?: string[], mx?: { preference: number, host: string }[], txt?: string[], cname?: string, ns?: string[] }> — each key is present only when that record type returned at least one entry, so use “mx” in result to test presence.

Throws: Resolves with an object omitting record types that errored or were empty; per-type lookup failures are swallowed (not thrown). Always resolves.

```
const r = await net.probe.dns("example.com", { types: ["a", "mx"] });
```

17.10.8.2 net.probe.ntp

```
ntp(host: string, opts?: { timeout?: number, port?: number | string }): Promise<{ serverTime: string, offsetMs: number, rttMs: number, stratum: number, referenceTime: string, rootDelayMs: number, rootDispersionMs: number }>
```

Query an NTPv4 server (UDP 123) and report offset, RTT, stratum, root delay / dispersion.

Parameters

- `host` (*string*) — The NTP server hostname or IP.
- `opts` (*{ timeout?: number, port?: number | string }, optional*) — `timeout` is the query timeout in ms (default 5000); `port` overrides the default UDP port 123.

Returns: `Promise<{ serverTime: string, offsetMs: number, rttMs: number, stratum: number, referenceTime: string, rootDelayMs: number, rootDispersionMs: number }>` — the server's time and reference time (RFC3339 nanos), clock offset and round-trip in ms, the stratum, and the root delay / dispersion in ms.

Throws: Rejects if the NTP query fails (unreachable server, timeout, malformed response).

```
const r = await net.probe.ntp("pool.ntp.org"); runtime.log(r.offsetMs);
```

17.10.8.3 net.probe.ping

```
ping(host: string, opts?: { mode?: "tcp" | "icmp" | "udp", count?: number, timeout?: number, port?: string }): Promise<{ host: string, ip: string, mode: string, sent: number, received: number, lossPercent: number, minMs: number, avgMs: number, maxMs: number }>
```

Reachability probe. `mode` `tcp` (default; dials `host:port`), `icmp` (real ICMP echo, needs raw-socket privileges), or `udp` (sends a datagram to a closed port and counts ICMP port-unreachable as reachable, needs `root` / `CAP_NET_RAW`). Returns `{ sent, received, lossPercent, minMs, avgMs, maxMs }`. Unreachable = received 0, no throw.

Parameters

- `host` (*string*) — The target host. Required.
- `opts` (*{ mode?: "tcp" | "icmp" | "udp", count?: number, timeout?: number, port?: string }, optional*) — `mode` selects the probe (default 'tcp' — opens count TCP connections; 'icmp' sends real ICMP echo and needs raw-socket privileges; 'udp' sends a datagram to a closed port and counts the ICMP port-unreachable reply as reachable, needs `root` / `CAP_NET_RAW`); `count` is the number of probes (default 4); `timeout` is the per-probe timeout in ms (default 5000); `port` is the TCP target port (default "80", tcp mode only).

Returns: `Promise<{ host: string, ip: string, mode: string, sent: number, received: number, lossPercent: number, minMs: number, avgMs: number, maxMs: number }>` — the resolved IP, the mode used, packets sent/received, loss percentage, and min/avg/max RTT in ms. A fully unreachable host resolves with `received:0` and `lossPercent:100` rather than rejecting.

Throws: Rejects if `host` is empty, `mode` is not one of 'tcp', 'icmp', or 'udp', DNS resolution fails (tcp mode), or the raw ICMP socket can't be opened (icmp/udp modes; typically missing raw-socket privileges). Individual lost packets are counted, not thrown.

```
const p = await net.probe.ping("example.com", { count: 3 }); runtime.log(p.lossPercent);
```

17.10.8.4 net.probe.smtp

```
smtp(host: string, opts?: { port?: string, timeout?: number, ehloName?: string }): Promise<{ host: string, port: string, banner: string, ehloDomain: string,
```

```
extensions: string[], starttls: boolean, authMechanisms: string[], sizeLimit:
number }>
```

SMTP capability probe (no mail sent). EHLO + parse extensions. Returns { banner, ehloDomain, extensions, starttls, authMechanisms, sizeLimit }. Connection failures throw.

Parameters

- `host` (*string*) — The SMTP server host. Required.
- `opts` (*{ port?: string, timeout?: number, ehloName?: string }, optional*) — `port` is the SMTP port (default “25”); `timeout` bounds the whole conversation in ms (default 10000); `ehloName` is the domain sent in EHLO (default “localhost”).

Returns: `Promise<{ host: string, port: string, banner: string, ehloDomain: string, extensions: string[], starttls: boolean, authMechanisms: string[], sizeLimit: number }>` — the greeting banner, the server’s EHLO greeting line, the raw advertised extension lines, whether STARTTLS is offered, the upper-cased AUTH mechanism names, and the SIZE limit (0 if unadvertised). No mail is sent.

Throws: Rejects if host is empty, the dial fails, or the greeting / EHLO cannot be read. A server that simply omits STARTTLS or AUTH reports them as false / empty — a finding, not an error.

```
const s = await net.probe.smtp("mail.example.com"); runtime.log(s.starttls,
s.authMechanisms);
```

17.10.8.5 net.probe.tcp

```
tcp(target: string, opts?: { timeout?: number, port?: string }): Promise<{ host:
string, port: number, ip: string, latencyMs: number }>
```

Dial a TCP target and report latency + resolved IP. Default timeout 5s.

Parameters

- `target` (*string*) — host:port to dial; a bare host uses `opts.port` (default 80).
- `opts` (*{ timeout?: number, port?: string }, optional*) — `timeout` is the dial timeout in ms (default 5000); `port` is the fallback port when `target` has no :port (default “80”).

Returns: `Promise<{ host: string, port: number, ip: string, latencyMs: number }>` — the parsed host, port, the resolved remote IP, and the connect latency in milliseconds.

Throws: Rejects if the dial fails (refused, unreachable, name resolution failure, or timeout).

```
const r = await net.probe.tcp("example.com:443"); runtime.log(r.latencyMs);
```

17.10.8.6 net.probe.tls

```
tls(target: string, opts?: { timeout?: number }): Promise<{ cn: string, issuer:
string, notBefore: string, notAfter: string, daysRemaining: number, dnsNames:
string[], serialNumber: string, fingerprintSha256: string }>
```

Open a TLS connection (InsecureSkipVerify; for probing only) and return the cert chain summary.

Parameters

- `target` (*string*) — host:port to dial; a bare host uses port 443. The host is sent as SNI.
- `opts` (*{ timeout?: number }, optional*) — timeout is the dial timeout in ms (default 5000).

Returns: `Promise<{ cn: string, issuer: string, notBefore: string, notAfter: string, daysRemaining: number, dnsNames: string[], serialNumber: string, fingerprintSha256: string }>` — leaf-certificate fields: common name, issuer CN, validity bounds (RFC3339), days until expiry, SAN DNS names, decimal serial, and the SHA-256 fingerprint as hex. Verification is skipped, so expired / mismatched certs still report.

Throws: Rejects if the dial / handshake fails, the connection is not TLS, or no peer certificates are presented.

```
const c = await net.probe.tls("example.com:443"); runtime.log(c.daysRemaining);
```

17.10.8.7 net.probe.traceroute

```
traceroute(host: string, opts?: { protocol?: "icmp" | "udp" | "tcp", port?: number, maxHops?: number, timeout?: number, probes?: number }): Promise<{ ttl: number; address: string | null; rttsMs: number[]; reached: boolean }[]>
```

Trace the network path to a host: `net.probe.traceroute(host, opts?) → Promise<hop[]>`. Sends probes with increasing TTL and reports each responding router. Needs root / CAP_NET_RAW (intermediate hops are seen via ICMP time-exceeded). `opts { protocol?: 'icmp'|'udp'|'tcp' (default 'icmp'), port?: number (udp 33434 / tcp 80), maxHops?: number (30), timeout?: number ms per probe (2000), probes?: number per hop (3) }`. IPv4 only.

Parameters

- `host` (*string*) — The destination host or IP.
- `opts` (*{ protocol?: "icmp" | "udp" | "tcp", port?: number, maxHops?: number, timeout?: number, probes?: number }, optional*) — protocol selects the probe type (icmp echo, udp to an incrementing high port, or tcp SYN via a TTL-limited connect). port is the udp/tcp target (ignored for icmp). maxHops caps the trace. timeout is the per-probe wait in ms. probes is the number of probes per hop.

Returns: One entry per hop (TTL 1..n): `ttl` is the hop number; `address` is the responding router/host IP (null if every probe at that TTL timed out); `rttsMs` are the round-trip times of the probes that answered; `reached` is true on the hop where the destination itself replied (the array ends there or at maxHops).

Throws: Rejects if the host doesn't resolve, the protocol is unknown, or the raw ICMP socket can't be opened (needs root / CAP_NET_RAW). Per-hop timeouts are normal (`address: null`), not errors.

```
// needs root / CAP_NET_RAW
const hops = await net.probe.traceroute("1.1.1.1", { protocol: "icmp", maxHops: 20 });
for (const h of hops) runtime.log(h.ttl, h.address ?? "*", h.rttsMs);
```

17.10.8.8 net.probe.whois

```
whois(domain: string, opts?: { timeout?: number }): Promise<{ raw: string,
domain?: { name: string, punycode: string, whoisServer: string, nameServers:
string[], status: string[], dnssec: boolean, createdAt: string, updatedAt:
string, expirationDate: string }, registrar?: { name: string } }>
```

Two-hop WHOIS via the IANA referral, returning the parsed record plus the raw response text.

Parameters

- `domain` (*string*) — The domain (or IP / ASN) to look up.
- `opts` (*{ timeout?: number }, optional*) — `timeout` is the wire-level WHOIS client timeout in ms (default 10000).

Returns: `Promise<{ raw: string, domain?: { name: string, punycode: string, whoisServer: string, nameServers: string[], status: string[], dnssec: boolean, createdAt: string, updatedAt: string, expirationDate: string }, registrar?: { name: string } }>` — `raw` is always the full WHOIS text; `domain` and `registrar` are best-effort parsed fields, omitted for TLDs the parser doesn't recognise.

Throws: Rejects if the WHOIS query itself fails (no referral, connection error, timeout). A parse failure is non-fatal — only `raw` is returned.

```
const w = await net.probe.whois("example.com");
runtime.log(w.domain?.expirationDate);
```

17.10.8.9 net.probe.wss

```
wss(url: string, opts?: { timeout?: number, ping?: boolean }): Promise<{ url:
string, connected: boolean, subprotocol: string, status: number, handshakeMs:
number, pingMs: number }>
```

WebSocket handshake probe. Opens ws:

Parameters

- `url` (*string*) — The WebSocket URL (ws:
- `opts` (*{ timeout?: number, ping?: boolean }, optional*) — `timeout` bounds the handshake and ping in ms (default 10000); `ping` toggles the ping/pong RTT measurement (default true).

Returns: `Promise<{ url: string, connected: boolean, subprotocol: string, status: number, handshakeMs: number, pingMs: number }>` — `connected` is true on a successful upgrade, `subprotocol` is the negotiated subprotocol (or empty), `status` is the HTTP status of the 101 upgrade, `handshakeMs` is the handshake time in ms, and `pingMs` is the ping/pong RTT (or -1 when the ping was skipped or unanswered). The connection is closed immediately.

Throws: Rejects if `url` is empty or the handshake fails (non-101, refused, bad URL). A failed ping leaves `pingMs` at -1 rather than rejecting.

```
const w = await net.probe.wss("wss://echo.websocket.org");
runtime.log(w.handshakeMs);
```

17.10.9 net.raw

17.10.9.1 net.raw.open

```
open(opts?: { iface?: string, filter?: string, readBuffer?: number }):  
Promise<{ link: string; send(spec: object | Uint8Array): Promise<{ bytesSent:  
number }>; onPacket(cb: (pkt: any) => void): void; onClose(cb: () => void): void;  
onError(cb: (msg: string) => void): void; close(): Promise<void> }>
```

Open a raw IPv4 packet engine: `net.raw.open({ iface?, filter?, readBuffer? })` → `Promise<handle>`. Sends crafted IPv4 packets (TCP flags / UDP / arbitrary IP protocol) via an `IP_HDRINCL` raw socket and receives replies via the capture path. Needs root / `CAP_NET_RAW`; Linux + macOS only (Windows rejects). `iface` defaults to the auto-detected default-route interface; `filter` is a tcpdump-like expression narrowing `onPacket`. The handle: `send(specOrBytes)` → `Promise<{ bytesSent }>`; `onPacket(cb)` delivers a decoded packet (same shape as `net.capture`); `onClose/onError`; `close()` → `Promise<void>`. `send spec`: { `dst`, `dstPort?`, `srcPort?`, `src?`, `proto?`: 'tcp'|'udp'|'ip', `protocol?`, `flags?`: `string[]`, `seq?`, `ack?`, `window?`, `ttl?`, `ipId?`, `payload?` }; or pass a `Uint8Array` to send a full IPv4 packet verbatim. Default flags ['SYN'], `ttl` 64, `window` 65535, `src` = egress IP, `srcPort` = random high.

Parameters

- `opts` (*{ `iface?`: string, `filter?`: string, `readBuffer?`: number }, optional*) — `iface` is the capture/egress interface (auto-detected if omitted); `filter` is a tcpdump-like expression evaluated post-decode; `readBuffer` sizes the inbound channel (default 64).

Returns: A handle: `link` is the capture link type; `send` crafts+fires a packet (structured spec or raw bytes) and resolves { `bytesSent` }; `onPacket` receives decoded reply packets; `close()` tears down the send socket and capture.

Throws: Rejects if the platform is Windows, the raw socket or capture can't be opened (needs root / `CAP_NET_RAW`), the egress interface can't be detected, or the filter is malformed. `send` throws on an invalid spec (missing `dst`, unknown TCP flag, bad port/ttl) and rejects on a write failure.

```
// needs root / CAP_NET_RAW  
const h = await net.raw.open({ filter: "tcp and src port 443" });  
h.onPacket(p => runtime.log(p.ip.src, p.tcp.flags));  
await h.send({ dst: "93.184.216.34", dstPort: 443, flags: ["SYN"] });  
await new Promise(r => setTimeout(r, 1000));  
await h.close();
```

17.10.9.2 net.raw.tcp

```
tcp(host: string, port: number, opts?: { flags?: string[], srcPort?: number, src?:  
string, seq?: number, ttl?: number, payload?: Uint8Array | string, timeout?:  
number, iface?: string }): Promise<{ ts: number; link: string; ip: { src: string;  
dst: string; protocol: string; ttl: number }; tcp: { srcPort: number; dstPort:  
number; seq: number; ack: number; flags: { syn: boolean; ack: boolean; fin:  
boolean; rst: boolean; psh: boolean; urg: boolean } }; payload?: Uint8Array;  
bytes: Uint8Array } | null>
```

One-shot raw TCP probe: `net.raw.tcp(host, port, opts?)` → `Promise<reply | null>`. Sends a single crafted TCP segment (default a SYN) and resolves with the first reply packet correlated by the 4-tuple, or null on timeout. SYN → SYN/ACK means open; RST means closed; null means filtered/no answer. Needs root / CAP_NET_RAW; Linux + macOS only.

Parameters

- `host` (*string*) — Destination host or IPv4 address. Required.
- `port` (*number*) — Destination TCP port. Required.
- `opts` (*{ flags?: string[], srcPort?: number, src?: string, seq?: number, ttl?: number, payload?: Uint8Array | string, timeout?: number, iface?: string }, optional*) — flags are the TCP flags to set (default ['SYN']); src/srcPort/seq/ttl/payload tune the crafted segment; timeout is the reply wait in ms (default 2000); iface overrides the auto-detected capture interface.

Returns: The decoded reply packet (same shape as `net.capture` packets), or null if no correlated reply arrived within the timeout. A SYN/ACK indicates the port is open; an RST indicates closed.

Throws: Rejects if the platform is Windows, host is empty, port is out of range, DNS resolution fails, or the raw socket / capture can't be opened (needs root / CAP_NET_RAW). A timeout is not an error — it resolves null.

```
// needs root / CAP_NET_RAW
const reply = await net.raw.tcp("scanme.nmap.org", 80, { flags: ["SYN"] });
if (reply && reply.tcp.flags.syn && reply.tcp.flags.ack) runtime.log("open");
else if (reply && reply.tcp.flags.rst) runtime.log("closed");
else runtime.log("filtered / no answer");
```

17.10.10 net.tcp

17.10.10.1 net.tcp.connect

```
connect(host: string, port: string | number, opts?: { timeout?: number,
readBuffer?: number }): Promise<{ remote: string, local: string, write(data:
string | Uint8Array): Promise<void>, onData(cb: (ev: { bytes: Uint8Array, text:
string }) => void): void, onClose(cb: () => void): void, onError(cb: (err: string)
=> void): void, close(): void }>
```

Open a TCP client socket: `net.tcp.connect(host, port, opts?)` → `Promise<handle>`. Push/callback read model — `handle.onData(cb)/onClose(cb)/onError(cb)` register listeners; `handle.write(data)` sends (string→UTF-8 / Uint8Array); `handle.remote/local` are the peer/local addresses; `handle.close()` shuts down. `opts { timeout?, readBuffer? }`.

Parameters

- `host` (*string*) — The remote host to dial.
- `port` (*string | number*) — The remote port.
- `opts` (*{ timeout?: number, readBuffer?: number }, optional*) — timeout is the dial timeout in ms (default 10000); readBuffer is the inbound channel capacity (default 64).

Returns: `Promise<{ remote: string, local: string, write(data: string | Uint8Array): Promise<void>, onData(cb: (ev: { bytes: Uint8Array, text: string }) => void): void, onClose(cb: () => void): void, onError(cb: (err: string) => void): void, close(): void }>` — a connected-socket handle. `remote/local` are the peer/local addresses. `write` resolves once the bytes are

written. `onData` fires per inbound chunk with both a `Uint8Array` and a UTF-8 text view; `onClose` fires when the stream ends; `onError` forwards non-EOF read/transport errors. `close()` tears down the connection.

Throws: The returned Promise rejects if the dial fails (refused, unreachable, timeout). After connect, write rejects on a write error and throws synchronously if called after close; transport read errors surface via the `onError` callback, not as rejections.

```
const sock = await net.tcp.connect("example.com", 80);
sock.onData(ev => runtime.log(ev.text));
await sock.write("GET / HTTP/1.0\r\n\r\n");
```

17.10.11 net.udp

17.10.11.1 net.udp.open

```
open(opts: { host?: string, port?: string | number, bind?: string, readBuffer?:
number }): Promise<{ local: string, send(data: string | Uint8Array):
Promise<void>, sendTo(data: string | Uint8Array, host: string, port: string |
number): Promise<void>, onMessage(cb: (ev: { bytes: Uint8Array, text: string,
address?: string, port?: number }) => void): void, onClose(cb: () => void): void,
onError(cb: (err: string) => void): void, close(): void }>
```

Open a UDP socket: `net.udp.open(opts) → Promise<handle>`. Connected mode `{ host, port }` exposes `send(data)`; bound mode `{ bind: '9999' }` exposes `sendTo(data, host, port)` and tags inbound events with `{ address, port }`. Push/callback model — `onMessage(cb)/onClose(cb)/onError(cb)`; `handle.local` is the bound address; `handle.close()` shuts down. `opts` also takes `readBuffer?`.

Parameters

- `opts` (`{ host?: string, port?: string | number, bind?: string, readBuffer?: number }`) — Selects the mode: connected mode needs `{ host, port }` (`net.DialUDP` to that peer); bound mode needs `{ bind }` (e.g. `'9999'`, `net.ListenUDP` on that local address). `readBuffer` is the inbound channel capacity (default 64). Provide exactly one of the two modes.

Returns: `Promise<handle>` — connected mode resolves to `{ local: string, send(data: string | Uint8Array): Promise<void>, onMessage, onClose, onError, close(): void }`; bound mode resolves to `{ local: string, sendTo(data: string | Uint8Array, host: string, port: string | number): Promise<void>, send (throws), onMessage, onClose, onError, close(): void }`. `onMessage` fires per datagram with `{ bytes: Uint8Array, text: string }` plus `{ address, port }` in bound mode. `local` is the bound address.

Throws: Rejects if neither `{ bind }` nor `{ host, port }` is supplied, or the dial / listen fails. `send/sendTo` reject on a write error and throw synchronously after close; calling `send` on a bound socket throws (use `sendTo`). Read errors surface via `onError` (a clean close ends silently).

```
const u = await net.udp.open({ host: "1.1.1.1", port: 53 });
u.onMessage(ev => runtime.log(ev.bytes.length));
await u.send(query);
```


17.11 runtime

Script-host scaffolding: logging, assertions, time, environment, runtime.argv.

17.11.1 runtime.argv

```
argv: string[]
```

Per-script argument vector: [programName, scriptPath, ...userArgs]. argv[0] is the program name (sercon), argv[1] is the running script path, and any args after `--` on the command line start at argv[2].

Returns: string[] — the per-run argument vector. argv[0] is the program name (“sercon”), argv[1] is the running script’s path, and entries from index 2 onward are the user arguments passed after `--` on the command line. This is a value (property), not a function.

Throws: Not callable — accessing it never throws; reading an out-of-range index yields undefined per normal array semantics.

```
const target = runtime.argv[2] ?? "default-host";
```

17.11.2 runtime.assert

17.11.2.1 runtime.assert.equal

```
equal(actual: unknown, expected: unknown, msg?: string): void
```

Throw when actual !== expected (strict equality on primitives, deep equality on objects). Optional msg appears in the error.

Parameters

- `actual` (*unknown*) — The value produced by the code under test.
- `expected` (*unknown*) — The value to compare against. Primitives use strict equality; objects/arrays use deep structural equality (key order ignored).
- `msg` (*string, optional*) — Optional message prefixed onto the thrown error; defaults to “assert.equal failed”.

Returns: void — returns nothing when the values match.

Throws: Throws an Error (“<msg>: expected <expected>, got <actual>”) when the values are not equal.

```
runtime.assert.equal(1 + 1, 2, "math still works");
```

17.11.2.2 runtime.assert.ok

```
ok(cond: unknown, msg?: string): void
```

Throw when cond is falsy. Optional msg appears in the error.

Parameters

- `cond` (*unknown*) — A value tested for truthiness (JS coercion: 0, “”, null, undefined, NaN and false are falsy).

- `msg` (*string, optional*) — Optional message used as the thrown error text; defaults to “assert.ok failed”.

Returns: void — returns nothing when `cond` is truthy.

Throws: Throws an Error carrying `msg` (or “assert.ok failed”) when `cond` is null or coerces to false.

```
runtime.assert.ok(user.id, "user must have an id");
```

17.11.3 runtime.clearDeadline

```
clearDeadline(): void
```

Remove the running script’s wall-clock kill deadline entirely (equivalent to `-timeout 0` / `setDeadline(0)`). The script then runs without a timeout.

Returns: void.

Throws: Does not throw.

```
runtime.clearDeadline(); // run without a timeout
```

17.11.4 runtime.clipboard

17.11.4.1 runtime.clipboard.available

```
available: boolean
```

True when a host clipboard backend is on PATH (macOS `pbcopy/pbpaste`; Linux `wl-clipboard` or `xclip/xsel`; Windows `clip` + PowerShell). Cheap, side-effect-free advisory — does not touch the clipboard; gate calls on it to self-skip on headless boxes.

Returns: boolean — false when no clipboard CLI is installed (e.g. a headless server). The authoritative signal is whether read/write throw.

Throws: Never throws.

```
if (!runtime.clipboard.available) runtime.log("no clipboard — skipping");
```

17.11.4.2 runtime.clipboard.imageAvailable

```
imageAvailable: boolean
```

True when a PNG image clipboard backend is usable on this host (macOS `pngpaste`; Linux `wl-clipboard` or `xclip` — not `xsel`; Windows PowerShell). Cheap advisory; does not touch the clipboard.

Returns: boolean — false when no image backend is installed (e.g. macOS without `pngpaste`). Gate `readImage/writeImage` on it.

Throws: Never throws.

```
if (runtime.clipboard.imageAvailable) { /* ... */ }
```

17.11.4.3 runtime.clipboard.read

```
read(...args: unknown[]): Promise<string>
```

Read the host OS system clipboard as UTF-8 text. Async (shells out to the platform clipboard tool). An empty clipboard resolves with "".

Returns: Promise resolving to the clipboard text ("" when the clipboard is empty or holds no text).

Throws: Rejects when no clipboard backend is on PATH (a clean "runtime.clipboard: no clipboard backend ..." message) or the underlying command fails / times out (5s).

```
const text = await runtime.clipboard.read();
```

17.11.4.4 runtime.clipboard.readImage

```
readImage(...args: unknown[]): Promise<Uint8Array | null>
```

Read the host clipboard image as PNG bytes. Async (shells out). Resolves null when the clipboard holds no image.

Returns: Promise resolving to the clipboard image as PNG bytes, or null when no image is present.

Throws: Rejects when no image backend is available (see imageAvailable) or the underlying command fails / times out (5s).

```
const png = await runtime.clipboard.readImage();
```

17.11.4.5 runtime.clipboard.write

```
write(text: string): Promise<void>
```

Replace the host OS system clipboard with the given text. Async (shells out to the platform clipboard tool); text is passed via stdin (no shell-injection risk).

Parameters

- `text` (*string*) — The text to place on the clipboard (non-string values are `String()`-coerced).

Returns: Promise resolving when the clipboard has been set.

Throws: Rejects when no clipboard backend is on PATH or the underlying command fails / times out (5s).

```
await runtime.clipboard.write("copied from sercon");
```

17.11.4.6 runtime.clipboard.writeImage

```
writeImage(png: Uint8Array): Promise<void>
```

Set the host clipboard image from PNG bytes. Async (shells out). The input must be a PNG (validated by signature) — other formats reject.

Parameters

- `png` (*Uint8Array*) — PNG image bytes (must begin with the PNG signature).

Returns: Promise resolving when the clipboard image is set.

Throws: Rejects when the data is not a PNG, no image backend is available, or the command fails / times out (5s).

```
await runtime.clipboard.writeImage(pngBytes);
```

17.11.5 runtime.env

17.11.5.1 runtime.env.get

```
get(name: string): string | undefined
```

Read an environment variable. Returns undefined when unset (not empty string).

Parameters

- `name` (*string*) — Environment variable name to look up.

Returns: string — the variable's value, or undefined when the variable is not set. A variable set to the empty string returns "", which is distinct from undefined.

Throws: Never throws.

```
const home = runtime.env.get("HOME") ?? "/tmp";
```

17.11.5.2 runtime.env.load

```
load(path: string, opts?: { override?: boolean }): Promise<{ [key: string]: string }>
```

Load a .env file and apply it to the process environment, so subsequent runtime.env.get (and any spawned subprocess) see the values. Parses KEY=VALUE lines (# comments, blank lines, optional `export`, surrounding quotes stripped; no shell expansion). An already-set variable is left untouched unless opts.override is true. Async; resolves to the parsed pairs.

Parameters

- `path` (*string*) — Path to the .env file.
- `opts` (*{ override?: boolean }, optional*) — override: overwrite variables already present in the environment (default false — existing values win).

Returns: A promise resolving to the parsed key/value pairs from the file (all of them, regardless of whether each was applied).

Throws: Rejects if the file is missing or unreadable, or if a line is malformed ("line N: ..."). Throws a TypeError if path is not a string.

```
await runtime.env.load(".env");
const url = runtime.env.get("DATABASE_URL");
```

17.11.6 runtime.getDeadline

```
getDeadline(): number | null
```

Return the milliseconds remaining until the running script's kill deadline, or null when no deadline is active (disabled or started with -timeout 0).

Returns: number — ms remaining (≥ 0) — or null when there is no active deadline.

Throws: Does not throw.

```
const left = runtime.getDeadline(); // e.g. 9871, or null
```

17.11.7 runtime.log

```
log(args: unknown[]): void
```

Print one space-separated line of the arguments to stdout. Primitives print raw; objects/arrays render as JSON (circular refs fall back to [object Object]). The script-side equivalent of console.log.

Parameters

- `args` (*unknown[]*) — Zero or more values to print. They are joined with single spaces; primitives stringify directly, objects/arrays are JSON-encoded.

Returns: void — writes a single newline-terminated line to stdout.

Throws: Never throws; unserialisable objects degrade to [object Object] rather than erroring.

```
runtime.log("count", 3, { ok: true }); // count 3 {"ok":true}
```

17.11.8 runtime.open

```
open(target: string): Promise<void>
```

Open a URL or file path in the OS default handler (the user's normal GUI browser for URLs) via the platform opener — macOS `open`, Linux `xdg-open` / `gnome-open`, Windows `rundll32/start` — feature-detected on PATH. Fire-and-forget: resolves once the opener is spawned, not when the browser closes. The target is passed as a single argument with no shell, so special characters can't inject.

Parameters

- `target` (*string*) — A URL or file path to hand to the OS default handler. Passed as one argument with no shell interpolation; must be non-empty.

Returns: `Promise<void>` — resolves once the opener process has been spawned (the browser's lifetime is not awaited).

Throws: Throws (“runtime.open: ...”) if target is missing/empty, if no OS opener is found on PATH (see runtime.openAvailable), or if the opener fails to start.

```
await runtime.open("https://example.com");
```

17.11.9 runtime.openAvailable

```
openAvailable: boolean
```

Whether an OS opener is available on PATH — an advisory for runtime.open. True when the platform opener (open / xdg-open / gnome-open / rundll32) is found. A value (property), not a function.

Returns: boolean — true if runtime.open can launch a handler on this host; false otherwise (in which case runtime.open throws).

Throws: Not callable — accessing it never throws.

```
if (runtime.openAvailable) await runtime.open(url);
```

17.11.10 runtime.secrets

17.11.10.1 runtime.secrets.available

```
available: boolean
```

True when an OS keystore backend (macOS Keychain, Linux Secret Service, Windows Credential Manager) is plausibly reachable this run. Cheap advisory hint — does not touch the keystore; gate calls on it to self-skip on headless boxes.

Returns: boolean — false on a host with no reachable keystore (e.g. a headless Linux box without a D-Bus session). The authoritative signal is whether get/set/delete throw.

Throws: Never throws.

```
if (!runtime.secrets.available) runtime.log("no keystore – skipping");
```

17.11.10.2 runtime.secrets.delete

```
delete(name: string, account: string): Promise<boolean>
```

Remove a secret from the OS keystore under prefix + name / account. Async (keystore I/O).

Parameters

- `name` (*string*) — Secret name within the sercon prefix namespace (keystore service is prefix+name).
- `account` (*string*) — Account/user the secret belongs to.

Returns: Promise resolving true when an item was removed, false when there was nothing to remove.

Throws: Rejects if name is missing/empty or account is missing (pass "" for a single-secret name), when the keystore backend is unreachable, or the delete fails ("runtime.secrets.delete: ..."). Bounded by a 10s timeout.

```
const removed = await runtime.secrets.delete("devshop", "tess@example.com");
```

17.11.10.3 runtime.secrets.get

```
get(name: string, account: string): Promise<string | null>
```

Read a string secret from the OS keystore. The keystore service is the configured prefix + name (default "sercon/"), so reads are confined to sercon's namespace. Async (keystore I/O).

Parameters

- `name` (*string*) — Secret name within the sercon prefix namespace (the keystore service is prefix+name, e.g. "sercon/devshop").
- `account` (*string*) — Account/user the secret belongs to (may be an empty string for a single-secret name).

Returns: Promise resolving to the stored secret string, or null when no such item exists.

Throws: Rejects if name is missing/empty or account is missing (pass "" for a single-secret name), when the keystore backend is unreachable, or the read fails (a clean "runtime.secrets.get: ..." error). Bounded by a 10s timeout (a blocking macOS consent prompt rejects rather than hangs).

```
const pw = await runtime.secrets.get("devshop", "tess@example.com");
```

17.11.10.4 runtime.secrets.set

```
set(name: string, account: string, secret: string): Promise<void>
```

Store or overwrite a string secret in the OS keystore under prefix + name / account. Async (keystore I/O).

Parameters

- `name` (*string*) — Secret name within the sercon prefix namespace (keystore service is prefix+name).
- `account` (*string*) — Account/user the secret belongs to.
- `secret` (*string*) — The secret value to store.

Returns: Promise resolving when the secret is written.

Throws: Rejects if name is missing/empty, account is missing (pass "" for a single-secret name), or secret is missing; when the keystore backend is unreachable; or the write fails ("runtime.secrets.set: ..."). Bounded by a 10s timeout.

```
await runtime.secrets.set("devshop", "tess@example.com", "hunter2");
```

17.11.11 runtime.setDeadline

```
setDeadline(ms: number): void
```

Set the running script's wall-clock kill deadline to now + ms (replacing any prior deadline; ms≤0 disables it). This is the same deadline the -timeout flag sets — NOT the JS global setTimeout (which schedules a callback). Use it to extend a long task or add a deadline to a -timeout 0 run.

Parameters

- `ms` (*number*) — Milliseconds from now until the run is killed; ≤ 0 disables the deadline (same as `clearDeadline()`).

Returns: void — applies immediately to the in-flight run.

Throws: Does not throw; a non-numeric argument coerces to 0 (disable).

```
runtime.setDeadline(30000); // give this run 30s from now
```

17.11.12 runtime.stderr

17.11.12.1 runtime.stderr.capture

```
capture(fn: () => void | Promise<void>): Promise<string>
```

Run fn (sync or async) with stderr captured to an in-memory buffer, and resolve to everything it wrote. Always exclusive — unlike `to / scoped`, capture never tees; use `scoped` with `{ tee: true }` if the terminal should also see the output.

Parameters

- `fn` (*() => void | Promise<void>*) — Called with no arguments; its own return value is ignored.

Returns: `Promise<string>` — everything written to stderr while fn ran, in write order. The redirect has already been restored by the time this resolves.

Throws: Rejects with whatever fn threw or its promise rejected with, after restoring the redirect. Throws synchronously if the argument is not a function.

```
const out = await runtime.stderr.capture(() => {
  console.error("one");
  console.error("two");
});
runtime.assert.equal(out, "one\ntwo\n");
```

17.11.12.2 runtime.stderr.reset

```
reset(): void
```

Drop every redirect this script has pushed onto stderr — the whole stack, not just the last push — closing any files they opened and reverting to the real process stderr. Called automatically at the start of every Run (including each script in `sercon a.ts b.ts` and each `-watch` re-run), so a script never inherits a previous script's redirect — and once more as the

process exits, after the CLI's last reporting write, so a line callback left pushed can't take the FAIL line or the `-verbose` duration with it.

Returns: void.

Throws: Never throws.

```
runtime.stderr.to("null");
runUntrusted();
runtime.stderr.reset(); // back to the real stderr, however deep the stack got
```

17.11.12.3 runtime.stderr.scoped

```
scoped(target: "stdout" | "stderr" | "null" | { file: string, append?: boolean } |
  ((line: string) => void), opts?: { tee?: boolean }, fn: () => void |
  Promise<void>): Promise<void>
```

Apply a target to stderr for the duration of fn (sync or async), then restore — even if fn throws or its returned promise rejects. Two call shapes: `scoped(target, fn)` or `scoped(target, opts, fn)`.

Parameters

- `target` (`"stdout" | "stderr" | "null" | { file: string, append?: boolean } | ((line: string) => void)`) — Same target union as `to`.
- `opts` (`{ tee?: boolean }`, optional) — Same as `to`'s opts; only meaningful in the three-argument form.
- `fn` `() => void | Promise<void>` — Called with no arguments. Its own return value is discarded — `scoped` always resolves to undefined.

Returns: `Promise<void>` — resolves once fn (and any promise it returns) settles. The redirect has already been restored by the time this resolves.

Throws: Rejects with whatever fn threw or its promise rejected with, after restoring the redirect. Throws synchronously (before touching the stream) for the same reasons as `to`, or if the last argument is not a function.

```
await runtime.stderr.scoped("null", () => {
  console.error("never printed");
});
console.error("back to normal");
```

17.11.12.4 runtime.stderr.silence

```
silence(): () => void
```

Discard everything written to stderr until the returned restore function is called. Shorthand for `stderr.to("null")`.

Returns: `() => void` — an idempotent restore function that removes the silence and reveals whatever was beneath it.

Throws: Never throws.

```
const unsilence = runtime.stderr.silence();
console.error("nobody sees this");
unsilence();
```

17.11.12.5 runtime.stderr.target

```
target(): { kind: "stream" | "null" | "file" | "callback" | "buffer"; tee:
boolean; depth: number; name?: "stdout" | "stderr"; path?: string; append?:
boolean }
```

Inspect the effective stderr destination without changing it.

Returns: The current top-of-stack destination: kind identifies its type (name is set only for kind “stream”; path/append only for kind “file”); tee reports whether it also writes to the destination beneath it; depth is how many redirects are currently pushed (0 means stderr is unredirected).

Throws: Never throws.

```
if (runtime.stderr.target().kind === "null") console.error("currently silenced");
```

17.11.12.6 runtime.stderr.to

```
to(target: "stdout" | "stderr" | "null" | { file: string, append?: boolean } |
((line: string) => void), opts?: { tee?: boolean }): () => void
```

Point stderr at a target and return a restore function. The target is “stdout”/“stderr” (fold onto that process stream), “null” (discard), { file, append? } (a file), or a function (called with each completed line). Redirects nest: the restore pops its own entry, so nested and out-of-order restores both behave, and calling it twice is a no-op.

Parameters

- **target** (“stdout” | “stderr” | “null” | { file: string, append?: boolean } | ((line: string) => void)) — Where the stream should go. “stdout” folds stderr onto the PROCESS stdout (so stdout→stderr and stderr→stdout cannot ping-pong); “null” discards; an object writes to a file, truncating unless append is true; a function receives each completed line without its newline, delivered on the next tick in write order.
- **opts** ({ tee?: boolean }, optional) — tee also writes to the destination beneath this one in the stack — so teeing on top of a silence() still writes only to the new target. tee is rejected with the “null” target.

Returns: () => void — an idempotent restore function that removes this redirect.

Throws: Throws on an unknown target name, an object target without a file property, a file that cannot be opened, or { tee: true } combined with the “null” target. A later write failure does NOT throw: the destination is marked dead and every subsequent write falls through to the destination BENEATH it in the stack — “beneath”, not “the process stream”, which coincide only when this is the sole redirect — so a destination that goes bad costs at most the one write that was in flight when it failed. The failure is reported once on the real stderr, not once per line, and the dead destination is never written to again. The process stream itself is never taken out of service — a failed write straight to stdout/stderr is

reported once and later writes are still attempted — so one transient error cannot silence the rest of the session. A function target has its own delivery caveats, none of which throw: the pending-line queue is bounded (1024 lines) — on overflow, or on a write that re-enters the handler (e.g. the handler’s own `console.error` while it runs), the write falls through to the destination beneath instead of blocking or being lost twice; any line still queued when the redirect is popped or the run ends is written to the destination beneath rather than delivered; and a handler that throws has that one line reported once on the real `stderr` (not retried, not recovered) without aborting the rest of the run.

```
const restore = runtime.stderr.to("stdout");
console.error("now on stdout too");
restore();
```

17.11.12.7 runtime.stderr.toFile

```
toFile(path: string, opts?: { append?: boolean, tee?: boolean }): () => void
```

Redirect `stderr` to a file, truncating unless `append` is true, and return a restore function. Shorthand for `stderr.to({ file, append }, { tee })`.

Parameters

- `path` (*string*) — Output file path. Opened immediately, at the call site — a bad path or a permission error surfaces here rather than at some later write.
- `opts` (*{ append?: boolean, tee?: boolean }, optional*) — `append` opens with `O_APPEND` instead of truncating (default false). `tee` also writes to whatever destination was beneath this one when it was pushed.

Returns: `() => void` — an idempotent restore function that closes the file and removes this redirect.

Throws: Throws (“toFile: ...”) if the file cannot be opened (missing parent directory, permission denied, etc). A later write failure does NOT throw: the destination is marked dead and every subsequent write falls through to the destination BENEATH it in the stack — “beneath”, not “the process stream”, which coincide only when this is the sole redirect — so a destination that goes bad costs at most the one write that was in flight when it failed. The failure is reported once on the real `stderr`, not once per line, and the dead destination is never written to again. The process stream itself is never taken out of service — a failed write straight to `stdout/stderr` is reported once and later writes are still attempted — so one transient error cannot silence the rest of the session.

```
const stop = runtime.stderr.toFile("/tmp/err.log", { append: true, tee: true });
console.error("on screen and in the file");
stop();
```

17.11.13 runtime.stdin

17.11.13.1 runtime.stdin.from

```
from(source: { file: string } | { text: string } | "stdin"): () => void
```

Push a new stdin source and return a restore function that pops it back off. `source` is `{ file: string }` (opened immediately), `{ text: string }` (an in-memory string), or “stdin” (the real process stdin, pushed as a new entry above whatever is currently active).

Parameters

- `source` (`{ file: string } | { text: string } | “stdin”`) — Which source becomes active. A `{ file }` is opened immediately, at the call site — a missing file or a permission error surfaces here. `{ text }` and “stdin” cannot fail to open.

Returns: `() => void` — an idempotent restore function that closes the file (if any) and pops this source back off, uncovering whatever was active before.

Throws: Throws (“from: ...”) if source is missing, is an object with neither `file` nor `text`, is an unrecognised string, or (for a `{ file }` source) the file cannot be opened.

```
const restore = runtime.stdin.from({ text: "a\nb\n" });
const first = await runtime.stdin.readLine();
restore();
```

17.11.13.2 runtime.stdin.fromFile

```
fromFile(path: string): () => void
```

Push a file as the stdin source and return a restore function. Shorthand for `stdin.from({ file: path })`.

Parameters

- `path` (*string*) — Path to the file to read from. Opened immediately, at the call site.

Returns: `() => void` — an idempotent restore function that closes the file and pops this source back off.

Throws: Throws (“fromFile: ...”) if the file cannot be opened (missing, permission denied).

```
const restore = runtime.stdin.fromFile("fixtures/input.txt");
const all = await runtime.stdin.read();
restore();
```

17.11.13.3 runtime.stdin.fromString

```
fromString(text: string): () => void
```

Push an in-memory string as the stdin source and return a restore function. Shorthand for `stdin.from({ text })`.

Parameters

- `text` (*string*) — Content to serve as stdin from now on.

Returns: `() => void` — an idempotent restore function that pops this source back off.

Throws: Never throws.

```
runtime.stdin.fromString("alpha\nbeta\n");
for await (const line of runtime.stdin.lines()) runtime.log(line);
```

17.11.13.4 runtime.stdin.lines

```
lines(): AsyncIterable<string>
```

Async-iterate the current stdin source one line at a time (no trailing newline), stopping at EOF. Equivalent to calling `readLine()` in a loop; `for await` is just more idiomatic.

Returns: An async iterator yielding each line as a string.

`for await (const line of runtime.stdin.lines()) break` simply stops calling `readLine` again — it does not close or reset the source.

Throws: The iterator's `next()` rejects if the underlying source returns a read error, propagating out of the `for await`.

```
for await (const line of runtime.stdin.lines()) {
  runtime.log(line.toUpperCase());
}
```

17.11.13.5 runtime.stdin.read

```
read(...args: unknown[]): Promise<string>
```

Read every remaining byte from the current stdin source as a UTF-8 string, blocking until EOF. Concurrent calls with `readBytes/readLine/lines` serialise, so two readers can never split a read.

Returns: `Promise<string>` — the rest of the source's bytes, decoded as UTF-8, from the current position through EOF.

Throws: Rejects if the underlying source returns a read error. Blocking on the real process stdin cannot be interrupted by the run's deadline — the run itself is still killed, but this call parks until the pipe closes; a file or string source always reaches EOF. Note: when the script itself came from stdin (`sercon -`), stdin is already drained and this resolves to "" immediately.

```
const body = await runtime.stdin.read();
runtime.log("got", body.length, "bytes");
```

17.11.13.6 runtime.stdin.readBytes

```
readBytes(...args: unknown[]): Promise<Uint8Array>
```

Read every remaining byte from the current stdin source as raw bytes, blocking until EOF. Concurrent calls with `read/readLine/lines` serialise, so two readers can never split a read.

Returns: `Promise<Uint8Array>` — the rest of the source's bytes from the current position through EOF. This is goja's Go-slice-backed wrapper around the underlying `[]byte`, not a native JS `Uint8Array`: `instanceof Uint8Array` is false, though `.length` and indexing work as expected.

Throws: Rejects if the underlying source returns a read error. Blocking on the real process stdin cannot be interrupted by the run's deadline — the run itself is still killed, but this call parks until the pipe closes. Note: when the script itself came from stdin (`sercon -`), stdin is already drained and this resolves to a zero-length result immediately.

```
const b = await runtime.stdin.readBytes();
runtime.log(b.length, b[0]);
```

17.11.13.7 runtime.stdin.readLine

```
readLine(...args: unknown[]): Promise<string | null>
```

Read one line from the current stdin source, without its trailing newline. Resolves to null at EOF. A final line with no trailing newline is still returned. Concurrent calls serialise, so two readers can never split a line.

Returns: `Promise<string | null>` — the next line without its newline, or null once the source is exhausted.

Throws: Rejects if the underlying source returns a read error. Note: when the script itself came from stdin (`sercon -`), stdin is already drained and this resolves to null immediately.

```
let line;
while ((line = await runtime.stdin.readLine()) !== null) {
  runtime.log("got", line);
}
```

17.11.13.8 runtime.stdin.reset

```
reset(): void
```

Drop every source swap this script has pushed onto stdin — the whole stack, not just the last push — closing any files they opened and reverting to the real process stdin. Called automatically at the start of every Run (including each script in `sercon a.ts b.ts` and each `-watch` re-run), so a script never inherits a previous script's source swap, and once more as the process exits: the same reset covers stdin, stdout and stderr together.

Returns: void.

Throws: Never throws.

```
runtime.stdin.fromString("test input\n");
// ... read it ...
runtime.stdin.reset(); // back to the real stdin
```

17.11.13.9 runtime.stdin.scoped

```
scoped(source: { file: string } | { text: string } | "stdin", fn: () => unknown |
Promise<unknown>): Promise<unknown>
```

Push a stdin source for the duration of `fn` (sync or async), then restore — even if `fn` throws or its returned promise rejects. Unlike the `stdout/stderr` scoped (which always resolves to undefined), this resolves to `fn`’s own resolved value.

Parameters

- `source` (`{ file: string } | { text: string } | "stdin"`) — Same source union as `from`.
- `fn` `() => unknown | Promise<unknown>` — Called with no arguments; its resolved value becomes scoped’s own resolved value.

Returns: `Promise<unknown>` — resolves to whatever `fn` returned (or its promise resolved with), after the source has already been restored.

Throws: Rejects with whatever `fn` threw or its promise rejected with, after restoring the source. Throws synchronously (before touching the stack) for the same reasons as `from`, or if the second argument is not a function.

```
const total = await runtime.stdin.scoped({ text: "1\n2\n3\n" }, async () => {
  let sum = 0;
  for await (const line of runtime.stdin.lines()) sum += Number(line);
  return sum;
});
```

17.11.13.10 runtime.stdin.source

```
source(): { kind: "stdin" | "file" | "text"; path?: string; tty: boolean }
```

Describe the currently active stdin source, without reading from it.

Returns: `{ kind, path?, tty }` — `kind` is which source is active (`path` is set only for kind “file”); `tty` is true only when `kind` is “stdin” and the real process stdin is a terminal — a file or string source is never a terminal, and a script itself read from stdin (`sercon -`) leaves the real stdin already drained.

Throws: Never throws.

```
if (runtime.stdin.source().tty) console.log("waiting for interactive input...");
```

17.11.14 runtime.stdout

17.11.14.1 runtime.stdout.capture

```
capture(fn: () => void | Promise<void>): Promise<string>
```

Run `fn` (sync or async) with stdout captured to an in-memory buffer, and resolve to everything it wrote. Always exclusive — unlike `to / scoped`, `capture` never tees; use `scoped` with `{ tee: true }` if the terminal should also see the output.

Parameters

- `fn` `() => void | Promise<void>` — Called with no arguments; its own return value is ignored.

Returns: `Promise<string>` — everything written to stdout while `fn` ran, in write order. The redirect has already been restored by the time this resolves.

Throws: Rejects with whatever fn threw or its promise rejected with, after restoring the redirect. Throws synchronously if the argument is not a function.

```
const out = await runtime.stdout.capture(() => {
  console.log("one");
  console.log("two");
});
runtime.assert.equal(out, "one\ntwo\n");
```

17.11.14.2 runtime.stdout.reset

```
reset(): void
```

Drop every redirect this script has pushed onto stdout — the whole stack, not just the last push — closing any files they opened and reverting to the real process stdout. Called automatically at the start of every Run (including each script in `sercon a.ts b.ts` and each `-watch` re-run), so a script never inherits a previous script's redirect — and once more as the process exits, after the CLI's last reporting write, so a line callback left pushed can't take the default-export JSON or the PASS/FAIL line with it.

Returns: void.

Throws: Never throws.

```
runtime.stdout.to("null");
runUntrusted();
runtime.stdout.reset(); // back to the real stdout, however deep the stack got
```

17.11.14.3 runtime.stdout.scoped

```
scoped(target: "stdout" | "stderr" | "null" | { file: string, append?: boolean } |
  ((line: string) => void), opts?: { tee?: boolean }, fn: () => void |
  Promise<void>): Promise<void>
```

Apply a target to stdout for the duration of fn (sync or async), then restore — even if fn throws or its returned promise rejects. Two call shapes: `scoped(target, fn)` or `scoped(target, opts, fn)`.

Parameters

- `target` (`"stdout"` | `"stderr"` | `"null"` | `{ file: string, append?: boolean }` | `((line: string) => void)`) — Same target union as `to`.
- `opts` (`{ tee?: boolean }`, optional) — Same as `to`'s `opts`; only meaningful in the three-argument form.
- `fn` `() => void` | `Promise<void>` — Called with no arguments. Its own return value is discarded — `scoped` always resolves to undefined.

Returns: `Promise<void>` — resolves once fn (and any promise it returns) settles. The redirect has already been restored by the time this resolves.

Throws: Rejects with whatever fn threw or its promise rejected with, after restoring the redirect. Throws synchronously (before touching the stream) for the same reasons as `to`, or if the last argument is not a function.


```
await runtime.stdout.scoped("null", () => {
  console.log("never printed");
});
console.log("back to normal");
```

17.11.14.4 runtime.stdout.silence

```
silence(): () => void
```

Discard everything written to stdout until the returned restore function is called. Shorthand for `stdout.to("null")`.

Returns: `() => void` — an idempotent restore function that removes the silence and reveals whatever was beneath it.

Throws: Never throws.

```
const unsilence = runtime.stdout.silence();
console.log("nobody sees this");
unsilence();
```

17.11.14.5 runtime.stdout.target

```
target(): { kind: "stream" | "null" | "file" | "callback" | "buffer"; tee:
boolean; depth: number; name?: "stdout" | "stderr"; path?: string; append?:
boolean }
```

Inspect the effective stdout destination without changing it.

Returns: The current top-of-stack destination: `kind` identifies its type (name is set only for kind “stream”; path/append only for kind “file”); `tee` reports whether it also writes to the destination beneath it; `depth` is how many redirects are currently pushed (0 means stdout is unredirected).

Throws: Never throws.

```
if (runtime.stdout.target().kind === "null") console.log("currently silenced");
```

17.11.14.6 runtime.stdout.to

```
to(target: "stdout" | "stderr" | "null" | { file: string, append?: boolean } |
((line: string) => void), opts?: { tee?: boolean }): () => void
```

Point stdout at a target and return a restore function. The target is “stdout”/“stderr” (fold onto that process stream), “null” (discard), `{ file, append? }` (a file), or a function (called with each completed line). Redirects nest: the restore pops its own entry, so nested and out-of-order restores both behave, and calling it twice is a no-op.

Parameters

- `target` (“stdout” | “stderr” | “null” | `{ file: string, append?: boolean }` | `((line: string) => void)`) — Where the stream should go. A stream name folds onto that PROCESS stream (so stdout→stderr and stderr→stdout cannot ping-pong); “null” discards; an object writes to a

file, truncating unless append is true; a function receives each completed line without its newline, delivered on the next tick in write order.

- `opts` (*{ tee?: boolean }, optional*) — tee also writes to the destination beneath this one in the stack — so teeing on top of a `silence()` still writes only to the new target. tee is rejected with the “null” target.

Returns: `() => void` — an idempotent restore function that removes this redirect.

Throws: Throws on an unknown target name, an object target without a `file` property, a file that cannot be opened, or `{ tee: true }` combined with the “null” target. A later write failure does NOT throw: the destination is marked dead and every subsequent write falls through to the destination BENEATH it in the stack — “beneath”, not “the process stream”, which coincide only when this is the sole redirect — so a destination that goes bad costs at most the one write that was in flight when it failed. The failure is reported once on the real `stderr`, not once per line, and the dead destination is never written to again. The process stream itself is never taken out of service — a failed write straight to `stdout/stderr` is reported once and later writes are still attempted — so one transient error cannot silence the rest of the session. A function target has its own delivery caveats, none of which throw: the pending-line queue is bounded (1024 lines) — on overflow, or on a write that re-enters the handler (e.g. the handler’s own `console.log` while it runs), the write falls through to the destination beneath instead of blocking or being lost twice; any line still queued when the redirect is popped or the run ends is written to the destination beneath rather than delivered; and a handler that throws has that one line reported once on the real `stderr` (not retried, not recovered) without aborting the rest of the run.

```
const restore = runtime.stdout.to({ file: "/tmp/out.log" }, { tee: true });
console.log("goes to both");
restore();
```

17.11.14.7 runtime.stdout.toFile

```
toFile(path: string, opts?: { append?: boolean, tee?: boolean }): () => void
```

Redirect `stdout` to a file, truncating unless append is true, and return a restore function. Shorthand for `stdout.to({ file, append }, { tee })`.

Parameters

- `path` (*string*) — Output file path. Opened immediately, at the call site — a bad path or a permission error surfaces here rather than at some later write.
- `opts` (*{ append?: boolean, tee?: boolean }, optional*) — append opens with `O_APPEND` instead of truncating (default false). tee also writes to whatever destination was beneath this one when it was pushed.

Returns: `() => void` — an idempotent restore function that closes the file and removes this redirect.

Throws: Throws (“toFile: ...”) if the file cannot be opened (missing parent directory, permission denied, etc). A later write failure does NOT throw: the destination is marked dead and every subsequent write falls through to the destination BENEATH it in the stack — “beneath”, not “the process stream”, which coincide only when this is the sole redirect — so a destination that goes bad costs at most the one write that was in flight when it failed. The

failure is reported once on the real stderr, not once per line, and the dead destination is never written to again. The process stream itself is never taken out of service — a failed write straight to stdout/stderr is reported once and later writes are still attempted — so one transient error cannot silence the rest of the session.

```
const stop = runtime.stdout.toFile("/tmp/out.log", { append: true, tee: true });
console.log("on screen and in the file");
stop();
```

17.11.15 runtime.termSize

```
termSize(): { columns: number; rows: number; tty: boolean }
```

Current terminal size of the controlling TTY (stdout) as { columns, rows, tty }. Synchronous (a single ioctl). When stdout is not a terminal (piped/redirected) tty is false and columns/rows fall back to \$COLUMNS/\$LINES, then 80x24 — so scripts can format tables/progress bars without special-casing the non-TTY path.

Returns: { columns, rows, tty } — terminal width/height in character cells; tty is true only when stdout is a real terminal (otherwise the values are the \$COLUMNS/\$LINES-or-80x24 fallback).

Throws: Never throws; a non-terminal stdout yields the fallback with tty:false.

```
const { columns } = runtime.termSize(); const bar = "=".repeat(Math.min(columns, 40));
```

17.11.16 runtime.time

17.11.16.1 runtime.time.format

```
format(ms: number, layout: string, tz?: string): string
```

Format a unix-ms timestamp through strftime tokens. Optional IANA tz (e.g. 'Europe/Stockholm'); default is the host's local zone.

Parameters

- `ms` (*number*) — Milliseconds since the Unix epoch (e.g. from `time.nowMs`). Coerced to an integer.
- `layout` (*string*) — strftime-style layout. Supported tokens: %Y %y %m %d %H %M %S %T %F %j %A %a %B %b %z %Z and %% (literal percent). Unknown %X tokens pass through verbatim.
- `tz` (*string, optional*) — IANA timezone name (e.g. "Europe/Stockholm", "UTC"). Defaults to the host's local zone when omitted/null/undefined.

Returns: string — the timestamp rendered in tz with the given layout.

Throws: Throws ("time.format: ...") if tz is not a loadable IANA timezone name.

```
const s = runtime.time.format(runtime.time.nowMs(), "%F %T", "UTC");
```

17.11.16.2 runtime.time.nowMs

```
nowMs(): number
```

Wall-clock milliseconds since the Unix epoch.

Returns: number — integer milliseconds since 1970-01-01T00:00:00Z (host wall clock).

Throws: Never throws.

```
const t0 = runtime.time.nowMs();
```

17.11.16.3 runtime.time.sleep

```
sleep(ms: number): Promise<void>
```

Resolve after `ms` milliseconds. Cancellable via the engine timeout.

Parameters

- `ms` (*number*) — Delay in milliseconds. Coerced to an integer; non-positive values resolve effectively immediately.

Returns: Promise<void> — resolves once the delay elapses.

Throws: Rejects if the run is cancelled or hits its timeout before the delay elapses (the underlying context is cancelled).

```
await runtime.time.sleep(250);
```

17.12 server

Network servers: HTTP/HTTPS listeners with routing, middleware, static files, WebSocket upgrade.

17.12.1 server.http

17.12.1.1 server.http.listen

```
listen(opts: { port: number; host?: string; routes: Record<string, ((req: Request, res: Response) => unknown) | { use?: ((req: Request, res: Response, next: () => Promise<void>) => unknown)[]; handler: (req: Request, res: Response) => unknown } >; use?: ((req: Request, res: Response, next: () => Promise<void>) => unknown)[]; onError?: (err: unknown, req: Request, res: Response) => unknown; maxBodyBytes?: number }): { address: string; stopped: Promise<void>; close(): Promise<void> }
```

Bind an HTTP listener: `server.http.listen({port, host?, routes, use?, onError?, maxBodyBytes?})` → handle with `.address`, `.close()`, `.stopped` Promise. `routes` is a map of stdlib `http.ServeMux` patterns (`'GET /users/{id}'`) to handlers `(req, res) => res.json({...})` or `{use: [...], handler: fn}` for per-route middleware. Optional `onError(err, req, res)` renders a custom response when a handler/middleware throws or rejects (else a stock 500). Request bodies are capped at `maxBodyBytes` (default 32 MiB) — an over-limit body gets a 413 before any route handler or middleware runs. Handlers can call `res.upgradeWebSocket(opts?)` to hijack the connection and return an `AsyncIterable<WSMessage>` with `.send` / `.close` — `for await (const msg of ws)` walks

frames; msg is {type:'text',text} or {type:'binary',bytes:Uint8Array}. Handlers can also call `res.sse(opts?)` to start a one-way Server-Sent Events stream — returns a handle with `send(data)` (string → `data`: or {event,data,id,retry} with object data JSON-encoded), `close()`, and a `closed` Promise (resolves on close or client disconnect); `opts`: {keepAlive?: ms, retry?: ms}.

Parameters

- `opts` ({ `port`: number; `host?`: string; `routes`: Record<string, ((`req`: Request, `res`: Response) => unknown) | { `use?`: ((`req`: Request, `res`: Response, `next`: () => Promise<void>) => unknown)[]; `handler`: (`req`: Request, `res`: Response) => unknown }>; `use?`: ((`req`: Request, `res`: Response, `next`: () => Promise<void>) => unknown)[]; `onError?`: (`err`: unknown, `req`: Request, `res`: Response) => unknown; `maxBodyBytes?`: number }) — Listener config. `port` is required; `host` defaults to “0.0.0.0”. `routes` maps Go 1.22+ ServeMux patterns (“GET /”, “POST /users/{id}”, “GET /assets/{rest...”}) to a handler function or a {`use`, `handler`} object for per-route middleware. `use` is a global middleware chain run before every route. `onError(err, req, res)` is invoked when a handler or middleware throws/rejects — render a response with `res.*` (it may be async); if it doesn’t finalize, or itself throws, the stock 500 is sent. `maxBodyBytes` caps the size of an inbound request body read into memory before dispatch; absent or `<=0` falls back to a 32 MiB default, and an over-limit body is rejected with 413 without invoking any JS. Under `sercon serve`, `-port-override` replaces `port`.

Returns: A server handle (returned synchronously): address is ‘tcp/host:port’ (resolved, so a port:0 ephemeral bind reports its OS-chosen port); stopped resolves when the server stops (rejects if Serve fails with a non-close error); `close()` begins a graceful 30s shutdown and resolves with the same stopped Promise.

Throws: Throws synchronously if `opts` is missing, `port` is 0/absent, `routes` is missing, a `use[]` entry or route value is not a function/valid {`use`, `handler`}, or the bind fails (e.g. address already in use).

```
const srv = server.http.listen({
  port: 8080,
  routes: { "GET /": (req, res) => res.json({ ok: true }) },
});
runtime.log(srv.address);
await srv.close();
```

17.12.1.2 server.http.static

```
static(opts: { dir: string; stripPrefix?: string; index?: string; etag?: boolean }): (req: Request, res: Response) => void
```

Static-file mount: `server.http.static({dir, stripPrefix, index?, etag?})` → handler. Assign to a wildcard route (GET /assets/{rest...}). Internally `stdlib http.FileServer` with `stripPrefix`; ETag/Last-Modified/range requests work; no directory listing.

Parameters

- `opts` ({ `dir`: string; `stripPrefix?`: string; `index?`: string; `etag?`: boolean }) — `dir` is the filesystem root to serve. `stripPrefix` is removed from the request path before lookup (set it

to the route's static prefix). index and etag are accepted but currently unused — http.FileServer already serves index.html and emits ETag/Last-Modified by default.

Returns: A route handler marker (returned synchronously). Assign it as a routes entry, typically under a wildcard pattern like 'GET /assets/{rest...}'. The route compiler unwraps it to a stdlib http.FileServer mounted under http.StripPrefix.

Throws: The call itself does not throw; an invalid dir surfaces as 404s at request time.

```
server.http.listen({
  port: 8080,
  routes: { "GET /assets/{rest...}": server.http.static({ dir: "../public",
stripPrefix: "/assets/" }) },
});
```

17.12.2 server.https

17.12.2.1 server.https.listen

```
listen(opts: { port: number; host?: string; cert: string | "self-signed"; key?:
string; routes: Record<string, ((req: Request, res: Response) => unknown) |
{ use?: ((req: Request, res: Response, next: () => Promise<void>) => unknown)[];
handler: (req: Request, res: Response) => unknown }>; use?: ((req: Request, res:
Response, next: () => Promise<void>) => unknown)[]; maxBodyBytes?: number }):
{ address: string; stopped: Promise<void>; close(): Promise<void> }
```

Like server.http.listen plus a cert (file path OR inline PEM string) and matching key. Set cert: "self-signed" to mint an ephemeral in-process dev cert (no key needed) instead — covers localhost / 127.0.0.1 / ::1 plus the listen host. No autocert (own production certs in your supervisor).

Parameters

- *opts* (*{ port: number; host?: string; cert: string | "self-signed"; key?: string; routes: Record<string, ((req: Request, res: Response) => unknown) | { use?: ((req: Request, res: Response, next: () => Promise<void>) => unknown)[]; handler: (req: Request, res: Response) => unknown }>; use?: ((req: Request, res: Response, next: () => Promise<void>) => unknown)[]; maxBodyBytes?: number }*) — Same shape as server.http.listen plus cert and key. cert is a filesystem path, an inline PEM string (detected by a leading '—BEGIN'), or the literal "self-signed" to generate an ephemeral P-256 cert in-process (key is then ignored). key is required for the path/PEM forms. TLS is pinned to a minimum of TLS 1.2. Self-signed certs fail normal client verification by design — dev only. maxBodyBytes caps the inbound request body (default 32 MiB); an over-limit body gets a 413 before any JS runs.

Returns: Same handle shape as server.http.listen; address is 'tcp/host:port'.

Throws: Throws synchronously on the same conditions as server.http.listen, plus if cert is missing, key is missing for a path/PEM cert, or the key pair fails to load/parse.

```
// Ephemeral dev cert — no openssl, no files:
const srv = server.https.listen({
  port: 8443,
```

```
cert: "self-signed",
routes: { "GET /": (req, res) => res.text("secure") },
});
```

17.12.2.2 server.https.static

```
static(opts: { dir: string; stripPrefix?: string; index?: string; etag?:
boolean }): (req: Request, res: Response) => void
```

Like `server.http.static`; same options.

Parameters

- `opts` (*{ dir: string; stripPrefix?: string; index?: string; etag?: boolean }*) — Identical to `server.http.static` — `dir` is the root, `stripPrefix` is removed from the path before lookup, `index`/`etag` are accepted but unused.

Returns: A route handler marker (returned synchronously); assign it to a wildcard route on an https listener.

Throws: The call itself does not throw; an invalid `dir` surfaces as 404s at request time.

```
server.https.listen({
  port: 8443, cert, key,
  routes: { "GET /assets/{rest...}": server.https.static({ dir: "./public",
stripPrefix: "/assets/" }) },
});
```

17.12.3 server.icmp

17.12.3.1 server.icmp.listen

```
listen(opts?: { network?: "ip4" | "ip6" }, handler?: (msg: { bytes: Uint8Array;
text: string; address: string; type: number; code: number }, reply: (opts?: { to?:
string; type?: number; code?: number; id?: number; seq?: number; payload?: string
| Uint8Array; body?: string | Uint8Array }) => Promise<void>) => void): { address:
string; close(): Promise<void> }
```

Bind a raw ICMP listener: `server.icmp.listen(opts?, (msg, reply) => {...})` → `handle { address: 'icmp/<addr>', close() }`. Raw ICMP has no ports — the socket receives ALL host ICMP traffic — and needs `root / CAP_NET_RAW` (synchronous bind throws otherwise). `opts { network?: 'ip4'|'ip6' (default 'ip4') }`. The handler runs once per received packet; `msg` is `{ bytes, text, address, type, code }` (the sender + parsed ICMP header) and `reply(opts?)` sends an ICMP message back to the sender (Echo by default, or a raw body), returning a Promise. Emits a READY line under `sercon serve` and joins graceful shutdown.

Parameters

- `opts` (*{ network?: "ip4" | "ip6" }, optional*) — network selects the IP version (default 'ip4'). There is no host/port — raw ICMP binds to all addresses.
- `handler` (*((msg: { bytes: Uint8Array; text: string; address: string; type: number; code: number }, reply: (opts?: { to?: string; type?: number; code?: number; id?: number; seq?: number; payload?: string | Uint8Array; body?: string | Uint8Array }) => Promise<void>) =>*

void, optional) — Invoked once per received ICMP packet. `msg` carries the marshalled body (bytes/text), the sender address, and the parsed type/code. `reply(opts?)` sends an ICMP message back to the sender (or `opts.to`): Echo mode { type?, code?, id?, seq?, payload? } or raw mode { type, code?, body } (body marshalled verbatim); it returns a Promise that resolves once written.

Returns: A server handle (returned synchronously): address is 'icmp/<local-addr>'; `close()` closes the socket and resolves. There is no per-connection handle (ICMP is connectionless) — `reply` is bound to the received packet's sender.

Throws: Throws synchronously if the handler is not a function, or if the raw socket can't be opened (typically because it needs root / CAP_NET_RAW). `reply` rejects on resolve/marshal/write errors and throws synchronously for the raw-body validation rules (a raw body requires type; body is mutually exclusive with id/seq/payload).

```
// Needs root / CAP_NET_RAW. Reply to every echo request with an echo reply:
const srv = server.icmp.listen({}, (msg, reply) => {
  if (msg.type === 8) reply({ type: 0, payload: msg.bytes });
});
runtime.log(srv.address);
await srv.close();
```

17.12.4 server.smtp

17.12.4.1 server.smtp.listen

```
listen(opts: { port: number; host?: string; hostname?: string; handlers: { onMail:
(env: Envelope) => boolean | string | void | Promise<boolean | string | void>;
onRcpt: (env: Envelope, to: string) => boolean | string | void | Promise<boolean |
string | void>; onData: (env: Envelope, msg: Message) => boolean | string | void |
Promise<boolean | string | void> }; auth?: { user: string, pass: string, env:
Envelope) => boolean | Promise<boolean>; starttls?: { cert: string; key: string };
allowInsecureAuth?: boolean; maxMessageBytes?: number; maxRecipients?: number;
sessionTimeout?: number }): { address: string; stopped: Promise<void>; close():
Promise<void> }
```

Bind an SMTP listener: `server.smtp.listen({port, hostname?, handlers: {onMail, onRcpt, onData}, auth?, starttls?, allowInsecureAuth?, maxMessageBytes?, maxRecipients?, sessionTimeout?})` → handle with `.address`, `.close()`, `.stopped` Promise. Handlers receive (envelope, ...) per stage; return true/undefined to accept, false to reject, a string for a 550 reason, throw for 451 temp-fail. `onData` receives a parsed Message with text/html bodies, attachments, and raw bytes.

Parameters

- `opts` ({ *port*: number; *host*?: string; *hostname*?: string; *handlers*: { *onMail*: (env: Envelope) => boolean | string | void | Promise<boolean | string | void>; *onRcpt*: (env: Envelope, to: string) => boolean | string | void | Promise<boolean | string | void>; *onData*: (env: Envelope, msg: Message) => boolean | string | void | Promise<boolean | string | void> }; *auth*?: { *user*: string, *pass*: string, *env*: Envelope) => boolean | Promise<boolean>; *starttls*?: { *cert*: string; *key*: string }; *allowInsecureAuth*?: boolean; *maxMessageBytes*?: number; *maxRecipients*?: number; *sessionTimeout*?: number }) — port is required; host (bind interface) defaults to "0.0.0.0"; hostname (advertised EHLO domain) defaults to the OS hostname.

handlers.onMail/onRcpt/onData are all required and run per protocol stage — each returns true/undefined to accept, false for a 550 reject, a string for '550 <string>', or throws for a 451 temp-fail. auth (optional) enables PLAIN+LOGIN SASL; return truthy to accept. starttls {cert, key} (paths or inline PEM) enables STARTTLS. allowInsecureAuth permits AUTH without TLS. maxMessageBytes defaults to 10 MiB (non-positive values ignored), maxRecipients to 100, sessionTimeout to 30000 (milliseconds).

Returns: A server handle (returned synchronously): address is 'tcp/host:port'; stopped resolves when the server stops (rejects on a non-close Serve error); close() shuts the listener down and resolves with the stopped Promise. The Envelope passed to handlers is { from, recipients, remote, helo, authenticatedUser?, tls?: { version, cipher } }; the Message passed to onData is { from, to, cc, subject, headers, body: { text, html }, attachments: { filename, contentType, bytes }[], raw: Uint8Array }.

Throws: Throws synchronously if opts is missing, port is 0/absent, handlers is missing, any of onMail/onRcpt/onData is absent or not a function, auth is present but not a function, the starttls key pair fails to load, or the bind fails.

```
const srv = server.smtp.listen({
  port: 2525,
  handlers: {
    onMail: (env) => true,
    onRcpt: (env, to) => to.endsWith("@example.com"),
    onData: (env, msg) => { runtime.log(msg.subject); },
  },
});
```

17.12.5 server.tcp

17.12.5.1 server.tcp.listen

```
listen(opts: { port: number; host?: string; readBuffer?: number }, handler: (conn:
{ remote: string; local: string; write(data: string | Uint8Array): Promise<void>;
onData(cb: (msg: { bytes: Uint8Array; text: string }) => void): void; onClose(cb:
() => void): void; onError(cb: (err: unknown) => void): void; close(): void }) =>
void): { address: string; close(): Promise<void> }
```

Bind a raw TCP server: server.tcp.listen({port, host?, readBuffer?}, conn => {...}) → handle { address: 'tcp/host:port', close() }. The connection handler runs once per accepted socket; conn is the SAME handle shape as net.tcp.connect — onData(cb) (cb gets {bytes, text}), onClose(cb), onError(cb), write(data) (string or Uint8Array), close(), and remote/local addresses. Synchronous bind (throws on bind error); port:0 binds an OS-chosen ephemeral port. Emits a READY line under `sercon` `serve` and joins graceful shutdown.

Parameters

- `opts` ({ *port*: number; *host*?: string; *readBuffer*?: number }) — port is the listen port (0 binds an OS-chosen ephemeral port). host defaults to all interfaces. readBuffer is the per-connection inbound channel capacity (frames buffered before backpressure), default 64.
- `handler` ((conn: { *remote*: string; *local*: string; *write*(data: string | Uint8Array): Promise<void>; *onData*(cb: (msg: { bytes: Uint8Array; text: string }) => void): void; *onClose*(cb: () => void): void; *onError*(cb: (err: unknown) => void): void; *close*(): void }) =>

void) — Invoked once per accepted connection. `conn` matches `net.tcp.connect`'s `handle`: register `onData/onClose/onError` callbacks, `write()` to send (returns a `Promise`), `close()` to tear down. `remote/local` are 'host:port' strings.

Returns: A server handle (returned synchronously): address is 'tcp/host:port' (the resolved bind address, so port:0 reports its ephemeral port). `close()` closes the listener and all accepted connections, then resolves.

Throws: Throws synchronously if `opts` is missing, the handler is not a function, or the bind fails (e.g. address already in use).

```
const srv = server.tcp.listen({ port: 0 }, (conn) => {
  conn.onData((msg) => conn.write("echo: " + msg.text));
});
runtime.log(srv.address);
await srv.close();
```

17.12.6 server.udp

17.12.6.1 server.udp.listen

```
listen(opts: { port: number; host?: string }, handler: (msg: { bytes: Uint8Array;
text: string; address: string; port: number }, reply: (data: string | Uint8Array)
=> Promise<void>) => void): { address: string; close(): Promise<void> }
```

Bind a raw UDP server: `server.udp.listen({port, host?}, (msg, reply) => {...})` → handle { address: 'udp/host:port', `close()` }. The handler runs once per inbound datagram; `msg` is {bytes, text, address, port} (the sender) and `reply(data)` (string or `Uint8Array`) sends a datagram back to that sender, returning a `Promise`. Synchronous bind (throws on bind error); port:0 binds an OS-chosen ephemeral port. Emits a `READY` line under `sercon serve` and joins graceful shutdown.

Parameters

- `opts` ({ *port*: number; *host?*: string }) — `port` is the listen port (0 binds an OS-chosen ephemeral port). `host` defaults to all interfaces.
- `handler` ((*msg*: { bytes: `Uint8Array`; text: string; address: string; port: number }, *reply*: (*data*: string | `Uint8Array`) => `Promise<void>`) => void) — Invoked once per inbound datagram. `msg` carries the payload (bytes/text) and the sender's address/port. `reply(data)` sends a datagram back to that sender and returns a `Promise` that resolves once written (rejects on a write error).

Returns: A server handle (returned synchronously): address is 'udp/host:port' (resolved, so port:0 reports its ephemeral port). `close()` closes the socket and resolves. There is no per-connection handle — UDP is connectionless, so `reply` is bound to the originating datagram's sender.

Throws: Throws synchronously if `opts` is missing, the handler is not a function, the address fails to resolve, or the bind fails.

```
const srv = server.udp.listen({ port: 0 }, (msg, reply) => {
  reply("pong: " + msg.text);
});
```

```
runtime.log(srv.address);
await srv.close();
```

17.13 services

Subprocess and external-CLI / service wrappers: shell, git, gh, AI providers, agent-browser automation, W3C WebDriver browser control, Typst typesetting, poppler-backed PDF render/extract, external-requirement diagnostics (doctor).

17.13.1 services.agentBrowser

17.13.1.1 services.agentBrowser.auth

Namespace object for the auth vault (session-independent): `auth.save`, `auth.list`, `auth.show`, `auth.delete`. Passwords are never placed in `argv` — `auth.save` sends the password via `stdin` (–password-stdin).

Returns: object — the auth namespace (not callable itself; use sub-methods). `auth.list` returns an array of profile names; `auth.show/delete` return an envelope; `auth.save` returns an envelope. Handle-level `auth.login(name)` is a separate method on the session handle.

Throws: Never throws directly; sub-methods throw if agent-browser is not on `PATH`, or required fields (name, url, username, password) are missing.

```
// Save a login profile (password sent via stdin, never via argv).
await services.agentBrowser.auth.save("prod", {
  url: "https://app.example.com/login",
  username: "admin",
  password: "s3cret",
  usernameSelector: "#user",
  passwordSelector: "#pass",
});

// List all saved profiles.
const profiles = await services.agentBrowser.auth.list();
runtime.log(JSON.stringify(profiles.data));

// Log in using a saved profile (handle method — requires an open session).
const b = services.agentBrowser.launch();
await b.open("https://app.example.com/login");
await b.auth.login("prod");
await b.close();
```

17.13.1.1.1 services.agentBrowser.auth.delete

```
delete(...args: unknown[])
```

17.13.1.1.2 services.agentBrowser.auth.list

```
list(...args: unknown[])
```

17.13.1.1.3 services.agentBrowser.auth.save

```
save(...args: unknown[])
```

17.13.1.1.4 services.agentBrowser.auth.show

```
show(...args: unknown[])
```

17.13.1.2 services.agentBrowser.available

```
available: boolean
```

True when the agent-browser CLI is on PATH. Sync boolean, resolved once per Run. Gate calls on this; every binding throws a clean error when the CLI is absent.

Returns: boolean — true if `agent-browser` is on PATH.

Throws: Never throws.

```
if (!services.agentBrowser.available) runtime.log("install agent-browser first");
```

17.13.1.3 services.agentBrowser.batch

```
batch(cmds: string[], opts?: { bail?: boolean }): Promise<Array<{ success: boolean, data: object, error: string | null }>>
```

Run multiple agent-browser command strings in a single round-trip. Each element of `cmds` is a full command string (e.g. 'get title'). Returns a JSON array of per-command results, not the usual envelope.

Parameters

- `cmds` (*string[]*) — Array of full command strings to execute sequentially. Required.
- `opts` (*{ bail?: boolean }, optional*) — `bail`: stop on the first failed command and return results up to that point.

Returns: `Promise<Array<{ success: boolean, data: object, error: string|null }>>` — a JSON array of per-command result envelopes (not the usual single envelope).

Throws: Throws if `cmds` is missing/not an array or agent-browser is not on PATH.

```
const results = await b.batch(["get title", "get url"], { bail: false });
for (const r of results) runtime.log(r.success, r.data);
```

17.13.1.4 services.agentBrowser.chat

```
chat(message: string, opts?: { model?: string }): Promise<{ success: boolean, data: object, error: string | null }>
```

Send a single natural-language instruction to the browser session's AI gateway. The AI interprets the message and drives the page. Requires an AI gateway configured in agent-browser; errors cleanly if not configured.

Parameters

- `message` (*string*) — Natural-language instruction for the AI to execute. Required.
- `opts` (*{ model?: string }, optional*) — `model` selects the AI model (uses the agent-browser default when omitted).

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope.

Throws: Throws if message is missing, if agent-browser is not on PATH, or if no AI gateway is configured.

```
// Requires agent-browser to have an AI gateway configured.
const r = await b.chat("Click the Login button");
runtime.log("chat ok:", r.success);
```

17.13.1.5 services.agentBrowser.clearDefaultOptions

```
clearDefaultOptions(): void
```

Reset the namespace-level launch defaults to an empty object, removing any values set by setDefaultOptions.

Returns: void

Throws: Never throws.

```
services.agentBrowser.clearDefaultOptions();
runtime.log(JSON.stringify(services.agentBrowser.defaultOptions())); // {}
```

17.13.1.6 services.agentBrowser.click

```
click(selector: string): Promise<{ success: boolean, data: object, error: string | null }>
```

Click an element matching a CSS selector.

Parameters

- `selector` (*string*) — CSS selector for the element to click. Required.

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope.

Throws: Throws if selector is missing, element not found, or agent-browser is not on PATH.

```
await b.click("#submit-btn");
```

17.13.1.7 services.agentBrowser.clipboard

```
clipboard(op: "read" | "write" | "copy" | "paste", text?: string):
Promise<{ success: boolean, data: object, error: string | null }>
```

Read from or write to the system clipboard. op is one of 'read', 'write', 'copy', 'paste'.

Parameters

- `op` (*"read" | "write" | "copy" | "paste"*) — Clipboard operation. Required.
- `text` (*string, optional*) — Text to write (only used when op is 'write').

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope.

Throws: Throws if op is missing or agent-browser is not on PATH.

```
await b.clipboard("write", "hello");
const r = await b.clipboard("read");
runtime.log(r.data?.text);
```

17.13.1.8 services.agentBrowser.close

```
close(): Promise<{ closed: boolean }>
```

Close the browser session. Idempotent: a second close is a no-op.

Returns: Promise<{ closed: boolean }> — { closed: true } on the first call; an empty object on subsequent calls.

Throws: Throws if the close command fails (e.g. agent-browser error).

```
await b.close();
```

17.13.1.9 services.agentBrowser.cmd

```
cmd(command: string, args: string[]): Promise<{ success: boolean, data: object,
error: string | null }>
```

Generic escape hatch: run any agent-browser command with the current session context and return the parsed envelope. Use this for subcommands sercon doesn't model yet.

Parameters

- `command` (*string*) — The agent-browser subcommand to run (e.g. 'get', 'scroll'). Required.
- `args` (*string[]*) — Additional arguments to pass to the subcommand (spread after the command).

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope.

Throws: Throws if command is missing or agent-browser is not on PATH.

```
const r = await b.cmd("get", "title");
runtime.log("title via cmd:", r.data?.title);
```

17.13.1.10 services.agentBrowser.cookies

```
cookies(): object
```

Handle sub-object for cookie management: cookies.get(), cookies.set(name, value, opts?), cookies.clear().

Returns: object — the cookies namespace (not callable itself; use sub-methods). Each sub-method returns Promise<{ success: boolean, data: object, error: string|null }>.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>hi</h1>");
await b.cookies.set("sid", "abc123", { sameSite: "Lax", httpOnly: true });
const jar = await b.cookies.get();
runtime.log("cookies:", JSON.stringify(jar.data));
await b.cookies.clear();
await b.close();
```

17.13.1.11 services.agentBrowser.defaultOptions

defaultOptions(): object

Return a shallow copy of the current namespace-level launch defaults. These are merged (under per-call opts) into every subsequent launch().

Returns: object — a plain-object copy of the current defaults map. Empty object when no defaults have been set.

Throws: Never throws.

```
services.agentBrowser.setDefaultOptions({ headed: false });
runtime.log(JSON.stringify(services.agentBrowser.defaultOptions())); //
{"headed":false}
```

17.13.1.12 services.agentBrowser.diff

diff(): object

Handle sub-object for page diffing: diff.snapshot(opts?), diff.screenshot(opts), diff.url(url1, url2).

Returns: object — the diff namespace (not callable itself; use sub-methods). Each sub-method returns Promise<{ success: boolean, data: object, error: string|null }>.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH, the session is closed, or a required option (e.g. baseline) is missing.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>hello</h1>");
const snap = await b.diff.snapshot();
runtime.log("diff.snapshot ok:", snap.success);
// Compare two URLs side-by-side (no open() needed for diff.url):
const cmp = await b.diff.url("data:text/html,<h1>a</h1>", "data:text/html,<h1>b</h1>");
runtime.log("diff.url ok:", cmp.success);
await b.close();
```

17.13.1.13 services.agentBrowser.eval

```
eval(url: string, js: string): Promise<{ success: boolean, data: object, error: string | null }>
```

One-shot shortcut: launch an ephemeral session, open url, evaluate a JS expression in the page, and close.

Parameters

- `url` (*string*) — URL to open. Required.
- `js` (*string*) — JavaScript expression to evaluate in the page context. Required.

Returns: `Promise<{ success: boolean, data: object, error: string|null }>` — agent-browser envelope; `data.result` holds the serialised return value.

Throws: Throws if url or js is missing; if agent-browser is not on PATH; on navigation or evaluation failure.

```
const r = await services.agentBrowser.eval("data:text/html,<title>Hi</title>",  
"document.title");  
runtime.log(r.data?.result); // "Hi"
```

17.13.1.14 services.agentBrowser.fill

```
fill(selector: string, text: string): Promise<{ success: boolean, data: object,  
error: string | null }>
```

Fill an input element with text.

Parameters

- `selector` (*string*) — CSS selector for the input element. Required.
- `text` (*string*) — Text to fill into the element.

Returns: `Promise<{ success: boolean, data: object, error: string|null }>` — agent-browser envelope.

Throws: Throws if selector is missing, element not found, or agent-browser is not on PATH.

```
await b.fill("#search", "typescript");
```

17.13.1.15 services.agentBrowser.find

```
find(locator: string, value: string, opts: { action: string, text?: string }):  
Promise<{ success: boolean, data: object, error: string | null }>
```

Locate an element using a semantic locator (role, text, label, etc.) and perform an action in one shot.

Parameters

- `locator` (*string*) — Locator type: role, text, label, placeholder, alt, title, testid. Required.
- `value` (*string*) — Value to match for the given locator type. Required.
- `opts` (*{ action: string, text?: string }*) — action is required (e.g. click, fill, hover, check). text is passed as the fill text when action is 'fill' or 'type'.

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope.

Throws: Throws if locator, value, or opts.action is missing, or if agent-browser is not on PATH.

```
await b.find("role", "button", { action: "click" });
await b.find("text", "Search", { action: "fill", text: "query" });
```

17.13.1.16 services.agentBrowser.get

```
get(what: string, selector?: string): Promise<{ success: boolean, data: object,
error: string | null }>
```

Get a page property. what is one of: text, html, value, attr, title, url, count, box, styles, cdp-url. Pass selector as the second argument for element-scoped queries.

Parameters

- `what` (*string*) — Property name: text, html, value, attr, title, url, count, box, styles, cdp-url. Required.
- `selector` (*string, optional*) — CSS selector to scope the query to a specific element.

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser wraps results in this envelope; the requested value lives under data (e.g. get("title") → { success, data: { title }, error }, get("value", "#sel") → { success, data: { value }, error }).

Throws: Throws if what is missing, the selector finds no element, or agent-browser is not on PATH.

```
const r = await b.get("title");
runtime.log(r.data?.title);

const t = await b.get("text", "#main");
runtime.log(t.data?.text);
```

17.13.1.17 services.agentBrowser.handle

17.13.1.17.1 services.agentBrowser.handle.snapshot

```
snapshot(opts?: { interactive?: boolean, compact?: boolean, depth?: number,
selector?: string }): Promise<{ success: boolean, data: object, error: string |
null }>
```

Return an accessibility tree snapshot of the current page. Useful for reading page structure without CSS selectors.

Parameters

- `opts` (*{ interactive?: boolean, compact?: boolean, depth?: number, selector?: string }, optional*) — interactive: include interactive-only elements (-i). compact: compact output (-c). depth: max tree depth (-d). selector: scope to a subtree (-s selector).

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope; data contains the accessibility tree.

Throws: Throws if agent-browser is not on PATH or the snapshot fails.

```
const snap = await b.snapshot({ interactive: true });
runtime.log("snapshot keys:", Object.keys(snap).join(", "));
```

17.13.1.18 services.agentBrowser.inspect

```
inspect(): Promise<{ success: boolean, data: object, error: string | null }>
```

Open the Chrome DevTools inspector on the current session and return the DevTools URL.

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope; data typically contains { url } for the DevTools connection.

Throws: Throws if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("https://example.com");
const r = await b.inspect();
runtime.log("DevTools:", r.data?.url);
await b.close();
```

17.13.1.19 services.agentBrowser.launch

```
launch(opts?: { session?: string, headed?: boolean, profile?: string, proxy?:
string, userAgent?: string, device?: string, colorScheme?: string,
ignoreHttpsErrors?: boolean, engine?: string, executablePath?: string, enable?:
string, args?: string, timeout?: number }): { session: string; open(url: string,
opts?: object): Promise<any>; back(): Promise<any>; forward(): Promise<any>;
reload(): Promise<any>; wait(selOrMs: string | number): Promise<any>;
connect(target: string): Promise<any>; frame(target: string): Promise<any>;
click(sel: string): Promise<any>; dblclick(sel: string): Promise<any>; hover(sel:
string): Promise<any>; focus(sel: string): Promise<any>; check(sel: string):
Promise<any>; uncheck(sel: string): Promise<any>; scrollIntoView(sel: string):
Promise<any>; fill(sel: string, text: string): Promise<any>; type(sel: string,
text: string): Promise<any>; press(key: string): Promise<any>; select(sel:
string, ...values: string[]): Promise<any>; scroll(dir: string, px?: number):
Promise<any>; drag(src: string, dst: string): Promise<any>; upload(sel: string,
files: string | string[]): Promise<any>; download(sel: string, path: string):
Promise<any>; keyboard: { type(text: string): Promise<any>; insertText(text:
string): Promise<any> }; mouse: { move(x: number, y: number): Promise<any>;
down(button?: string): Promise<any>; up(button?: string): Promise<any>; wheel(dy:
number, dx?: number): Promise<any> }; get(what: string, sel?: string):
Promise<any>; isVisible(sel: string): Promise<any>; isEnabled(sel: string):
Promise<any>; isChecked(sel: string): Promise<any>; eval(code: string):
Promise<any>; snapshot(opts?: object): Promise<any>; console(opts?: object):
Promise<any>; errors(opts?: object): Promise<any>; highlight(sel: string):
Promise<any>; find(locator: string, value: string, opts: { action: string, text?:
string }): Promise<any>; locator(spec: object | string, value?: string): object;
set: { viewport(w: number, h: number, scale?: number): Promise<any>; device(name:
string): Promise<any>; geo(lat: number, lng: number): Promise<any>; offline(on?:
boolean): Promise<any>; headers(headers: Record<string, string>): Promise<any>;
credentials(user: string, pass: string): Promise<any>; media(scheme?: "dark" |
"light", reducedMotion?: boolean): Promise<any> }; record: { start(path: string,
url?: string): Promise<any>; stop(): Promise<any> }; screenshot(path?: string,
```

```

opts?: { selector?: string, full?: boolean, annotate?: boolean, format?: "png" |
"jpeg", quality?: number }): Promise<{ path?: string, size?: number, bytes?:
number[], format: string }>; pdf(path?: string): Promise<{ path?: string, size?:
number, bytes?: number[], format: string }>; network: { route(url: string, opts?:
{ abort?: boolean, body?: unknown, resourceType?: string }): Promise<any>;
unroute(url?: string): Promise<any>; requests(opts?: { clear?: boolean, filter?:
string, type?: string, method?: string, status?: string }): Promise<any>;
request(id: string): Promise<any>; har: { start(path?: string): Promise<any>;
stop(path?: string): Promise<any> } }; cookies: { get(): Promise<any>; set(name:
string, value: string, opts?: { url?: string, domain?: string, path?: string,
httpOnly?: boolean, secure?: boolean, sameSite?: "Strict" | "Lax" | "None",
expires?: number }): Promise<any>; clear(): Promise<any> }; storage: { local:
{ get(key?: string): Promise<any>; set(key: string, value: string): Promise<any>;
clear(): Promise<any> }; session: { get(key?: string): Promise<any>; set(key:
string, value: string): Promise<any>; clear(): Promise<any> } }; tabs: { list():
Promise<any>; new(url?: string, opts?: { label?: string }): Promise<any>;
close(ref?: string): Promise<any>; select(ref: string): Promise<any> }; diff:
{ snapshot(opts?: { baseline?: string, selector?: string, compact?: boolean,
depth?: number }): Promise<any>; screenshot(opts: { baseline: string, output?:
string, threshold?: number }): Promise<any>; url(url1: string, url2: string):
Promise<any> }; trace: { start(): Promise<any>; stop(path?: string):
Promise<any> }; profiler: { start(opts?: { categories?: string }): Promise<any>;
stop(path?: string): Promise<any> }; inspect(): Promise<any>; clipboard(op: "read"
| "write" | "copy" | "paste", text?: string): Promise<any>; vitals(url?: string):
Promise<any>; pushstate(url: string): Promise<any>; react: { tree(): Promise<any>;
inspect(id: string): Promise<any>; renders: { start(): Promise<any>; stop():
Promise<any> }; suspense(opts?: { onlyDynamic?: boolean }): Promise<any> };
stream: { enable(opts?: { port?: number }): Promise<any>; disable(): Promise<any>;
status(): Promise<any> }; chat(message: string, opts?: { model?: string }):
Promise<any>; cmd(command: string, ...args: string[]): Promise<any>; batch(cmds:
string[], opts?: { bail?: boolean }): Promise<any>; auth: { login(name: string):
Promise<any> }; close(): Promise<any> }

```

Allocate a browser session and return a handle. Synchronous (no browser starts until the first command). Pass `opts.session` to name the session; otherwise a unique id is generated. Launch flags (headed, profile, proxy, userAgent, device, colorScheme, ignoreHttpsErrors, engine, executablePath, enable, args) are threaded into every call the handle makes. Sessions the script does not close() are best-effort closed when the Run ends.

Parameters

- `opts` (*{ session?: string, headed?: boolean, profile?: string, proxy?: string, userAgent?: string, device?: string, colorScheme?: string, ignoreHttpsErrors?: boolean, engine?: string, executablePath?: string, enable?: string, args?: string, timeout?: number }, optional*) — Launch flags captured for the lifetime of the handle and threaded into every subprocess call. `session` names the agent-browser session (auto-generated when omitted). `timeout` is the per-call agent-browser subprocess timeout in milliseconds (default 30000, 0 disables); when a call exceeds the limit the Promise rejects with a clear 'timed out' error instead of hanging forever.

Returns: A handle object with a read-only session string and methods: open, back, forward, reload, wait, connect, click, dblclick, hover, focus, fill, type, press, check, uncheck, select, scroll, scrollIntoView, drag, upload, download, keyboard.{type,insertText}, mouse.{move,down,up,wheel}, get, isVisible, isEnabled, isChecked, eval, snapshot, console, errors,

highlight, find, locator, close. Every async method resolves to an agent-browser envelope { success: boolean, data: object, error: string|null }; drill into .data for the actual values.

Throws: launch() itself does not throw for a missing CLI (it allocates only); the first method call throws if agent-browser is not on PATH.

```
const b = services.agentBrowser.launch({ headed: false });
await b.open("https://example.com");
const r = await b.get("title");
runtime.log(r.data?.title);
await b.close();
```

17.13.1.20 services.agentBrowser.network

```
network(): object
```

Handle sub-object for network interception and monitoring: network.route, network.unroute, network.requests, network.request, network.har.start, network.har.stop.

Returns: object — the network namespace (not callable itself; use sub-methods). Each sub-method returns Promise<{ success: boolean, data: object, error: string|null }>.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>hi</h1>");
await b.network.route("**/api/*", { abort: true });
const reqs = await b.network.requests({ clear: true });
runtime.log("requests:", JSON.stringify(reqs.data));
await b.close();
```

17.13.1.21 services.agentBrowser.open

```
open(url: string): Promise<{ success: boolean; data: object; error: string | null }>
```

Navigate the browser session to a URL. The URL may be http/https, a data: URI, or any scheme the browser supports.

Parameters

- `url` (*string*) — The URL to navigate to. Required.

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope; data typically contains { url, title } after navigation.

Throws: Throws if url is missing, if agent-browser is not on PATH, or if navigation fails.

```
const b = services.agentBrowser.launch();
await b.open("https://example.com");
await b.close();
```

17.13.1.22 services.agentBrowser.pdf

```
pdf(url: string, path?: string): Promise<{ path: string, size: number, format: string } | { bytes: number[], format: string }>
```

One-shot shortcut: launch an ephemeral session, open url, capture a PDF, and close.

Parameters

- `url` (*string*) — URL to open. Required.
- `path` (*string, optional*) — Output file path. When supplied the PDF is written there and the result has { path, size, format }; when omitted the PDF bytes are returned as a number[] in { bytes, format }.

Returns: Promise — path given: { path: string, size: number, format: string }; no path: { bytes: number[], format: string }.

Throws: Throws if url is missing; if agent-browser is not on PATH; on navigation, capture, or I/O failure.

```
const pdf = await services.agentBrowser.pdf("data:text/html,<h1>Hi</h1>");
runtime.log(new Uint8Array(pdf.bytes).length, pdf.format); // e.g. 5678 pdf
```

17.13.1.23 services.agentBrowser.profiler

```
profiler(opts?: { categories?: string }): object
```

Handle sub-object for V8 CPU profiling: profiler.start(opts?) begins profiling (opts.categories narrows the V8/Blink categories), profiler.stop(path?) stops it.

Parameters

- `opts` (*{ categories?: string }, optional*) — categories is a comma-separated list of V8/Blink profiling categories (e.g. 'v8,blink').

Returns: object — the profiler namespace (not callable itself; use sub-methods). Each sub-method returns Promise<{ success: boolean, data: object, error: string|null }>.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>hi</h1>");
await b.profiler.start({ categories: "v8,blink" });
// ... interact ...
await b.profiler.stop("/tmp/profile.json");
await b.close();
```

17.13.1.24 services.agentBrowser.pushstate

```
pushstate(url: string): Promise<{ success: boolean, data: object, error: string | null }>
```

Perform a client-side SPA navigation using history.pushState without a full page reload.

Parameters

- `url` (*string*) — The URL to push into the browser history. Required.

Returns: `Promise<{ success: boolean, data: object, error: string|null }>` — agent-browser envelope.

Throws: Throws if url is missing or agent-browser is not on PATH.

```
await b.pushstate("/app/dashboard");
```

17.13.1.25 services.agentBrowser.react

```
react(): object
```

Handle sub-object for React DevTools integration: `react.tree()`, `react.inspect(id)`, `react.render.start/stop()`, `react.suspense(opts?)`. Requires the session launched with `launch({ enable: 'react-devtools' })`.

Returns: object — the react namespace (not callable itself; use sub-methods). Each sub-method returns `Promise<{ success: boolean, data: object, error: string|null }>`. agent-browser returns a clear error when react-devtools was not enabled at launch time.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH, the session is closed, or react-devtools was not enabled at launch.

```
const b = services.agentBrowser.launch({ enable: "react-devtools" });
await b.open("https://react-app.example.com");
const tree = await b.react.tree();
runtime.log(JSON.stringify(tree.data).slice(0, 200));
const suspense = await b.react.suspense({ onlyDynamic: true });
runtime.log("suspense ok:", suspense.success);
await b.close();
```

17.13.1.26 services.agentBrowser.record

```
record(): object
```

Namespace object for video recording on an open handle: `record.start(path, url?)` and `record.stop()`.

Returns: object — the record namespace (not callable itself; use sub-methods).

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("https://example.com");
await b.record.start("/tmp/clip.webm");
// ... interact ...
await b.record.stop();
await b.close();
```

17.13.1.27 services.agentBrowser.screenshot

```
screenshot(url: string, path?: string, opts?: { selector?: string, full?: boolean,
annotate?: boolean, format?: "png" | "jpeg", quality?: number }): Promise<{ path:
string, size: number, format: string } | { bytes: number[], format: string }>
```

One-shot shortcut: launch an ephemeral session, open url, capture a screenshot, and close. Equivalent to launch()+open(url)+screenshot(path?,opts?)+close() but in a single call.

Parameters

- `url` (*string*) — URL to open (http/https or data: URI). Required.
- `path` (*string, optional*) — Output file path. When supplied the screenshot is written there and the result has { path, size, format }; when omitted the image bytes are returned as a number[] (byte-value array) in { bytes, format }.
- `opts` (*{ selector?: string, full?: boolean, annotate?: boolean, format?: "png" | "jpeg", quality?: number }, optional*) — Capture options. selector scopes the capture to an element. full captures the full page. format defaults to png. quality (0–100) applies to jpeg.

Returns: Promise — path given: { path: string, size: number, format: string }; no path: { bytes: number[], format: string } where bytes is a plain JS number[] (byte-value array); wrap with new Uint8Array(bytes) to get a typed array.

Throws: Throws if url is missing; if agent-browser is not on PATH; on navigation, capture, or I/O failure.

```
const shot = await services.agentBrowser.screenshot("data:text/html,<h1>Hi</h1>");
runtime.log(new Uint8Array(shot.bytes).length, shot.format); // e.g. 12345 png
```

17.13.1.28 services.agentBrowser.set

```
set(): object
```

Namespace object with browser-settings sub-methods on an open handle: set.viewport, set.device, set.geo, set.offline, set.headers, set.credentials, set.media.

Returns: object — the set namespace (not callable itself; use sub-methods).

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>Test</h1>");
await b.set.viewport(1920, 1080);
await b.set.device("iPhone 12");
await b.close();
```

17.13.1.29 services.agentBrowser.setDefaultOptions

```
setDefaultOptions(opts: object): void
```

Replace the namespace-level launch defaults with the supplied object. Merged (under per-call opts) into every subsequent launch(). Affects only the current Run.

Parameters

- `opts` (*object*) — A plain object of launch option key/value pairs. The entire defaults map is replaced (not merged) with this object.

Returns: void

Throws: Never throws.

```
services.agentBrowser.setDefaultOptions({ headed: false, proxy: "http://proxy:3128" });
const b = services.agentBrowser.launch(); // headed:false + proxy inherited
```

17.13.1.30 services.agentBrowser.snapshot

```
snapshot(url: string, opts?: { interactive?: boolean, compact?: boolean, depth?: number, selector?: string }): Promise<{ success: boolean, data: object, error: string | null }>
```

One-shot shortcut: launch an ephemeral session, open url, take an accessibility-tree snapshot, and close.

Parameters

- `url` (*string*) — URL to open. Required.
- `opts` (*{ interactive?: boolean, compact?: boolean, depth?: number, selector?: string }*, *optional*) — Snapshot options forwarded to the handle's snapshot() method.

Returns: Promise<{ success: boolean, data: object, error: string|null }> — agent-browser envelope; data contains the accessibility tree.

Throws: Throws if url is missing; if agent-browser is not on PATH; on navigation or snapshot failure.

```
const snap = await services.agentBrowser.snapshot("data:text/html,<h1>Hi</h1>", { compact: true });
runtime.log(JSON.stringify(snap.data).slice(0, 100));
```

17.13.1.31 services.agentBrowser.storage

```
storage(): object
```

Handle sub-object for web storage: storage.local.{get,set,clear} and storage.session.{get,set,clear}.

Returns: object — the storage namespace with local and session sub-objects. Each sub-method returns Promise<{ success: boolean, data: object, error: string|null }>.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.


```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>hi</h1>");
await b.storage.local.set("theme", "dark");
const theme = await b.storage.local.get("theme");
runtime.log("theme:", JSON.stringify(theme.data));
await b.storage.session.set("token", "xyz");
await b.close();
```

17.13.1.32 services.agentBrowser.stream

```
stream(opts?: { port?: number }): object
```

Handle sub-object for live streaming of browser events: `stream.enable(opts?)`, `stream.disable()`, `stream.status()`. Streaming makes page events available over a local WebSocket.

Parameters

- `opts` (*{ port?: number }, optional*) — port selects the streaming port (default chosen by agent-browser).

Returns: object — the stream namespace (not callable itself; use sub-methods). Each sub-method returns `Promise<{ success: boolean, data: object, error: string|null }>`.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
await b.stream.enable({ port: 9229 });
const status = await b.stream.status();
runtime.log("streaming:", status.data?.enabled);
await b.stream.disable();
```

17.13.1.33 services.agentBrowser.tabs

```
tabs(): object
```

Handle sub-object for tab management: `tabs.list()`, `tabs.new(url?, opts?)`, `tabs.close(ref?)`, `tabs.select(ref)`. Tab refs are `t1/t2/...` or user labels.

Returns: object — the tabs namespace (not callable itself; use sub-methods). Each sub-method returns `Promise<{ success: boolean, data: object, error: string|null }>`.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH, the session is closed, or a tab ref is required but missing.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>tab1</h1>");
await b.tabs.new("data:text/html,<h1>tab2</h1>", { label: "second" });
const list = await b.tabs.list();
runtime.log("tabs:", JSON.stringify(list.data));
await b.tabs.select("t1");
await b.tabs.close("second");
await b.close();
```

17.13.1.34 services.agentBrowser.trace

```
trace(): object
```

Handle sub-object for Chrome DevTools tracing: `trace.start()` begins a trace, `trace.stop(path?)` stops it and optionally saves to a file.

Returns: object — the trace namespace (not callable itself; use sub-methods). Each sub-method returns `Promise<{ success: boolean, data: object, error: string|null }>`.

Throws: Never throws directly; sub-methods throw if agent-browser is not on PATH or the session is closed.

```
const b = services.agentBrowser.launch();
await b.open("data:text/html,<h1>hi</h1>");
await b.trace.start();
// ... interact ...
await b.trace.stop("/tmp/trace.json");
await b.close();
```

17.13.1.35 services.agentBrowser.version

```
version(...args: unknown[]): Promise<string>
```

The agent-browser CLI version string.

Returns: `Promise<string>` — the version reported by `agent-browser --version`.

Throws: Throws if the agent-browser CLI is not on PATH.

```
runtime.log(await services.agentBrowser.version());
```

17.13.1.36 services.agentBrowser.vitals

```
vitals(url?: string): Promise<{ success: boolean, data: object, error: string | null }>
```

Collect Core Web Vitals (LCP, FID, CLS, TTFB, etc.) for the current page or a given URL.

Parameters

- `url` (*string, optional*) — URL to navigate to before measuring. When omitted, vitals are measured on the currently loaded page.

Returns: `Promise<{ success: boolean, data: object, error: string|null }>` — agent-browser envelope; data contains the Core Web Vitals metrics.

Throws: Throws if agent-browser is not on PATH or the session is closed.

```
const v = await b.vitals();
runtime.log("LCP:", v.data?.lcp);
```

17.13.2 services.ai

17.13.2.1 services.ai.providers

```
providers(): string[]
```

Which of `claude` / `codex` / `copilot` / `gemini` are on `PATH`, in preference order.

Returns: `string[]` — the subset of supported AI CLIs found on `PATH`, in preference order (`claude`, `codex`, `copilot`, `gemini`); an empty array when none are installed. Synchronous (not a `Promise`).

Throws: Never throws.

```
const ps = services.ai.providers(); // e.g. ["claude", "gemini"]
```

17.13.2.2 services.ai.send

```
send(opts: { prompt: string, provider?: "claude" | "codex" | "copilot" | "gemini",  
system?: string, context?: string, timeout?: number }): Promise<{ provider:  
string; output: string; exitCode: number }>
```

Run a one-shot prompt through a provider. `opts` { `prompt` (required), `provider?`, `system?`, `context?`, `timeout?` }. Returns { `provider`, `output`, `exitCode` }. Non-zero exit is data; no provider throws.

Parameters

- `opts` (*{ prompt: string, provider?: "claude" | "codex" | "copilot" | "gemini", system?: string, context?: string, timeout?: number }*) — `prompt` is required. `provider` names the CLI to use; when omitted, the first provider on `PATH` (in preference order) is chosen. `system` and `context` are prepended to the prompt as "System: ..." / "Context: ..." blocks (a portable substitute for each CLI's own flags). `timeout` in ms (default 120000).

Returns: `Promise<{ provider: string, output: string, exitCode: number }>` — `provider` is the CLI that ran; `output` is its trimmed `stdout` (or `stderr` when `stdout` is empty on a non-zero exit); `exitCode` is 0 on success.

Throws: Throws if `opts.prompt` is missing/empty, no provider is on `PATH` (when `provider` is unset), the named provider is unknown, the CLI fails to spawn, or on timeout / context cancellation. A non-zero exit code does NOT throw — it resolves with that `exitCode`.

```
const r = await services.ai.send({ prompt: "Say hi", provider: "claude" });  
runtime.log(r.output);
```

17.13.3 services.doctor

```
doctor(requires?: string[]): Promise<{ ok: boolean; satisfied: boolean; unmet:  
string[]; tools: { name: string; category: string; purpose: string; installed:  
boolean; version: string | null; ok: boolean; detail?: string }[] }>
```

Report on every external tool sercon can use (`git`, `gh`, AI providers, `agent-browser`, `chromedriver/geckodriver`, `typst`, `recon/curl`, `clipboard/image` tools): `installed?`, `version`,

purpose — and validate the chromedriver↔Chrome major-version match. Optionally assert a script's prerequisites via `requires`.

Parameters

- `requires` (*string[], optional*) — Feature/category names (e.g. “webdriver”, “typst”, “ai”, “git”, “gh”, “agentBrowser”, “clipboard”, “image”, “http”) OR specific binaries (e.g. “chromedriver”, “pngpaste”, “claude”) to assert. Each unmet requirement (absent or in a compatibility conflict) lands in the returned `unmet` array — a missing optional tool is reported, not thrown. An unrecognized name throws (catches typos).

Returns: `Promise<{ ok, satisfied, unmet, tools }>` — `ok` is false only when a compatibility conflict was detected (e.g. chromedriver↔Chrome major mismatch); `satisfied` is true when every entry in `requires` is met (`unmet` is empty); `unmet` lists the requested requirements that are absent or conflicted; `tools` is the full report, one entry per probed tool (`{ name, category, purpose, installed, version (string|null), ok, optional detail }`). A missing tool is `installed:false, ok:true` (optional, not a failure).

Throws: Throws if a `requires` entry matches neither a known category/feature nor a known binary name (an unknown requirement). Missing tools do NOT throw — they are reported via `installed:false` and, when requested, listed in `unmet`.

```
const r = await services.doctor(["git"]);
runtime.log("git satisfied:", r.satisfied, "of", r.tools.length, "tools");
const opt = await services.doctor(["typst", "webdriver"]);
runtime.log("unmet optional features:", JSON.stringify(opt.unmet));
```

17.13.4 services.exec

17.13.4.1 services.exec.http

```
http(method: string, url: string, opts?: { headers?: Record<string, string>,
body?: string, timeout?: number, follow?: boolean, insecure?: boolean, backend?:
"auto" | "recon" | "curl" }): Promise<{ status: number; headers: Record<string,
string>; body: string; durationMs: number; backend: "recon" | "curl" }>
```

Make an HTTP request by shelling out to `recon` (preferred) or `curl` (fallback). `4xx/5xx` resolve as status; transport errors and timeouts throw. `opts.backend = 'auto' | 'recon' | 'curl'`.

Parameters

- `method` (*string*) — HTTP verb (GET, POST, PUT, DELETE, PATCH, HEAD); lower-case input is uppercased before forwarding.
- `url` (*string*) — Target URL; must be fully qualified (the backend requires a scheme + host).
- `opts` (*{ headers?: Record<string, string>, body?: string, timeout?: number, follow?: boolean, insecure?: boolean, backend?: "auto" | "recon" | "curl" }, optional*) — `headers` emits one `-H "Name: Value"` per entry. `body` is written to a temp file and sent via `-data-binary` so CR/LF stay intact. `timeout` in ms (default 30000). `follow` toggles `-L` to follow 3xx redirects. `insecure` toggles `-k` to skip TLS verification. `backend` picks the tool: `'auto'` (default) prefers `recon` then `curl`; `'recon'` or `'curl'` require that specific binary on `PATH`.

Returns: Promise<{ status: number, headers: Record<string, string>, body: string, durationMs: number, backend: “recon” | “curl” }> — status is the final HTTP status code; headers have lower-cased names (last response block on a redirect chain); body is the UTF-8 decoded response body; backend is whichever tool ran.

Throws: Throws if method or url is missing/empty; if the requested backend (or, for ‘auto’, neither recon nor curl) is on PATH; on transport errors (DNS failure, connection refused, TLS handshake); on timeout or context cancellation; or if the response headers can’t be parsed. HTTP 4xx/5xx do NOT throw — they resolve with that status.

```
const r = await services.exec.http("GET", "https://example.com");
runtime.log(r.status, r.backend);
```

17.13.4.2 services.exec.interactive

```
interactive(cmd: string | string[], opts?: { cwd?: string, env?: Record<string,
string>, timeout?: number }): Promise<{ exitCode: number; success: boolean;
durationMs: number }>
```

Run a subprocess wired to sercon’s own terminal — the interactive counterpart to exec.shell. Inherits stdin/stdout/stderr so genuinely interactive children work (docker exec - it, ssh, interactive mysql/redis-cli REPLs, pagers, full-screen TUIs). On Unix with a real TTY it allocates a pty and switches to raw mode (restored on exit); on a non-TTY stdin or on Windows it inherits the raw handles. Nothing is captured — the result is just { exitCode, success, durationMs }.

Parameters

- `cmd` (*string | string[]*) — A string is passed to the host shell (/bin/sh -c on Unix, cmd /C on Windows) so quoting, pipes, and redirects work. A string[] is treated as argv: argv[0] is run directly with no shell.
- `opts` (*{ cwd?: string, env?: Record<string, string>, timeout?: number }, optional*) — `cwd` sets the working directory. `env` entries are merged on top of the inherited environment (they do not replace it). `timeout` is in ms with NO default (0 / absent = run until the child exits, since an interactive shell or REPL is expected to stay open); when set, the process tree is killed on expiry and the call throws.

Returns: Promise<{ exitCode: number, success: boolean, durationMs: number }> — the child’s exit code (0 on success), success (exitCode === 0), and wall-clock spawn-to-exit time. No stdout/stderr is captured; the child wrote directly to the terminal.

Throws: Throws if cmd is missing, an empty string, an empty array, or a non-string array element; if the host binary is not on PATH or fails to start; or if the timeout (or context cancellation) fires before exit — terminal state is restored first. A non-zero exit code does NOT throw — it resolves with success:false.

```
const r = await services.exec.interactive("ssh user@host");
runtime.log("session ended, exit", r.exitCode);
```

17.13.4.3 services.exec.shell

```
shell(cmd: string | string[], opts?: { timeout?: number, cwd?: string, stdin?:
string, env?: Record<string, string>, pane?: string | Pane, pty?: boolean }):
Promise<{ stdout: string; stderr: string; exitCode: number; success: boolean;
durationMs: number }>
```

Run a subprocess and wait for it to exit. String `cmd` → `/bin/sh -c` (or `cmd /C` on Windows); array `cmd` → `argv` (no shell). Non-zero exits resolve normally; spawn failures and timeouts throw.

Parameters

- `cmd` (*string* / *string[]*) — A string is passed verbatim to the host shell (`/bin/sh -c` on Unix, `cmd /C` on Windows) so quoting, pipes, and redirects work. A `string[]` is treated as `argv`: `argv[0]` is run directly with no shell, so use this form when arguments contain whitespace or shell metacharacters you don't want re-interpreted.
- `opts` (*{ timeout?: number, cwd?: string, stdin?: string, env?: Record<string, string>, pane?: string | Pane, pty?: boolean }*, optional) — `timeout` in ms (default 30000); on expiry the process tree is killed and the call throws. `cwd` sets the working directory. `stdin` is fed to the process's standard input. `env` entries are merged on top of the inherited environment (they do not replace it). `pane` (a `tui.pane` name or `Pane` handle) streams `stdout`+`stderr` live into a TUI pane — in that mode the result's `stdout`/`stderr` strings stay empty. `pty` (default `false`) runs the command under a pseudo-terminal so it believes it is a terminal and emits color/progress; with a pane the output is rendered there, without a pane it is captured into `stdout` (`stderr` stays empty since a `pty` merges both streams). Unix only — on Windows `pty` is ignored and the normal pipe path is used.

Returns: `Promise<{ stdout: string, stderr: string, exitCode: number, success: boolean, durationMs: number }>` — `stdout`/`stderr` are captured (empty when streamed to a pane); `exitCode` is 0 on success; `success` is `exitCode === 0`; `durationMs` is wall-clock spawn-to-exit time.

Throws: Throws if `cmd` is missing, an empty string, an empty array, or a non-string array element; if the host binary is not on `PATH` or fails to start; if the timeout (or context cancellation) fires before exit; or if a named pane is referenced without a prior `tui.layout`. A non-zero exit code does NOT throw — it resolves with `success:false`.

```
const r = await services.exec.shell("echo hi");
if (r.success) runtime.log(r.stdout.trim());
```

17.13.4.4 services.exec.stream

```
stream(cmd: string | string[], onLine: (line: string, stream: "stdout" | "stderr")
=> void, opts?: { cwd?: string, env?: Record<string, string>, stdin?: string,
timeout?: number }): Promise<{ exitCode: number; success: boolean; durationMs:
number }>
```

Run a subprocess and stream its `stdout`/`stderr` to a callback line by line as output arrives (unlike `exec.shell`, which buffers). String `cmd` → `/bin/sh -c` (or `cmd /C` on Windows); array `cmd` → `argv` (no shell). Resolves `{ exitCode, success, durationMs }` on exit; non-zero exits resolve normally; spawn failures and timeouts reject.

Parameters

- `cmd` (*string* / *string[]*) — A string is passed to the host shell (`/bin/sh -c` on Unix, `cmd /C` on Windows) so pipes, redirects, and globs work. A `string[]` is treated as `argv`: `argv[0]` is run directly with no shell.
- `onLine` ((*line*: *string*, *stream*: “*stdout*” / “*stderr*”) => *void*) — Called once per output line as it arrives. `line` has its trailing newline stripped; `stream` is ‘`stdout`’ or ‘`stderr`’. A final line without a trailing newline is still delivered. Required — a non-function throws synchronously.
- `opts` ({ *cwd*?: *string*, *env*?: *Record*<*string*, *string*>, *stdin*?: *string*, *timeout*?: *number* }, *optional*) — `cwd` sets the working directory; `env` entries merge on top of the inherited environment; `stdin` is fed to the process. `timeout` is in ms with NO default (0 / absent = run until exit, unlike `exec.shell`’s 30000); when set, the process tree is killed on expiry and the call rejects.

Returns: `Promise`<{ `exitCode`: `number`, `success`: `boolean`, `durationMs`: `number` }> — resolves on process exit. `exitCode` is 0 on success; `success` is `exitCode === 0`; `durationMs` is wall-clock spawn-to-exit time. Output is delivered via `onLine`, not captured into the result.

Throws: Throws synchronously if `cmd` is missing or `onLine` is not a function. The `Promise` rejects if the host binary is not on `PATH` or fails to start, or if the timeout (or context cancellation) fires before exit. A non-zero exit code does NOT reject — it resolves with `success:false`.

```
const r = await services.exec.stream("echo one; echo two", (line, stream) => {
  runtime.log(stream, line);
});
runtime.log("exit", r.exitCode);
```

17.13.5 services.gh

17.13.5.1 services.gh.authStatus

```
authStatus(...args: unknown[]): Promise<{ authenticated: boolean; user: string;
raw: string }>
```

Probe `gh`’s auth state. Missing `gh` / unauthenticated resolve with { `authenticated`: `false`, ... } — only context cancellation throws.

Returns: `Promise`<{ `authenticated`: `boolean`, `user`: `string`, `raw`: `string` }> — `authenticated` is true only when `gh api user` succeeds; `user` is the resolved login (“” when not authenticated); `raw` is the login on success or the underlying `gh` error / “`gh not on PATH`” otherwise.

Throws: Throws only on context cancellation. A missing `gh` binary or an unauthenticated session resolve with `authenticated:false` rather than throwing.

```
const a = await services.gh.authStatus();
if (a.authenticated) runtime.log("logged in as", a.user);
```

17.13.5.2 services.gh.prList

```
prList(opts?: { cwd?: string, state?: string, limit?: number, author?: string }):  
Promise<Array<{ number: number; title: string; state: string; author: string;  
headRefName: string; baseRefName: string; url: string; createdAt: string;  
updatedAt: string }>>
```

List pull requests on the cwd's repo (or opts.cwd). Defaults: open state, limit 30. Filters: state / limit / author.

Parameters

- `opts` (*{ cwd?: string, state?: string, limit?: number, author?: string }, optional*) — `cwd` selects the repo (defaults to the engine's working directory, which `gh` uses to detect the repo). `state` filters by PR state ("open" default, "closed", "merged", "all"). `limit` caps results (default 30; must be positive). `author` filters to PRs opened by that login.

Returns: `Promise<Array<{ number: number, title: string, state: string, author: string, headRefName: string, baseRefName: string, url: string, createdAt: string, updatedAt: string }>>` — one object per PR; `author` is flattened from `gh`'s `{ login }` wrapper to the bare login string; `createdAt/updatedAt` are ISO 8601 timestamps.

Throws: Throws if `limit` is ≤ 0 , `gh` is not on `PATH`, `gh` exits non-zero (not authenticated, not a GitHub repo, etc.), the JSON can't be parsed, or on context cancellation.

```
const prs = await services.gh.prList({ state: "open", limit: 5 });  
for (const pr of prs) runtime.log(pr.number, pr.title);
```

17.13.5.3 services.gh.repoView

```
repoView(repo?: string, opts?: { cwd?: string }): Promise<{ name: string; owner:  
string; description: string; url: string; defaultBranch: string; visibility:  
string }>
```

Repo metadata. With no arg uses `cwd`'s repo; pass 'owner/name' for any repo `gh` can see. `owner` + `defaultBranch` are pre-flattened.

Parameters

- `repo` (*string, optional*) — "owner/name" of any repo `gh` can access. Omit (or pass `opts` as the first arg) to view the repo detected from `cwd`.
- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout `gh` uses to detect the current repo when `repo` is omitted.

Returns: `Promise<{ name: string, owner: string, description: string, url: string, defaultBranch: string, visibility: string }>` — `owner` is flattened from `gh`'s `{ login }` wrapper to the bare login; `defaultBranch` is flattened from `defaultBranchRef.name` (" " if absent); key order matches `gh`'s output.

Throws: Throws if `gh` is not on `PATH`, `gh` exits non-zero (repo not found, not authenticated, etc.), the JSON can't be parsed, or on context cancellation.

```
const r = await services.gh.repoView("cli/cli");  
runtime.log(r.owner, r.defaultBranch);
```


17.13.6 services.git

17.13.6.1 services.git.add

```
add(paths: string | string[], opts?: { cwd?: string }): Promise<{ paths: string[] }>
```

Stage one path (string) or several (string[]).

Parameters

- `paths` (*string* | *string[]*) — Path or paths to stage. Passed after a `--` separator so paths that look like flags (`-foo`) are handled literally.
- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout; defaults to the engine's working directory.

Returns: `Promise<{ paths: string[] }>` — the list of paths that were staged.

Throws: Throws if `paths` is missing, an empty string, or contains a non-string array element; if `git` is not on `PATH`; or if `git add` exits non-zero (e.g. a `paths` spec matching nothing).

```
await services.git.add(["src/a.ts", "src/b.ts"]);
```

17.13.6.2 services.git.branch

```
branch(opts?: { cwd?: string }): Promise<{ current: string; detached: boolean; all: string[] }>
```

Current branch (empty when HEAD is detached) plus the list of local branches.

Parameters

- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout to inspect; defaults to the engine's working directory.

Returns: `Promise<{ current: string; detached: boolean; all: string[] }>` — `current` is the checked-out branch name ("" when detached); `detached` is true on a detached HEAD; `all` lists every local branch (`refs/heads`) by short name.

Throws: Throws if `git` is not on `PATH`, the directory is not a `git` repository, or the underlying `git` command fails. A detached HEAD is reported via `detached:true`, not a throw.

```
const b = await services.git.branch();
runtime.log(b.detached ? "(detached)" : b.current);
```

17.13.6.3 services.git.commit

```
commit(message: string, opts?: { cwd?: string, allowEmpty?: boolean }): Promise<{ sha: string }>
```

Create a commit; returns the post-commit HEAD SHA. `opts.allowEmpty` toggles `--allow-empty`.

Parameters

- `message` (*string*) — Commit message (passed as a single `-m` argument).

- `opts` (*{ cwd?: string, allowEmpty?: boolean }, optional*) — `cwd` selects the checkout. `allowEmpty` adds `--allow-empty` so a commit succeeds with no staged changes (release markers, etc.); defaults to `false`.

Returns: `Promise<{ sha: string }>` — `sha` is the full SHA of the newly created HEAD commit.

Throws: Throws if message is missing or blank, git is not on PATH, or git commit exits non-zero (e.g. nothing staged and `allowEmpty` is `false`).

```
const c = await services.git.commit("chore: bump", { allowEmpty: true });
```

17.13.6.4 services.git.diffStat

```
diffStat(opts?: { cwd?: string, revRange?: string }): Promise<{ files: number;
insertions: number; deletions: number }>
```

Aggregate `{ files, insertions, deletions }` from `git diff --shortstat`. Default `revRange` `HEAD 1..HEAD`.

Parameters

- `opts` (*{ cwd?: string, revRange?: string }, optional*) — `cwd` selects the checkout. `revRange` is the diff range (default “HEAD 1..HEAD”, the last commit).

Returns: `Promise<{ files: number, insertions: number, deletions: number }>` — counters parsed from `git diff --shortstat`. An empty diff returns all zeros.

Throws: Throws if git is not on PATH, the directory is not a git repository, or git diff exits non-zero (e.g. a bad `revRange`).

```
const d = await services.git.diffStat();
runtime.log(d.files, d.insertions, d.deletions);
```

17.13.6.5 services.git.isClean

```
isClean(opts?: { cwd?: string }): Promise<boolean>
```

True iff `git status --porcelain` is empty.

Parameters

- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout; defaults to the engine’s working directory.

Returns: `Promise<boolean>` — `true` when the working tree has no staged, unstaged, or untracked changes.

Throws: Throws if git is not on PATH, the directory is not a git repository, or git status exits non-zero.

```
if (await services.git.isClean()) runtime.log("clean");
```

17.13.6.6 services.git.log

```
log(opts?: { cwd?: string, limit?: number, revRange?: string }):
Promise<Array<{ sha: string; shortSha: string; author: string; email: string;
timestamp: number; subject: string }>>
```

Recent commits as { sha, shortSha, author, email, timestamp, subject }. opts.limit / opts.revRange.

Parameters

- `opts` (*{ cwd?: string, limit?: number, revRange?: string }, optional*) — `cwd` selects the checkout. `limit` caps the number of commits (default 50; must be positive). `revRange` selects the range/ref to walk (default “HEAD”).

Returns: `Promise<Array<{ sha: string, shortSha: string, author: string, email: string, timestamp: number, subject: string }>>` — newest first; timestamp is the author Unix epoch seconds; subject is the commit’s first line.

Throws: Throws if `limit` is ≤ 0 , `git` is not on `PATH`, the directory is not a git repository, or `git log` exits non-zero (e.g. an unknown `revRange`).

```
const log = await services.git.log({ limit: 5 });
runtime.log(log[0].subject);
```

17.13.6.7 services.git.revParse

```
revParse(rev: string, opts?: { cwd?: string }): Promise<string>
```

Full 40-char SHA for the given rev. Invalid refs throw.

Parameters

- `rev` (*string*) — Any revision git understands (branch, tag, HEAD, short SHA, HEAD 2, etc.).
- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout; defaults to the engine’s working directory.

Returns: `Promise<string>` — the full 40-character commit SHA the rev resolves to.

Throws: Throws if `rev` is missing or empty, `git` is not on `PATH`, the directory is not a git repository, or the rev cannot be resolved (git’s own error message is included).

```
const sha = await services.git.revParse("HEAD");
```

17.13.6.8 services.git.runText

```
runText(args: string | string[], opts?: { cwd?: string }): Promise<{ stdout:
string; stderr: string; exitCode: number }>
```

Escape hatch: run any `git <args>`, get { stdout, stderr, exitCode } — `exitCode` is data, not a throw.

Parameters

- `args` (*string | string[]*) — git arguments (without the leading “git”), e.g. [“tag”, “-list”]. A bare string is treated as a single argument.

- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout; defaults to the engine's working directory.

Returns: `Promise<{ stdout: string, stderr: string, exitCode: number }>` — captured streams plus git's exit code, so callers can react to any exit status without `try/catch`.

Throws: Throws if `args` is missing, an empty string, or an empty array; if git is not on `PATH`; or on a spawn failure / context cancellation. A non-zero exit code does NOT throw — it is returned in `exitCode`.

```
const r = await services.git.runText(["tag", "--list"]);
if (r.exitCode === 0) runtime.log(r.stdout);
```

17.13.6.9 services.git.status

```
status(opts?: { cwd?: string }): Promise<Array<{ path: string; indexStatus: string; workingStatus: string }>>
```

Parsed `git status --porcelain` entries: `{ path, indexStatus, workingStatus }`.

Parameters

- `opts` (*{ cwd?: string }, optional*) — `cwd` selects the checkout; defaults to the engine's working directory.

Returns: `Promise<Array<{ path: string, indexStatus: string, workingStatus: string }>>` — one entry per changed path; `indexStatus` / `workingStatus` are the porcelain v1 X / Y status characters (e.g. "M", "A", "?"). An empty array means a clean tree.

Throws: Throws if git is not on `PATH`, the directory is not a git repository, or git status exits non-zero.

```
for (const e of await services.git.status())
  runtime.log(e.indexStatus + e.workingStatus, e.path);
```

17.13.7 services.pdf

17.13.7.1 services.pdf.available

```
available: boolean
```

True when the poppler `pdftoppm` binary is on `PATH` (the core PDF capability). Sync boolean, resolved once per Run. Gate calls on this — every pdf operation throws a clean error when its binary is absent.

Returns: boolean — true if `pdftoppm` is found on `PATH`.

Throws: Never throws.

```
if (!services.pdf.available) {
  runtime.log("install poppler-utils first (brew install poppler / apt install poppler-utils)");
}
```

17.13.7.2 services.pdf.backend

```
backend: string | null
```

The active PDF backend name, or null when no backend is available. Currently “poppler” when pdftoppm is on PATH; future-proofs against a backend swap.

Returns: string|null — “poppler” when available, otherwise null.

Throws: Never throws.

```
runtime.log("pdf backend:", services.pdf.backend ?? "none");
```

17.13.7.3 services.pdf.info

```
info(src: string): Promise<{ pages?: number; title?: string; author?: string;
creator?: string; producer?: string; creationDate?: string; modDate?: string;
encrypted?: boolean; tagged?: boolean; pageSize?: string; fileSize?: string;
pdfVersion?: string }>
```

Read a PDF’s metadata via `pdffinfo` : page count, title/author, encryption, page size, PDF version, and tagging.

Parameters

- `src` (*string*) — Path to the source PDF.

Returns: Promise<object> — best-effort metadata; `pages` is the page count and drives multi-page loops. `creationDate/modDate` are the raw `pdffinfo` date strings when present. Fields `pdffinfo` does not report are omitted.

Throws: Throws if `src` is missing/empty; if `pdffinfo` is not on PATH; if `pdffinfo` exits non-zero (trimmed `stderr` included); or on timeout.

```
const meta = await services.pdf.info("report.pdf");
runtime.log("pages:", meta.pages, "encrypted:", meta.encrypted);
```

17.13.7.4 services.pdf.toHtml

```
toHtml(src: string, opts?: { pages?: string, dest?: string }): Promise<string |
{ path: string }>
```

Convert a PDF to a single self-contained HTML document via `pdftohtml` (`-i -noframes`). Without `dest`, returns the HTML string; with `dest`, writes it there.

Parameters

- `src` (*string*) — Path to the source PDF.
- `opts` (*{ pages?: string, dest?: string }, optional*) — `pages` limits conversion to “N” or “F-L” (1-based). `dest` writes the HTML to that path instead of returning a string.

Returns: Promise<string|{path}> — the HTML when no `dest`; { `path` } when `dest` is set.

Throws: Throws if `src` is missing; if `pages` is malformed; if `pdftohtml` is not on PATH; if `pdftohtml` exits non-zero; or on timeout.

```
const html = await services.pdf.toHtml("report.pdf", { pages: "1" });
runtime.log(html.length, "bytes of HTML");
```

17.13.7.5 services.pdf.toImage

```
toImage(src: string, opts?: { page?: number, firstPage?: number, lastPage?:
number, format?: "png" | "jpeg" | "tiff", dpi?: number, dest?: string }):
Promise<{ format: string; page?: number; bytes?: Uint8Array; paths?: string[] }>
```

Render PDF page(s) to PNG/JPEG/TIFF via `pdftoppm`. With a single `page` and no `dest`, returns the rendered image as bytes; with a `dest` prefix or a page range, writes files and returns their paths.

Parameters

- `src` (*string*) — Path to the source PDF.
- `opts` (*{ page?: number, firstPage?: number, lastPage?: number, format?: "png" | "jpeg" | "tiff", dpi?: number, dest?: string }, optional*) — `page` selects a single 1-based page; `firstPage`/`lastPage` select a range (page wins when set). `format` defaults to `png`. `dpi` sets resolution (default poppler's 150). `dest` is an output path/prefix; when omitted, a single `page` is returned as bytes; a multi-page/whole-document render REQUIRES `dest` (omitting it throws).

Returns: `Promise<object>` — single page + no `dest`: `{ format, page, bytes }`. With `dest`: `{ format, paths }` listing the written files (sorted).

Throws: Throws if `src` is missing; if `format` is not `png/jpeg/tiff`; if the page range is invalid; if `pdftoppm` is not on `PATH`; if `pdftoppm` exits non-zero; if a multi-page/whole-document render is requested without a `dest` or on timeout.

```
// Single page → PNG bytes.
const img = await services.pdf.toImage("report.pdf", { page: 1, format: "png" });
runtime.log(img.format, img.bytes.length);
// All pages → files on disk.
const all = await services.pdf.toImage("report.pdf", { dest: "/tmp/page" });
runtime.log("wrote", all.paths.length, "images");
```

17.13.7.6 services.pdf.toText

```
toText(src: string, opts?: { pages?: string, layout?: boolean, dest?: string }):
Promise<string | { path: string }>
```

Extract text from a PDF via `pdftotext`. Without `dest`, returns the extracted text string; with `dest`, writes it to that file.

Parameters

- `src` (*string*) — Path to the source PDF.
- `opts` (*{ pages?: string, layout?: boolean, dest?: string }, optional*) — `pages` limits extraction to "N" or "F-L" (1-based). `layout` preserves the original physical layout (-layout). `dest` writes to a file instead of returning a string (use it for very large documents).

Returns: Promise<string|{path}> — the extracted text when no dest; { path } when dest is set.

Throws: Throws if src is missing; if pages is malformed; if pdftotext is not on PATH; if pdftotext exits non-zero; if the buffered output exceeds the size cap (use dest); or on timeout.

```
const txt = await services.pdf.toText("report.pdf", { pages: "1-2" });
runtime.log(txt.slice(0, 80));
```

17.13.7.7 services.pdf.tools

Per-binary availability for the poppler tools sercon uses, so a partial install can still serve the operations it covers.

Returns: object — each poppler binary mapped to whether it is on PATH.

Throws: Never throws.

```
if (!services.pdf.tools.pdftotext) runtime.log("text extraction unavailable");
```

17.13.7.7.1 services.pdf.tools.pdfinfo

```
pdfinfo
```

17.13.7.7.2 services.pdf.tools.pdfhtml

```
pdfhtml
```

17.13.7.7.3 services.pdf.tools.pdfppm

```
pdfppm
```

17.13.7.7.4 services.pdf.tools.pdftotext

```
pdftotext
```

17.13.7.8 services.pdf.version

```
version(...args: unknown[]): Promise<string>
```

The poppler version string (from `pdfppm -v`, falling back to `pdfinfo -v`).

Returns: Promise<string> — the trimmed poppler version line.

Throws: Throws if no poppler binary is on PATH, or on timeout / context cancellation.

```
runtime.log(await services.pdf.version());
```

17.13.8 services.typst

17.13.8.1 services.typst.available

```
available: boolean
```

True when the typst CLI is on PATH. Sync boolean, resolved once per Run. Gate calls on this — every other typst binding throws a clean error when the CLI is absent.

Returns: boolean — true if `typst` is found on PATH.

Throws: Never throws.

```
if (!services.typst.available) {  
  runtime.log("install typst (brew install typst) first");  
}
```

17.13.8.2 services.typst.compile

```
compile(opts: { input?: string, source?: string, output?: string, format?: "pdf" |  
  "png" | "svg", root?: string, inputs?: Record<string, string>, ppi?: number,  
  fontPaths?: string[], timeout?: number }): Promise<{ format: string; bytes?:  
  Uint8Array; path?: string }>
```

Compile a Typst document to PDF/PNG/SVG. Provide exactly one of `input` (a .typ path) or `source` (inline Typst). With no `output`, a PDF is compiled to a temp file and returned as bytes (PDF only); with an `output` path the result is written there and `format` is inferred from the extension (png/svg require an output path).

Parameters

- `opts` (*{ input?: string, source?: string, output?: string, format?: "pdf" | "png" | "svg", root?: string, inputs?: Record<string, string>, ppi?: number, fontPaths?: string[], timeout?: number }*) — Provide exactly one of `input` (a path to a .typ file) or `source` (inline Typst markup) — passing both, or neither, throws. `output` is the destination path; when omitted a PDF is produced in a temp dir and returned as bytes (PDF only — png/svg require an output path). `format` (pdf/png/svg) is inferred from output's extension when omitted, defaulting to pdf when there is no output. `root` sets the project root for absolute imports. `inputs` are sys.inputs key/value pairs passed as `-input k=v` (deterministic order). `ppi` sets PNG resolution (png only). `fontPaths` adds `-font-path` search dirs. `timeout` in ms (default 60000). Inline source caveat: `source` is written to a temp main.typ, so relative imports/reads resolve against that temp dir, not your cwd — use `input` (or `root`) when the document imports or reads sibling files.

Returns: `Promise<{ format, bytes?, path? }>` — `format` echoes the chosen format. With no `output`: `{ format, bytes }` where `bytes` is the compiled PDF as a `Uint8Array`. With an `output` path: `{ format, path }` where `path` is the written file.

Throws: Throws if neither or both of `input`/`source` are given; if `format` is not pdf/png/svg; if png/svg is requested without an output path; if the output extension can't be mapped to a format; if typst is not on PATH; if typst exits non-zero (the trimmed stderr is included); or on timeout / context cancellation.

```
// Inline source → PDF bytes.  
const pdf = await services.typst.compile({ source: "= Hi\nBody." });  
runtime.log(pdf.format, pdf.bytes.length);
```



```
// .typ file → PNG on disk.
await services.typst.compile({ input: "report.typ", output: "/tmp/report.png",
  ppi: 144 });
```

17.13.8.3 services.typst.fonts

```
fonts(...args: unknown[]): Promise<string[]>
```

List the font families typst can see (from `typst fonts`), de-duplicated and sorted.

Returns: `Promise<string[]>` — sorted, unique font family names available to typst.

Throws: Throws if typst is not on PATH or on timeout / context cancellation.

```
const families = await services.typst.fonts();
runtime.log("fonts:", families.length);
```

17.13.8.4 services.typst.query

```
query(opts: { selector: string, input?: string, source?: string, field?: string,
  one?: boolean, root?: string, inputs?: Record<string, string>, timeout?:
  number }): Promise<unknown>
```

Query a compiled Typst document for elements matching a selector and return the result as parsed JSON. Provide exactly one of `input` or `source` selector is required.

Parameters

- `opts` (*{ selector: string, input?: string, source?: string, field?: string, one?: boolean, root?: string, inputs?: Record<string, string>, timeout?: number }*) — selector is required — a Typst query selector (e.g. “<label>”, “heading”, “figure”). Provide exactly one of `input` (a .typ path) or `source` (inline Typst). `field` extracts a single field from each match (–`field`). `one` returns just the first match instead of an array (–`one`). `root` sets the project root; `inputs` are –`input` `k=v` sys.inputs pairs; `timeout` in ms (default 60000). Same inline-source caveat as `compile`: `source` is written to a temp file, so relative imports/reads resolve there.

Returns: `Promise<unknown>` — the parsed JSON typst emits: an array of matched elements (or matched field values when `field` is set), or a single value when `one` is set.

Throws: Throws if selector is missing; if neither or both of `input`/`source` are given; if typst is not on PATH; if typst exits non-zero (the trimmed `stderr` is included); if the output isn’t valid JSON; or on timeout / context cancellation.

```
const v = await services.typst.query({
  source: "#metadata(42) <answer>",
  selector: "<answer>", field: "value", one: true,
});
runtime.log(v); // 42
```

17.13.8.5 services.typst.version

```
version(...args: unknown[]): Promise<string>
```

The typst CLI version string (from `typst --version`).

Returns: `Promise<string>` — the trimmed version line reported by `typst --version` (e.g. “typst 0.12.0 (...)”).

Throws: Throws if typst is not on PATH or on timeout / context cancellation.

```
runtime.log(await services.typst.version());
```

17.13.9 services.webdriver

17.13.9.1 services.webdriver.available

```
available: boolean
```

True when a W3C WebDriver binary (chromedriver or geckodriver) is on PATH. Sync boolean, resolved once per Run. Gate calls on this before using probe or connect.

Returns: `boolean` — true if chromedriver or geckodriver is found on PATH.

Throws: Never throws.

```
if (!services.webdriver.available) {  
  runtime.log("install chromedriver or geckodriver first");  
}
```

17.13.9.2 services.webdriver.connect

```
connect(opts?: { browser?: "chrome" | "firefox", headless?: boolean, url?: string,  
args?: string[], capabilities?: object, commandTimeout?: number }):  
Promise<{ get(url: string): Promise<{ ok: true }>; url(): Promise<string>;  
title(): Promise<string>; back(): Promise<{ ok: true }>; forward(): Promise<{ ok:  
true }>; refresh(): Promise<{ ok: true }>; find(by: string, value: string):  
Promise<{ click(): Promise<{ ok: true }>; sendKeys(text: string): Promise<{ ok:  
true }>; clear(): Promise<{ ok: true }>; submit(): Promise<{ ok: true }>; text():  
Promise<string>; getAttribute(name: string): Promise<string>; cssValue(name:  
string): Promise<string>; tagName(): Promise<string>; isDisplayed():  
Promise<boolean>; isEnabled(): Promise<boolean>; isSelected(): Promise<boolean>;  
find(by: string, value: string): Promise<any>; findAll(by: string, value: string):  
Promise<any[]>; screenshot(path?: string): Promise<{ path?: string; size?: number;  
bytes?: number[]; format: "png" }>>; findAll(by: string, value: string):  
Promise<Array<{ click(): Promise<{ ok: true }>; sendKeys(text: string):  
Promise<{ ok: true }>; clear(): Promise<{ ok: true }>; submit(): Promise<{ ok:  
true }>; text(): Promise<string>; getAttribute(name: string): Promise<string>;  
cssValue(name: string): Promise<string>; tagName(): Promise<string>;  
isDisplayed(): Promise<boolean>; isEnabled(): Promise<boolean>; isSelected():  
Promise<boolean>; find(by: string, value: string): Promise<any>; findAll(by:  
string, value: string): Promise<any[]>; screenshot(path?: string):  
Promise<{ path?: string; size?: number; bytes?: number[]; format: "png" }>>;  
source(): Promise<string>; screenshot(path?: string): Promise<{ path?: string;  
size?: number; bytes?: number[]; format: "png" }>; executeScript(js: string,  
args?: unknown[]): Promise<unknown>; executeScriptAsync(js: string, args?:  
unknown[]): Promise<unknown>; cookies(): Promise<object[]>; setCookie(c: { name:  
string; value: string; path?: string; domain?: string; secure?: boolean;  
httpOnly?: boolean; expiry?: number }): Promise<{ ok: true }>; deleteCookie(name:
```

```

string): Promise<{ ok: true }>; deleteAllCookies(): Promise<{ ok: true }>;
setImplicitWait(ms: number): Promise<{ ok: true }>; waitFor(by: string, value:
string, opts?: { timeout?: number; visible?: boolean; enabled?: boolean }):
Promise<any>; clickWhenReady(by: string, value: string, opts?: { timeout?: number;
visible?: boolean; enabled?: boolean; poll?: number }): Promise<{ ok: true }>;
windowHandles(): Promise<string[]>; currentWindow(): Promise<string>;
switchToWindow(handle: string): Promise<{ ok: true }>; newWindow(type?: "tab" |
>window"): Promise<{ handle: string; type: string }>; closeWindow():
Promise<string[]>; switchToFrame(target: number | string | object): Promise<{ ok:
true }>; frameChain(targets: (number | string | object)[]): Promise<{ ok: true }>;
switchToParentFrame(): Promise<{ ok: true }>; switchToDefaultContent():
Promise<{ ok: true }>; acceptAlert(): Promise<{ ok: true }>; dismissAlert():
Promise<{ ok: true }>; alertText(): Promise<string>; sendAlertText(text: string):
Promise<{ ok: true }>; maximize(): Promise<{ x: number; y: number; width: number;
height: number }>; minimize(): Promise<{ x: number; y: number; width: number;
height: number }>; fullscreen(): Promise<{ x: number; y: number; width: number;
height: number }>; setWindowRect(rect: { width?: number; height?: number; x?:
number; y?: number }): Promise<{ x: number; y: number; width: number; height:
number }>; getWindowRect(): Promise<{ x: number; y: number; width: number; height:
number }>; hover(el: object): Promise<{ ok: true }>; dragAndDrop(src: object, dst:
object): Promise<{ ok: true }>; keyChord(...keys: string[]): Promise<{ ok: true }
>; performActions(sequence: unknown[]): Promise<{ ok: true }>; releaseActions():
Promise<{ ok: true }>; cdp(command: string, params?: object): Promise<unknown>;
cdpClick(by: string, value: string, opts?: { timeout?: number; poll?: number;
button?: "left" | "right" | "middle"; scrollIntoView?: boolean; offsetX?: number;
offsetY?: number }): Promise<{ clicked: true; x: number; y: number; targetId?:
string }>; targets(): Promise<Array<{ targetId: string; type: string; url: string;
title: string }>>; attach(target: string | { targetId: string }):
Promise<{ targetId: string; sessionId: string; cdp(method: string, params?:
object): Promise<unknown>; detach(): Promise<{ detached: true }> }>; quit():
Promise<{ closed: true }> }>

```

Connect to a running WebDriver server (opts.url) or start an installed local chromedriver/geckodriver and dial it. Returns a session handle whose methods drive the browser. Sessions are quit on Run end if the script does not call quit() explicitly.

Parameters

- **opts** (*{ browser?: "chrome" | "firefox", headless?: boolean, url?: string, args?: string[], capabilities?: object, commandTimeout?: number }, optional*) — browser selects the driver binary (default 'chrome'). headless defaults to true. url, if given, dials an already-running driver at that base URL instead of starting one. args appends extra browser flags. capabilities is an escape hatch for raw W3C capability overrides merged last. commandTimeout (ms, default 30000) bounds each low-level WebDriver request so a driver blocked behind an open alert or an unreachable endpoint can't hang the call; 0 or negative disables the per-command deadline. quit()/Run-end also cancel any in-flight command.

Returns: Promise resolving to a session handle with methods: get(url), url(), title(), back(), forward(), refresh(), find(by, value) → element handle, findAll(by, value) → element handle[], source(), screenshot(path?), executeScript(js, args?), executeScriptAsync(js, args?), cookies(), setCookie(c), deleteCookie(name), deleteAllCookies(), setImplicitWait(ms), waitFor(by, value, opts?) (opts.enabled waits past a disabled→enabled flip; opts.visible

waits for visibility), `clickWhenReady`(by, value, opts?) (waits — by default visible+enabled — in the active frame, then issues a native trusted click that fires React handlers; `opts.poll` sets the inter-attempt interval), `quit()`. `find/clickWhenReady` operate in the active frame set via `switchToFrame/frameChain`. Phase 2 session methods: `windowHandles()`, `currentWindow()`, `switchToWindow(handle)`, `newWindow(type?)`, `closeWindow()`, `switchToFrame(selectorOrIndexOrElement)`, `frameChain([...targets])` (nested: switch from the top document through each level), `switchToParentFrame()`, `switchToDefaultContent()`, `acceptAlert()`, `dismissAlert()`, `alertText()`, `sendAlertText(text)`, `maximize()`, `minimize()`, `fullscreen()`, `setWindowRect({width?,height?,x?,y?})`, `getWindowRect()`, `hover(el)`, `dragAndDrop(src,dst)`, `keyChord(...keys)`, `performActions(sequence)`, `releaseActions()`. `cdp(command, params?)` sends one raw Chrome DevTools Protocol command (Chrome-only) and returns its result; `cdpClick`(by, value, opts?) locates the element across the whole frame tree (including nested cross-origin iframes), scrolls it into view, and dispatches a trusted CDP mouse click at its true viewport coordinates — the way to activate a button the W3C Element Click hit-tests to a parent iframe and intercepts (e.g. a Klarna Checkout “Pay order”). `cdp/cdpClick` throw on firefox. `cdpClick` now crosses out-of-process iframe (OOPIF) boundaries by dispatching input over a browser-level CDP connection; `targets()` lists the browser’s CDP targets (incl. cross-site OOPIF iframe: `targets`) and `attach(target)` returns a session handle whose `cdp(method, params?)` runs a CDP command against that target (e.g. an OOPIF) and `detach()` releases it. `targets()/attach()` require a CDP-capable session (local chromedriver, or a Grid advertising `se:cdp`). Element handles also expose `hover()` and `dragTo(target)` and carry an `elementId` string. `executeScript/executeScriptAsync` return element handles when the script returns an element (or a top-level array of elements). Locator strategies: `css`, `xpath`, `id`, `name`, `tag`, `className`, `linkText`, `partialLinkText`.

Throws: Throws if no url is given and the driver binary for the selected browser is not on PATH; if dialing the driver fails (driver not running, wrong port); or if the browser launch fails. Subsequent session method calls throw if the session is already closed.

```
if (!services.webdriver.available) {
  runtime.log("no driver on PATH – skip");
} else {
  const d = await services.webdriver.connect({ browser: "chrome", headless:
true });
  try {
    await d.get("data:text/html,<title>hi</title><h1 id=h>Hello</h1>");
    runtime.log("title:", await d.title());
    const el = await d.find("id", "h");
    runtime.log("text:", await el.text());
    await d.executeScript("return 1+2", []);
  } finally {
    await d.quit();
  }
}
```

17.13.9.3 services.webdriver.probe

```
probe(opts: { url: string }): Promise<{ ready: boolean; status?: number; error?:
string }>
```

Check whether a WebDriver endpoint responds at `opts.url/status`. Returns `{ ready, status }` on HTTP success or `{ ready: false, error }` on transport failure. Does not throw on network errors.

Parameters

- `opts` (*{ url: string }*) — url is required — the base URL of a running WebDriver server (e.g. `'http://127.0.0.1:9515'`).

Returns: `Promise<{ ready, status? } | { ready: false, error }>` — `ready` is true when the endpoint returns HTTP 200; `status` is the HTTP status code when a response was received; `error` is the transport error message when the request failed entirely.

Throws: Throws if `opts.url` is missing or empty. Transport failures resolve with `{ ready: false, error }` rather than throwing.

```
const r = await services.webdriver.probe({ url: "http://127.0.0.1:9515" });
runtime.log("ready:", r.ready);
```

17.14 text

String / regex / charset / data manipulation — all text-shaped transforms.

17.14.1 text.case

17.14.1.1 text.case.ada

```
ada(input: string): string
```

Convert to `Ada_Case` (every word capitalized, underscore-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (`camelCase`, `snake_case`, `kebab-case`, `spaces`, `dot/path`, and acronym runs like `HTTPServer`).

Returns: string — e.g. `"My_Var_Name"`.

Throws: Throws a `TypeError` if `input` is missing, null, or undefined.

```
text.case.ada("my-var"); // "My_Var"
```

17.14.1.2 text.case.camel

```
camel(input: string): string
```

Convert to `camelCase` (first word lower, rest capitalized, no separator).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (`camelCase`, `snake_case`, `kebab-case`, `spaces`, `dot/path`, and acronym runs like `HTTPServer`).

Returns: string — e.g. `"myVarName"`.

Throws: Throws a `TypeError` if `input` is missing, null, or undefined.

```
text.case.camel("my-var-name"); // "myVarName"
```

17.14.1.3 text.case.camelSnake

```
camelSnake(input: string): string
```

Convert to camel_Snake_Case (first word lower, rest capitalized, underscore-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. "my_Var_Name".

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.camelSnake("my-var"); // "my_Var"
```

17.14.1.4 text.case.capital

```
capital(input: string): string
```

Convert to Capital Case (every word capitalized, space-separated). Synonym of title.

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. "My Var Name".

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.capital("my_var_name"); // "My Var Name"
```

17.14.1.5 text.case.cobol

```
cobol(input: string): string
```

Alias of screamingKebab (COBOL-CASE).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. "MY-VAR-NAME".

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.cobol("myVar"); // "MY-VAR"
```

17.14.1.6 text.case.convert

```
convert(input: string, name: string): string
```

Convert input to the named case (dynamic dispatch; accepts canonical names and aliases).

Parameters

- `input` (*string*) — The string to convert.
- `name` (*string*) — A case name from `names()` (e.g. “snake”, “kebab”) or an alias (header/cobol/slug).

Returns: string — input rendered in the requested case.

Throws: Throws a `TypeError` if input/name is missing; throws if name is not a known case (the message lists valid names).

```
text.case.convert("userID", "kebab"); // "user-id"
```

17.14.1.7 text.case.detect

```
detect(input: string): string
```

Best-effort guess of the input’s case name (heuristic).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like `HTTPServer`).

Returns: string — the first case name whose converter reproduces the input exactly, or “unknown”. Multiword inputs detect cleanly; a single lowercase word resolves to “camel”; empty input is “unknown”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.detect("my_var"); // "snake"
```

17.14.1.8 text.case.dot

```
dot(input: string): string
```

Convert to dot.case (all lower, dot-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like `HTTPServer`).

Returns: string — e.g. “my.var.name”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.dot("myVarName"); // "my.var.name"
```

17.14.1.9 text.case.flat

```
flat(input: string): string
```

Convert to flatcase (all lower, no separator). Lossy: boundaries are gone and cannot be recovered.

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “myvarname”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.flat("myVarName"); // "myvarname"
```

17.14.1.10 text.case.header

```
header(input: string): string
```

Alias of `train` (Header-Case, e.g. Content-Type).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “My-Var-Name”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.header("content-type"); // "Content-Type"
```

17.14.1.11 text.case.kebab

```
kebab(input: string): string
```

Convert to kebab-case (all lower, hyphen-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “my-var-name”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.kebab("myVarName"); // "my-var-name"
```

17.14.1.12 text.case.names

```
names(): string[]
```


List the canonical case names (drives `convert()` and `detect()`; excludes aliases).

Returns: `string[]` — the 16 canonical case names in priority order.

Throws: None.

```
text.case.names(); // ["camel","pascal","snake",...]
```

17.14.1.13 text.case.pascal

```
pascal(input: string): string
```

Convert to PascalCase (every word capitalized, no separator).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: `string` — e.g. “MyVarName”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.pascal("my_var"); // "MyVar"
```

17.14.1.14 text.case.path

```
path(input: string): string
```

Convert to path/case (all lower, slash-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: `string` — e.g. “my/var/name”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.path("myVarName"); // "my/var/name"
```

17.14.1.15 text.case.screamingKebab

```
screamingKebab(input: string): string
```

Convert to SCREAMING-KEBAB-CASE (all upper, hyphen-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: `string` — e.g. “MY-VAR-NAME”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.screamingKebab("myVar"); // "MY-VAR"
```

17.14.1.16 text.case.screamingSnake

```
screamingSnake(input: string): string
```

Convert to SCREAMING_SNAKE_CASE (all upper, underscore-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. "MY_VAR_NAME".

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.screamingSnake("myVar"); // "MY_VAR"
```

17.14.1.17 text.case.sentence

```
sentence(input: string): string
```

Convert to Sentence case (first word capitalized, rest lower, space-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. "My var name".

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.sentence("my_var_name"); // "My var name"
```

17.14.1.18 text.case.slug

```
slug(input: string): string
```

Alias of kebab. NOTE: kebab only — does NOT transliterate or strip diacritics/punctuation.

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. "my-var-name".

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.slug("My Var"); // "my-var"
```

17.14.1.19 text.case.snake

```
snake(input: string): string
```

Convert to snake_case (all lower, underscore-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “my_var_name”.

Throws: Throws a TypeError if input is missing, null, or undefined.

```
text.case.snake("myVarName"); // "my_var_name"
```

17.14.1.20 text.case.split

```
split(input: string): string[]
```

Tokenize any input into an array of lowercased words (the primitive every converter builds on).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string[] — lowercased words; boundaries detected at lower→upper transitions, acronym→word (HTTPServer→[http,server]), and separators (_ - . / whitespace). Empty/separator-only input → [].

Throws: Throws a TypeError if input is missing, null, or undefined.

```
text.case.split("getHTTPCode"); // ["get", "http", "code"]
```

17.14.1.21 text.case.title

```
title(input: string): string
```

Convert to Title Case (every word capitalized, space-separated). Synonym of capital.

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “My Var Name”.

Throws: Throws a TypeError if input is missing, null, or undefined.

```
text.case.title("my_var_name"); // "My Var Name"
```

17.14.1.22 text.case.train

```
train(input: string): string
```

Convert to Train-Case (every word capitalized, hyphen-separated).

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “My-Var-Name”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.train("my_var"); // "My-Var"
```

17.14.1.23 text.case.upperFlat

```
upperFlat(input: string): string
```

Convert to UPPERFLATCASE (all upper, no separator). Lossy: boundaries are gone.

Parameters

- `input` (*string*) — The string to convert. Word boundaries are auto-detected (camelCase, snake_case, kebab-case, spaces, dot/path, and acronym runs like HTTPServer).

Returns: string — e.g. “MYVARNAME”.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.case.upperFlat("myVar"); // "MYVAR"
```

17.14.2 text.charset

17.14.2.1 text.charset.decode

```
decode(input: string | Uint8Array | ArrayBuffer, charset: string): Promise<string>
```

Decode bytes in a named charset to a UTF-8 string.

Parameters

- `input` (*string | Uint8Array | ArrayBuffer*) — Bytes encoded in `charset`.
- `charset` (*string*) — WHATWG/HTML5 encoding name or alias (UTF-8, ISO-8859-1, Windows-1252, Shift_JIS, GBK, ...).

Returns: `Promise<string>` — the bytes decoded to a UTF-8 string.

Throws: Rejects if input is an unsupported type, the charset name is unknown, or the bytes are invalid for that encoding.

```
const s = await text.charset.decode(bytes, "Windows-1252");
```

17.14.2.2 text.charset.detect

```
detect(input: string | Uint8Array | ArrayBuffer): Promise<{ charset: string;  
confidence: number; language?: string; candidates: { charset: string; confidence:  
number; language?: string }[] }>
```

Detect the most-likely charset of a byte sequence (saintfish/chardet). Resolves to the top guess plus the full candidate list, each scored 0–100.

Parameters

- `input` (*string* | *Uint8Array* | *ArrayBuffer*) — Bytes to sniff. A string is taken as its raw UTF-8 bytes.

Returns: `Promise<{ charset, confidence, language?, candidates }>` — the top match plus all candidates; confidence is chardet's 0–100 score. language is present only when chardet reports one.

Throws: Rejects if input is empty, an unsupported type, or chardet finds no candidates.

```
const r = await text.charset.detect(bytes);
runtime.log(r.charset, r.confidence);
```

17.14.2.3 text.charset.encode

```
encode(input: string, charset: string): Promise<Uint8Array>
```

Encode a UTF-8 string to bytes in the named charset.

Parameters

- `input` (*string*) — UTF-8 string to encode.
- `charset` (*string*) — Target WHATWG/HTML5 encoding name or alias.

Returns: `Promise<Uint8Array>` — the string encoded as bytes in the target charset.

Throws: Rejects if the charset name is unknown, or a character has no representation in the target encoding (no lossy fallback — characters are not silently dropped).

```
const bytes = await text.charset.encode("café", "ISO-8859-1");
```

17.14.3 text.diff

17.14.3.1 text.diff.compare

```
compare(a: string | Uint8Array | ArrayBuffer, b: string | Uint8Array |
ArrayBuffer, opts?: { context?: number; fromFile?: string; toFile?: string }):
Promise<{ identical: boolean; binary: boolean; added: number; removed: number;
diff: string; format: "unified" }>
```

Unified-diff two text inputs. `opts`: `context` (default 3), `fromFile` / `toFile` (default 'a' / 'b'). Binary inputs return `{ binary: true }` with an empty diff.

Parameters

- `a` (*string* | *Uint8Array* | *ArrayBuffer*) — The 'from' / left side. A string is taken as its UTF-8 bytes.
- `b` (*string* | *Uint8Array* | *ArrayBuffer*) — The 'to' / right side.
- `opts` (*{ context?: number; fromFile?: string; toFile?: string }, optional*) — `context` is the number of unchanged lines around each hunk (default 3); `fromFile` / `toFile` are the header labels (default 'a' / 'b').

Returns: Promise<{ identical, binary, added, removed, diff, format }> — diff holds the unified-diff text (empty when identical or binary); added/removed count body +/- lines excluding file headers; binary is true when either input has a NUL byte in its first 8 KB.

Throws: Rejects if either input is an unsupported type, or the unified-diff generation fails.

```
const d = await text.diff.compare("a\n", "b\n");
runtime.log(d.diff, d.added, d.removed);
```

17.14.4 text.jq

17.14.4.1 text.jq.query

```
query(data: unknown, filter: string): Promise<unknown>
```

Run a jq filter over data and return the first emitted value (or null).

Parameters

- `data` (*unknown*) — Input value — any JSON-like JS value (object/array/scalar). Passed to gojq directly; integers of any width are normalised so queries don't blow up.
- `filter` (*string*) — A jq filter expression, e.g. `“.users[0].name”` or `“.items | length”`.

Returns: Promise<unknown> — the first value the filter emits, or null when it emits nothing (e.g. an optional path like `.a.b?` that misses).

Throws: Rejects if data is undefined, the filter string is empty, the filter fails to parse, or evaluation produces an error.

```
const name = await text.jq.query(obj, "users[0].name");
```

17.14.4.2 text.jq.queryAll

```
queryAll(data: unknown, filter: string): Promise<unknown[]>
```

Run a jq filter and drain the iterator into an array.

Parameters

- `data` (*unknown*) — Input value — any JSON-like JS value.
- `filter` (*string*) — A jq filter expression. Use this when the filter explodes a stream, e.g. `“.”` or `“.users[].id”`.

Returns: Promise<unknown[]> — every value the filter emits, in order (empty array when it emits nothing).

Throws: Rejects if data is undefined, the filter string is empty, the filter fails to parse, or evaluation produces an error.

```
const ids = await text.jq.queryAll(obj, "users[].id");
```

17.14.5 text.markdown

17.14.5.1 text.markdown.toHtml

```
toHtml(md: string, opts?: { gfm?: boolean, hardBreaks?: boolean }): string
```

Render a Markdown string to an HTML string (CommonMark, backed by the pure-Go goldmark). GFM extensions (tables, strikethrough, task lists, autolinks) are on by default; raw HTML in the source is escaped.

Parameters

- `md` (*string*) — The Markdown source text.
- `opts` (*{ gfm?: boolean, hardBreaks?: boolean }, optional*) — `gfm` (default `true`) enables GitHub-Flavored Markdown extensions — tables, strikethrough, task lists, autolinks. `hardBreaks` (default `false`) renders a single source newline as `<br>` instead of a space.

Returns: The rendered HTML fragment (e.g. “`<h1>Hi</h1>\n<ul>\n<li>a</li>\n</ul>\n`”).

Throws: Throws (“text.markdown.toHtml: ...”) if rendering fails (rare — malformed input is handled leniently, not rejected).

```
const html = text.markdown.toHtml("# Hi\n\n a\n- b");
```

17.14.6 text.preg

17.14.6.1 text.preg.match

```
match(pattern: string, subject: string): { match: string; groups: string[]; index: number } | null
```

First hit of `/pattern/flags` against `subject`, or `null`. Returns `{ match, groups, index }`; optional groups that didn't match surface as empty strings.

Parameters

- `pattern` (*string*) — A PHP-style `/regex/flags` delimited pattern (forward-slash delimiter only). Engine is Go RE2. Flags: `i` (case-insensitive), `m` (multiline `^/$`), `s` (dotall). `u/U/x` are rejected.
- `subject` (*string*) — The string to search.

Returns: `{ match, groups, index }` for the first match, where `match` is the full match, `groups` holds the numbered submatches after group 0 (unmatched optionals as “”), and `index` is the byte offset; `null` when there is no match.

Throws: Throws if the pattern is empty, lacks a leading/closing `/`, uses an unsupported flag (`u/U/x`), or fails RE2 compilation (e.g. backreferences/lookaround, which RE2 does not support).

```
const m = text.preg.match("/(\\d+)/", "x42"); // { match: "42", groups: ["42"], index: 1 }
```

17.14.6.2 text.preg.matchAll

```
matchAll(pattern: string, subject: string): { match: string; groups: string[];
index: number }[]
```

Every hit of /pattern/flags against subject, as an array of { match, groups, index } objects.

Parameters

- `pattern` (*string*) — A PHP-style /regex/flags delimited pattern (RE2 engine; i/m/s flags).
- `subject` (*string*) — The string to search.

Returns: Array of { match, groups, index } objects, one per non-overlapping match; empty array when there is none.

Throws: Throws on the same conditions as `preg.match` (bad delimiter, unsupported flag, RE2 compile failure).

```
const all = text.preg.matchAll("/\\d+/", "1 22 333"); // 3 matches
```

17.14.6.3 text.preg.replace

```
replace(pattern: string, replacement: string, subject: string): string
```

Substitute every match of /pattern/flags in subject. Replacement uses Go's \$1 / \${1} backref syntax — PHP's \1 form is NOT translated.

Parameters

- `pattern` (*string*) — A PHP-style /regex/flags delimited pattern (RE2 engine; i/m/s flags).
- `replacement` (*string*) — Replacement template using Go's RE2 syntax: \$1 / \${name} reference groups; use \${1} to disambiguate. PHP's \1 backrefs are NOT supported.
- `subject` (*string*) — The string to transform.

Returns: string — subject with every match replaced (all occurrences).

Throws: Throws on the same conditions as `preg.match` (bad delimiter, unsupported flag, RE2 compile failure).

```
text.preg.replace("/(\\w+)@/", "${1}_at_", "a@b"); // "a_at_b"
```

17.14.7 text.preg2

17.14.7.1 text.preg2.match

```
match(pattern: string, subject: string): { match: string; groups: string[]; index:
number } | null
```

First hit of /pattern/flags via `regexp2` (PCRE). Supports lookahead/lookbehind/backreferences. Same { match, groups, index } shape as `preg`. No linear-time guarantee.

Parameters

- `pattern` (*string*) — A `/regex/flags` delimited pattern run on the .NET-flavoured regexp2 engine (lookaround, backreferences, possessive quantifiers). Flags: `i`, `m`, `s`, and `x` (ignore-pattern-whitespace, unavailable in preg). `u`/`U` are rejected.
- `subject` (*string*) — The string to search.

Returns: `{ match, groups, index }` for the first match (same shape as `preg2.match`); null when there is no match.

Throws: Throws if the pattern is empty, lacks a leading/closing `/`, uses an unsupported flag (`u`/`U`), fails regexp2 compilation, or errors during matching. Backtracking engine: guard untrusted input with a timeout.

```
const m = text.preg2.match("/(?<=@)\\w+/", "a@host"); // { match: "host", ... }
```

17.14.7.2 text.preg2.matchAll

```
matchAll(pattern: string, subject: string): { match: string; groups: string[];
index: number }[]
```

Every hit of `/pattern/flags` via regexp2 (PCRE), as an array of `{ match, groups, index }`.

Parameters

- `pattern` (*string*) — A `/regex/flags` delimited pattern on the regexp2 engine (`i`/`m`/`s`/`x` flags).
- `subject` (*string*) — The string to search.

Returns: Array of `{ match, groups, index }` objects, one per match; empty array when there is none.

Throws: Throws on the same conditions as `preg2.match`. Backtracking engine: guard untrusted input with a timeout.

```
const all = text.preg2.matchAll("/\\w+/", "a b c"); // 3 matches
```

17.14.7.3 text.preg2.replace

```
replace(pattern: string, replacement: string, subject: string): string
```

Substitute every match of `/pattern/flags` via regexp2. Replacement uses .NET `$1` / `${1}` syntax. Backtracking engine — keep a timeout around untrusted input.

Parameters

- `pattern` (*string*) — A `/regex/flags` delimited pattern on the regexp2 engine (`i`/`m`/`s`/`x` flags).
- `replacement` (*string*) — Replacement template using .NET substitution syntax: `$1` / `${1}` reference groups, `$$` is a literal dollar.
- `subject` (*string*) — The string to transform.

Returns: string — subject with every match replaced (startAt 0, count -1).

Throws: Throws on the same conditions as `preg2.match`, or if replacement fails. Backtracking engine: guard untrusted input with a timeout.

```
text.preg2.replace("/(\\w)\\1/", "X", "aabb"); // backref-aware: "XX"
```

17.14.8 text.stego

17.14.8.1 text.stego.embed

```
embed(cover: string, payload: string | Uint8Array, opts?: { password?: string }): string
```

Hide a payload inside cover text using zero-width characters (U+200B / U+200C), appended as an invisible trailing run — the visible text is unchanged. A non-empty password encrypts the payload (AES-256-GCM). Returns the cover text with the hidden run.

Parameters

- `cover` (*string*) — The visible cover text.
- `payload` (*string* / *Uint8Array*) — Data to hide. A string is stored as UTF-8 text (extract returns a string); a *Uint8Array* is stored as binary.
- `opts` (*{ password?: string }, optional*) — password encrypts the payload with AES-256-GCM.

Returns: The cover text followed by the invisible zero-width payload run.

Throws: Throws a *TypeError* if payload is neither a string nor *Uint8Array*.

```
const out = text.stego.embed("Hello there.", "meet at noon", { password: "s3cret" });
```

17.14.8.2 text.stego.extract

```
extract(stegoText: string, opts?: { password?: string }): string | Uint8Array
```

Recover a payload hidden by `text.stego.embed`. Scans the string for zero-width carrier characters, verifies the sercon header, and returns the payload as a string (if embedded as text) or a *Uint8Array*. There is no `capacity()` for text — the hidden run is appended, so capacity is effectively unbounded.

Parameters

- `stegoText` (*string*) — Text produced by `text.stego.embed` (or a copy that preserved the zero-width characters).
- `opts` (*{ password?: string }, optional*) — The password used at embed time, required when the payload was encrypted.

Returns: The recovered payload — a string when embedded as text, otherwise a *Uint8Array*.

Throws: Throws if no sercon stego payload is present, if the payload is encrypted but no password is given, or if decryption fails (wrong password / corrupt).

```
const msg = text.stego.extract(out, { password: "s3cret" });
```

17.14.9 text.str

17.14.9.1 text.str.base64Decode

```
base64Decode(input: string): string
```

Decode standard (RFC 4648) base64 with padding. The standard alphabet only — URL-safe input (containing `-` or `_`) is NOT auto-detected and will throw; use `base64UrlDecode` for that.

Parameters

- `input` (*string*) — Standard-alphabet base64 string with `=` padding. URL-safe characters (`-` / `_`) are not accepted.

Returns: string — the decoded bytes interpreted as a UTF-8 string.

Throws: Throws a `TypeError` if input is missing/null/undefined; throws if the input is not valid standard base64 (including URL-safe alphabet input).

```
text.str.base64Decode("aGk="); // "hi"
```

17.14.9.2 text.str.base64Encode

```
base64Encode(input: string): string
```

Standard base64 (with padding).

Parameters

- `input` (*string*) — UTF-8 string to encode (encoded as its raw bytes).

Returns: string — RFC 4648 standard base64 with `=` padding.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.base64Encode("hi"); // "aGk="
```

17.14.9.3 text.str.base64UrlDecode

```
base64UrlDecode(input: string): string
```

Decode URL-safe (RFC 4648 §5) base64. Tolerant of both padded and unpadded input (trailing `=` is optional). Use `base64Decode` for the standard `+ / /` alphabet.

Parameters

- `input` (*string*) — URL-safe base64 string (`-` / `_` alphabet); `=` padding optional.

Returns: string — the decoded bytes interpreted as a UTF-8 string.

Throws: Throws a `TypeError` if input is missing/null/undefined; throws if the input is not valid URL-safe base64.

```
text.str.base64UrlDecode("YT9i"); // "a?b"
```

17.14.9.4 text.str.base64UrlEncode

```
base64UrlEncode(input: string): string
```

URL-safe base64 (RFC 4648 §5: `-` / `_` alphabet), without `=` padding — safe to drop in URLs, filenames, or JWT segments.

Parameters

- `input` (*string*) — UTF-8 string to encode (encoded as its raw bytes).

Returns: string — URL-safe base64 with no padding.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.base64UrlEncode("a?b"); // "YT9i"
```

17.14.9.5 text.str.br2nl

```
br2nl(input: string): string
```

Inverse of `nl2br`: `<br>`, `<br/>`, `<br />` → `'\n'`.

Parameters

- `input` (*string*) — Source text. Any case-insensitive `<br>`, `<br/>`, or `<br />` variant is matched.

Returns: string — input with each `<br>` variant replaced by a single `\n`.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.br2nl("a<br/>b"); // "a\nb"
```

17.14.9.6 text.str.htmlEntityDecode

```
htmlEntityDecode(input: string): string
```

Decode named and numeric HTML entities to their UTF-8 equivalents.

Parameters

- `input` (*string*) — Text containing HTML entities (named like `&` or numeric like `' / '`).

Returns: string — input with recognised entities decoded to their UTF-8 characters.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.htmlEntityDecode("a & b"); // "a & b"
```

17.14.9.7 text.str.lpad

```
lpad(input: string, len: number, padChar?: string): string
```

Shortcut for `pad(side: 'left')`.

Parameters

- `input` (*string*) — The string to pad on the left.
- `len` (*number*) — Target rune length.
- `padChar` (*string, optional*) — Pad string; defaults to a single space.

Returns: string — input left-padded to len runes.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.lpad("7", 3, "0"); // "007"
```

17.14.9.8 text.str.ltrim

```
ltrim(input: string, mask?: string): string
```

Like trim, left side only.

Parameters

- `input` (*string*) — The string to trim.
- `mask` (*string, optional*) — Cutset of characters to strip from the left. Defaults to the whitespace set “\t\n\r\v\f”.

Returns: string — input with leading mask characters removed.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.ltrim("--x", "-"); // "x"
```

17.14.9.9 text.str.nl2br

```
nl2br(input: string, xhtml?: boolean): string
```

Replace newlines with `<br>` (or `<br/>` when `xhtml=true`).

Parameters

- `input` (*string*) — Source text. CRLF is normalised to LF first, so each line break yields one tag.
- `xhtml` (*boolean, optional*) — When truthy, emit the self-closing `<br/>` instead of `<br>`. Defaults to false.

Returns: string — input with each `\n` replaced by the chosen `<br>` tag followed by the original newline.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.nl2br("a\nb"); // "a<br>\nb"
```

17.14.9.10 text.str.normalizeNewlines

```
normalizeNewlines(input: string, style?: "lf" | "crlf" | "cr"): string
```

Canonicalise any mix of `\r\n`, `\r`, `\n` to the requested style (`'lf'` | `'crlf'` | `'cr'`).

Parameters

- `input` (*string*) — Text with any mix of CRLF, CR, and LF line endings.
- `style` (`"lf"` | `"crlf"` | `"cr"`, *optional*) — Target line-ending style. Defaults to `'lf'`.

Returns: string — input with every line ending rewritten to the requested style.

Throws: Throws a `TypeError` if style is not one of `'lf'`, `'crlf'`, or `'cr'`.

```
text.str.normalizeNewlines("a\r\nb", "lf"); // "a\nb"
```

17.14.9.11 text.str.pad

```
pad(input: string, len: number, padChar?: string, side?: "right" | "left" | "both"): string
```

Pad to `len` with `padChar` (default `' '`). `side` is `'right'` (default), `'left'`, or `'both'`.

Parameters

- `input` (*string*) — The string to pad. Returned unchanged when its rune length already meets or exceeds `len`.
- `len` (*number*) — Target rune length. Measured in runes, not bytes.
- `padChar` (*string, optional*) — Pad string; truncated to fit the needed width. Defaults to a single space.
- `side` (*"right" | "left" | "both", optional*) — Which side(s) to pad. `'both'` splits the deficit with the extra rune going right. Defaults to `'right'`; any unknown value is treated as `'right'`.

Returns: string — input padded to `len` runes on the chosen side(s).

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.pad("7", 3, "0", "left"); // "007"
```

17.14.9.12 text.str.printf

```
printf(format: string, ...args: unknown[]): void
```

sprintf + write to stdout.

Parameters

- `format` (*string*) — A Go fmt format string (same verbs as `sprintf`).
- `args` (*...unknown[], optional*) — Values substituted into the verbs.

Returns: void — writes the formatted text directly to process stdout; returns nothing.

Throws: Throws a `TypeError` if called with no arguments (format string required).

```
text.str.printf("%d items\n", 3);
```

17.14.9.13 text.str.reverse

```
reverse(input: string): string
```

Rune-aware reversal — `reverse('café')` is `'éfac'`.

Parameters

- `input` (*string*) — The string to reverse. Reversed by Unicode code point, not byte, so multi-byte runes stay intact.

Returns: string — the input reversed rune-by-rune.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.reverse("café"); // "éfac"
```

17.14.9.14 text.str.rpad

```
rpadd(input: string, len: number, padChar?: string): string
```

Shortcut for `pad(side: 'right')`.

Parameters

- `input` (*string*) — The string to pad on the right.
- `len` (*number*) — Target rune length.
- `padChar` (*string, optional*) — Pad string; defaults to a single space.

Returns: string — input right-padded to len runes.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.rpad("7", 3, "."); // "7.."
```

17.14.9.15 text.str.rtrim

```
rtrim(input: string, mask?: string): string
```

Like `trim`, right side only.

Parameters

- `input` (*string*) — The string to trim.
- `mask` (*string, optional*) — Cutset of characters to strip from the right. Defaults to the whitespace set “`\t\n\r\v\f`”.

Returns: string — input with trailing mask characters removed.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.rtrim("x..", "."); // "x"
```

17.14.9.16 text.str.sprintf

```
sprintf(format: string, ...args: unknown[]): string
```

Go’s `fmt` verbs (`%s`, `%d`, `%x`, `%2f`, `%v`, `%t`, `%q`, ...) — not PHP’s.

Parameters

- `format` (*string*) — A Go `fmt` format string. Uses Go verbs: `%s` string, `%d` integer, `%f` / `%2f` float, `%x` hex, `%v` default, `%t` bool, `%q` quoted, `%%` literal percent.
- `args` (*...unknown[], optional*) — Values substituted into the verbs (passed through `.Export()`, so JS numbers arrive as Go `int64/float64`).

Returns: string — the formatted result.

Throws: Throws a `TypeError` if called with no arguments (format string required). Verb/arg mismatches are not thrown — Go renders `%!verb(...)` error markers inline.

```
text.str.Sprintf("%s=%d", "n", 5); // "n=5"
```

17.14.9.17 text.str.stripHtml

```
stripHtml(input: string): string
```

Remove HTML tags and decode common entities.

Parameters

- `input` (*string*) — HTML source. Anything matching `<...>` is removed.

Returns: `string` — the input with all `<...>` tag spans deleted.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.stripHtml("<b>hi</b>"); // "hi"
```

17.14.9.18 text.str.trim

```
trim(input: string, mask?: string): string
```

Strip whitespace (or any char in the optional mask string) from both ends.

Parameters

- `input` (*string*) — The string to trim.
- `mask` (*string, optional*) — Cutset: any character in this string is trimmed (PHP-style, not a prefix). Defaults to the whitespace set `"\t\n\r\v\f"`.

Returns: `string` — input with leading and trailing mask characters removed.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.trim(" hi "); // "hi"
```

17.14.9.19 text.str.urlDecode

```
urlDecode(input: string): string
```

Inverse of `urlEncode`.

Parameters

- `input` (*string*) — A form-encoded string (+ decodes to space, `%XX` to bytes).

Returns: `string` — the decoded value.

Throws: Throws a `TypeError` if input is missing/null/undefined; throws on malformed percent-escapes.

```
text.str.urlDecode("a+b%26c"); // "a b&c"
```


17.14.9.20 text.str.urlEncode

```
urlEncode(input: string): string
```

Form-encoding ('+' for space). For path segments use encodeURIComponent (provided by goja).

Parameters

- `input` (*string*) — String to percent-encode. Uses application/x-www-form-urlencoded rules: space becomes +.

Returns: string — the form-encoded value.

Throws: Throws a `TypeError` if input is missing, null, or undefined.

```
text.str.urlEncode("a b&c"); // "a+b%26c"
```

17.15 tui

Multi-pane terminal UI: layout, pane, write, focus.

17.15.1 tui.layout

```
layout(tree: { name: string; title?: string; weight?: number } | { rows: object[]; weight?: number } | { cols: object[]; weight?: number }): void
```

Declare the pane layout for this Run. Tree nodes: { name, title?, weight?, autoscroll? } (leaf), { rows: [...], weight? } (vertical split), { cols: [...], weight? } (horizontal split). The root node also accepts { mouse?: boolean }. Throws on duplicate names, empty rows/cols, unknown keys, or under `-watch`.

Parameters

- `tree` (*{ name: string; title?: string; weight?: number } | { rows: object[]; weight?: number } | { cols: object[]; weight?: number }*) — The root layout node. Exactly one of name / rows / cols must be set per node. A leaf (name) becomes a bordered pane addressable via `tui.pane(name)`; name must be a non-empty string and unique across the whole tree. rows stacks children top-to-bottom; cols places them side-by-side; both arrays must be non-empty. weight (positive integer, default 1) sets the child's proportional share of its parent's space. title (string, leaf only) seeds the pane's border caption. Any other key is rejected. The tree is realised over the full terminal as a `tview Flex` when `stdout` is a TTY; otherwise it falls back to prefixed-line output. autoscroll (boolean, leaf only, default true) controls whether the pane follows the tail as new lines arrive; set false to keep it pinned at the top. mouse (boolean, root only, default false) enables mouse support: the wheel scrolls the pane under the cursor (without changing which pane is focused) and a left-click focuses the pane under the cursor — at the cost of the terminal's native click-drag text selection (use Shift/Option+drag to select while mouse mode is on). wrap (string, leaf only, default "char") sets line wrapping: "char" wraps mid-word, "word" wraps at word boundaries, "off" disables wrapping (long lines scroll horizontally). color (boolean, leaf only, default true) renders subprocess ANSI as pane colors; set false to strip ANSI and show plain text. wrap and color affect TTY rendering only (the non-TTY fallback always emits plain prefixed lines).

Returns: void — installs the layout and brings up the UI (TTY) or the fallback line writer (non-TTY); the controller is torn down automatically at Run end.

Throws: Throws if called under `-watch`; if layout was already called this Run; if the tree argument is missing/null/undefined; if any node violates the structure rules (not an object, more than one of name/rows/cols, missing all three, empty rows/cols, unknown key, non-string or empty name, duplicate name, title on a non-leaf, or non-positive/non-integer weight) — the error includes the tree path (e.g. “rows[1].cols[0]”); or if the terminal screen / fallback writer fails to start.

```
tui.layout({ cols: [{ name: "log", title: "Log" }, { name: "out", weight: 2 }] });
tui.pane("log").writeln("started");
```

17.15.2 tui.onKey

```
onKey(handler: (key: { name: string; rune: string; ctrl: boolean; alt: boolean;
shift: boolean }) => void): () => void
```

Register a callback invoked on every keypress (TTY mode) except Ctrl-C. Returns an unsubscribe function. No-op in non-TTY (fallback) mode.

Parameters

- `handler` *((key: { name: string; rune: string; ctrl: boolean; alt: boolean; shift: boolean }) => void)* — Called for each keypress with a key descriptor. name is the tcell key name (“Enter”, “Up”, “Tab”, “Ctrl-A”, “F1”, ...) or “Rune” for a printable character (rune holds it). For Ctrl+letter combinations the name carries the combo (e.g. “Ctrl-A”) and the ctrl flag may be false, so check name for control keys. Built-in navigation (Tab focus, PgUp/PgDn/arrows/Home/End scroll) still runs in addition to the handler (coexist model). Ctrl-C always aborts the script and is never delivered.

Returns: An unsubscribe function; call it to stop receiving keys. In non-TTY mode onKey registers nothing and returns a no-op unsubscribe. Note: registering onKey alone does not keep the Run alive — keep an outstanding await (e.g. waitKey or a sleep) if the script should stay open.

Throws: Throws if called before tui.layout, or if the argument is not a function.

```
tui.layout({ rows: [{ name: "log" }] });
const off = tui.onKey((k) => { if (k.name === "Rune" && k.rune === "q") off(); });
```

17.15.3 tui.pane

```
pane(name: string): { write(text: string): void; writeln(text: string): void;
clear(): void; title(text: string): void }
```

Return a Pane handle for a declared pane. Throws if the name wasn’t in the layout. Handle methods: write(text), writeln(text), clear(), title(text). services.exec.shell({pane}) streams subprocess I/O into a pane.

Parameters

- `name` *(string)* — The leaf name declared in the tui.layout tree.

Returns: A Pane handle. `write(text)` appends text (subprocess ANSI SGR colors are translated to pane colors); `writeln(text)` appends text followed by a newline; `clear()` empties the pane (no-op in the non-TTY fallback); `title(text)` updates the pane's border caption (no-op in fallback). All methods return undefined and are safe to call from any callback. The handle can also be passed as the pane option to `services.exec.shell` to stream a subprocess's stdout/stderr live into the pane.

Throws: Throws if `tui.layout` has not been called yet this Run, or if name was not declared as a leaf in the layout (the message lists the available pane names).

```
const p = tui.pane("out");
p.title("Output");
p.writeln("hello");
```

17.15.4 tui.waitKey

```
waitKey(...args: unknown[]): Promise<{ name: string; rune: string; ctrl: boolean;
alt: boolean; shift: boolean }>
```

Resolve with the next keypress (TTY mode). One-shot — await again for the next key. Rejects in non-TTY (fallback) mode.

Returns: A Promise resolving to the next key descriptor (same shape as `onKey`'s argument). Concurrent `waitKey` calls resolve FIFO — one keypress resolves the oldest pending call. While a `waitKey` is pending the TUI stays open (this is the idiomatic “press any key to close” hold). Ctrl-C aborts and is never delivered.

Throws: Rejects if called before `tui.layout`, or in non-TTY mode (no interactive terminal), or if the TUI is closed while waiting.

```
tui.layout({ rows: [{ name: "log" }] });
tui.pane("log").writeln("Done. Press any key to close.");
await tui.waitKey();
```

17.16 web

Fetch & parse web documents: RSS/Atom/JSON feeds (`web.feed`), sitemaps incl. gzip + index expand (`web.sitemap`), and lenient HTML scraping with CSS + XPath (`web.html`). Each offers `parse(string)` and `async load(url, opts?)`.

17.16.1 web.feed

17.16.1.1 web.feed.load

```
load(url: string, opts?: { timeout?: number | string; headers?: Record<string,
string>; follow?: boolean; userAgent?: string; username?: string; password?:
string; maxBytes?: number }): Promise<{ feedType: string; title: string;
description: string; link: string; updated: string | null; items: Array<{ title:
string; link: string; published: string | null; updated: string | null; content:
string; summary: string; author: string; guid: string; categories: string[]; raw:
Record<string, unknown> }> }>
```

Fetch a URL and parse it as a feed (see `feed.parse`). Reuses the `net.http` option surface and sends a default User-Agent unless overridden. Throws on a non-2xx response.

Parameters

- `url` (*string*) — The feed URL to GET.
- `opts` (*{ timeout?: number | string; headers?: Record<string, string>; follow?: boolean; userAgent?: string; username?: string; password?: string; maxBytes?: number }*, optional) — Fetch options: timeout (ms or duration string), headers, follow redirects, userAgent, basic-auth username/password, maxBytes (response body size cap, default 256 MiB).

Returns: A promise resolving to the normalized feed object.

Throws: Throws on transport failure or a non-2xx response, and on malformed feed content.

```
const f = await web.feed.load("https://example.com/feed.xml");
```

17.16.1.2 web.feed.parse

```
parse(source: string): { feedType: string; title: string; description: string; link: string; updated: string | null; items: Array<{ title: string; link: string; published: string | null; updated: string | null; content: string; summary: string; author: string; guid: string; categories: string[]; raw: Record<string, unknown> }> }
```

Parse RSS, Atom, or JSON-feed text into a normalized feed model. Format is auto-detected; `feedType` reports it. RSS/Atom field differences are unified (pubDate/updated, description/summary). Each item carries a `raw` escape hatch with format-specific extras (enclosures, namespaced elements like `media:dc:`).

Parameters

- `source` (*string*) — The feed document text (RSS/Atom XML or JSON Feed).

Returns: The normalized feed object.

Throws: Throws on empty/malformed/undetectable feed input.

```
const f = web.feed.parse(xml); f.items[0].title;
```

17.16.2 web.html

17.16.2.1 web.html.load

```
load(url: string, opts?: { timeout?: number | string; headers?: Record<string, string>; follow?: boolean; userAgent?: string; username?: string; password?: string; maxBytes?: number }): Promise<{ find(selector: string): unknown; findAll(selector: string): unknown[]; xpath(expr: string): unknown; xpathAll(expr: string): unknown[]; text(): string; html(): string; innerHTML(): string; tag(): string; attr(name: string): string | null; attrs(): Record<string, string>; }>
```

Fetch a URL and parse the response as lenient HTML (see `html.parse`). Reuses the `net.http` option surface with a default User-Agent. Throws on a non-2xx response.

Parameters

- `url` (*string*) — The page URL to GET.

- `opts` (*{ timeout?: number | string; headers?: Record<string, string>; follow?: boolean; userAgent?: string; username?: string; password?: string; maxBytes?: number }*, optional) — Fetch options: timeout (ms or duration string), headers, follow redirects, userAgent, basic-auth username/password, maxBytes (response body size cap, default 256 MiB).

Returns: A promise resolving to the document Node handle.

Throws: Throws on transport failure or a non-2xx response.

```
const doc = await web.html.load("https://example.com"); doc.findAll("a").map(a =>
a.attr("href"));
```

17.16.2.2 web.html.parse

```
parse(source: string): { find(selector: string): unknown; findAll(selector:
string): unknown[]; xpath(expr: string): unknown; xpathAll(expr: string):
unknown[]; text(): string; html(): string; innerHTML(): string; tag(): string;
attr(name: string): string | null; attrs(): Record<string, string>; }
```

Parse HTML leniently (real-world tag soup is accepted, never throws on bad markup) into a chainable Node. Query with CSS (`find/findAll`) or XPath (`xpath/xpathAll`); read with `text/html/innerHTML/tag/attr/attrs`. Sub-queries are scoped to the receiver node (use `.`).

Parameters

- `source` (*string*) — The HTML document text.

Returns: A Node handle rooted at the document.

Throws: Does not throw on malformed markup; only on unreadable input.

```
const doc = web.html.parse(html); doc.find("h1").text();
```

17.16.3 web.sitemap

17.16.3.1 web.sitemap.load

```
load(url: string, opts?: { expand?: boolean } & { timeout?: number | string;
headers?: Record<string, string>; follow?: boolean; userAgent?: string; username?:
string; password?: string; maxBytes?: number }): Promise<{ type: "urlset" |
"sitemapindex"; urls: Array<{ loc: string; lastmod?: string; changefreq?: string;
priority?: number }>; sitemaps: string[]; errors: Array<{ url: string; error:
string }> }>
```

Fetch a sitemap URL and parse it. Transparently decompresses `.xml.gz` (gzip magic-byte detection on the body; a Content-Encoding: gzip response is unwrapped by the HTTP transport). For a `sitemapindex`, pass `{ expand:true }` to fetch all child sitemaps (bounded; per-child errors collected in `errors[]`) and merge their urls into `urls`.

Parameters

- `url` (*string*) — The sitemap URL to GET (may be `.xml.gz`).
- `opts` (*{ expand?: boolean } & { timeout?: number | string; headers?: Record<string, string>; follow?: boolean; userAgent?: string; username?: string; password?: string; maxBytes?:*

number }, optional) — expand:true fetches & merges child sitemaps for an index. Plus the standard fetch options.

Returns: A promise resolving to the sitemap object.

Throws: Throws on transport failure, non-2xx, or a non-sitemap document. Per-child expand failures are captured in errors[], not thrown.

```
const sm = await web.sitemap.load("https://example.com/sitemap.xml", { expand: true });
```

17.16.3.2 web.sitemap.parse

```
parse(source: string): { type: "urlset" | "sitemapindex"; urls: Array<{ loc: string; lastmod?: string; changefreq?: string; priority?: number }>; sitemaps: string[]; errors: Array<{ url: string; error: string }> }
```

Parse a sitemap document (urlset or sitemapindex) into {type, urls, sitemaps, errors}. urls carry loc/lastmod/changefreq/priority; an index lists child sitemap URLs in sitemaps .

Parameters

- `source` (*string*) — The sitemap XML text (decompressed).

Returns: The parsed sitemap object.

Throws: Throws when the document is neither <urlset> nor <sitemapindex>.

```
const sm = web.sitemap.parse(xml); sm.urls.map(u => u.loc);
```

This manual covers sercon v0.102.0. Whenever you add, remove, or change a flag, a binding, or the script API, update this file alongside the help screen (`--help`), the examples walkthrough (`--examples`), and the `CHANGELOG.md` .